MLOPS & CLOUD

AI · ML · DS / REVISION NOTES

COURSERA · DEEPLEARNING.AI

Machine Learning Modeling Pipelines in Production

MLOps Specialization, Course 3 — AutoML, resource management, analysis and interpretability

51 SECTIONS

464 FIGURES

412 PAGES

Compiled October 2026

github.com/ankitgupta1729/AI-ML-DS

Contents

LECTURE 1 TensorFlow Model Analysis (TFMA) 4
LECTURE 2 Manual Hyperparameter Tuning Is Not Scalable 7
LECTURE 3 Search Strategy 11
LECTURE 4 Automated Machine Learning (AutoML) 26
LECTURE 5 Micro Architecture Search Space 32
LECTURE 6 Reinforcement Learning 38
LECTURE 7 Weight Inheritance/Network Morphisms 48
LECTURE 8 Amazon SageMaker Autopilot 54
LECTURE 9 Steps to Classify Images Using AUtoML Vision 69
LECTURE 10 A Note About Neural Networks 72
LECTURE 11 Curse of Dimensionality in the Distance Function 83
LECTURE 12 More Features Require More Training Data 93
LECTURE 13 Why Reduce Dimensionality? 99
LECTURE 14 Bucketizing and Feature Crossing 104
LECTURE 15 Linear Dimensionality Reduction 116
LECTURE 16 Principal Component Analysis (PCA) 119
LECTURE 17 Independent Components Analysis (ICA) 130
LECTURE 18 Mobile Inference 138
LECTURE 19 Why Quantize Neural Networks? 146
LECTURE 20 Post Training Quantization 158
LECTURE 21 Quantize Custom Keras Layer 167
LECTURE 22 Sparsity Increases with Training 176
LECTURE 23 Types of Distributed Training 195
LECTURE 24 Parallelize Data Transformation 205
LECTURE 25 Pipeline Parallelism 218
LECTURE 26 Knowledge Distillation 230
LECTURE 27 First Quantitative Results of Distillation 233
LECTURE 28 Train Teacher 241

LECTURE 29 Performance Metrics vs Optimization Objectives 248
LECTURE 30 One Model vs Multiple Models Over Time 256
LECTURE 31 EVAL Config and TFMA 265
LECTURE 32 Robustness Metrics 271
LECTURE 33 Benchmark Models 277
LECTURE 34 Random Attacks 279
LECTURE 35 Create the Adversarial Image 295
LECTURE 36 Residual Analysis 305
LECTURE 37 Remediation Techniques 308
LECTURE 38 Fairness Indicators 312
LECTURE 39 About the CelebA Dataset 319
LECTURE 40 Statistical Process Control 328
LECTURE 41 Need for Explainability in AI 345
LECTURE 42 Interpretability of ML Models 352
LECTURE 43 More Advanced Models: TensorFlow Lattice 365
LECTURE 44 Model Agnostic Methods 379
LECTURE 45 Partial Dependence Plots: Examples 381
LECTURE 46 Advantages of Permutation Feature Importance 387
LECTURE 47 Disadvantages of Shapley Values 391
LECTURE 48 SHAP Summary Plot 399
LECTURE 49 Testing Concept Activation Vectors (TCAV) 403
LECTURE 50 Local Interpretable Model-agnostic Explanations (LIME) 405
LECTURE 51 AI Explanations: Feature Attributions 406

LECTURE 1

TensorFlow Model Analysis (TFMA)

Welcome to this third course in the Machine Learning Engine for Production in MLOps specialization. The first course have presented an overview of the entire machine learning project life cycle. In the second course, you saw data pipelines for production machine learning. This third course is about building and maintaining models for different serving environments. The modules covered in this course, will tightly integrate with the data pipelines you built in the second course. I'm thrilled to reintroduce this course's instructor Robert Crowe, who is TensorFlow Developer Engineer at Google and one of the world's experts in MLOps. Thanks, Andrew. This course focuses on implementing tools and techniques to effectively manage your modeling resources and best serve batch and real-time inference requests.

In this course, you'll implement effective search strategies for the best model that will scale for various serving needs while constraining model complexity and hardware requirements. You'll optimize and manage this compute storage and IO resources your model needs in production environments during its life cycle.

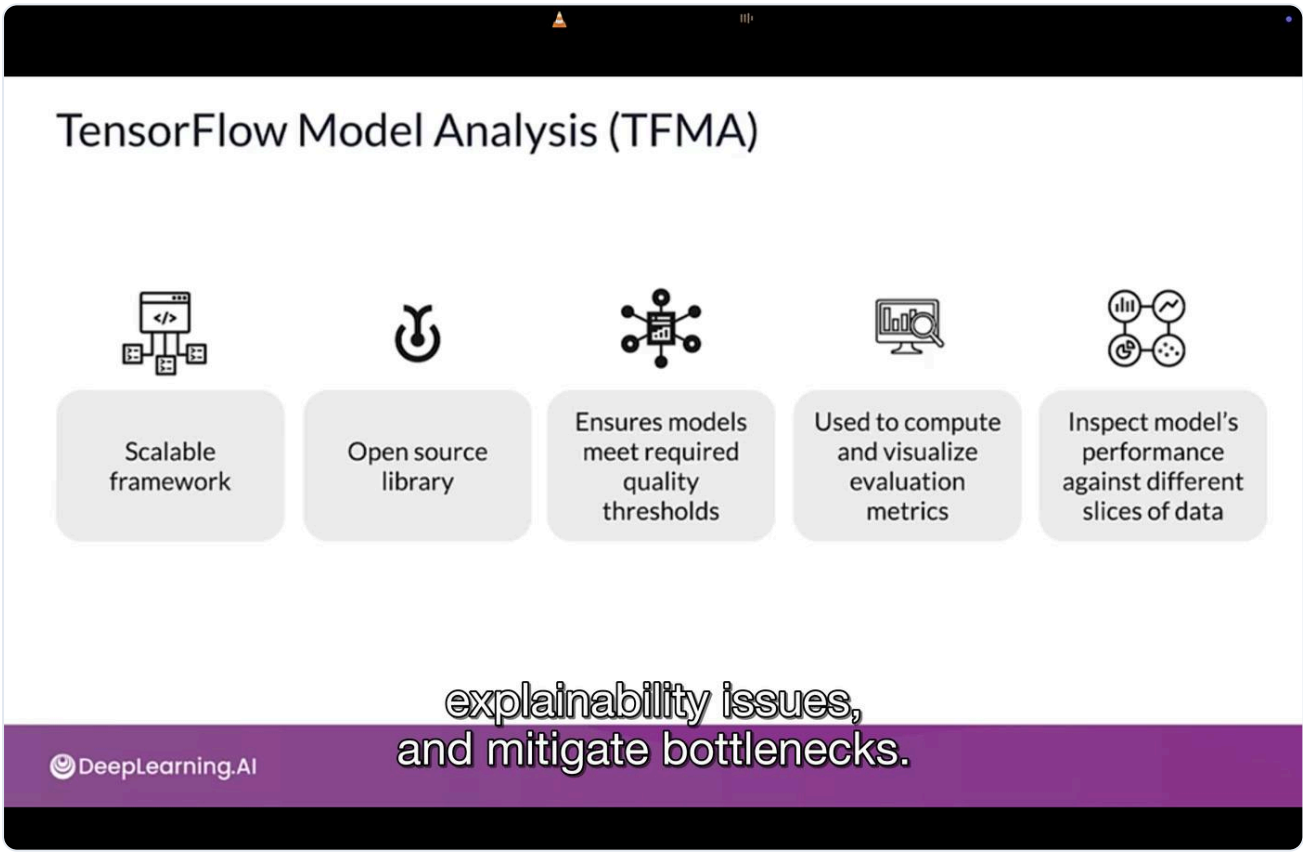

FIGURE 1.1 | TensorFlow Model Analysis (TFMA)

In this journey, you will continue to use the TFX library and rely on tools like AutoML for finding the best suitable model and TensorFlow model analysis to address model fairness, explainability issues, and mitigate bottlenecks.

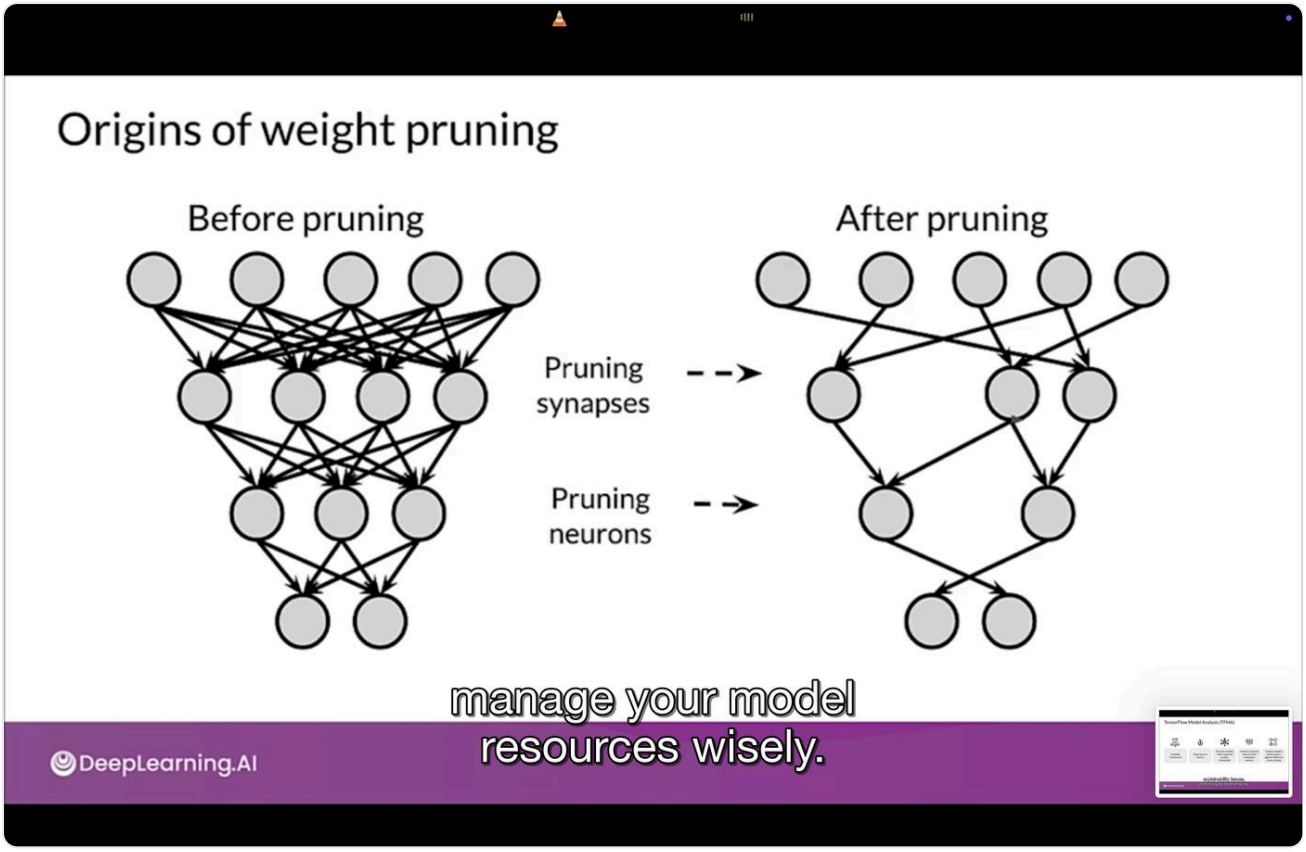

FIGURE 1.2

Along the way, you'll learn about more specialized scenarios like dimensionality reduction and pruning to manage your model resources wisely.

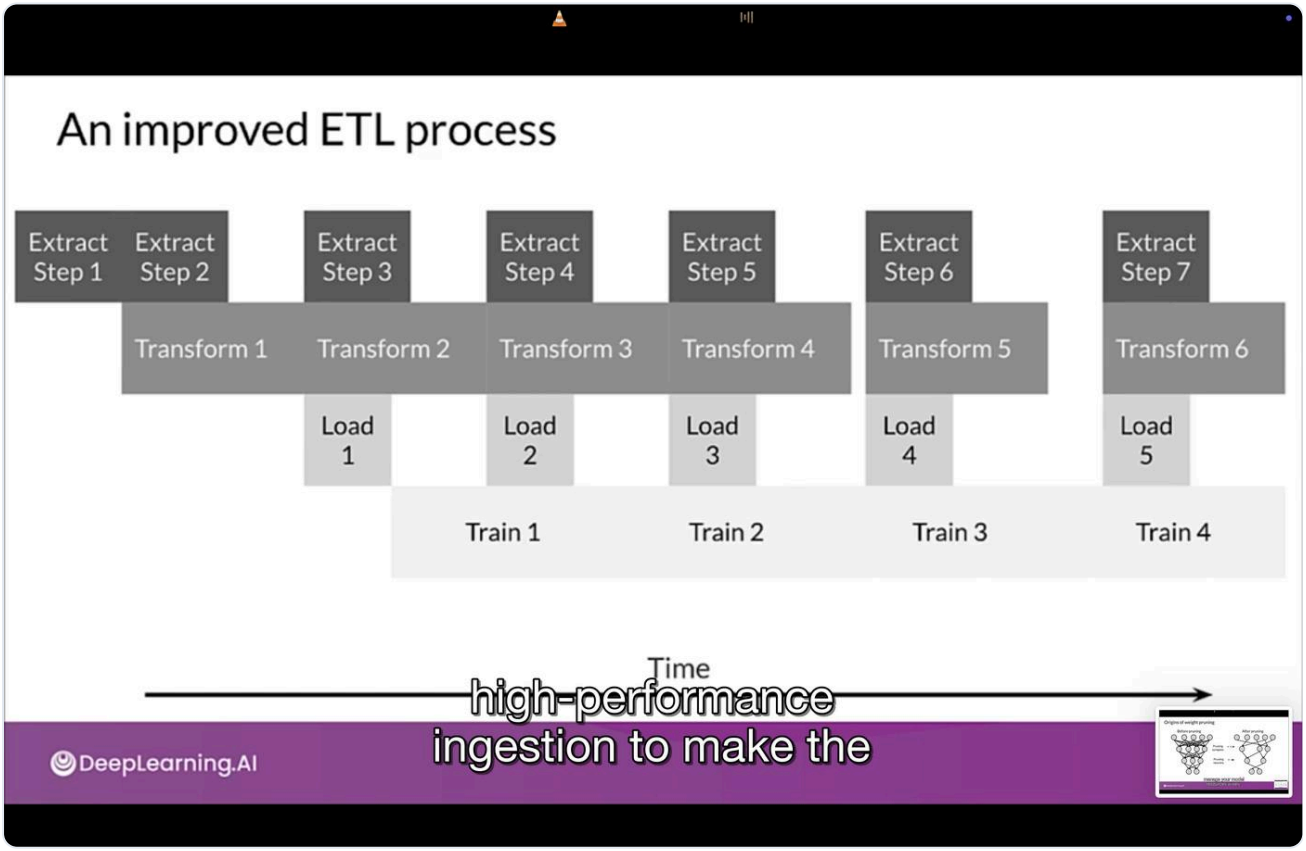

FIGURE 1.3

Also, you'll be exploring pipelining and parallelism and high-performance ingestion to make the most of your computational resources. A Production ML system must run nonstop, at the minimum cost while producing the maximum performance. In this course, you'll learn how to use well-established tools and methodologies for doing all of this effectively and efficiently. Thanks Robert, I'm excited about this. Anyone that masters these techniques will be much more effective at building production machine learning systems, and also be well-equipped to promptly address any explainability and fairness issues that may arise. With that, let's dive into building model pipelines.

LECTURE 2

Manual Hyperparameter Tuning Is Not Scalable

Welcome back. We have a lot of great stuff to talk about this week. First, we're going to start with a discussion of hyperparameter tuning, and we'll do an exercise using Keras Tuner. Then we'll dive into AutoML, and really look to try to understand it at some depth. Then we'll actually do an exercise using AutoML on the Cloud. It's going to be fun. Let's get started. We'll get started with neural architecture search. We'll start with a discussion of something that you might be more familiar with, hyperparameter tuning. As I think you'll see there are similarities between hyperparameter tuning and neural architecture search. Neural architecture search, or NAS, takes a prominent role in recent ML developments.

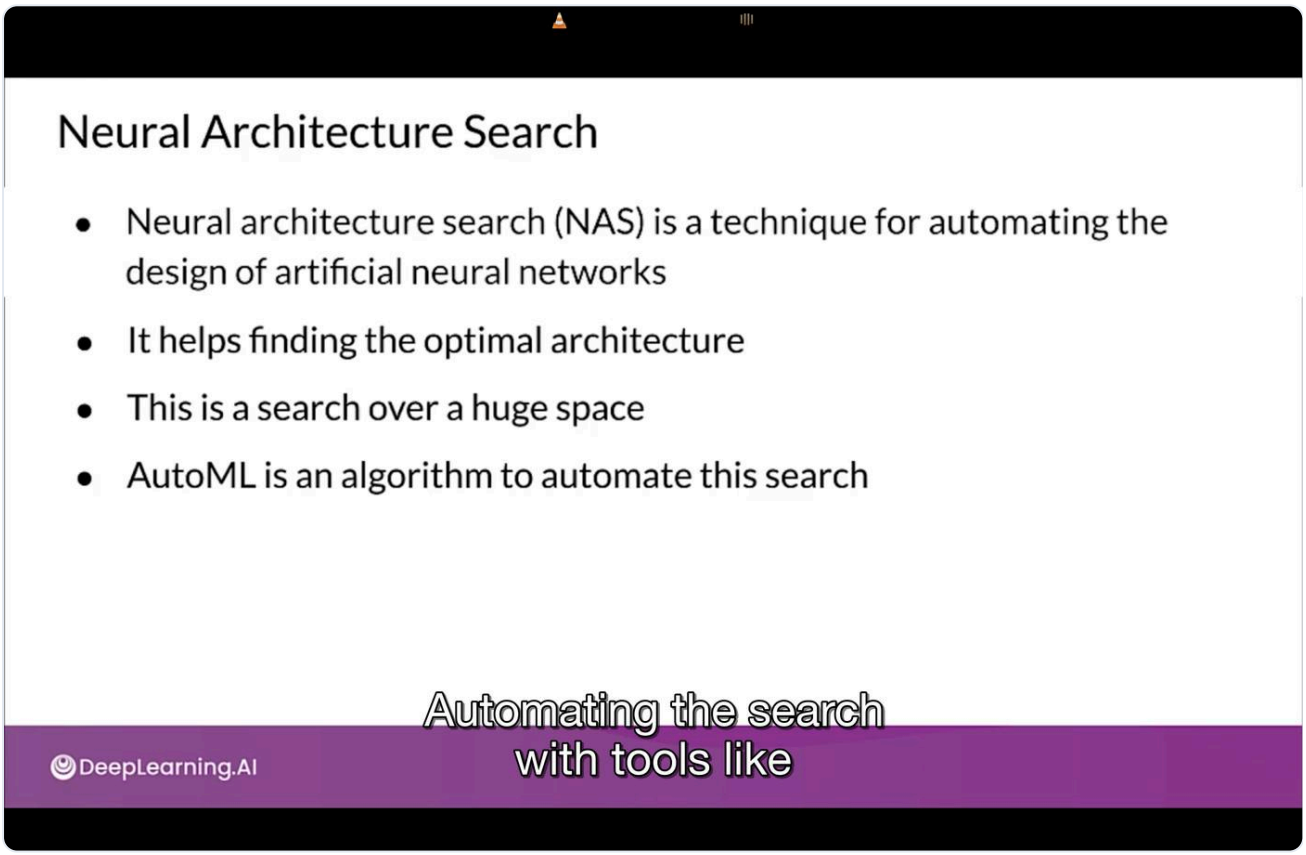

FIGURE 2.1 | Neural Architecture Search

In essence, it's a technique for automating the design of neural networks. Models found by NAS are often on par with or outperform hand designed architectures for many types of problem. The goal of NAS is to find the optimal architecture. Keep in mind that modern neural networks cover a huge parameter space. Automating the search with tools like AutoML makes a lot of sense. Before taking a deep dive into AutoML, let's understand the problem it solves by analyzing one of the most tedious processes in ML modeling you've done naively, which is hyperparameter tuning. In machine learning models, there are two types of parameters. There are model parameters.

Those are parameters that the model must learn using the training set. They're fitted or trained parameters of our models. Usually that means weights and biases. Then we have hyperparameters.

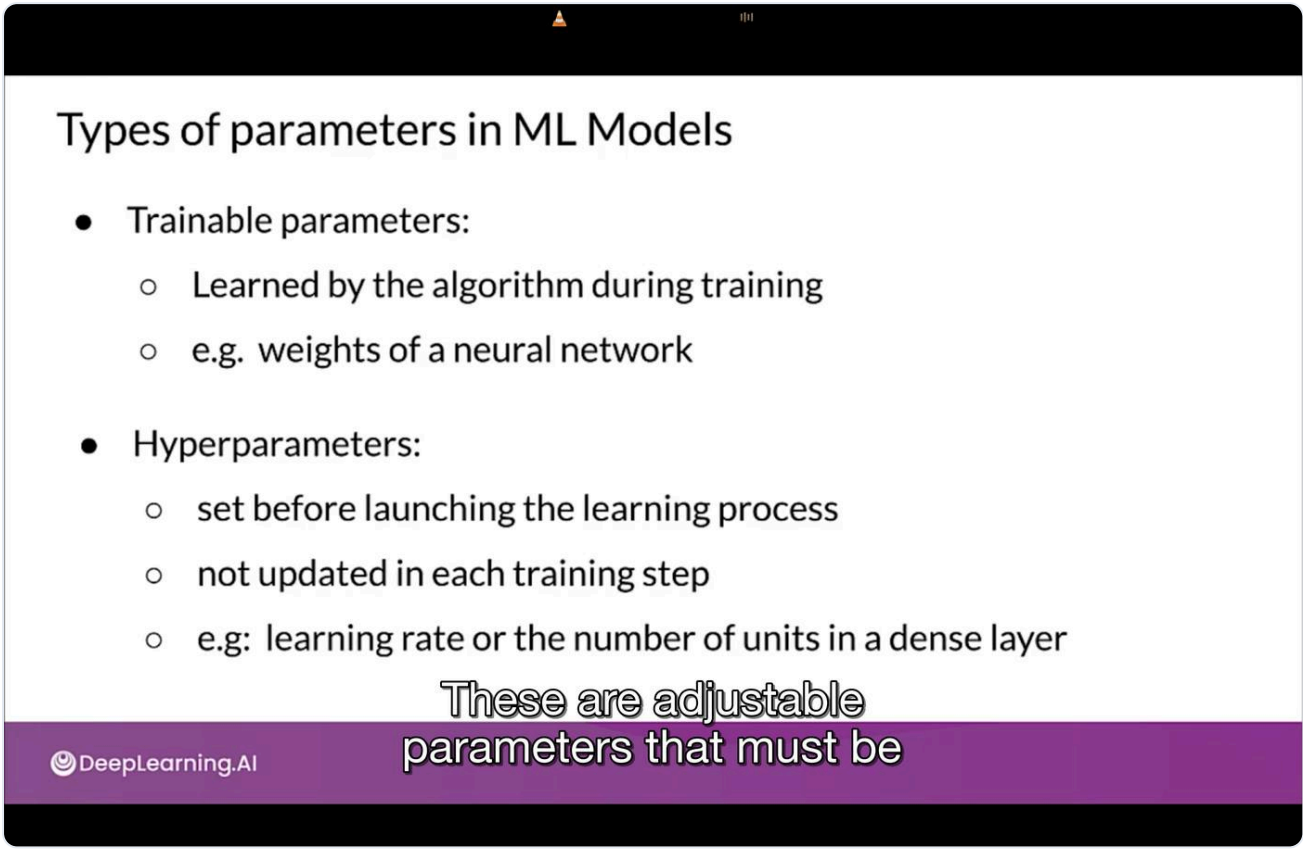

FIGURE 2.2 | Types of Parameters in ML Models

These are adjustable parameters that must be tuned in order to create a model with optimal performance, but unlike model parameters, hyperparameters are not automatically optimized during the training process. They need to be set before model training begins, and they affect how the model trains. Hyperparameter tuning can have a high impact on model performance, and unfortunately, the number of hyperparameters can be substantial even for small models. For instance, in a shallow DNN, you can make hyperparameter choices for architecture options, for activation functions, for weight initialization strategy, and optimization hyperparameters, and others.

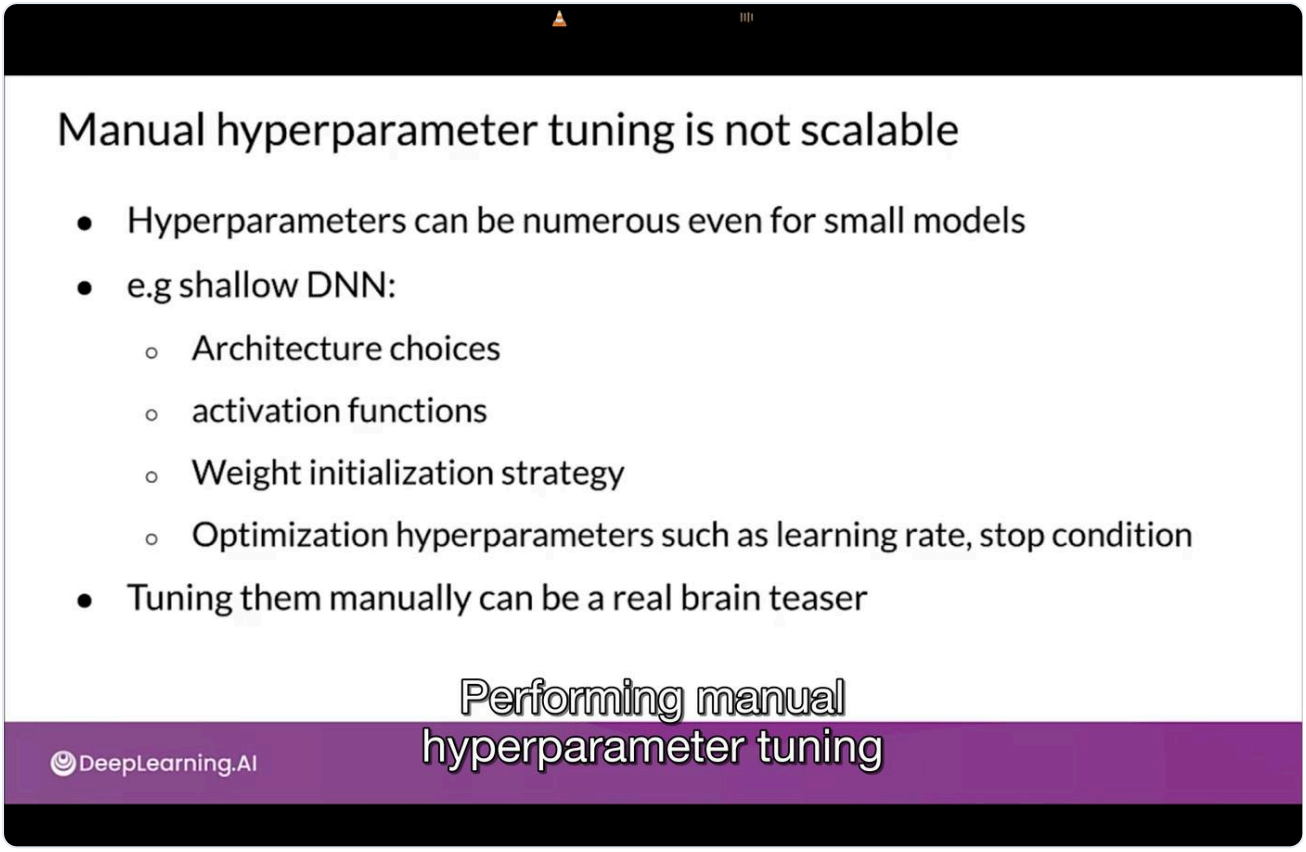

FIGURE 2.3 | Manual Hyperparameter Tuning Is Not Scalable

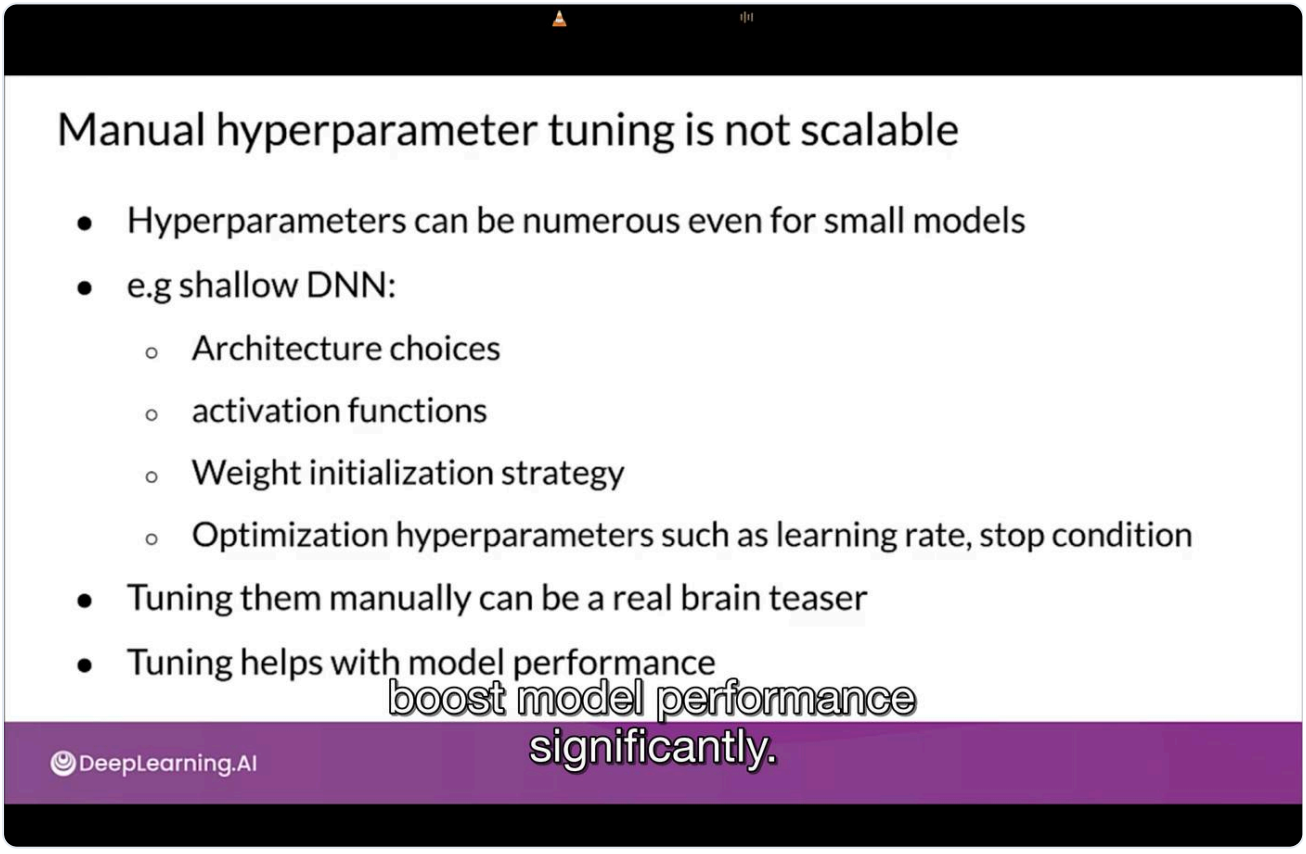

FIGURE 2.4 | Manual Hyperparameter Tuning Is Not Scalable

Performing manual hyperparameter tuning can be a real brain teaser, which is a nice way to put it, since you're going to need to keep track of what you've tried and launch different experiments, so a manual process like that is tedious. Nonetheless, when done properly, hyperparameter tuning helps boost model performance significantly. Several open source libraries have been created using various approaches to hyperparameter tuning. The Keras team has released one of the best, Keras Tuner, which is a library to easily perform hyperparameter tuning with TensorFlow 2.0. It offers a variety of different tuning methods, including random search, Hyperband, and Bayesian optimization.

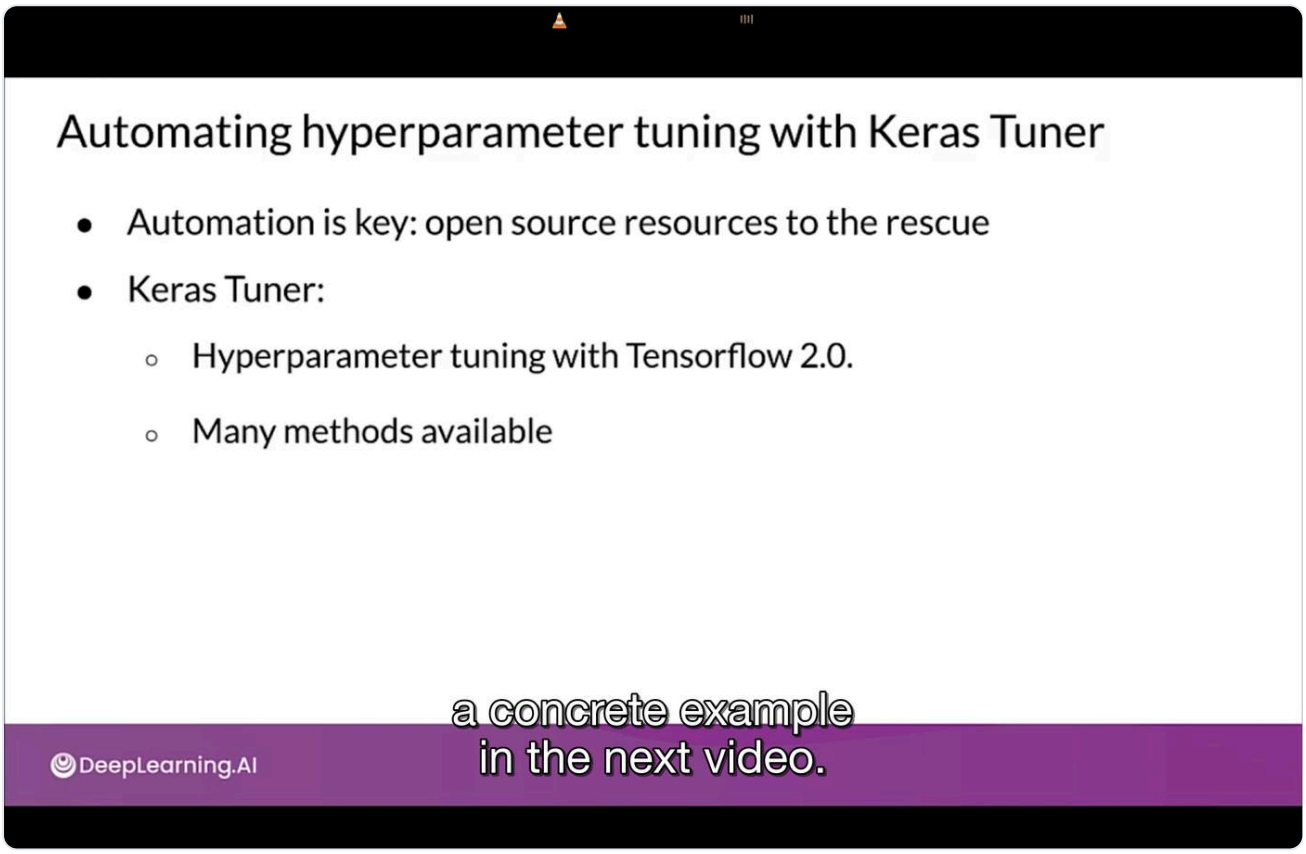

FIGURE 2.5 | Automating Hyperparameter Tuning with Keras Tuner

Let's take a look at a concrete example in the next video.

LECTURE 3

Search Strategy

Now let's get started with a demo of Keras Autotuner.

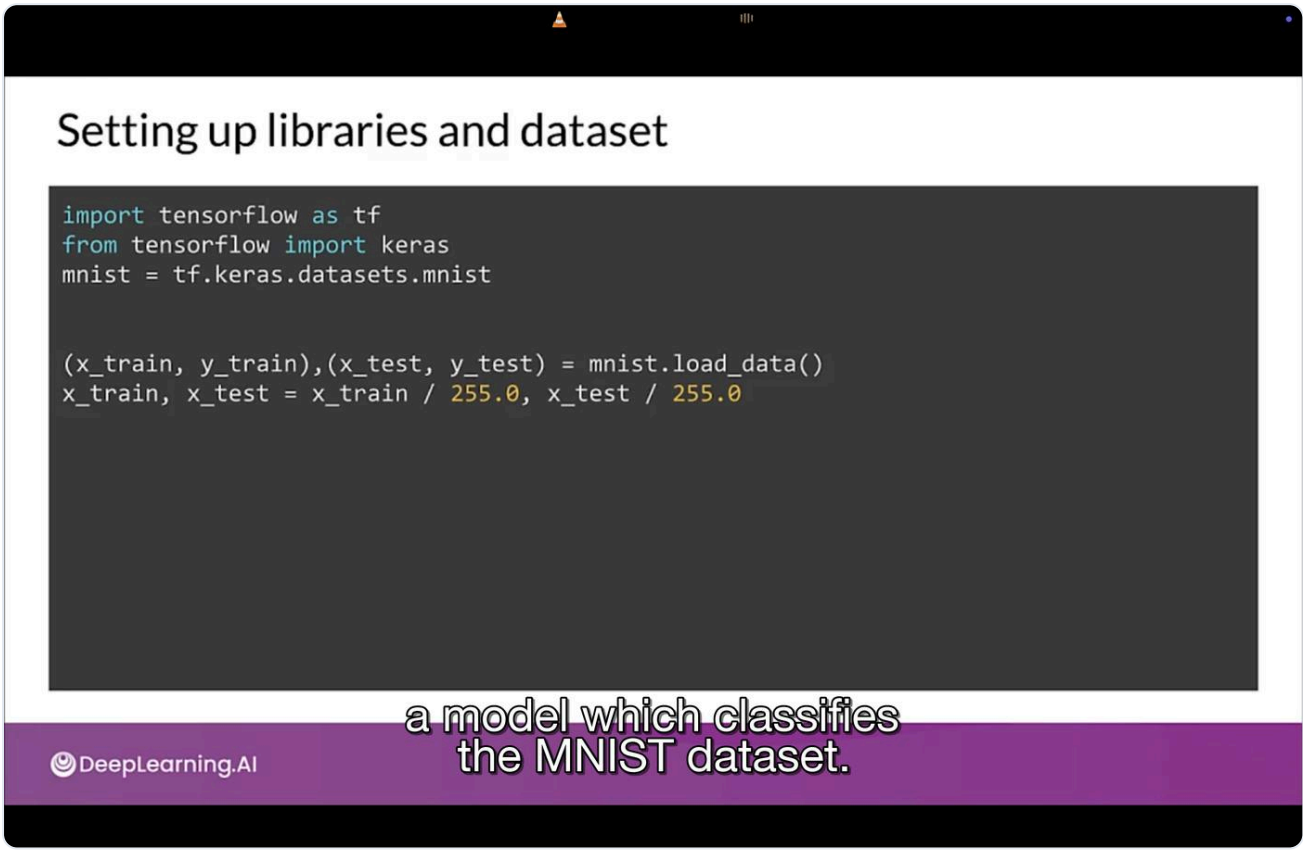

FIGURE 3.1 | Setting Up Libraries and Dataset

In this example, you'll start by manually setting the hyper parameters for a model which classifies the MNIST dataset. For simplicity, let's work with the most basic neural network architecture consisting of one input layer, one hidden layer, and one output layer. Let's begin by importing the required libraries and dataset.

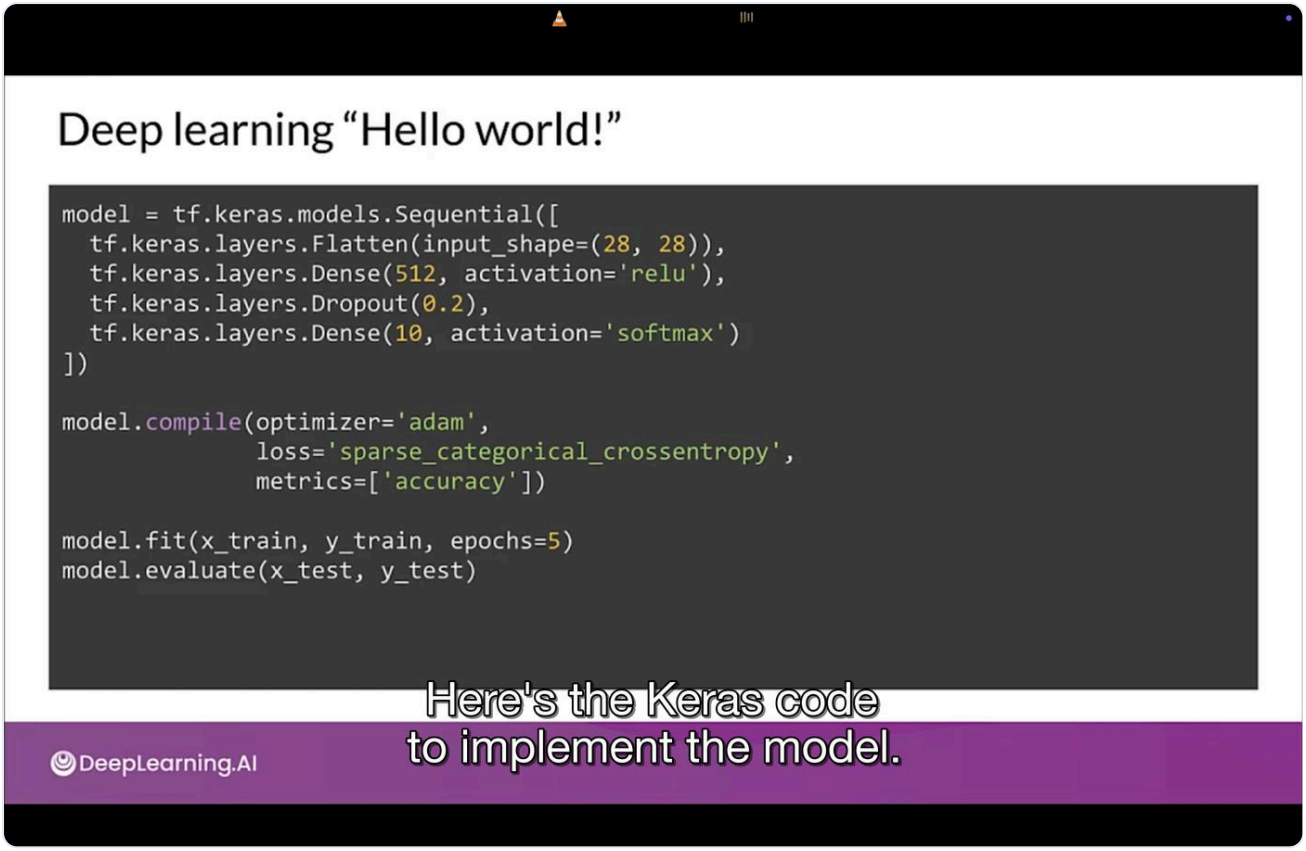

FIGURE 3.2

Here's the Keras code to implement the model. This is maybe the hello world of deep learning where you have a data set and of fashion items and you want to build a basic model to recognize them. You can train a model in just a few seconds, that is about 97% accurate on this dataset.

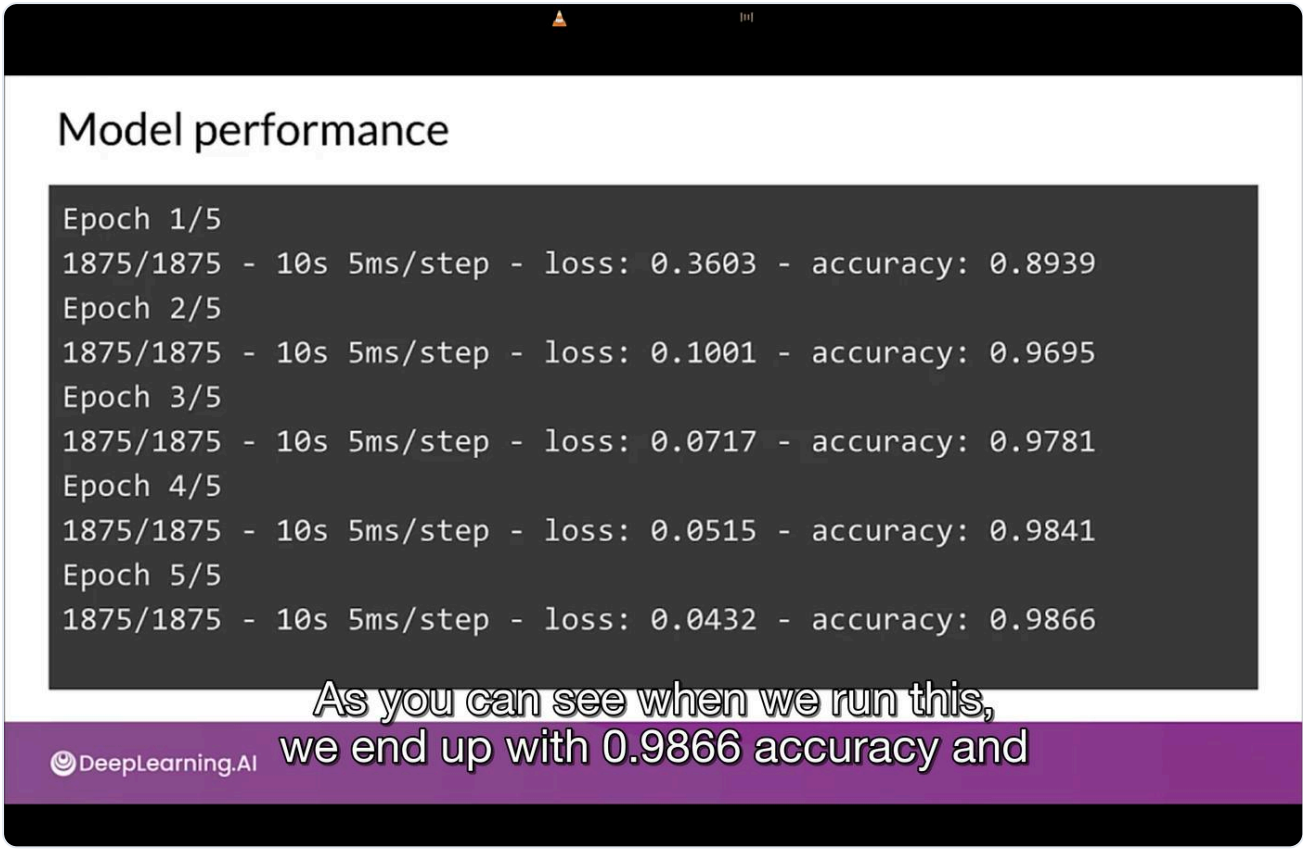

FIGURE 3.3

As you can see when we run this, we end up with 0.9866 accuracy and each epic takes a little over 10 seconds. But can we do better?

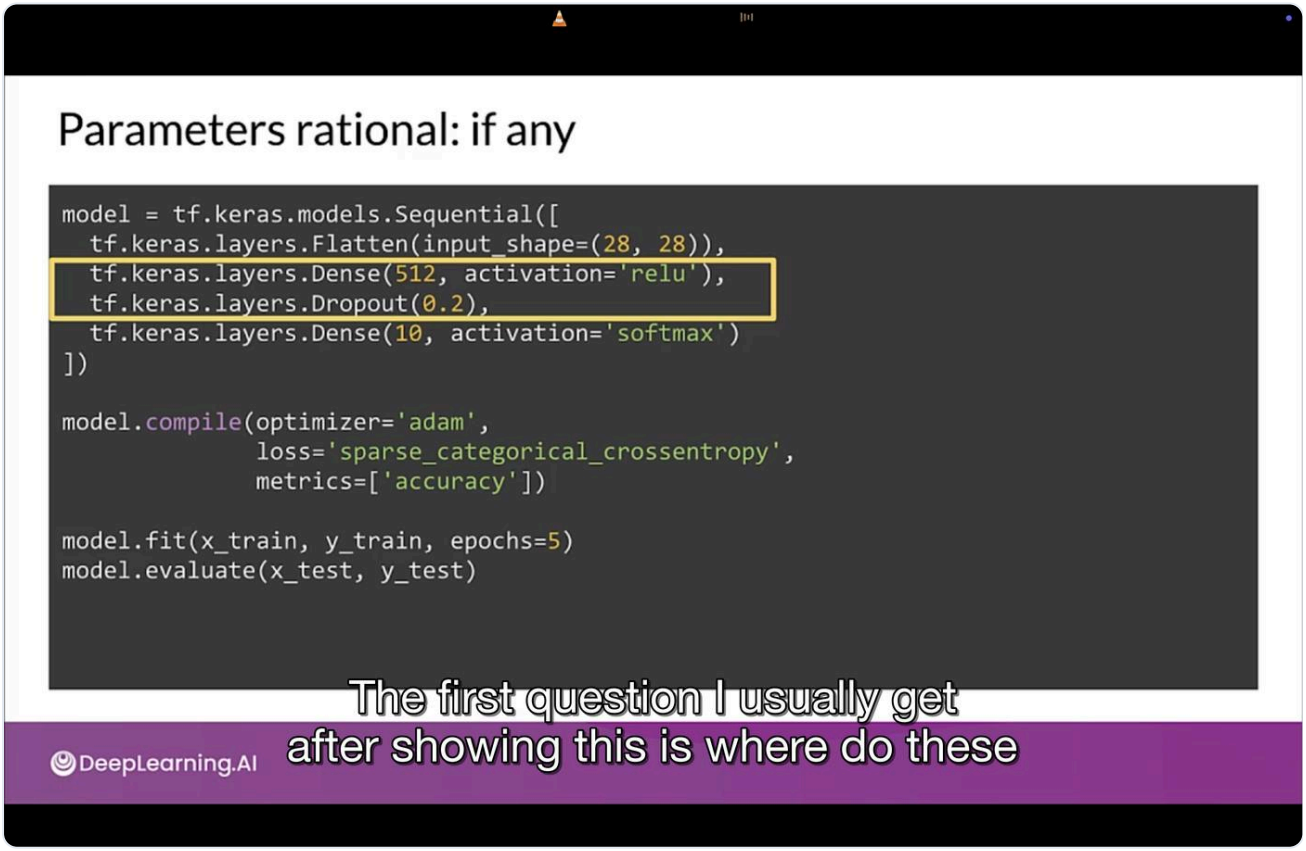

FIGURE 3.4

The first question I usually get after showing this is where do these numbers come from? Why does this architecture have 512 neurons? Why not another number? And why does the dropout use 0.2, why not 0.5 or 0.1? Often, the answer is that I used a lot of trial and error to come up with these numbers. Or as is often the case, I just copy the code from a working example and didn't really think about it. Let's think about whether this is an attribute choice or not. The natural questions are will the model do better with more hidden units or less? Will it learn faster with less units? Can a smaller architecture do that without losing accuracy?

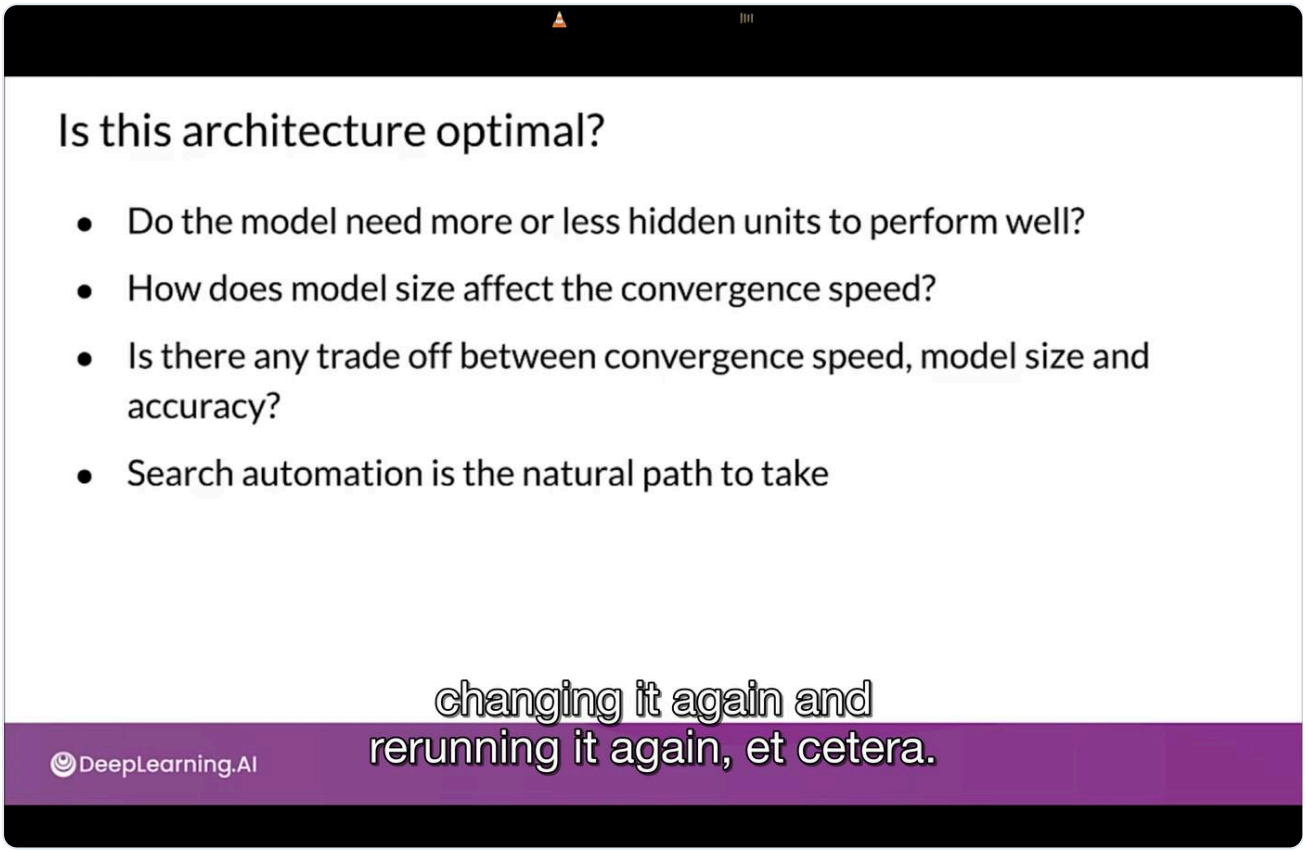

FIGURE 3.5 | Is This Architecture Optimal?

You can experiment by changing it, rerunning it, changing it again and rerunning it again, et cetera. But that's a lot of work. What if you could automate this.

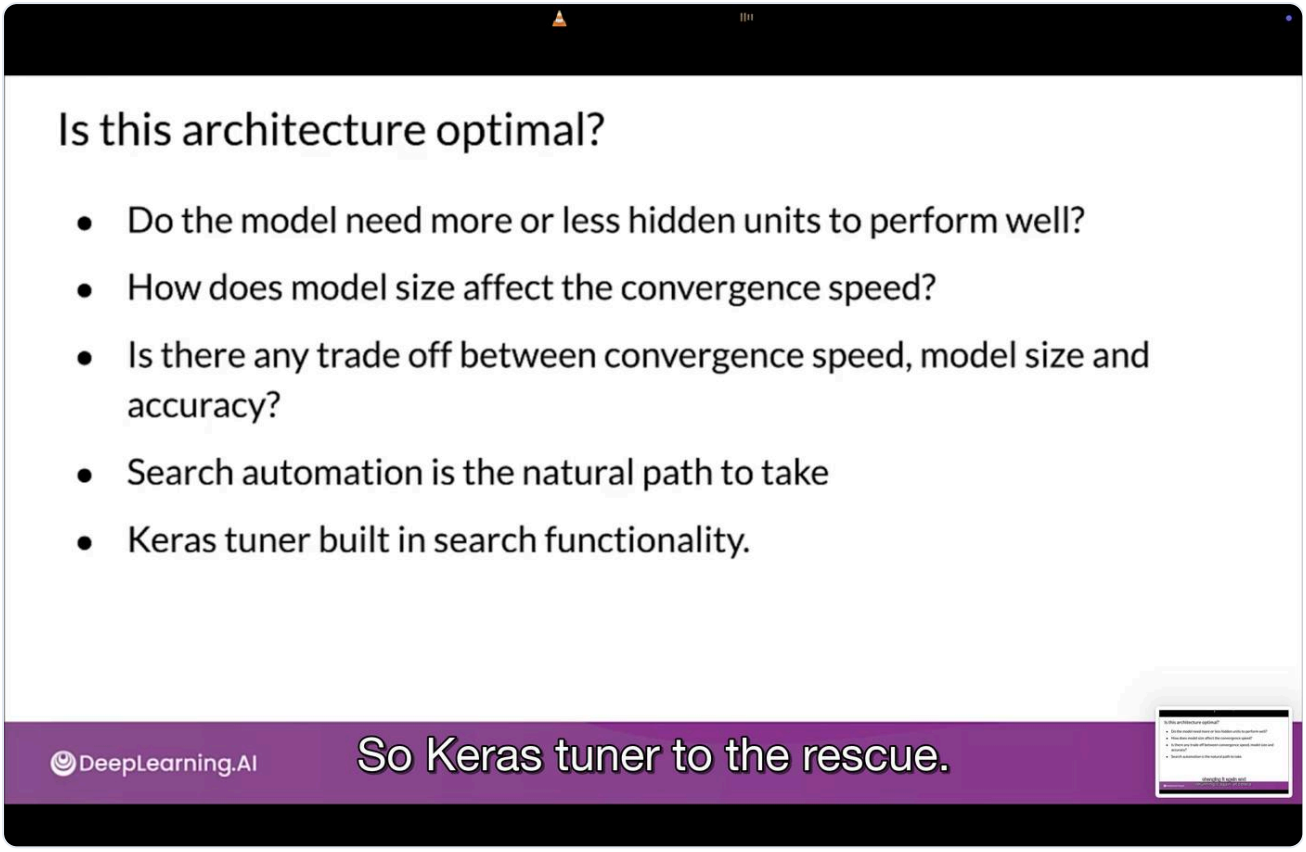

FIGURE 3.6 | Is This Architecture Optimal?

So Keras tuner to the rescue. Let's look at how to install and use it, and maybe change the number of neurons in this space from 16 to 256 in steps of 16. And how to run it to get the best results.

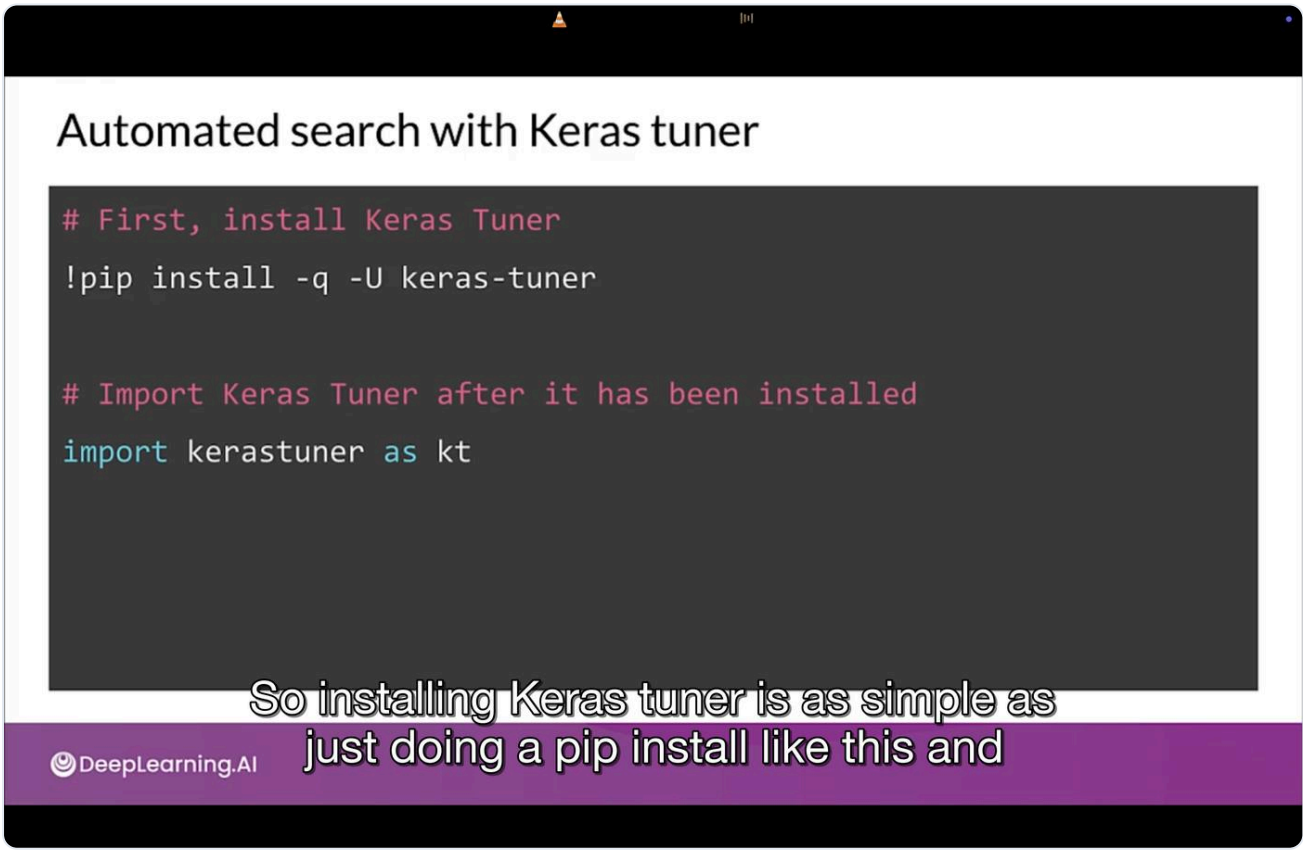

FIGURE 3.7 | Automated Search with Keras Tuner

So installing Keras tuner is as simple as just doing a pip install like this and and then importing it. So we'll import it as kt in this example.

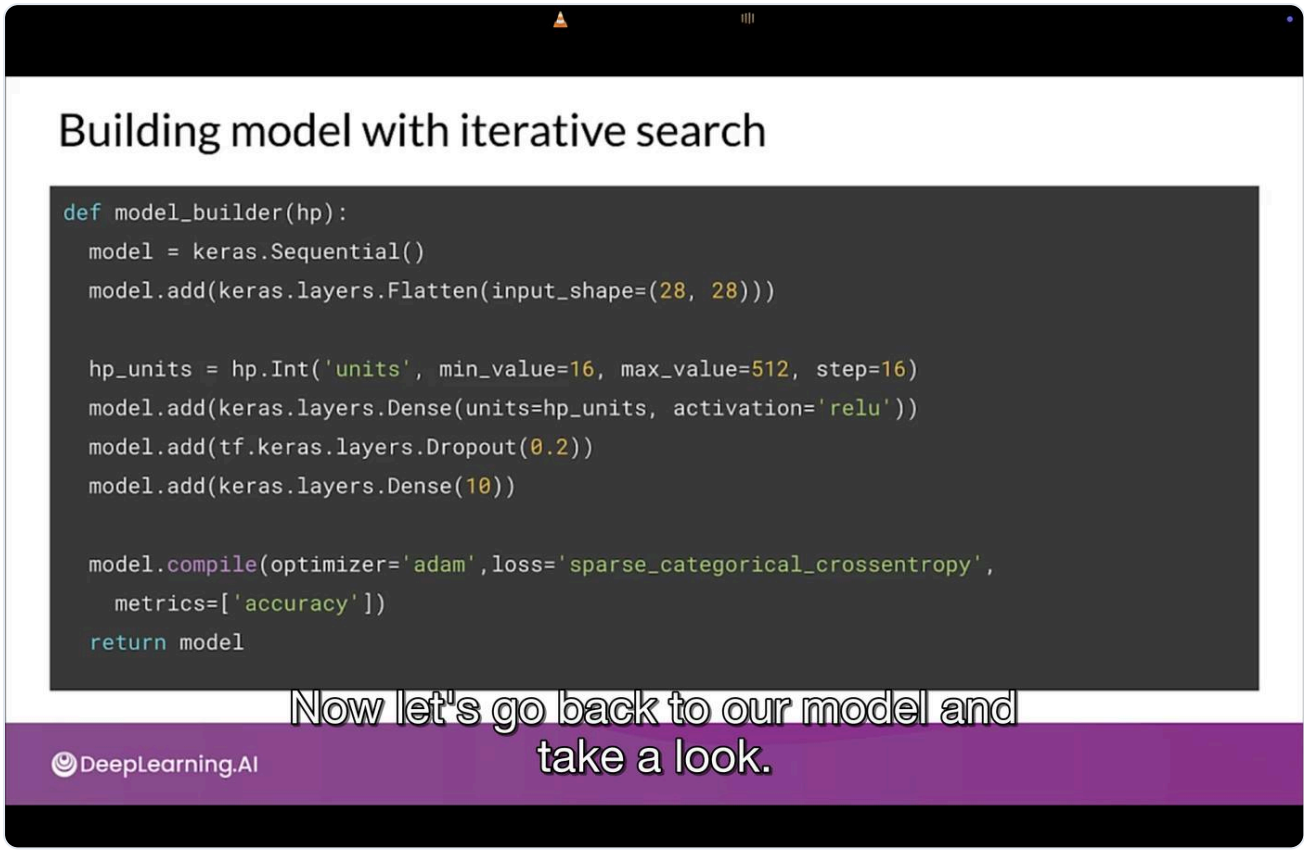

FIGURE 3.8

Now let's go back to our model and take a look. Instead of hard coding 512 neurons into the hidden layer, we'll use this code instead. Notice that the number of units in the first dense layer is set to hp.Int. This will be tuned by the Keras tuner. So this sets up an integer value starting at 16 and going to 512 in steps of 16. Later you'll see how Keras tuner will run the model multiple times, gathering metrics each time and optimizing for the best one. In this case, the value is what drives the number of times kt does its thing. Next, you'll define your search strategy.

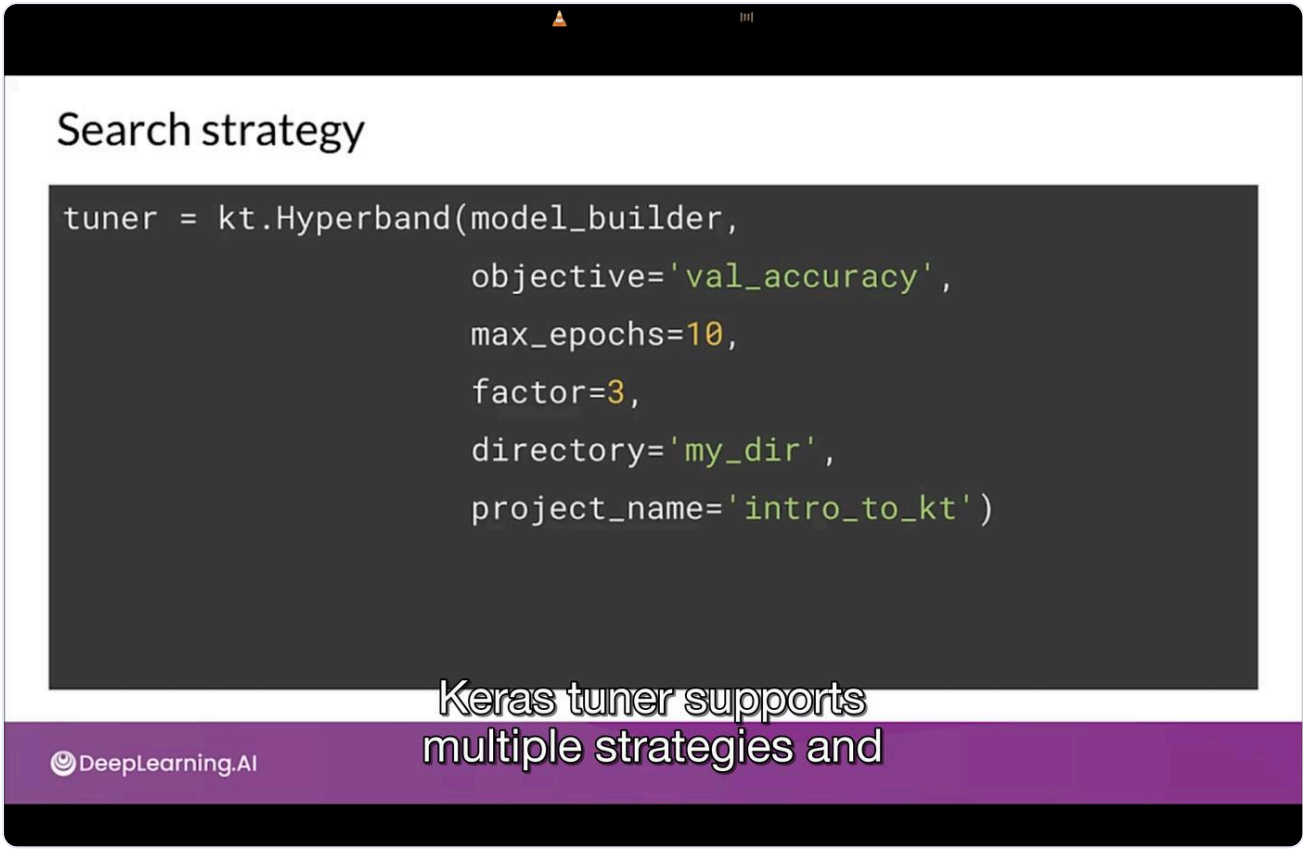

FIGURE 3.9 | Search Strategy

Keras tuner supports multiple strategies and you define which one you want to use like this. In this case, I'm using the Hyperband strategy.

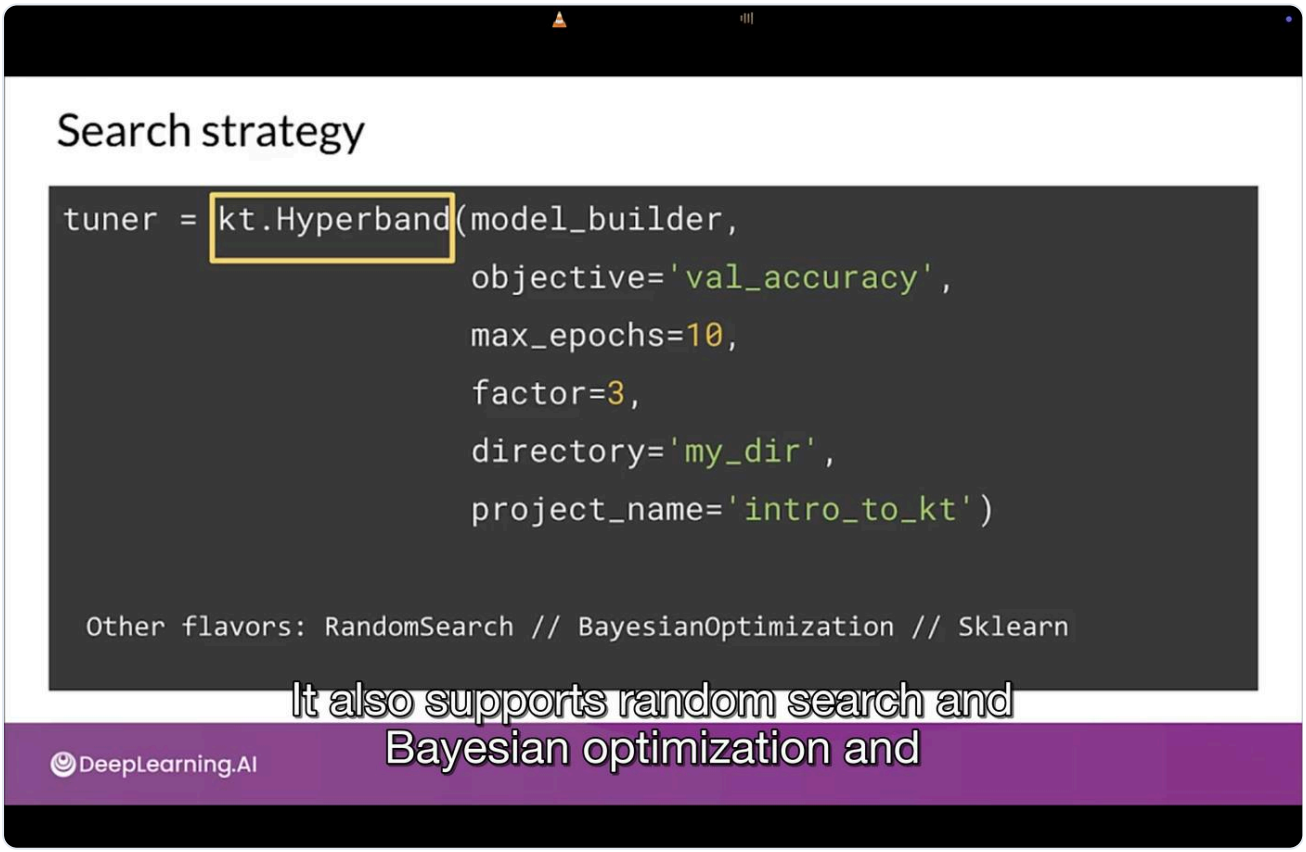

FIGURE 3.10 | Search Strategy

It also supports random search and Bayesian optimization and Sklearn strategies. You can learn more about these at the Keras tuner site at Keras-team.github.io/kerastuner. The parameters you choose will vary based on your strategy. But the important one to note is the objective.

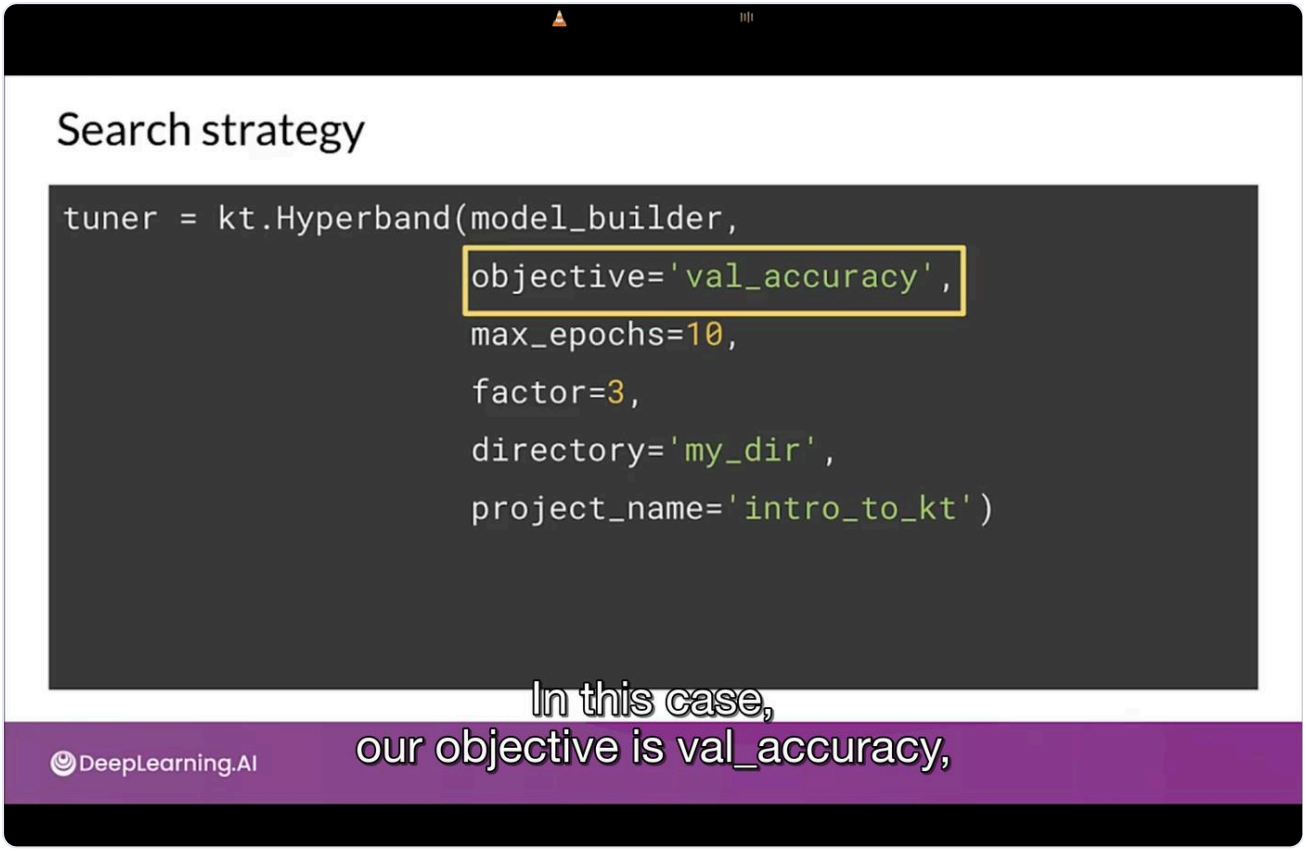

FIGURE 3.11 | Search Strategy

In this case, our objective is val_accuracy, so we want to maximize on the validation accuracy. You can find details on the rest at the Keras site.

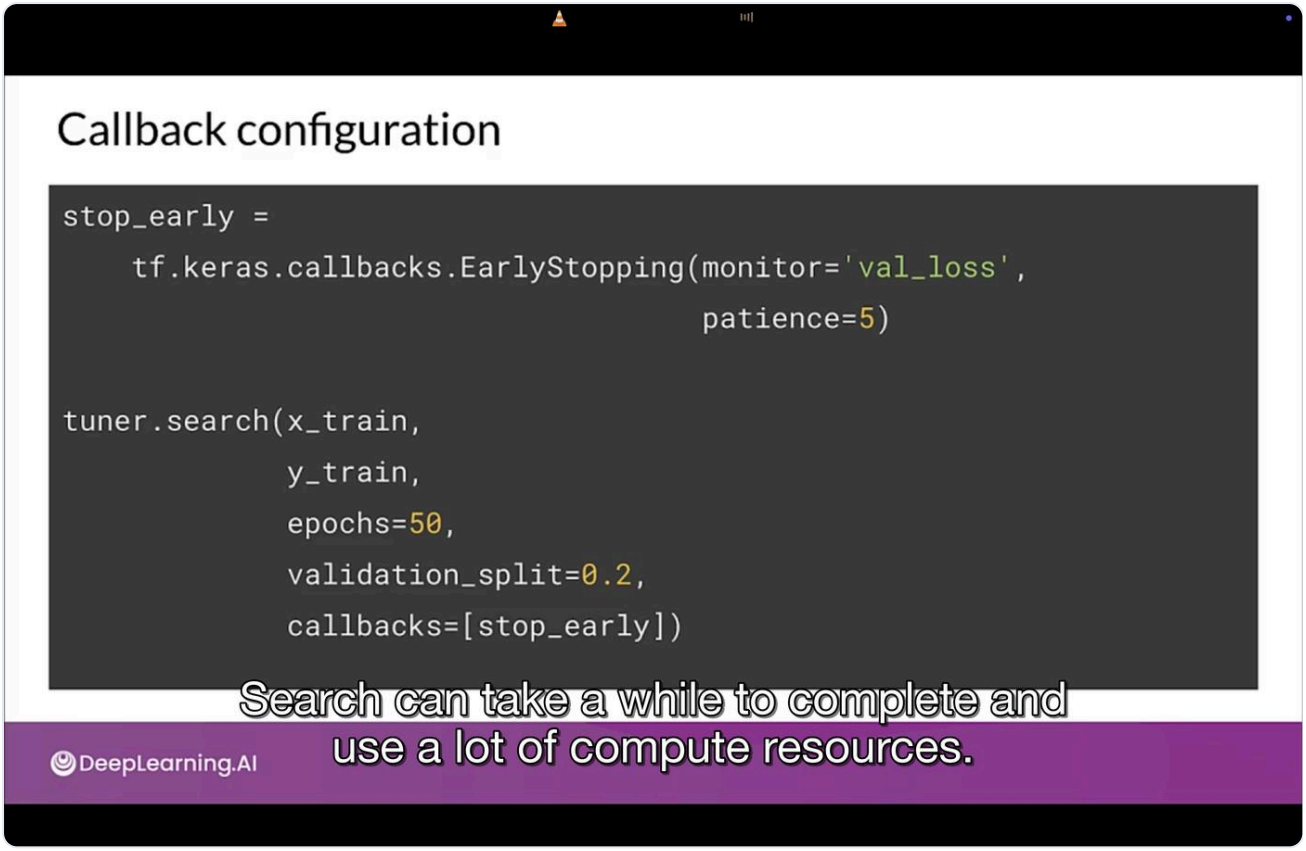

FIGURE 3.12

Search can take a while to complete and use a lot of compute resources. But you can configure a an early stopping callback that stops the search when the conditions are met.

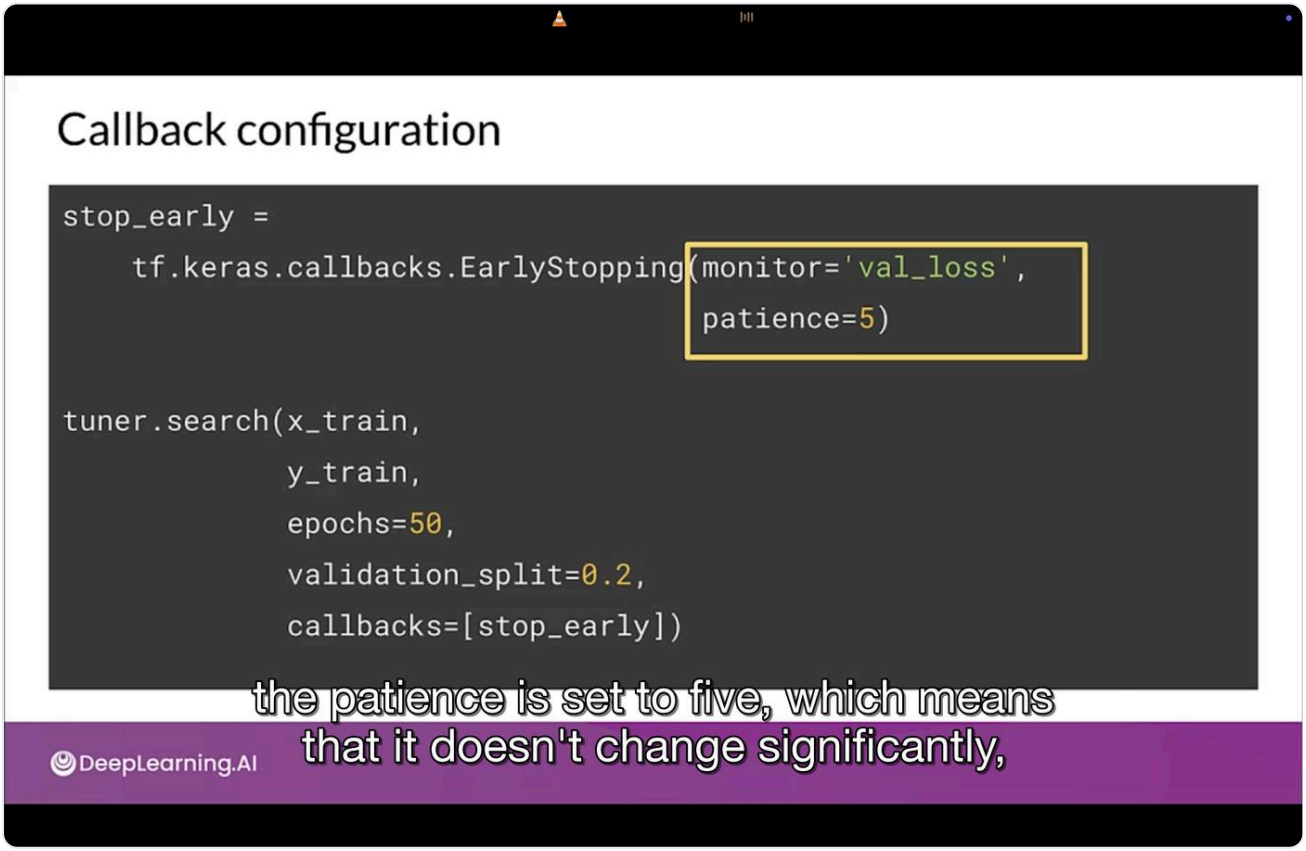

FIGURE 3.13

So for example, here I'm monitoring the validation loss and the patience is set to five, which means that it doesn't change significantly, or if it doesn't change significantly in five epics, then stop searching on this iteration.

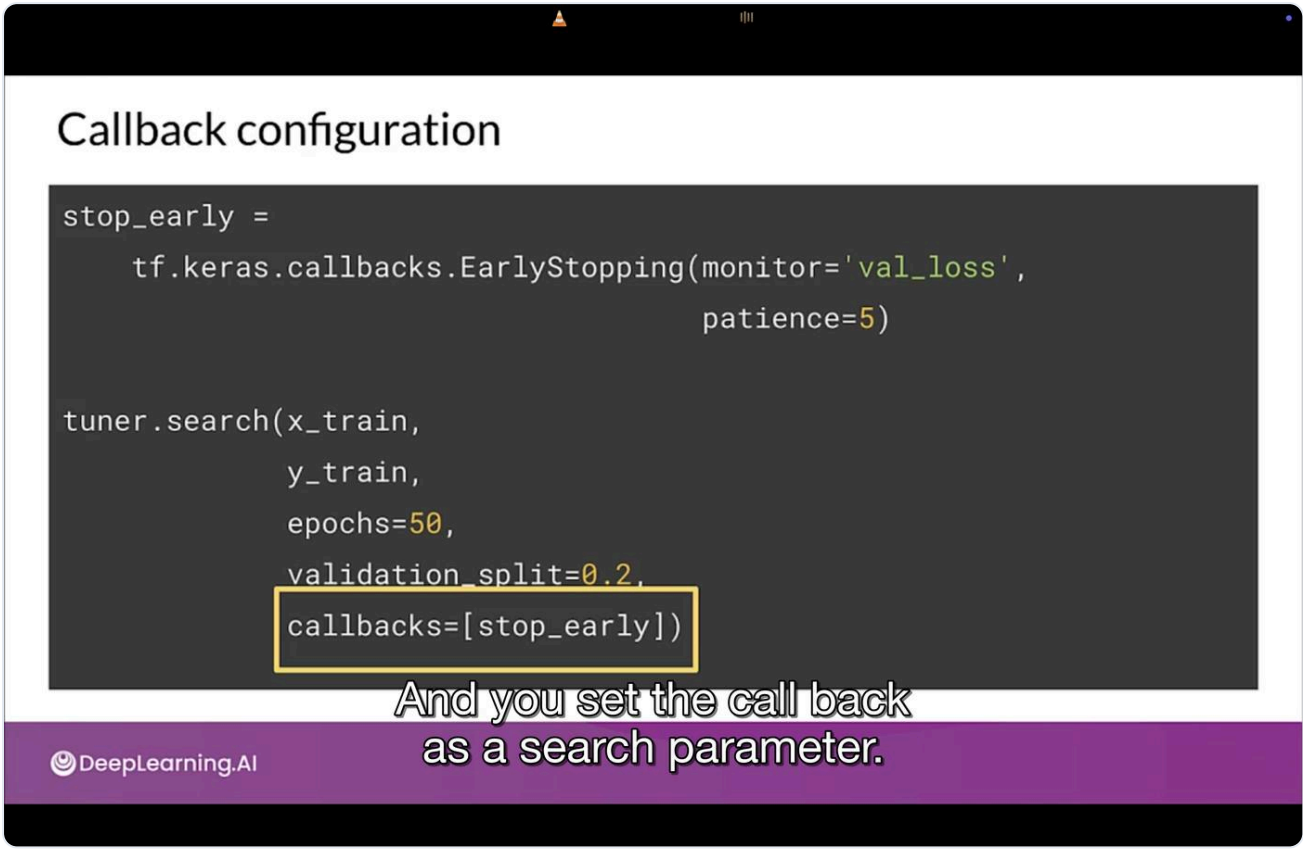

FIGURE 3.14

And you set the call back as a search parameter. The rest of the parameter specify how to search, such as the data and label, the number of epics to train for, and the validation split. So for example how much data you will use to validate against the training set.

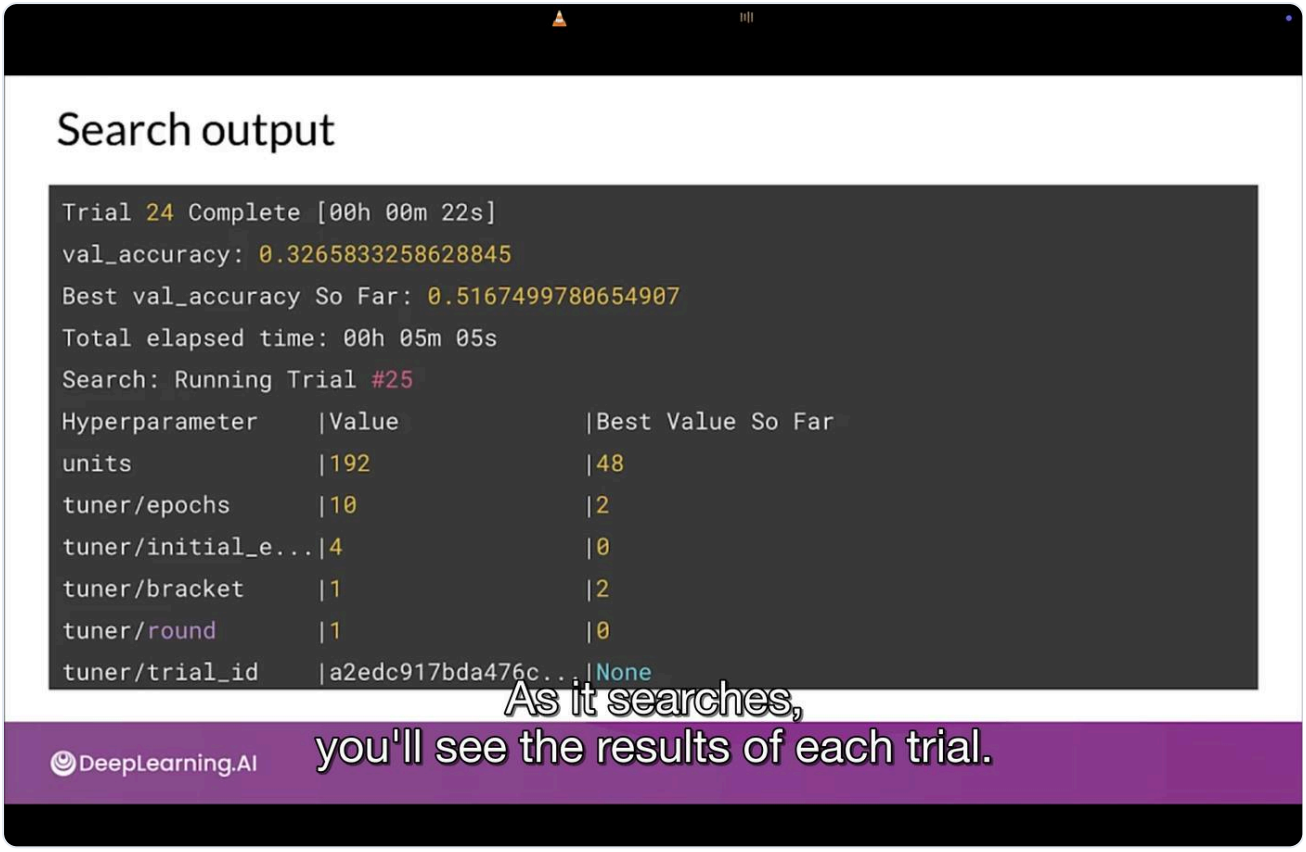

FIGURE 3.15 | Search Output

As it searches, you'll see the results of each trial. We're only searching on one parameter, the units in the dense hidden layer. As you can see as it updates, you can keep track of the best value so far.

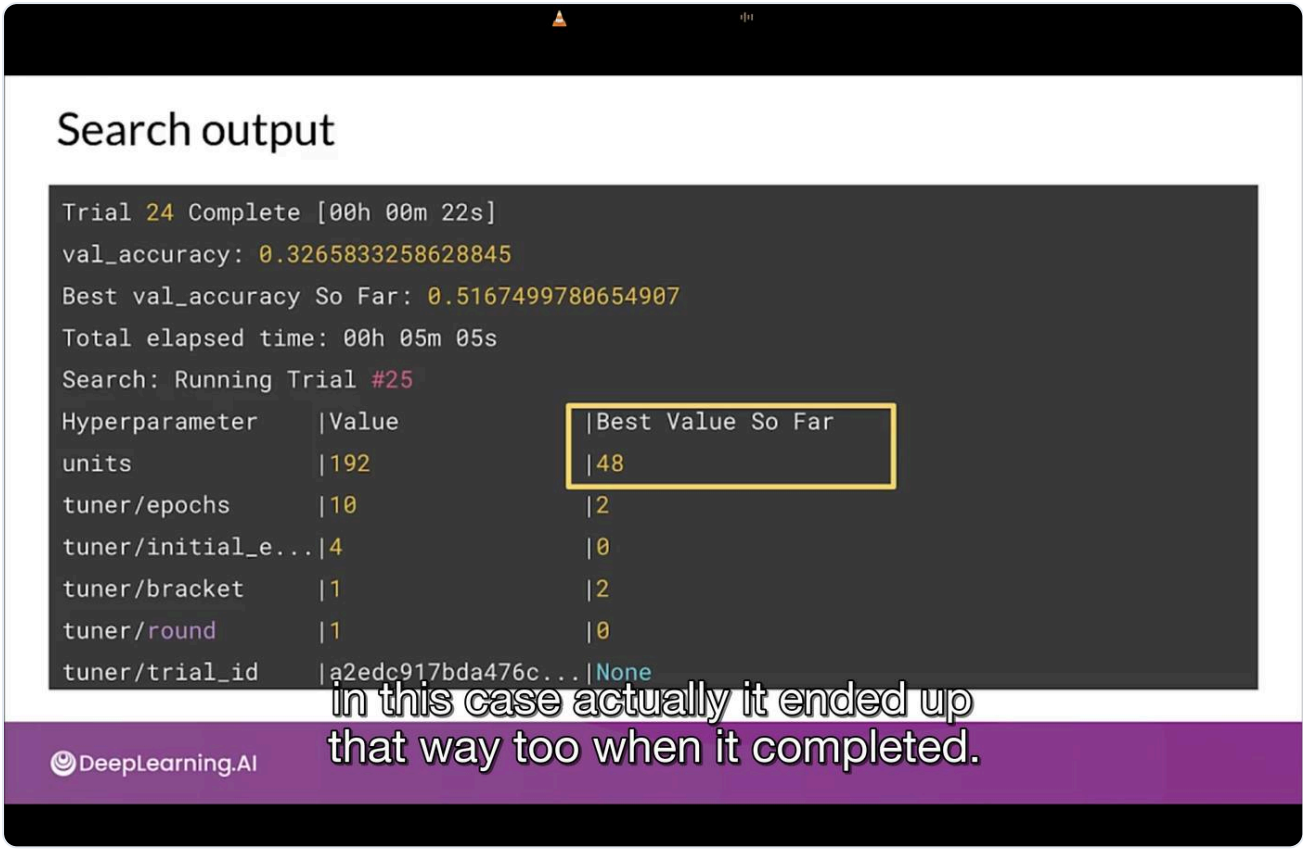

FIGURE 3.16 | Search Output

Which at this point that we have in the slide is 48, and in this case actually it ended up that way too when it completed.

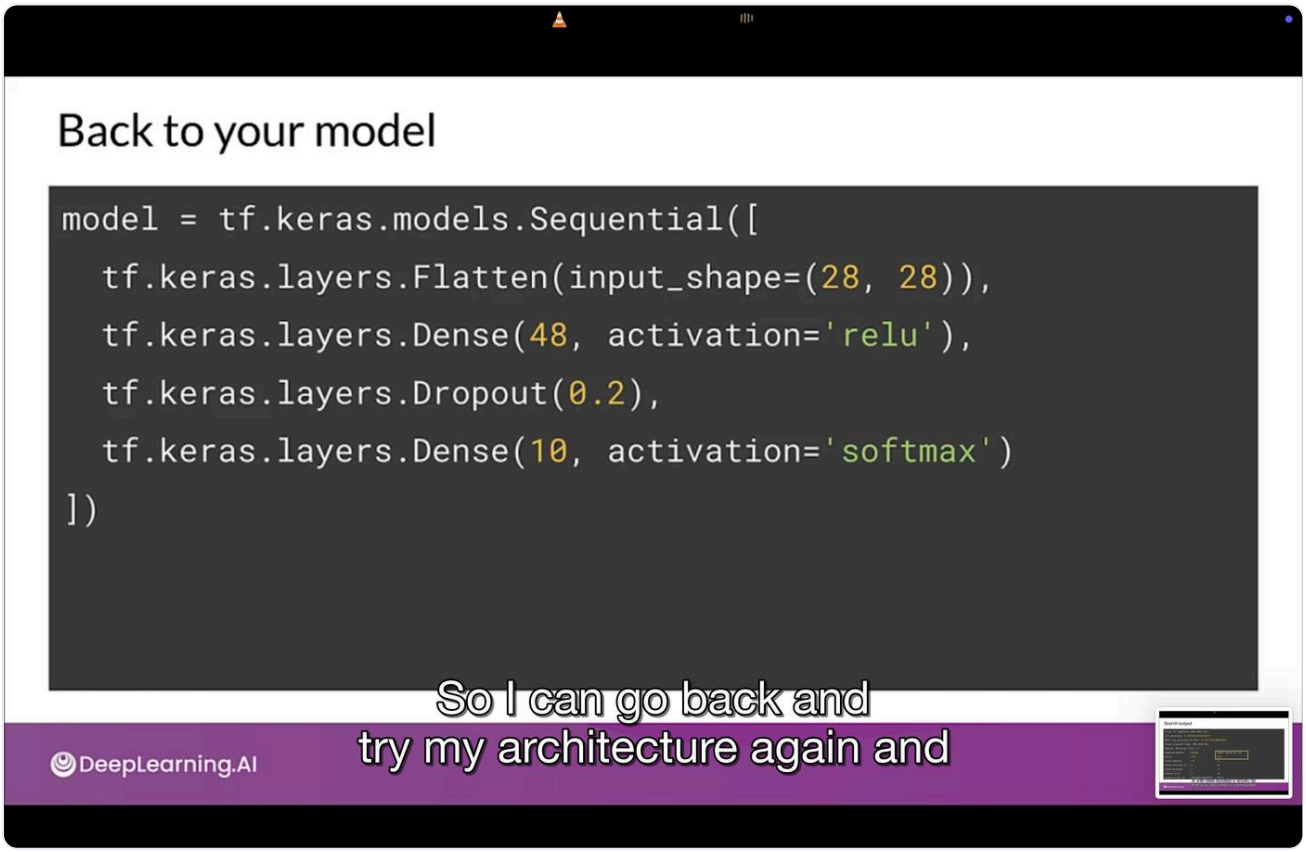

FIGURE 3.17 | Back to Your Model

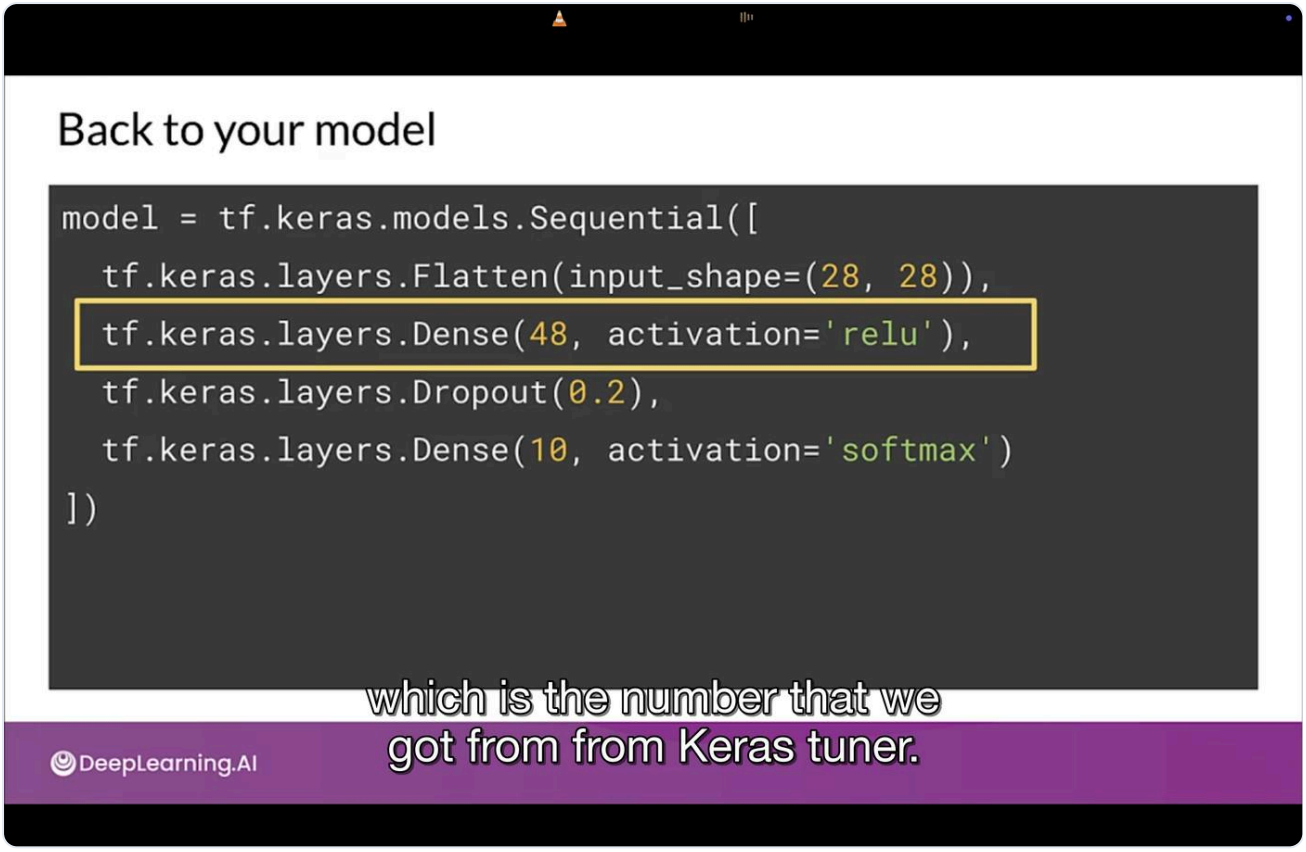

FIGURE 3.18 | Back to Your Model

So I can go back and try my architecture again and use the results of the Keras tuner to set the number of units manually, this time with 48 neurons in the layer, which is the number that we got from from Keras tuner.

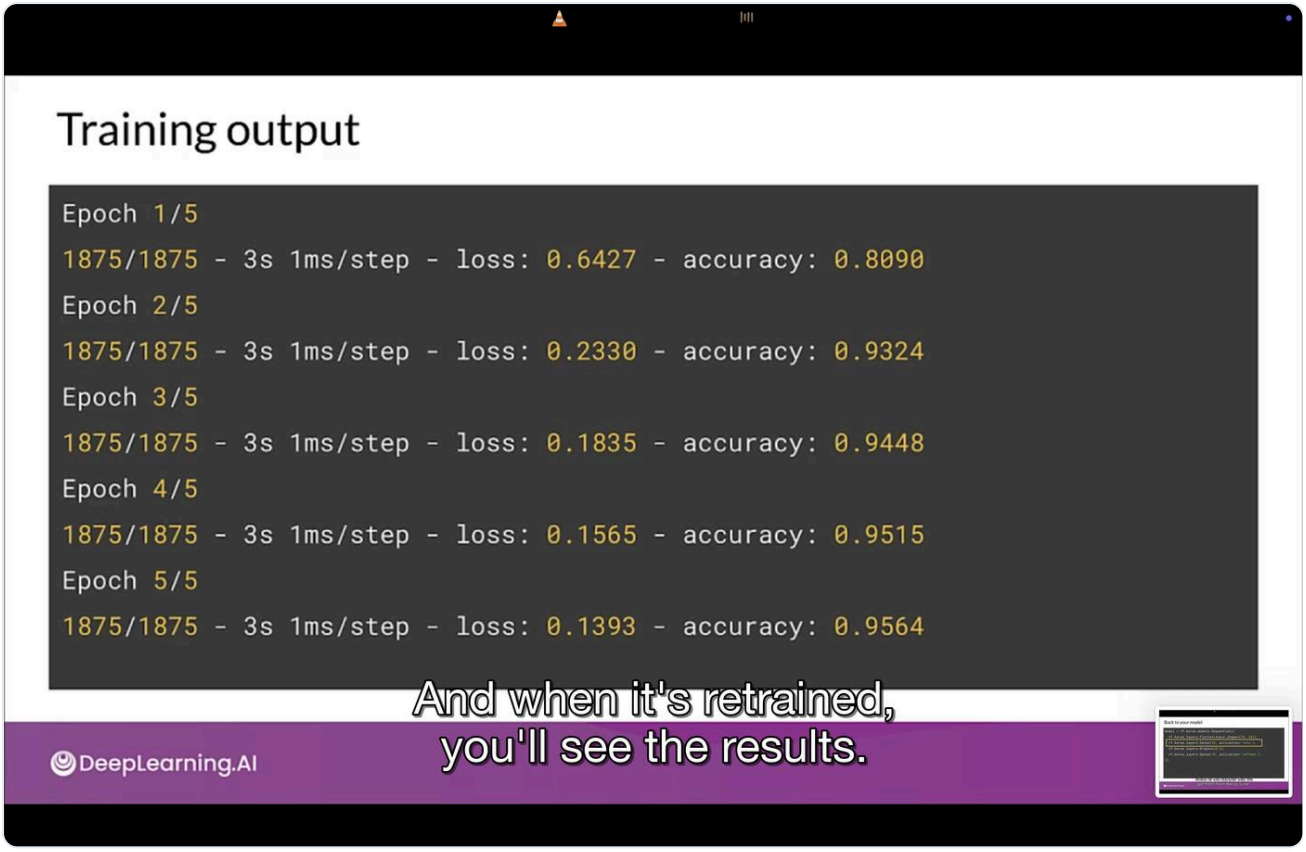

FIGURE 3.19 | Training Output

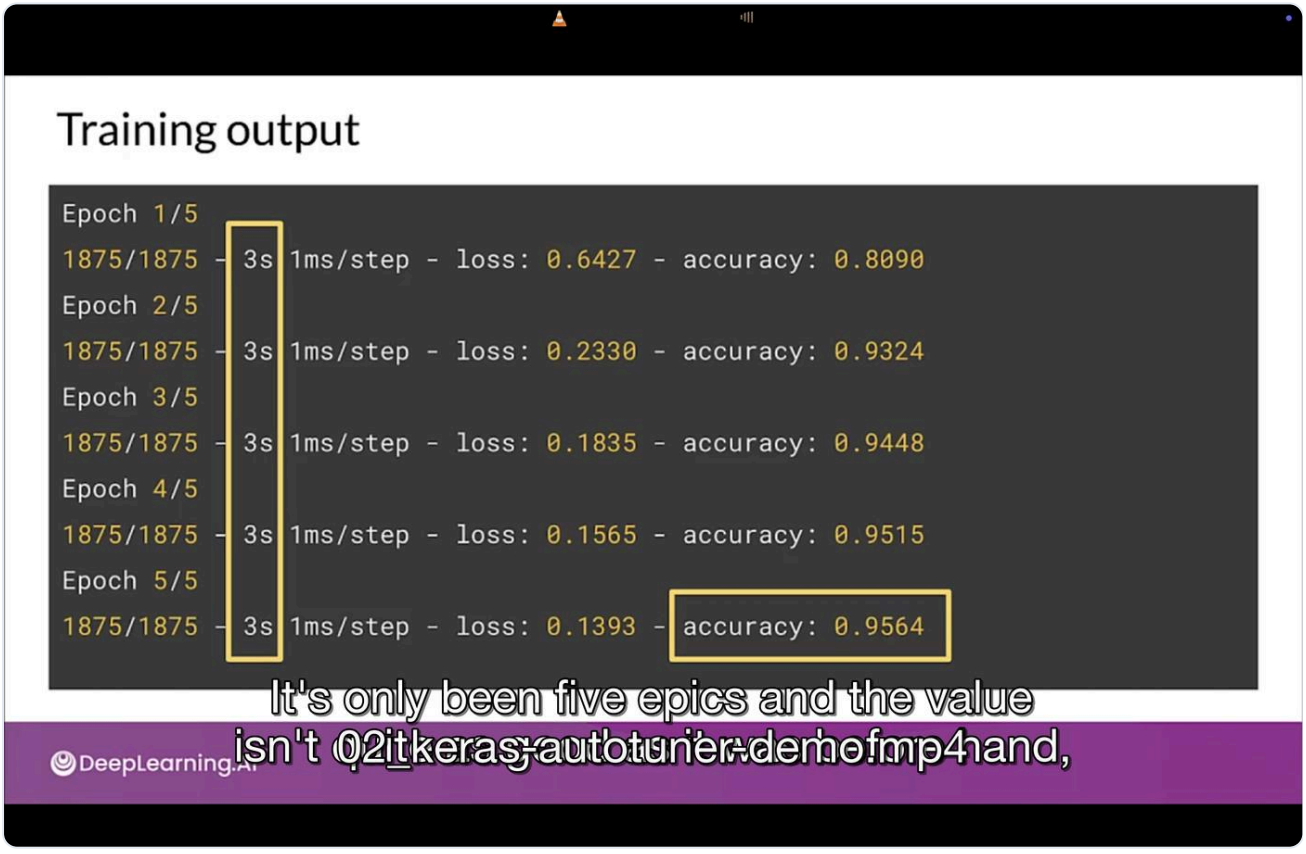

FIGURE 3.20 | Training Output

And when it's retrained, you'll see the results. It's only been five epics and the value isn't quite as good as it was before hand, but our epics are three times quicker than they were. So I can maybe train for longer to get better results knowing that at the very least I've optimized part of my architecture.

LECTURE 4

Automated Machine Learning (AutoML)

So now that we've looked at hyper parameter tuning, let's take a look at actual AutoML. In this lesson, we'll be discussing AutoML. A set of very versatile tools to automate the machine learning process end to end. Finding the right model is an important piece of the puzzle. Neural Architecture Search is a process to help find relevant models. You'll be learning about search spaces and strategies which are key for both hyper parameter tuning and AutoML. We'll also discuss tools to quantify performance estimation on the explored models in your search.

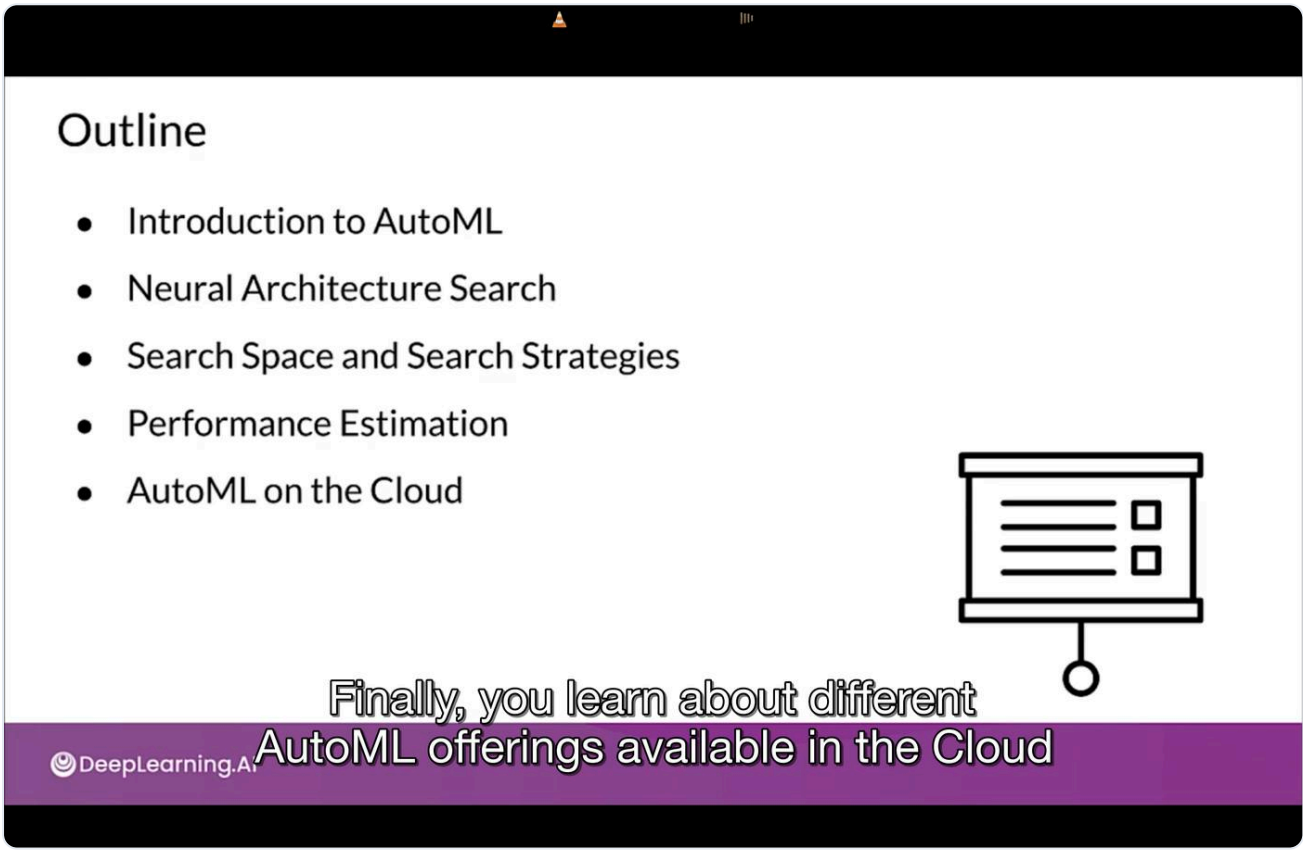

FIGURE 4.1

Finally, you learn about different AutoML offerings available in the Cloud by major service providers. Automated Machine Learning or AutoML is aimed at enabling developers with very little experience in machine learning, to make use of machine learning models and techniques. It tries to automate the process of machine learning end to end, in order to produce simple solutions, faster creation of those solutions and models that sometimes outperform even hand tuned models.

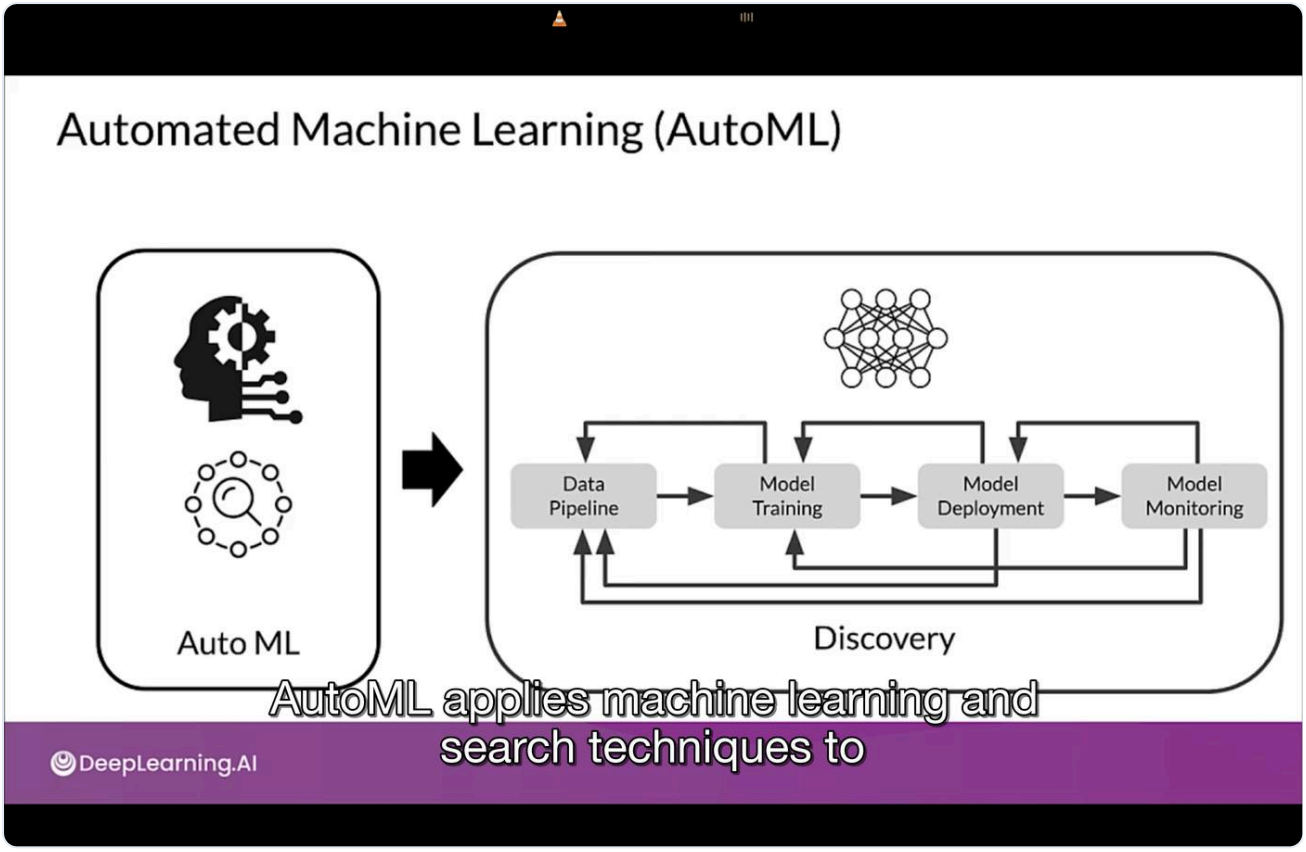

FIGURE 4.2 | Automated Machine Learning (AutoML)

AutoML applies machine learning and search techniques to the process of creating machine learning models and pipelines. It covers the complete pipeline from the raw data set to the deployable machine learning model. In traditional machine learning, we write code for all of the phases of the process. We start off with ingesting and cleansing the raw data and then perform feature selection and feature engineering.

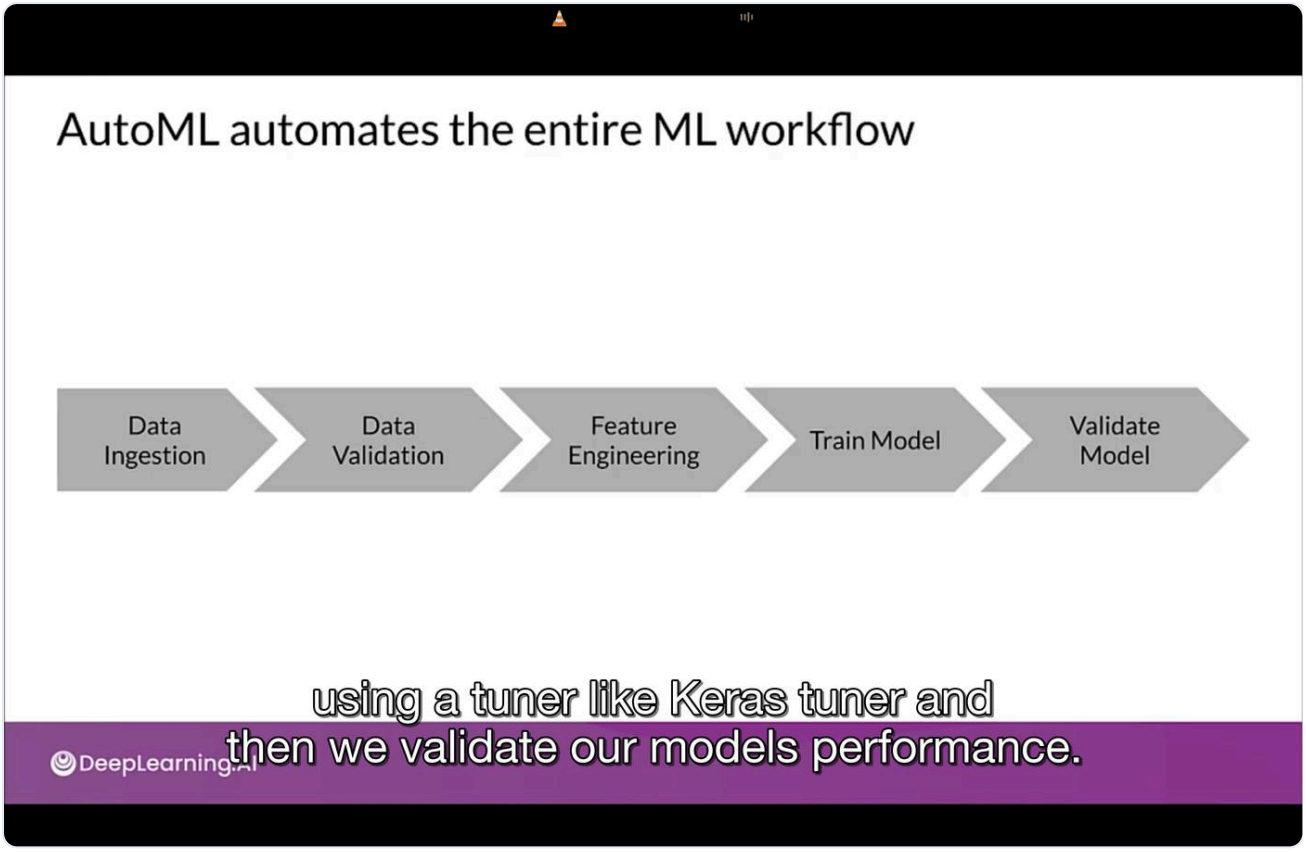

FIGURE 4.3 | AutoML Automates the Entire ML Workflow

We select a model architecture for a task, train or model and perform hyper parameter tuning, maybe manually or using a tuner like Keras tuner and then we validate our models performance. ML requires a lot of manual programming and a highly specialized skill set. Auto ML aims to automate the entire ML workflow. If we can provide the AutoML system with raw data and our model validation requirements. It goes through all of the phases in the ML workflow and performs the iterative process of ML development in a systematic way until the model is trained. Neural Architecture Search or NAS is at the heart of AutoML. There are three main parts to Neural Architecture Search, a search space, a search strategy and a performance estimation strategy.

The search space defines the range of architectures which can be represented. To reduce the size of the search problem, we need to limit the search space to the architectures which are best suited to the problem that we're trying to model. This helps reduce the search space, but it also means that a human bias will be introduced, which might prevent Neural Architecture Search from finding architectural blocks that go beyond current human knowledge. The search strategy defines how we explore the search space. We want to explore the search based quickly, but this might lead to premature convergence to a sub optimal region in the search space. The objective of Neural Architecture Search is to find architecture is that perform well on our data.

The performance estimation strategy helps in measuring and comparing the performance of various architectures.

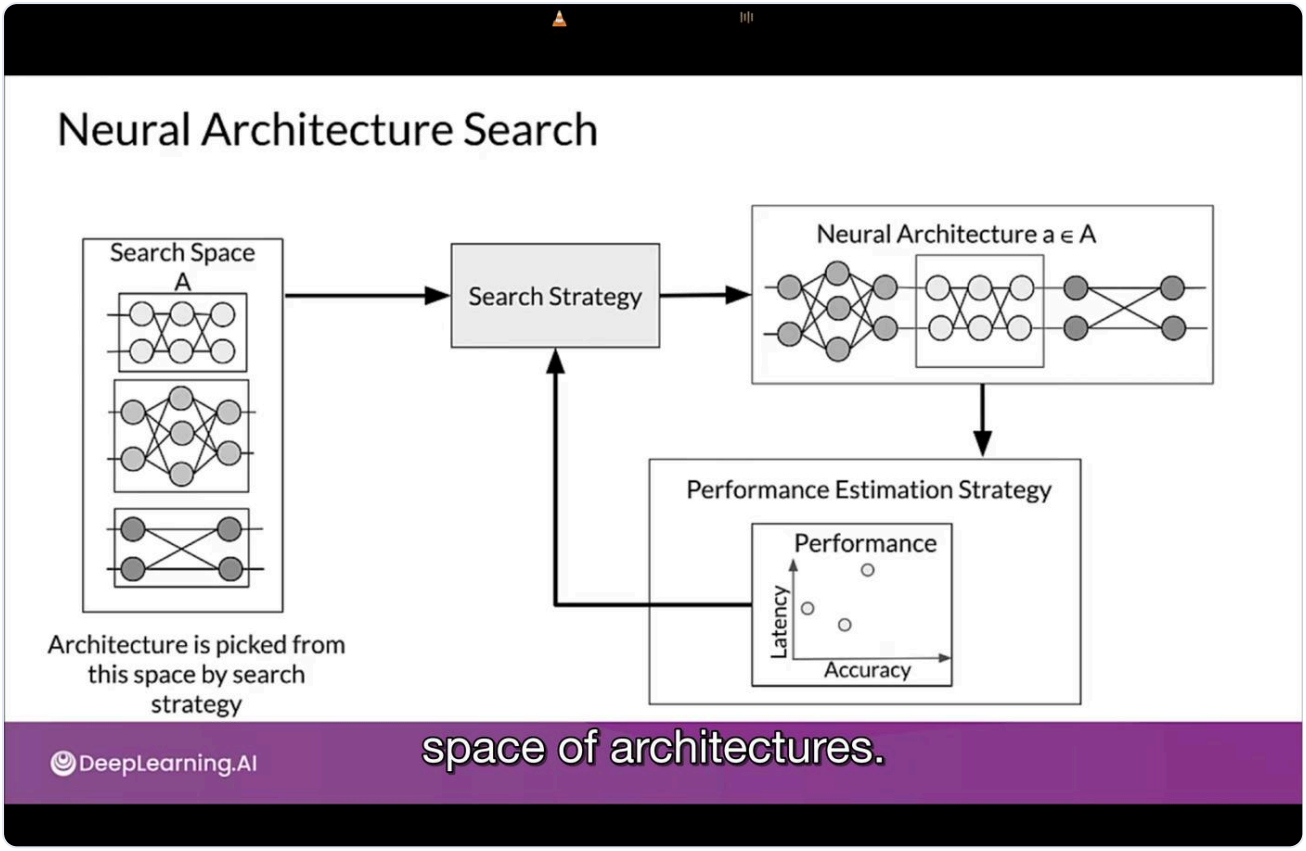

FIGURE 4.4

A search strategies selects an architecture from a predefined search space of architectures. The selected architecture is passed to a performance estimation strategy, which returns it's estimated performance to the search strategy. The search space, search strategy and performance estimation strategy are the key components of Neural Architecture Search and will be discussing each of them in turn. AutoML facilitates building models in an automated fashion. Furthermore, AutoML is not restricted to a specific family or subset of models.

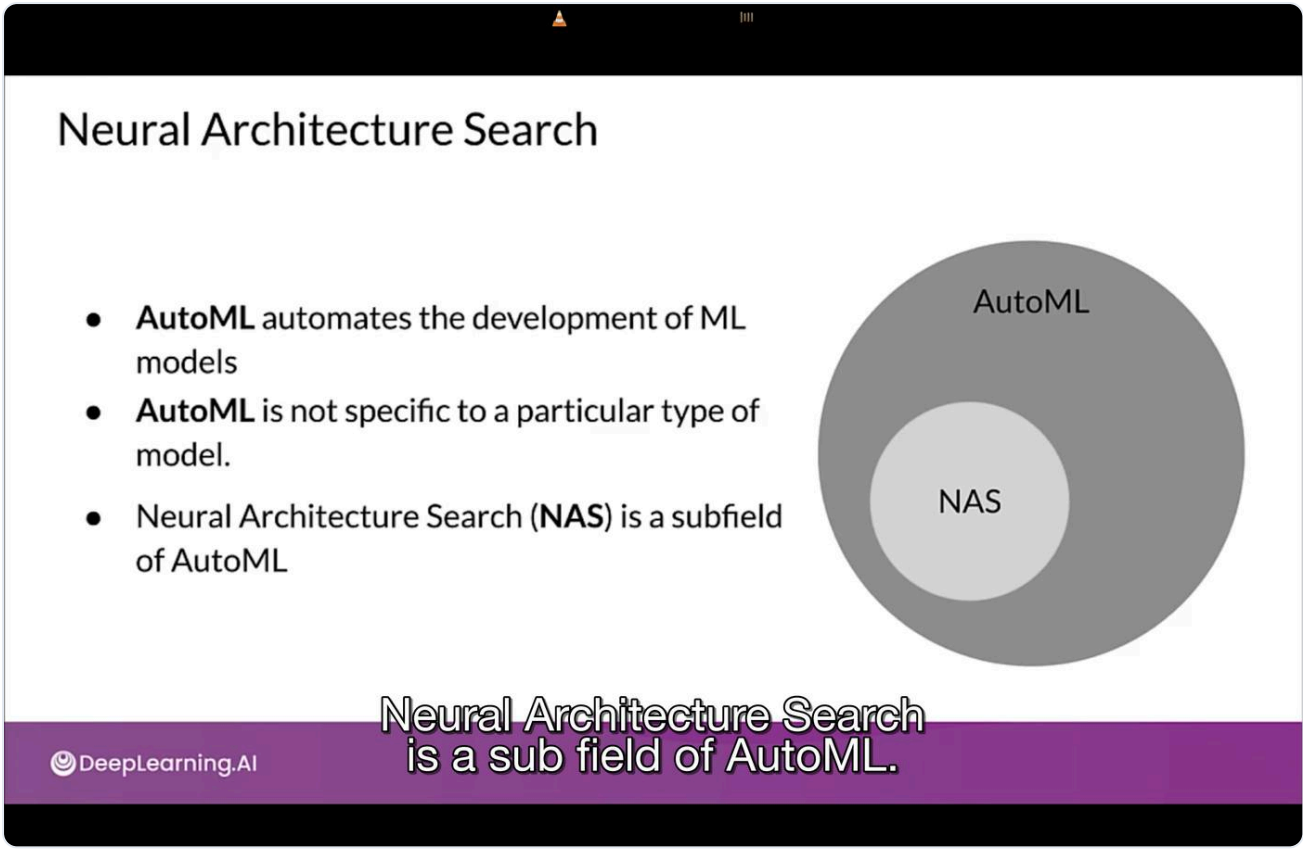

FIGURE 4.5

Neural Architecture Search is a sub field of AutoML.

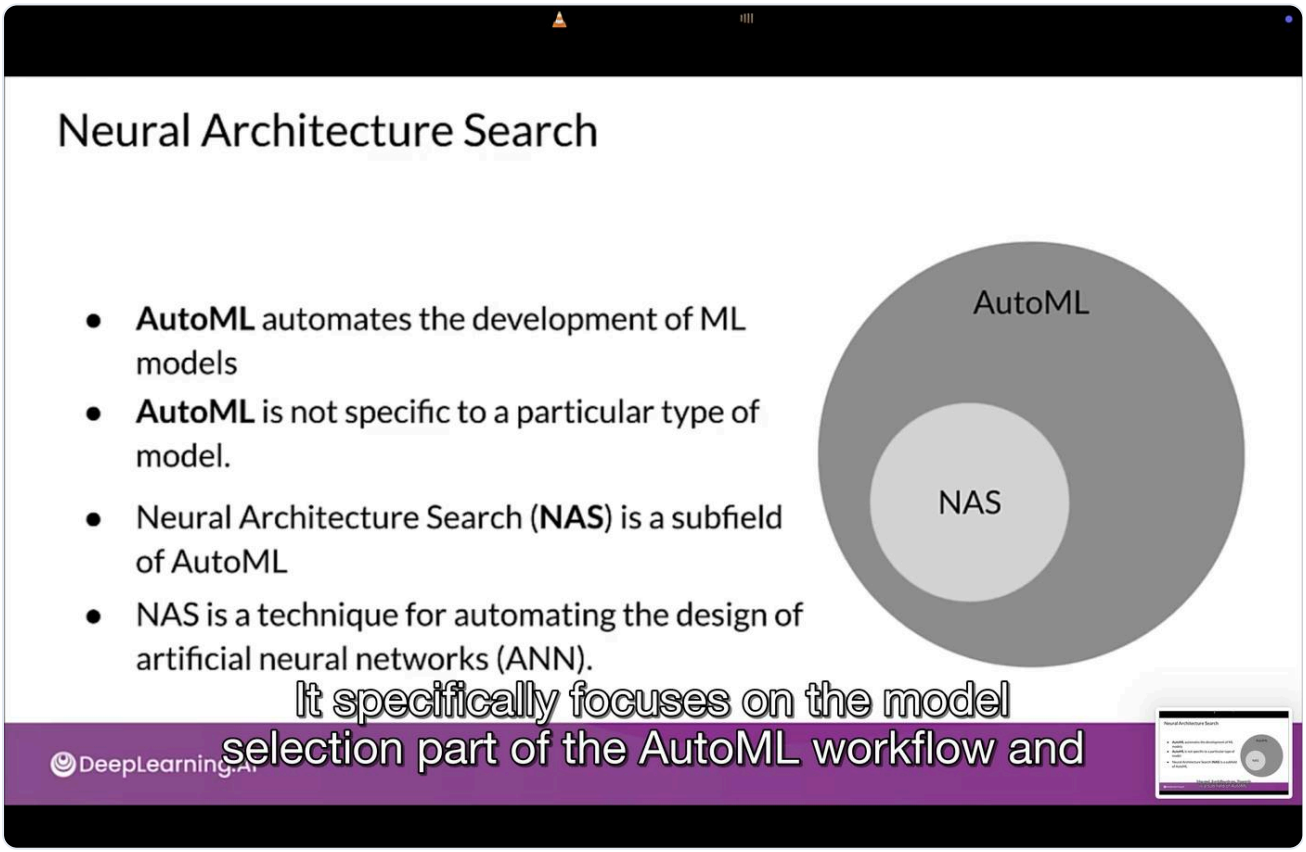

FIGURE 4.6 | Neural Architecture Search

It specifically focuses on the model selection part of the AutoML workflow and the design of neural networks. Neural Architecture Search has been used to design architectures that are on par with or outperform hand designed models. Let's look at a real-world use of AutoML. Meredith Digital is a publishing company specializing in multiple formats of media and entertainment. Meredith Digital uses AutoML to train models, mostly natural language based to automate content classification. AutoML speeds up the classification process by reducing the process for months to just a few days. It also helps by providing insightful, actionable recommendations to help build customer loyalty.

It also helps by identifying new user trends and customer interests, to adapt content to better serve customers.

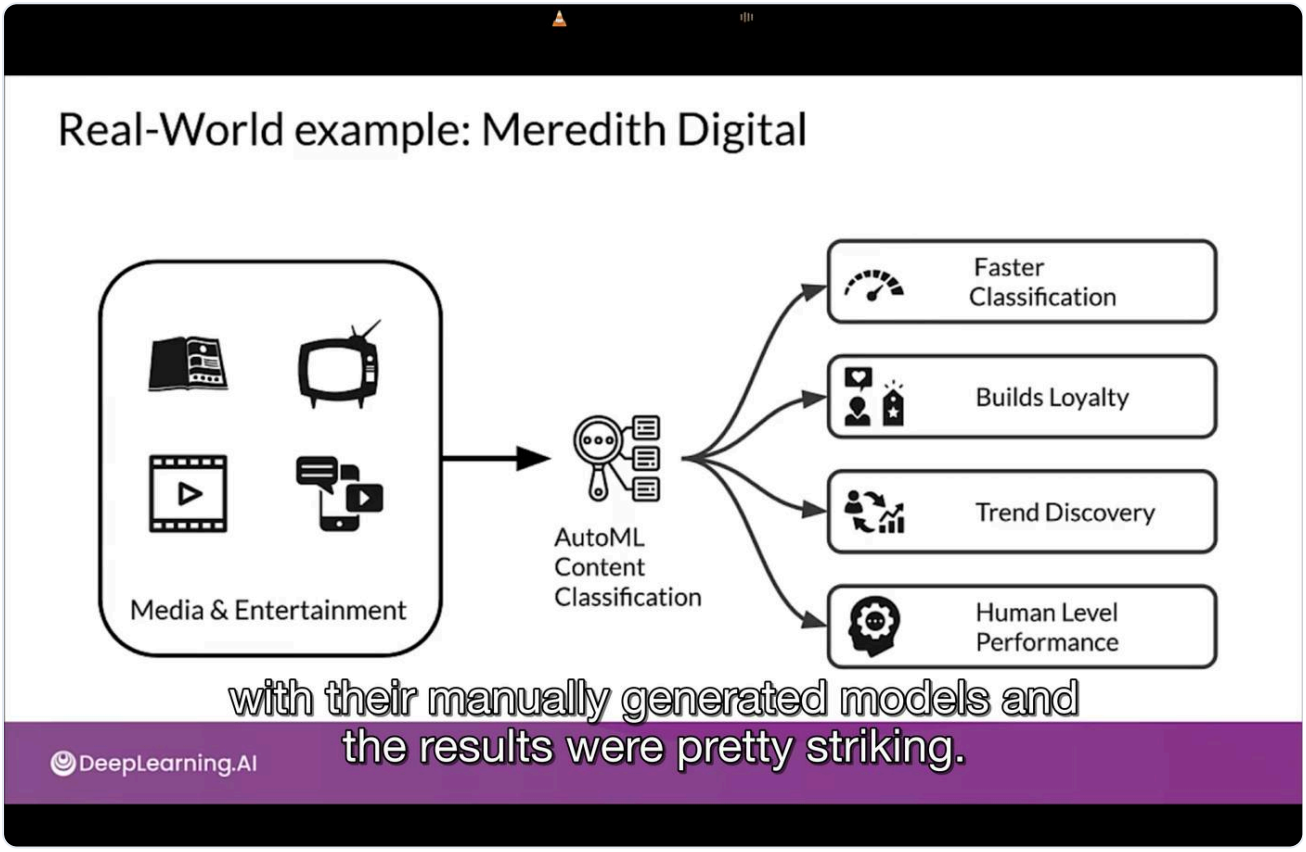

FIGURE 4.7

To test its effectiveness, they conducted a test where they compared AutoML with their manually generated models and the results were pretty striking. The Google Cloud AutoML natural language tools provided content classification that was comparable to human level performance.

LECTURE 5

Micro Architecture Search Space

Now let's continue our discussion of AutoML by discussing search spaces. There are two main types of search spaces, macro and micro.

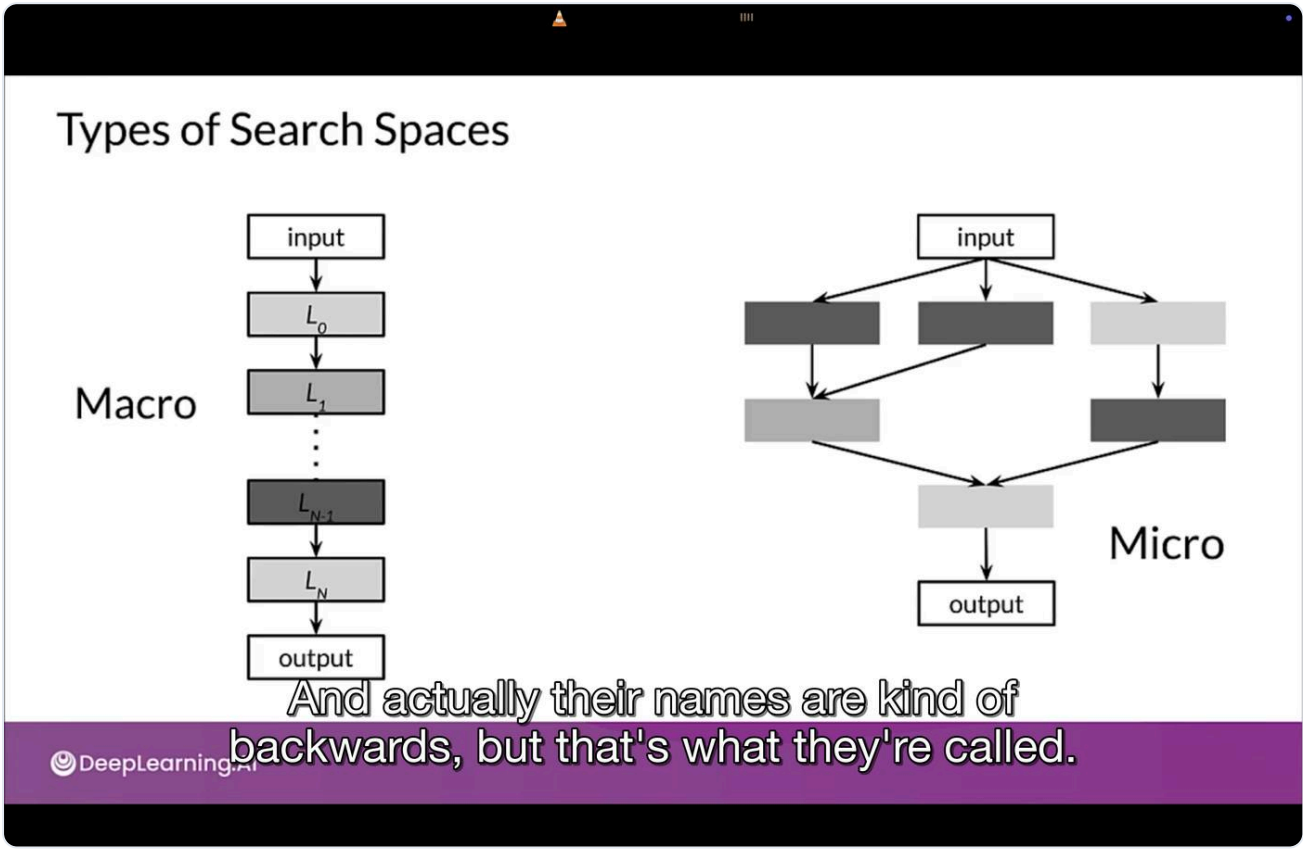

FIGURE 5.1 | Types of Search Spaces

And actually their names are kind of backwards, but that's what they're called. Let's look at both.

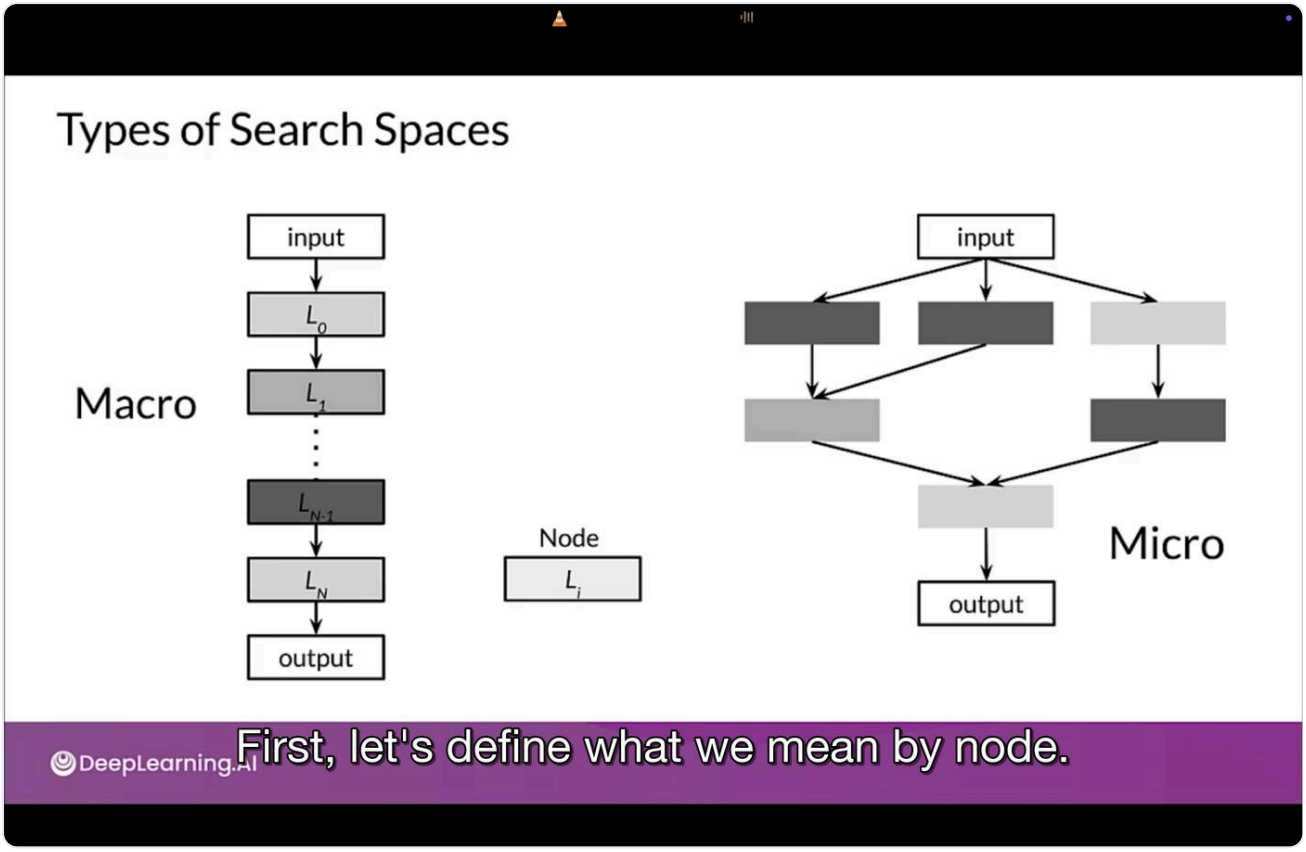

FIGURE 5.2 | Types of Search Spaces

First, let's define what we mean by node. A node is a layer in a neural network like a convolution or pooling layer. In this illustration, each color represents a different layer type.

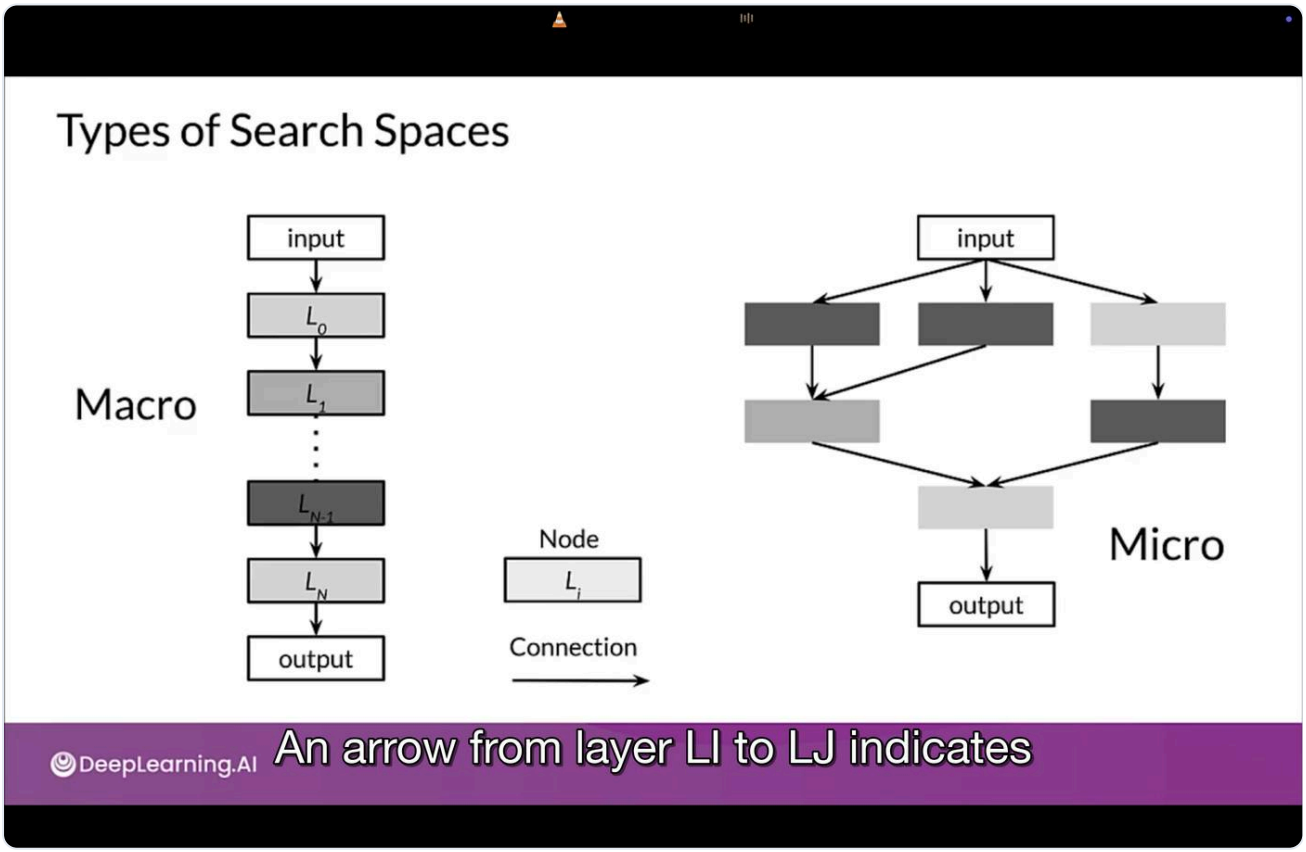

FIGURE 5.3 | Types of Search Spaces

An arrow from layer LI to LJ indicates that layer LJ receives the output of LI as input. A macro search space contains the individual layers and connection types of a neural network. And neural architecture search searches within that space for the best model, building the model layer by layer.

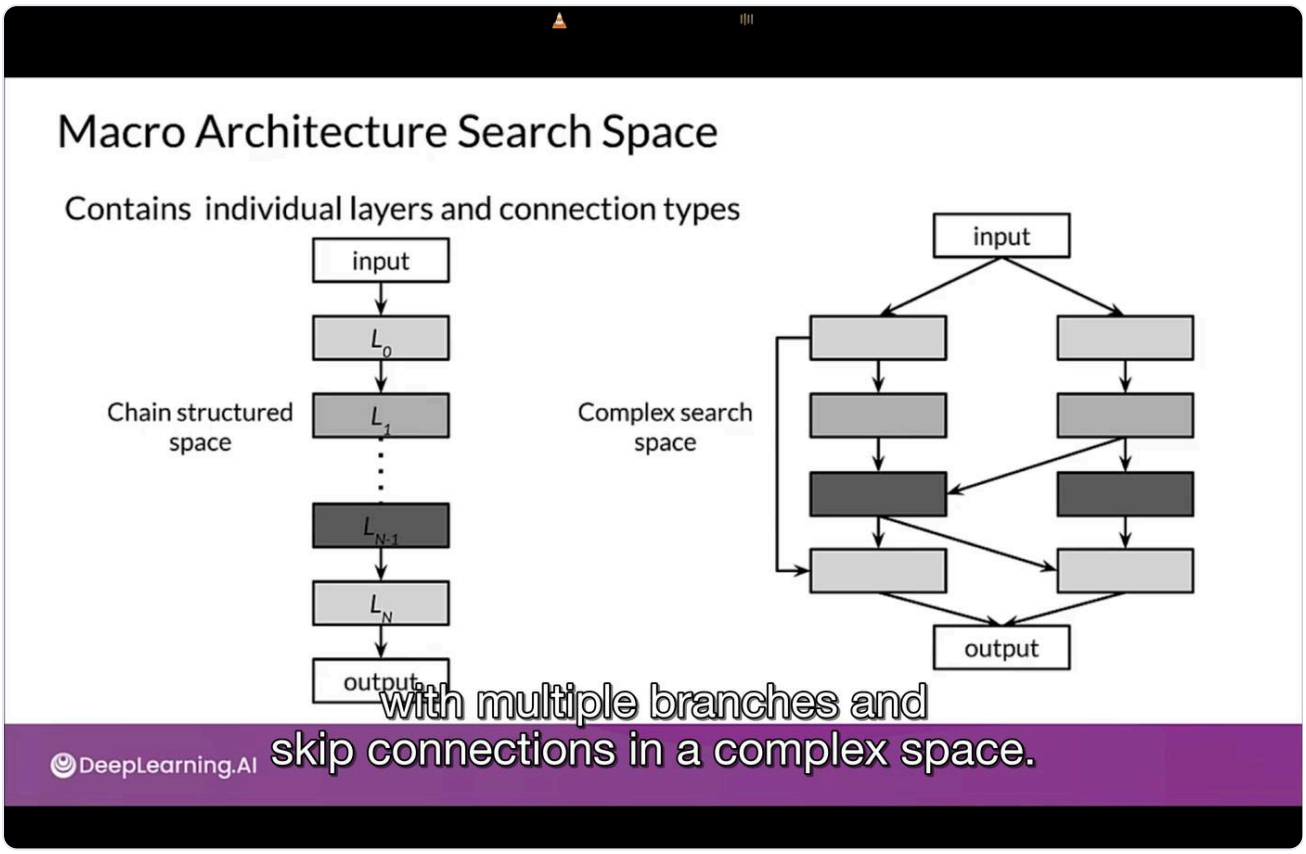

FIGURE 5.4 | Macro Architecture Search Space

As shown here, a network can be built very simply by stacking individual layers referred to as a chain structured space or with multiple branches and skip connections in a complex space.

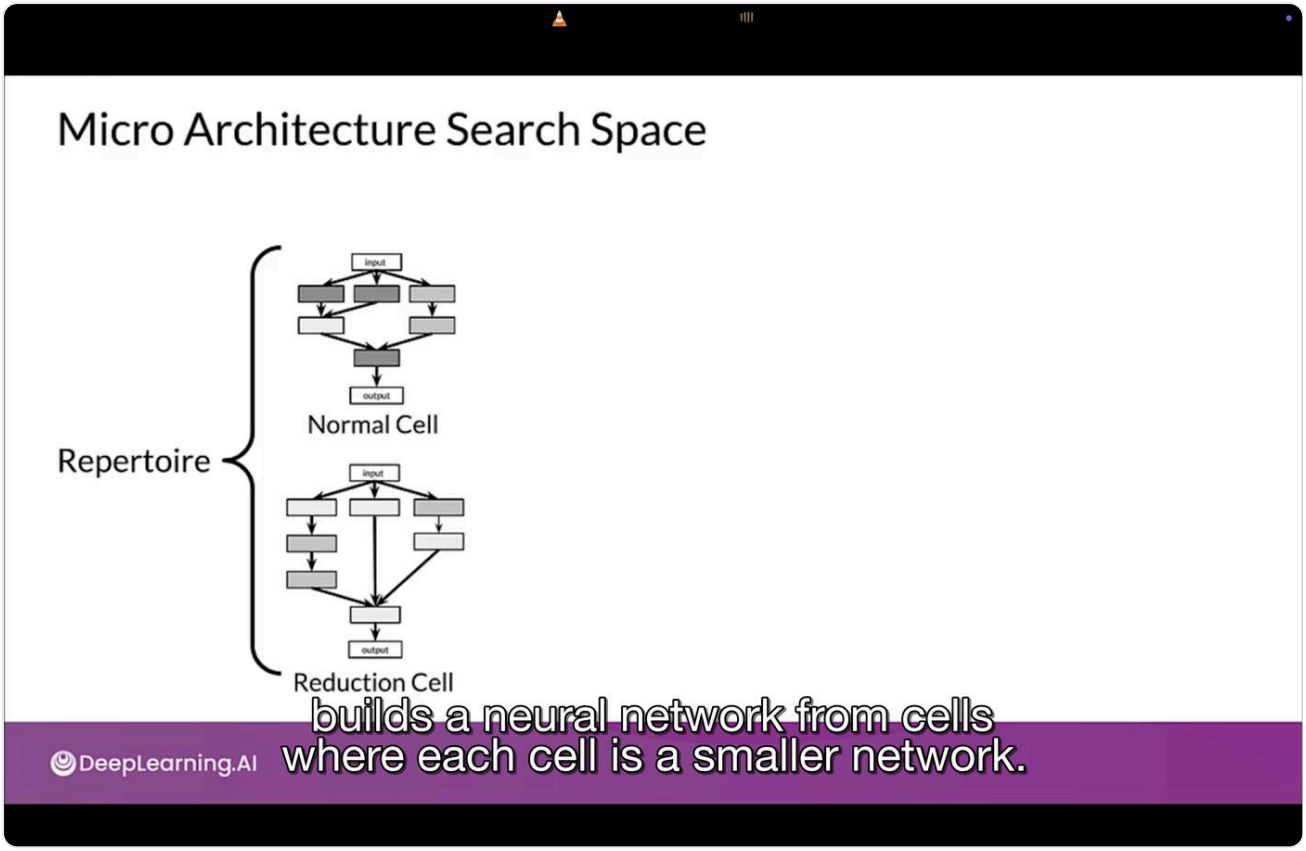

FIGURE 5.5 | Micro Architecture Search Space

By contrast, in a micro search space, neural architecture search builds a neural network from cells where each cell is a smaller network. Here are two different types of cells, a normal cell on the top and a reduction cell on the bottom.

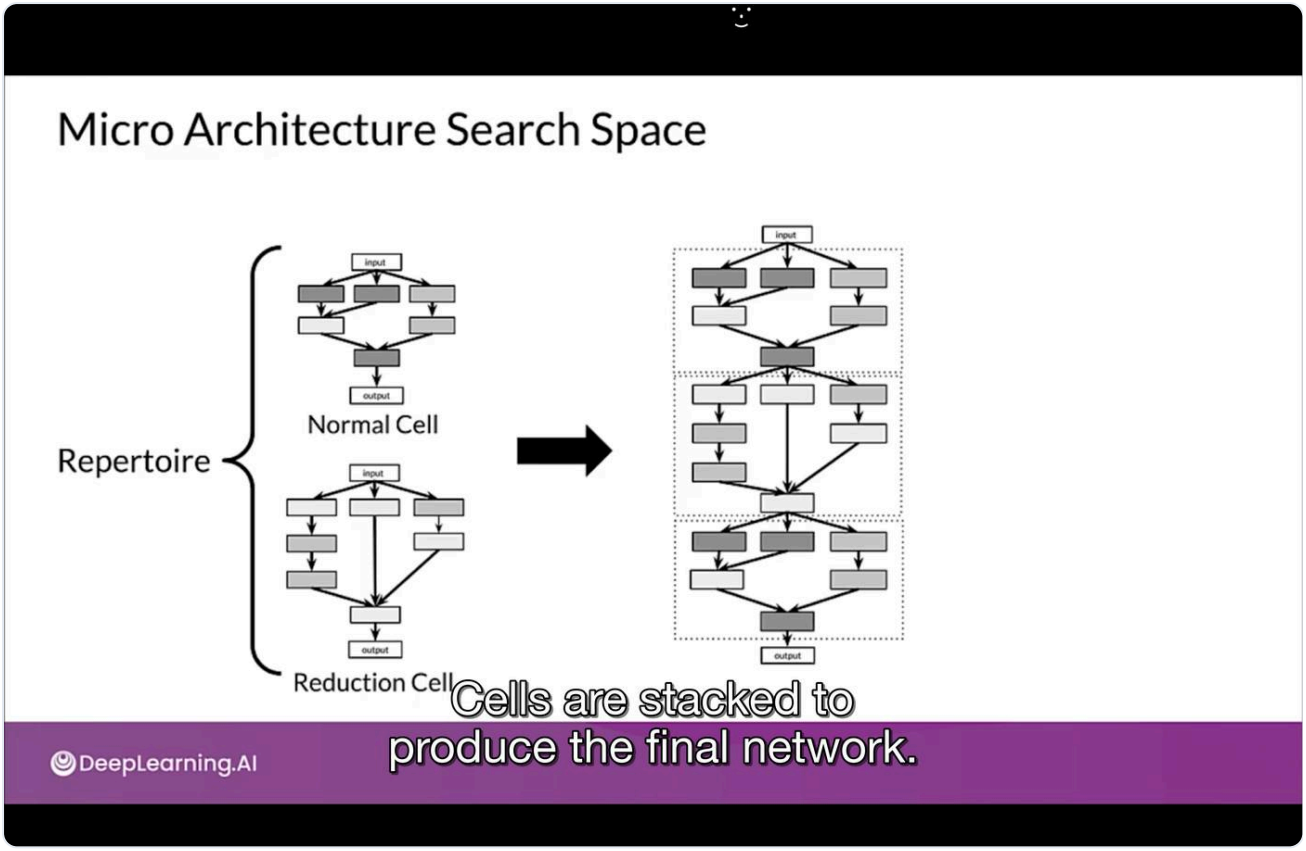

FIGURE 5.6 | Micro Architecture Search Space

Cells are stacked to produce the final network. This approach has been shown to have significant performance advantages compared to a macro approach.

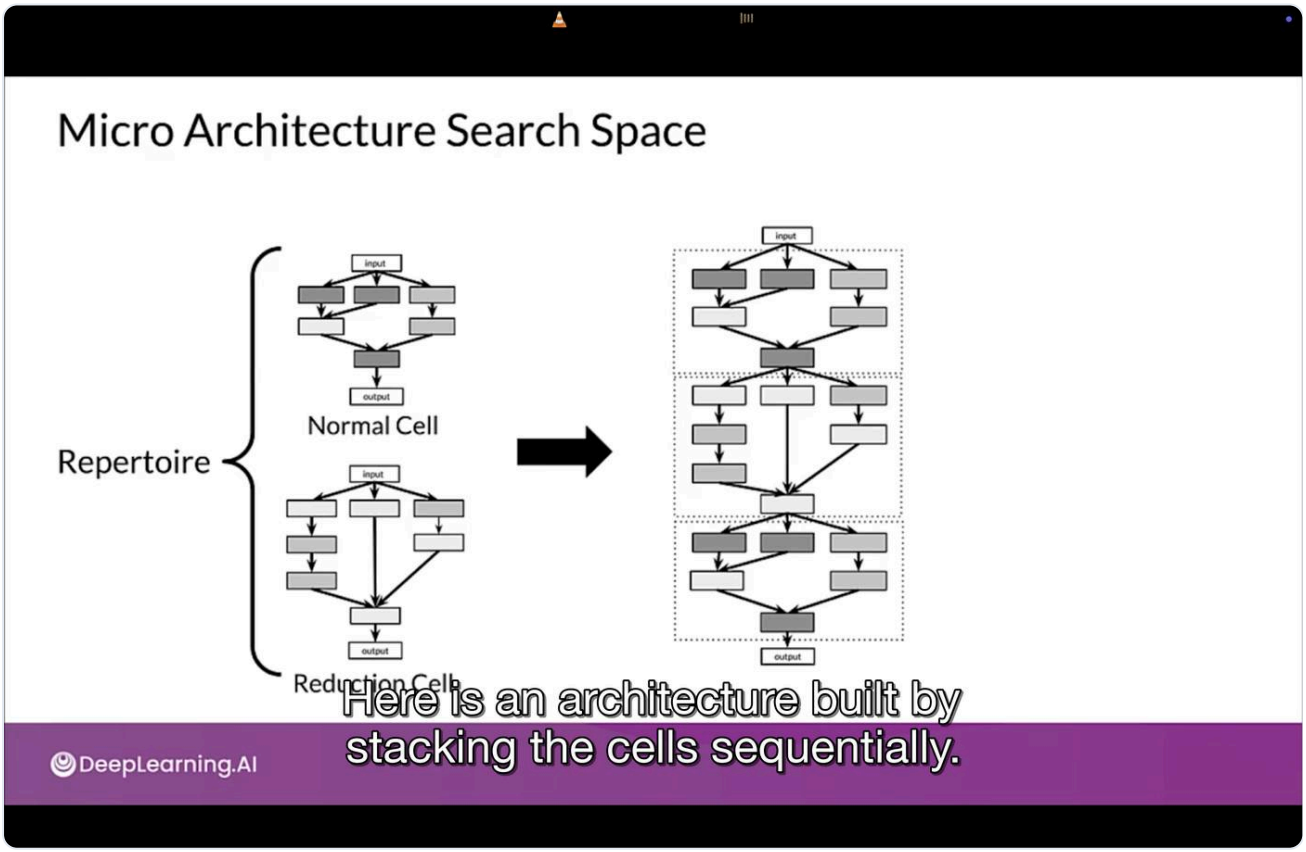

FIGURE 5.7 | Micro Architecture Search Space

Here is an architecture built by stacking the cells sequentially.

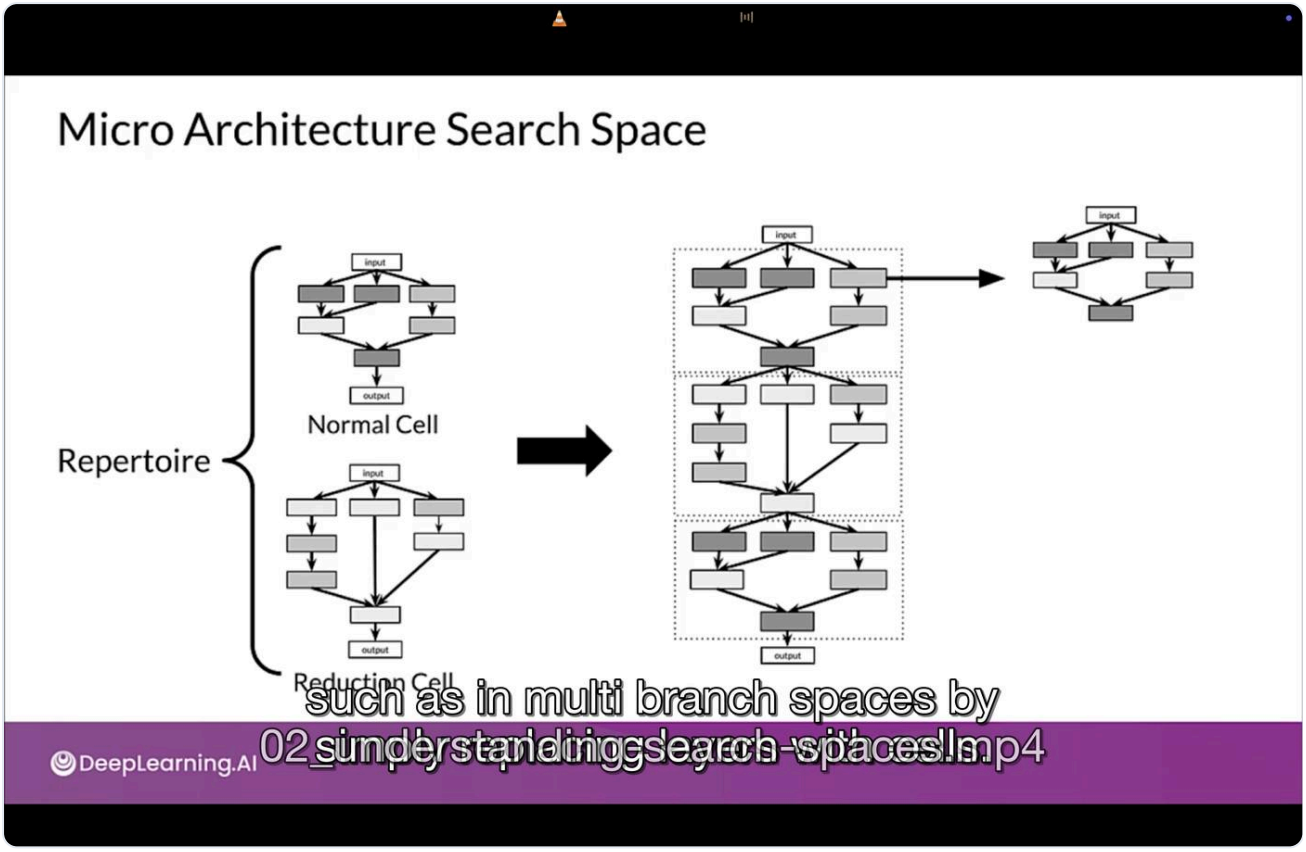

FIGURE 5.8 | Micro Architecture Search Space

Note that cells can also be combined in a more complex manner, such as in multi branch spaces by simply replacing layers with cells.

LECTURE 6

Reinforcement Learning

But how does neural architecture search decide which options in the search space to try next? It needs to have a search strategy, let's discuss that next. Neural architecture search searches through the search base for the architecture that produces the best performance. A variety of different approaches can be used for that search. These include grid search, random search, Bayesian optimization, evolutionary algorithms and reinforcement learning.

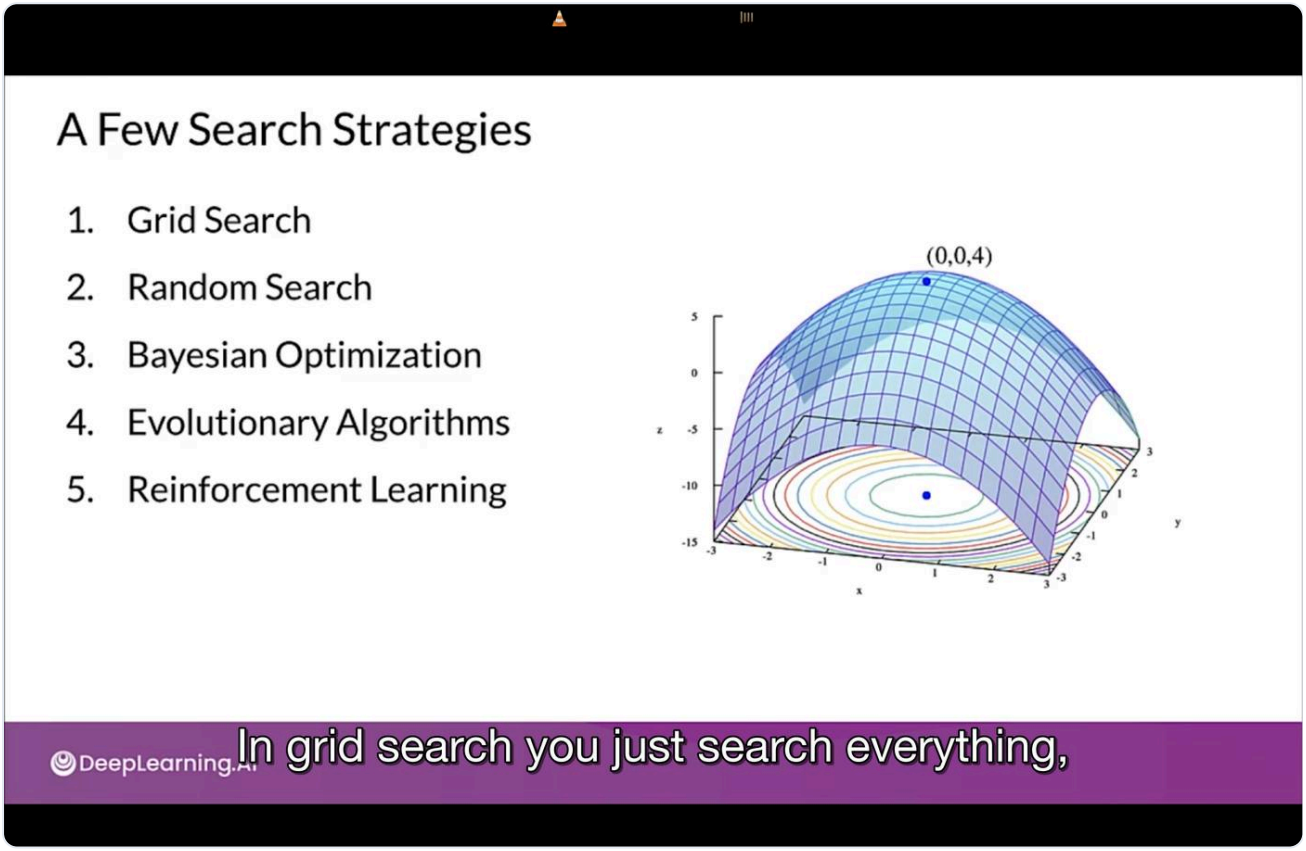

FIGURE 6.1 | Random Search

In grid search you just search everything, that means you cover every option we have in the search base. In random search, you select the next option randomly within the search space.

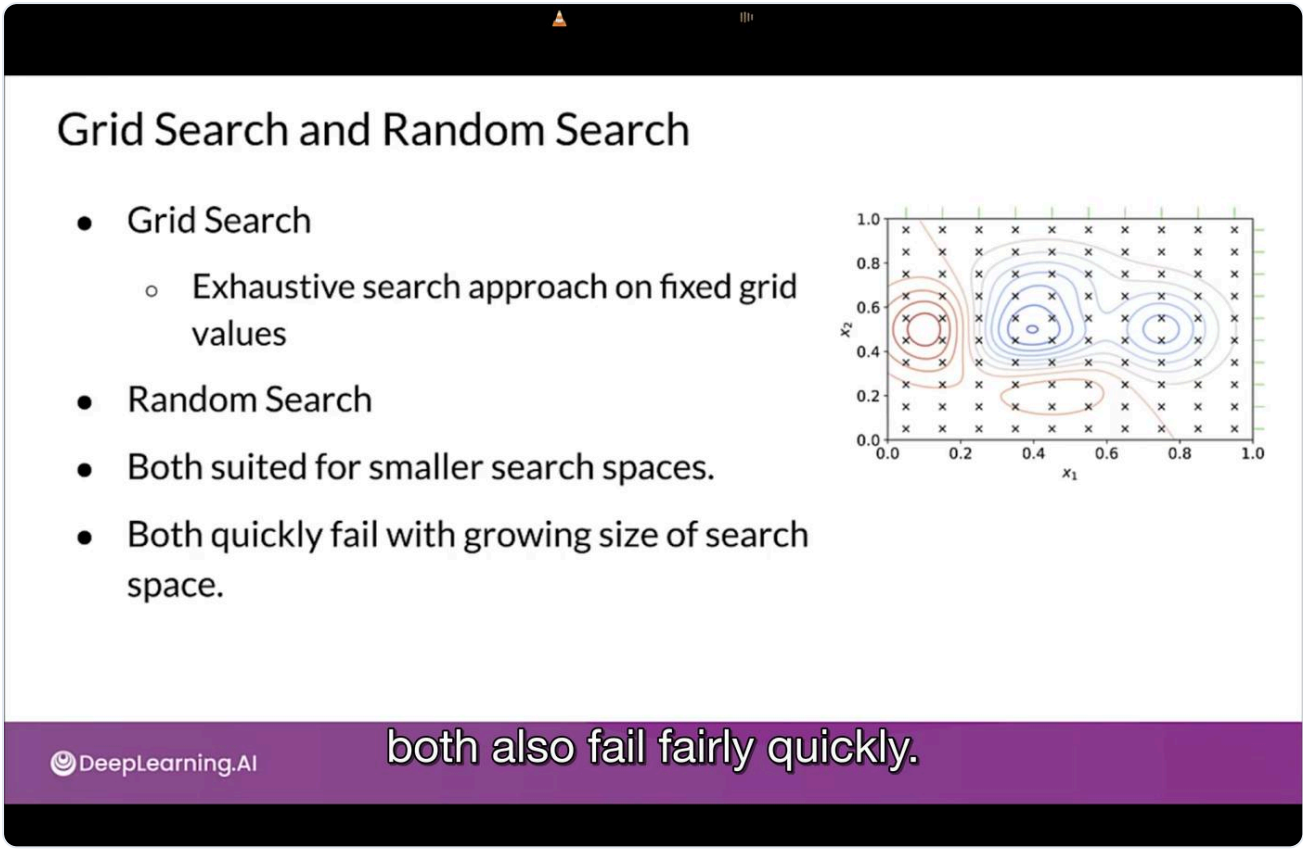

FIGURE 6.2 | Grid Search and Random Search

Both of these work reasonably well in smaller search bases, but both also fail fairly quickly. When the search space grows beyond a certain size, which is all too common. Bayesian optimization is a little more sophisticated. It assumes that a specific probability distribution, which is typically a Gaussian distribution is underlying the performance of model architectures. So you use observations from tested architectures to constrain the probability distribution and guide the selection of the next option.

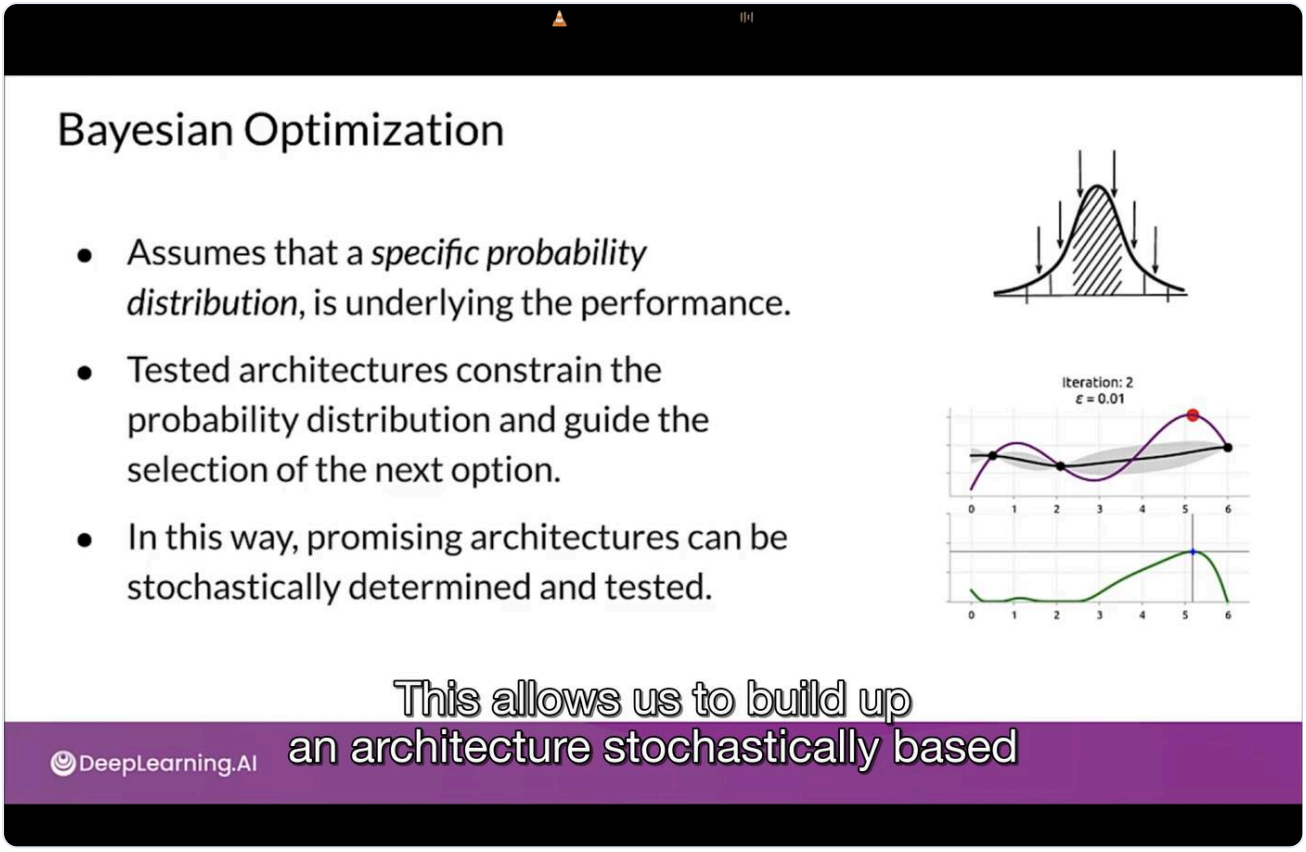

FIGURE 6.3 | Bayesian Optimization L

This allows us to build up an architecture stochastically based on the test results and the constrained distribution. Neural architecture search can also use an evolutionary method to search, here's how it works. First, an initial population of n different model architecture is randomly generated, the performance of each individual. In other words, each architecture is evaluated as defined by the performance estimation strategy, which we'll talk about next.

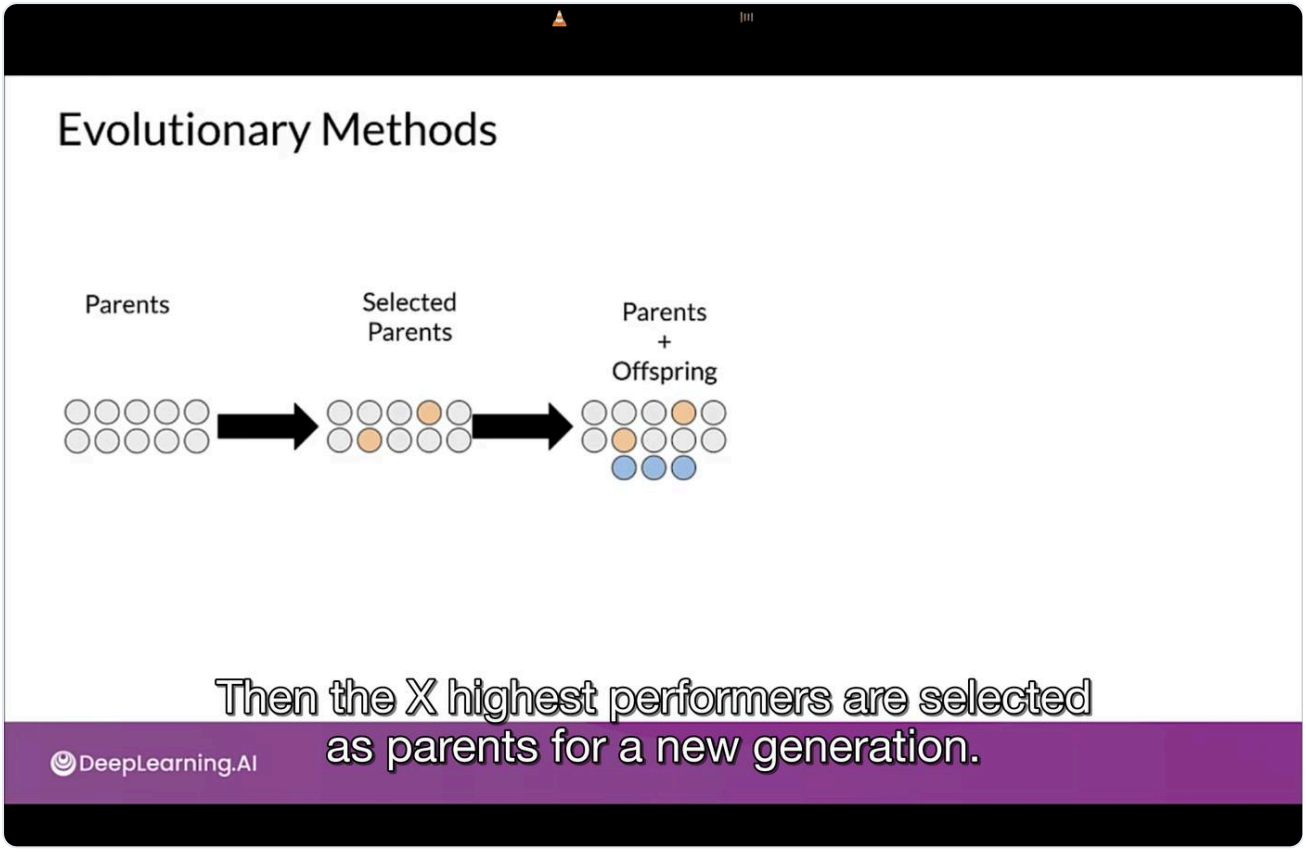

FIGURE 6.4

Then the X highest performers are selected as parents for a new generation. This new generation of architectures might be copies of the respective parents with induced random alterations or mutations. Or they might arise from combinations of the parents, the performance of the offspring is assessed. Again using the performance estimation strategy. The list of possible mutations can include operations like adding or removing a layer. Adding or removing a connection, changing the size of a layer or changing another hyper parameter.

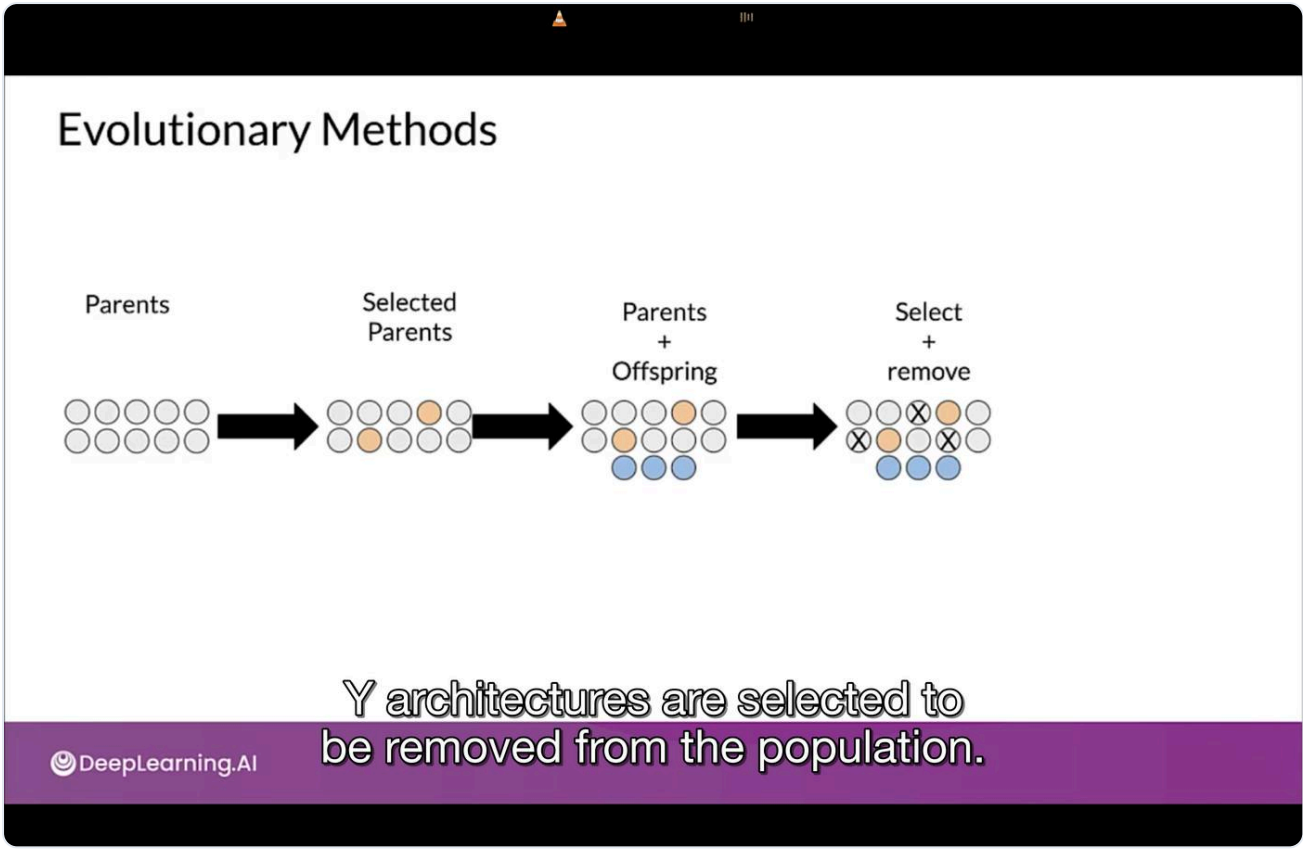

FIGURE 6.5 | Parents Selected Parents Select

Y architectures are selected to be removed from the population. This might be the Y worst performers, the Y oldest individuals in the population. Or a selection of individuals based on a combination of these parameters.

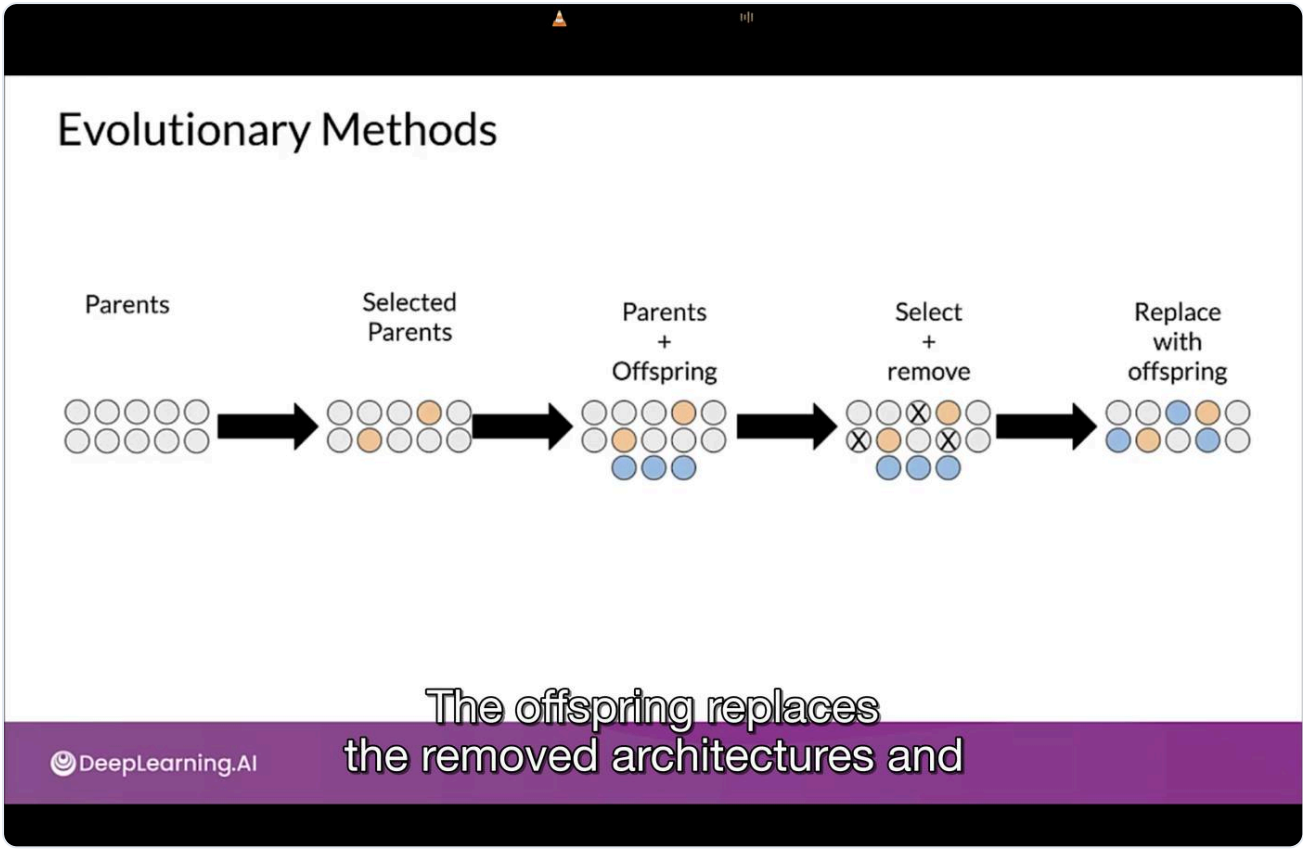

FIGURE 6.6 | Parents Selected Parents Select Replace

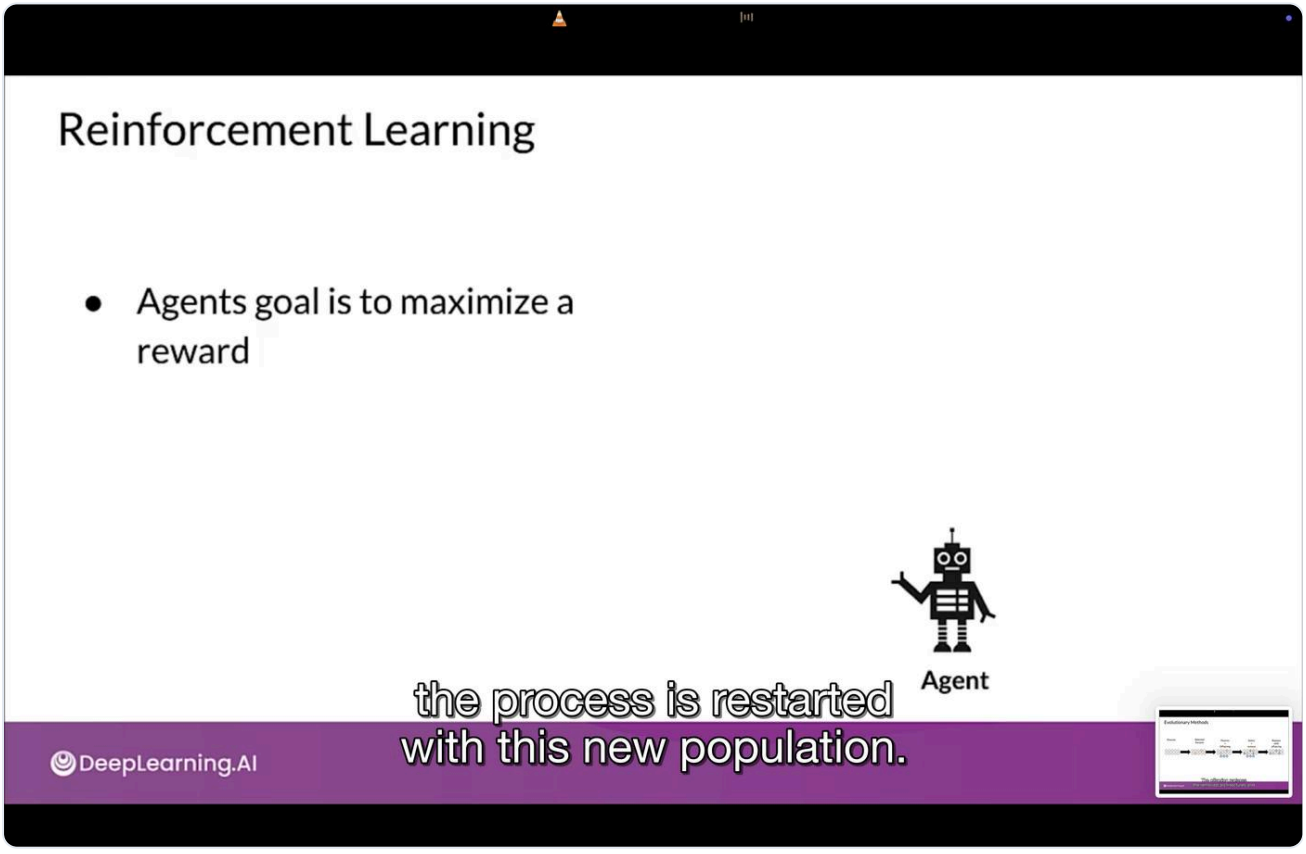

FIGURE 6.7 | Reinforcement Learning

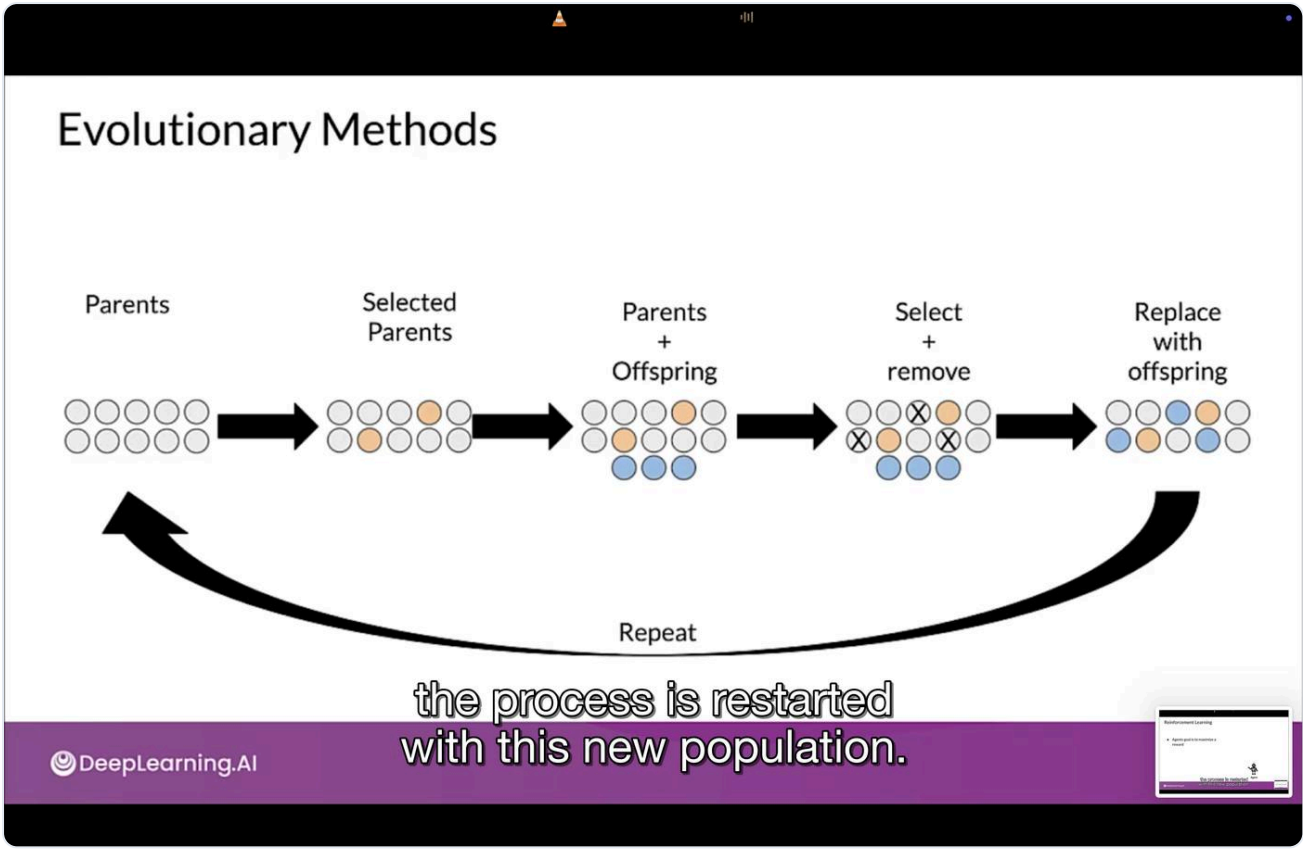

FIGURE 6.8 | Parents Selected

The offspring replaces the removed architectures and the process is restarted with this new population. In reinforcement learning agents take actions in an environment, trying to maximize a reward. After each action, the state of the agent and the environment is updated and a reward is issued based on a performance metric.

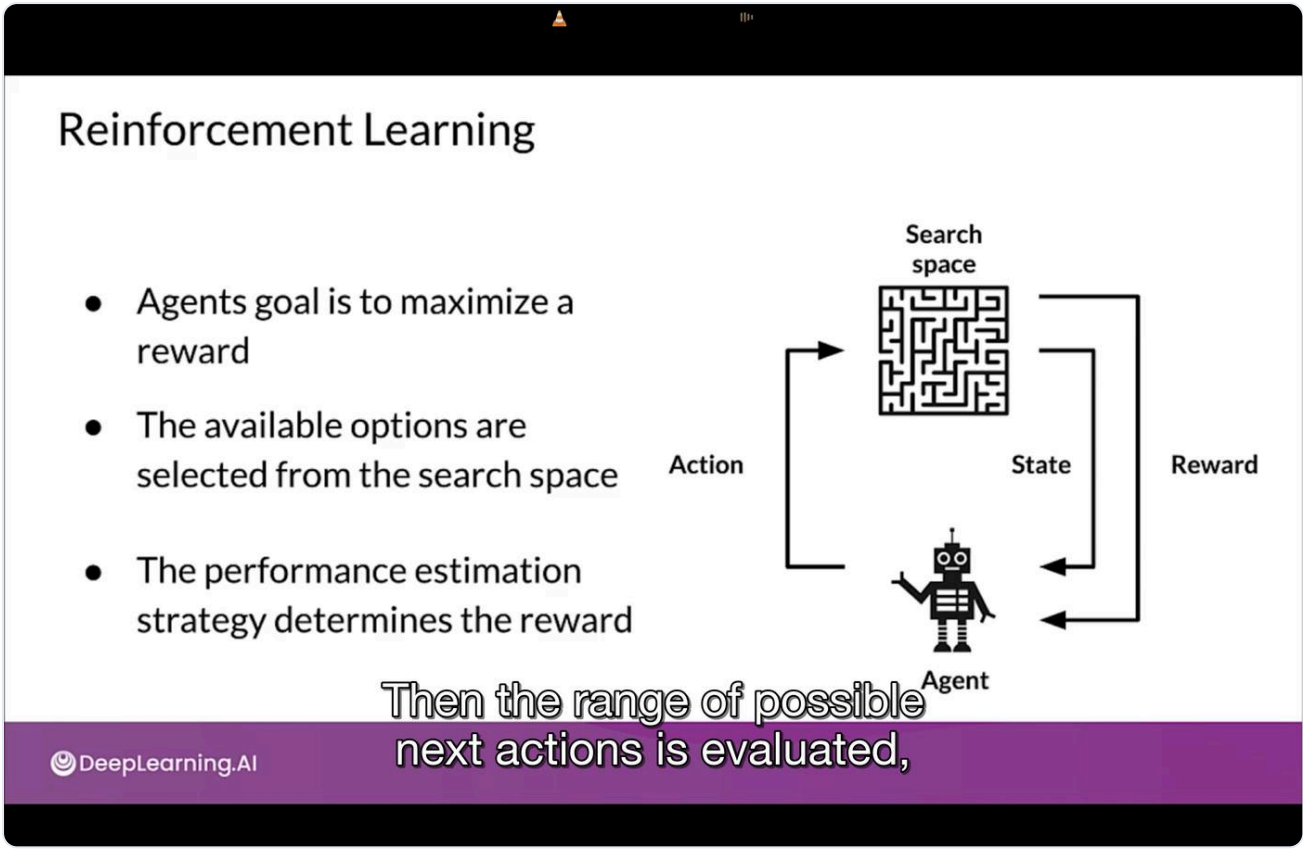

FIGURE 6.9 | Reinforcement Learning

Then the range of possible next actions is evaluated, the environment in this case is our search space. And the reward function is our performance estimation strategy. A neural network can also be specified by a variable length string where the elements of the string specify individual network layers. That enables us to use a recurrent neural network or RNN to generate that string, as we might do for an NLP model. The RNN that generates the string is referred to as the controller, after training the network referred to as the child network on real data.

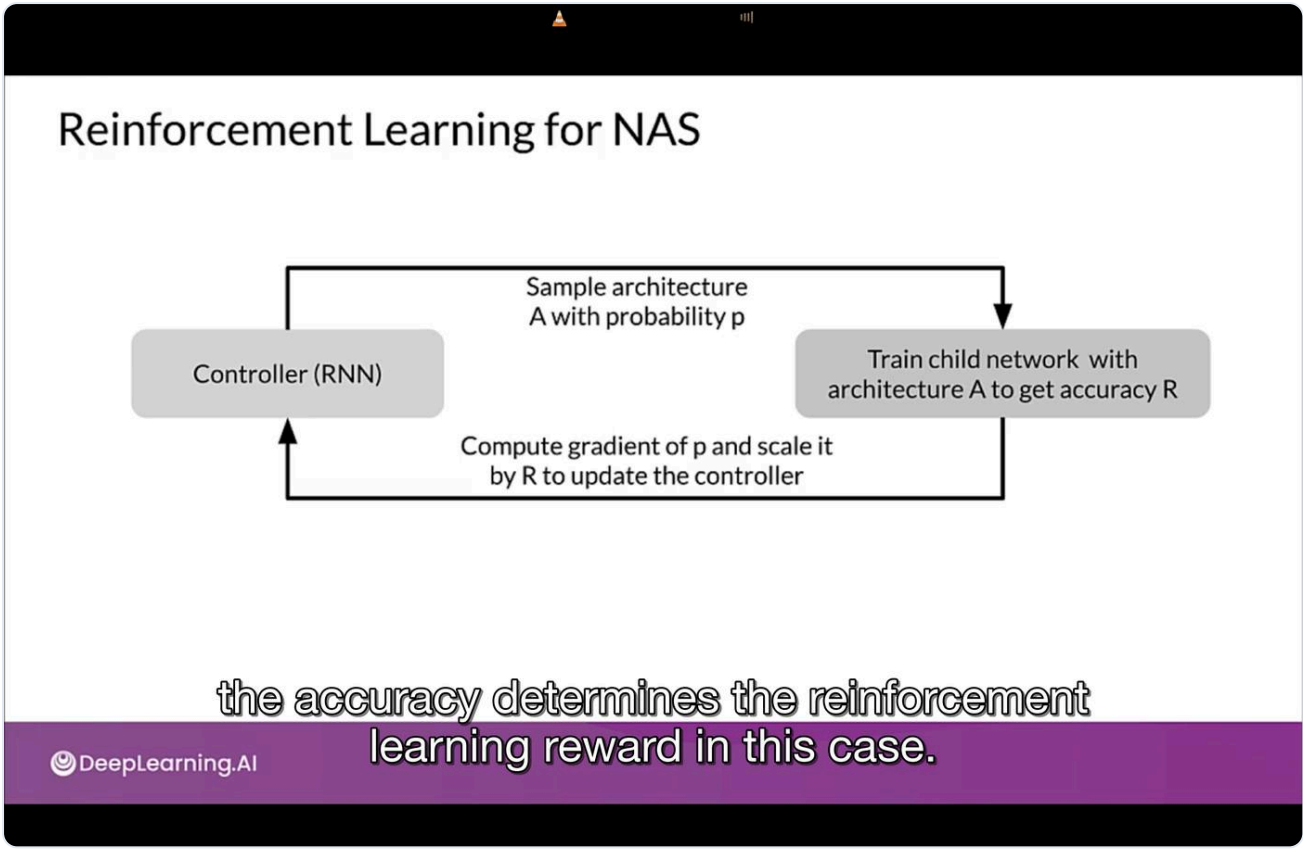

FIGURE 6.10 | Reinforcement Learning for NAS

We can measure the accuracy on the validation set, the accuracy determines the reinforcement learning reward in this case. Based on the accuracy, we can compute the policy gradient to update the controller RNN.

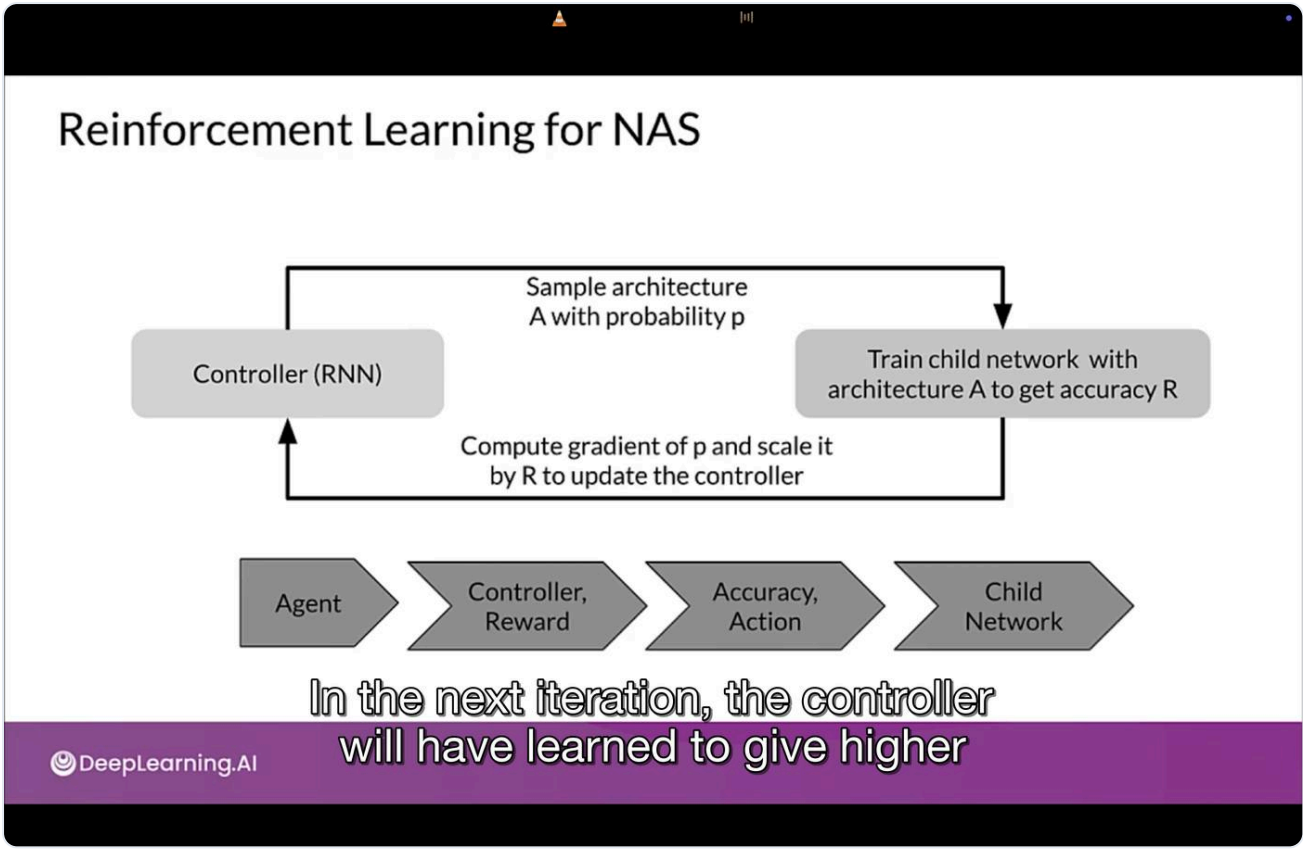

FIGURE 6.11 | Reinforcement Learning for NAS

In the next iteration, the controller will have learned to give higher probabilities to architectures that result in higher accuracies during training. This is how the controller will learn to improve its search over time, for example, on the CIFAR-10 data set. This method starting from scratch, can design a new network architecture that rivals the best human designed architecture in terms of test set accuracy.

LECTURE 7

Weight Inheritance/Network Morphisms

Neural architecture search relies on being able to measure the accuracy or effectiveness of the different architectures that it tries. This requires a performance estimation strategy. Let's talk about that now. The search strategies in neural architecture search need to estimate the performance of generated architectures so that they can generate architectures that perform better. Let's discuss some of the performance estimation strategies which are used in neural architecture search. The simplest approach is to measure the validation accuracy of each architecture that is generated like we saw with the reinforcement learning approach. This becomes computational heavy, especially for large search spaces and complex networks.

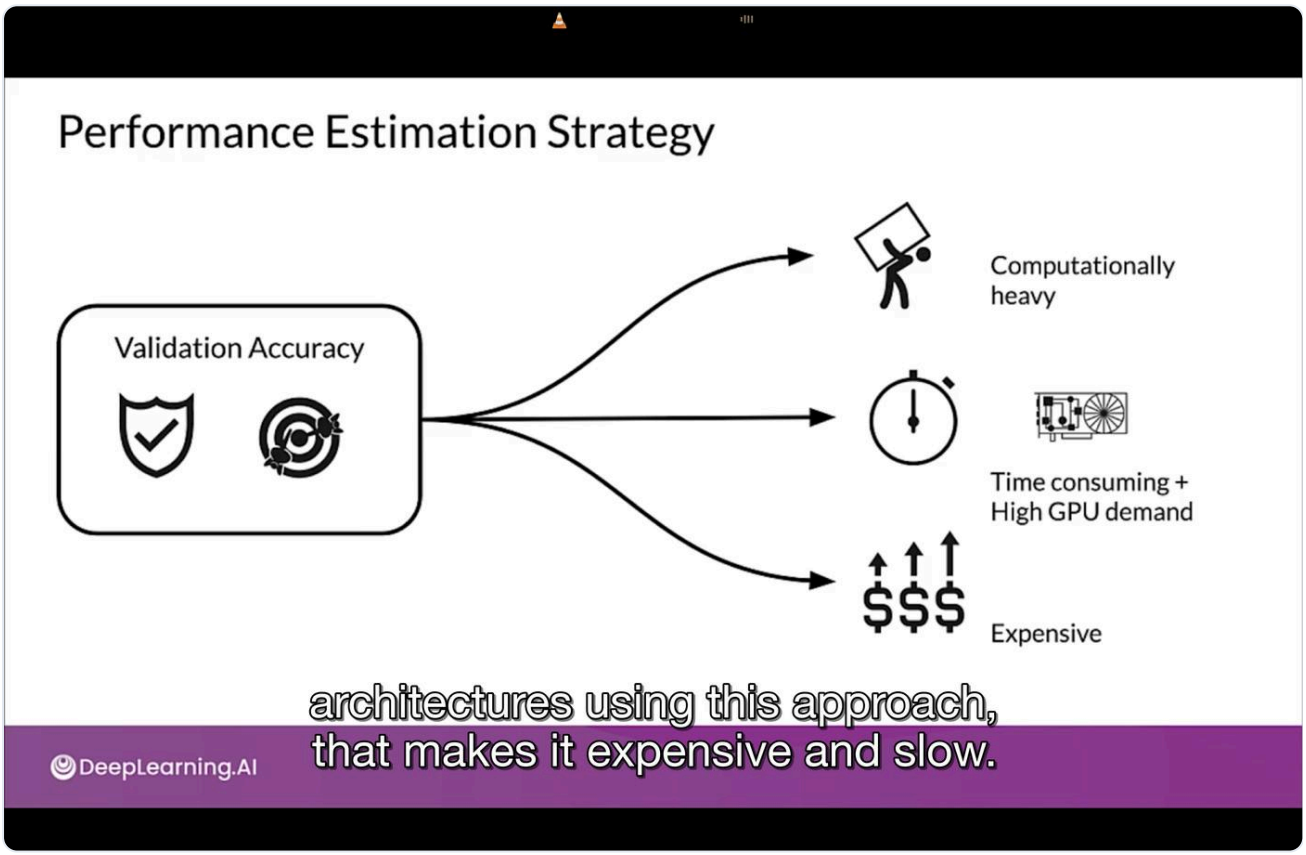

FIGURE 7.1 | Performance Estimation Strategy

And as a result, it can take several GPU days to find the best architectures using this approach, that makes it expensive and slow. It also makes neural architecture search impractical for many use cases. Is there a way to reduce performance cost estimation? Several strategies have been proposed, including lower fidelity estimates, learning curve extrapolation, and weight inheritance or network morphism.

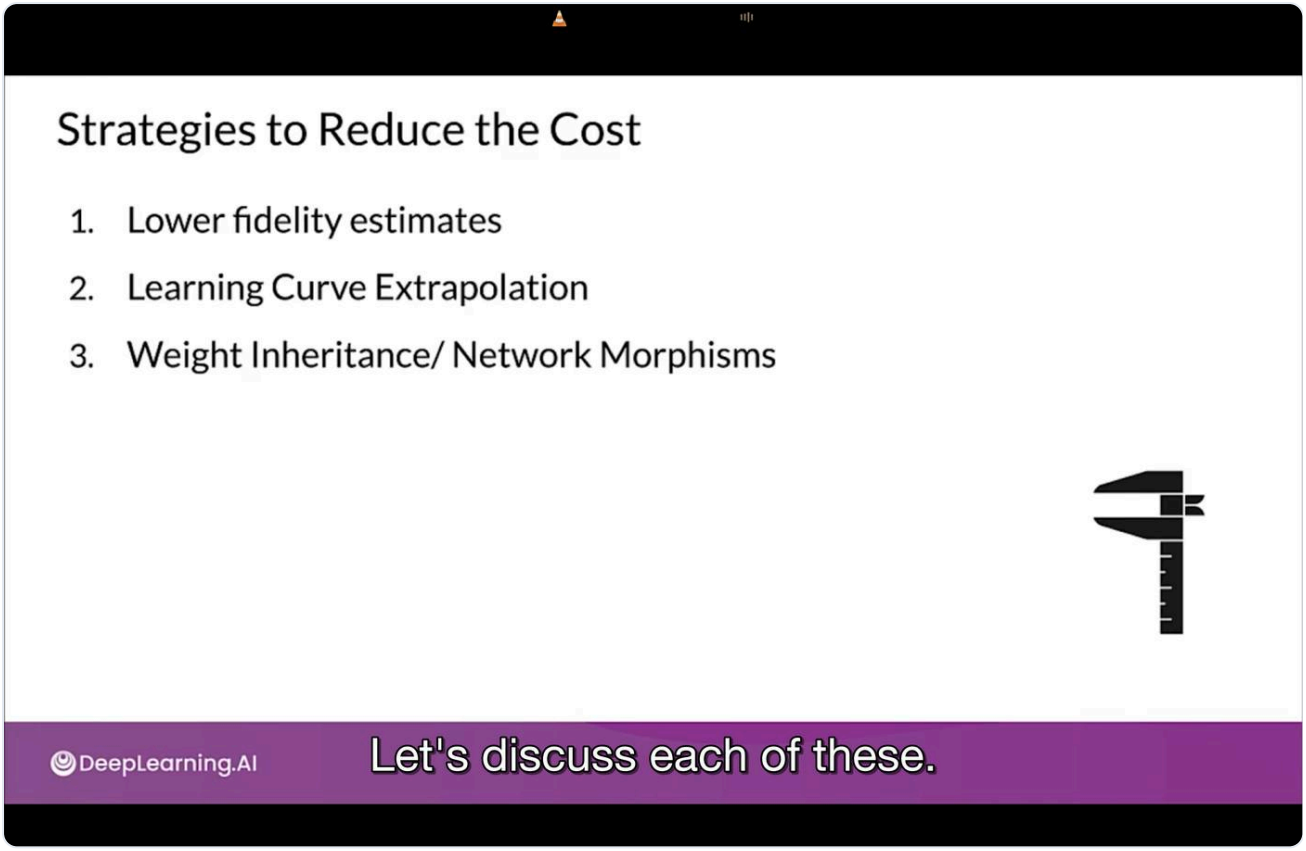

FIGURE 7.2 | Strategies to Reduce the Cost

Let's discuss each of these. Lower fidelity or lower precision estimates try to reduce the training time by reframing the problem to make it easier to solve by training on a subset of data or using lower resolution images, for example, or using fewer filters per layer and fewer cells. It greatly reduces the computational cost, but ends up underestimating performance. That's okay if you can make sure that the relative ranking of the architectures does not change due to their lower fidelity estimates.

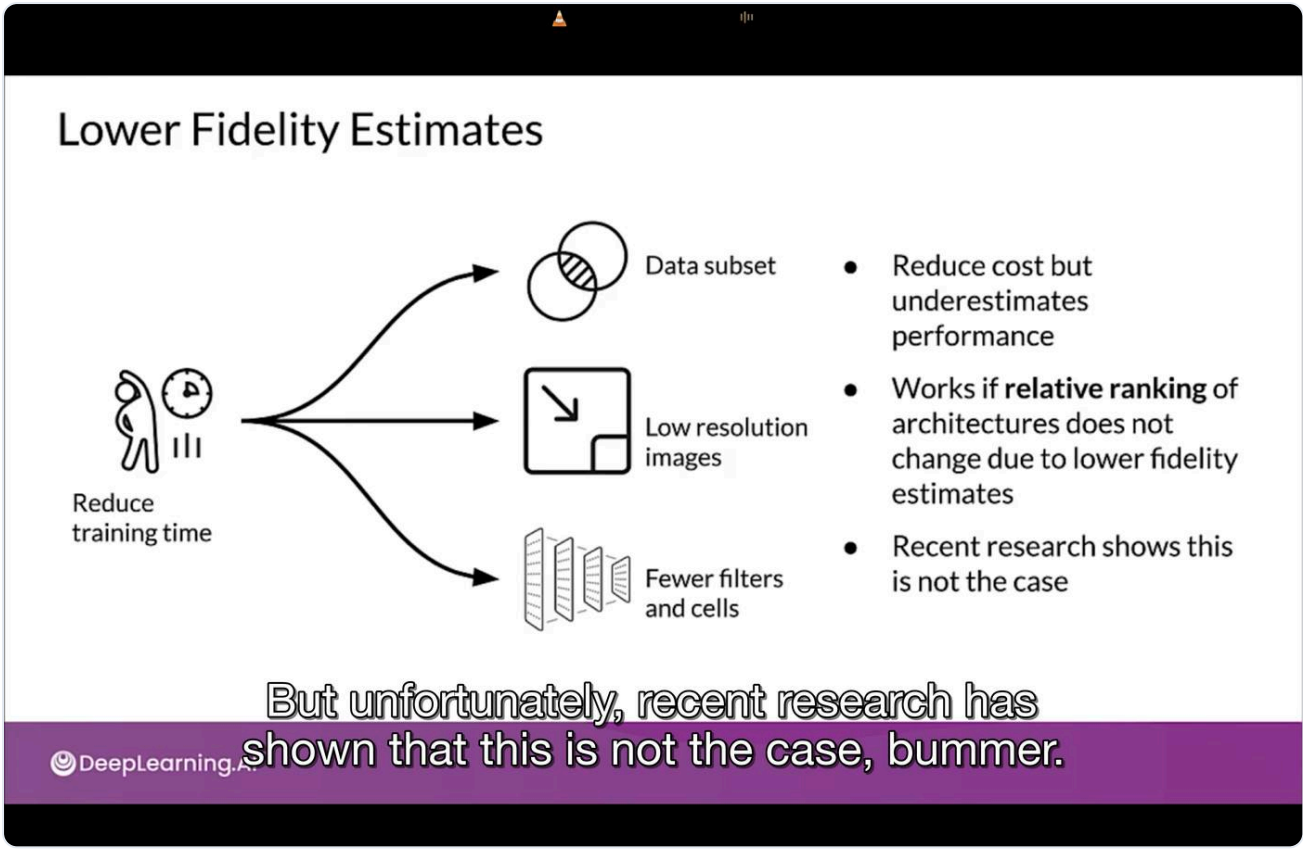

FIGURE 7.3 | Lower Fidelity Estimates

But unfortunately, recent research has shown that this is not the case, bummer. What else can we try? Learning Curve Extrapolation is based on the assumption that you have mechanisms to predict the learning curve reliably, and so extrapolation is a sensitive and valid choice. On the right there are learning curves for different architectures. Based on a few iterations and available knowledge, the method extrapolates the initial learning curves and terminates all architectures that perform poorly.

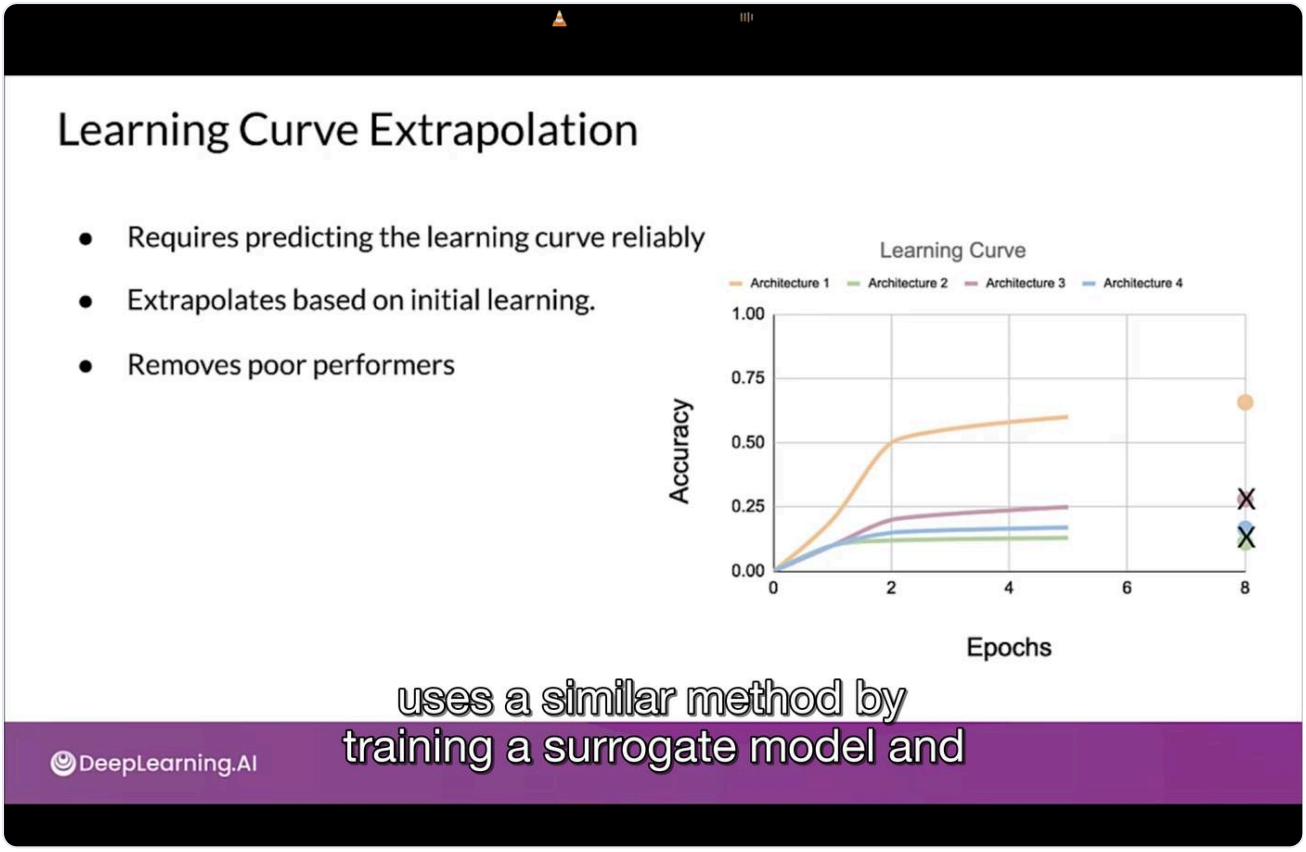

FIGURE 7.4 | Learning Curve Extrapolation

The progressive neural architecture search algorithm, which is one of the approaches for neural architecture search, uses a similar method by training a surrogate model and using it to predict the performance using architectural properties. Another method for speeding up architecture search initializes the weights of novel architectures based on the weights of other architectures that have been trained before, similar to the way that transfer learning works. One way of achieving this is referred to as network morphism. Network morphism modifies the architecture without changing the underlying function.

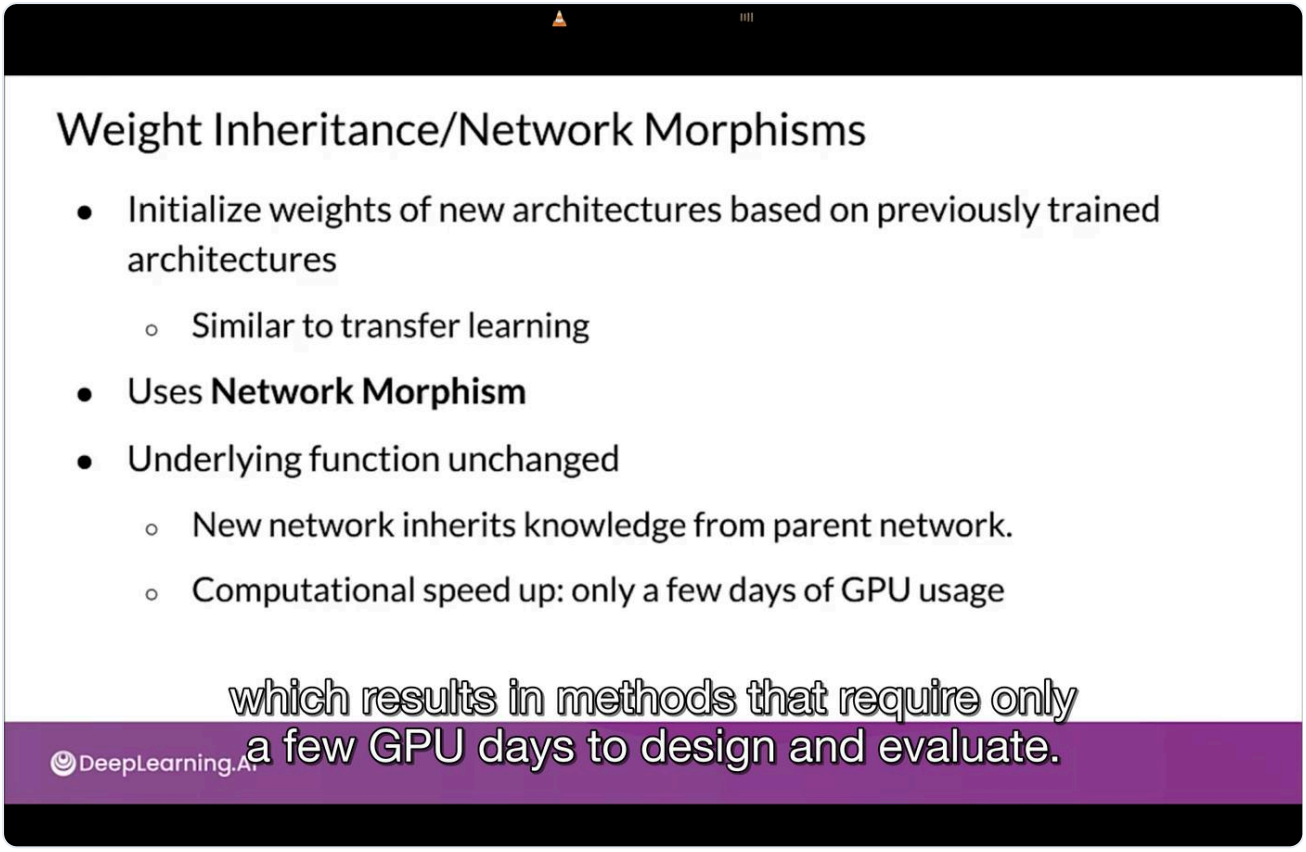

FIGURE 7.5 | Weight Inheritance/Network Morphisms

This is advantageous because the network inherits knowledge from the parent network, which results in methods that require only a few GPU days to design and evaluate.

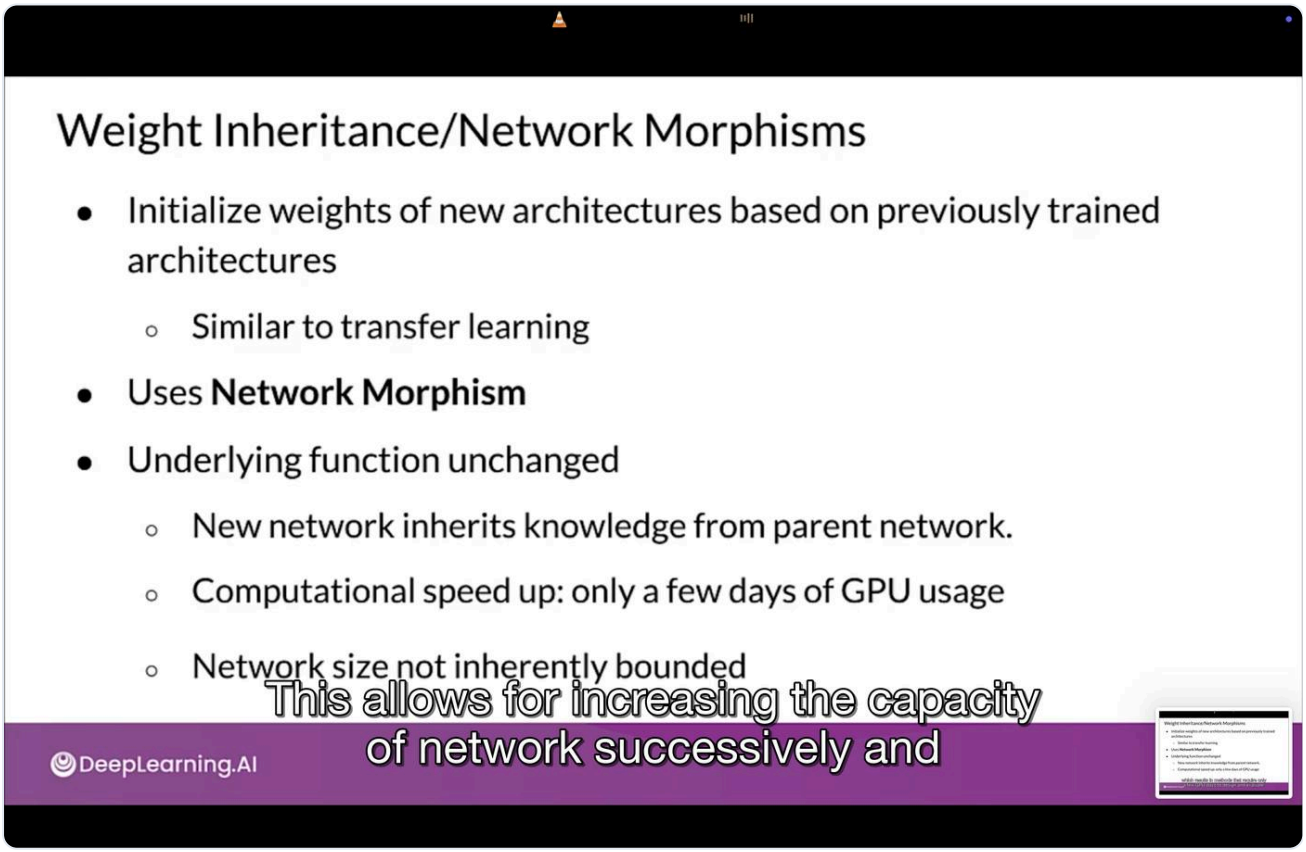

FIGURE 7.6 | Weight Inheritance/Network Morphisms

This allows for increasing the capacity of network successively and retaining high performance without requiring training from scratch. One advantage of this approach is that it allows for search bases that don't have an inherent upper bound on the architecture's size.

LECTURE 8

Amazon SageMaker Autopilot

One of the easiest ways to use AutoML is by using a Cloud service. Let's talk about some of those now.

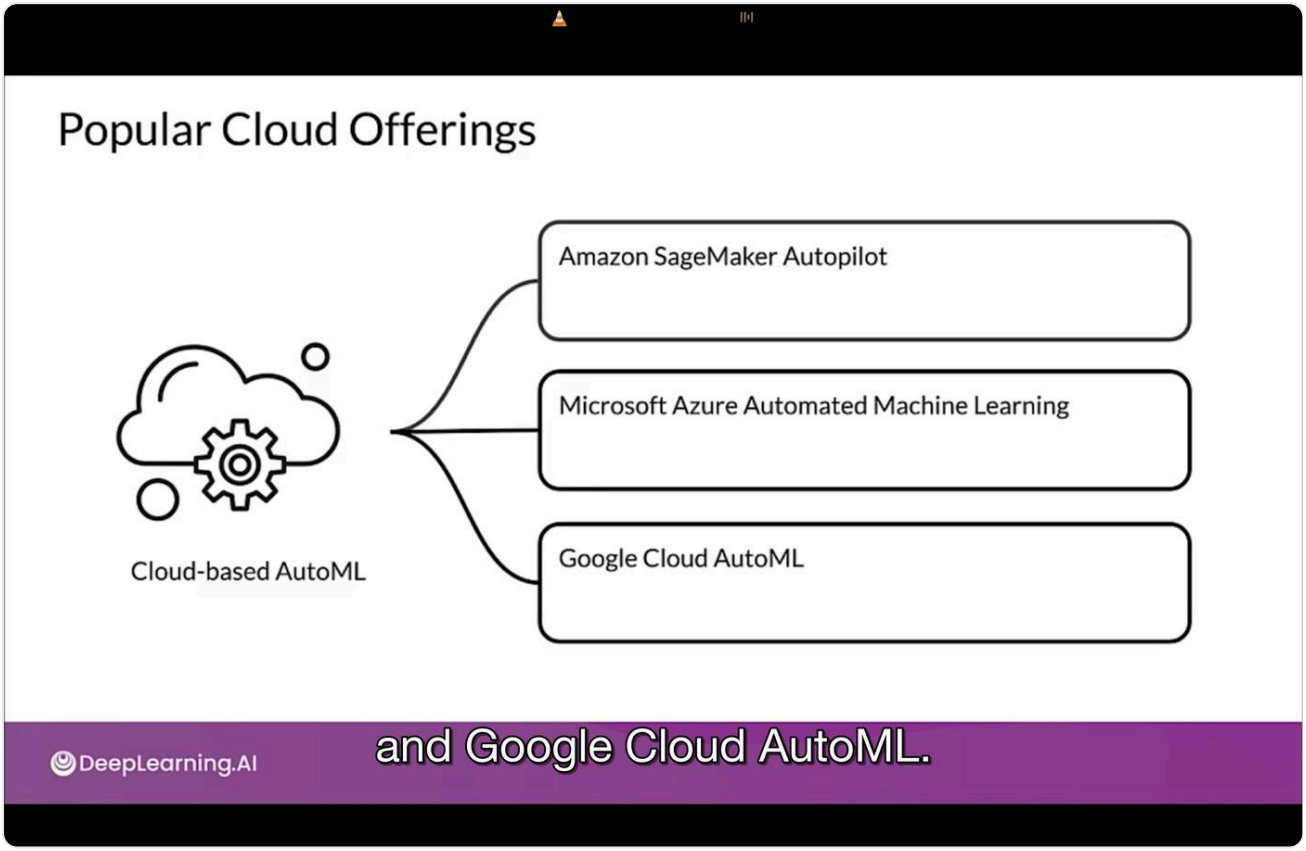

FIGURE 8.1 | Popular Cloud Offerings

We'll review three popular offerings; Amazon SageMaker Autopilot, Microsoft Azure Automated Machine Learning, and Google Cloud AutoML. In the exercises, you'll train a model using Google Cloud AutoML. Let's begin with Amazon SageMaker Autopilot. Amazon SageMaker Autopilot automatically trains and tunes the best machine learning models for classification or regression based on your data, while allowing you to maintain full control and visibility. Starting with your raw data, you select a label or target. Autopilot then searches for candidate models for you to review and choose from. All of these steps are documented on executable notebooks that give you full control and reproducibility of the process.

FIGURE 8.2 | Amazon SageMaker Autopilot

This includes a leader board of model candidates to help you select the best model for your needs.

FIGURE 8.3 | Amazon SageMaker Autopilot

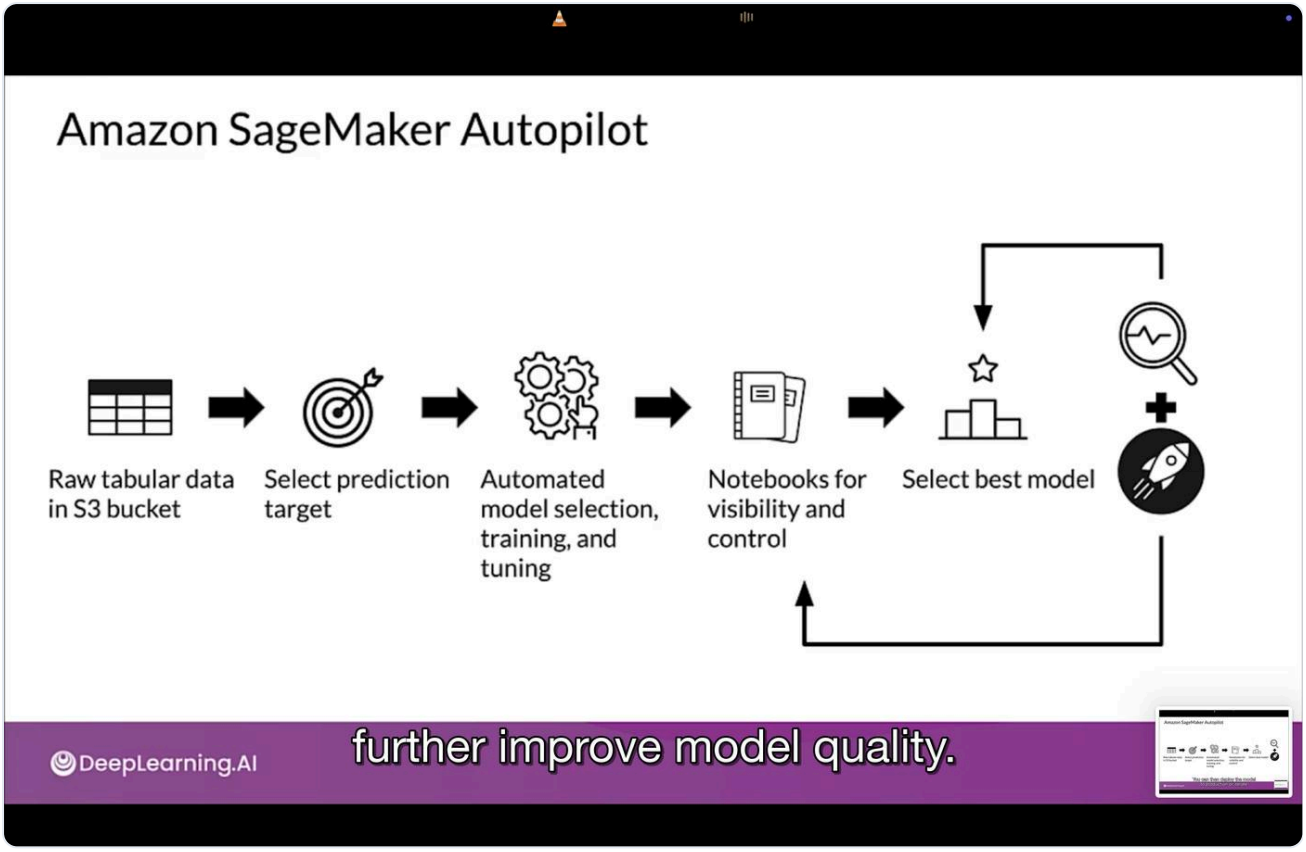

FIGURE 8.4 | Amazon SageMaker Autopilot

You can then deploy the model to production or iterate on the recommended solutions to further improve model quality. Autopilot is optimized for quick iteration. It generates high-quality models quickly. After the initial set of iterations, autopilot creates a leaderboard of models ranked by performance. You can see which features in your dataset were selected by each model. You can then deploy a model to production.

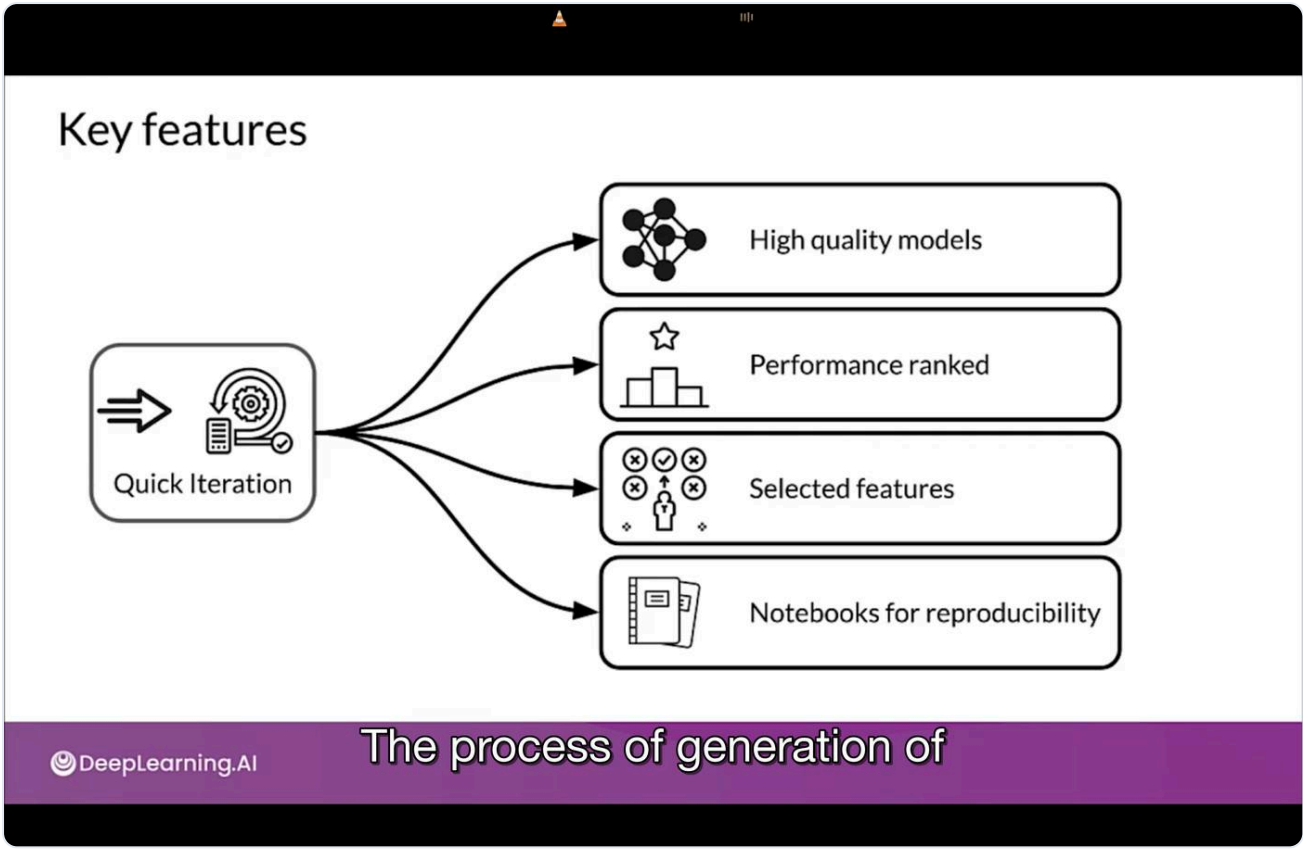

FIGURE 8.5 | Quick Iteration Selected Features

The process of generation of the models is completely transparent. Autopilot allows you to create a SageMaker notebook from any model it created. You can then check the Notebook to dive into details of the models implementations. If need be, you can refine the model and recreate it from the Notebook at any point in time. Amazon suggests that autopilot can be applied for a number of different use cases.

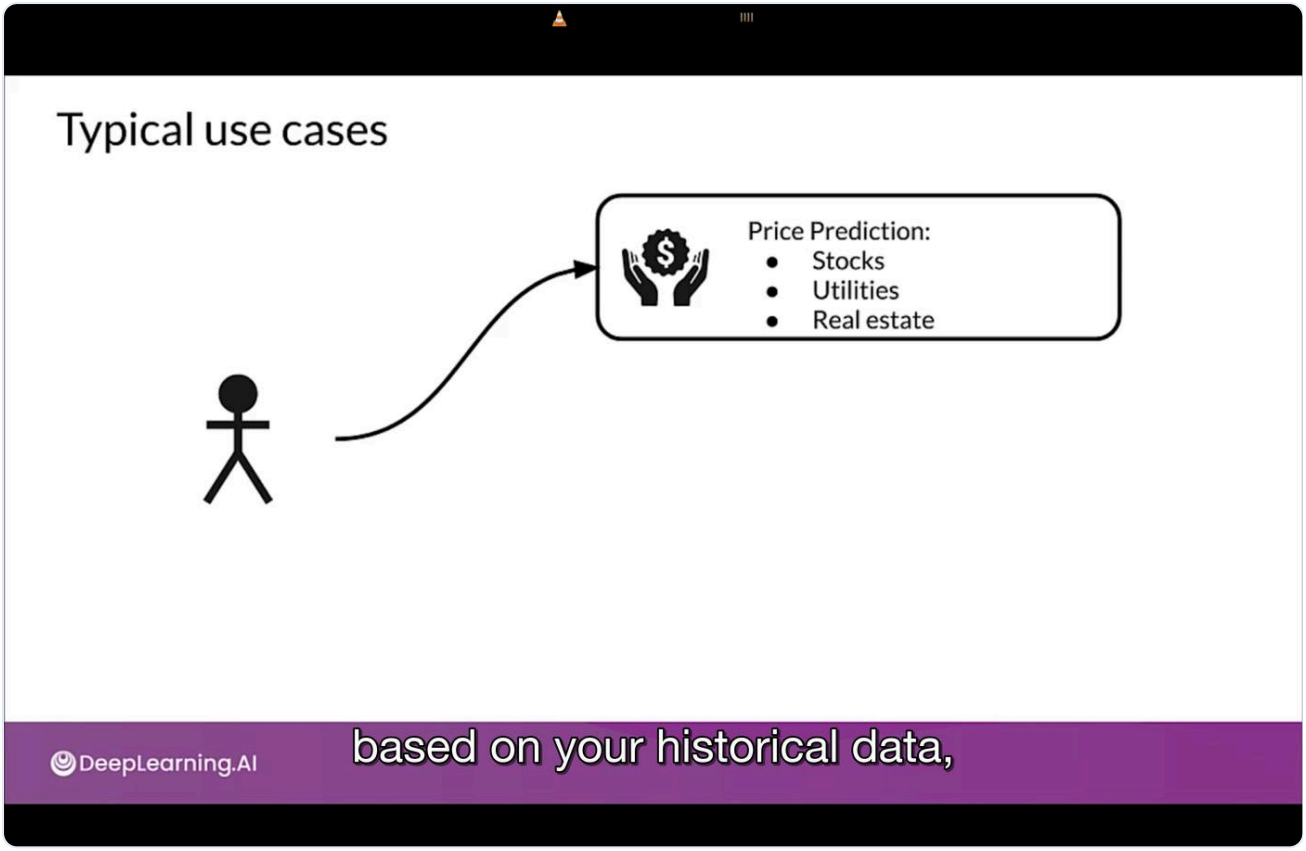

FIGURE 8.6 | Price Prediction:

For example, it can be used to predict future prices to help you make sound investment decisions based on your historical data, such as demand, seasonal trends, and the price of other commodities. Price predictions are particularly useful in the financial services sector to predict the price of stocks or real estate, to predict real estate prices, or energy and utilities to predict the prices of natural resources.

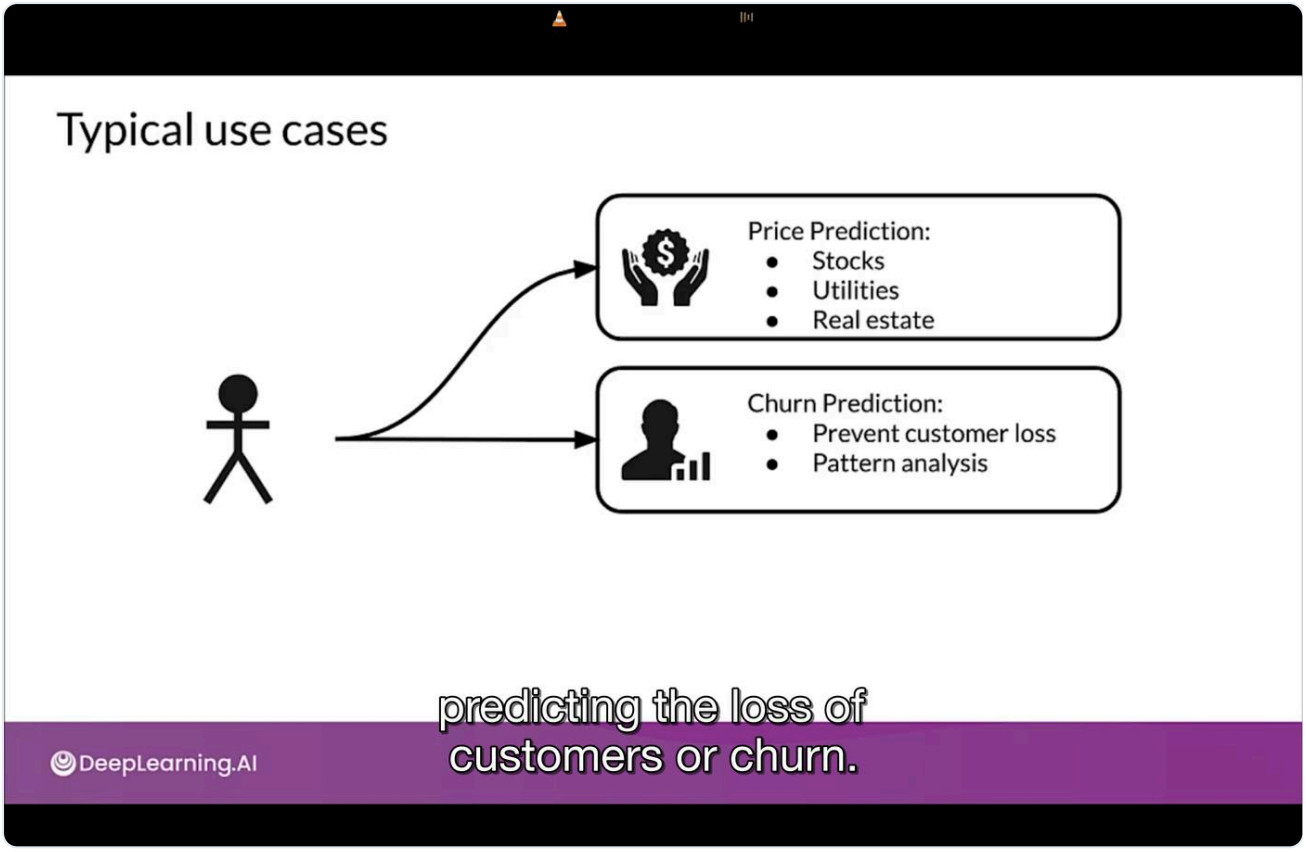

FIGURE 8.7 | Price Prediction:

Churn prediction can be useful in predicting the loss of customers or churn. Companies are always looking for ways to eliminate churn. Churn prediction works by learning patterns in your existing data and identifying patterns in the new datasets so that the model can predict which customers are most likely to churn.

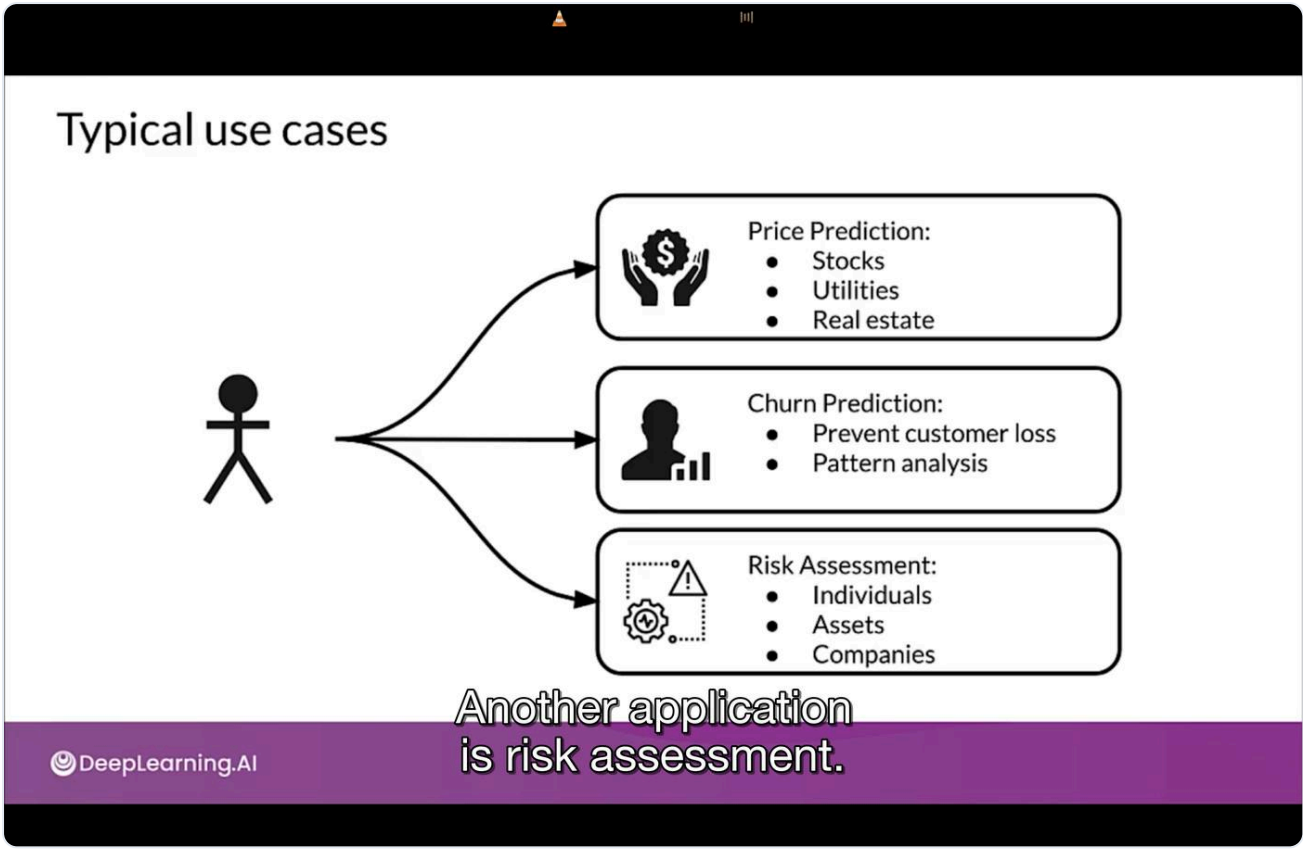

FIGURE 8.8 | Price Prediction:

Another application is risk assessment. Risk assessment requires identifying and analyzing potential events that may negatively impact individuals, assets, and your company. Risk assessment models are trained using your existing datasets so that you can optimize the models predictions for your business. Next, let's discuss Microsoft Azure Automated Machine Learning. Microsoft Azure Automated Machine Learning aims to empower professional and non-professional data scientists to build machine learning models rapidly. It automates time-consuming and iterative task of model development, so basically it does AutoML.

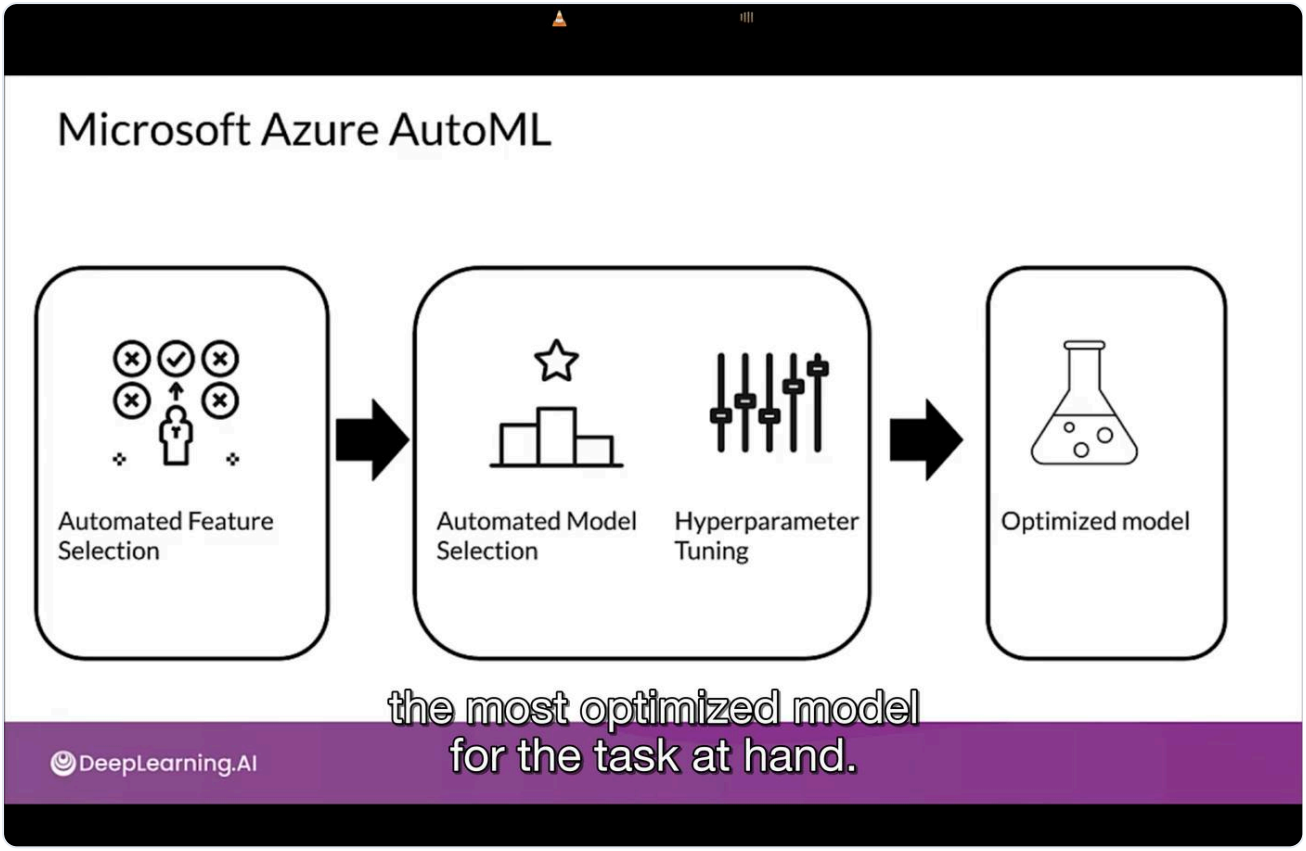

FIGURE 8.9 | Microsoft Azure AutoML

It starts with automatic feature selection, followed by model selection and hyperparameter tuning on the selected model to generate the most optimized model for the task at hand. You can either create your models using a no code UI or using code first notebooks. You can quickly customize your models, apply control settings to iterations, thresholds, validations, blocked algorithms, and other experimental criteria. It also provides tools to fully automate the feature engineering process. You can also easily visualize and profile your data to spot trends, as well as discover common errors and inconsistencies in your data.

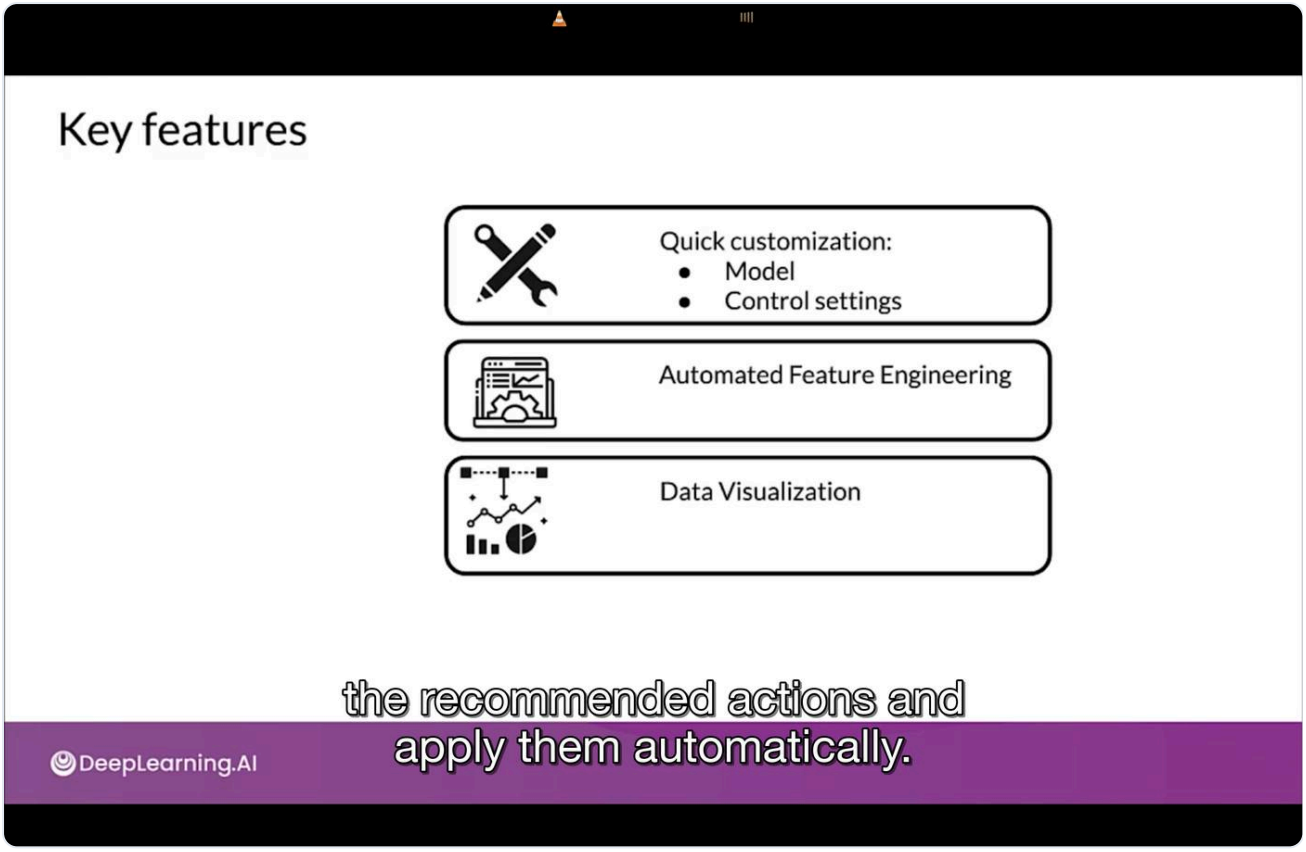

FIGURE 8.10

This helps you better understand the recommended actions and apply them automatically.

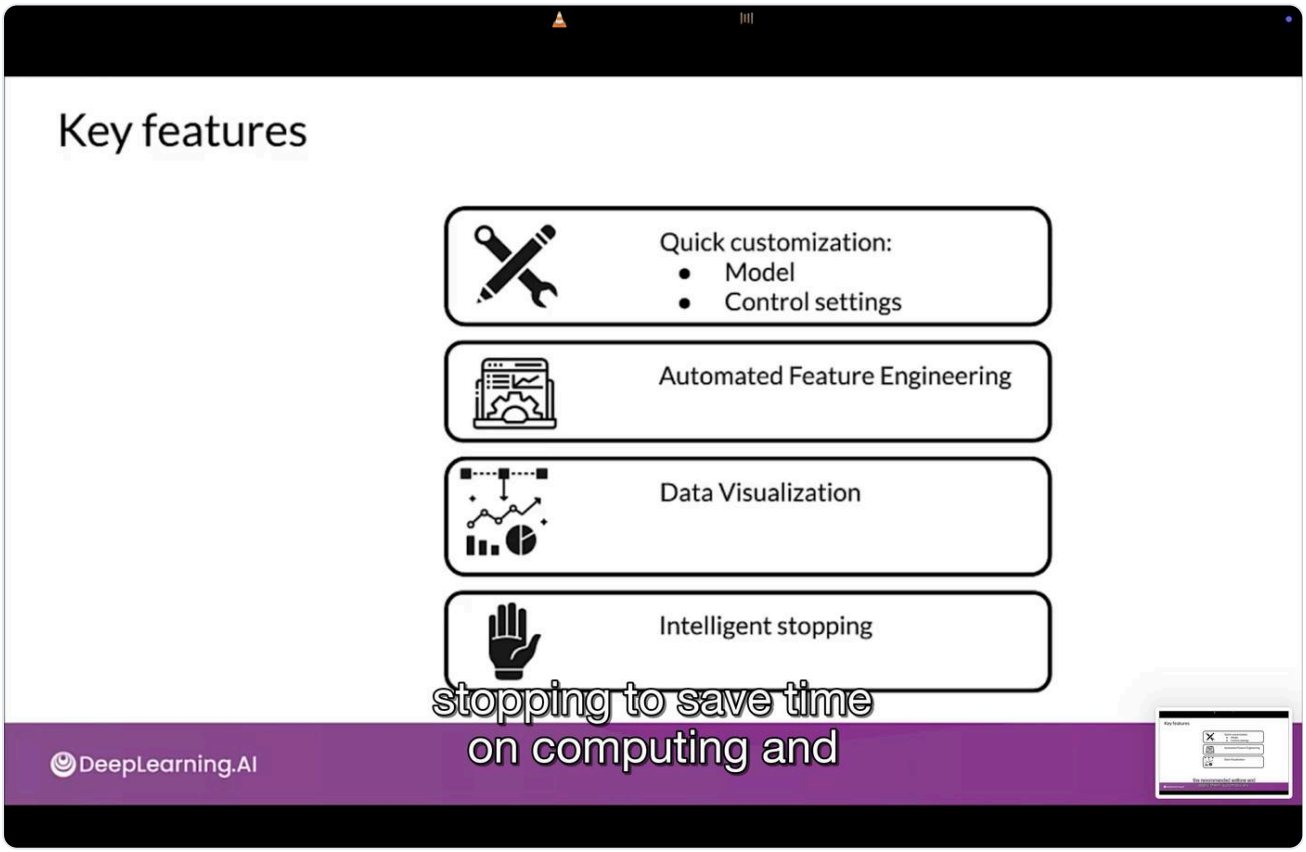

FIGURE 8.11

It also provides intelligent stopping to save time on computing and prioritize the primary metric and subsampling to streamline experiment runs and speed results. It also has built-in support for experiment runs summaries, and detailed metrics visualizations to help you understand your models and compare model performance. Model interpretability helps evaluate model fit for raw and engineered features and provides insights into feature importance.

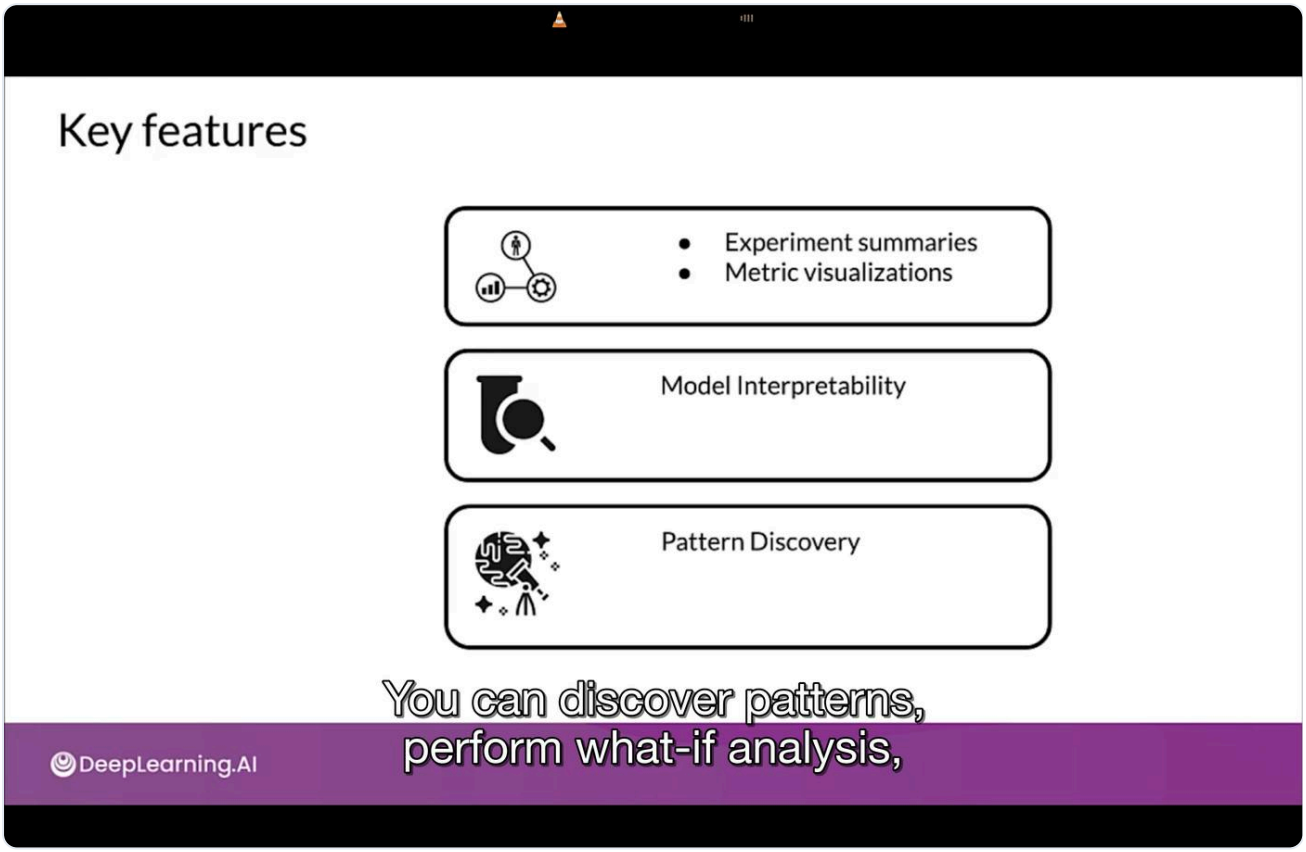

FIGURE 8.12 | Model Interpretability

You can discover patterns, perform what-if analysis, and develop deeper understanding of models to support transparency and trust in your business. Now, let's discuss Google Cloud AutoML, which you'll be using in the upcoming exercise. Google Cloud AutoML is a suite of machine learning products that enables developers with limited machine-learning expertise to train high-quality models specific to their business needs. It relies on Google, state of the art transfer learning and neural architecture search technologies. Cloud AutoML leverages more than 10 years of Google research to help your machine-learning models achieve faster performance and more accurate predictions.

You can use Cloud AutoML, simple graphical user interface to train, evaluate, improve, and deploy models based on your data.

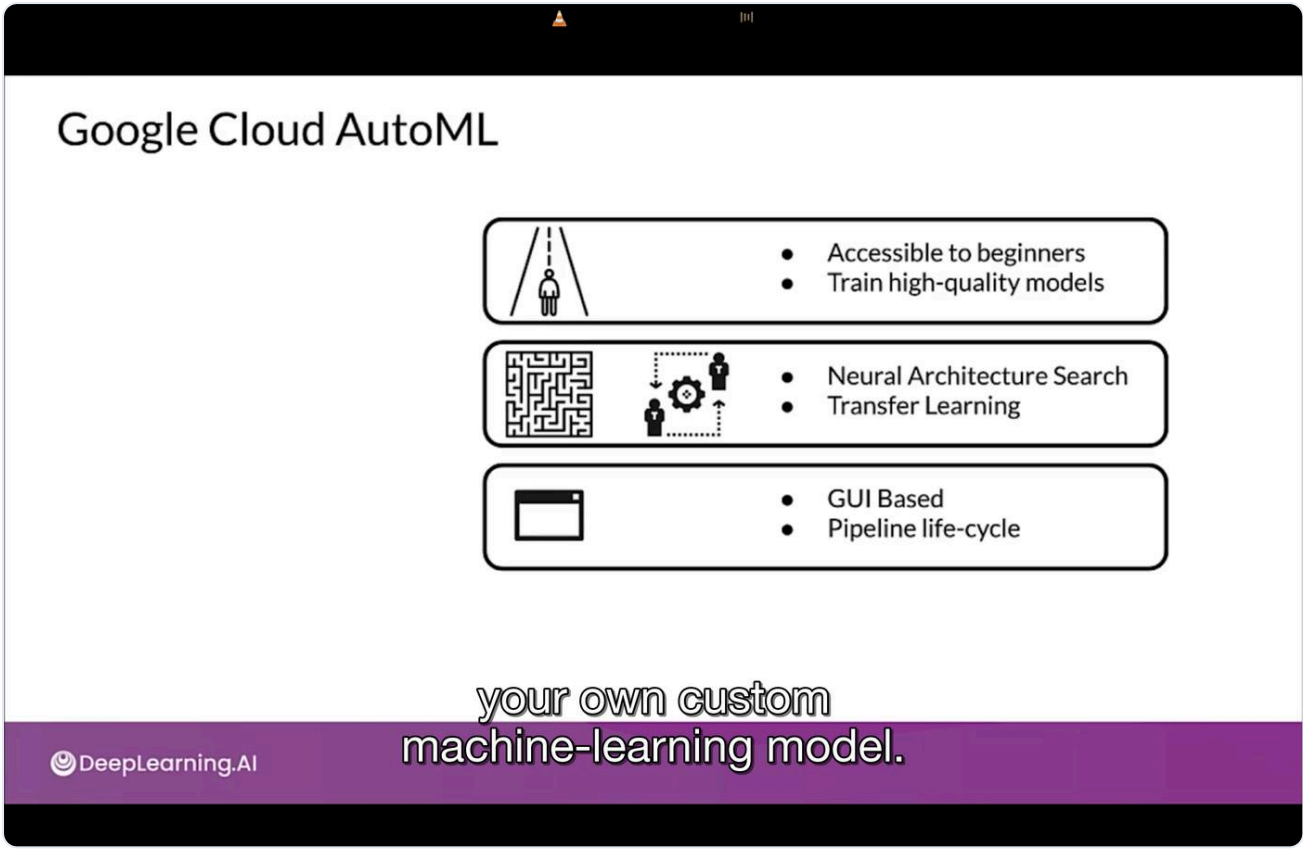

FIGURE 8.13

You're really only a few minutes away from your own custom machine-learning model.

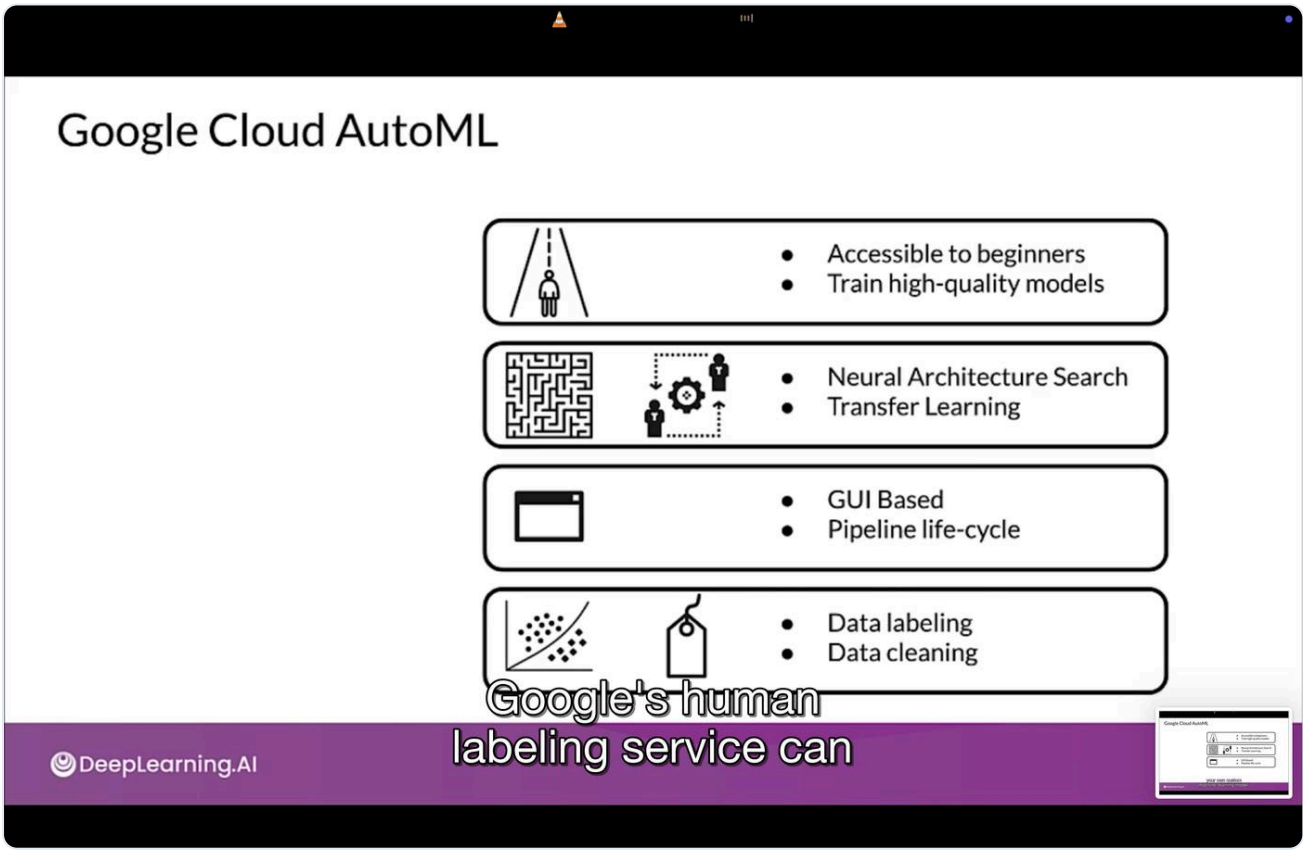

FIGURE 8.14 | Google Cloud AutoML

Google's human labeling service can also put a team of people to work annotating or cleaning your labels to make sure that your models are being trained on high-quality data. Because different problems and different data need to be treated differently, Cloud AutoML isn't just one thing. It's a suite of different products, each focused towards particular use cases and data types. For example, for image data, there's AutoML Vision and for video data, there's AutoML Video Intelligence, for natural language, there's AutoML natural language and for translation, there's AutoML Translation.

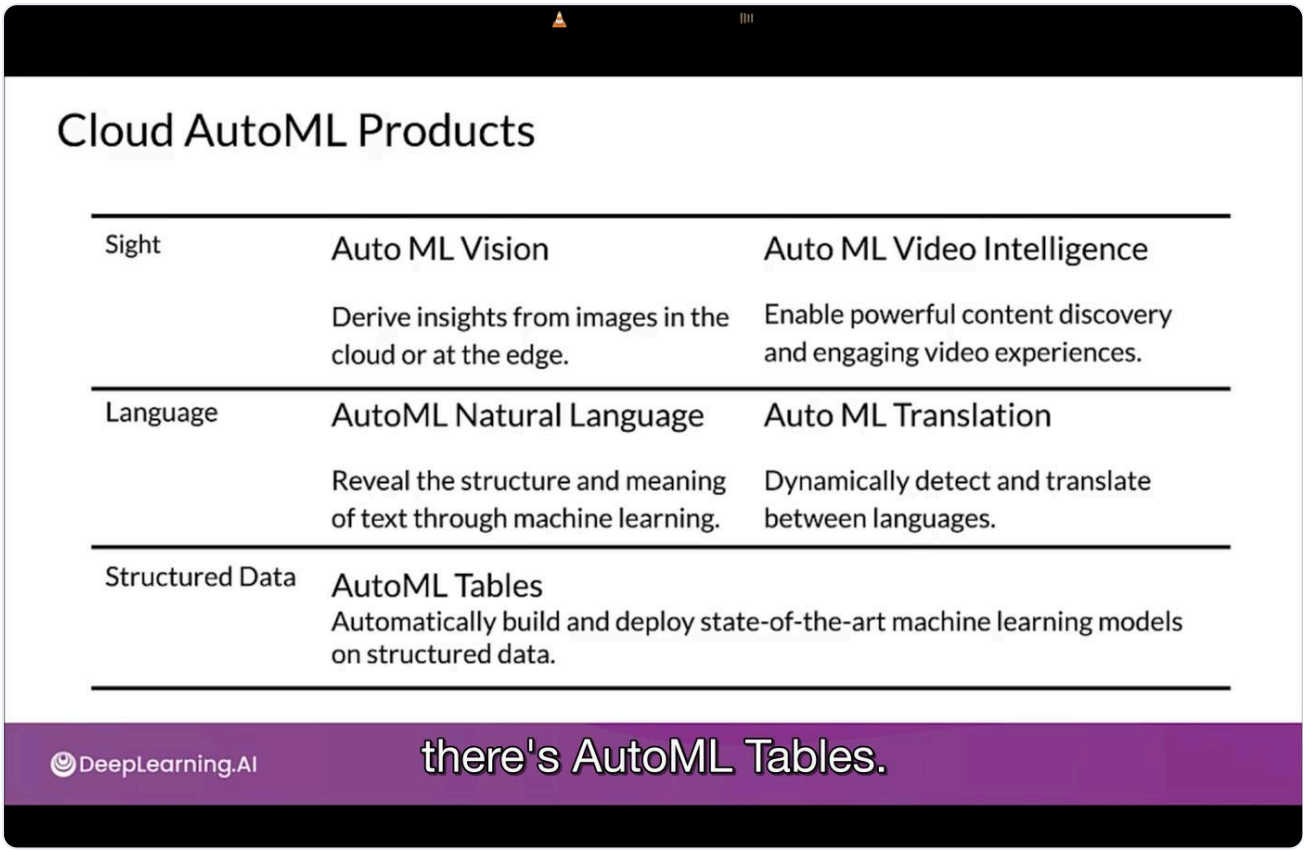

FIGURE 8.15 | Cloud AutoML Products

Finally, for general structured data, there's AutoML Tables. Some of these are broken down even further. For image data, for example, there's both vision and classification and Vision Object Detection.

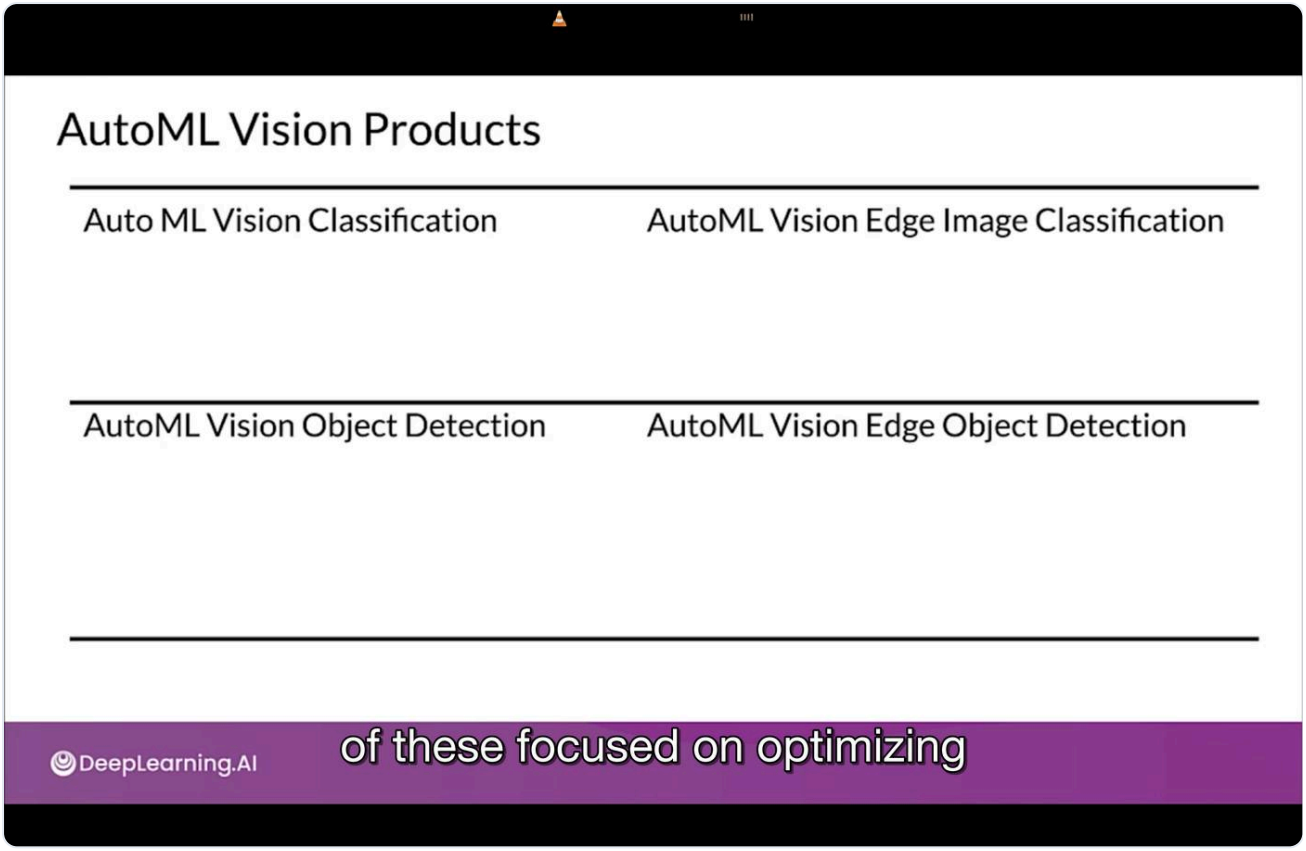

FIGURE 8.16 | AutoML Vision Products

Then there are also Edge versions of both of these focused on optimizing for running inference at the edge in mobile applications or IoT devices. For video, there's both Video Intelligence classification and video object detection.

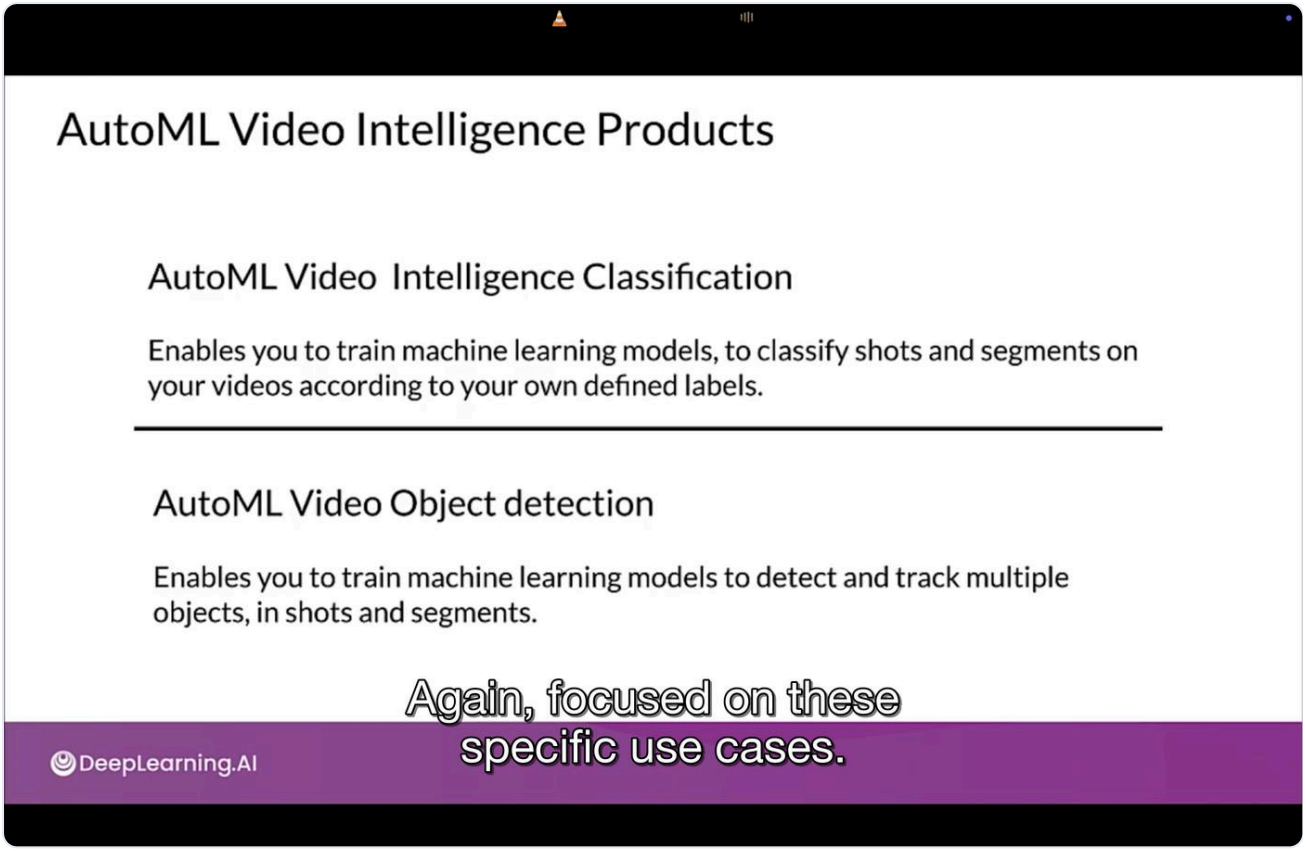

FIGURE 8.17 | AutoML Video Intelligence Products

Again, focused on these specific use cases. How do all three of these different Cloud services operate under the hood? Well, no one knows exactly or can't say if they work for a given provider. However, it is safe to assume that the algorithms at play will be similar to the ones that we've discussed in previous lessons. Also, keep in mind that ML Engineering for Production is a rapidly changing field and new technologies and developments emerge every few months. These new advancements are typically incorporated and exploited to their greatest extent by the AutoML providers.

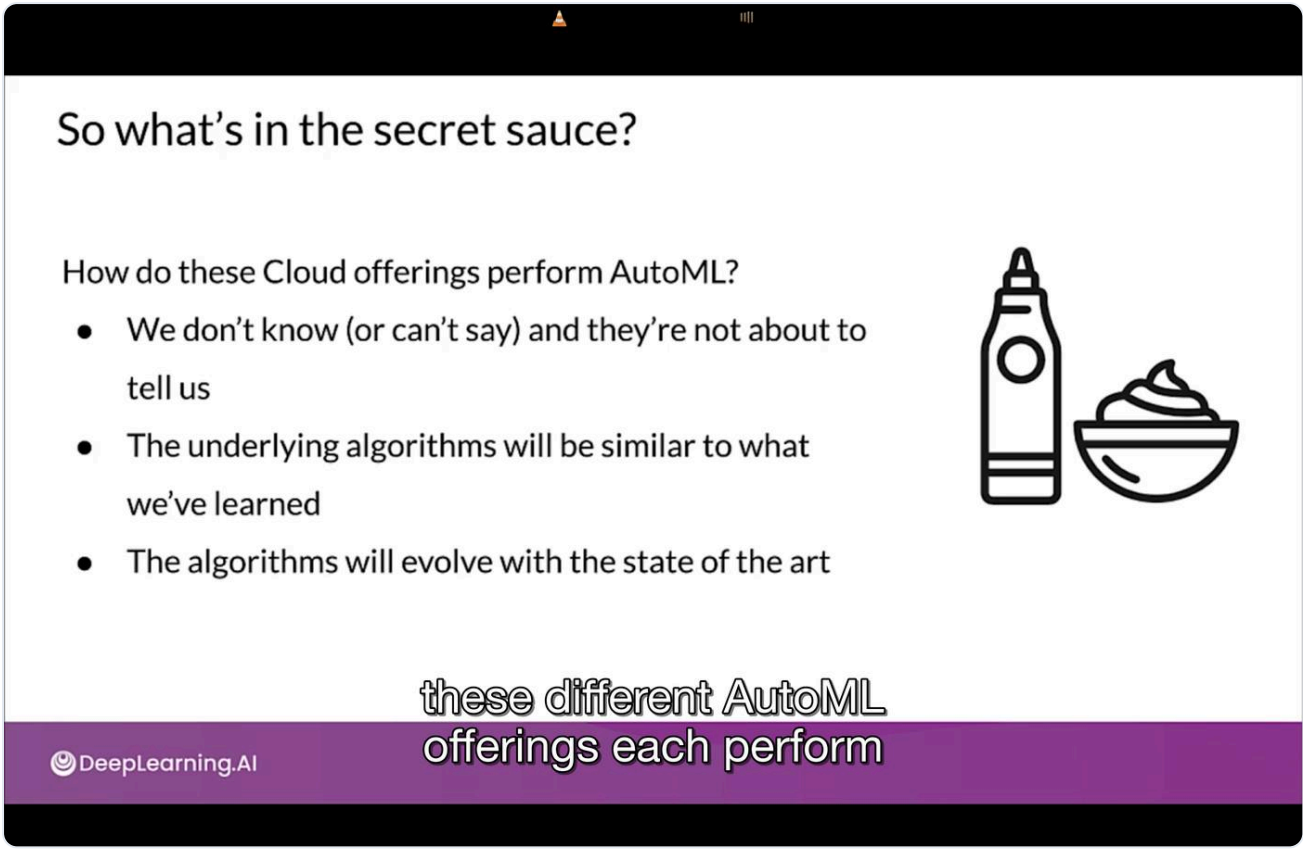

FIGURE 8.18 | How Do These Cloud Offerings Perform AutoML?

Also notice that these different AutoML offerings each perform the same operations that you perform or should perform as you train your model, including feature selection and feature engineering, and cleaning your labels. AutoML designers understand the importance of these activities.

LECTURE 9

Steps to Classify Images Using AUtoML Vision

Now let's take a quick look at the assignment. There are six steps to classifying images using Google Cloud AutoML Vision. First, you set up AutoML and then you create the dataset in Vision by first uploading images to Cloud Storage and then proceeding to create the dataset. This dataset contains images of Clouds, and we're also running on a Cloud. Don't get confused. AutoML then performs the actual training by selecting the best model for the task.

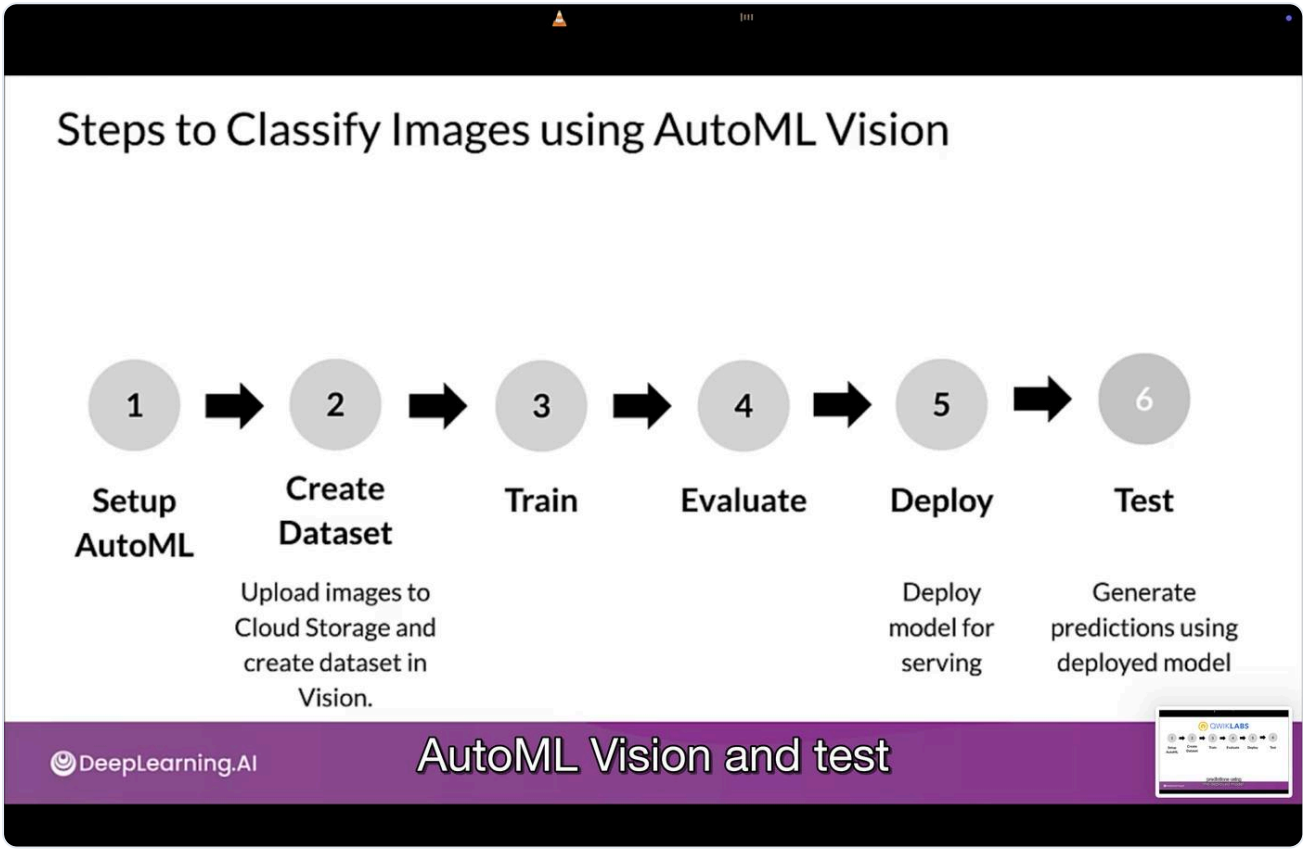

FIGURE 9.1 | Steps to Classify Images Using AUtoML Vision

Finally, we can deploy the model trained by AutoML Vision and test the model by generating predictions using the deployed model. So that you can perform the exercise in a Cloud environment we'll be using Qwiklabs.

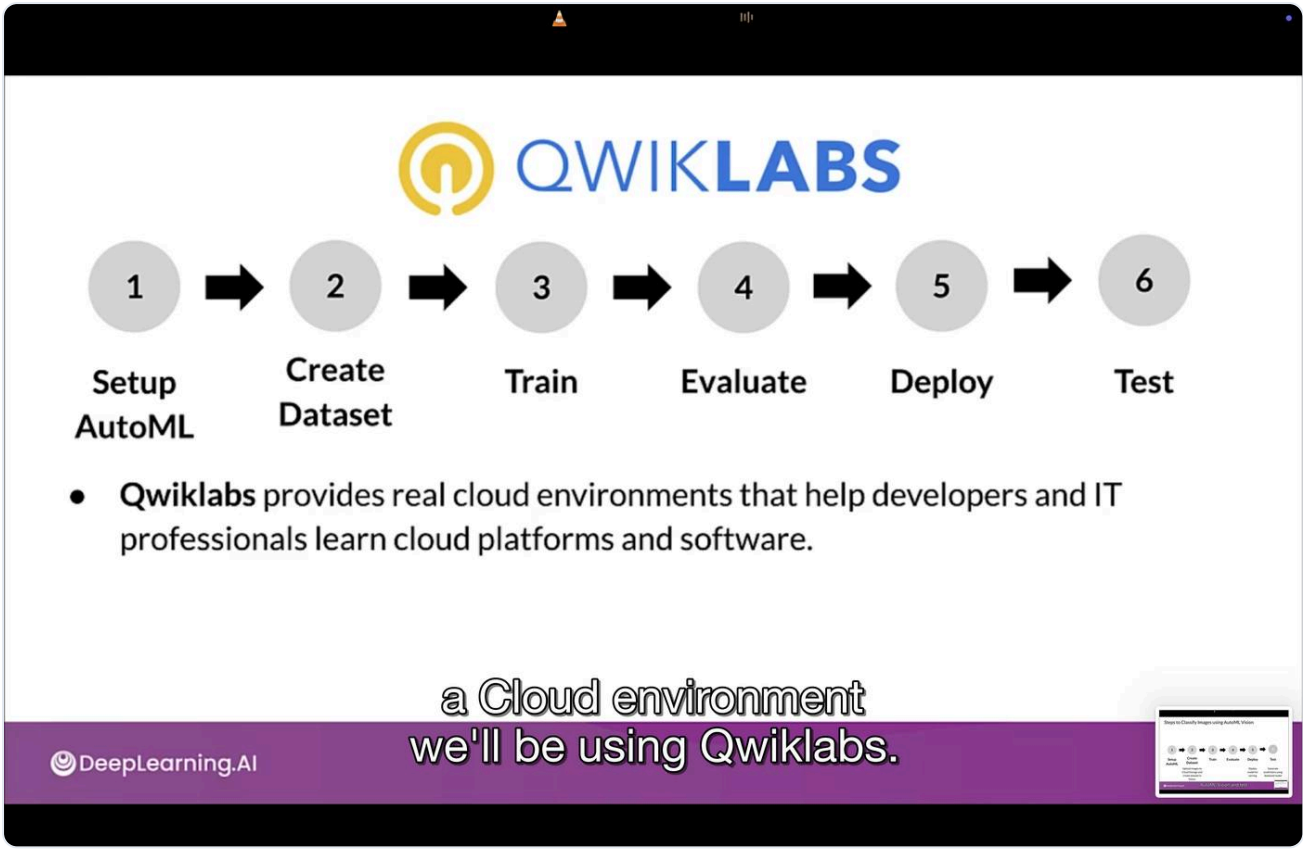

FIGURE 9.2

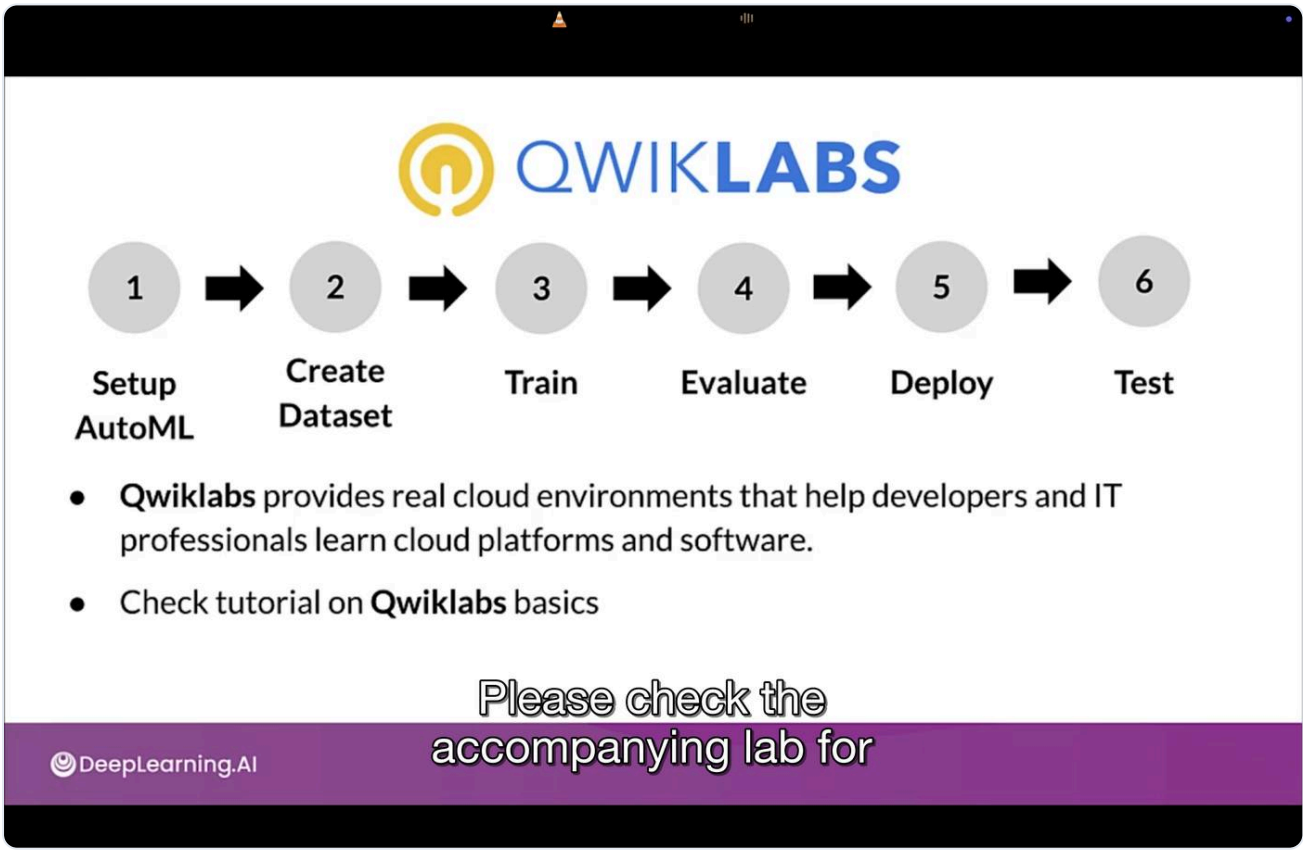

FIGURE 9.3

Qwiklabs provides real cloud environments that help developers and IT professionals learn how to apply cloud platforms and software. You'll be using Qwiklabs to learn how to train models using Google Cloud AutoML. This exercise has six steps, which you'll perform live on the Google Cloud using Qwiklabs. Please check the accompanying lab for details on how to get started with Qwiklabs. Now it's your turn to try AutoML.

LECTURE 10

A Note About Neural Networks

Welcome back this week, is all about model optimization and resource management. We'll be talking about a lot of interesting issues including dimensionality and ways to reduce it and why it's important. Then we'll look at some really resource constrained scenarios including mobile and IOT applications. And ways to deal with resource usage in those situations such as quantization and pruning. So we have a lot to talk about, let's get started. The compute,storage and IOT Systems that your model requires will determine how much it costs to put your model into production and maintain it during its entire lifetime. This week we'll be taking a look at some important techniques that can help us manage model resource requirements.

We'll begin by discussing dimensionality and how it affects our models performance and resource requirements.

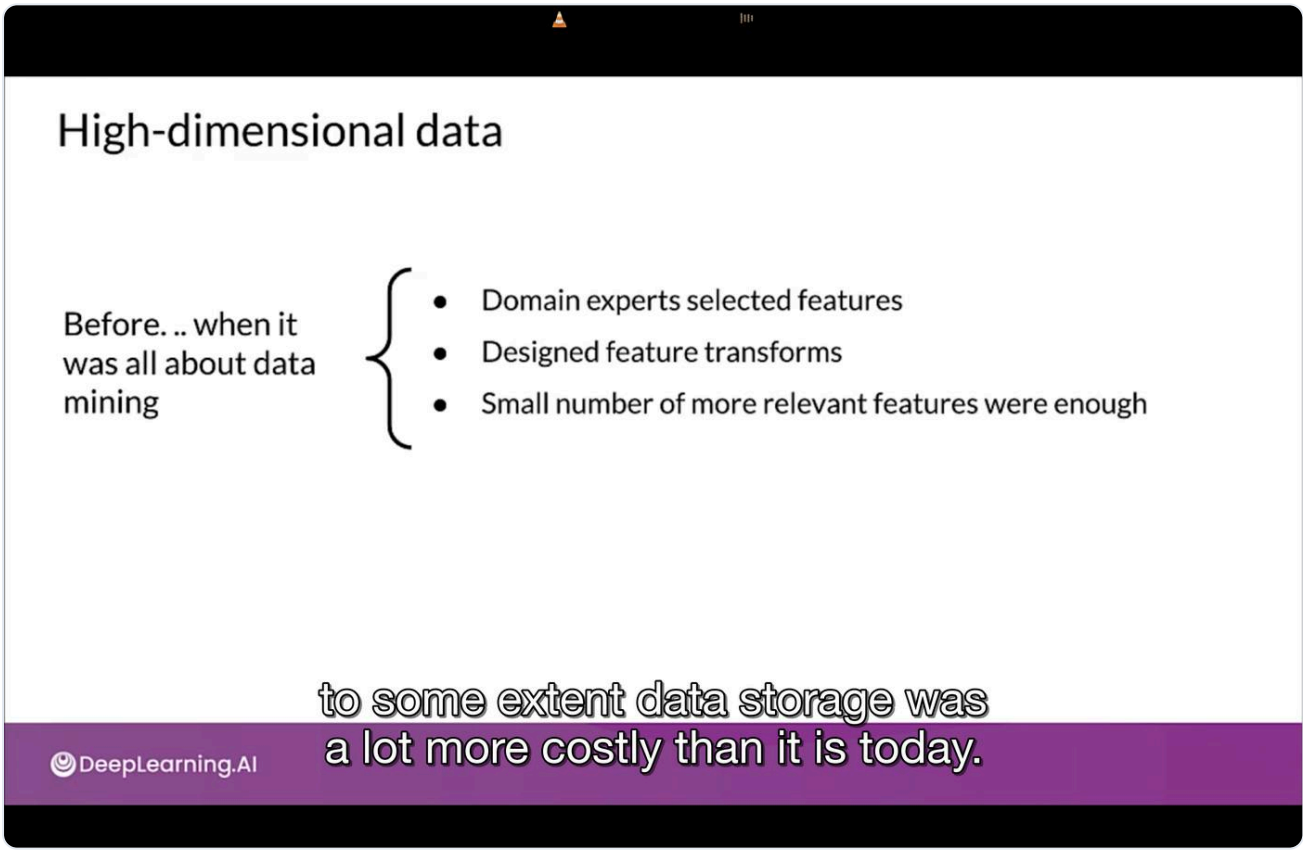

FIGURE 10.1

In the not so distant past data generation and to some extent data storage was a lot more costly than it is today. Back then a lot of domain experts would carefully consider which features or variables to measure before designing their experiments and feature transforms. As a result, data sets were expected to be well designed and potentially contain only a small number of relevant features.

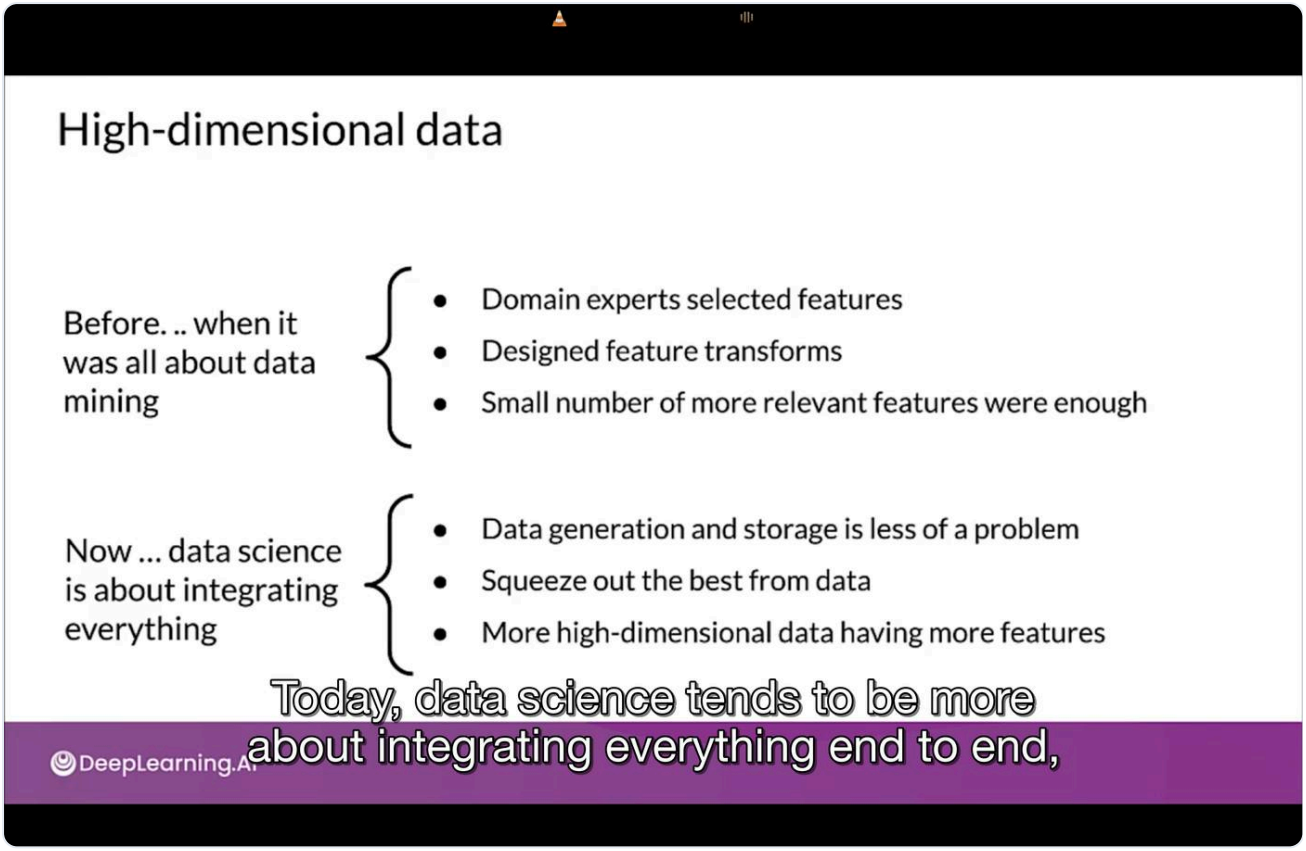

FIGURE 10.2

Today, data science tends to be more about integrating everything end to end, generating and storing data is becoming faster, easier and less expensive. So there's a tendency for people to measure everything they can and include ever more complex feature transformations. As a result, datasets are often high dimensional, containing a large number of features, although the relevancy of each feature for analyzing this data is not always clear. Before going to deep, let's discuss a common misconception about neural networks. Many developers correctly assume that when they train their neural network models, the model itself, as part of the training process, will learn to ignore features that don't provide predictive information by reducing their weights to zero or close to zero.

Well, this is true, the result is not an efficient model. Much of the model can end up being shut off when running inference to generate predictions but those unused parts of the model are still there. They take up space and they consume compute resources as the model server traverses the computation graph. Those unwanted features could also introduce unwanted noise into the data, which can degrade model performance.

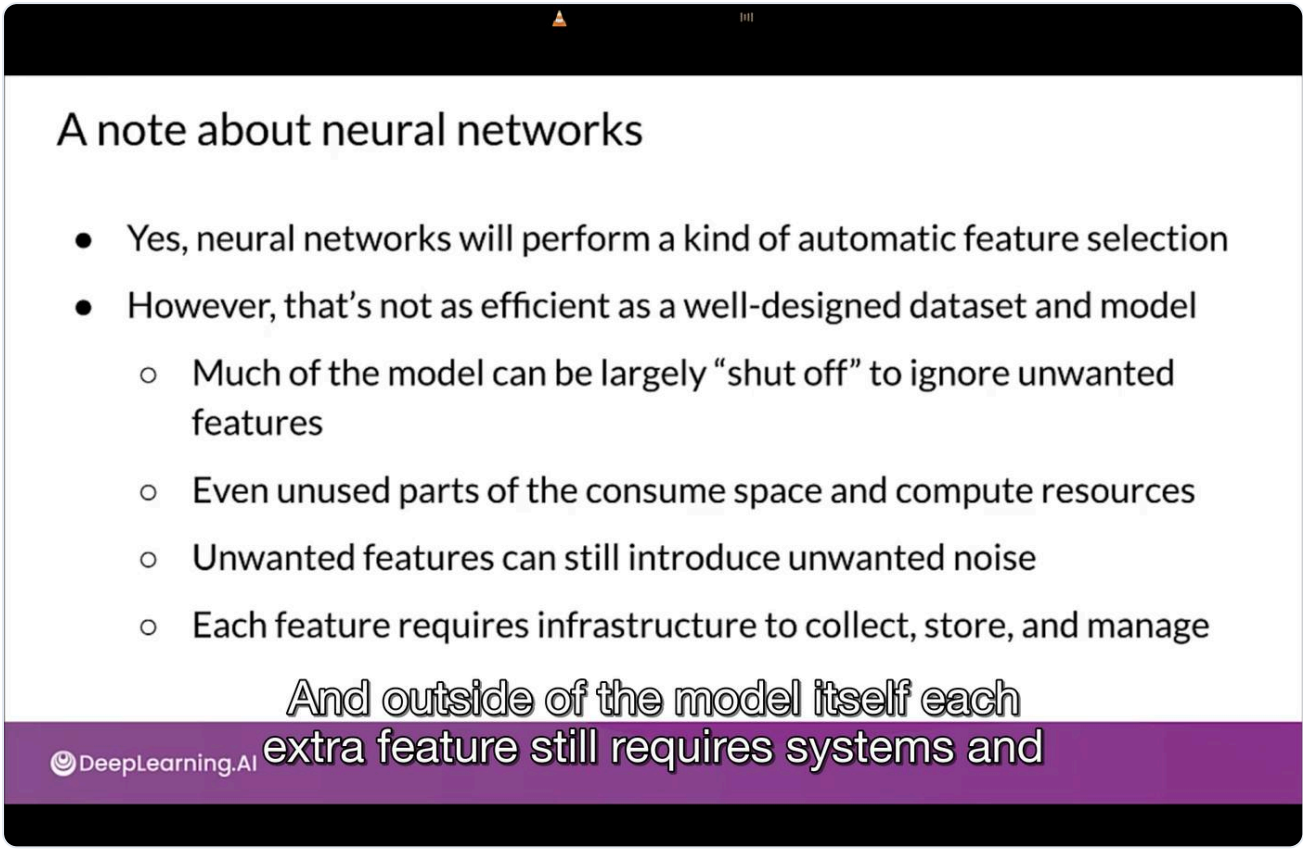

FIGURE 10.3 | A Note About Neural Networks

And outside of the model itself each extra feature still requires systems and infrastructure to collect that data, store it, manage updates, etc, which adds cost and complexity to the overall system. That includes monitoring for problems with the data and the effort to fix those problems if and when they happen. Those costs continue for the lifetime of the product or service that you're deploying, which could easily be years. There are techniques for optimizing models with weights that are close to zero, which you'll learn more about later in this course. But in general, you shouldn't just throw everything at your model and rely on your training process to determine which features are actually useful.

In machine learning we often have to deal with high dimensional data.

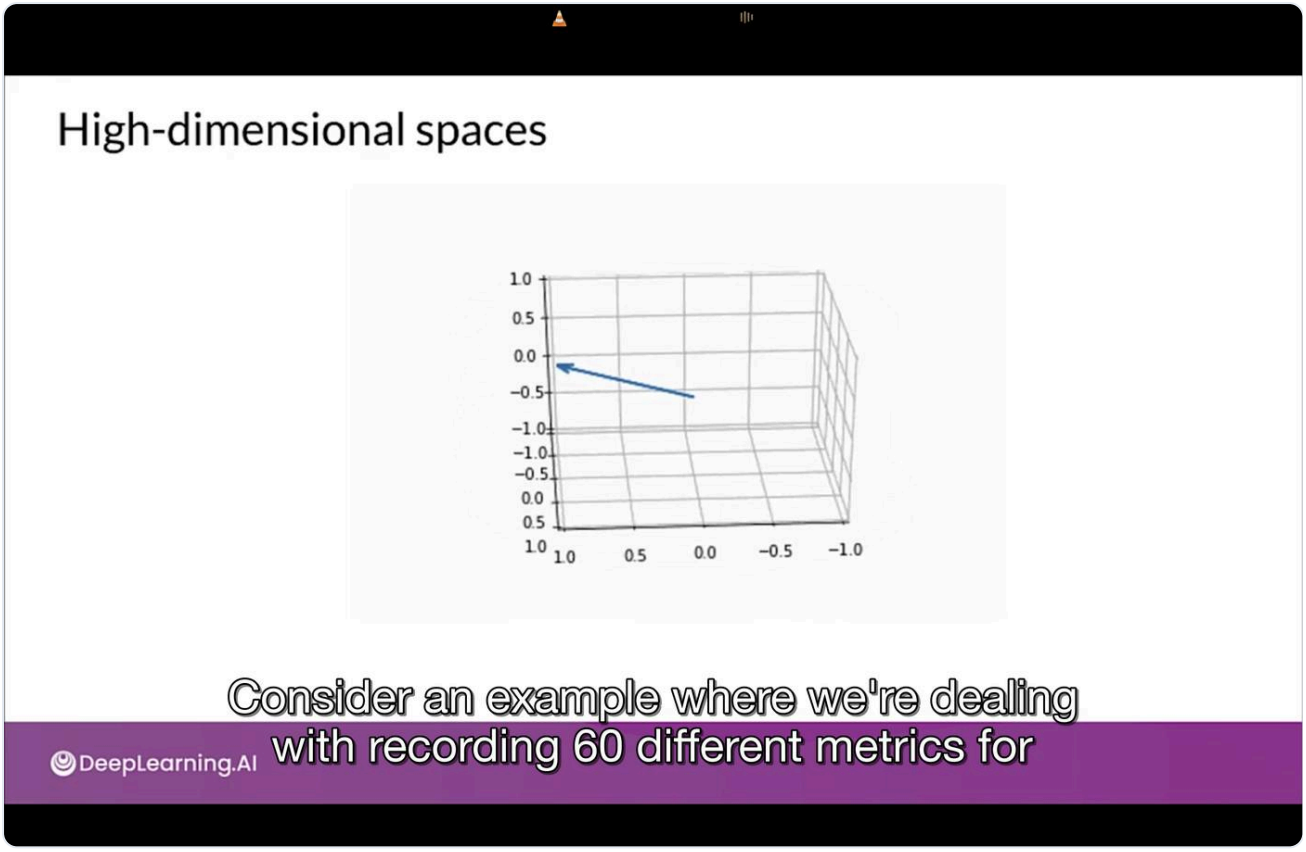

FIGURE 10.4

Consider an example where we're dealing with recording 60 different metrics for each of our shoppers, that means we're working in a space with 60 dimensions. In other cases, if you're trying to analyze grayscale images that are 50 by 50, you're working in an area with 2500 dimensions. If the dimensions or if the images rather are RGB, the dimensionality increases to 7500 dimensions. In this case we have one dimension for each color channel in each pixel of the image.

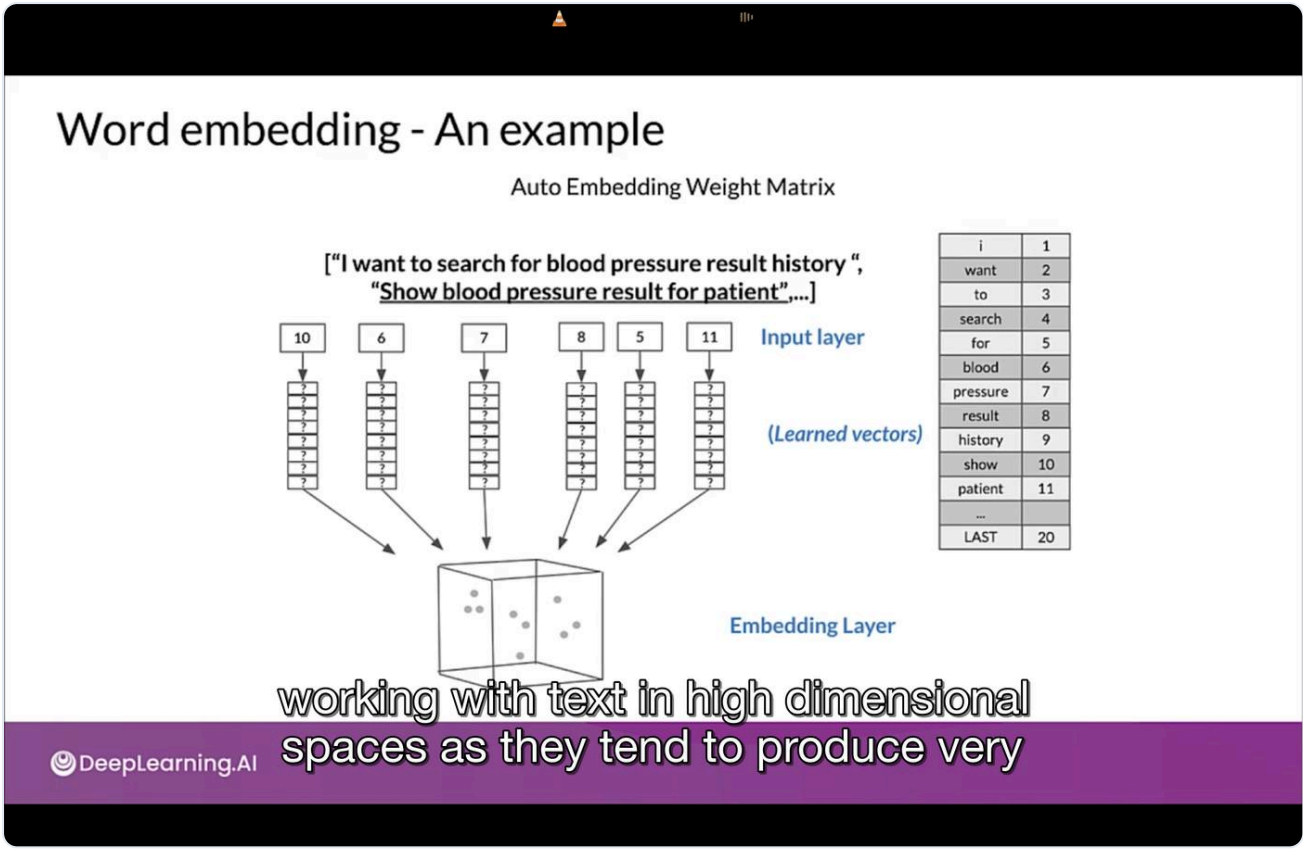

FIGURE 10.5 | Word Embedding - an Example

Some feature representations such as one hot encoding are problematic for working with text in high dimensional spaces as they tend to produce very sparse representations that do not scale well. One way to overcome this is to use an embedding layer that tokenizes the sentences and assigns a float value to each word. This leads to a more powerful vector representation that respects the timing and sequence of the words in a given sentence. This representation can be automatically learned during training. The cube labeled embedding layer in this figure is a conceptual representation of those vectors in a high dimensional space. Let's look at a concrete example of word of embedding using caress.

Let's start by loading the necessary libraries and modules into finding some important parameters.

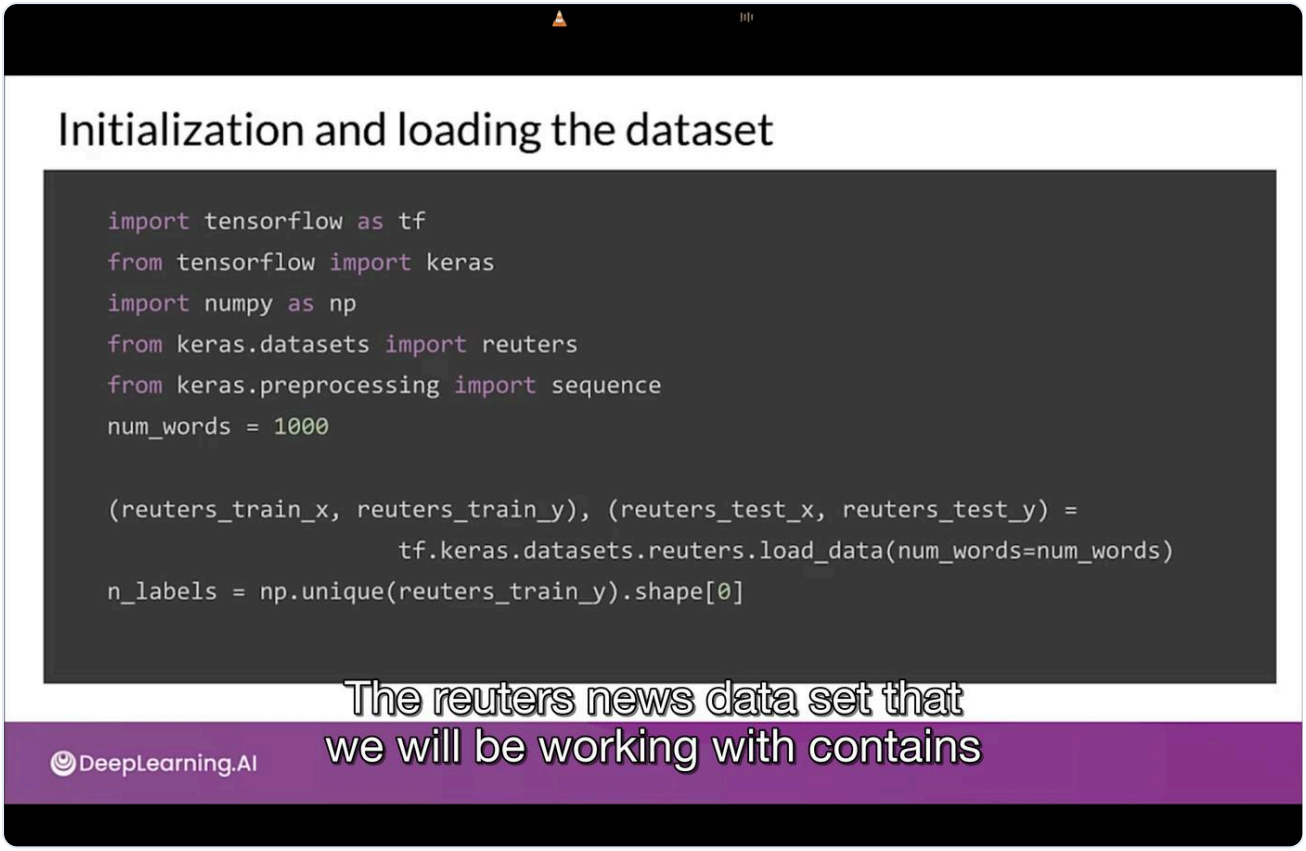

FIGURE 10.6

The reuters news data set that we will be working with contains 11,228 newswires labeled over 46 topics. The documents are already encoded in such a way that each word is indexed by an integer, its overall frequency in the data set. While loading the data set, we specify the number of words will work with 1000 so that the least repeated words are considered unknown. Let's further pre-process the data, so it's ready for training a model. First this converts the training vector Y into a categorical variable for both train and test.

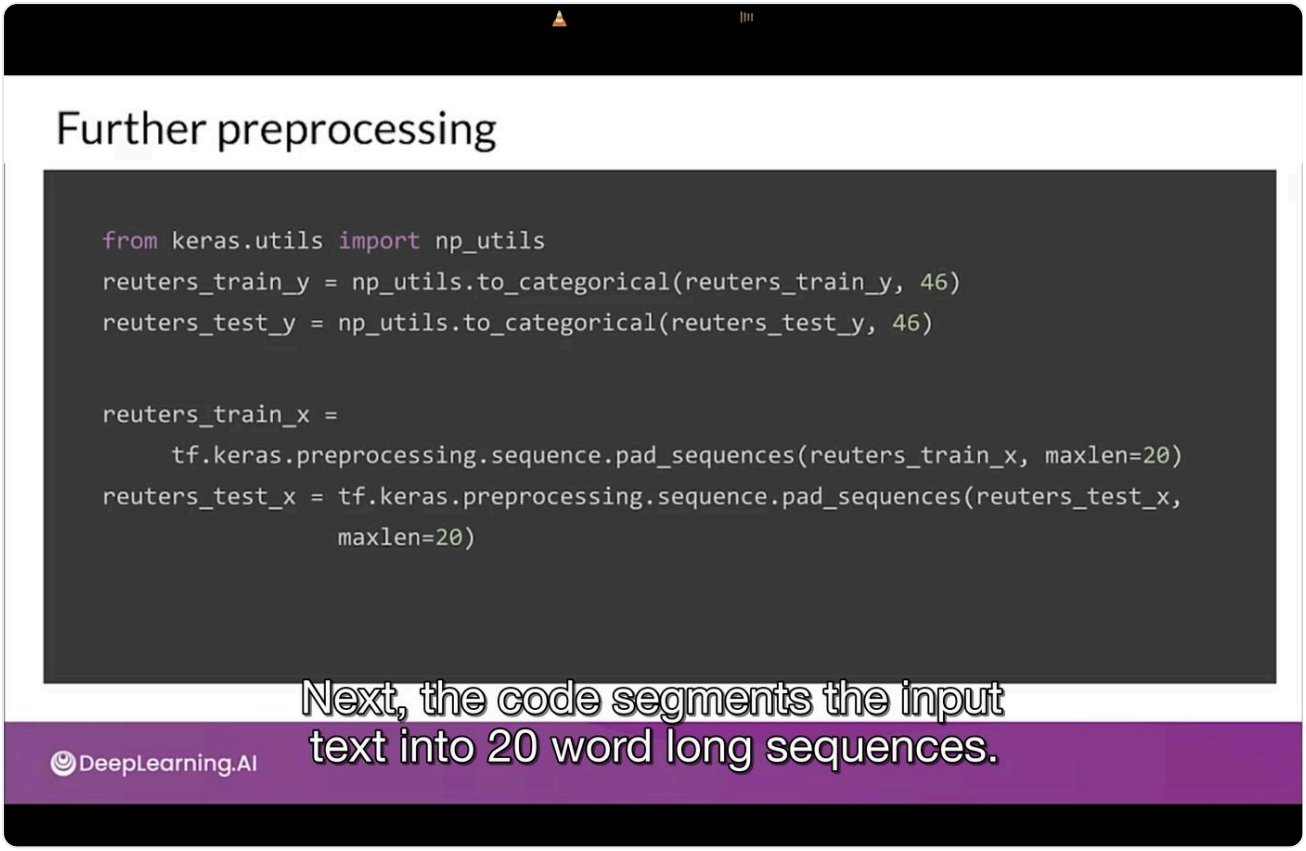

FIGURE 10.7

Next, the code segments the input text into 20 word long sequences. Building the network is the next logical step.

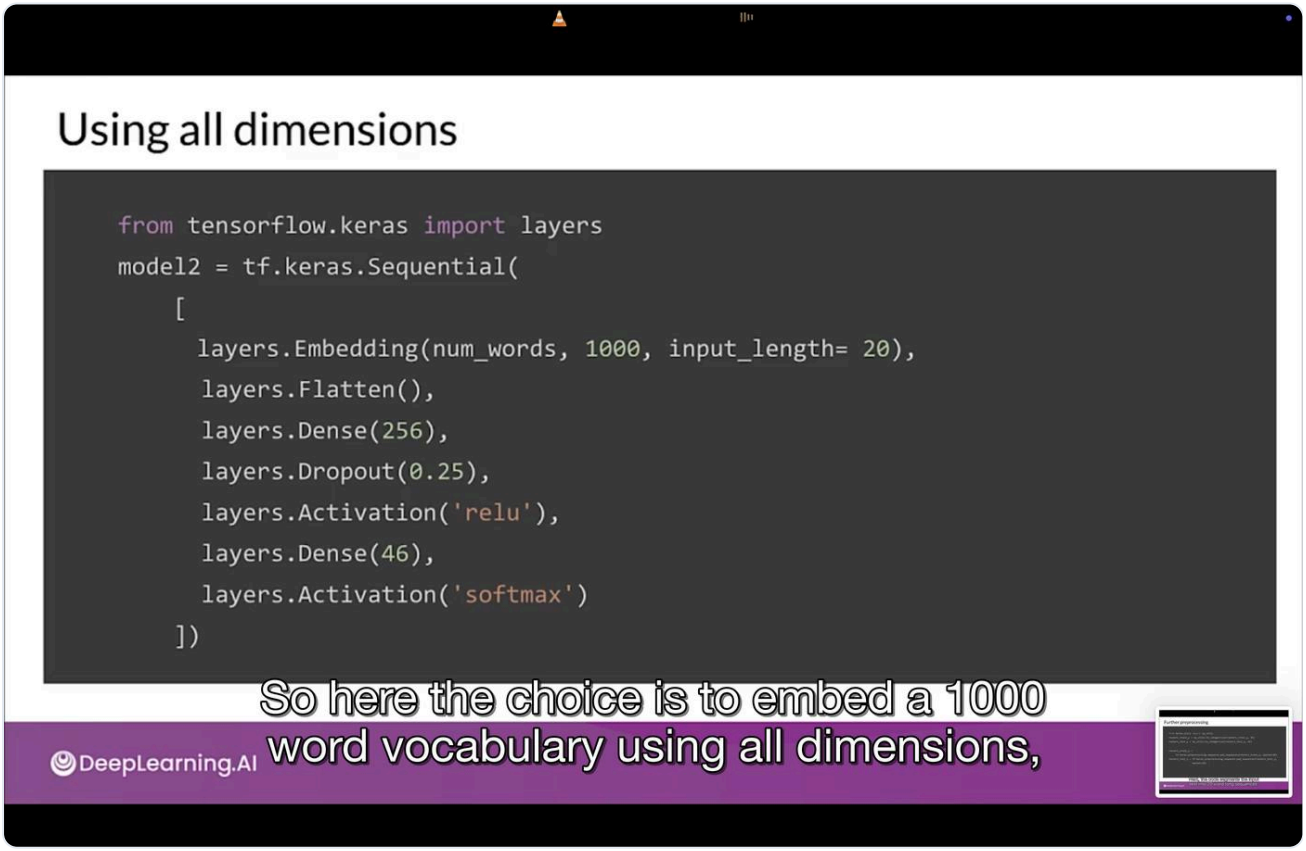

FIGURE 10.8

So here the choice is to embed a 1000 word vocabulary using all dimensions, here we're using all the dimensions of the data. The last layer is dense with dimension 46 since the target variable is a 46 dimensional vector of categories.

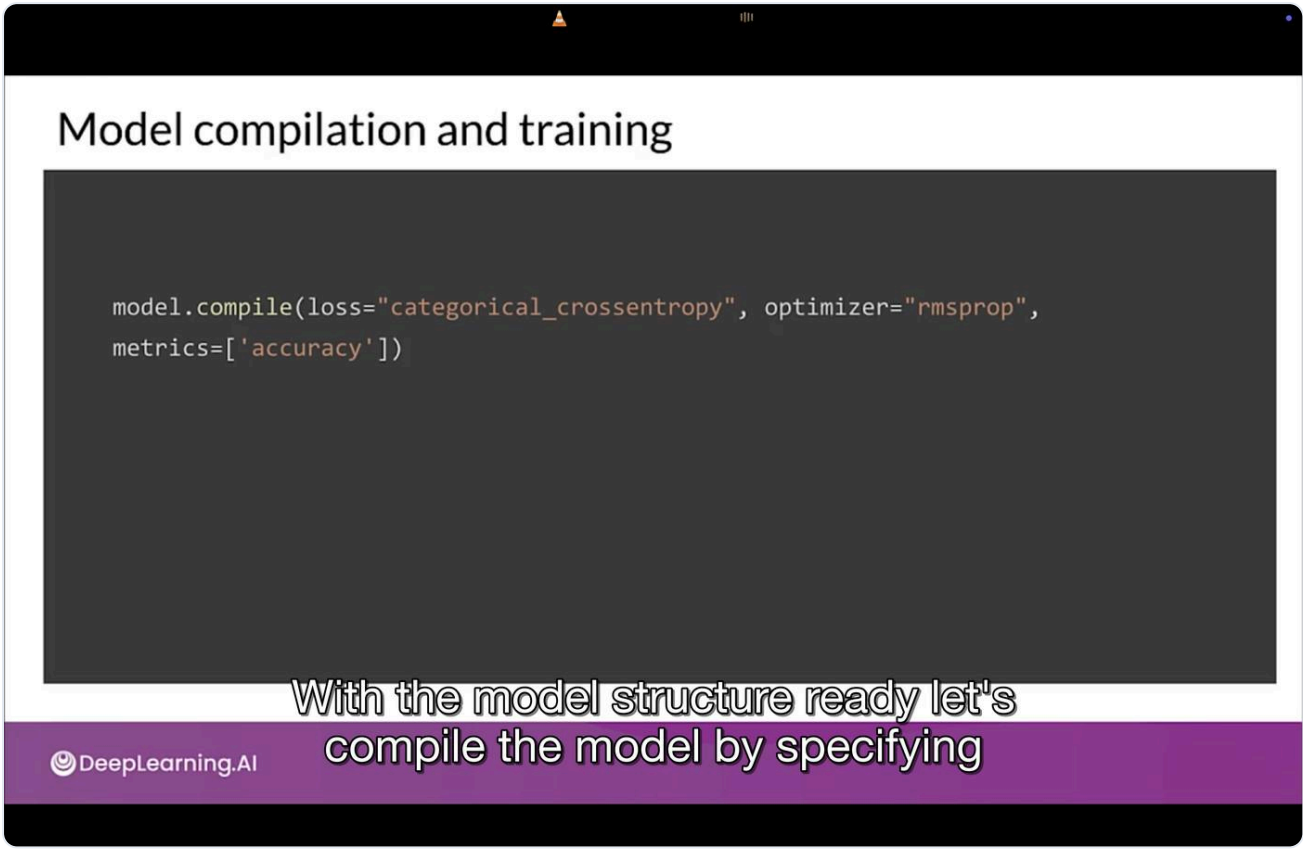

FIGURE 10.9

With the model structure ready let's compile the model by specifying the loss optimizer and output metric. For this problem the natural choices are categorical crossentropy loss, rmsprop optimization and accuracy is the metric.

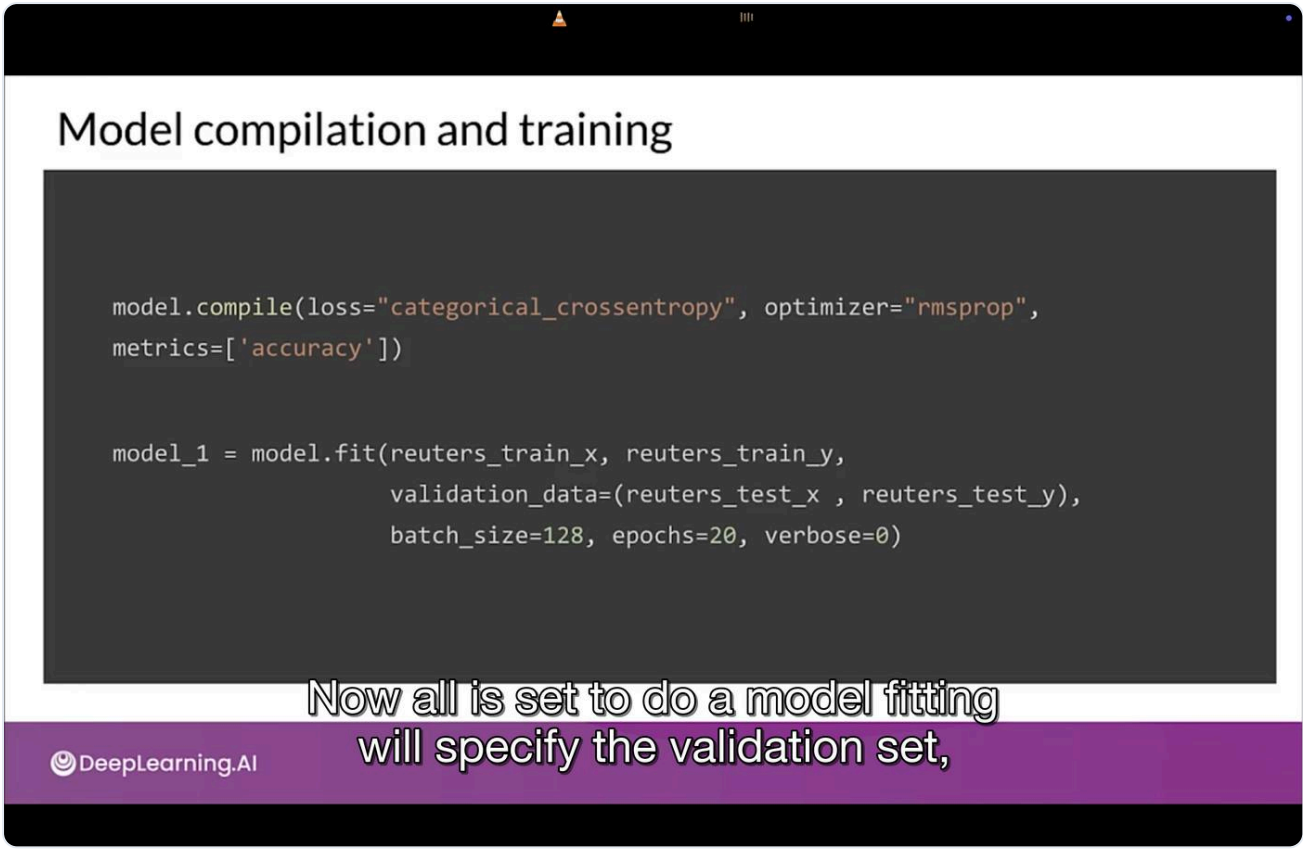

FIGURE 10.10 | Model Compilation and Training

Now all is set to do a model fitting will specify the validation set, batch size and the number of epics for training.

FIGURE 10.11 | Example with a Higher Number of Dimensions

Here is the accuracy as a function of training epics and the model loss as a function of training epics. Notice that after about two epics are training data results in significantly higher accuracies and lower losses compared to the validation set. This is a clear indication that the model is severely overfitting, this may be the result of using all the dimensions of the data. So the model is picking up nuances in the training set that do not generalize well.

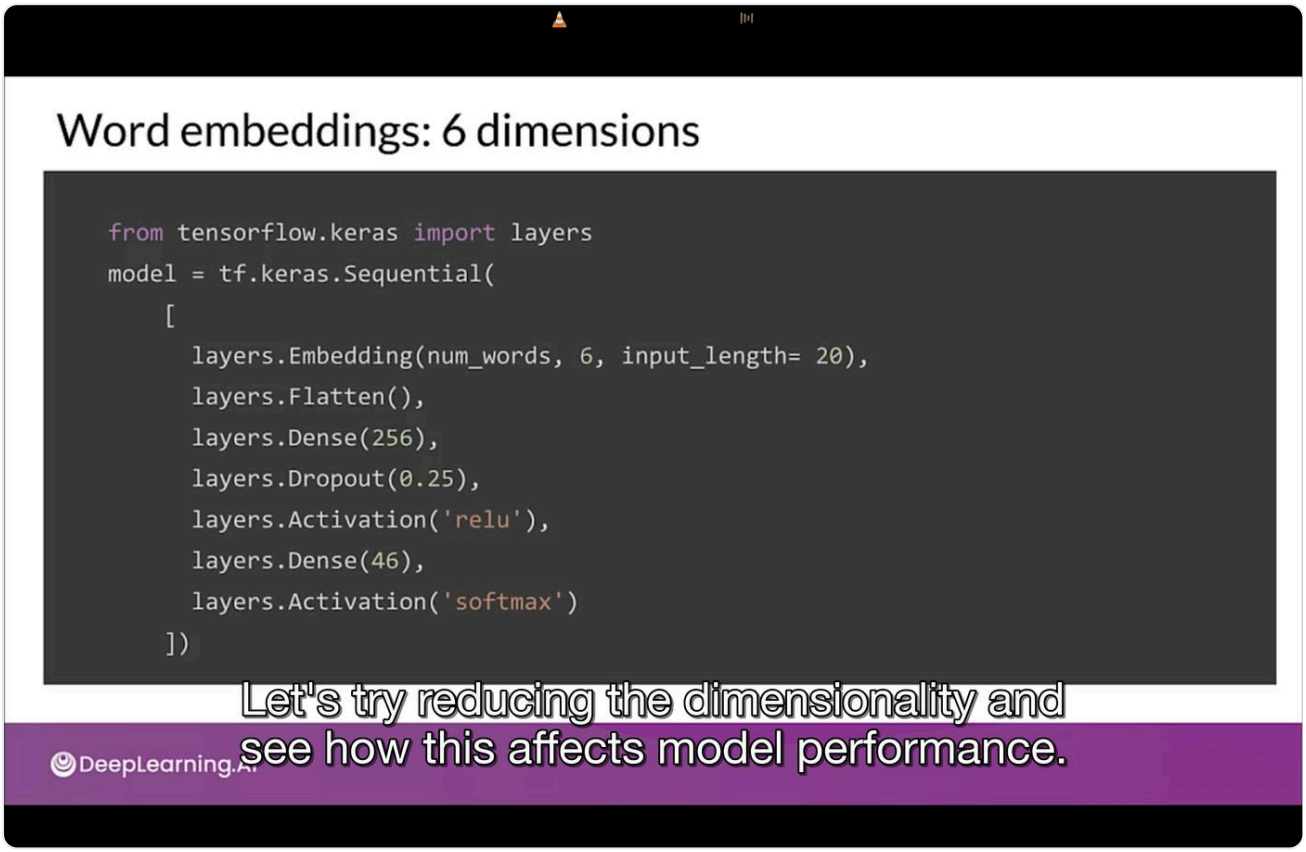

FIGURE 10.12

Let's try reducing the dimensionality and see how this affects model performance. For that let's embed a 1000 word vocabulary into 6 dimensions, this is roughly a reduction of a fourth root factor. The model remains unchanged otherwise.

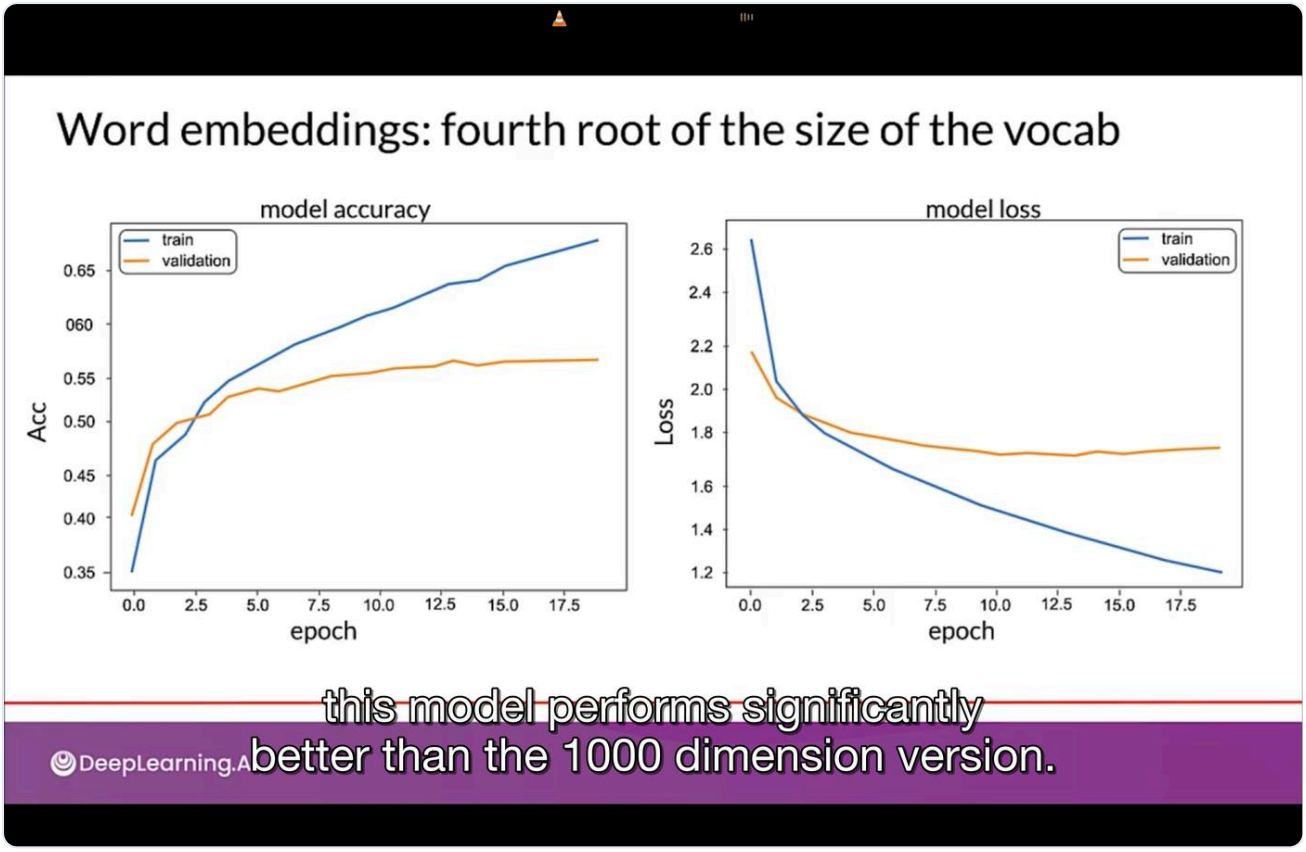

FIGURE 10.13

There may still be some overfitting, but with that one change, this model performs significantly better than the 1000 dimension version.

LECTURE 11

Curse of Dimensionality in the Distance Function

Let's talk about the curse of dimensionality and why this is a very important topic when building models.

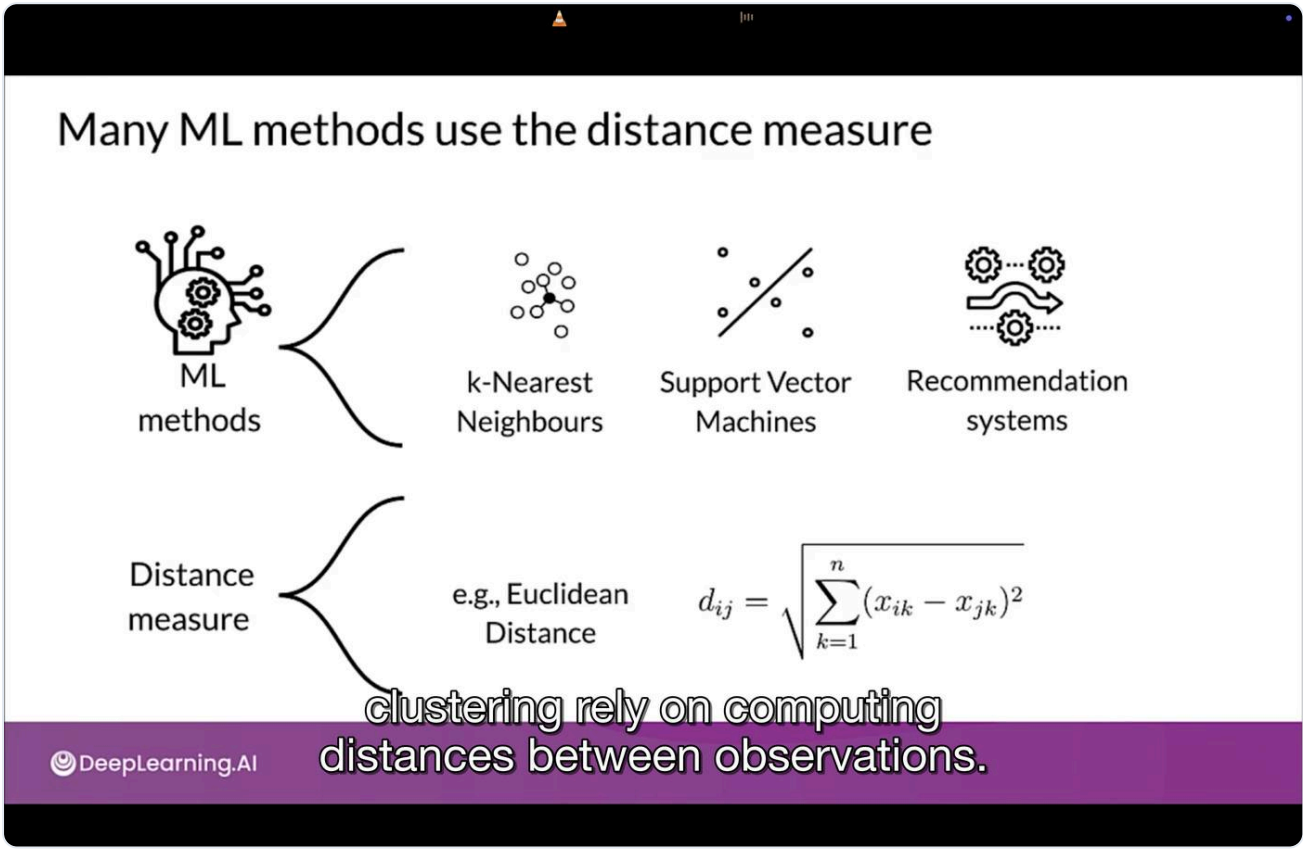

FIGURE 11.1 | Many ML Methods Use the Distance Measure

Many common machine learning tasks like segmentation and clustering rely on computing distances between observations. For example, supervised classification uses the distance between observations to assign a class, k-nearest neighbors is a basic example of this. Support vector machines or SVMs deal with projecting observations using kernels based on the distance between the observations after projection. Another example is recommendation systems that use a distance based similarity measure between the user and the item attribute vectors. There could even be other forms of distance being used. So distance plays an important role in understanding dimensionality.

One of the most common distance metrics is Euclidean distance, which is simply a linear distance between two points in a multi-dimensional space. The Euclidean distance between two dimensional vectors with Cartesian coordinates is calculated using this familiar formula. But why is distance important? Let's look at some issues with measuring distance in high dimensional spaces. For example you might wonder why data being high dimensional can be an issue. In

extreme cases where we have more features and observations, we run the risk of massively overfitting our model. But in the more general case when we have too many features, observations become harder to cluster.

Too many dimensions can cause every observation in your data set to appear equidistant from all the others. And because clustering using a distance measures such as Euclidean distance to quantify the similarity between observations, this is a big problem. If the distances are all approximately equal, all the observations appear equally alike and no meaningful clusters can be formed. As dimensionality grows, the contrast provided by the usual metrics decreases.

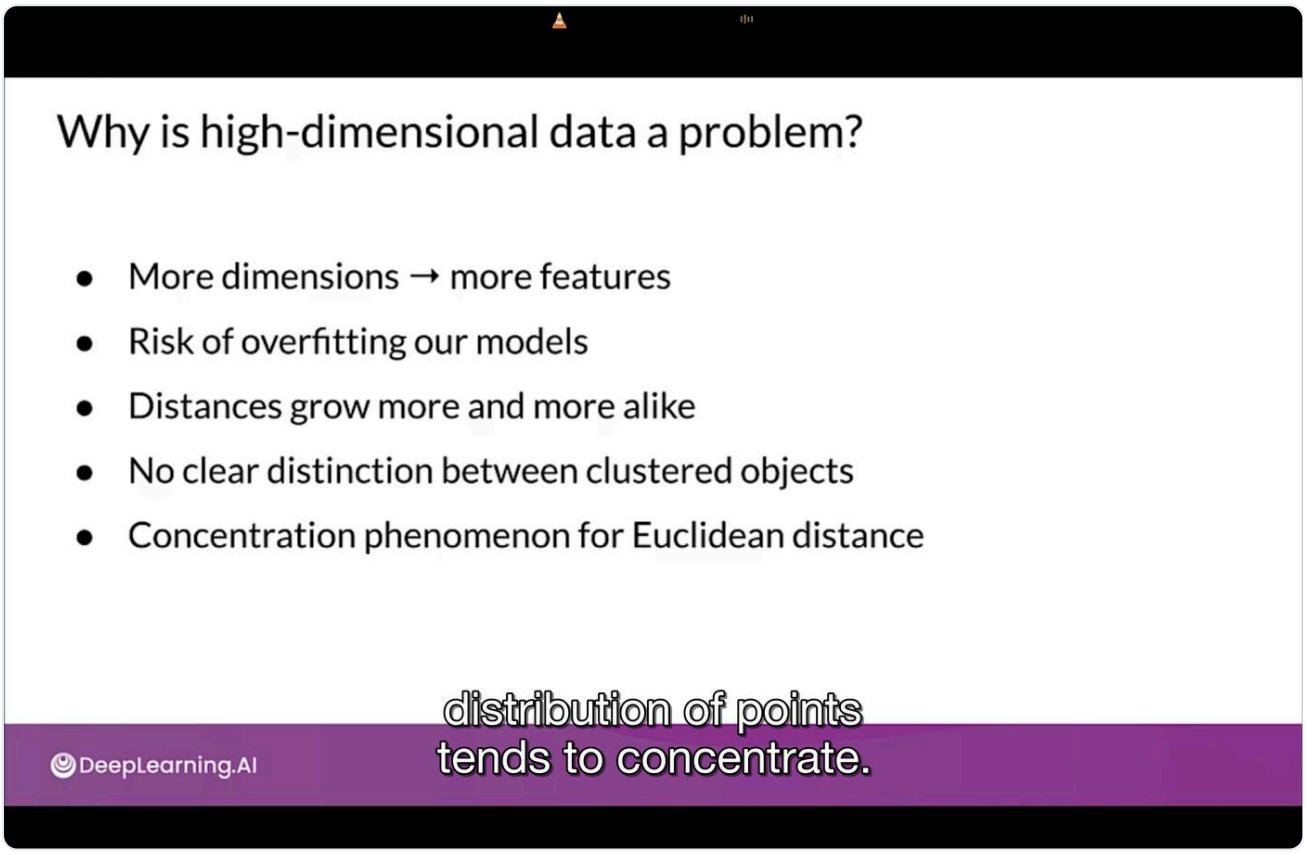

FIGURE 11.2 | Why Is High-dimensional Data a Problem?

In other words, the distribution of norms in a given distribution of points tends to concentrate. That can cause unexpected behavior in high dimensional spaces. This phenomenon is called the curse of dimensionality. It includes situations where non-intuitive properties of data are observed in these high dimensional spaces. This is specifically related to the usability and interpretation of distances and volumes. When it comes to the curse of dimensionality, there are two things to consider. On the one hand, machine learning is good at analyzing data when dealing with many dimensions. However, we humans aren't adept at finding patterns and data that may be spread out across several dimensions, especially if those dimensions are interrelated in counterintuitive ways.

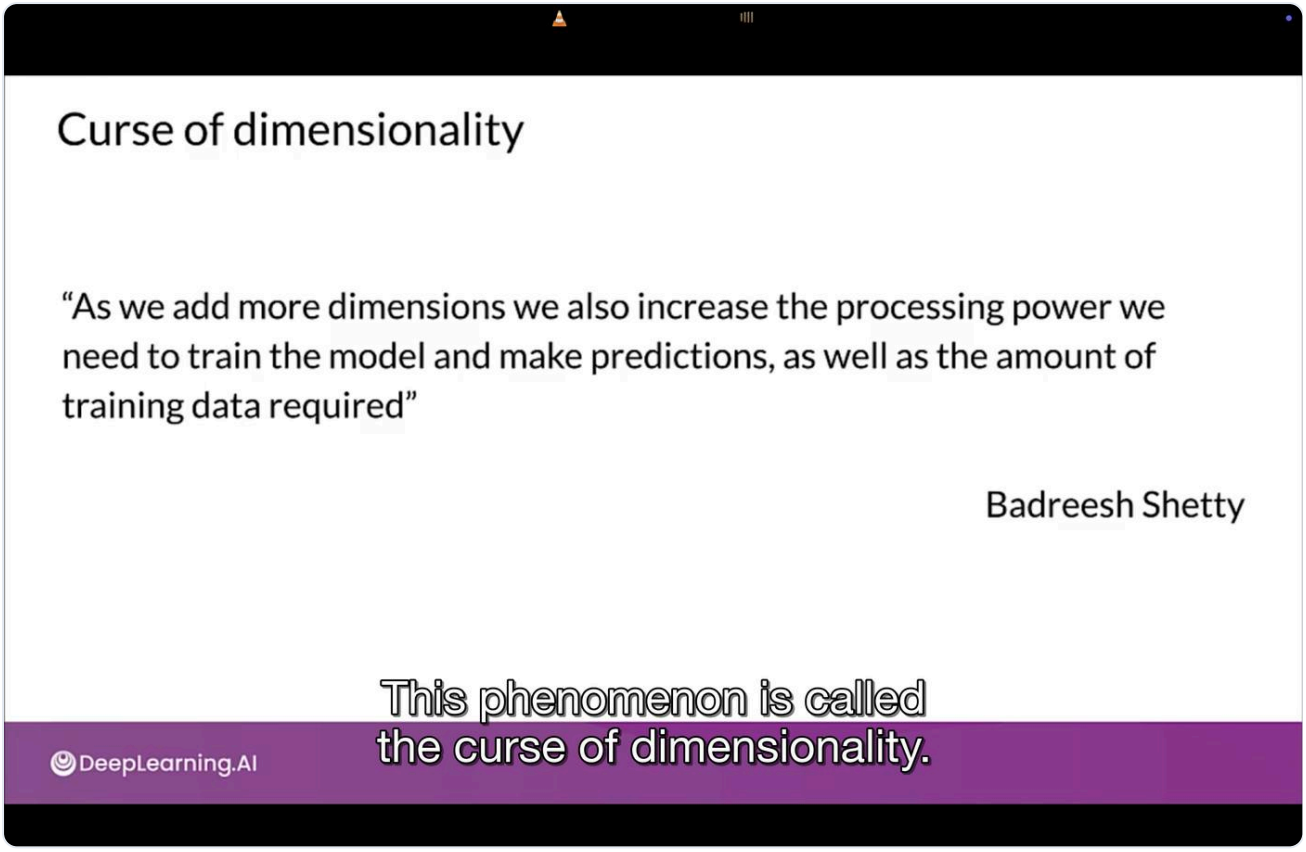

FIGURE 11.3

On the other hand, as we add more dimensions, we also increase the processing power we need to analyze the data and at the same time we also increase the amount of training data required to make meaningful models. If you're curious, Richard Bellman first coined the term curse of dimensionality over half a century ago in 1961 in his book Adaptive Control Processes, A Guided Tour. So adding more features can easily create problems. This could include redundant or irrelevant features appearing in data. Moreover, noise is added when features don't provide predictive power for our models. On top of that, more features make it harder for one to interpret and visualize data.

Finally, more features mean more data, so you need to have more storage and more processing power to process it.

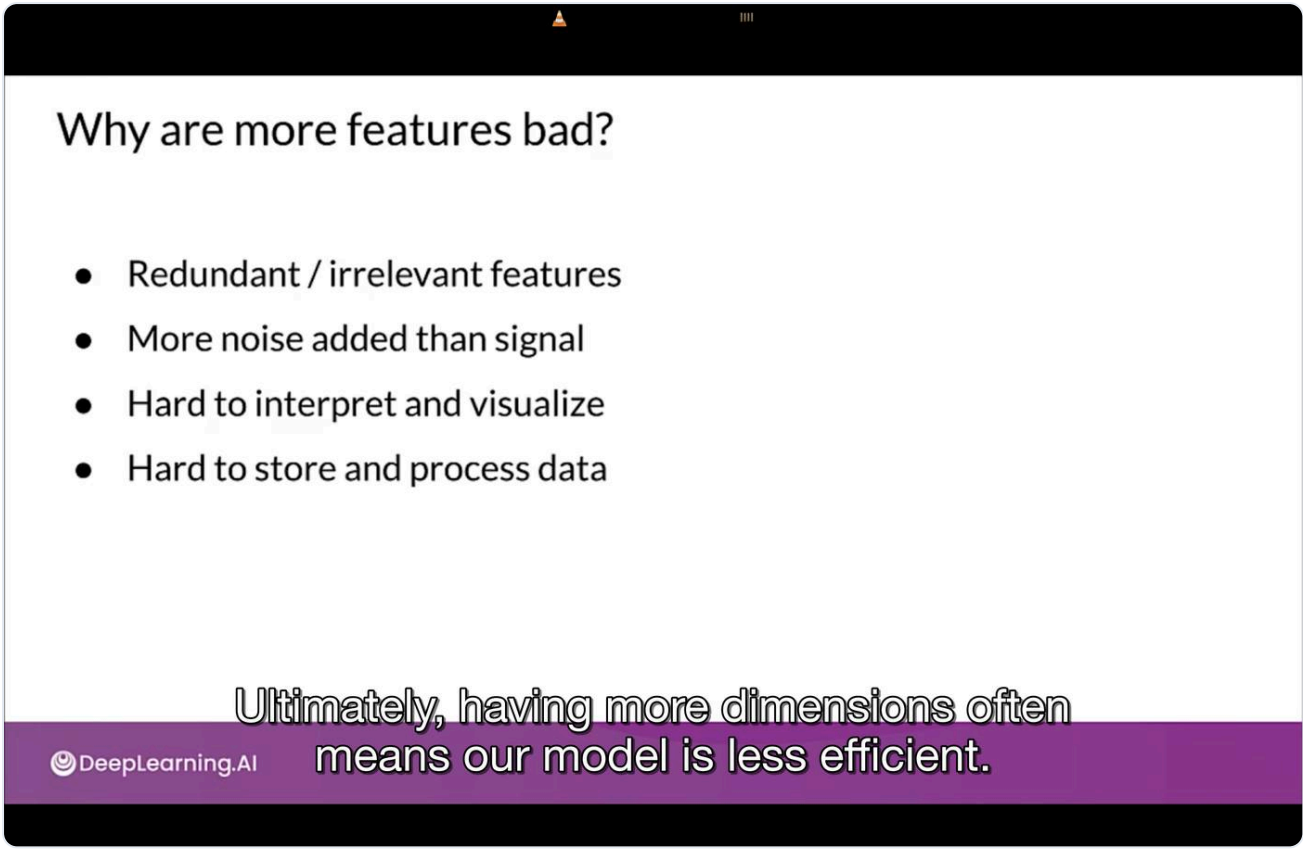

FIGURE 11.4 | Why Are More Features Bad?

Ultimately, having more dimensions often means our model is less efficient.

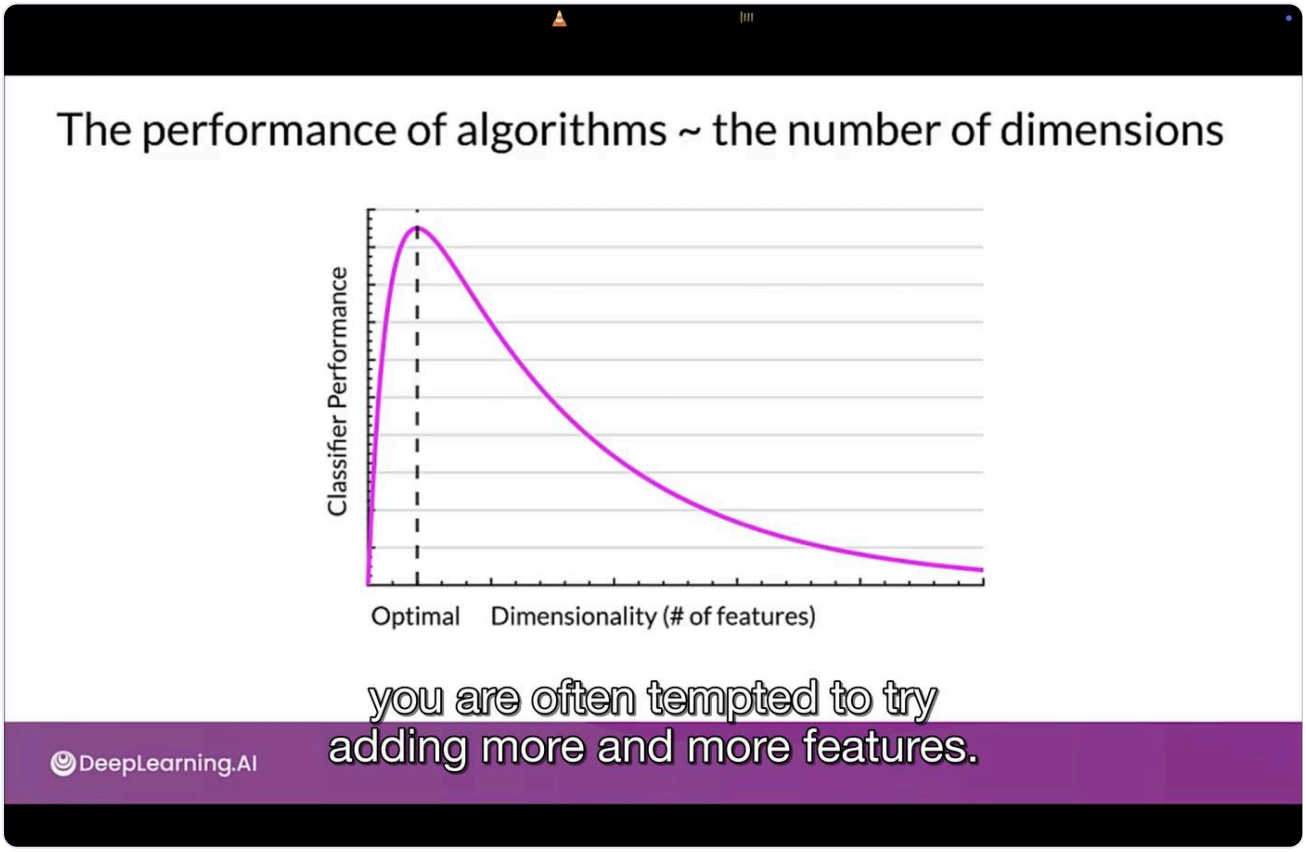

FIGURE 11.5 | Classifier Performance

When you have problems getting your model to perform, you are often tempted to try adding more and more features. But as you add more features, you reach a certain point where your model's performance degrades. This graph shows this well. Here you see that as the dimensionality increases, the classifiers performance increases until the optimum number of features is reached. A key point to understand here is that you are increasing the dimensionality without increasing the number of training samples and that results in a steady decrease in classifier performance after the optimum. Let's look at another problem with dimensionality to uncover what is behind this behavior.

Let's look deeper to try to understand why more dimensions can hurt your models. Let's start by understanding why the number of parameters of a function impact the difficulty of learning that function. Take for example, the parameters of a line function. In this case the list is finite. It simply means discretizing the features. Let's simplify this by using numbers from 1-5. Assuming that you have a function with a single parameter, then there are only five possible values that this parameter can take. What happens if you add a second parameter that can also take five values. Well, let's see.

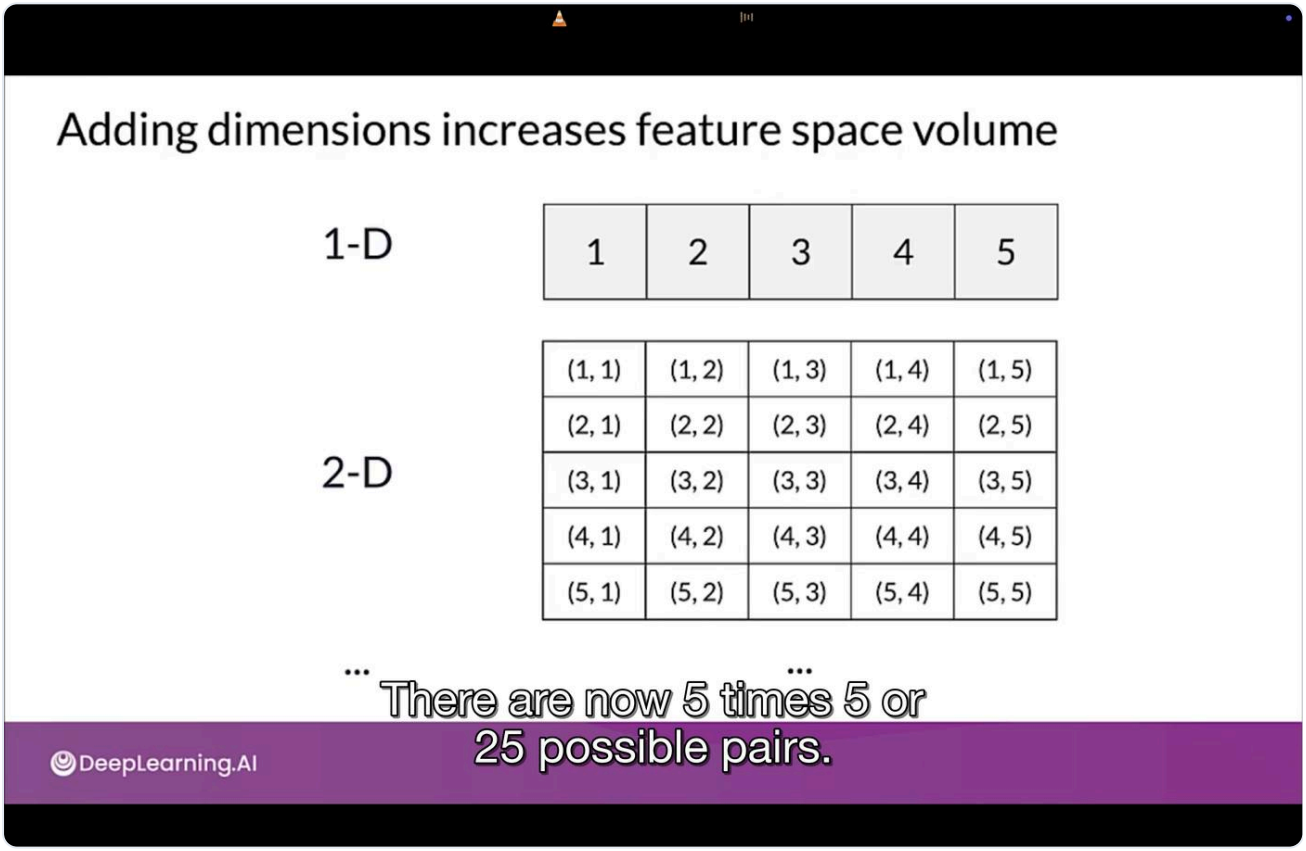

FIGURE 11.6 | Adding Dimensions Increases Feature Space Volume

There are now 5 times 5 or 25 possible pairs. This is a simple example using discrete parameter values, but of course it's even worse with continuous variables. What happens if you had a third parameter? Now the representation is a cube, you probably see the formula behind this reasoning. If we have n values that a parameter can take and m parameters, you end up with n to the m possible parameter values. The number of parameter values grows exponentially. Now, how

big of a problem is it? Well, it's a very big problem indeed which is why this is called the curse of dimensionality. An increase in the number of dimensions of a data set means there are more entries in the feature vector representing each training example.

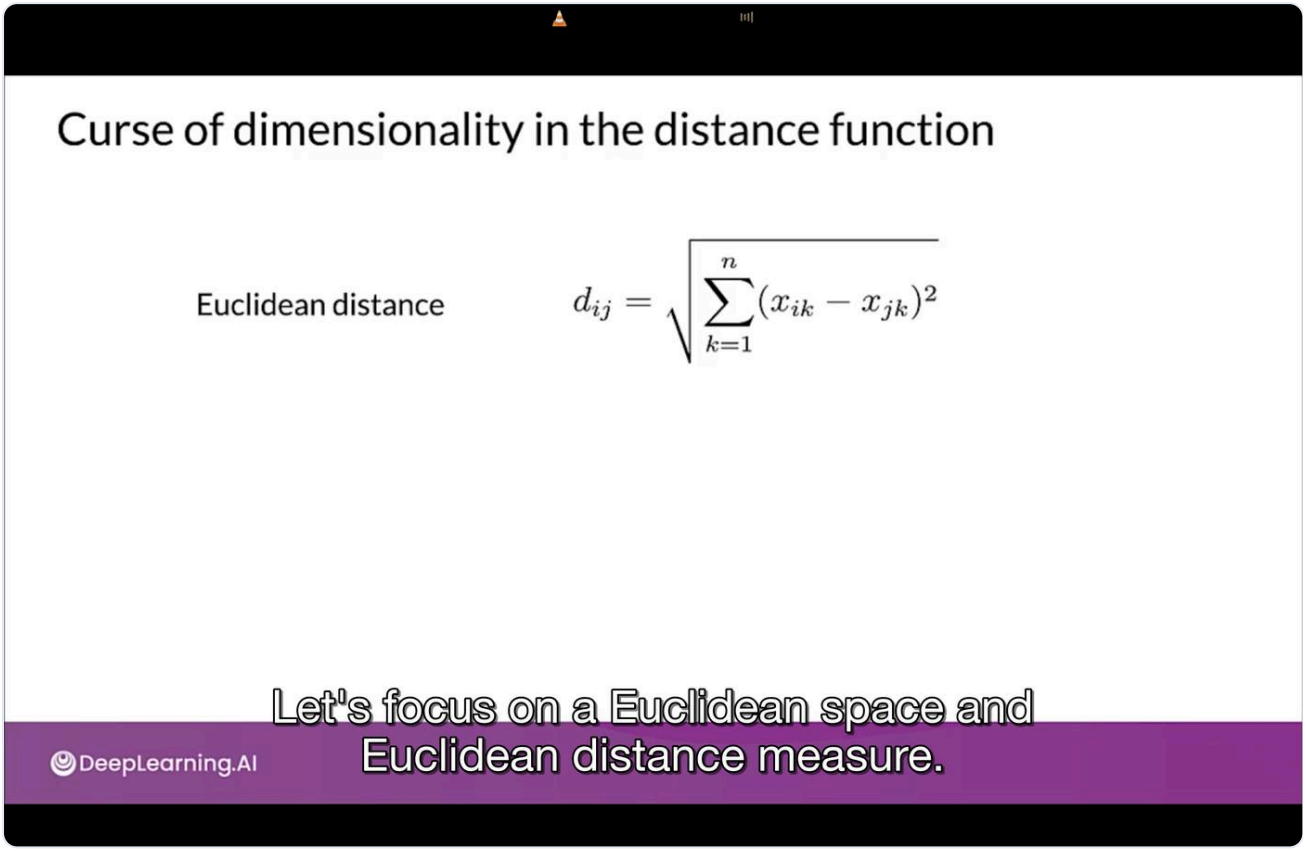

FIGURE 11.7 | Curse of Dimensionality in the Distance Function

Let's focus on a Euclidean space and Euclidean distance measure.

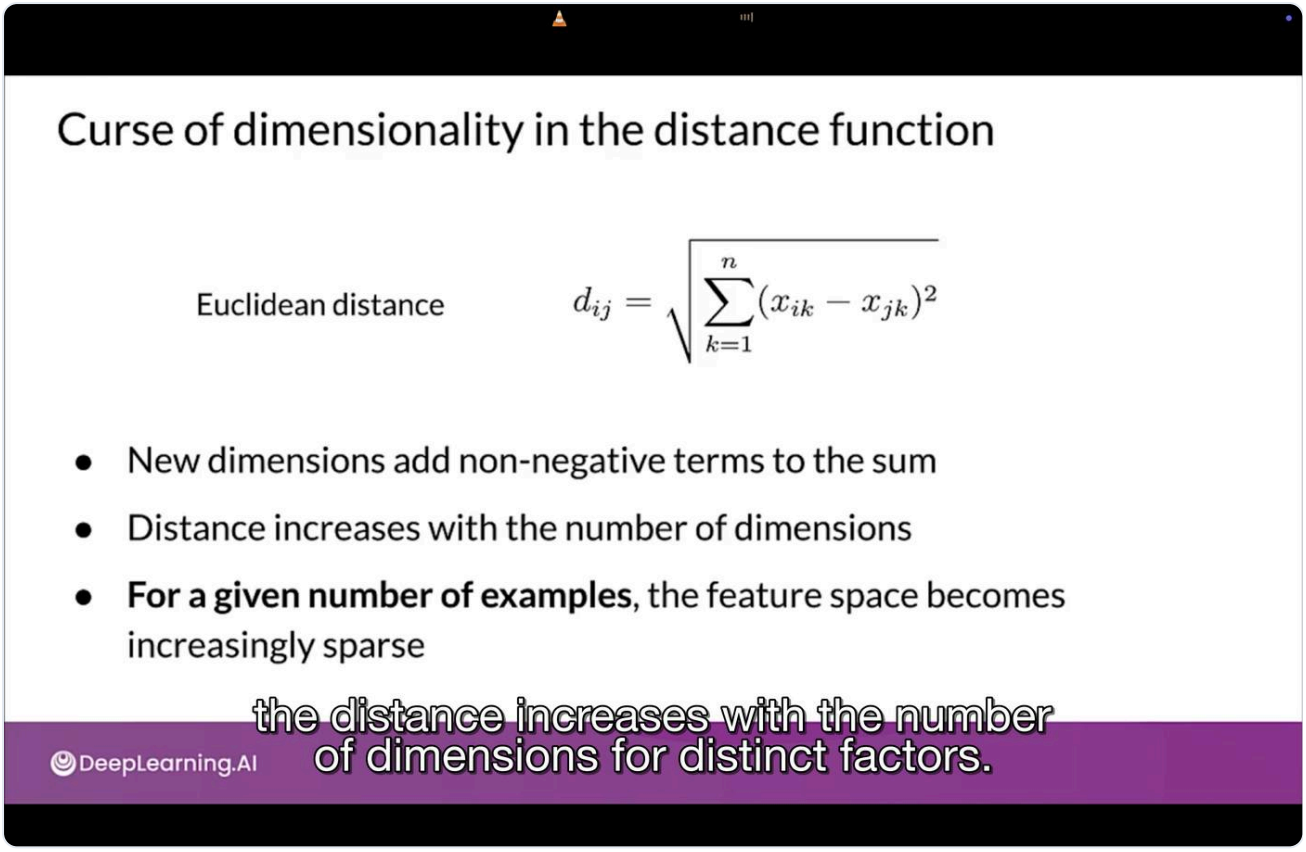

FIGURE 11.8 | Curse of Dimensionality in the Distance Function

Each new dimension adds a non-negative term to the sum, so the distance increases with the number of dimensions for distinct factors. In other words, as the number of features grows for a given number of training examples, the feature space becomes increasingly sparse with more distance between training examples. Because of that the lower data density requires more training examples to keep the average distance between data points the same. It's also important that the examples that you add are significantly different from the examples that you already have that are already present in the sample. Here the argument is built using Euclidean distance, but it is true for any properly defined distance measure.

Now, let's understand why this happens.

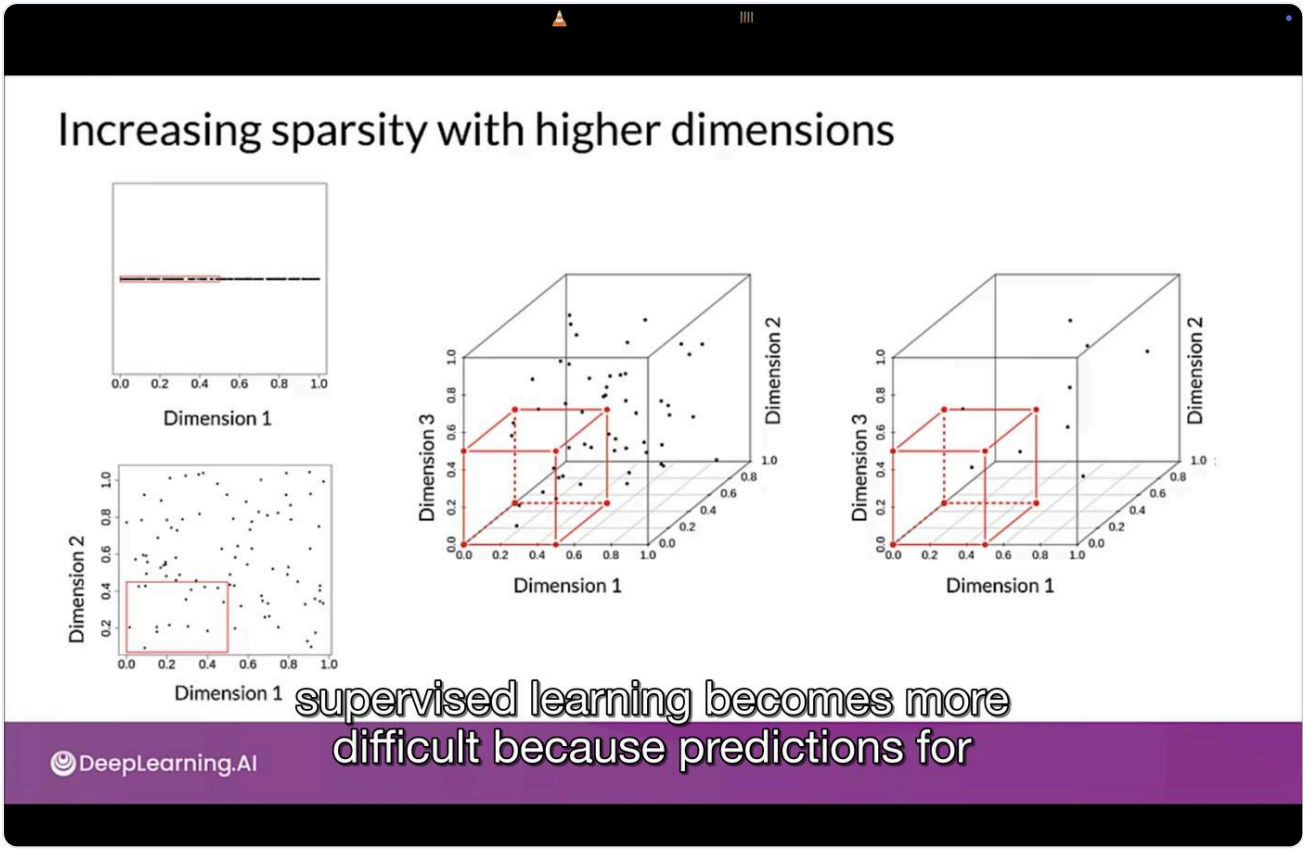

FIGURE 11.9 | Increasing Sparsity with Higher Dimensions

When the distance between observations grows, supervised learning becomes more difficult because predictions for new samples are less likely to be based on learning from similar training examples. The size of the feature space grows exponentially as the number of features increases making it much harder to generalize efficiently. The variance increases and there's a higher chance of overfitting to noise in more dimensions resulting in poor generalization performance. In practice, features can also be correlated or do not exhibit much variation. For these reasons, there is a need to reduce dimensionality. The challenge is to keep as much of the predictive information as possible using as few features as possible.

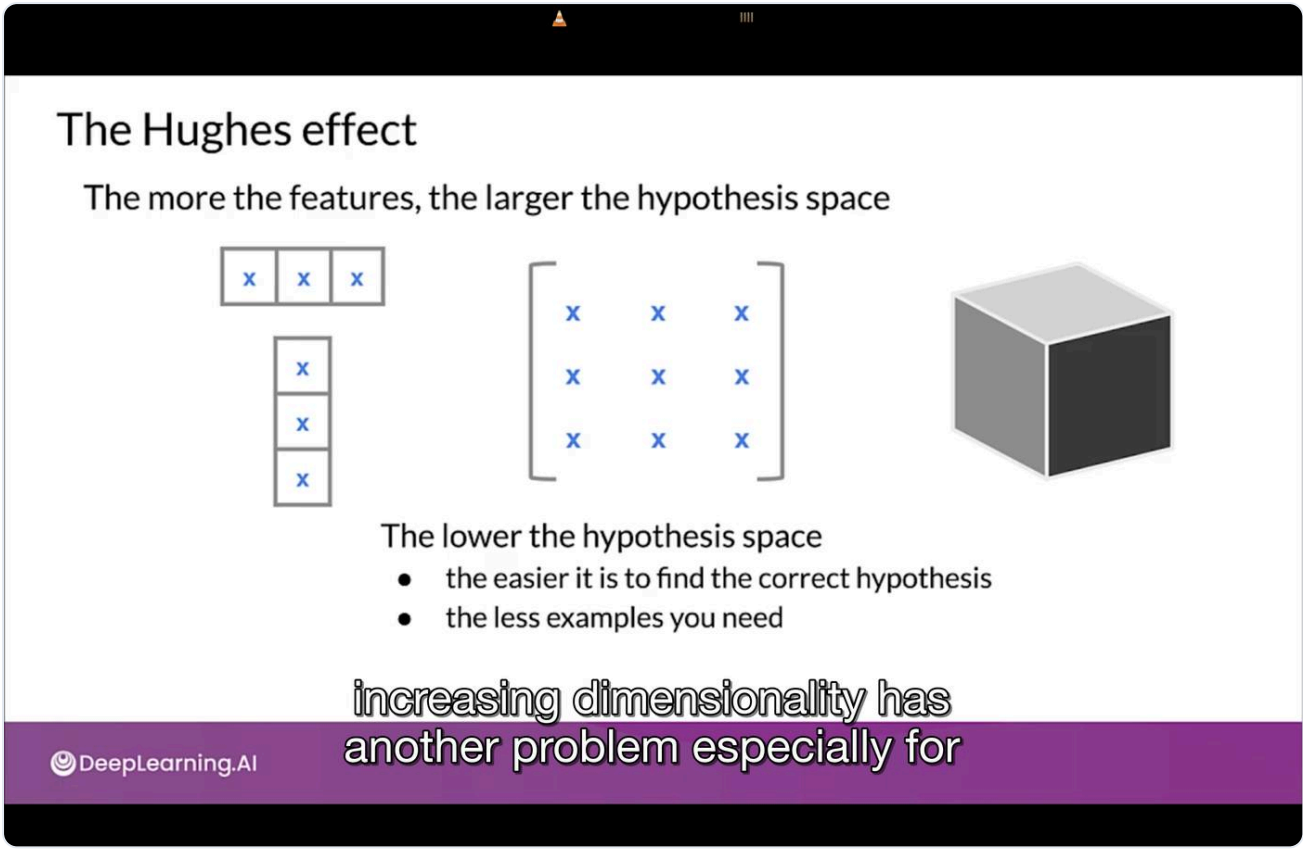

FIGURE 11.10 | The Hughes Effect

Regardless of which modeling approach you're using, increasing dimensionality has another problem especially for classification which is known as the Hughes effect. This is a phenomenon that demonstrates the improvement in classification performance as the number of features increases until we reach an optimum where we have enough features.

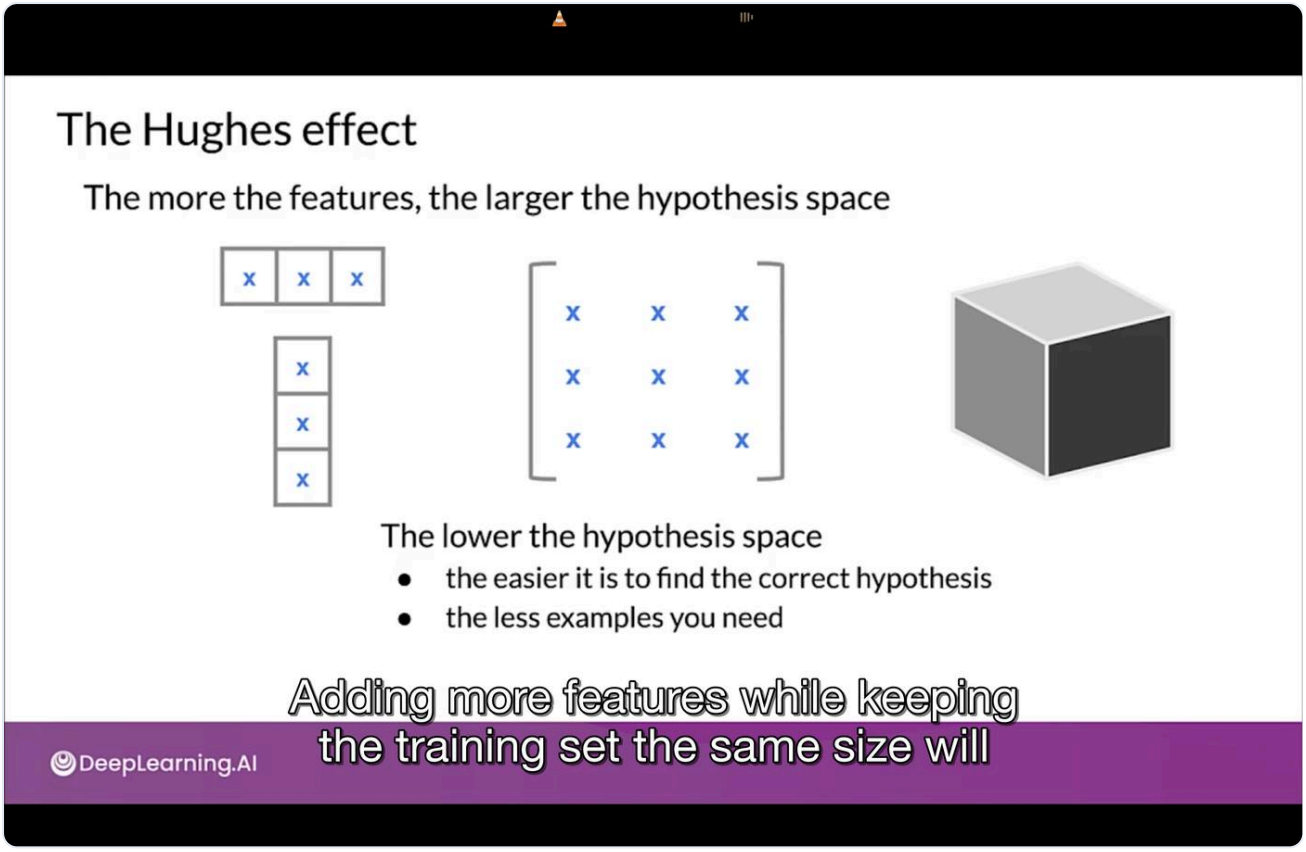

FIGURE 11.11 | The Hughes Effect

Adding more features while keeping the training set the same size will degrade the classifiers performance. We saw this earlier in our graph. In classification, the goal is to find a function that discriminates between two or more classes. You can do this by searching for hyperplanes in space that separate these categories. The more dimensions you have, the easier it is to find a hyperplane during training, but at the same time the harder it is to match that performance when generalizing to unseen data. And the less training data you have, the less sure you are that you identify the dimensions that matter for discriminating between categories.

LECTURE 12

More Features Require More Training Data

Let's look at an example of how dimensionality reduction can help our models perform better, apart from distances and volumes. Increasing the number of dimensions can create other problems. Processor and memory requirements often scale non linearly with an increase in the number of dimensions, due to an exponential rise in feasible solutions many optimization methods cannot reach a global optima and get stuck in a local optima.

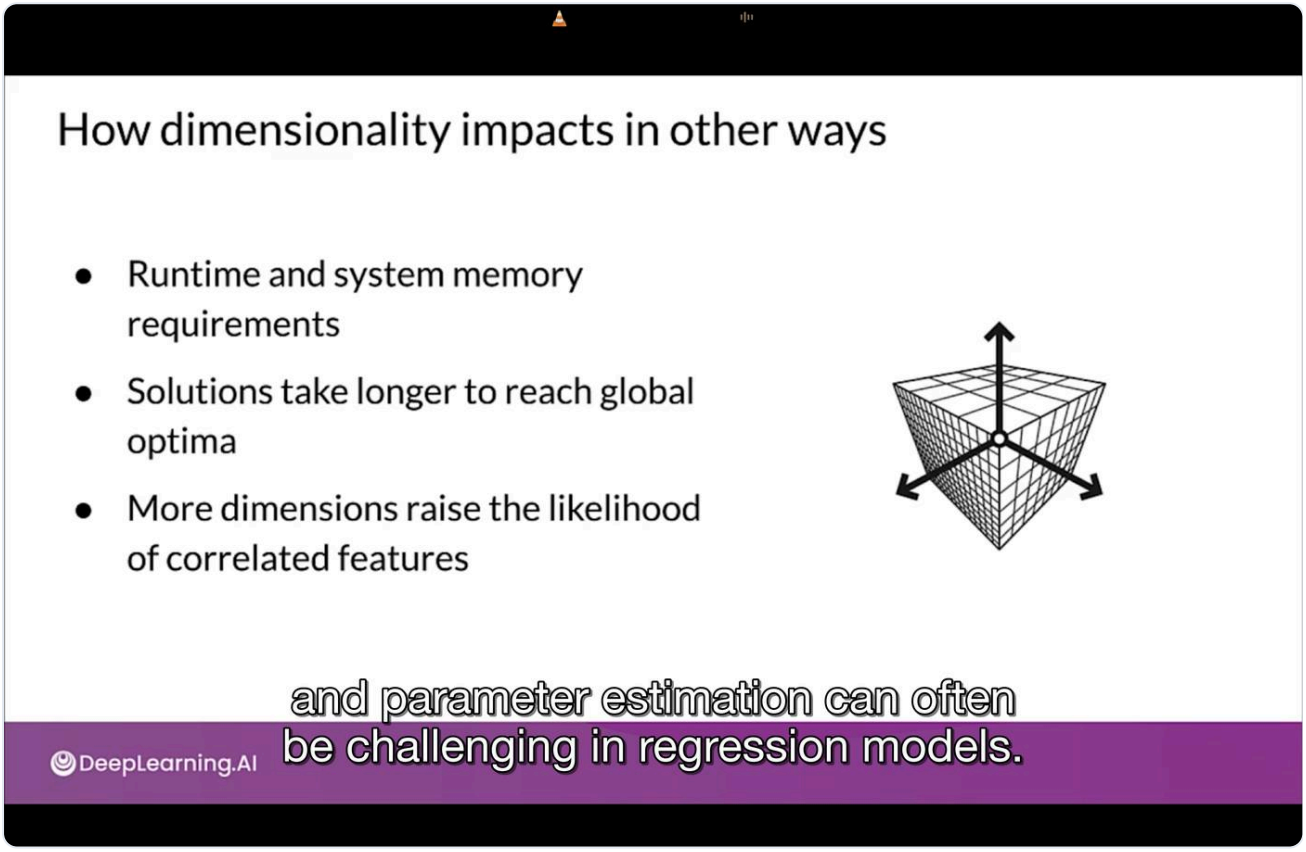

FIGURE 12.1 | How Dimensionality Impacts in Other Ways

More dimensions also often increases the likelihood of having correlated features and parameter estimation can often be challenging in regression models. So let's look at how more features can make training a model harder. With an example, when you create a model, you design it for a certain number of features or dimensions, you might be tempted to add more features to get a better model, but more features can actually hurt your model. Each feature holds information that may or may not help your model predict accurately, as you add more and more features, you need to add more and more training examples along the range of values for those features.

The amount of training data needed increases exponentially with each added feature. That means that the volume of training data increases exponentially. And we need to make sure that the training data covers the same regions of the feature space as the prediction requests that will receive, all of these can reduce the ability of your model to generalize. Well, along with this, the number of trainable variables in the model also increases.

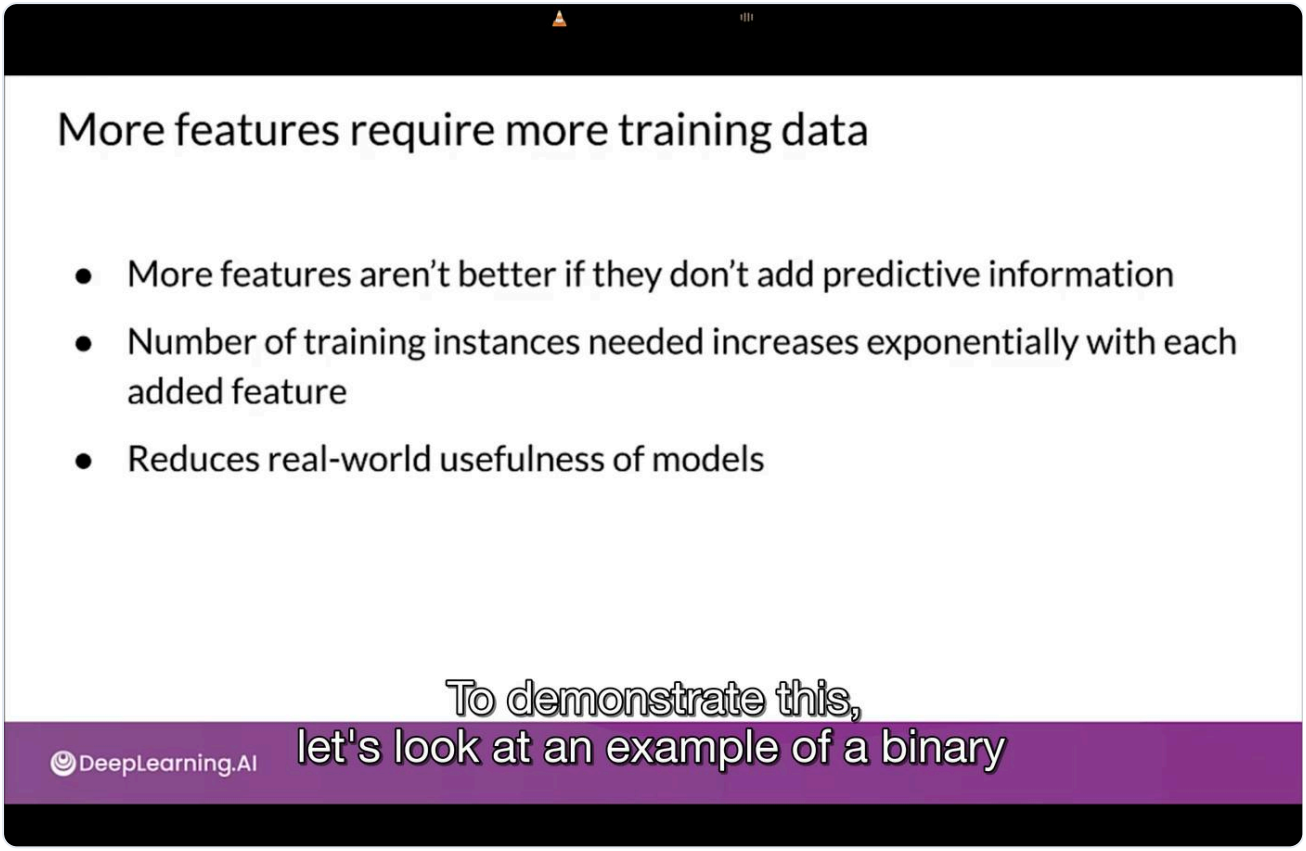

FIGURE 12.2 | More Features Require More Training Data

To demonstrate this, let's look at an example of a binary classification model for the Cleveland heart disease data set when a single additional feature is added. Let's start by creating a structured classification model for the Cleveland heart disease data set.

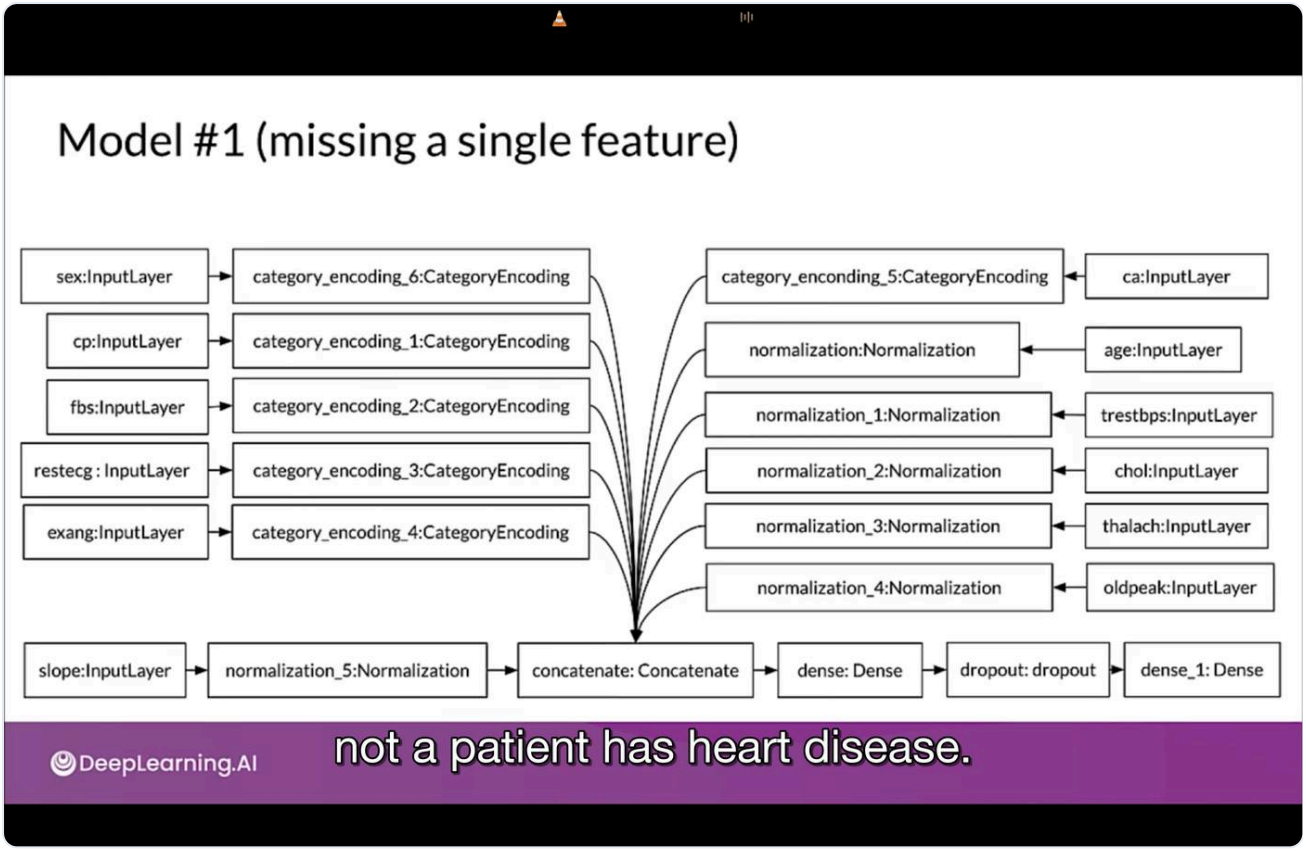

FIGURE 12.3

This dataset has 14 features to predict whether or not a patient has heart disease. This is a binary classification problem. This first model omits one of the original features called foul, go look at this and how adding a feature to the dataset impacts the number of trainable variables in the model.

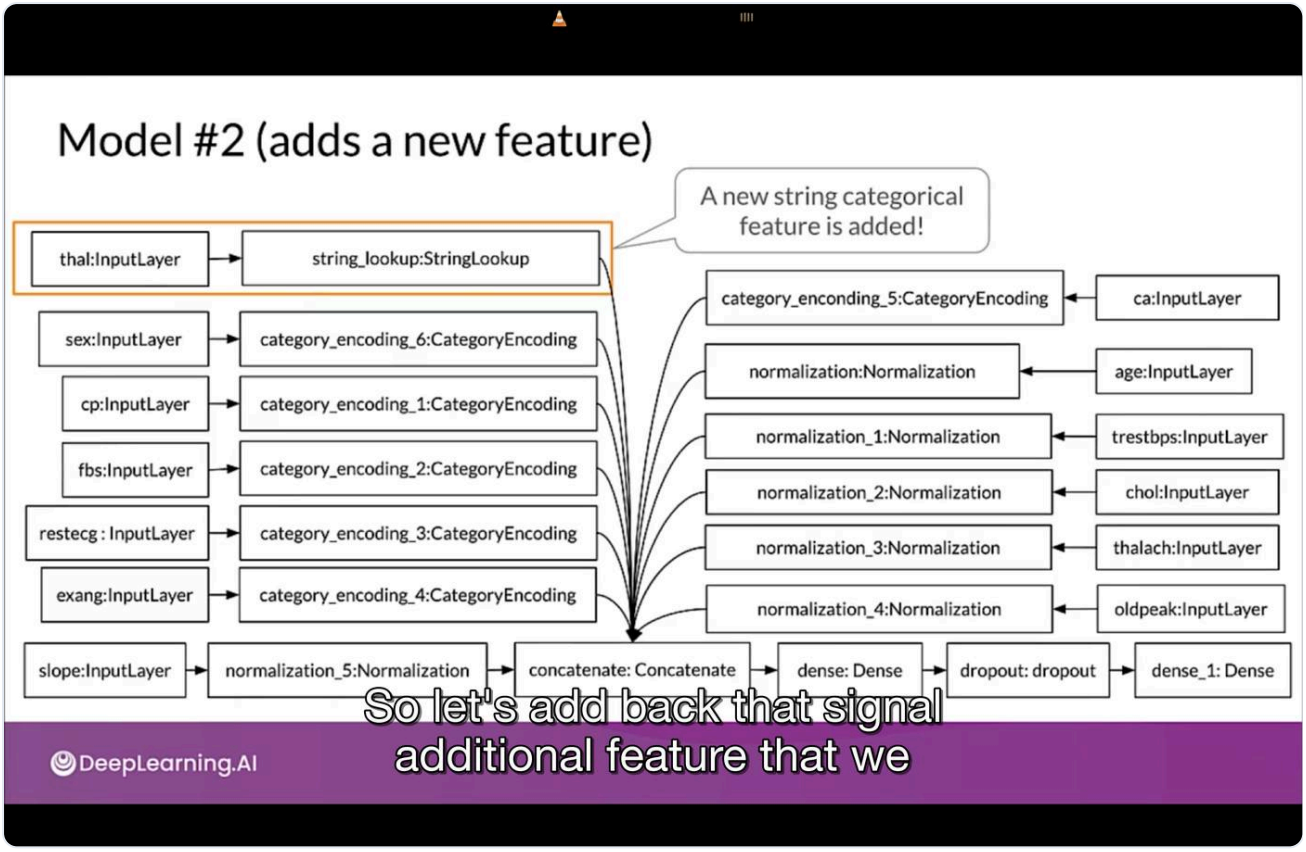

FIGURE 12.4

So let's add back that signal additional feature that we removed in the first model. For this let's encode this as a categorical string feature using Keras preprocessing layers which is available in tensorflow. Next, let's see how adding it impacts your original model in terms of the number of trainable parameters.

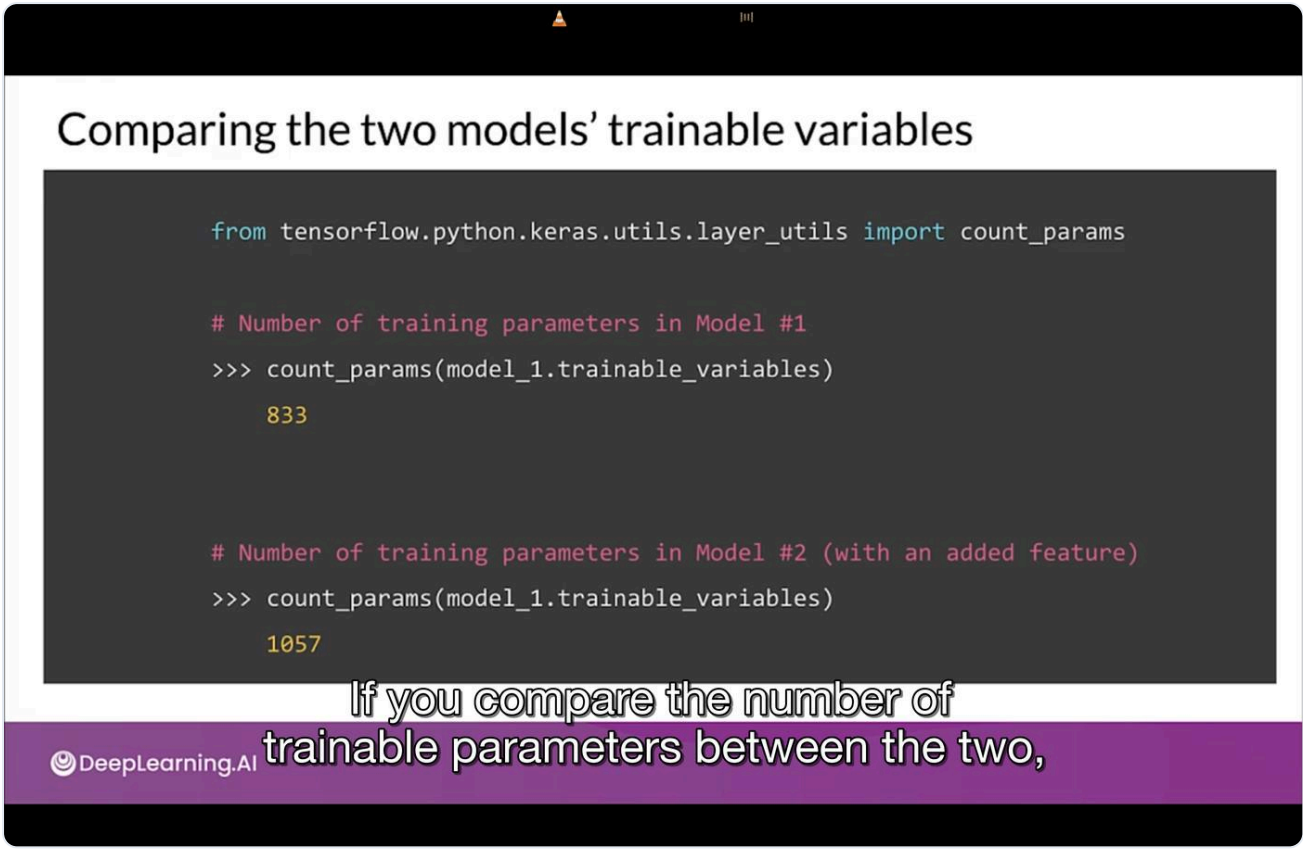

FIGURE 12.5

If you compare the number of trainable parameters between the two, even adding only one feature results in a 27% increase. That's a considerable growth in the number of parameters to say the least. That will make training slower and more expensive. You'll also need to increase the size of the training data set which will make the training even slower and more expensive. The main point in this section is that when data dimensionality becomes too large, the performance of a classifier decreases and the demand for resources increases. The question, that is, what does too large really mean? Unfortunately, there is no fixed rule for how many features should be used in a machine learning problem.

In fact, this depends on the amount of training data available, the variance in that data, the complexity of the decision surface and the type of classify are used. It also depends on which features actually contain predictive information that will help your model train.

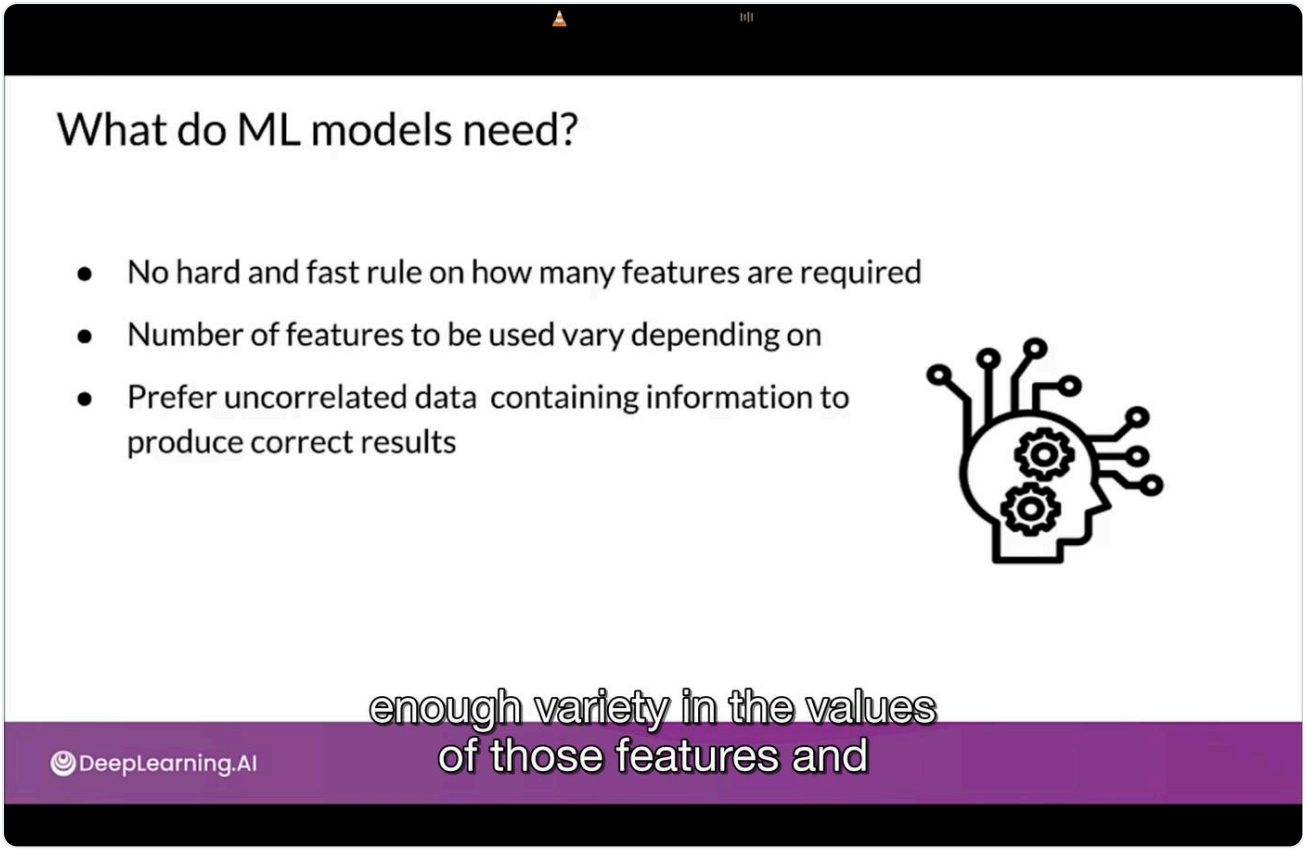

FIGURE 12.6 | What Do ML Models Need?

You want enough data with the best features and enough variety in the values of those features and enough predictive information in those features to maximize the performance of your model while simplifying it as much as possible.

LECTURE 13

Why Reduce Dimensionality?

Now that we've discussed how your data has an impact on the performance of your models and the resources required to train and serve those models. Let's look at some manual techniques for doing dimensionality reduction. When pre processing a set of features to create a new feature set, it's important to retain as much predictive information as possible, without predictive information all the data in the world won't help your model learn. Features must be representative of the predictive information in the data set. This information also needs to be in a form that will help your model learn. While some inherent features can be obtained directly from raw data, you often need derived features, normalized, engineered or embedded features.

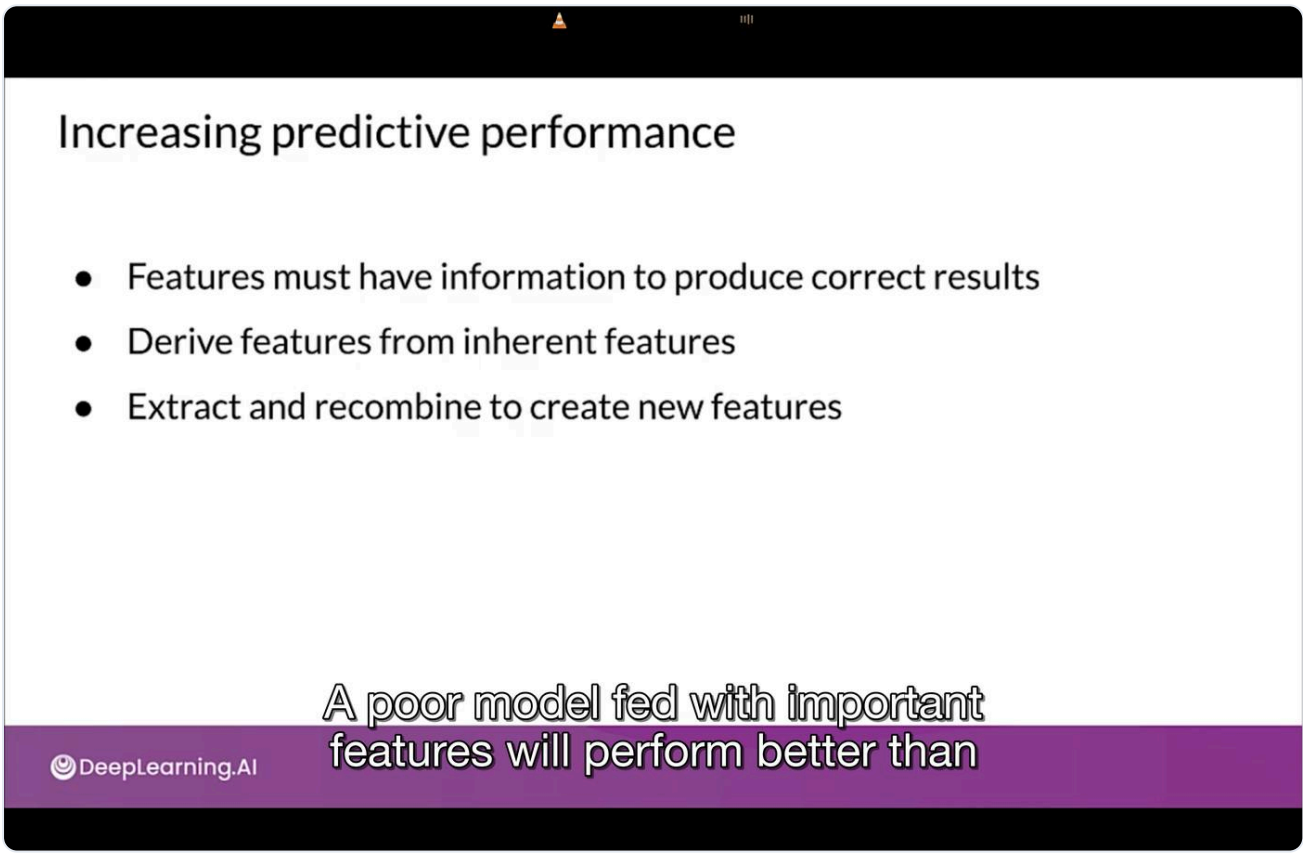

FIGURE 13.1

A poor model fed with important features will perform better than a fantastic model fed with low quality or bad features. Many domains involve huge numbers of features and dimensions. Often the first pick of features is an expression of domain knowledge, that can often result in more features than we really need or want.

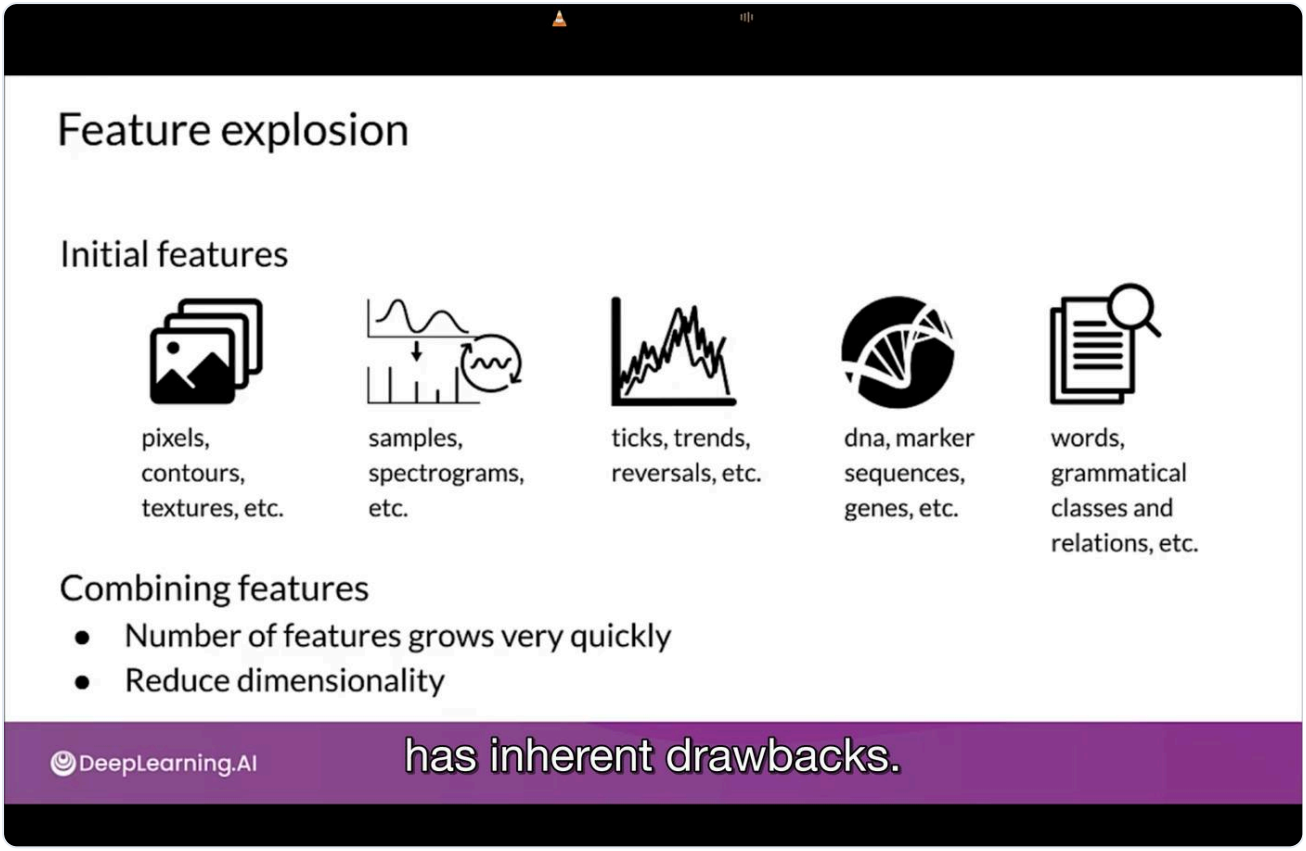

FIGURE 13.2

As we saw in our discussion of the curse of dimensionality this has inherent drawbacks. This means that you need to reduce dimensionality, or more precisely, the number of features you're including in your dataset while retaining or improving the amount of predictive information contained in the data. Dimensionality reduction looks for patterns and data and uses these patterns to re express the data in a lower dimensional form. This makes computation much more efficient, which can be a significant factor in a world of big models and big data sets. However, dimensionality reductions most essential function is to reduce the data set to its bare bones, discarding noisy features that cause significant problems for supervised learning tasks like regression and classification.

In many real world applications it is the dimensionality reduction that makes predictions possible.

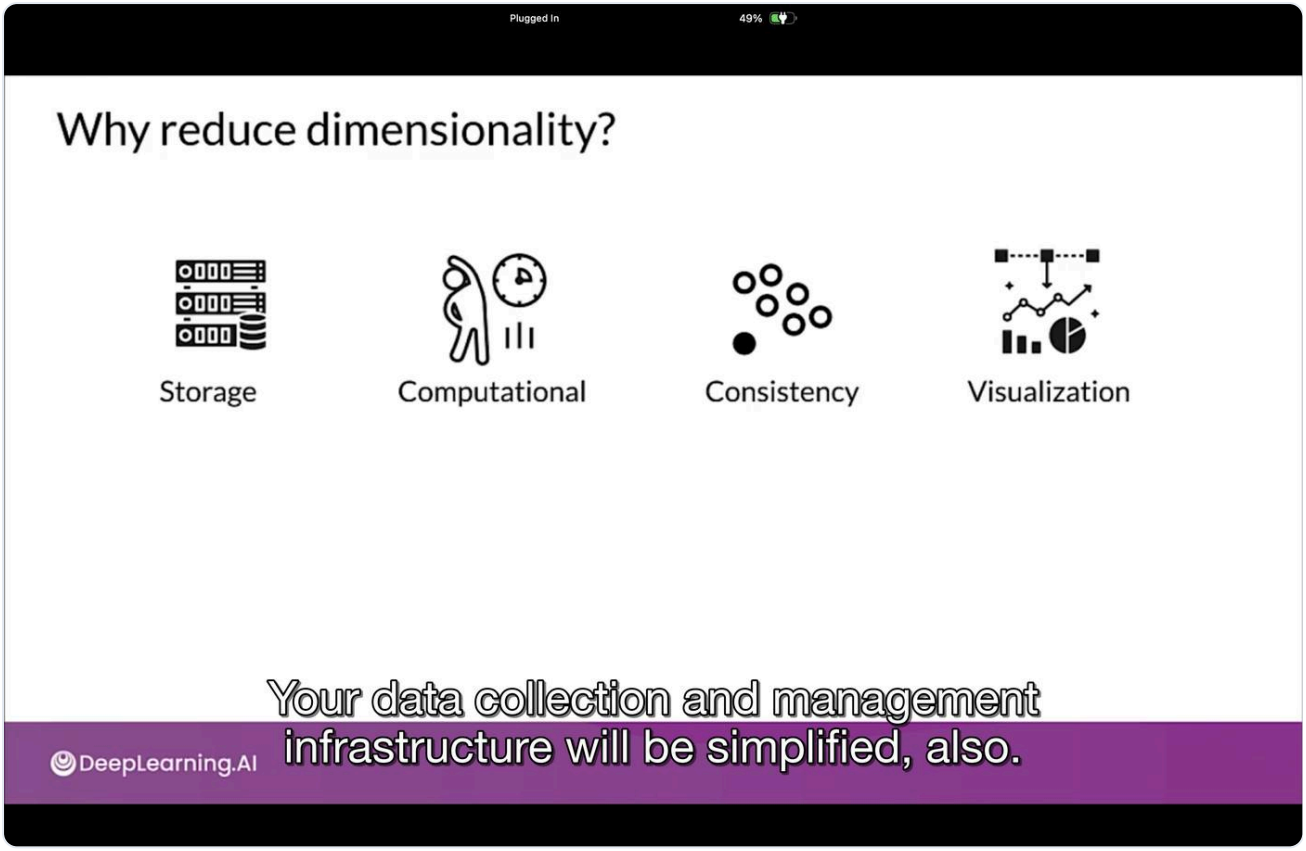

FIGURE 13.3 | Why Reduce Dimensionality?

Your data collection and management infrastructure will be simplified, also. Another factor to consider is that some algorithms do not perform well when we have large dimensions. It also reduces multicollinearity by removing redundant features, it helps when we're trying to visualize the data. And as we discussed earlier, it isn't easy to visualize data in higher dimensions, so reducing our space to 2D or 3D may help us to plot and observe patterns more clearly.

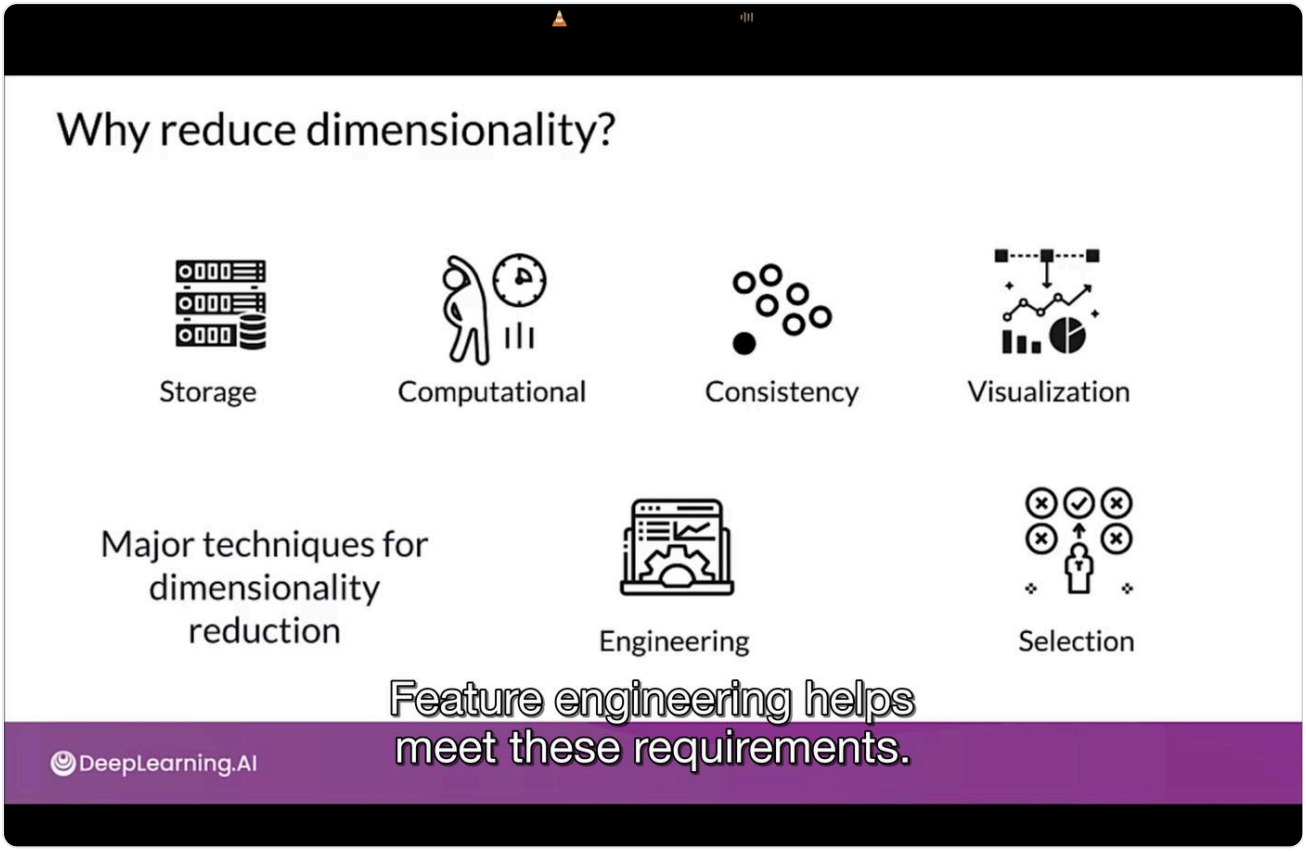

FIGURE 13.4 | Why Reduce Dimensionality?

Feature engineering helps meet these requirements. It builds valuable information from raw data by reformatting, combining and transforming primary features into new ones until it yields a new set of data that results in a better model. In addition, features selection examines a set of potential features, select some of them and discards the rest. Feature selection is applied either to prevent redundancy and or to remove irrelevancy in the original features or just get to a limited number of features to avoid issues. Since we've already seen various feature selection techniques, let's look instead at feature engineering. The best results come down to you, the practitioner crafting the features.

This is one of the areas where machine learning engineering is somewhat of an art form. Feature importance and feature selection can help inform you about the objective utility of features, but those features have to come from somewhere. You often need to manually create them. This requires spending a lot of time with the actual sample data and thinking about the underlying form of the problem, the structures in the data and how best to express them for predictive modeling algorithms. With tabular data It often means a mixture of aggregating and or combining features to create new features and decomposing or splitting features to also create new features.

With textual data, it often means devising document or content specific indicators relevant to the problem. With image data, it can often mean using filters to pick out relevant structures like pixels, contours, shapes and textures.

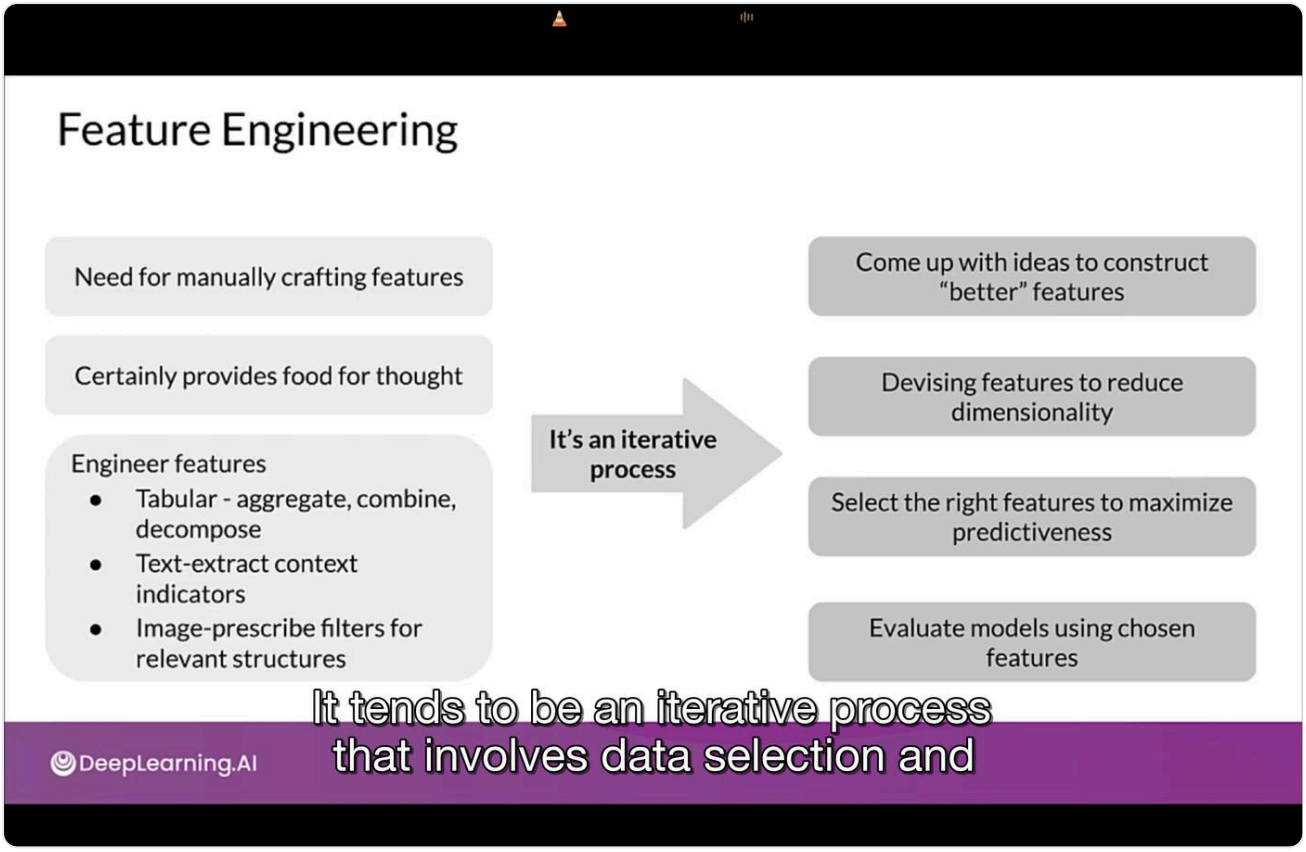

FIGURE 13.5 | Feature Engineering

It tends to be an iterative process that involves data selection and model evaluation again and again. The process usually starts with brainstorming features. Here you really get into the problem, look at a lot of data, study feature engineering on other problems and see what you can learn. Then you move on to devising new features. It depends on your problem, but you may use automatic feature extraction, manual feature engineering or a mixture of the two. Next you pick the right features using feature important scoring and feature selection methods to prepare one or more views of your data. Finally, you measure the model's performance on unseen data using the chosen features.

LECTURE 14

Bucketizing and Feature Crossing

Now that you've seen that the dimensionality of your data has an impact on the performance of your models and the resources required to train and serve those models. This is even more important when dealing with resource constraints scenarios such as mobile deployments. So you need to carefully manage the dimensionality of your data, which often means looking for ways to reduce it. Let's look now at some manual techniques for doing dimensionality reduction.

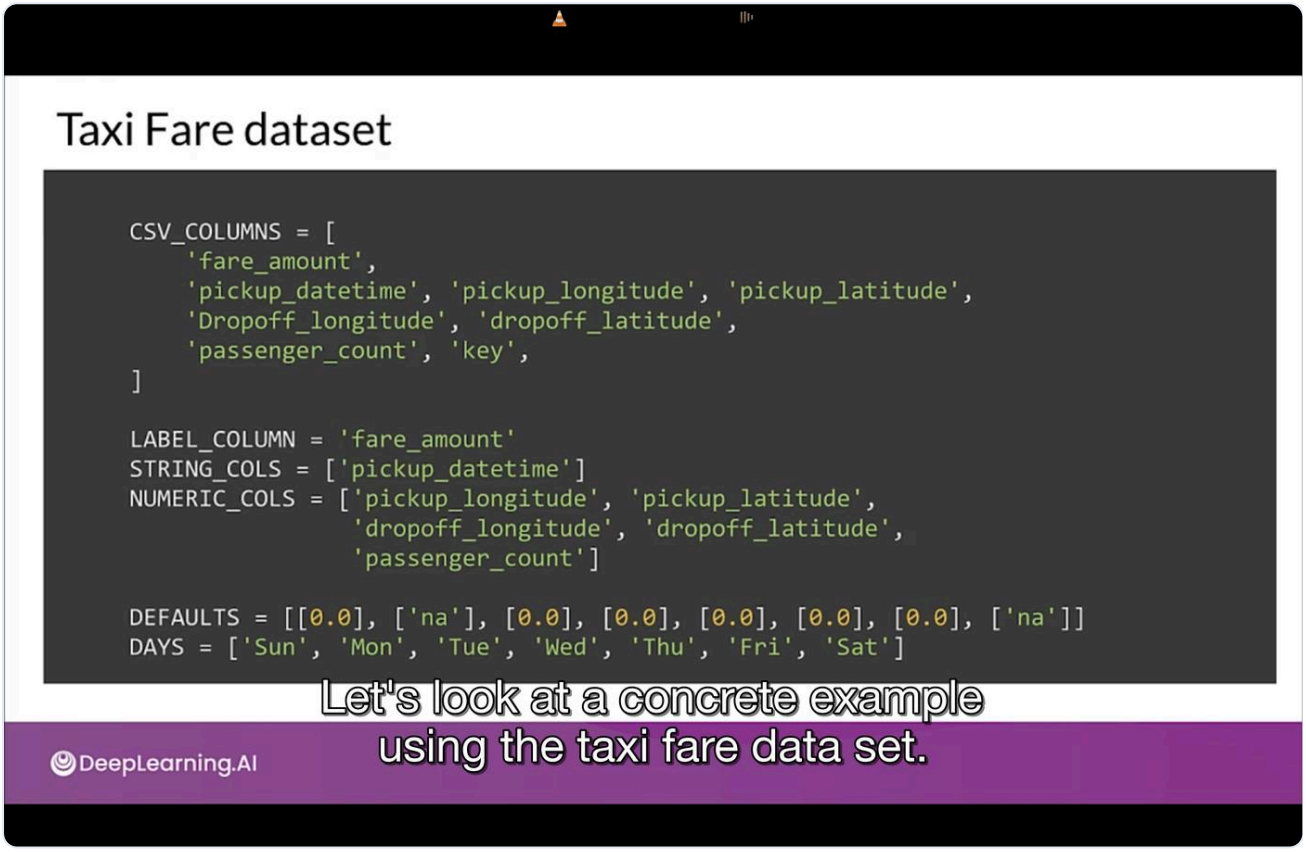

FIGURE 14.1

Let's look at a concrete example using the taxi fare data set. The data set consists of 106,545 taxi rides. The objective is to predict the fair amount of each ride based on a variety of features such as time and location of pickup, time travel and distance, number of passengers etcetera. As usual, the first steps are downloading the data set which is in CSV format separating the variables into string and numeric types and defining useful constants and parameters.

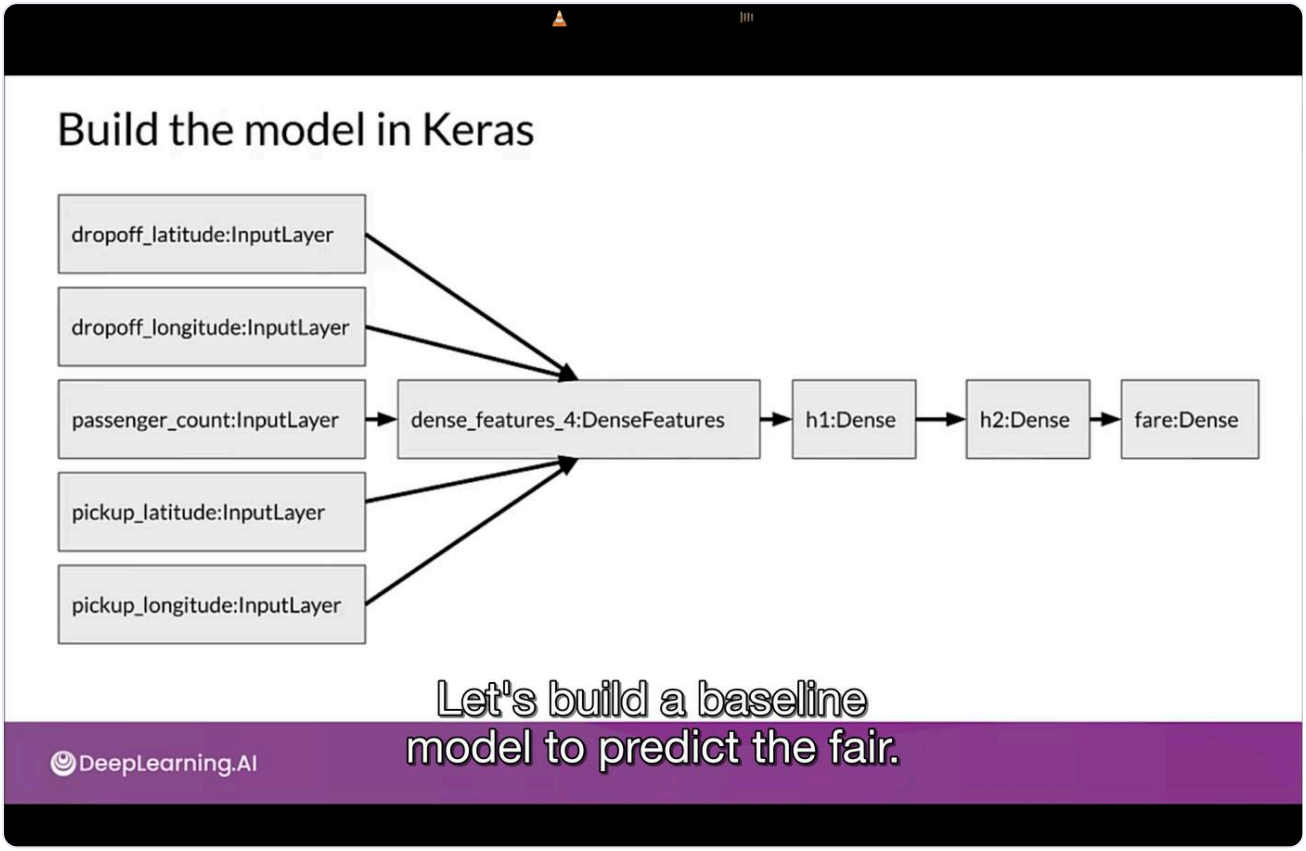

FIGURE 14.2 | Build the Model in Keras

Let's build a baseline model to predict the fair. We'll try using these features. Drop off, latitude, drop off longitude passenger count, pick up latitude and pick up longitude. The network consists of a concatenation of dense hidden layers with the last one producing a fair prediction output.

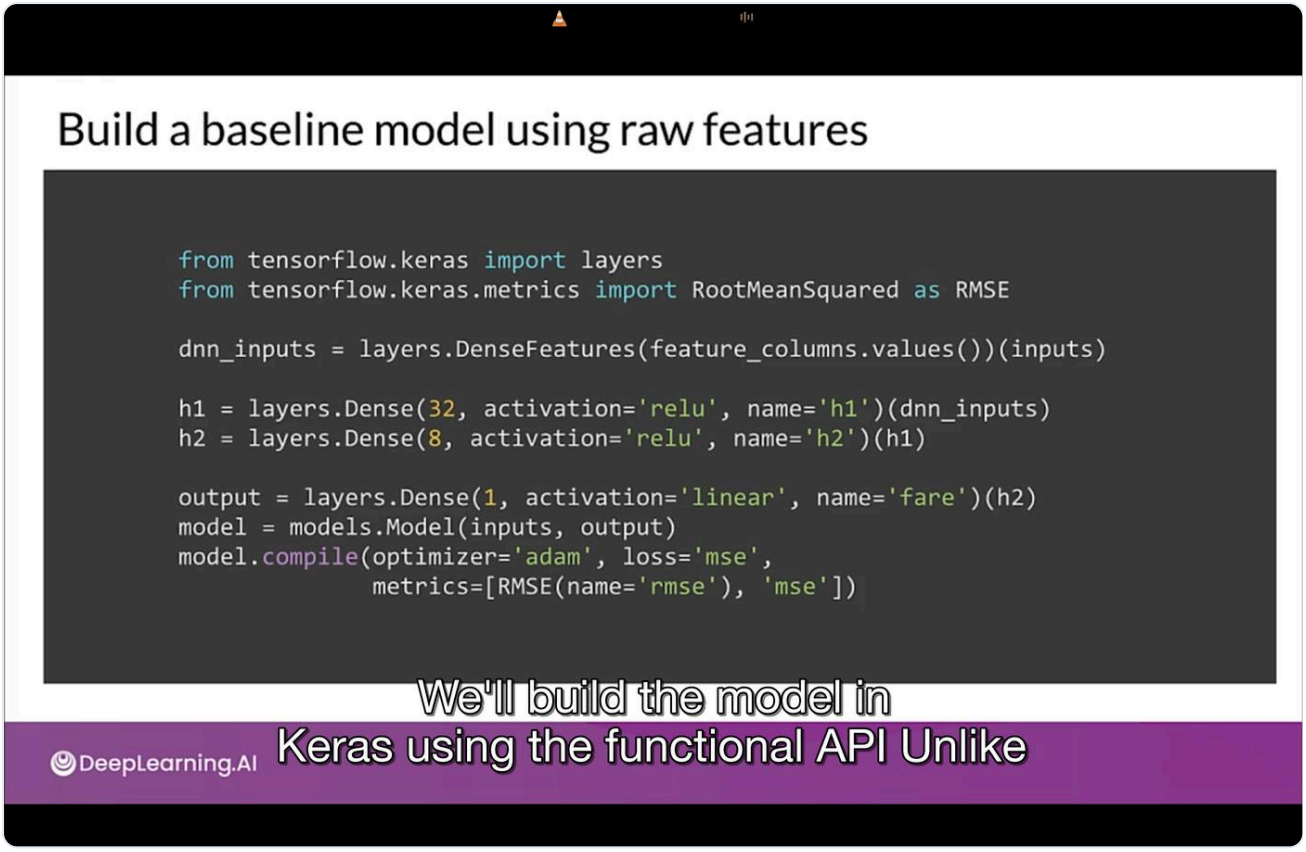

FIGURE 14.3 | Build a Baseline Model Using Raw Features

We'll build the model in Keras using the functional API Unlike the sequential API you'll need to specify the input and hidden layers. Note that you're creating a linear regression baseline model with no feature engineering as a quick reminder, a baseline model is a naive implementation that helps with setting expectations for model performance. After setting up the model for training and creating the data sets, you're ready to train the baseline model. To train the model simply call model dot fit.

FIGURE 14.4 | Train the Model

Let's look at training and validation performance using the root mean squared error loss over the trading epics, ideally you want the validation RMSE to be close to the training set. Now, let's try to improve the model.

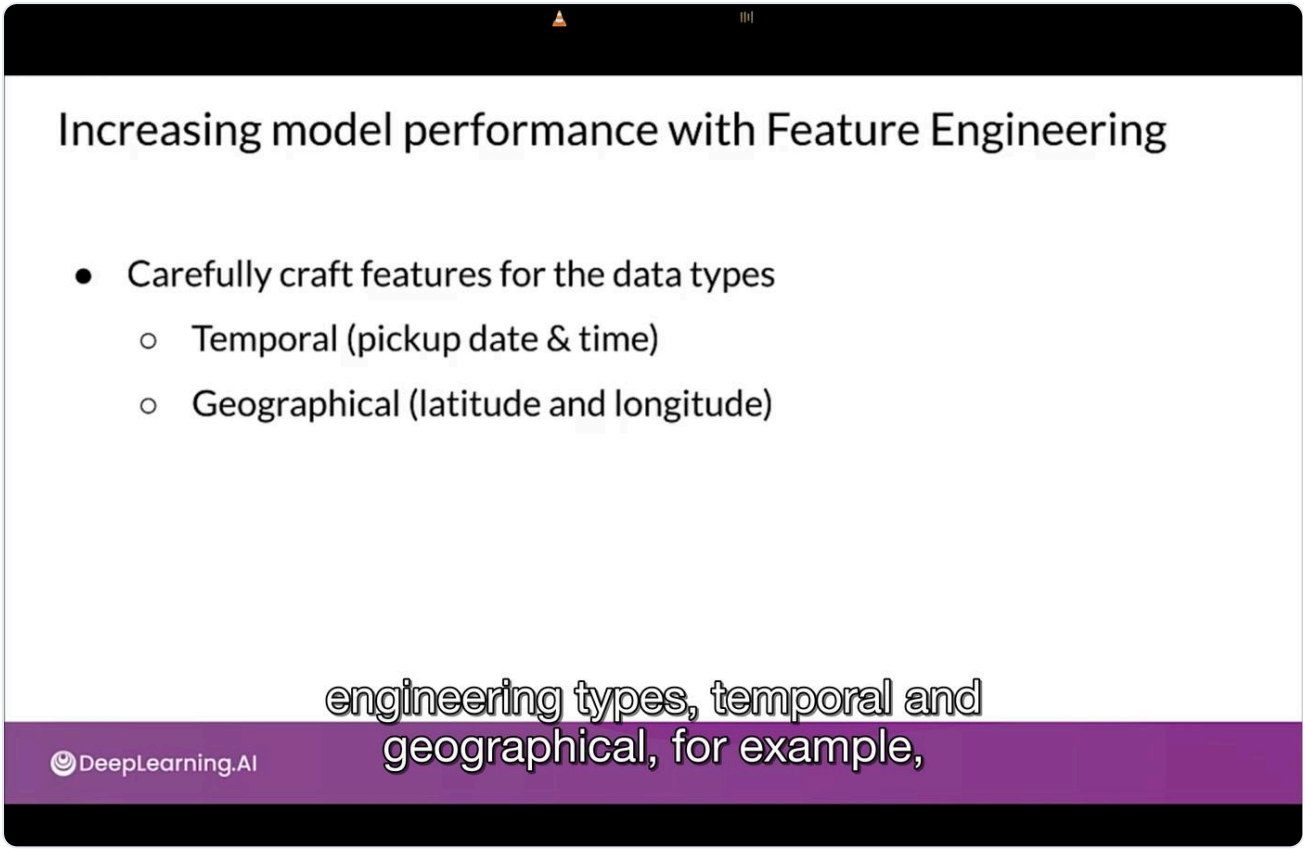

FIGURE 14.5 | Increasing Model Performance with Feature Engineering

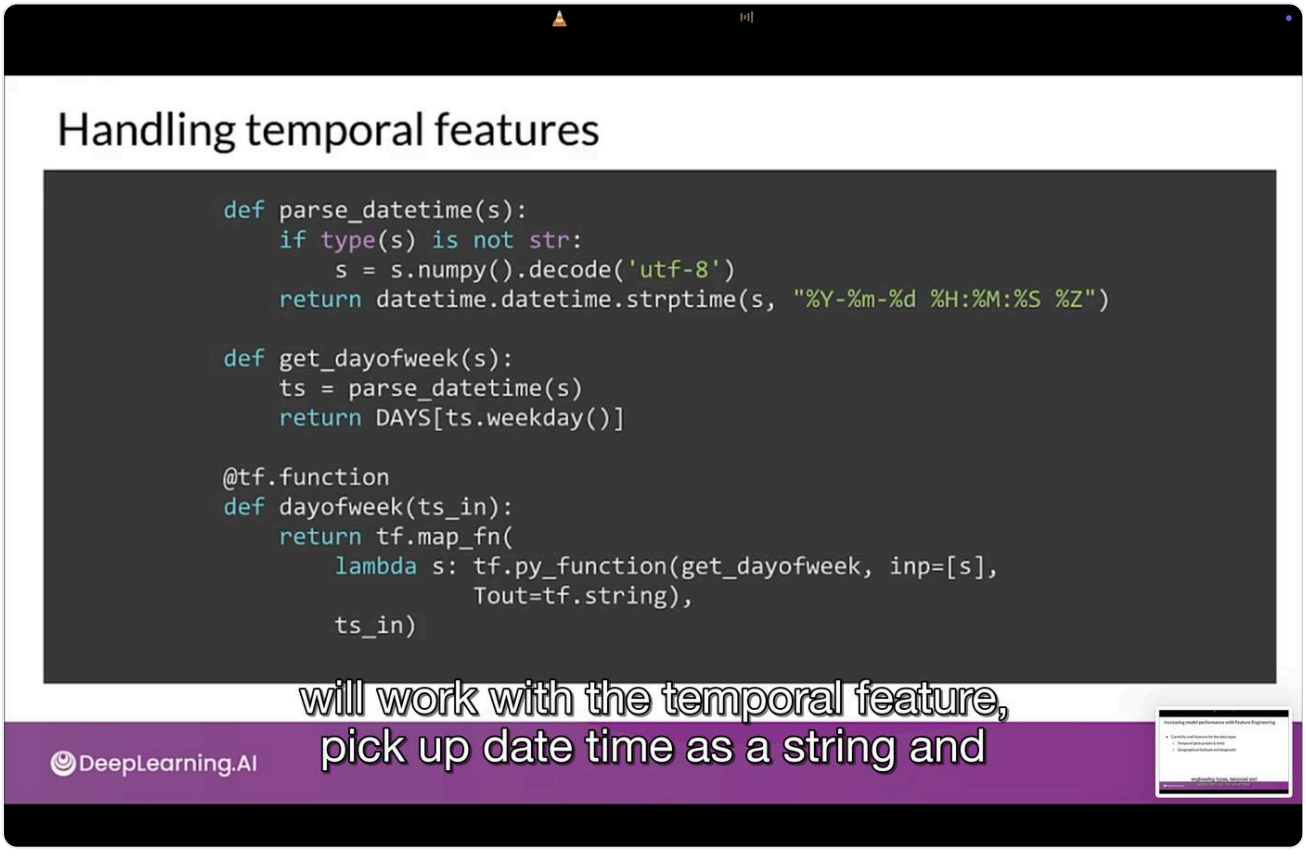

FIGURE 14.6

To improve the model's performance, let's create two new feature, engineering types, temporal and geographical, for example, will work with the temporal feature, pick up date time as a string and we will need to handle this within the model. First, you'll include pickup date time as a feature and then you'll need to modify the model to handle it as a string feature.

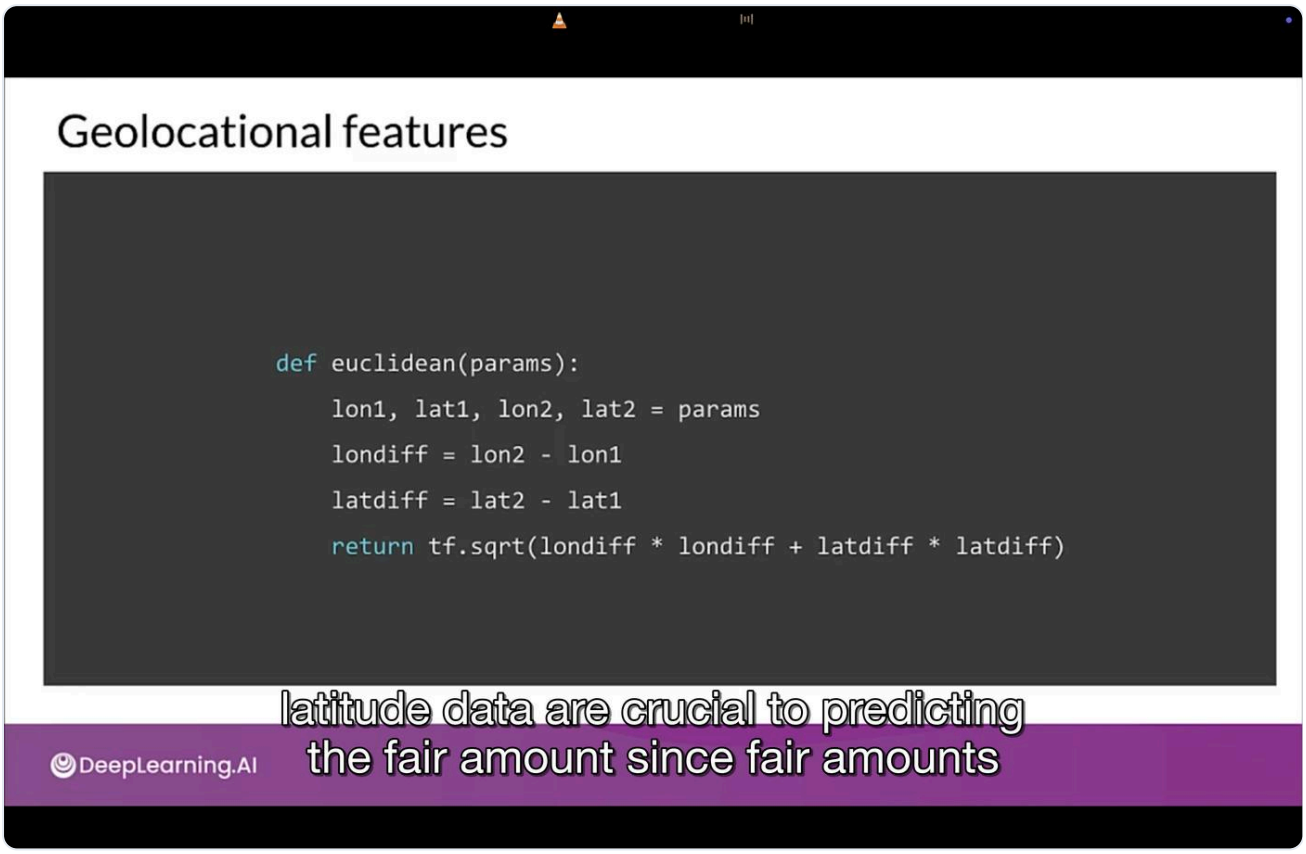

FIGURE 14.7

The pickup or drop off longitude and latitude data are crucial to predicting the fair amount since fair amounts in new york city taxis are largely determined by the distance traveled. As such we need to teach the model of the Euclidean distance between the pickup and drop off points. Recall that latitude and longitude allows us to specify any location on Earth using a set of coordinates. The dataset contains information regarding the pickup and drop off coordinates. However, there is no information regarding the distance between the pickup and drop off points. So let's create a new feature that calculates the distance between each pair of pick up and drop off points.

You can do this using the Euclidean distance, which is a straight line distance between any two coordinate points, but note that this will only be a rough indicator of the actual driving distance. It's very important for numerical variables to get scaled before they're fed into the neural network. Let's use min max scaling, also called normalization on the geo location features. Later in our model, you'll see that these values are shifted and re scaled so they end up ranging from 0 to 1. We'll use domain knowledge to create a function named scale longitude where you pass all the longitude values and add 78 to each value. Note that are scaling longitude values range from negative 72 negative 78.

So the value 78 is the maximum longitudinal value. The difference or delta between negative 70 and negative 78 is eight. The function adds 78 to each longitudinal value and then divides by eight to return a scaled value.

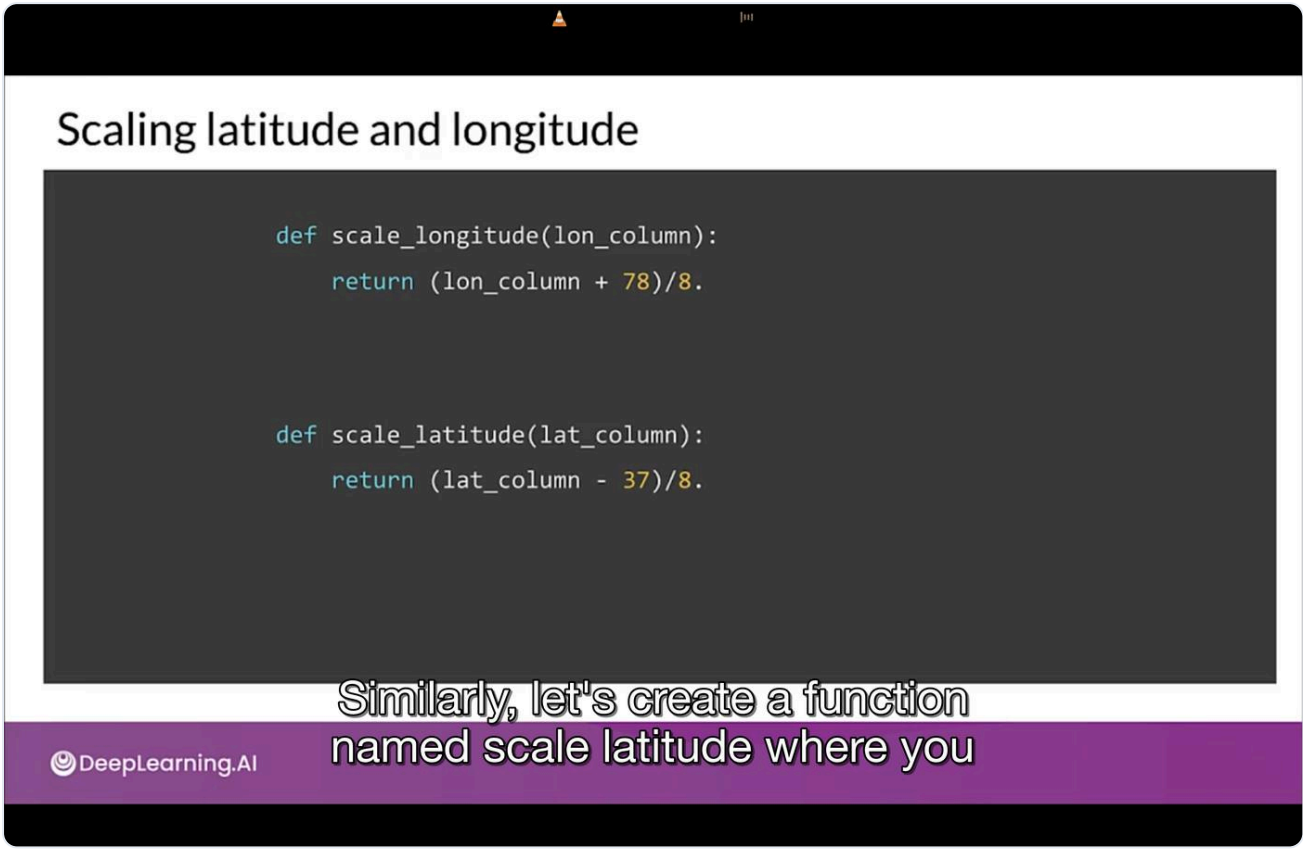

FIGURE 14.8 | Scaling Latitude and Longitude

Similarly, let's create a function named scale latitude where you pass in all the latitudinal values and subtract 37 from each value. Note that are scaling latitude range is from 37 to 45. Thus the value 37 is the minimal latitudinal value. The delta or difference between negative 37 and negative 45 also happens to be eight. The function that subtracts 37 from each latitudinal value, and then divides by eight to return a scaled value. Next let's create a geo transform function.

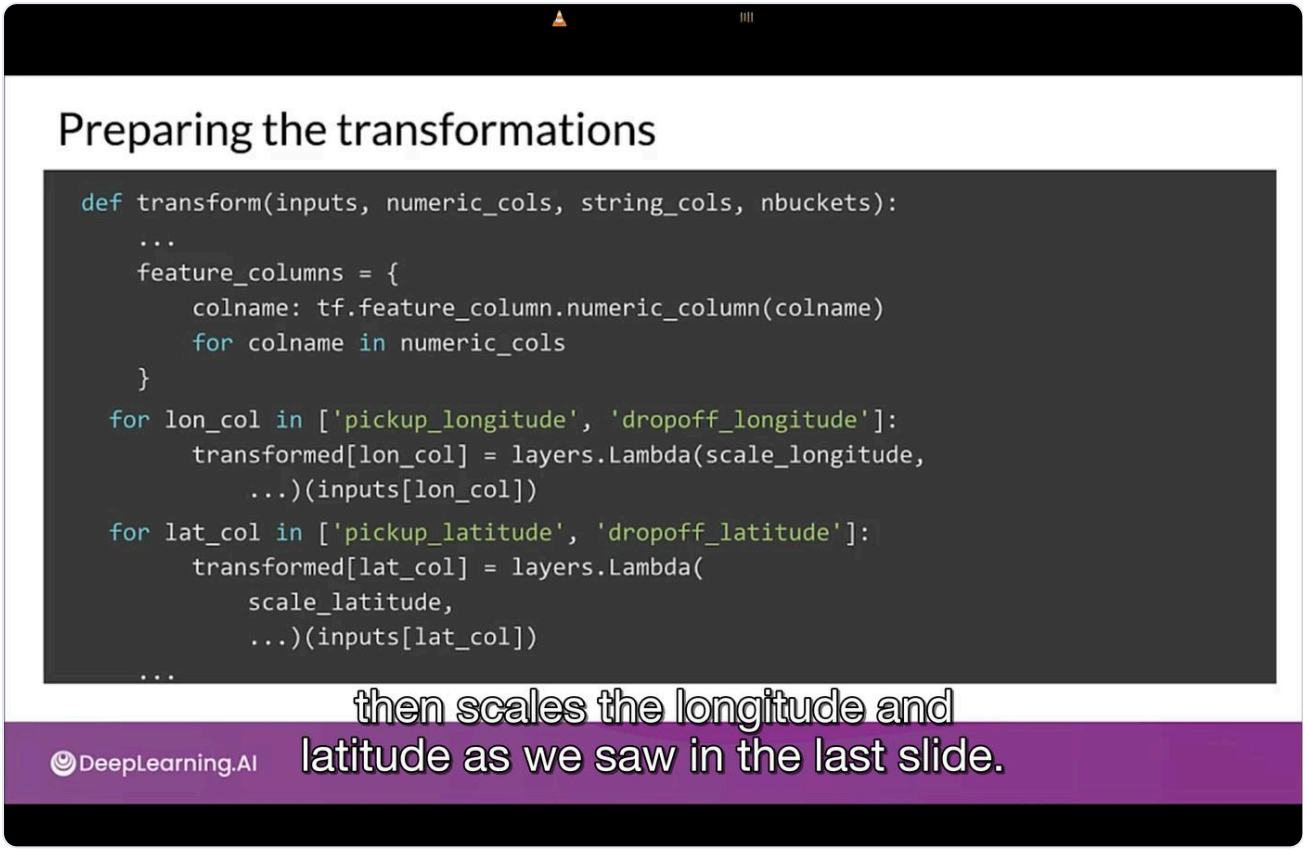

FIGURE 14.9

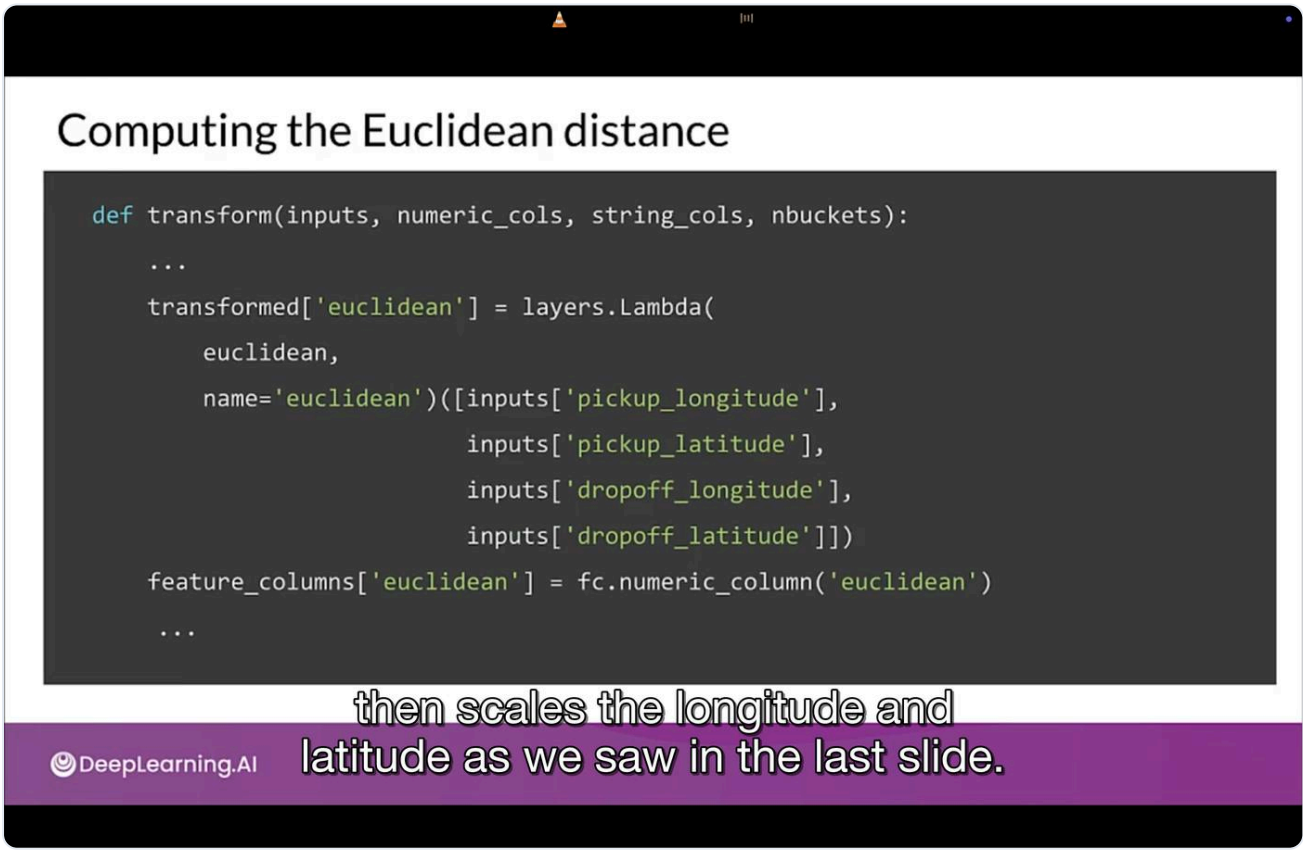

FIGURE 14.10 | Computing the Euclidean Distance

This function passes in your numerical and string column features as inputs to the model and then scales the longitude and latitude as we saw in the last slide. Then we compute the Euclidean distance based on the geo location parameters. Unless the specific geometry of the earth is relevant to your data, a bucketized version of the map is likely to be more useful than the raw inputs. This requires bucketizing the dimensions of latitude and longitude separately and then cross them effectively doing a two dimensional bucketizing of location data.

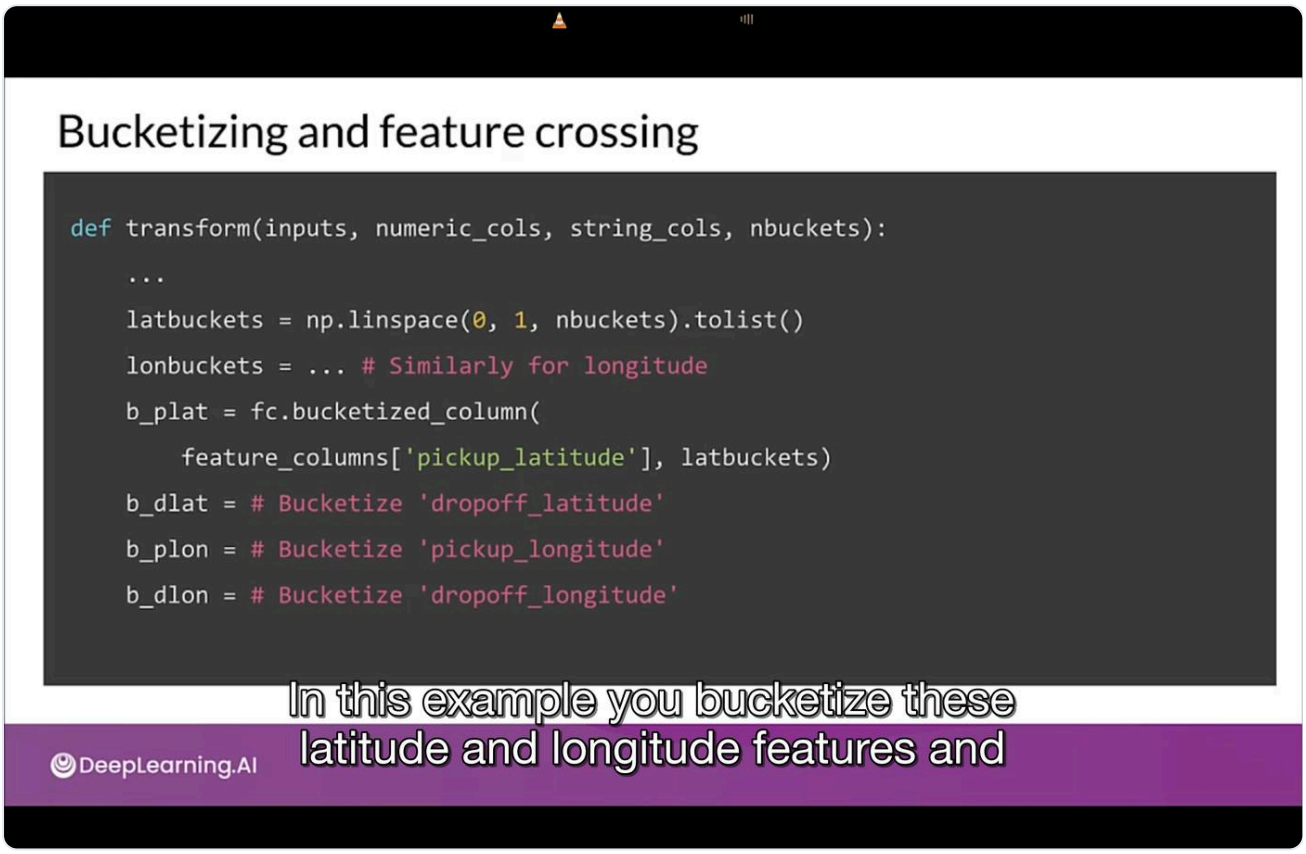

FIGURE 14.11 | Bucketizing and Feature Crossing

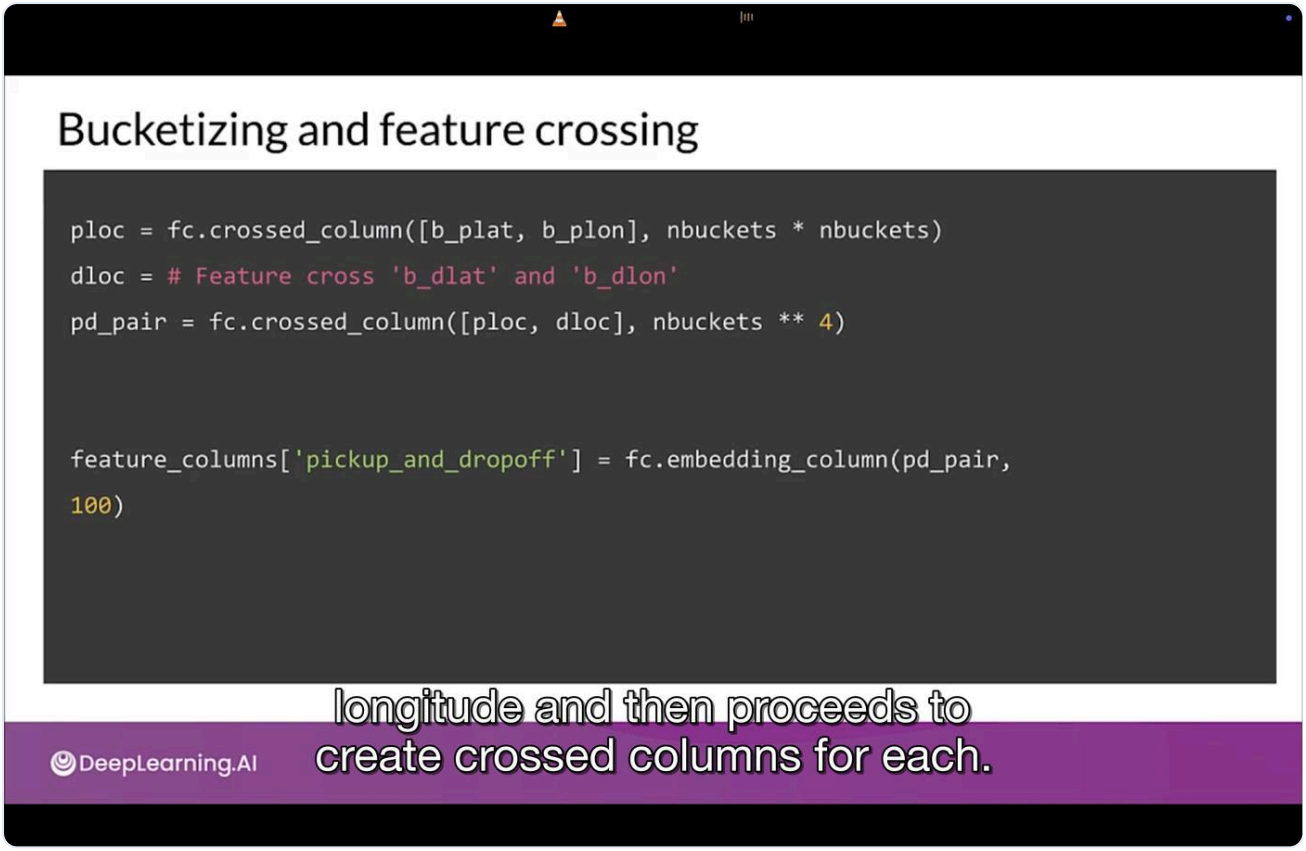

FIGURE 14.12 | Bucketizing and Feature Crossing

In this example you bucketize these latitude and longitude features and create feature crosses out of the geo locational features here the code creates bucket sized columns for pick up and drop off latitude and longitude and then proceeds to create crossed columns for each.

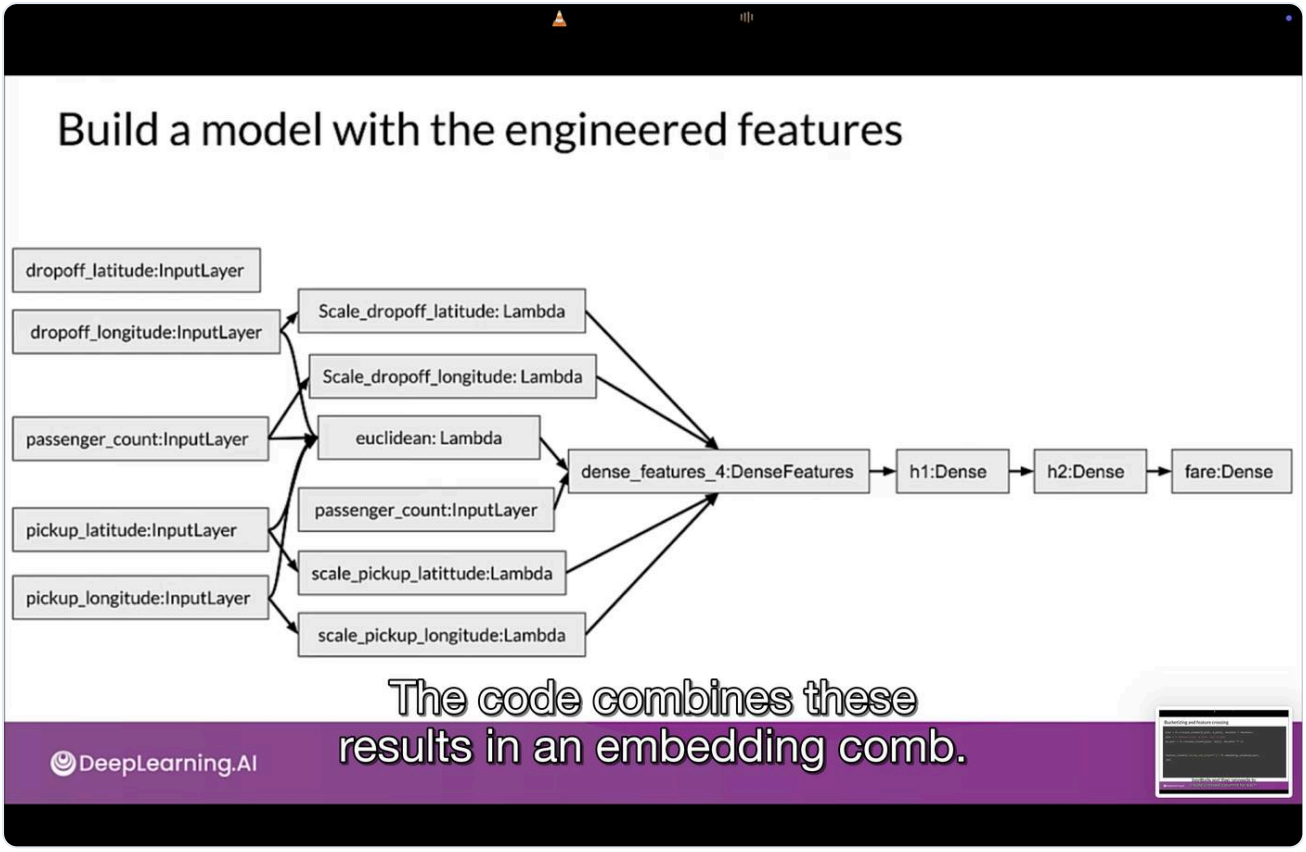

FIGURE 14.13 | Build a Model with the Engineered Features

The code combines these results in an embedding comb. This is the new architecture of your model. As with your first attempted model, you need to create a model in Keras is using the functional API. This will, of course, leverage all the feature engineering that you've done so far. Let's take a look at the performance of this new model.

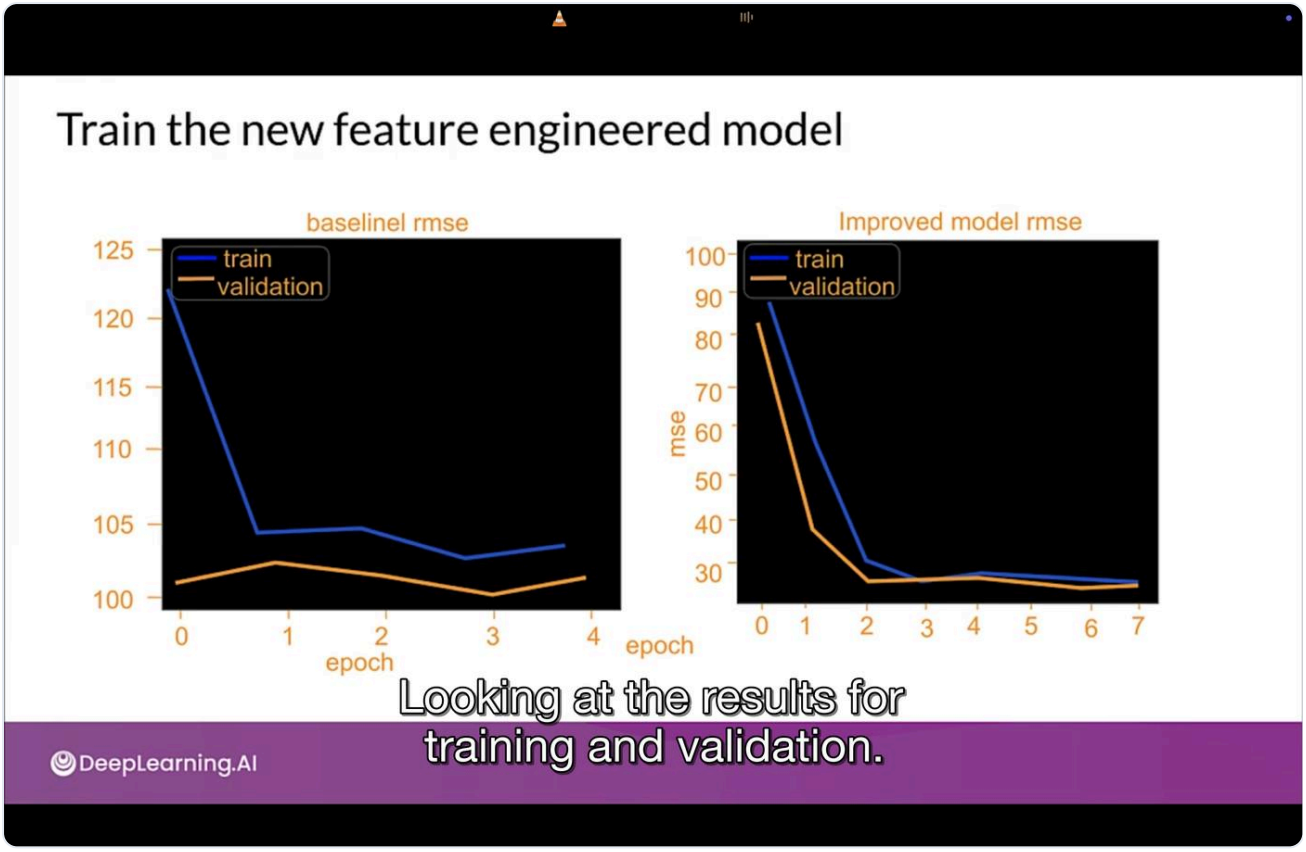

FIGURE 14.14 | Train the New Feature Engineered Model

Looking at the results for training and validation. It's clear that the model with feature engineering on the right is a significant improvement over the baseline model on the left.

LECTURE 15

Linear Dimensionality Reduction

In addition to manually reducing the dimensionality of your datasets, you can also apply several algorithmic approaches to do dimensionality reduction. Let's look at some of those now. Let's look at techniques that you can use to reduce dimensionality automatically. First, let's build some intuition on how linear dimensionality reduction actually works. In this approach, you linearly project n-dimensional data onto a smaller k- dimensional subspace. Here, k is usually much smaller than n. There are infinitely many dimensional subspaces that we can project data onto.

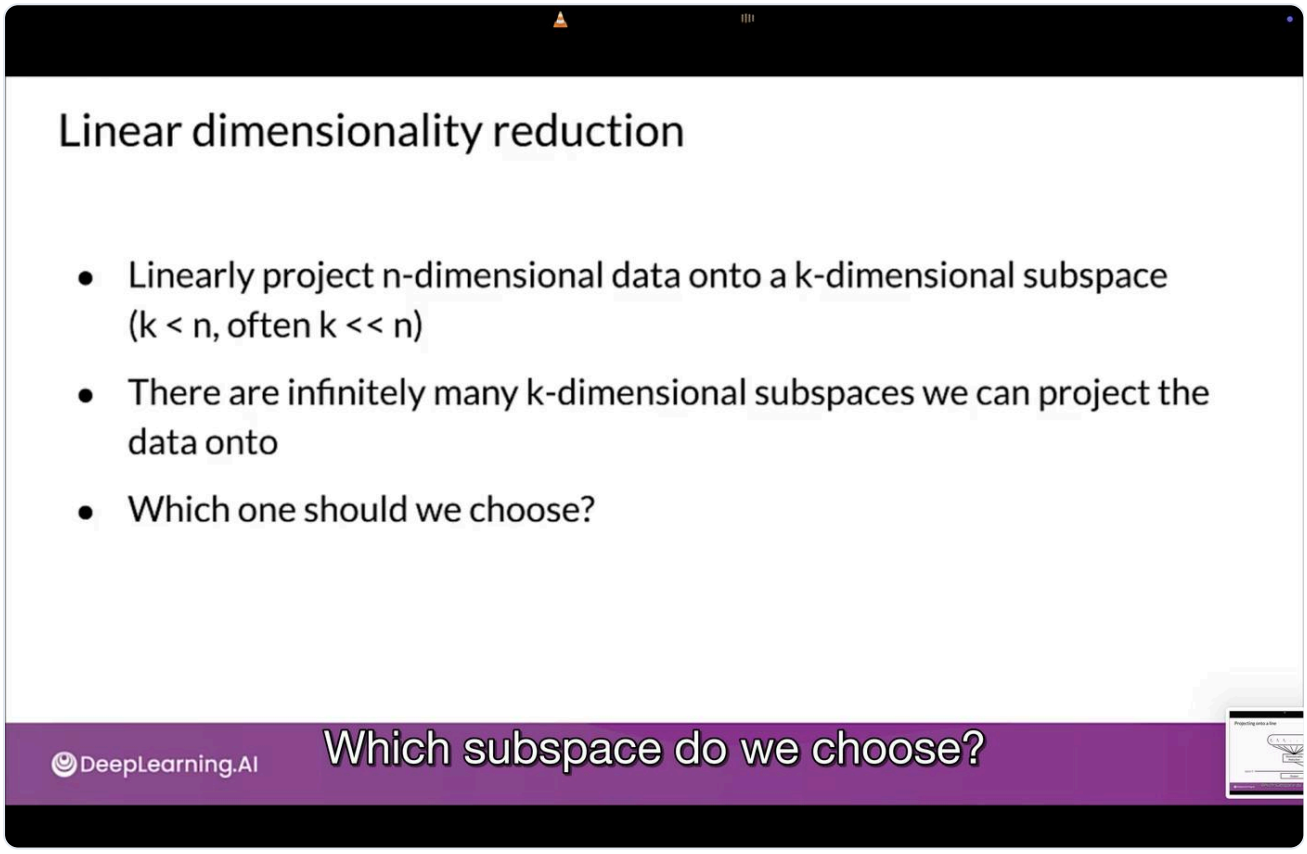

FIGURE 15.1 | Linear Dimensionality Reduction

Which subspace do we choose? To understand how sub-spaces are chosen.

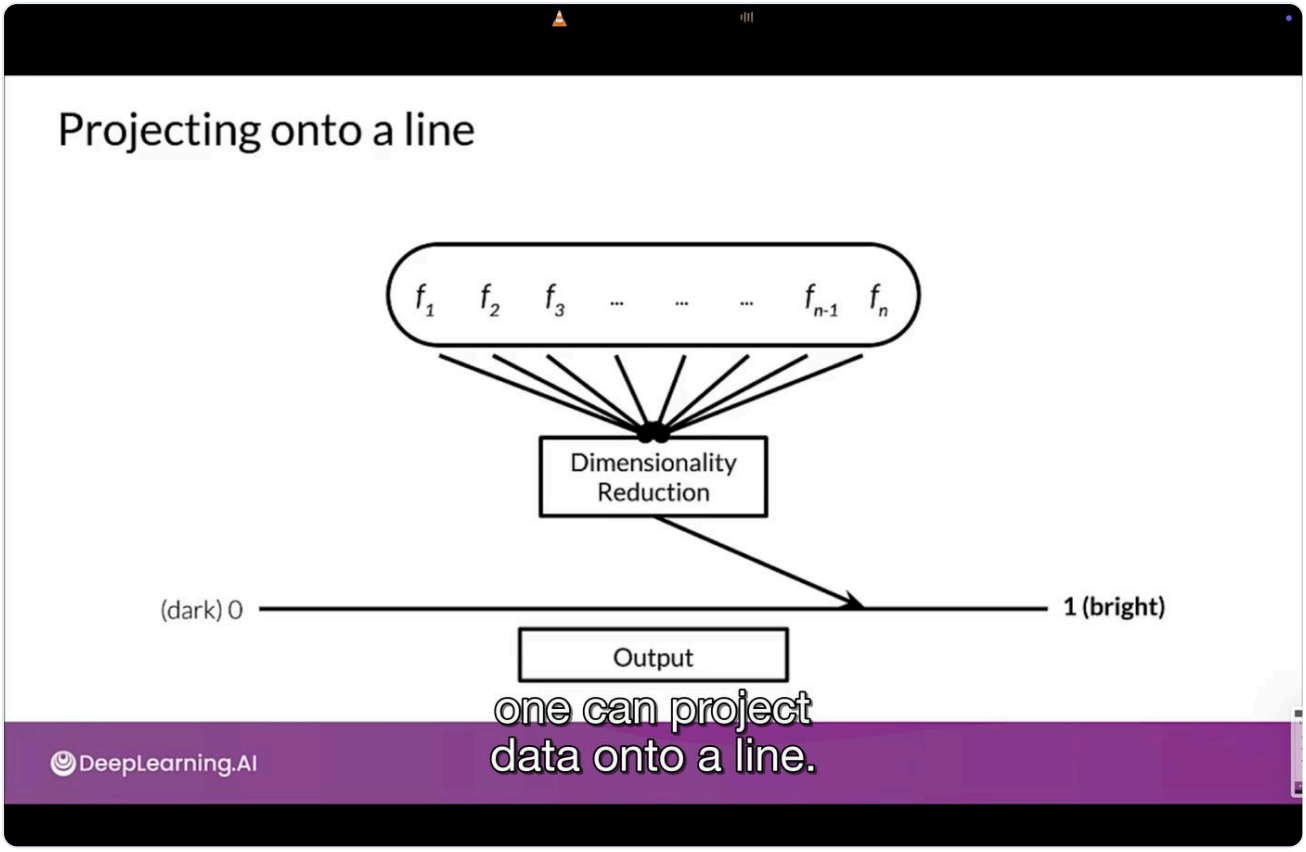

FIGURE 15.2

Let's take a step backwards and look how one can project data onto a line. To start, let's think of features as vectors existing in a high-dimensional space. Visualizing them would reveal a lot about the distribution of the data though it's impossible for us humans to see so many dimensions all at once. Instead, you need to project the data onto a lower dimension, which might allow you to visualize the data more directly. This projection is called an embedding. Computing this requires taking each sample and calculating a single number to describe it. A benefit of reducing to one-dimension is that the numbers and the examples can be sorted on a line.

In this example, we're taking images and reducing the information they contain to just one-dimension their average pixel brightness, which we can then visualize as a point on a line. Coming back to subspaces, there are several ways to choose these k-dimensional subspaces. For example, in a classification tests, you typically want to have the maximum separation among classes. Linear discriminant analysis, or LDA, generally works well for that. For regression, you want to maximize the correlation between the projected data and the output, where Partial least squares, or PLS, works well.

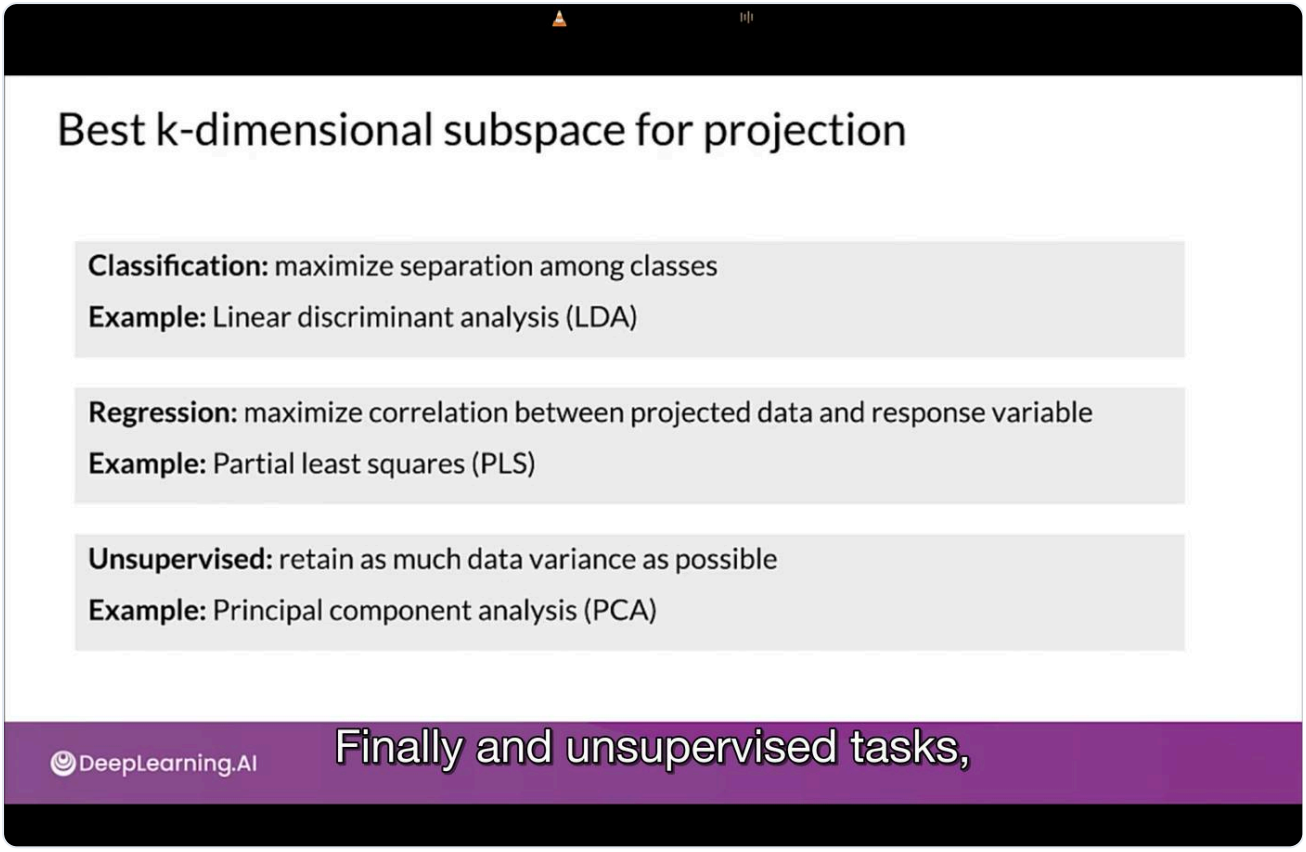

FIGURE 15.3 | Classification: Maximize Separation Among Classes

Finally and unsupervised tasks, we typically want to retain as much of the variance as possible. Principal component analysis, or PCA, is the most widely used technique for doing that.

LECTURE 16

Principal Component Analysis (PCA)

There are several algorithmic approaches for doing dimensionality reduction and principal component analysis, or PCA is one of the most widely used. To start, let's look at how principal component analysis or PCA works.

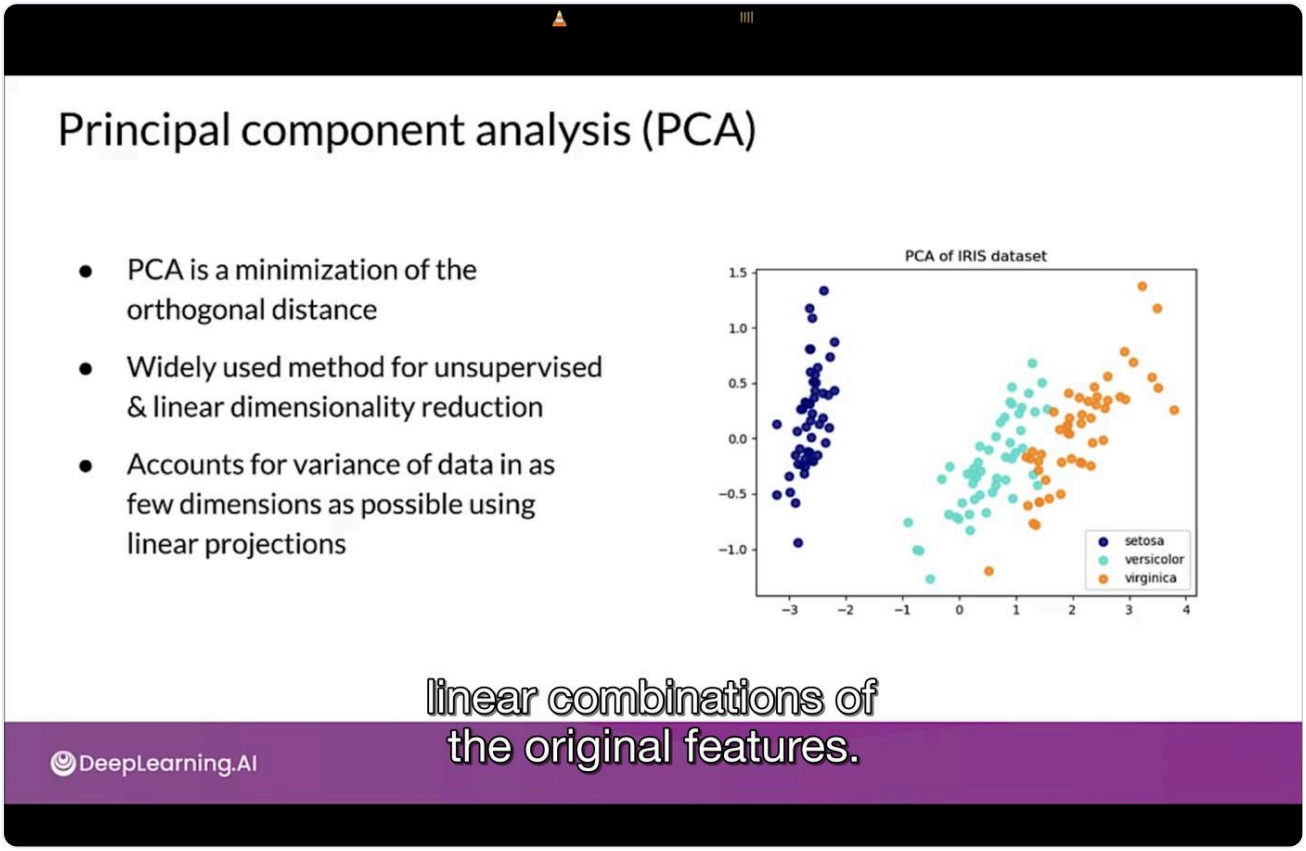

FIGURE 16.1 | Principal Component Analysis (PCA)

This is an unsupervised algorithm that creates linear combinations of the original features. PCA performs dimensionality reduction in two steps, starting with decorrelation, where it doesn't change the dimensionality of the data at all. In the first step, PCA rotates the samples so that they are aligned with the coordinate axes. In fact, there is more than this. PCA also shifts the samples so that they have a mean of zero. These scatter plots show the effect of PCA applied to three features of the IRIS dataset.

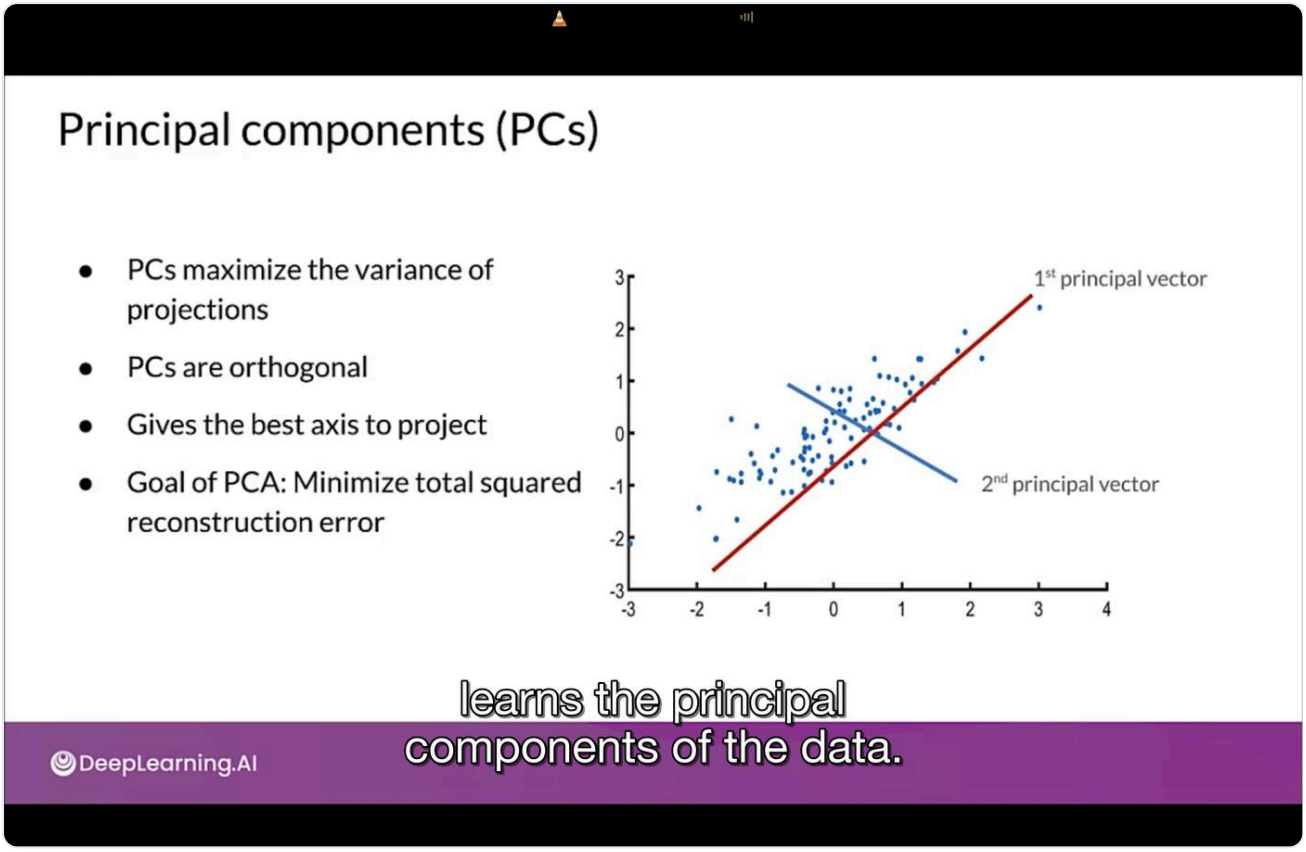

FIGURE 16.2

Finally, PCA is called principal component analysis because it learns the principal components of the data. These are the directions in which the samples vary the most depicted here in red. It is the principal components that PCA aligns with the coordinate axis. The goal of PCA is that it tries to find a lower dimensional surface onto which to project the data so as to minimize the squared projection error. In other words, to minimize the square of the distance between each point and the location of where it gets projected. The result will be to maximize the variance of the projections. The first principal component is the projection direction that maximizes the variance of the projected data.

The second principal component is the projection direction that is orthogonal to the first principal component and maximizes the remaining variance of the projected data.

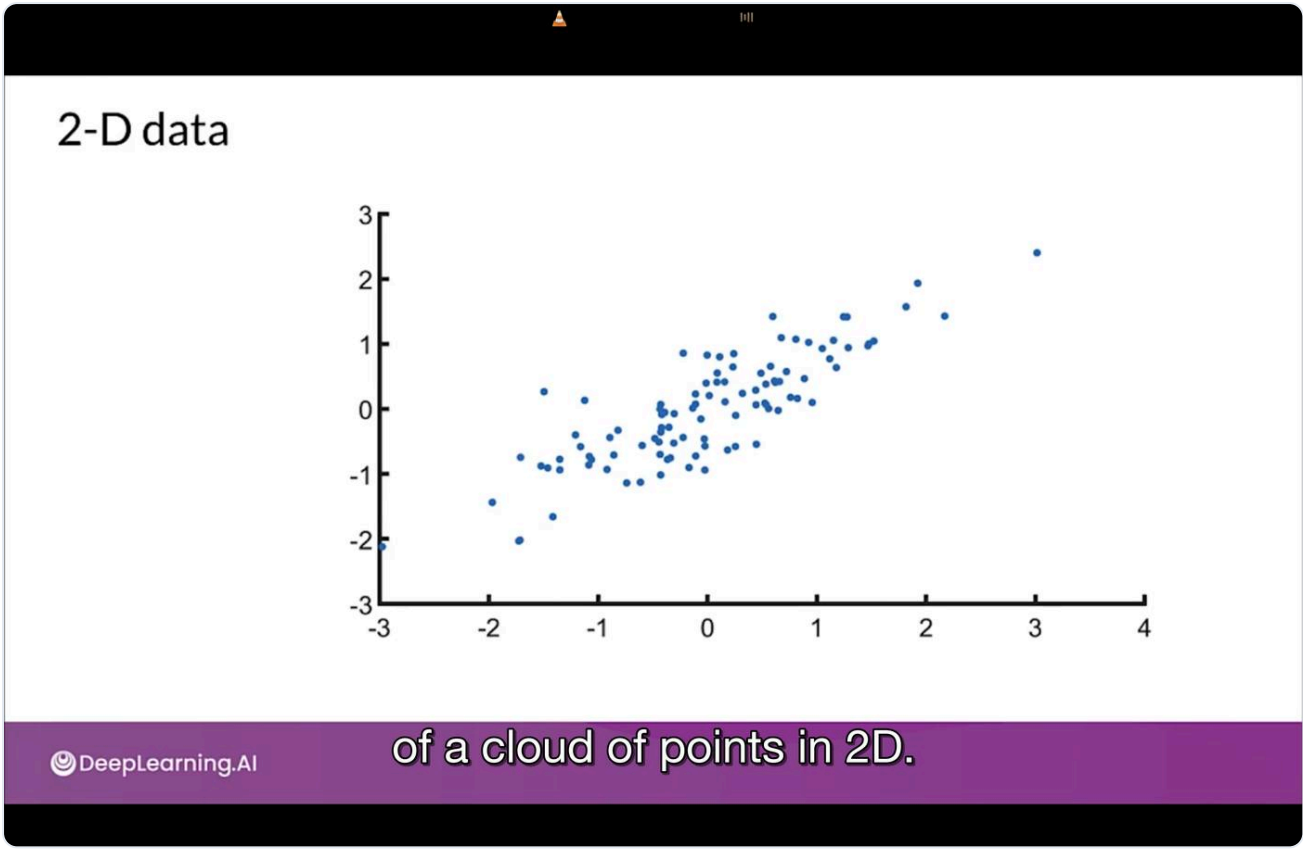

FIGURE 16.3

Here's a toy example consisting of a cloud of points in 2D. Let's try to apply PCA to this two-dimensional data and see what happens.

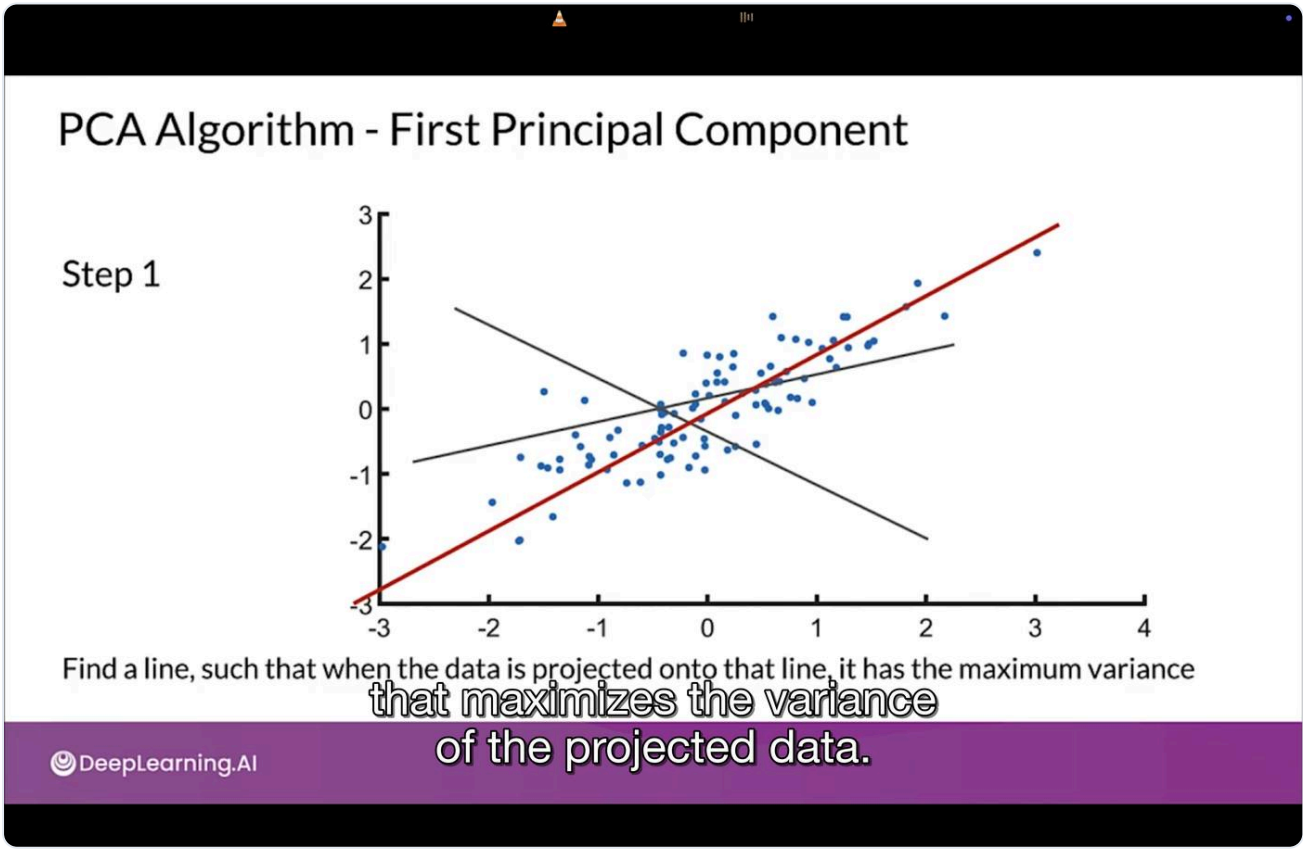

FIGURE 16.4 | PCA Algorithm - First Principal Component

The first principal component is a projection direction that maximizes the variance of the projected data. In the plot, you can see three attempts at producing such a line. In this case, it's quite obvious that the variance is maximized in the direction indicated by the red line.

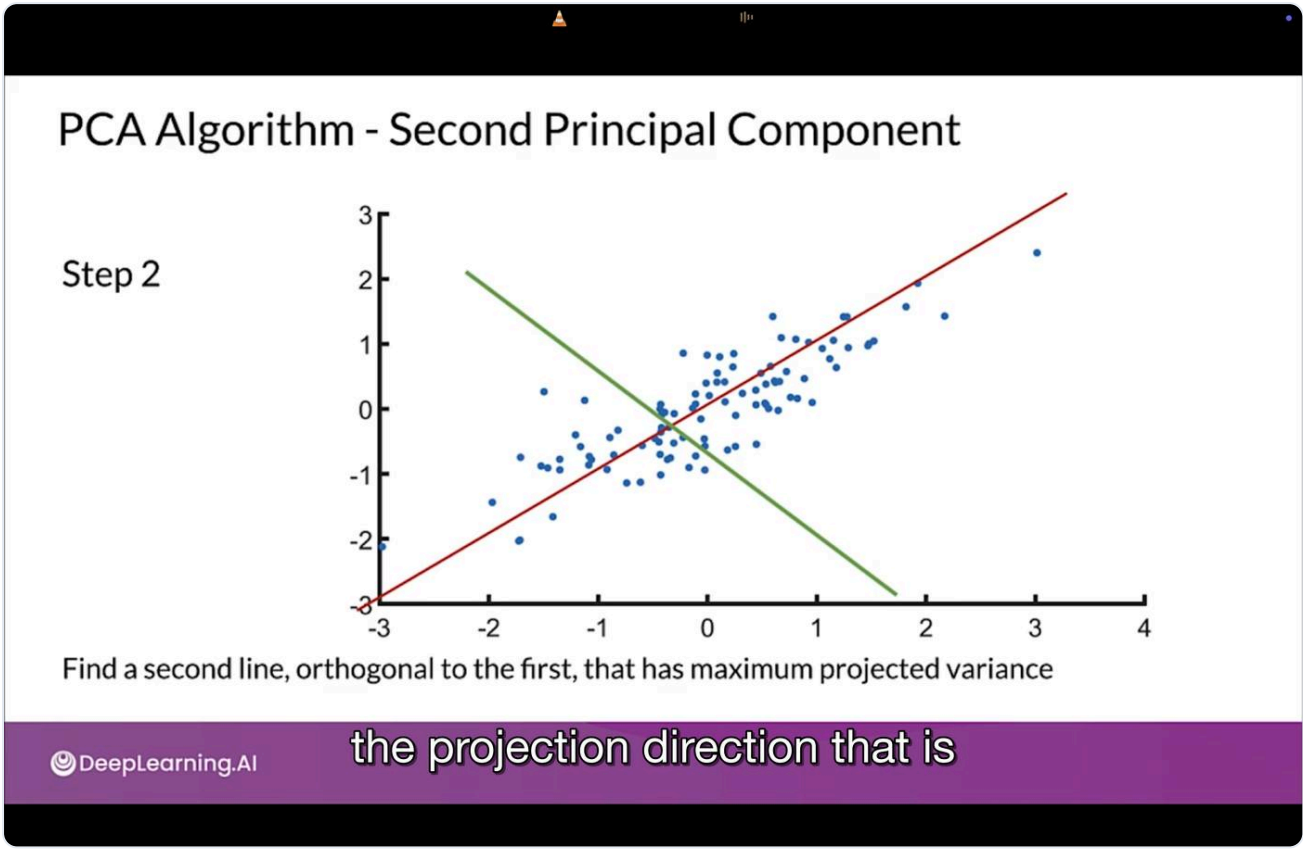

FIGURE 16.5 | PCA Algorithm - Second Principal Component

The second principal component is the projection direction that is orthogonal to the first principal component and maximizes the remaining variance of the projected data indicated here by the green line.

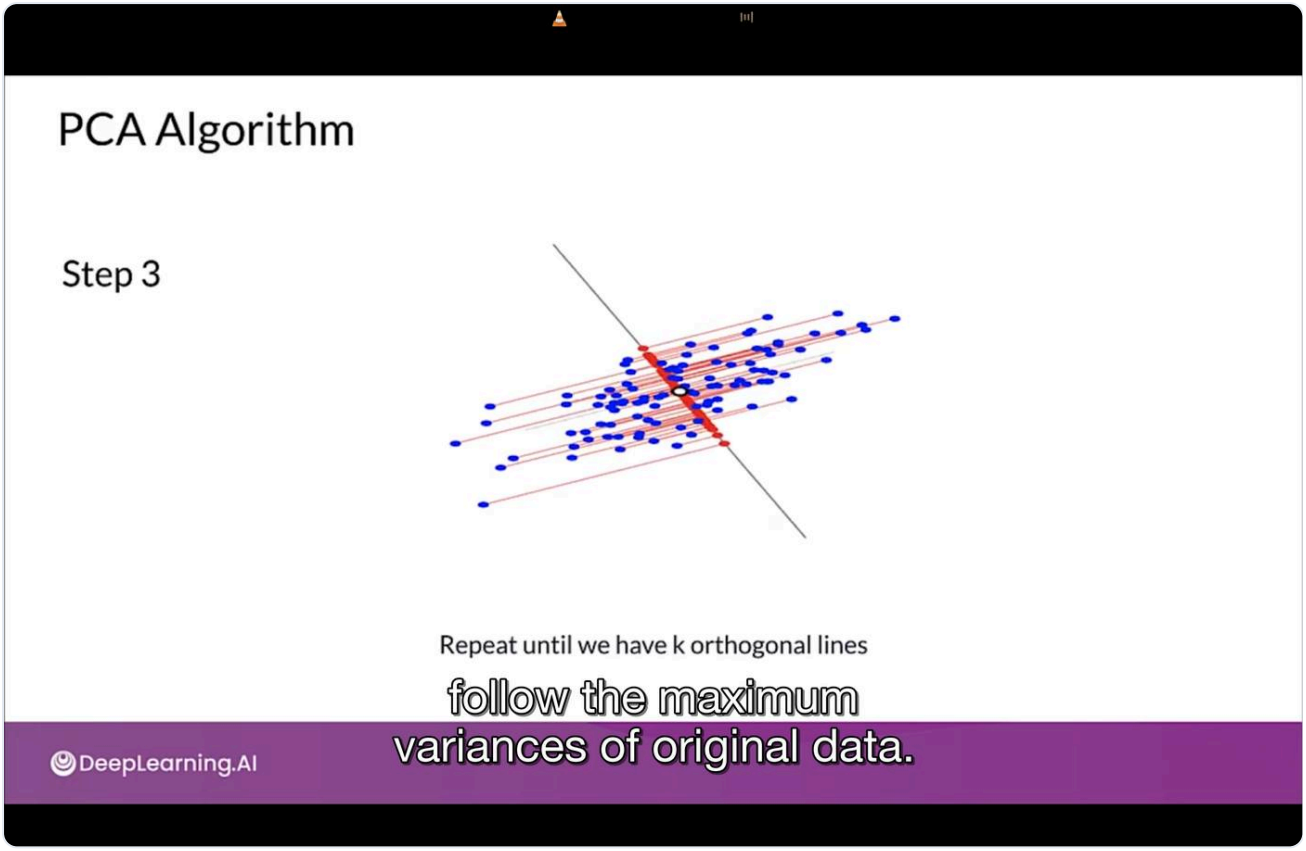

FIGURE 16.6

The full set of principal components comprises a new orthogonal basis for feature space whose axis follow the maximum variances of original data. These projections are simply transformed to the new k-dimensional reduced space. This means that when you're projecting your original data onto the first k principal components, you're reducing the dimensionality of the data. Later, you can recover the original space from this reduced dimensionality projection. This reconstruction will of course, have some amount of error, but this is often negligible and acceptable given the other benefits of dimensionality reduction.

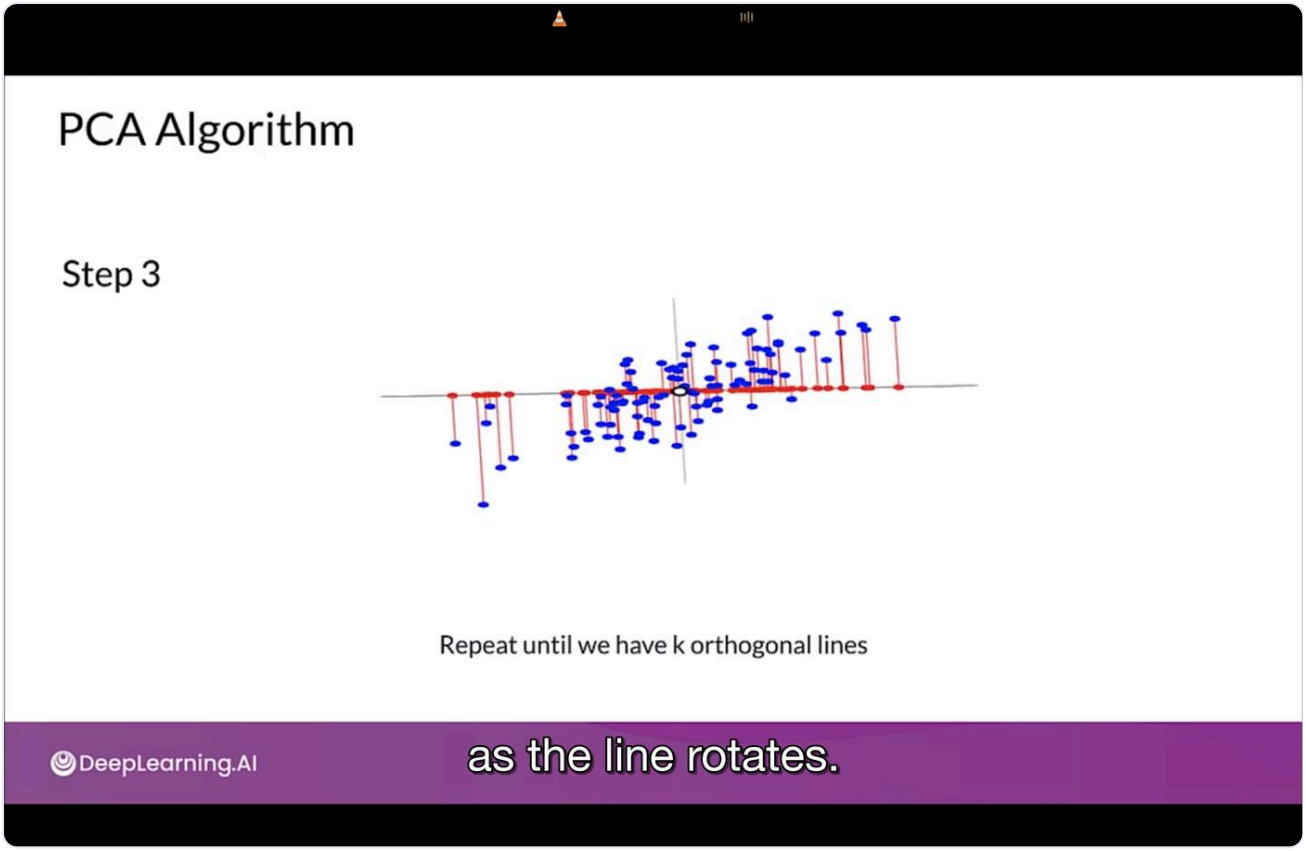

FIGURE 16.7

Also, look at how the red dots change as the line rotates. That's the variance. Can you see when it reaches its maximum? Second, if we reconstruct the original two characteristics, the blue dots from the new ones, the red dots, the reconstruction error will be given by the length of the connecting red line. Observe how the length of the red lines changes while the line rotates. Can you see where the total length reaches a minimum?

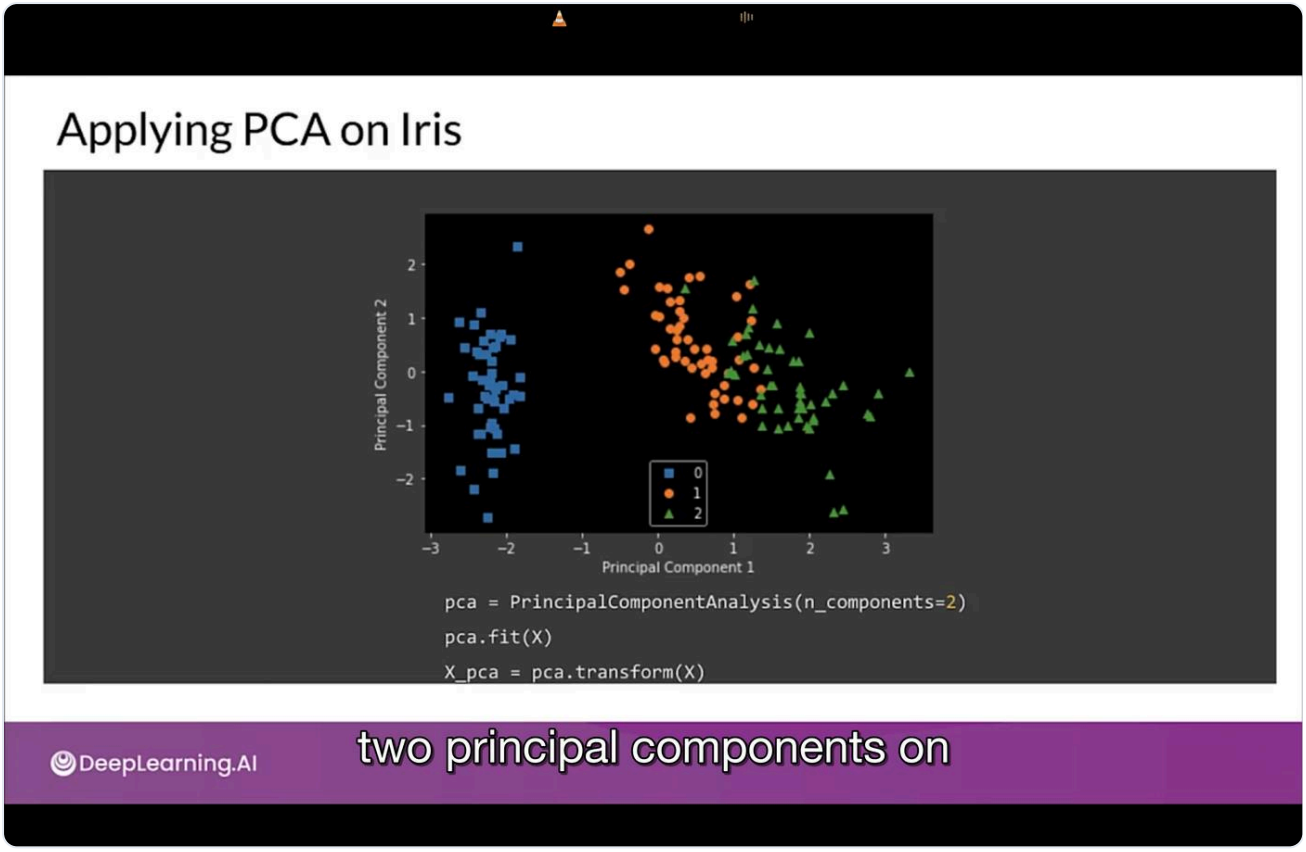

FIGURE 16.8

Here we apply the PCA algorithm with two principal components on the IRIS dataset and visualize the results.

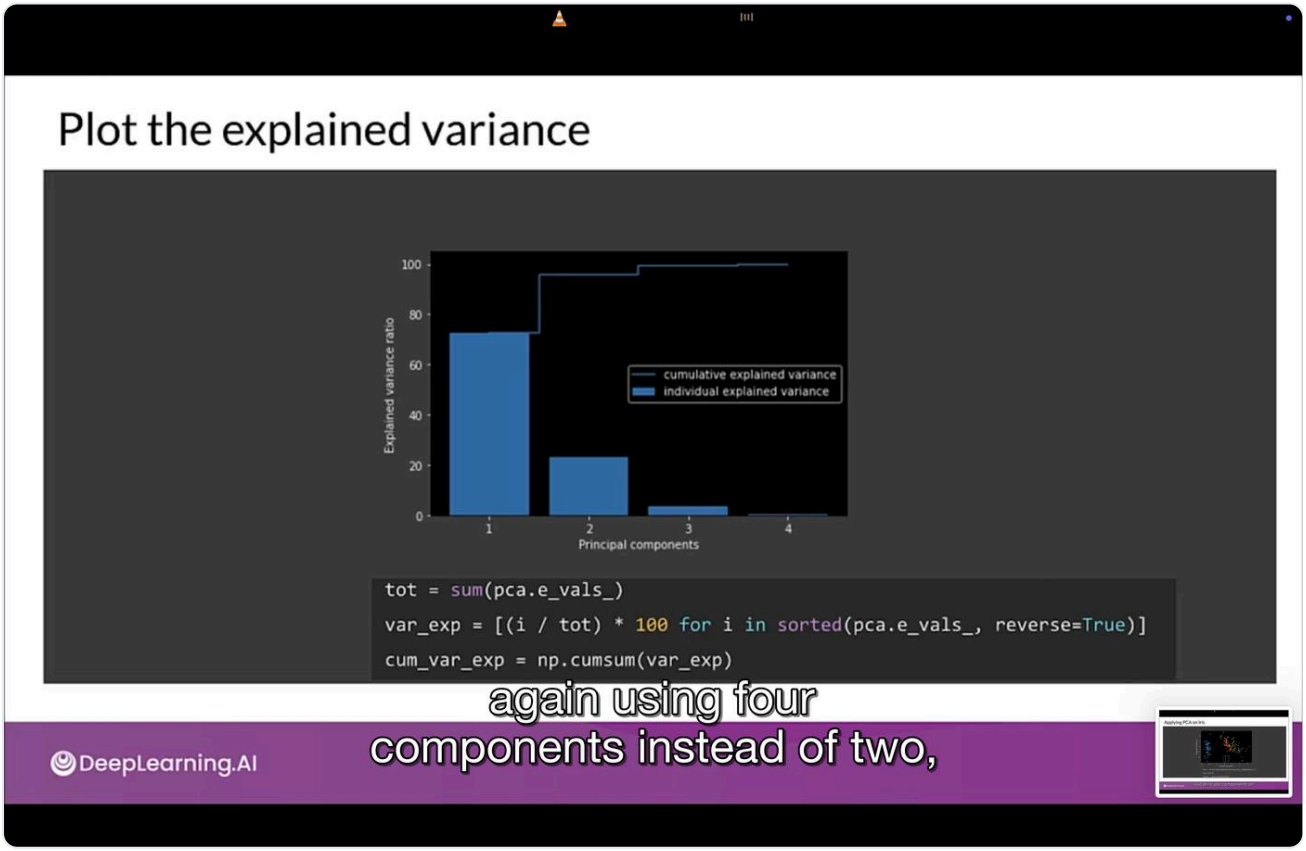

FIGURE 16.9 | Plot the Explained Variance

Now assuming you've applied PCA again using four components instead of two, let's visualize how much variance has been explained using these four components. If you look at the relative variance, you might lose some information, but if the eigenvalues are small, not much information is lost. Principal components are orthogonal in nature as we've seen before and this means that they are uncorrelated. Also, they are ranked in order of their explained variance. The first principle component explains the most variance in the dataset, and the second explains the second most variance and so on. Therefore, you can reduce dimensionality by limiting the number of principal components to keep based on the cumulative explained variance.

For example, you might decide to keep only as many principal components as are needed to reach a cumulative explained variance of 90 percent.

FIGURE 16.10

The factor loadings are the unstandardized values of the eigenvectors. We can interpret the loadings as the covariances or correlations.

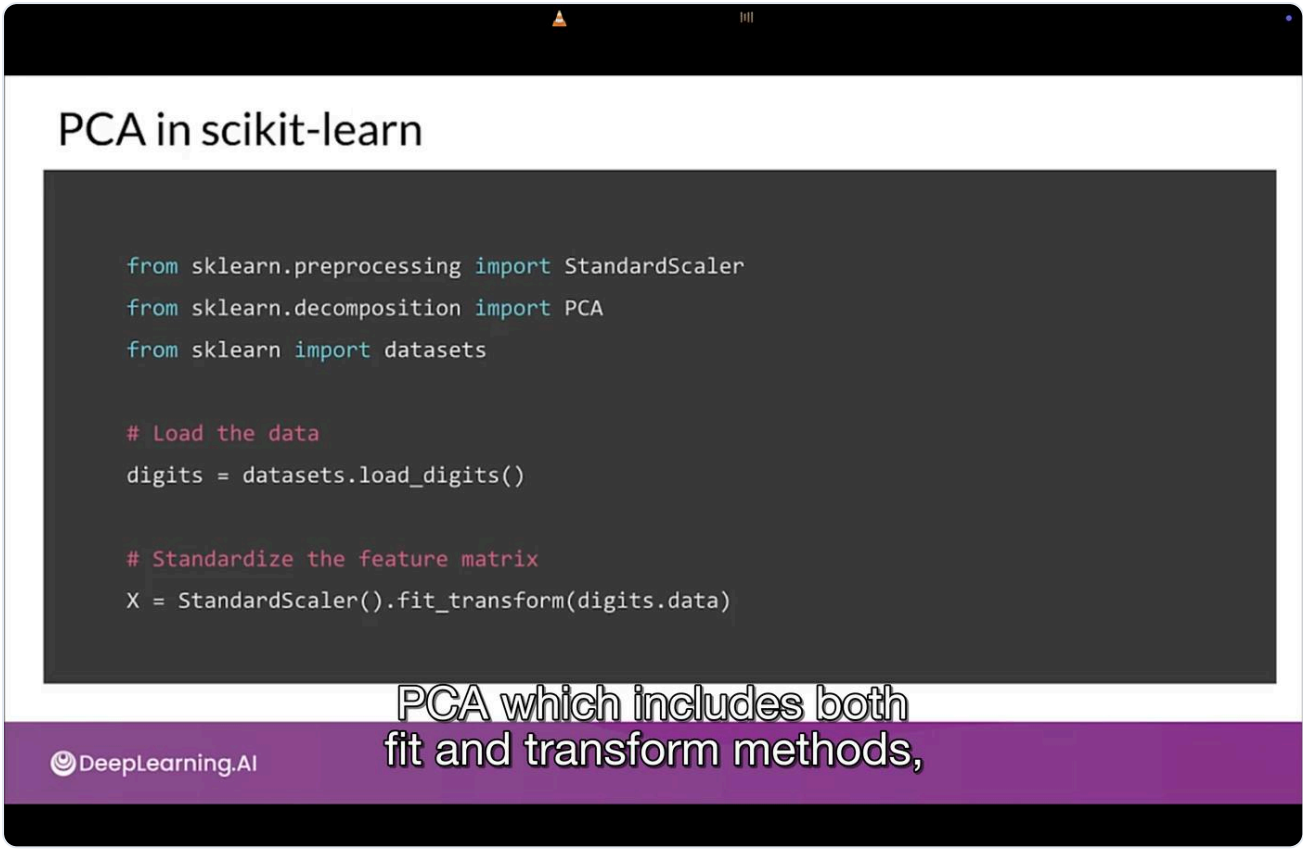

FIGURE 16.11

Scikit-learn has an implementation of PCA which includes both fit and transform methods, just like the standard scalar operation, as well as a fit transform method which combines both fit and transform. The fit method learns how to shift and rotate the samples, but it doesn't actually change them. The transform method, on the other hand, applies the transformation the fit learned. In particular, the transfer method can be applied to new unseen samples. Before applying PCA, let's use the standard scaler on the features.

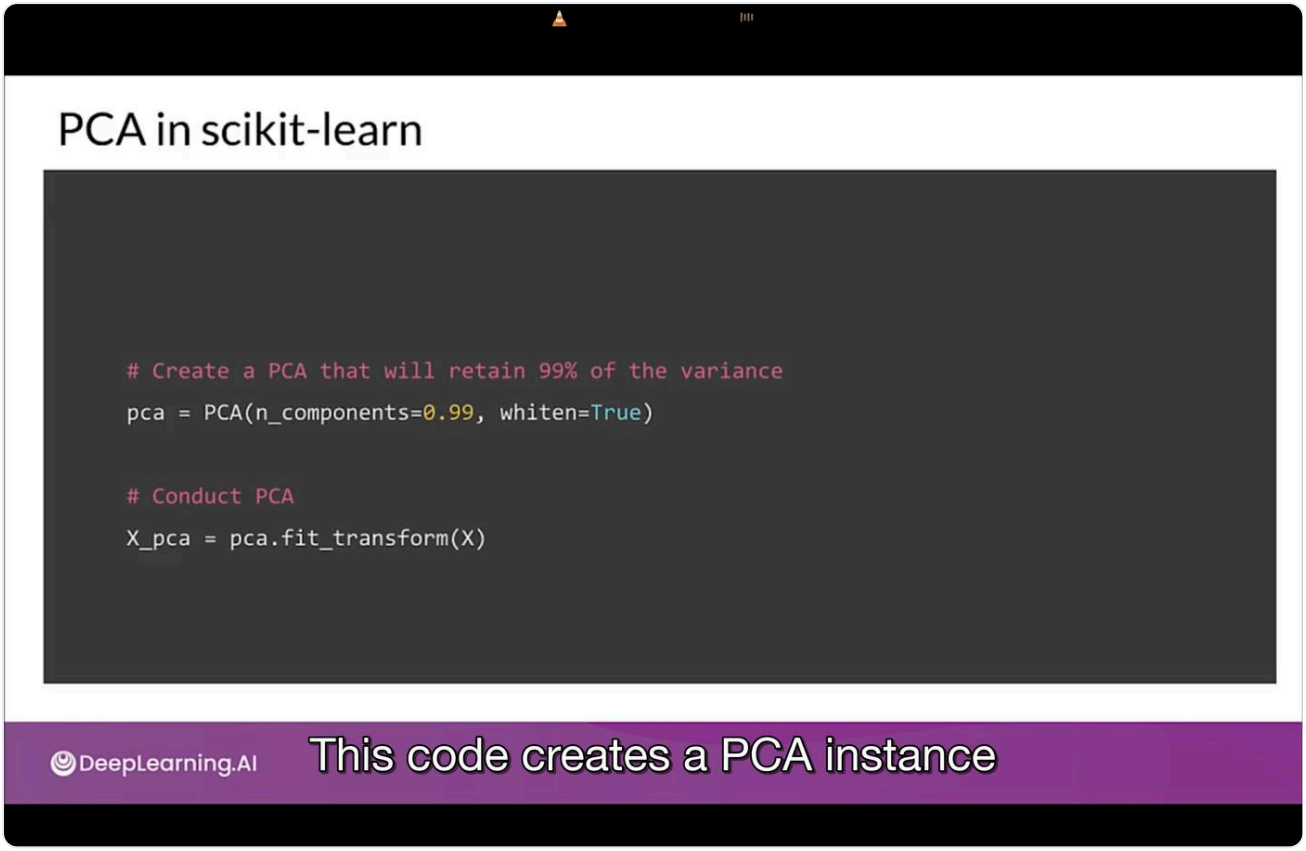

FIGURE 16.12

This code creates a PCA instance that will retain 99 percent of the variance fitted to the data and apply the transform which was learned. Summing all of that up.

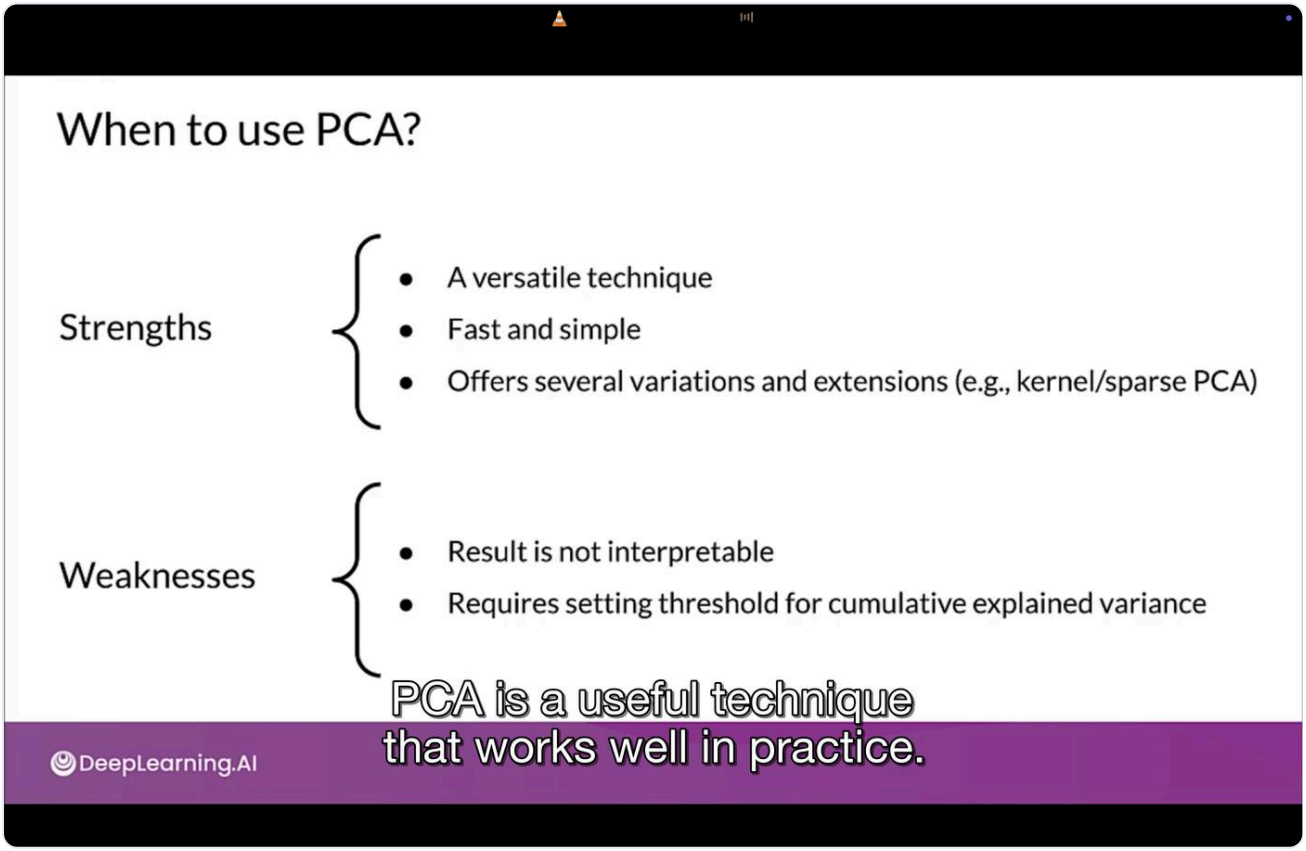

FIGURE 16.13 | When to Use PCA?

PCA is a useful technique that works well in practice. It's fast and simple to implement, which means you can easily test algorithms with and without PCA to compare performance. In addition, PCA offers several variations and extensions. For example, kernel PCA or sparse PCA, etc., to tackle specific roadblocks. However, the resulting principal components are not interpretable, which may be a deal breaker in some settings where interpretability is important. In addition, you must still manually set or tune a threshold for cumulative explained variance. Other than this, PCA is especially useful when visually studying clusters of observations in high dimensions.

This could be when you are still exploring the data. For example, you may have reason to believe that the data are inherently low rank, which means that there are many attributes, but only a few attributes which mostly determine the rest through a linear association.

LECTURE 17

Independent Components Analysis (ICA)

In addition to the techniques we've already discussed, there are several more algorithmic approaches to do dimensionality reduction. Let's look at some of those now, some techniques are focused on particular kinds of problems. For example, staying with unsupervised approaches, there are techniques such as single value decomposition or SVD, and independent component analysis or ICA. In Matrix Factorization techniques, you could use non-negative matrix factorization.

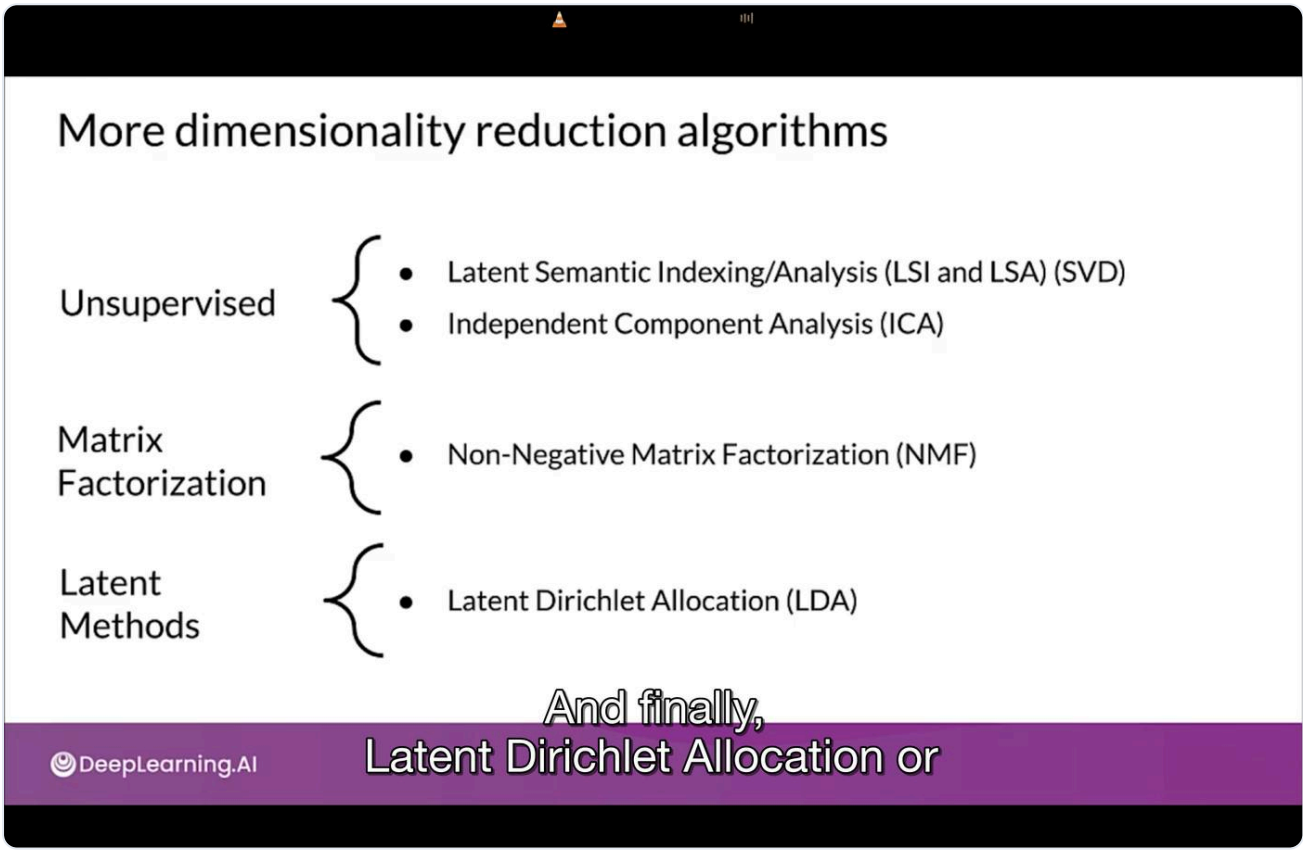

FIGURE 17.1 | More Dimensionality Reduction Algorithms

And finally, Latent Dirichlet Allocation or LDA is one of the more popular latent dimensionality reduction methods. Let's discuss single value decomposition or SVD. Matrices can be seen as linear transformations in space. PCA, which we discussed previously relies on eigen-decomposition, which can only be done for square matrices, of course, you don't always have square matrices. In TF-IDF, for example, high frequency of terms may not really be fruitful, in some cases, rare words contribute more. In general, the importance of words increases if the number of occurrences of these words within the same document also increases. On the other hand, the importance will be decreased for words which occur frequently in the corpus.

The challenges that the resulting matrix is very sparse and not square. To decompose these types of matrices, which can't be decomposed with eigendecomposition, we can use techniques such as singular value decomposition or SVD.

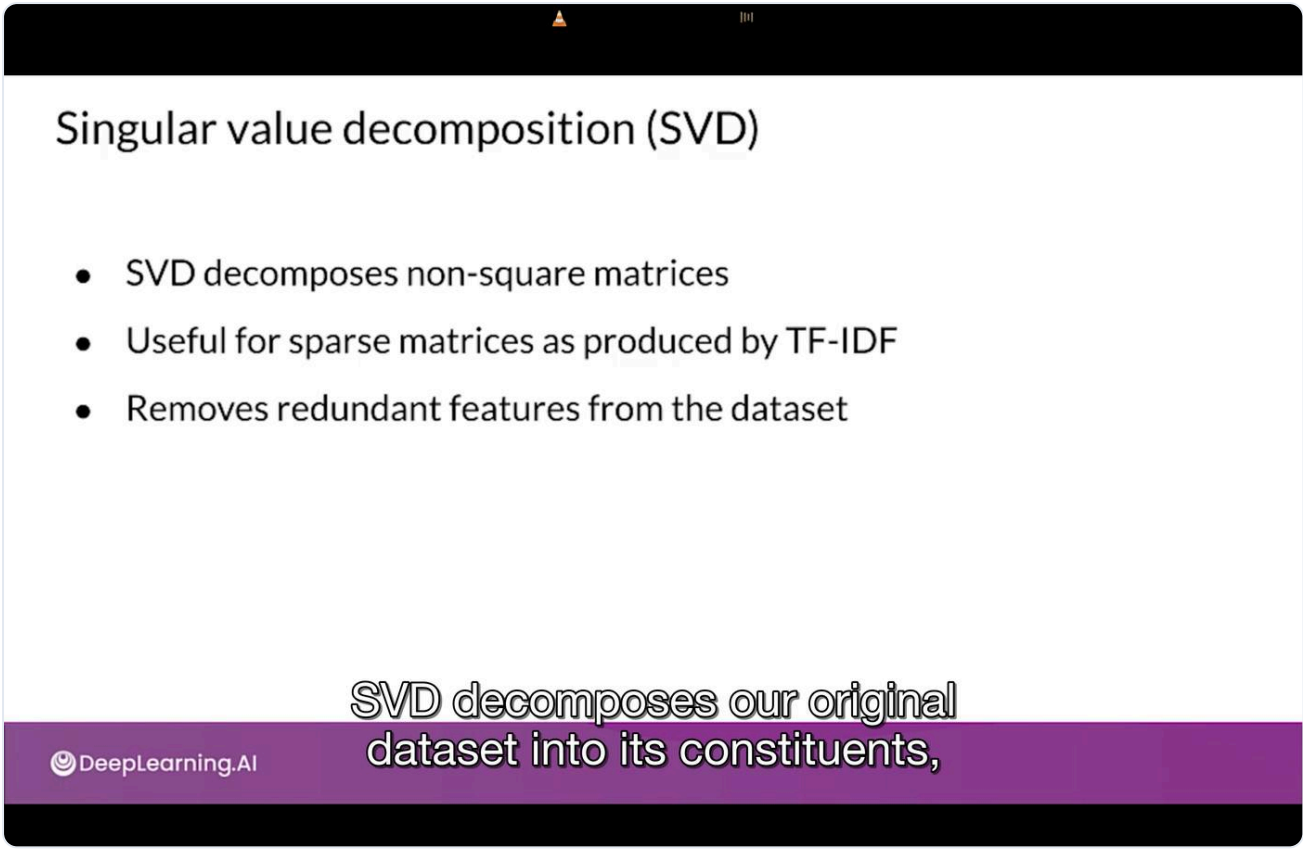

FIGURE 17.2 | Singular Value Decomposition (SVD)

SVD decomposes our original dataset into its constituents, resulting in a reduction of dimensionality, it's used to remove redundant features for the dataset. Independent component analysis or ICA is another algorithm and is based on information theory. It's also one of the most widely used dimensionality reduction techniques. PCAs and ICAs's significant difference is that PCA looks for uncorrelated factors, while ICA looks for independent factors.

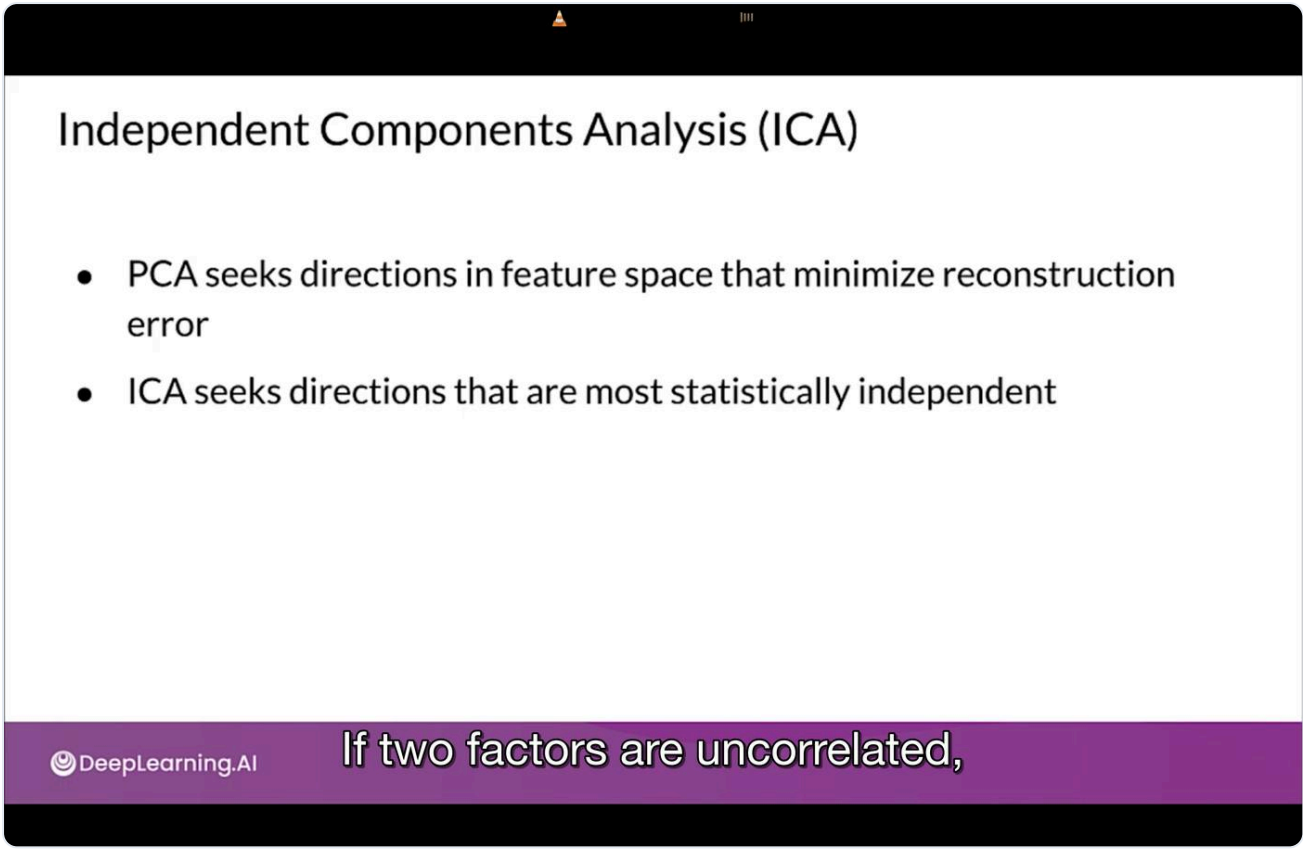

FIGURE 17.3 | Independent Components Analysis (ICA)

If two factors are uncorrelated, it means that there is no linear relation between them. If they're independent, it means that they are not dependent on other variables.

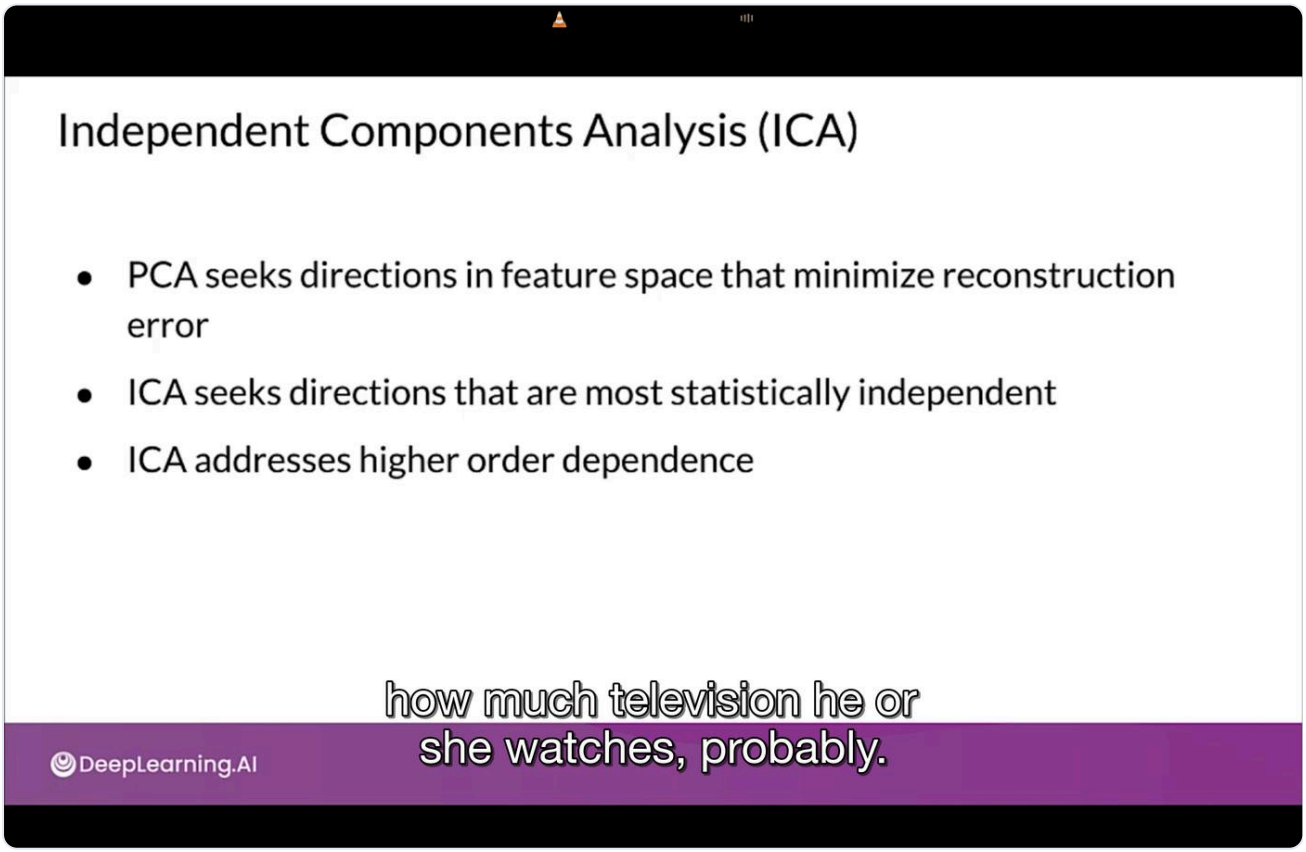

FIGURE 17.4 | Independent Components Analysis (ICA)

For example, a person's age is independent of what that person eats or how much television he or she watches, probably. Although maybe you've noticed patterns in food choices and TV watching among different age groups, anyway, I digress. Independent component analysis separates a multivariant signal into additive components that are maximally independent.

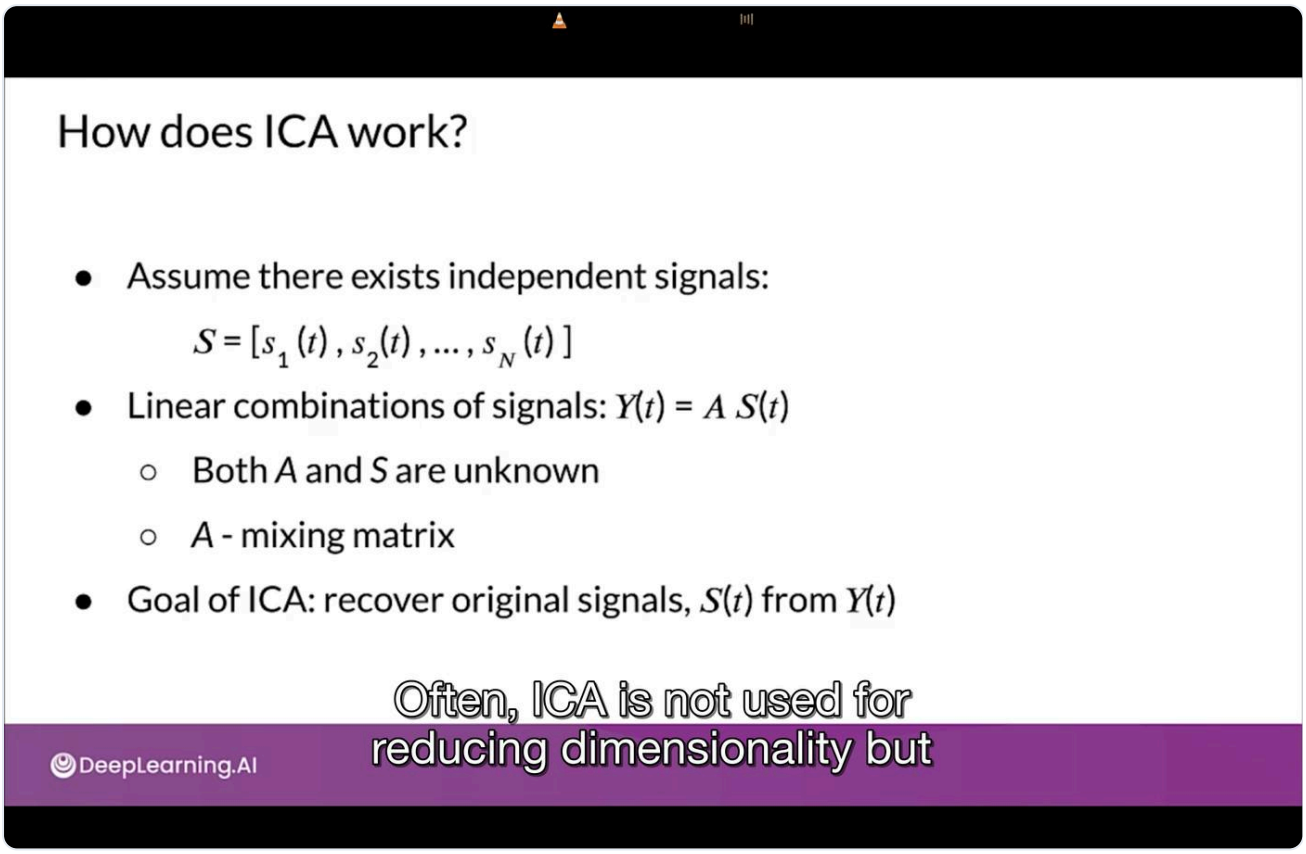

FIGURE 17.5 | How Does ICA Work?

Often, ICA is not used for reducing dimensionality but for separating superimposed signals. Since the model does not include a noise term, for the model to be correct, whitening must be applied. This can be done in various ways, including using one of the PCA variants. ICA further assumes that there exists independent signals, S, and a linear combination of signals, Y. The goal of ICA Is to recover the original signals, S, from Y, ICA assumes that the given variables are linear mixtures of some unknown latent variables. It also assumes that these latent variables are mutually independent. In other words, they're not dependent on other variables and hence they are called the independent components of the observed data.

Let's compare PCA and ICA visually to get a better understanding of how they're different. Both are statistical transformations, that is PCA uses information extracted from second order statistics, while ICA goes up to higher order statistics.

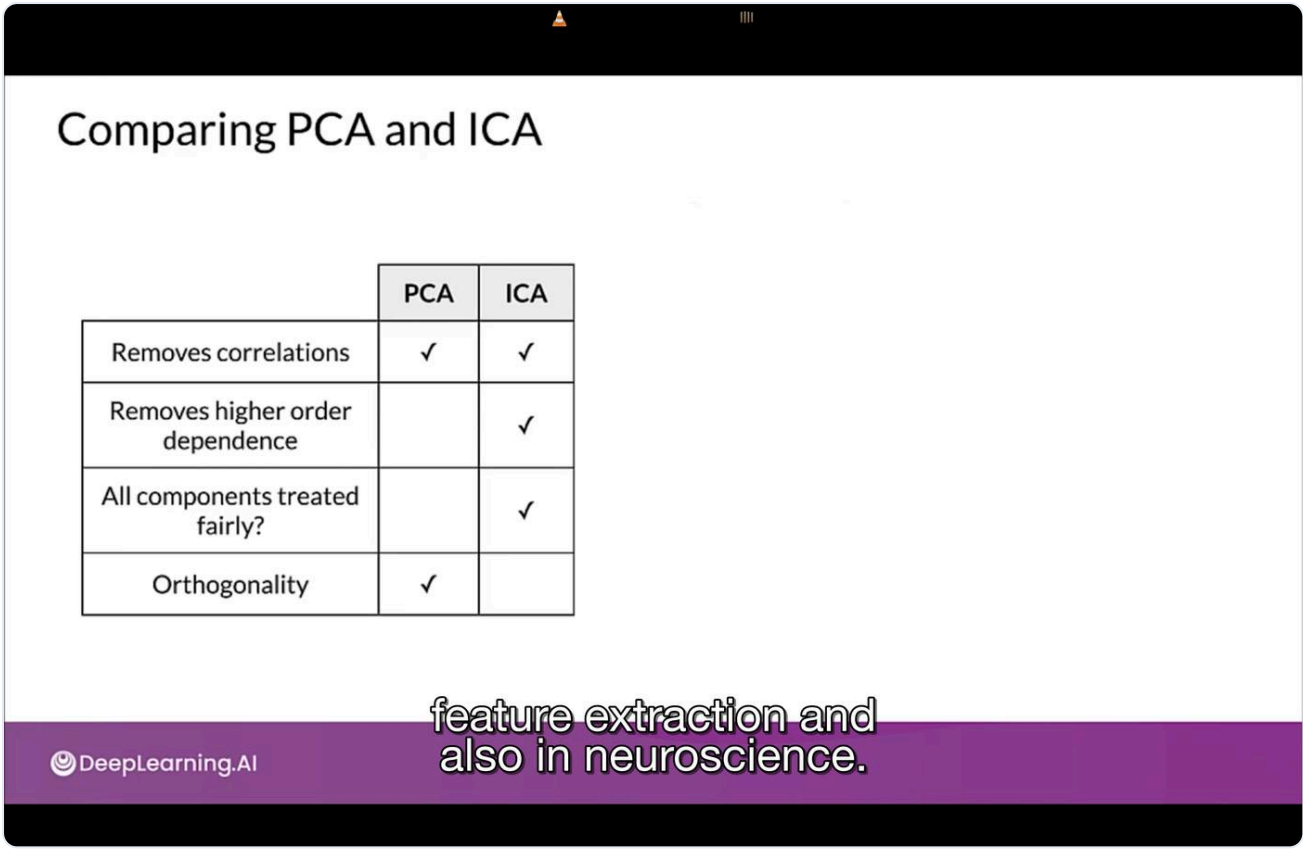

FIGURE 17.6 | Removes Correlations Y V

Both are used in various fields like blind source separation, feature extraction and also in neuroscience. ICA is an algorithm that finds directions in the feature space corresponding to projections which are highly non-Gaussian.. Unlike PCA, these directions need not be orthogonal in the original feature space, but they are orthogonal in the whitened feature space, in which all directions correspond to the same variance. PCA, on the other hand, finds orthogonal directions in the raw feature space that corresponded directions accounting for maximum variance. Let's look at a simulation of two independent sources using a highly non-Gaussian process on the left.

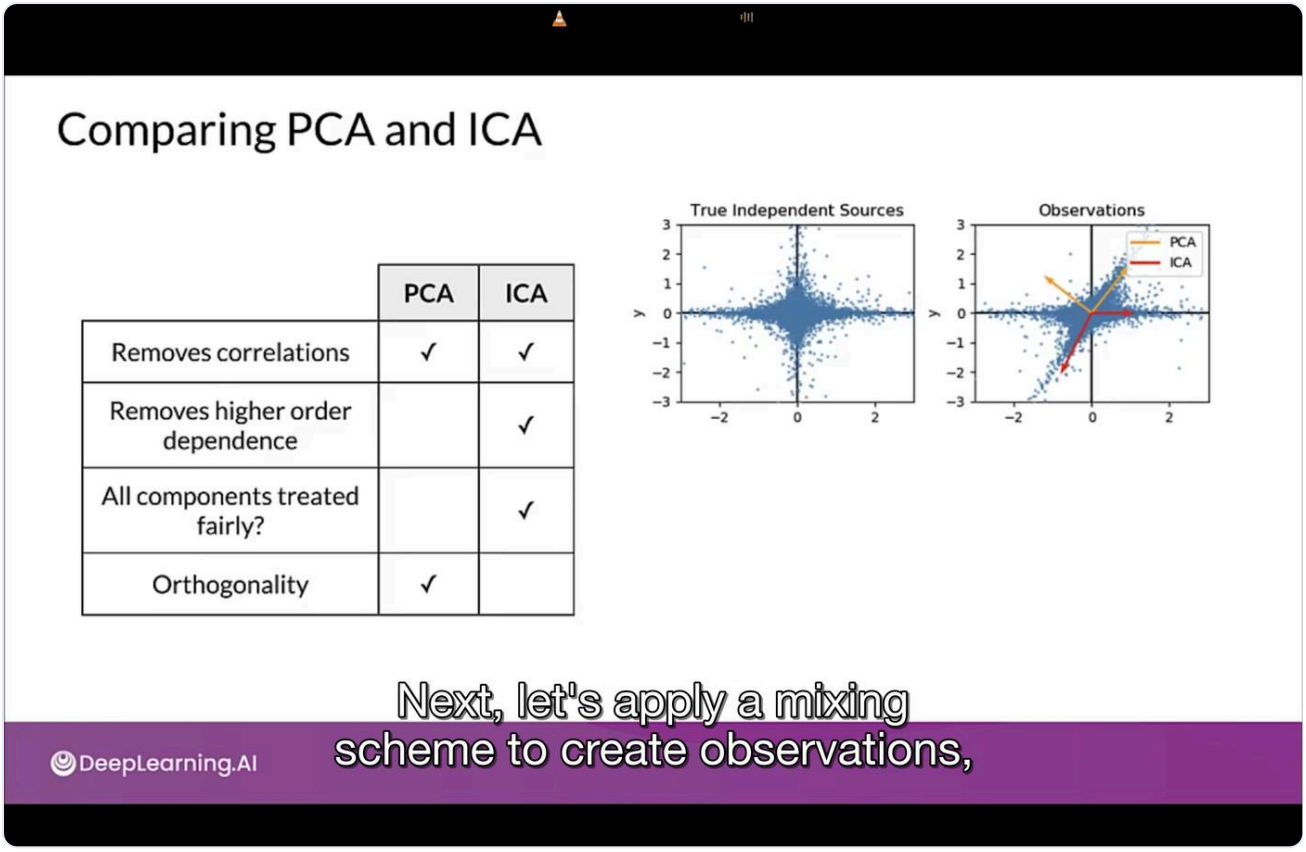

FIGURE 17.7 | True Independent Sources Observations

Next, let's apply a mixing scheme to create observations, in this raw observation space, directions identified by PCA are represented by orange vectors. Then, let's represent the signal in the PCA space after whitening by the variance corresponding to the PCA vectors.

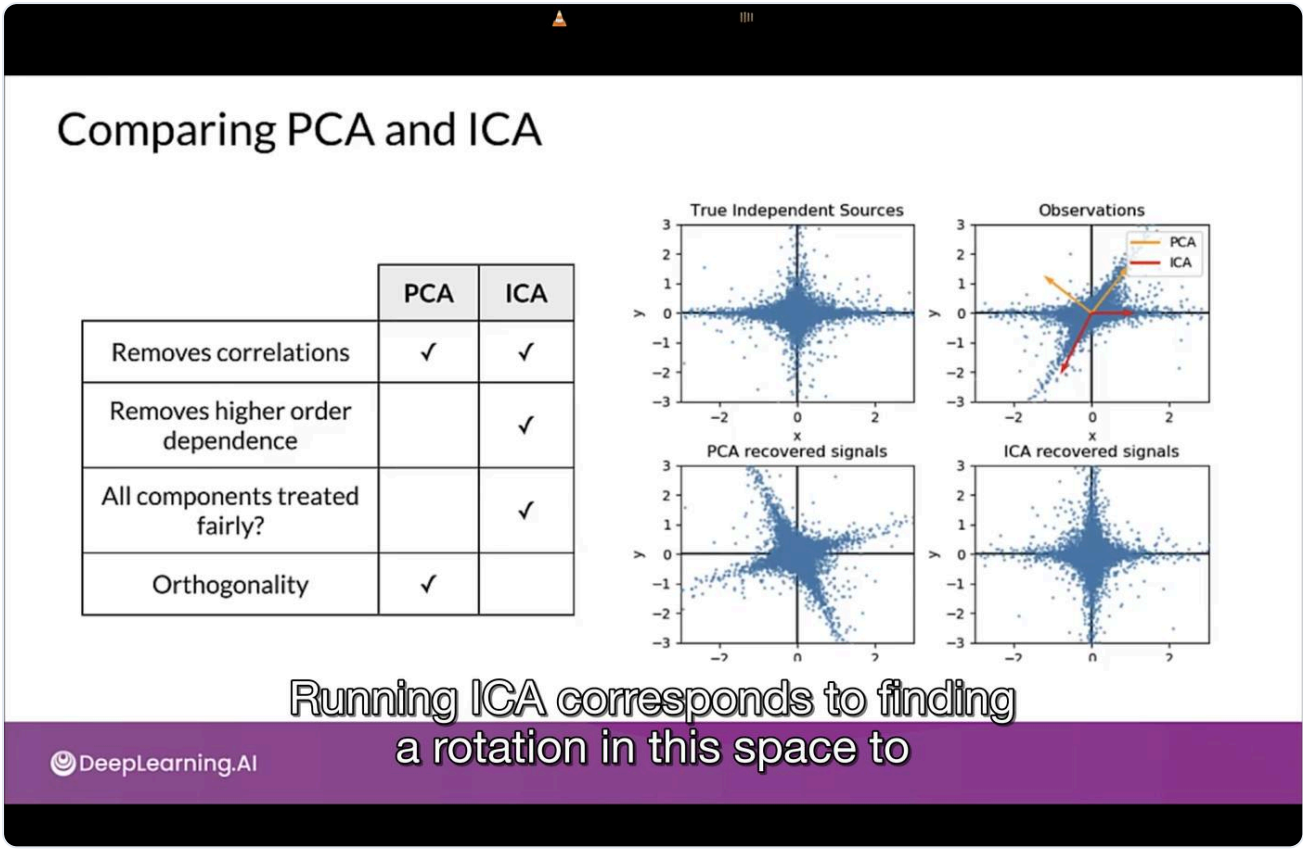

FIGURE 17.8 | 3 True Independent Sources Observations

Running ICA corresponds to finding a rotation in this space to identify the directions which are the most non-Gaussian. This is in the lower right, from this figure, you can see that PCA removes correlations but not higher order dependence. On the contrary, ICA removes correlations along with higher order dependence. When it comes to the importance of components, PCA, considers some of them to be more important than others. ICA, on the other hand, considers all components to be equally important. Now, let's discuss a dimensionality reduction technique called non-negative Matrix Factorization or NMF. NMF expresses samples as a combination of interpretable parts.

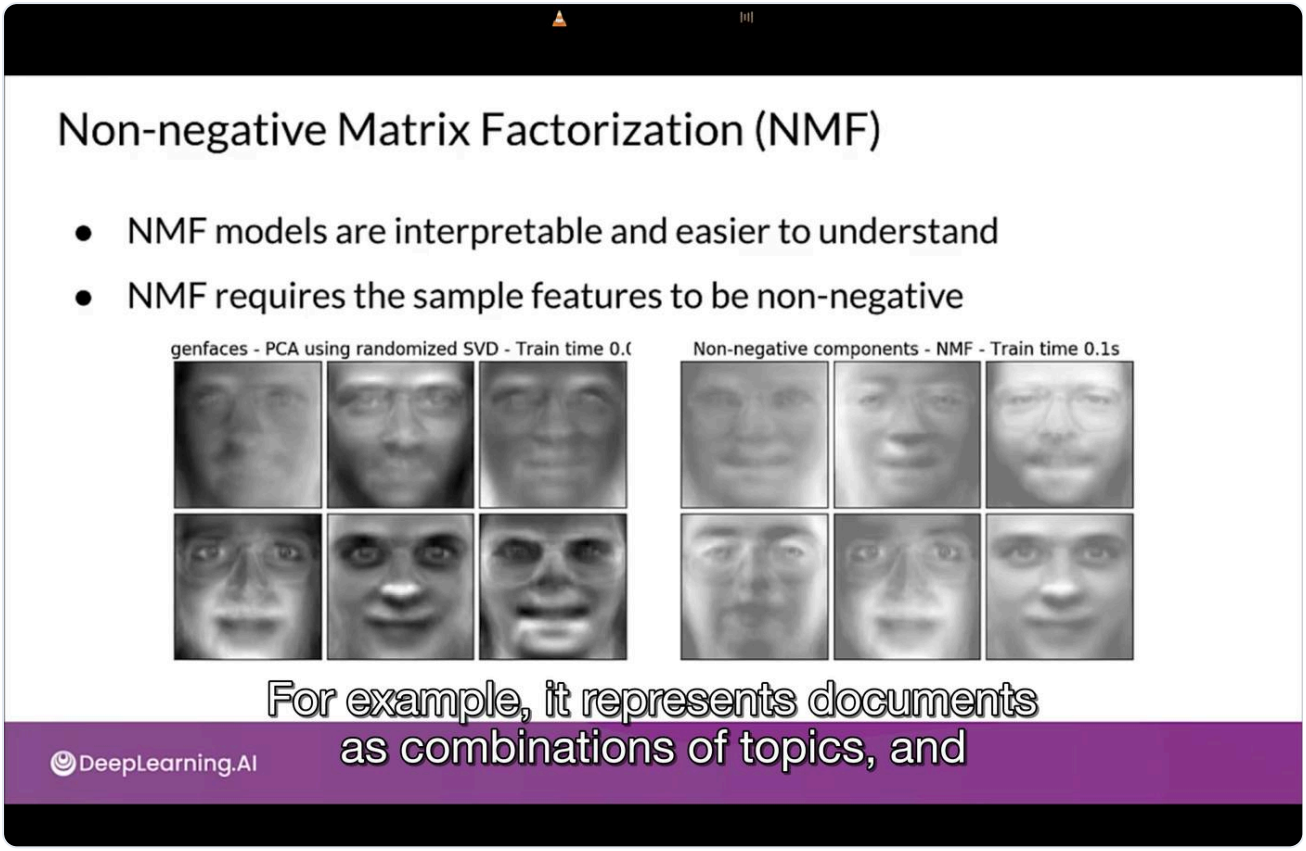

FIGURE 17.9 | Non-negative Matrix Factorization (NMF)

For example, it represents documents as combinations of topics, and images in terms of commonly occurring visual patterns. NMF, like PCA, is a dimensionality reduction technique, in contrast to PCA, however, NMF models are interpretable. This means NMF models are easier to understand and much easier for us to explain to others. NMF can't be applied to every dataset however, it requires the sample features to be non-negative, so the values must be greater than or equal to zero. It has been observed that when carefully constrained, NMF can produce a parts-based representation of the dataset, resulting in interpretable models. This example displays 16 sparse components found by NMF from the images in the Olivetti faces dataset on the right, compared with the PCA eigenfaces on the left.

LECTURE 18

Mobile Inference

Model optimization is another area of focus where you can further optimize performance and resource requirements. The goal is to create models that are as efficient and accurate as possible and to achieve the highest performance at the least cost. Let's look at some advanced techniques for that now. Let's start by looking at some of the issues around the mobile, IoT and embedded applications. Machine learning is increasingly becoming part of more and more devices and products.

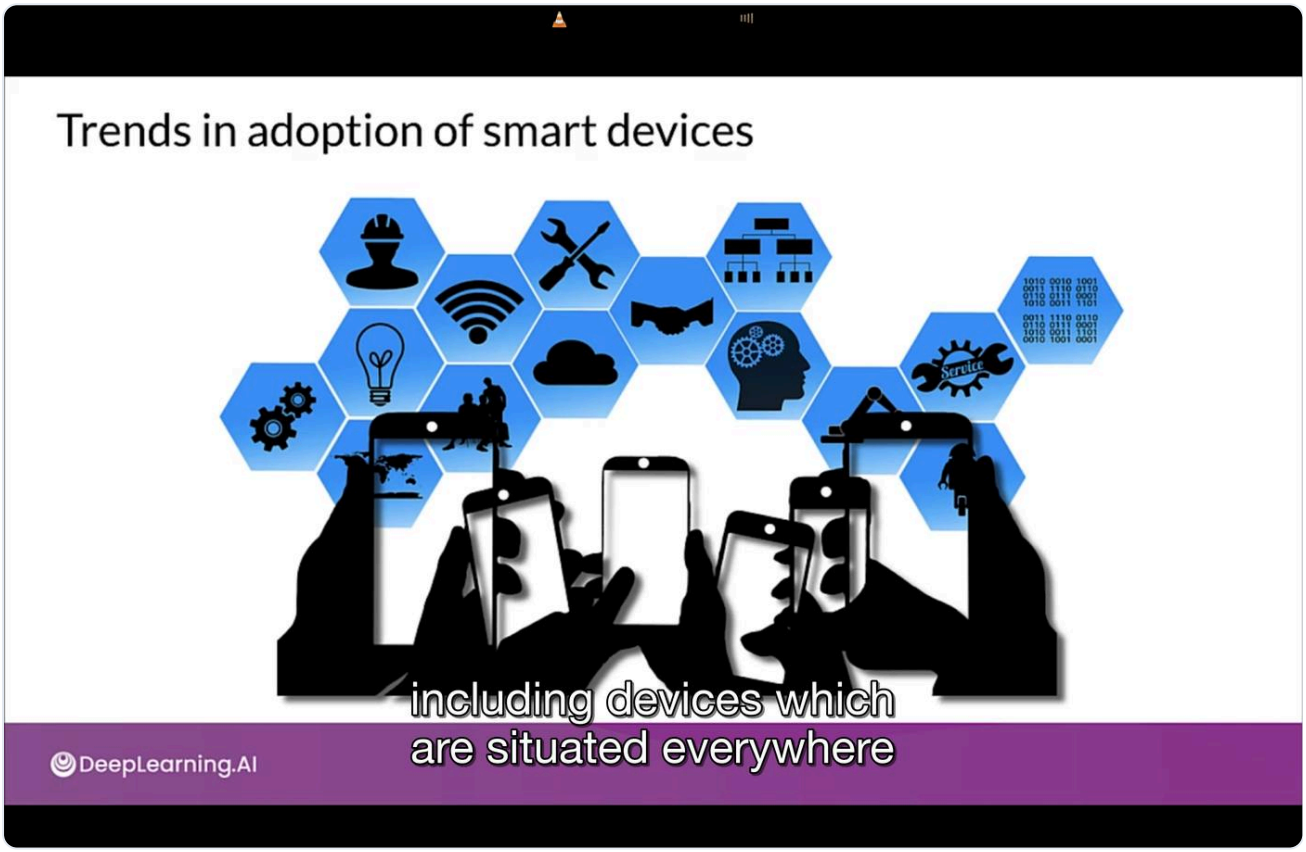

FIGURE 18.1

This includes the rapid growth of mobile and IoT applications, including devices which are situated everywhere from farmers fields to train tracks. Businesses are using the data which these devices generate to train machine learning models to improve their business processes, products, and services. Even digital advertisers spend more on mobile than desktop. There are already billions of mobile and edge computing devices, and that number will continue to grow rapidly in the next decade. McKinsey predicts that by 2025, the overall economic impact of IoT and mobile could reach trillions of dollars, surpassing many sectors like automation of knowledge work or Cloud technology.

As these devices become more and more ubiquitous and powerful, many of the machine learning tasks, which you think of as requiring months of high powered compute time, will become part of more and more fairly common devices. Now let's look at some of the reasons those trends

occur in the first place. Traditionally, you can think of deploying machine learning models in the Cloud. This requires a server to run inference and return the results. But with the advance of machine learning research for applications on lower power devices, this processing can be offloaded to a device and run locally. This enables more opportunities for including machine learning is part of a device's core functionality.

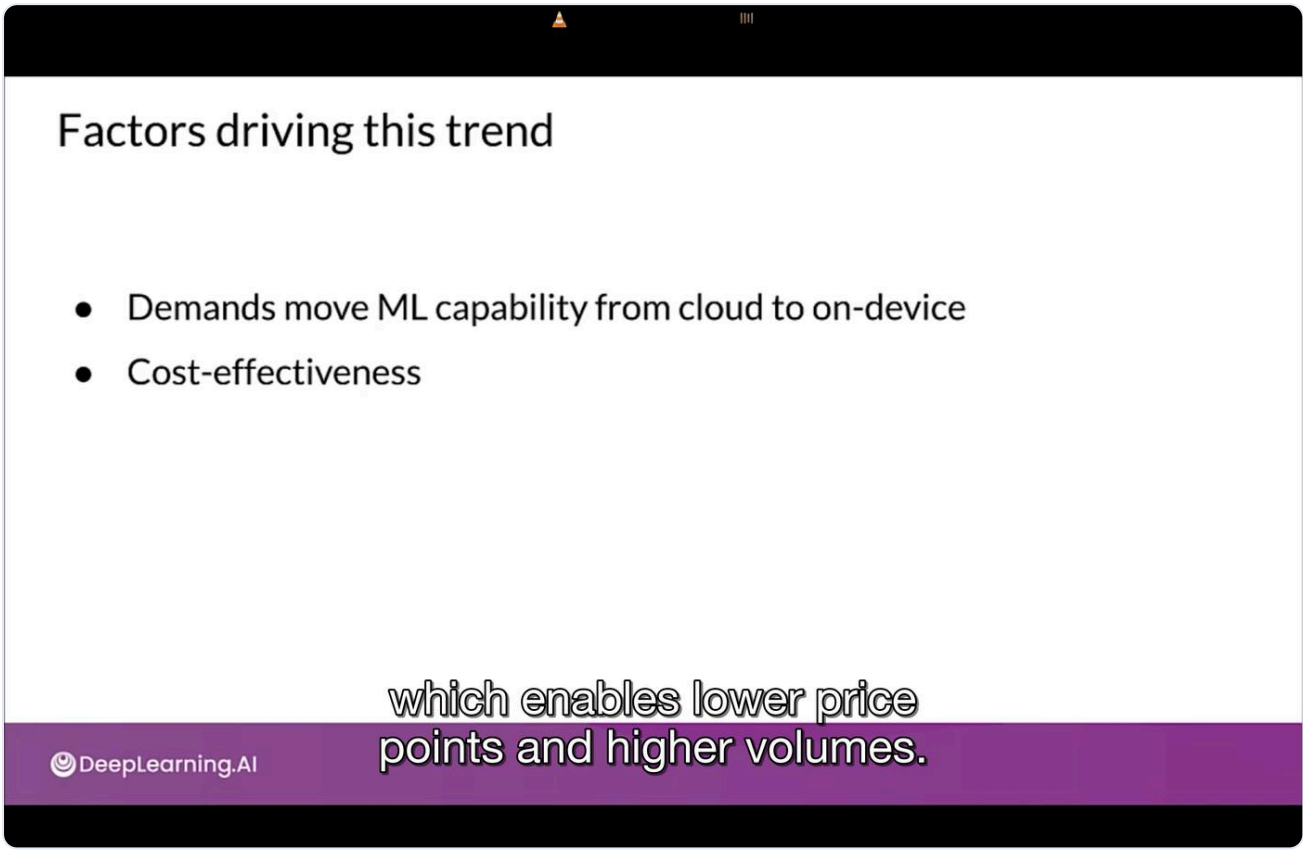

FIGURE 18.2

Moreover, the hardware costs for these devices continues to fall, which enables lower price points and higher volumes.

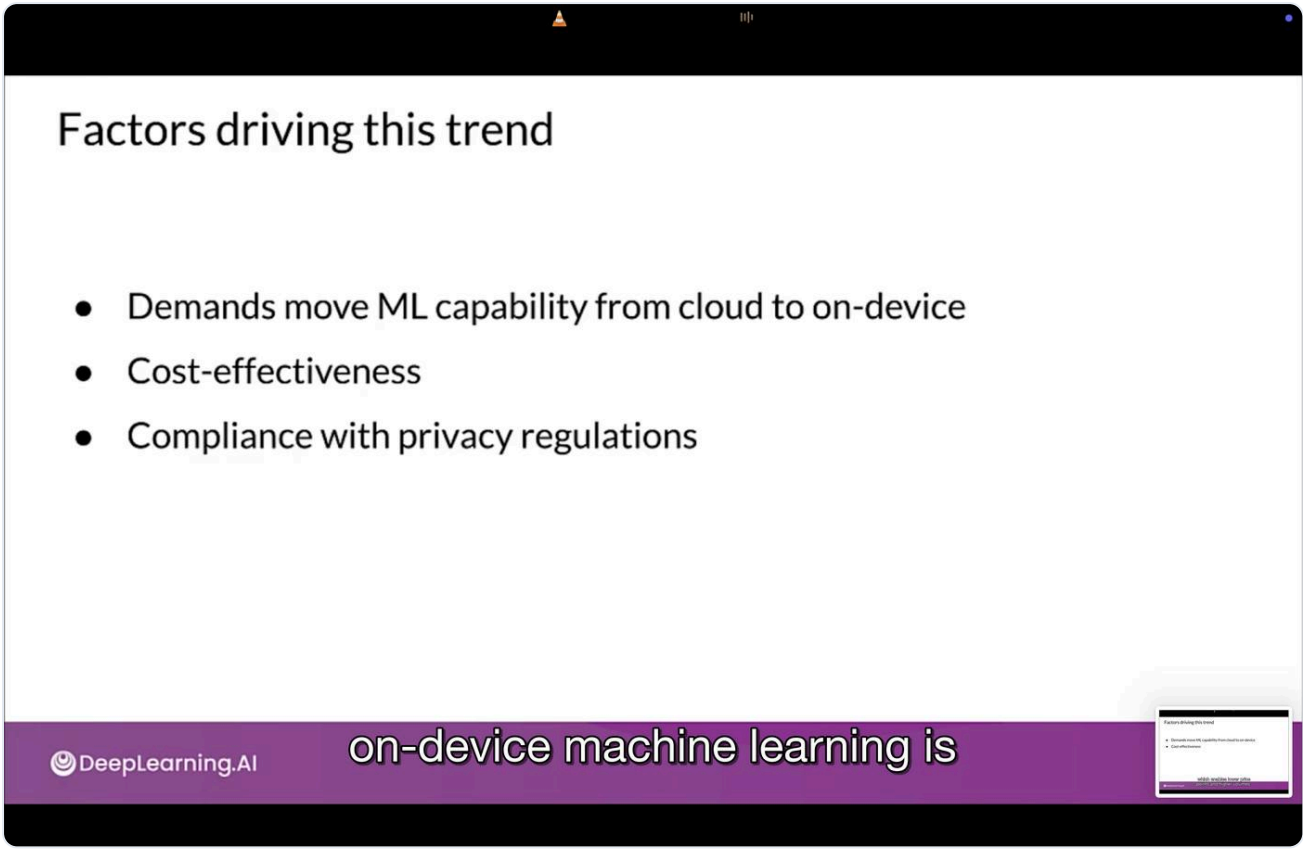

FIGURE 18.3

A key aspect of on-device machine learning is that in most cases it ensures greater compliance with privacy regulations by keeping user data on the device. How should you deploy your models so that they can be used to generate real value? If you host them on a server, the mobile or IoT device needs to be connected so that it can make a network request. Another option is to embed the model on a mobile device directly. In your case, can you always rely on the device to have a network connection?

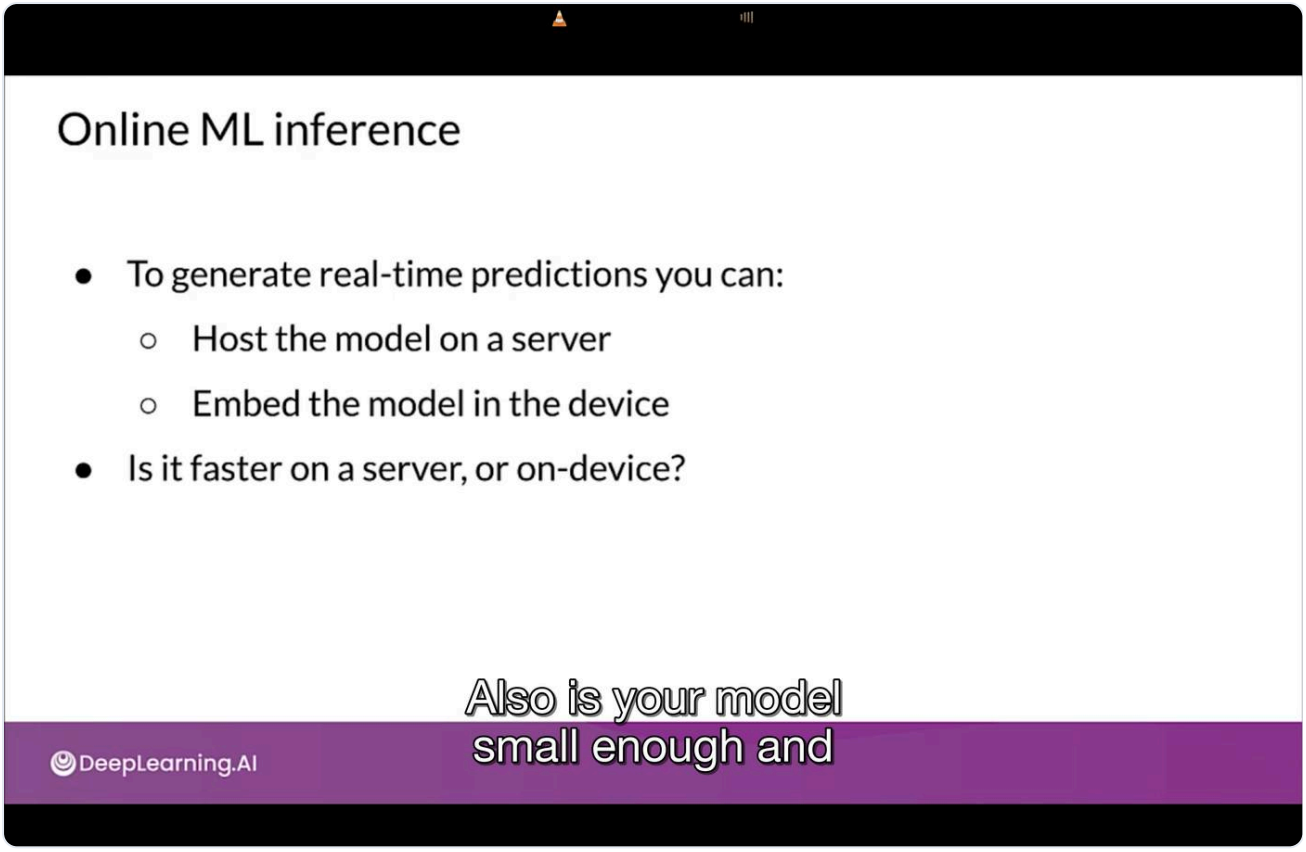

FIGURE 18.4

Also is your model small enough and fast enough to perform inference on the device?

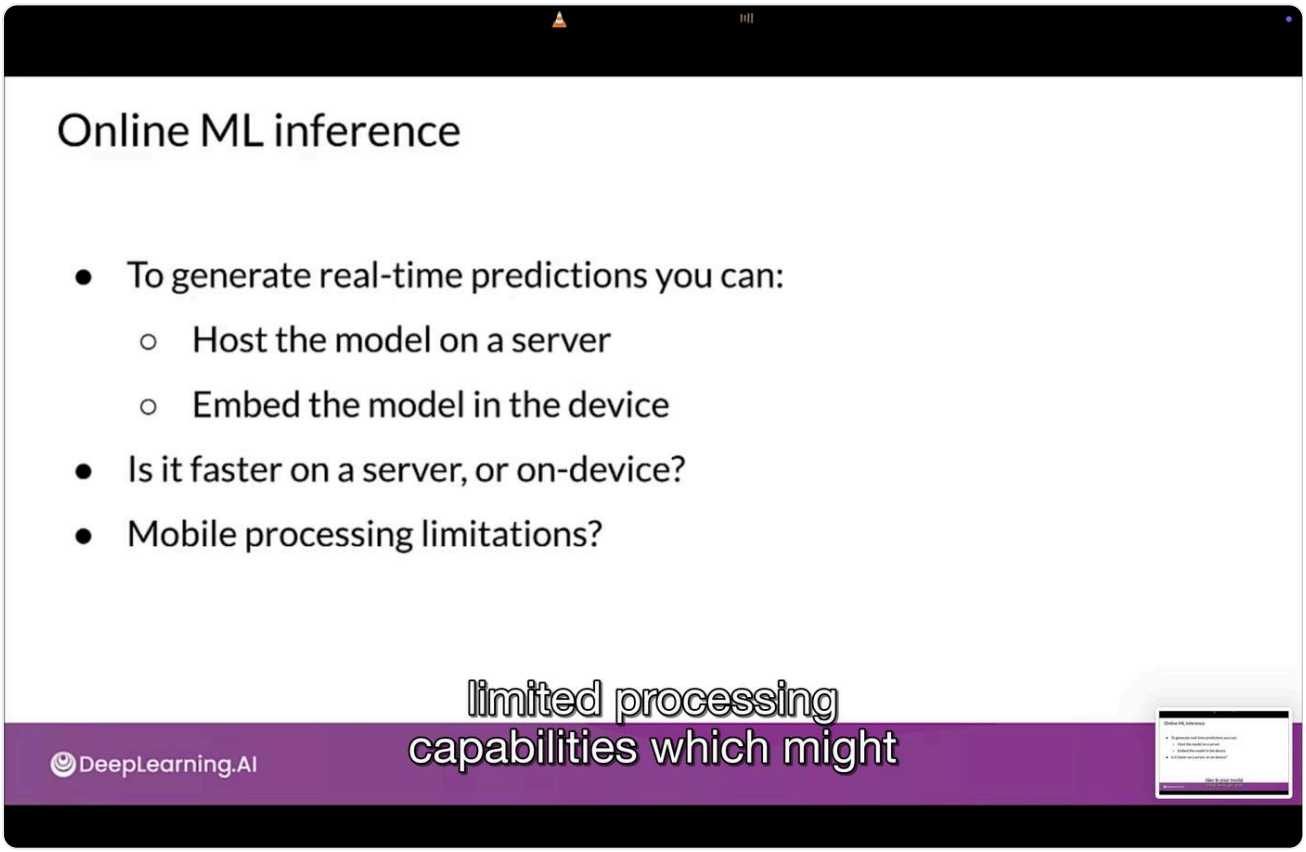

FIGURE 18.5

Additionally, mobile devices offer limited processing capabilities which might affect which types of models you can embed in them. Furthermore, does the device have all the access it needs to the data that it needs? Or does it need things like historical data that are only available on a server? Suppose you're trying to build an app that applies different effects to photos. In a scenario where the models being hosted on a server, the app needs to first send the photo to the server, and then the server feeds the picture through a model to apply the desired effect, and a few seconds later, it sends the modified image back to the app.

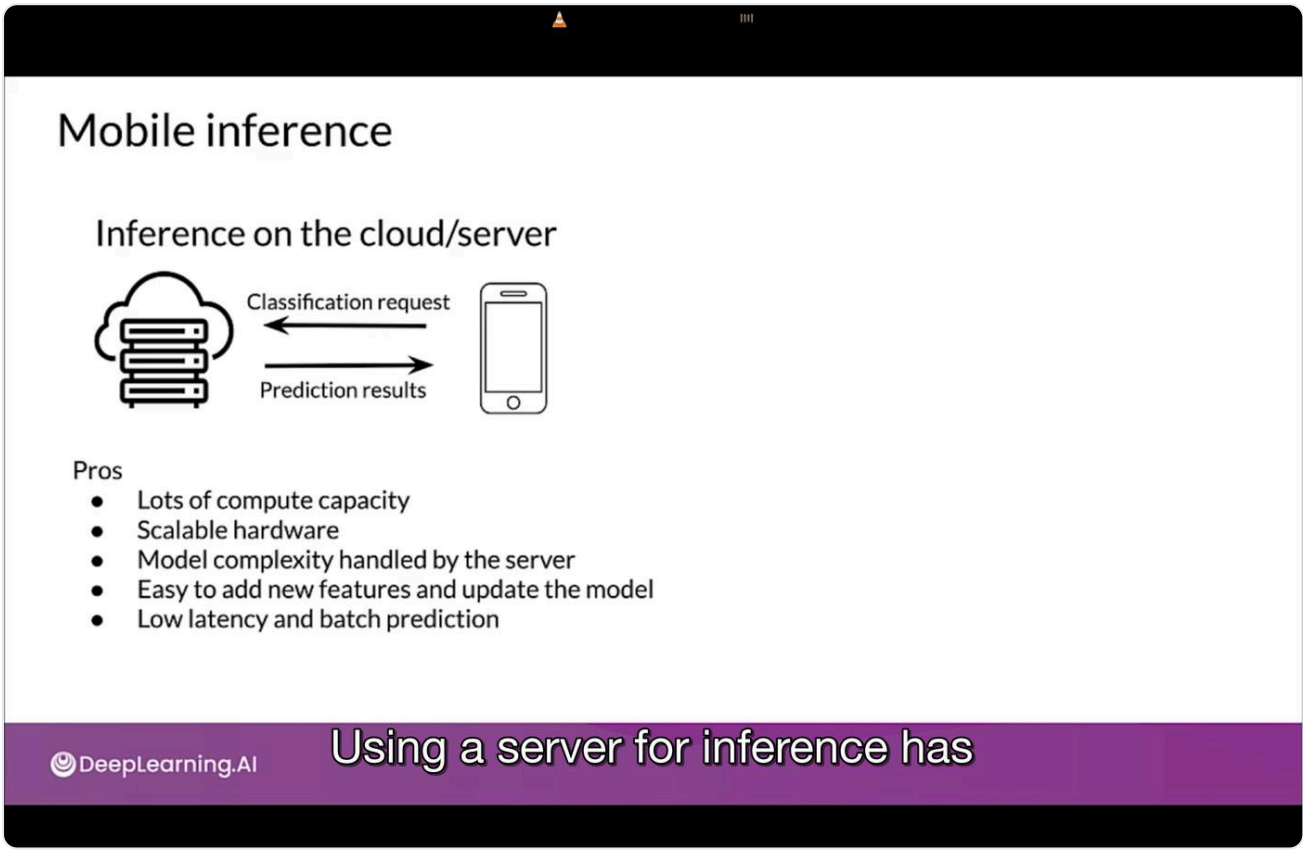

FIGURE 18.6 | Mobile Inference

Using a server for inference has the advantage that it keeps the mobile app simple. The server encapsulates all of the model complexity. This means that you can update the model or add new features anytime you want. To deploy the improved model, you update the model on the server. That means that you probably don't have to update the app itself, unless you need to change the request that the app sends.

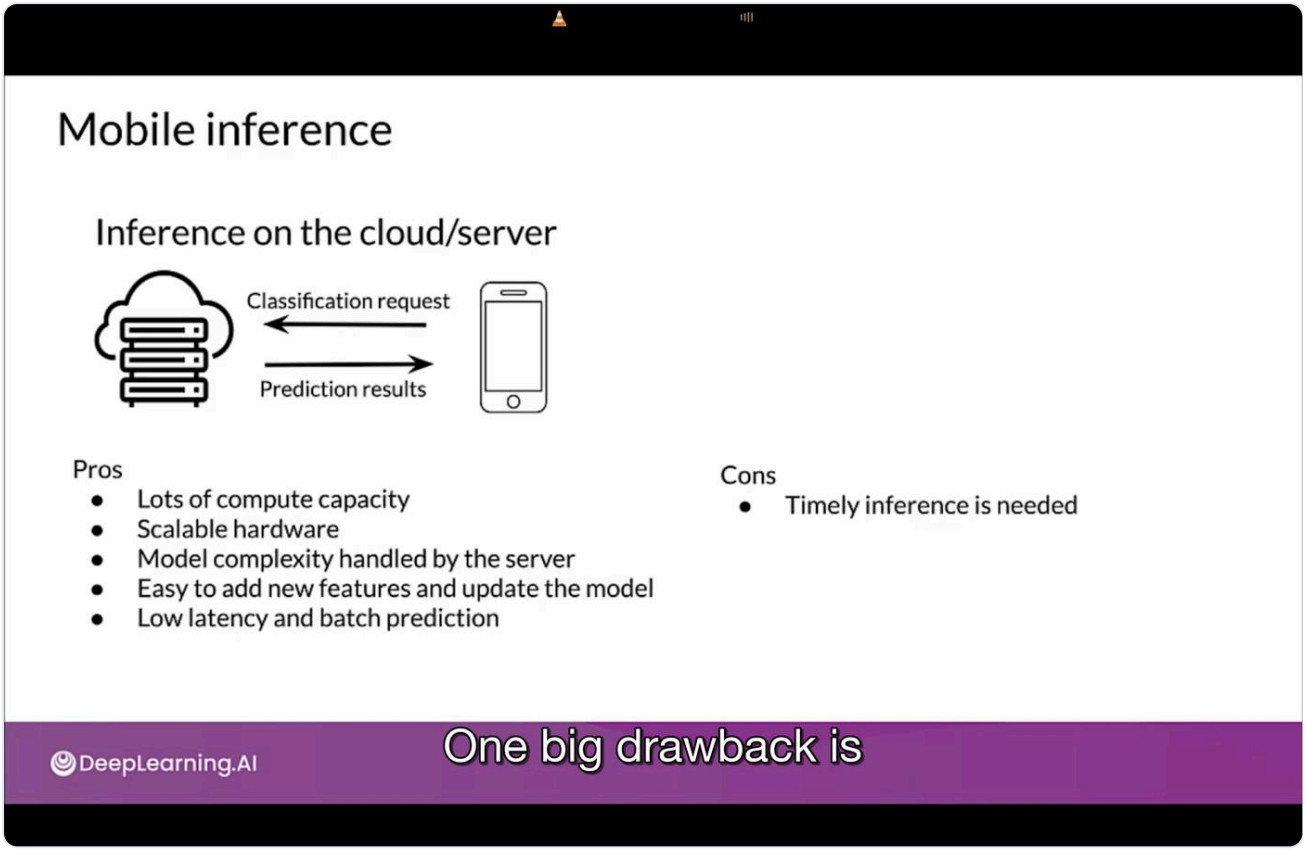

FIGURE 18.7 | Mobile Inference

One big drawback is the timely inference is a strong requirement in this setting. In case of on-device inference, you load the trained model into the app. Since the model runs in the app, you don't need to send a request over the Internet and wait for a reply. Instead, the prediction happens fast and you don't need a network connection.

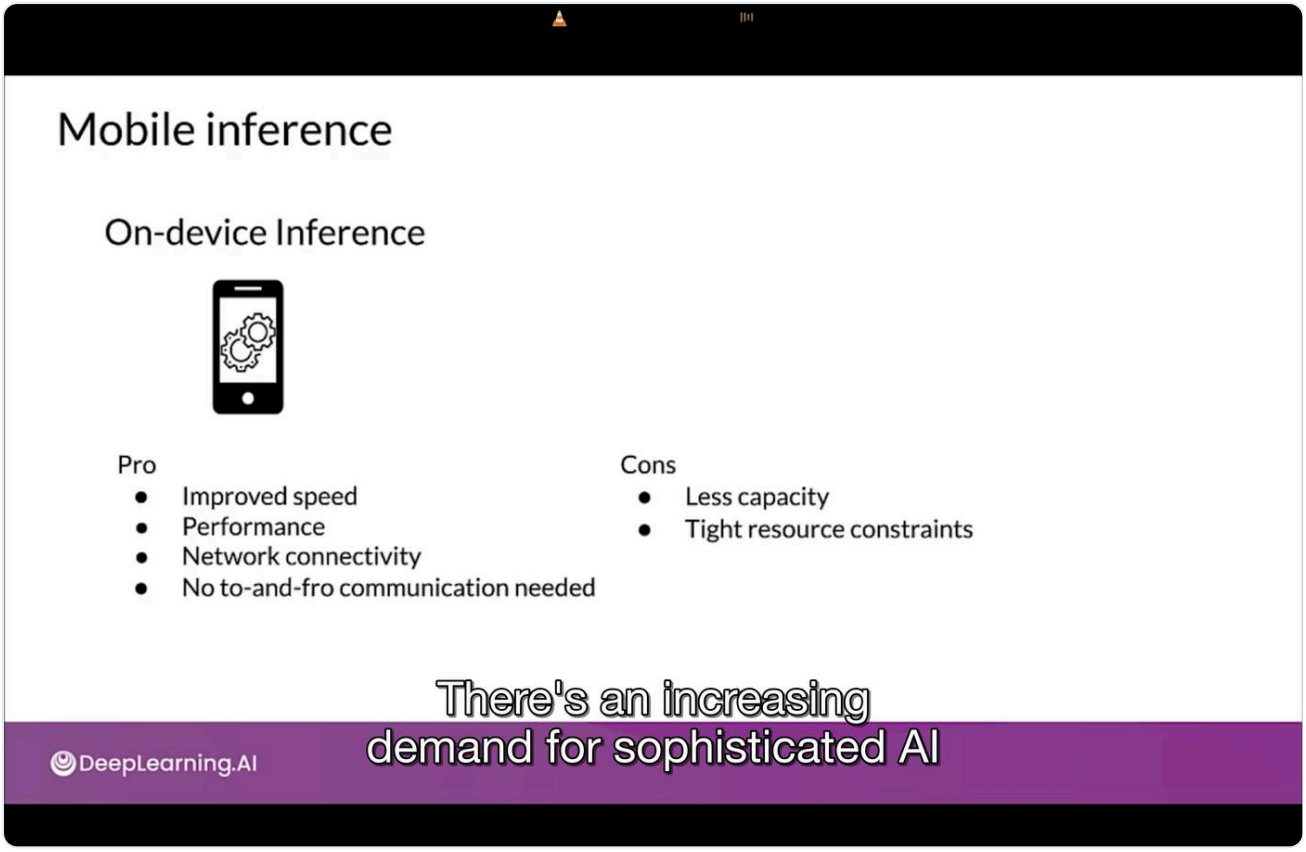

FIGURE 18.8 | Mobile Inference

There's an increasing demand for sophisticated AI enabled services like image and speech recognition, natural language processing, visual search, and personalized recommendations. At the same time, datasets are growing. Networks are becoming more complex. Privacy is increasingly becoming an issue and latency requirements are tightening to meet user expectations. All of these trends influence the choice of where to generate predictions from your trained models, which in turn affects the architecture and complexity of the models that you train.

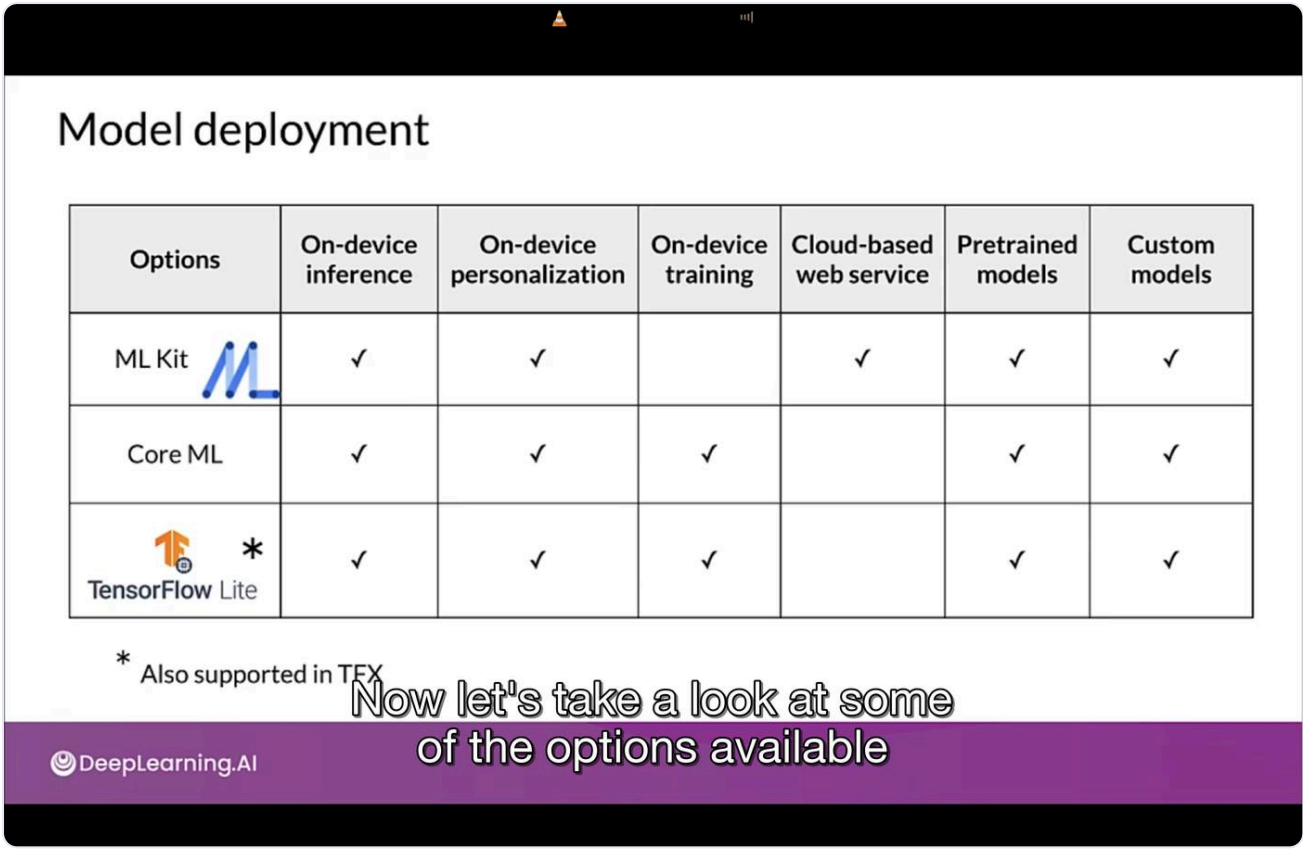

FIGURE 18.9

Now let's take a look at some of the options available today to deploy models to mobile apps. ML Kit for Firebase offers ready to use APIs. You can also deploy your own TensorFlow Lite models if you don't find a base API that covers your use case. It targets mobile platforms and uses TensorFlow Lite, the Google Cloud Vision API and Android Neural Networks API to provide on-device machine learning, such as facial recognition, barcode scanning, and object detection among others. ML Kit gives you both on-device and Cloud APIs, meaning you can also use the APIs when there's no network connection. The Cloud based APIs make use of the Google Cloud Platform.

With ML Kit, you can upload models through the Firebase Console and let the service take care of hosting and serving the models to your apps users. Another advantage is that since ML Kit is available through Firebase, it's possible to take advantage of the broader Firebase platform. With Core ML, you can build your model or use a pretrained model. To use your model, you first need to create a model using third-party frameworks. Then you convert your model to the Core ML model format. Supported frameworks includes Scikit-learn, Keras, Caffe, and XGBoost. There are also some pre-trained models ready for use. TensorFlow Lite was developed by Google and has APIs for many programming languages including Java, C++, Python, Swift, and Objective-C.

It's optimized for on-device applications and provides an interpreter tuned for on-device machine learning. Custom models are converted to TensorFlow Lite format and their size is optimized to increase efficiency. TensorFlow Lite also supports optimizing models for IoT and embedded applications, for devices with as little as 20k of memory. TensorFlow Lite models can be trained and evaluated in TFX pipelines, which is important when the model will become part of a production, product, or service.

LECTURE 19

Why Quantize Neural Networks?

Now let's discuss some techniques for optimizing your models. Especially for deployment scenarios, such as mobile and IoT, where the capabilities of the device are extremely limited compared to running on a server or in the Cloud. However, before doing so, let's clarify that these techniques can benefit any model regardless of where it is deployed since they reduce the compute resources required to serve the model.

FIGURE 19.1

FIGURE 19.2

FIGURE 19.3

Quantization involves transforming a model into an equivalent representation that uses parameters and computations at a lower precision. This improves the model's execution performance and efficiency, but it can often result in lower model accuracy. Let's use an analogy to understand this better. Think of an image. As you might know, a picture is a grid of pixels where each pixel has a certain number of bits. Now if you try reducing the continuous color spectrum of real-life to discrete colors, we're quantizing or approximating the image.

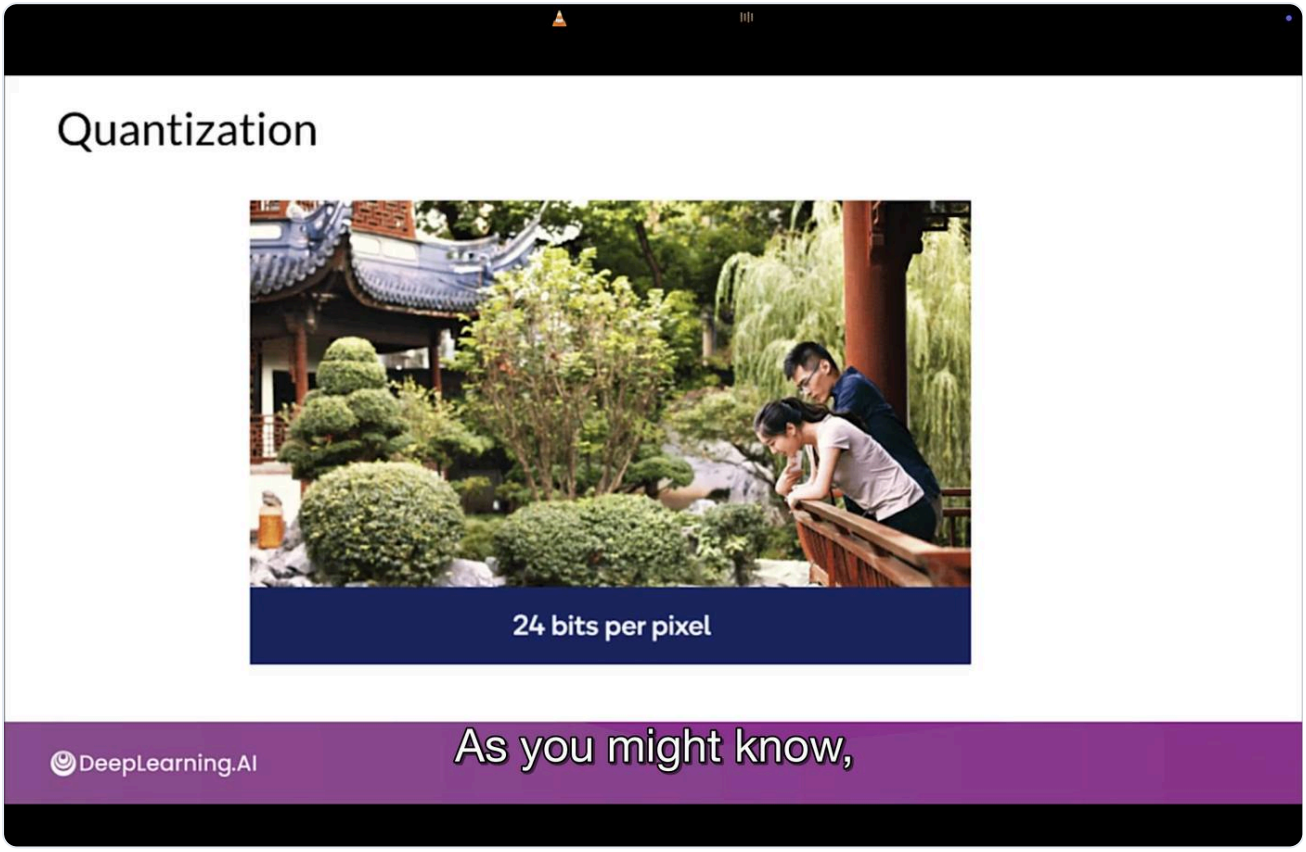

FIGURE 19.4

In this animation, you can see that a black and white image could be represented with one bit per pixel, while a typical picture with color has 24 bits per pixel. Quantization, in essence, lessens or reduces the number of bits needed to represent information. However, you may notice that as you reduce the number of pixels beyond a certain point, depending on the image, it may get harder to recognize what that image is. Neural network models can take up a lot of this space. For example, AlexNet which requires around 200 megabytes of disk space. Nearly all of that size is taken up with the weights for the neural connections, as there are often many millions of these in a single model.

Because they're all slightly different floating-point numbers. Simple compression-like zipping doesn't compress them well unless we make models less dense. Most straightforward motivation for quantization is to shrink file sizes. For mobile apps, especially it's often impractical to store a 200-megabyte model on the phone just to run a single app. Compressing higher precision models is necessary.

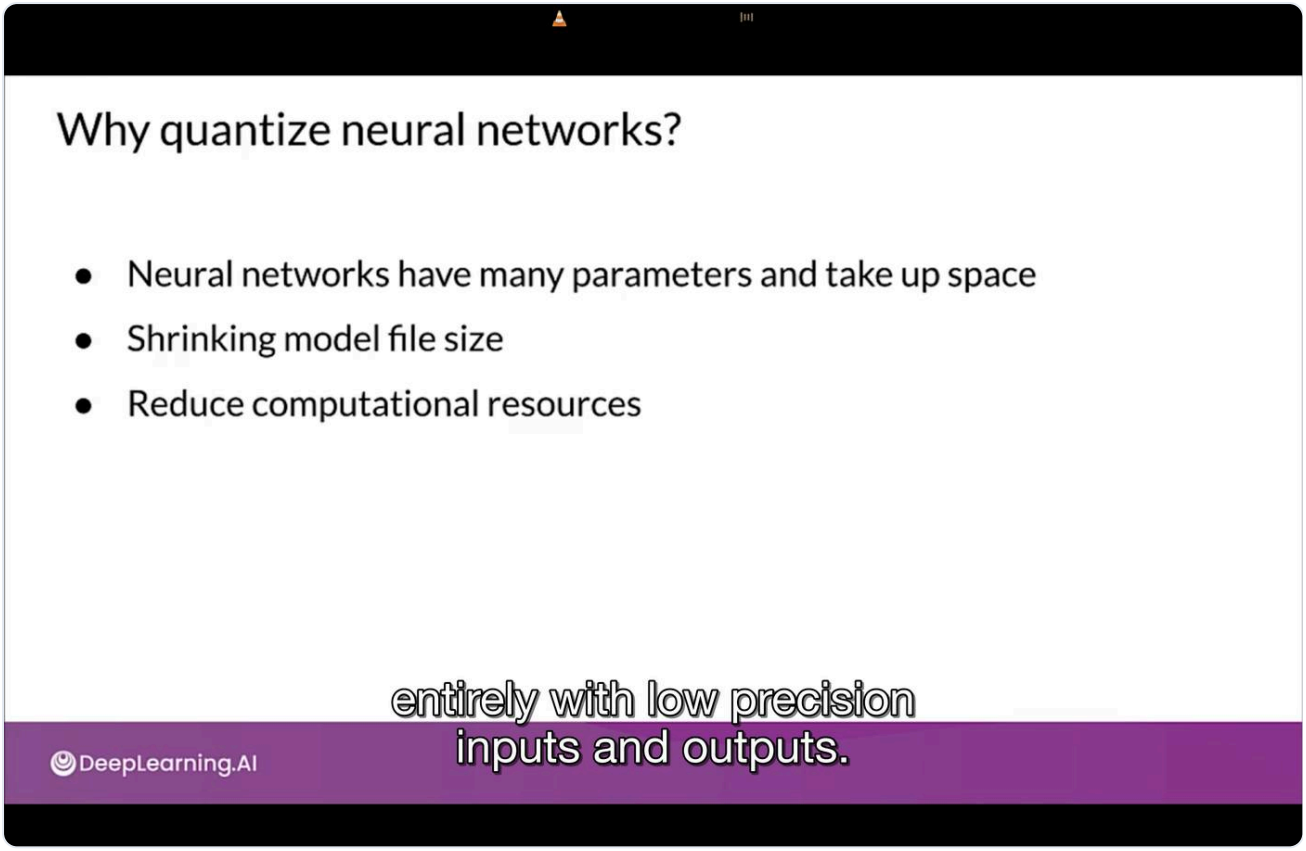

FIGURE 19.5 | Why Quantize Neural Networks?

Another reason to quantize is to reduce the computational resources that you need to do inference calculations by running them entirely with low precision inputs and outputs. This is a lot more difficult since it requires changes everywhere you do calculations, but it offers potential rewards.

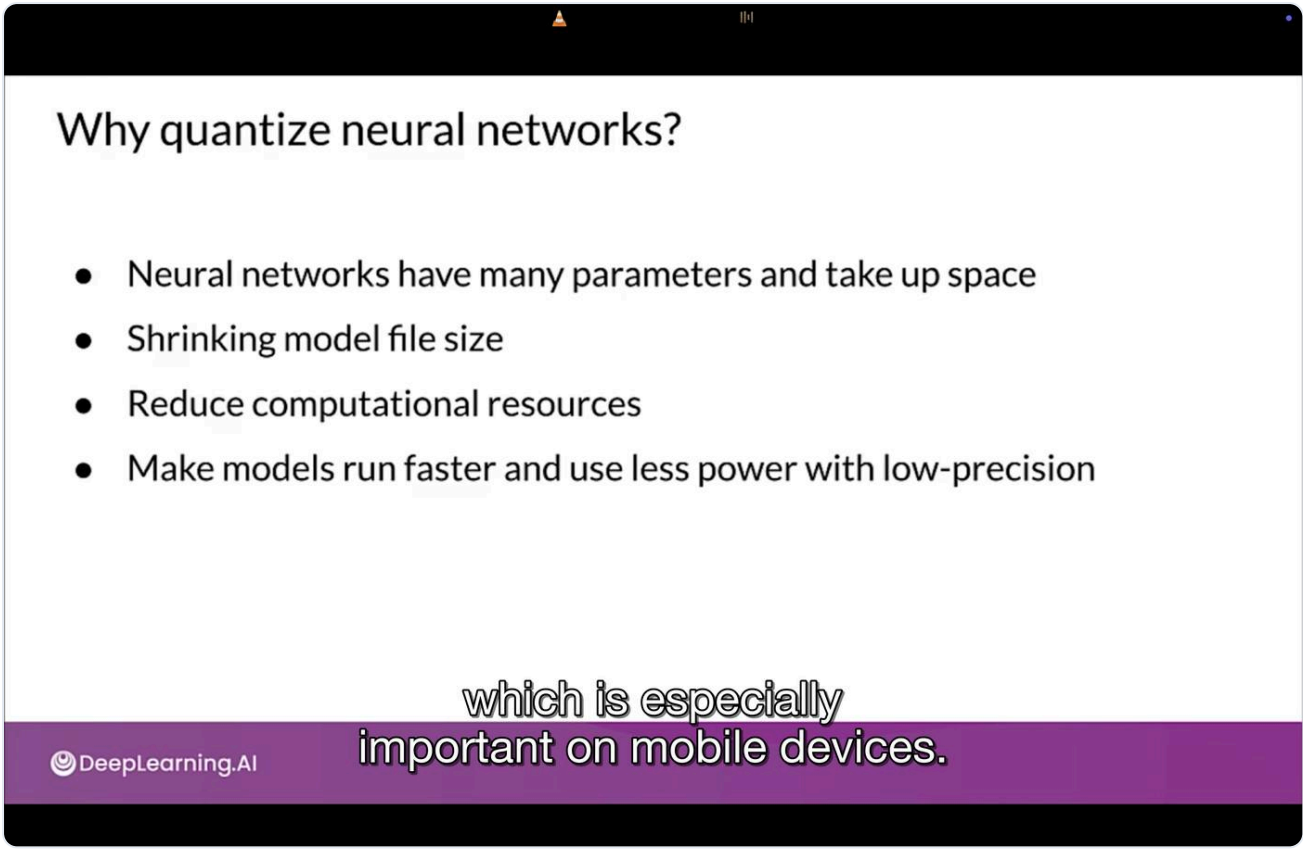

FIGURE 19.6 | Why Quantize Neural Networks?

Doing this will help you run your models faster and use less power, which is especially important on mobile devices. It also opens the door to a lot of embedded systems that can't run floating-point efficiently, enabling many applications in the IoT world. Now, let's take a look at what quantization means. MobileNets are a family of architectures that achieve a state-of-the-art trade-off between on-device latency and ImageNet classification accuracy. A recent publication demonstrated how integer-only quantization could further improve the trade-off on common hardware.

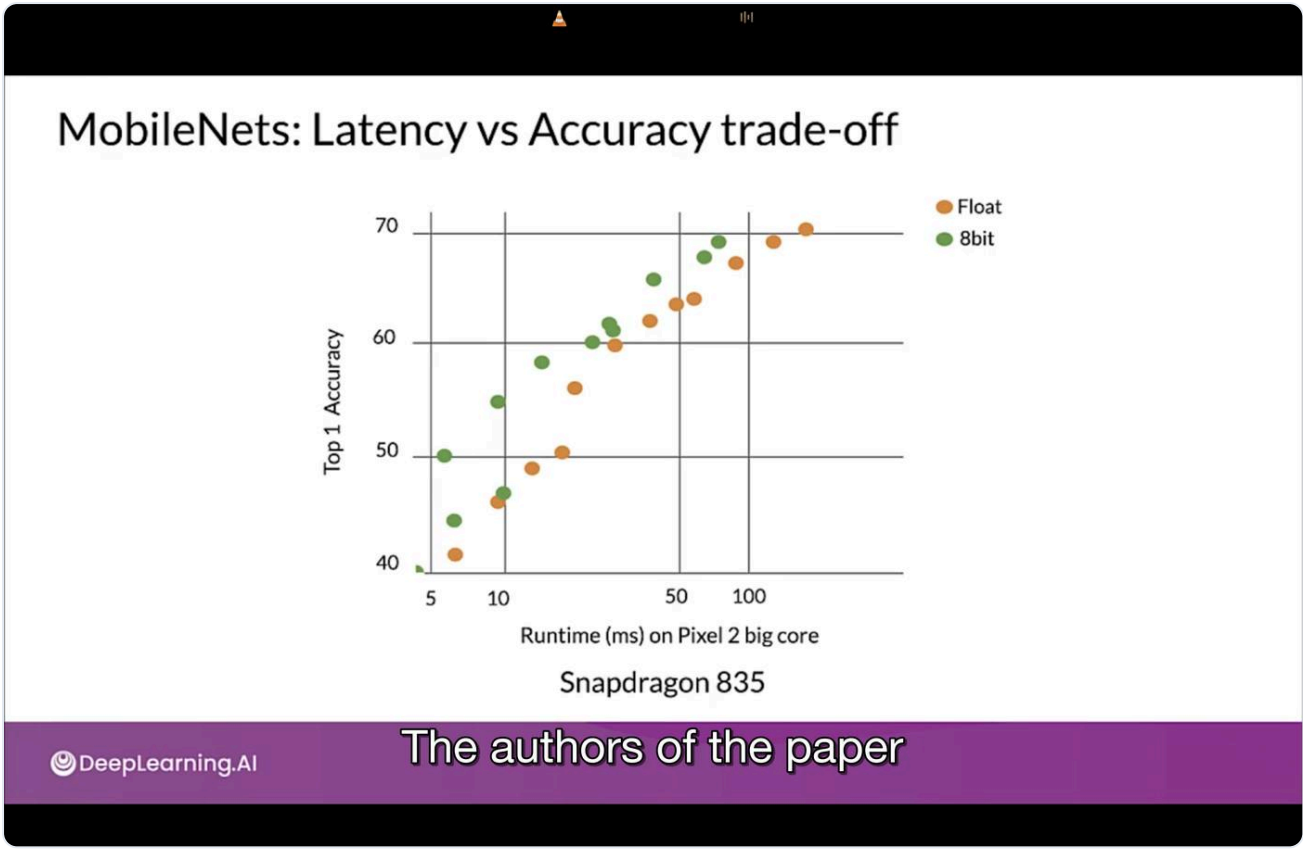

FIGURE 19.7 | MobileNets: Latency vs Accuracy Trade-off

The authors of the paper benchmarked the MobileNet architecture with varying depth multipliers and resolutions on ImageNet on three types of Qualcomm cores. This plot is for the Snapdragon 835 chip. You can see that for any given level of accuracy, latency time is lower for the 8-bit version of the model. Arithmetic with lower bit depth is faster, assuming that the hardware supports it. Even though floating-point computation is no longer slower than integer on modern CPUs, operations with 32-bit floating-point will almost always be slower than, say, eight-bit integers. Moving from 32 bits to eight bits, we usually get speedups of 4x reduction in memory.

Lighter deployment models mean they have less storage space and are easier to share over smaller bandwidths and easier to update.

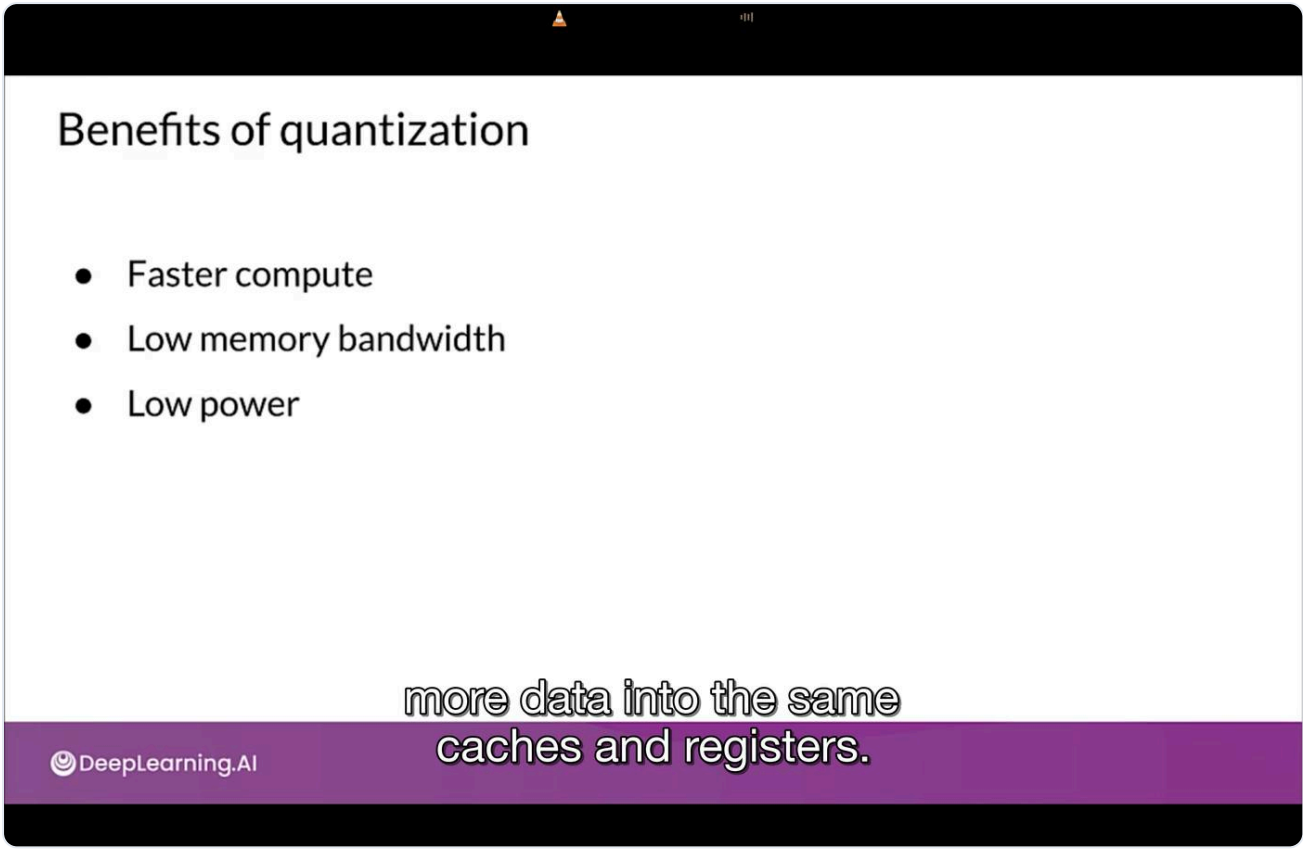

FIGURE 19.8

Lower bit depths also mean we can squeeze more data into the same caches and registers. This makes it possible to build applications with better caching capabilities that reduce power usage and run faster.

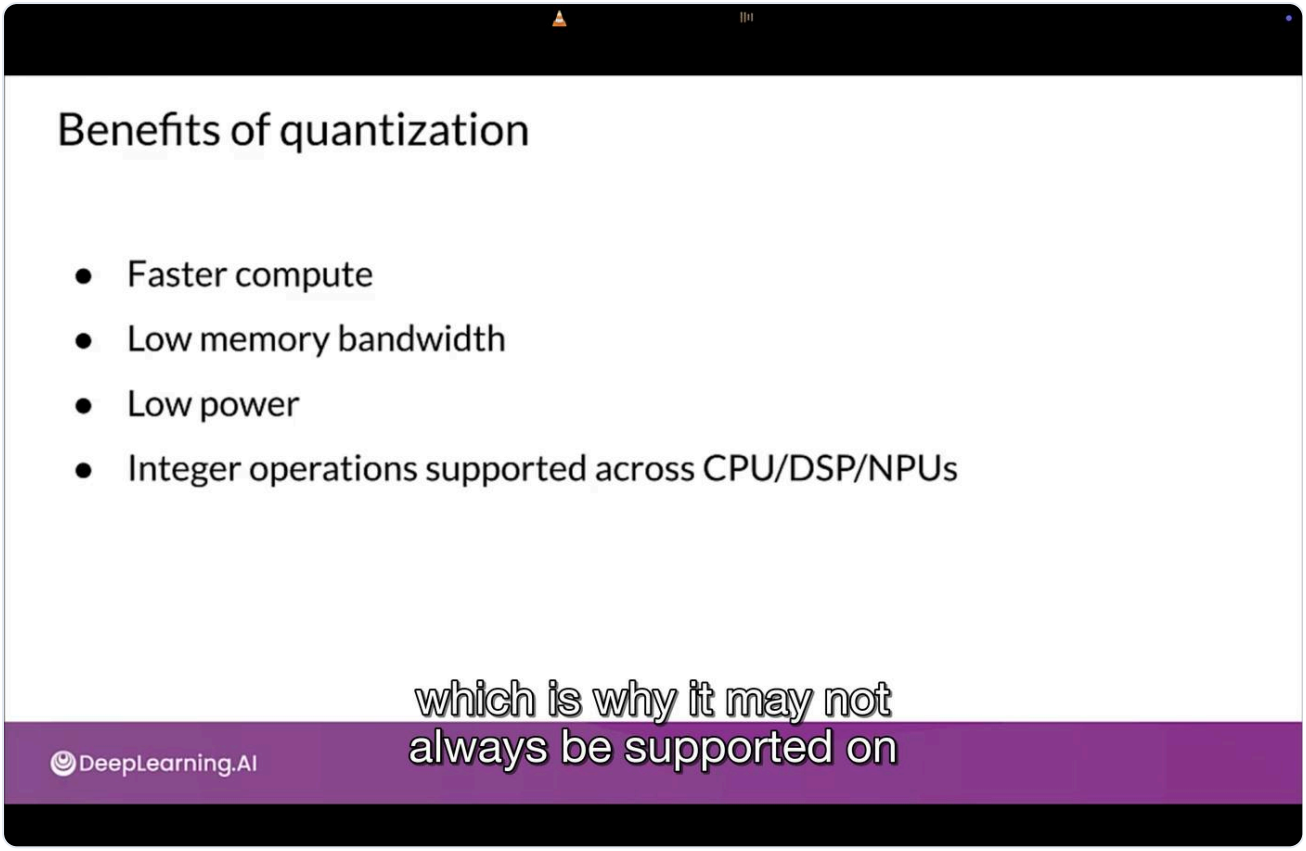

FIGURE 19.9

Floating-point arithmetic is hard, which is why it may not always be supported on microcontrollers and on some ultra low-powered embedded devices, such as drones, watches, or IoT devices. Integer support, on the other hand, is always readily available. Neural networks consist of activation nodes, the connections between the nodes, a weight parameter associated with each connection, and a bias term.

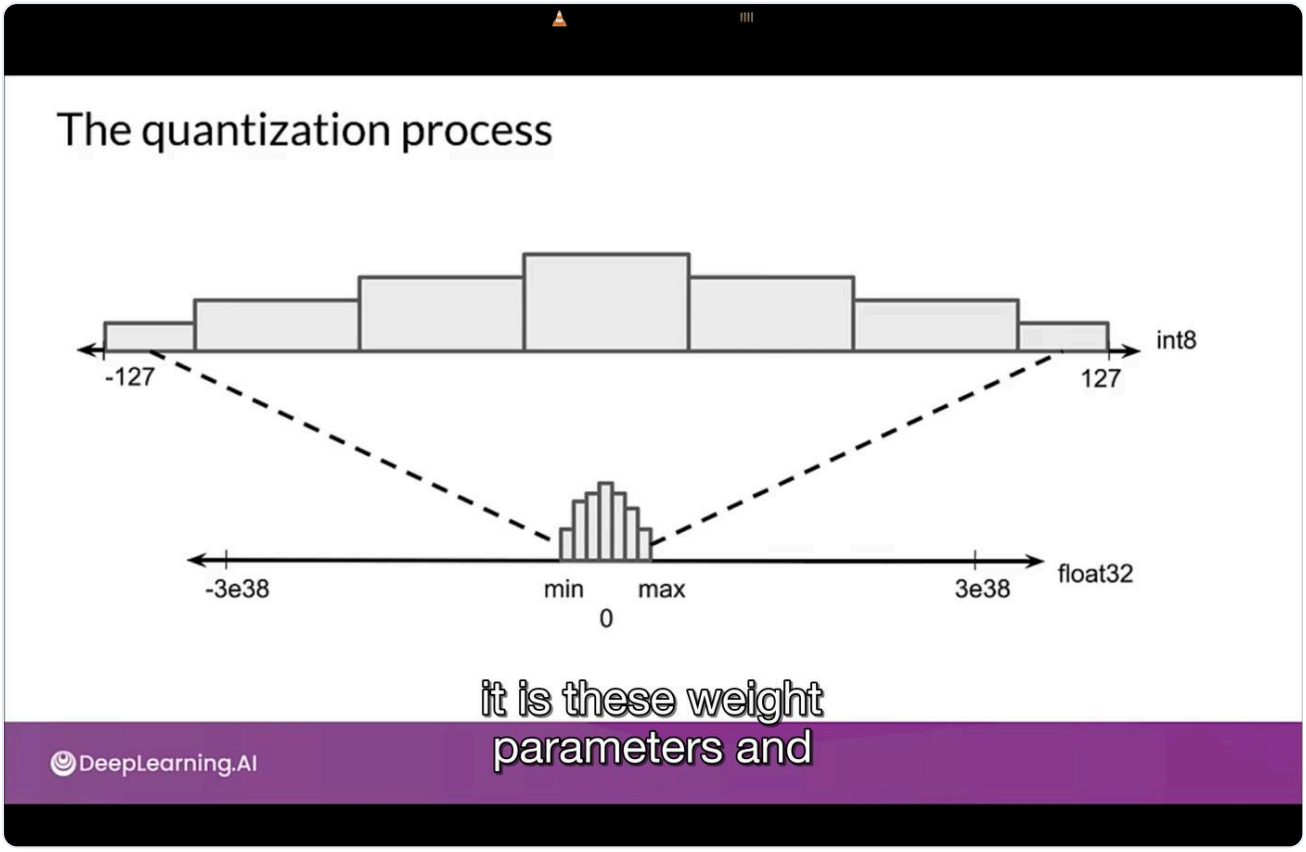

FIGURE 19.10 | The Quantization Process

When it comes to quantizing these networks, it is these weight parameters and activation node computations that we're trying to quantize. Quantization squeezes a small range of floating-point values into a fixed number of information buckets. As you can see in this diagram. This process is lossy in nature. But the weights and activations of a particular layer often tend to lie in a small range, which can be estimated beforehand. This means we don't need the ability to store a range in the same data type, allowing us to concentrate our precious fewer bits within a smaller range. Say negative three to positive three. As you might imagine, it will be crucial to accurately know this smaller range.

If done right, quantization causes only a small loss of precision, which usually doesn't change the output significantly. Now, let's see what parts of a model are affected after applying quantization. One of them could be static parameters like the weights of layers, and others could be dynamic ones like activations inside networks. You could also have transformations like adding, modifying, or removing operations, coalescing different operations, and so on.

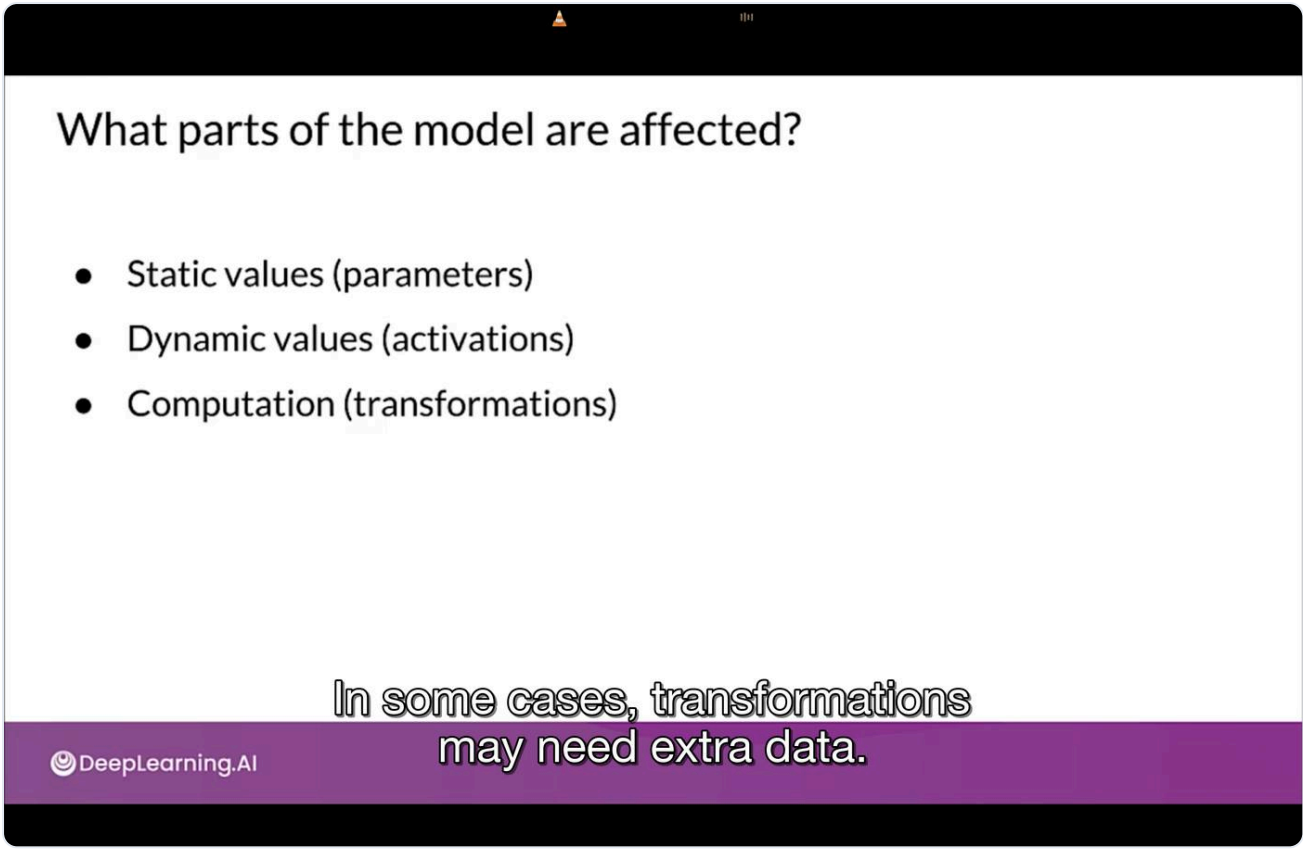

FIGURE 19.11 | What Parts of the Model Are Affected?

In some cases, transformations may need extra data. You'll see how this is the case in one of the techniques of quantization where some unlabeled data is used to determine scaling parameters. Optimizations can often result in changes in model accuracy, which must be considered during the application development process. The accuracy changes depend on the individual model and data being optimized and are difficult to predict ahead of time. Generally, models that are optimized for size and latency will lose some amount of accuracy. Depending on your application, this may or may not impact your user's experience. In rare cases, certain models may actually gain some accuracy as a result of the optimization process.

In terms of interpretability, there are some effects which may be imposed on the model after quantization.

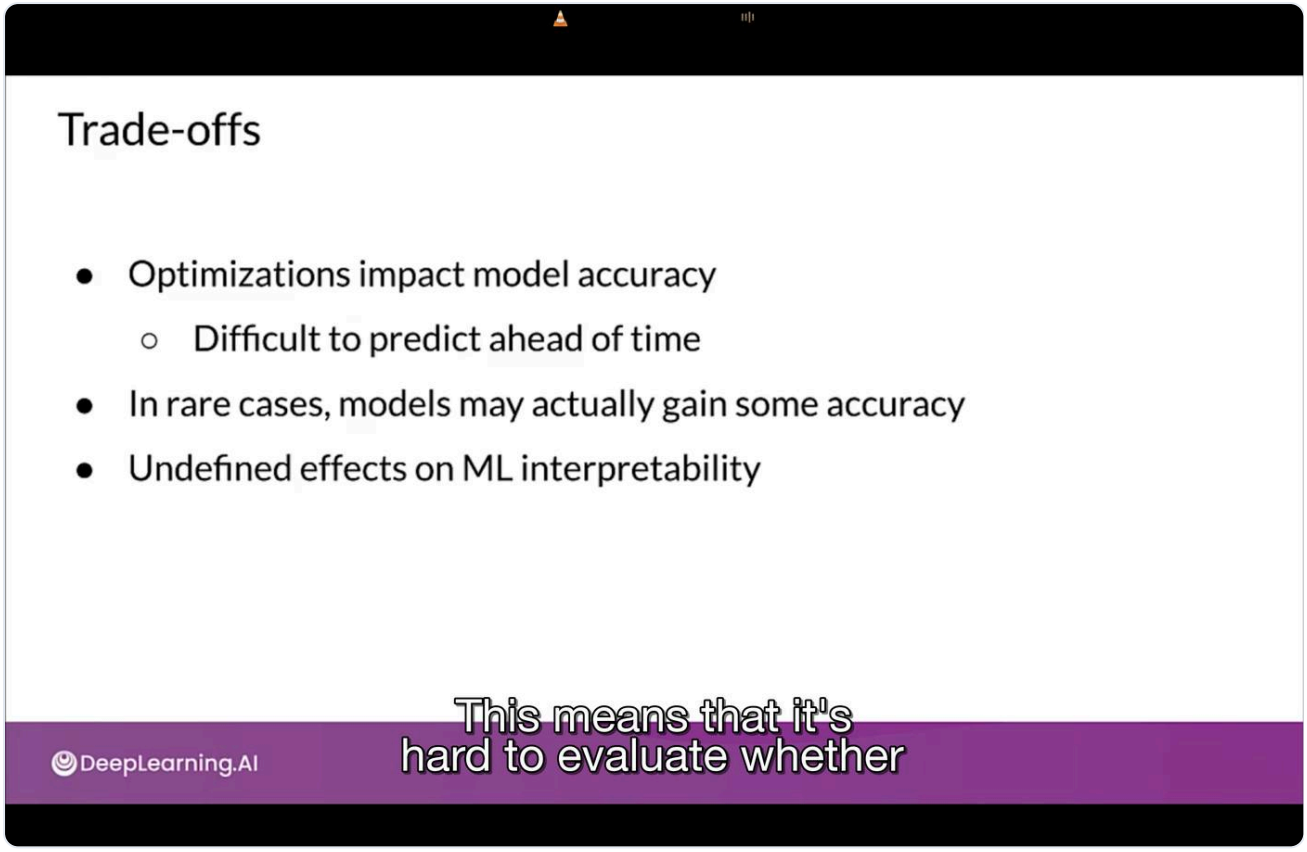

FIGURE 19.12

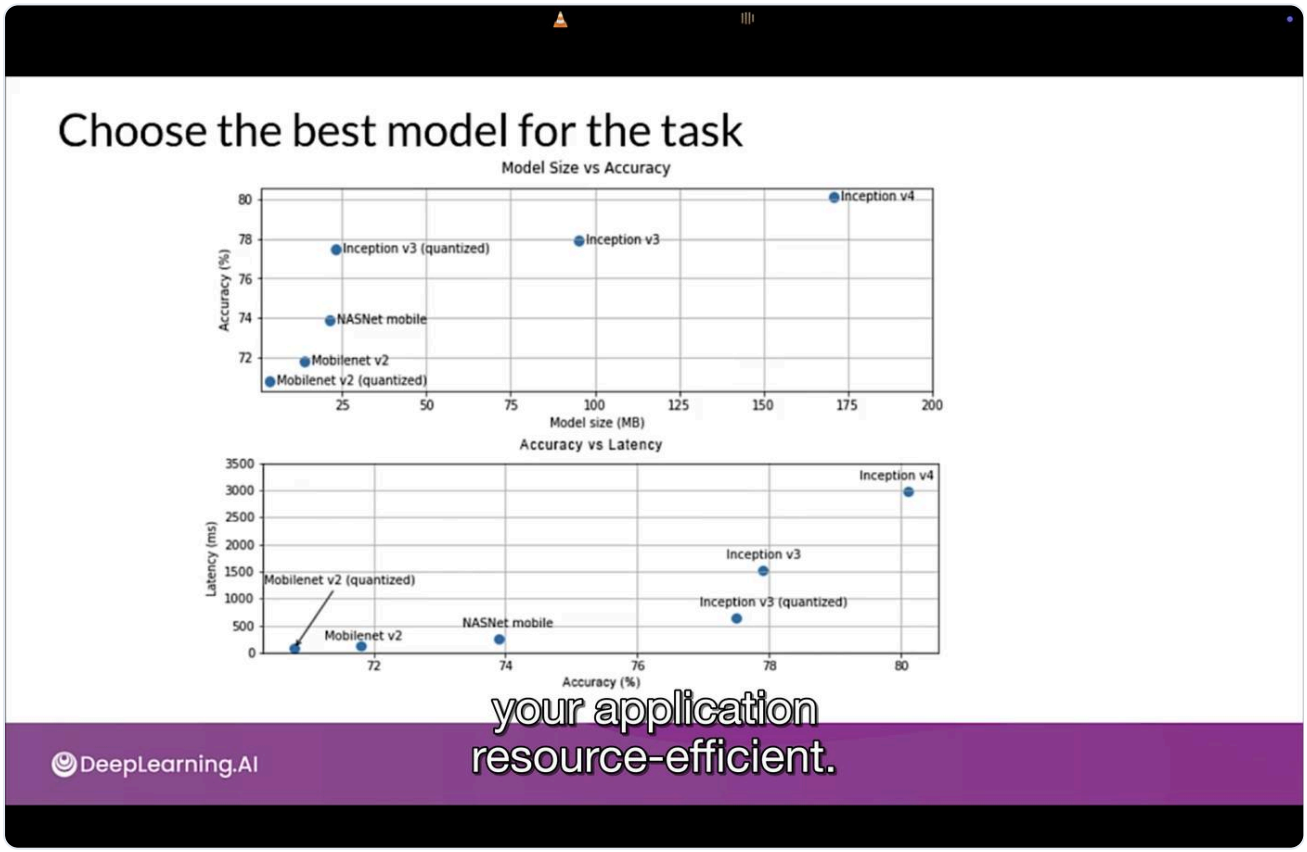

FIGURE 19.13 | Model Size vs Accuracy

This means that it's hard to evaluate whether transforming a layer was going in the right or wrong direction. Mobile and embedded devices have limited computational resources, so it's important to keep your application resource-efficient. Depending on the task, you will need to make a trade-off between model accuracy and model complexity. If your task requires high accuracy, then you may need a large and complex model. For tasks that require less precision, it's better to use a smaller, less complex model. Because they not only use less disk space in memory, but they are also generally faster and more energy-efficient. For example, these graphs show accuracy and latency trade-offs for some common image classification models.

One example of models optimized for mobile devices are MobileNets, which are optimized for mobile vision applications. Once you've selected a candidate model that is right for your task, it's a good practice to profile and benchmark your model.

LECTURE 20

Post Training Quantization

You can do quantization during training or after the model has been trained. Let's look, first, at post-training quantization.

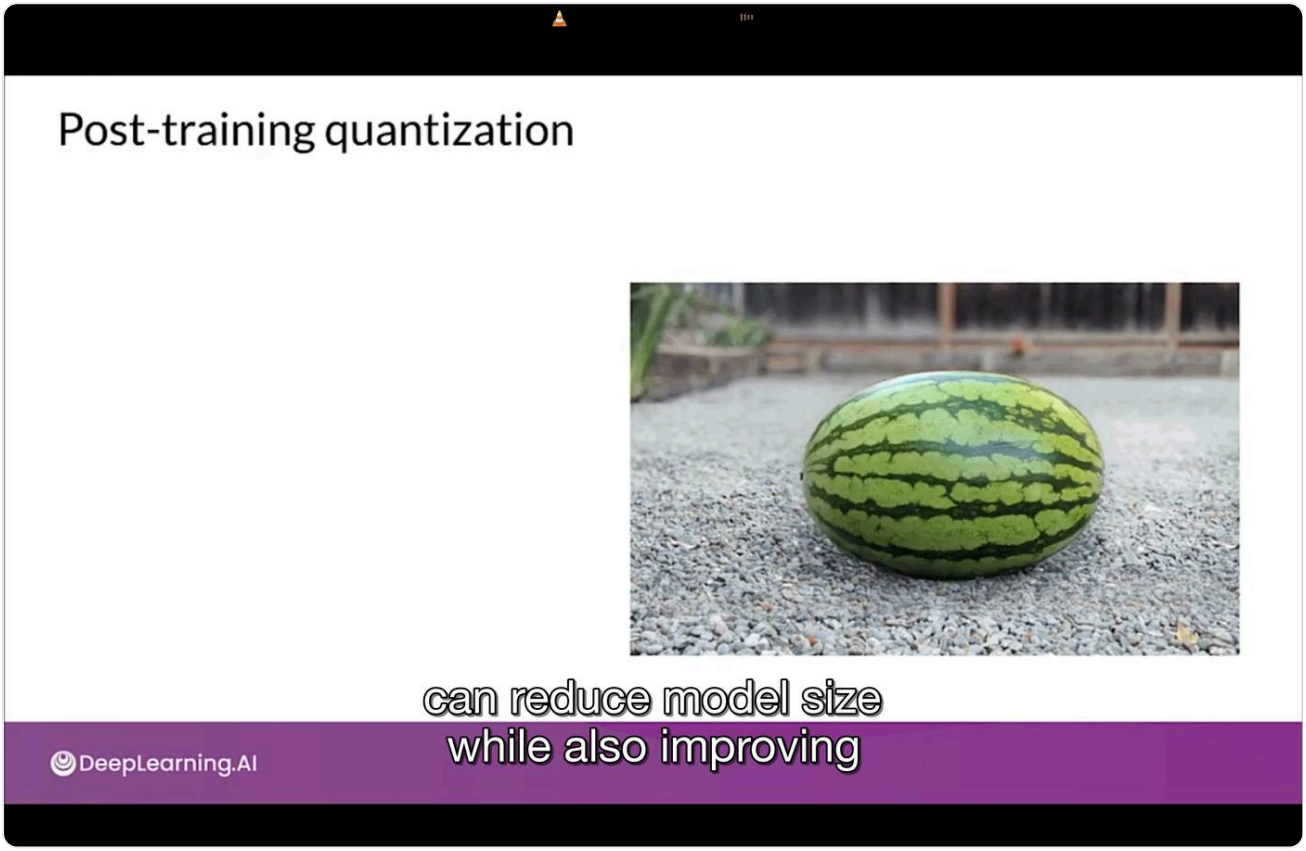

FIGURE 20.1

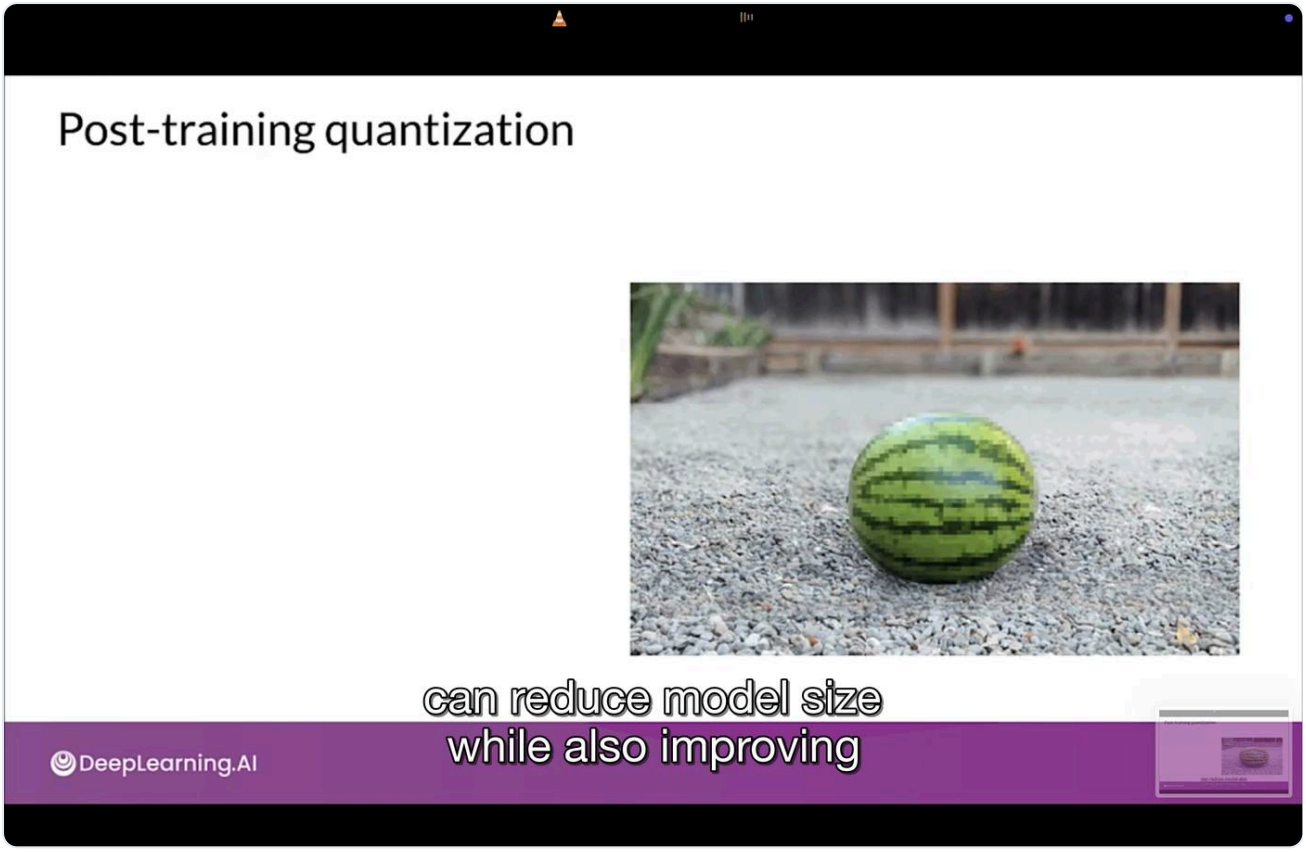

FIGURE 20.2

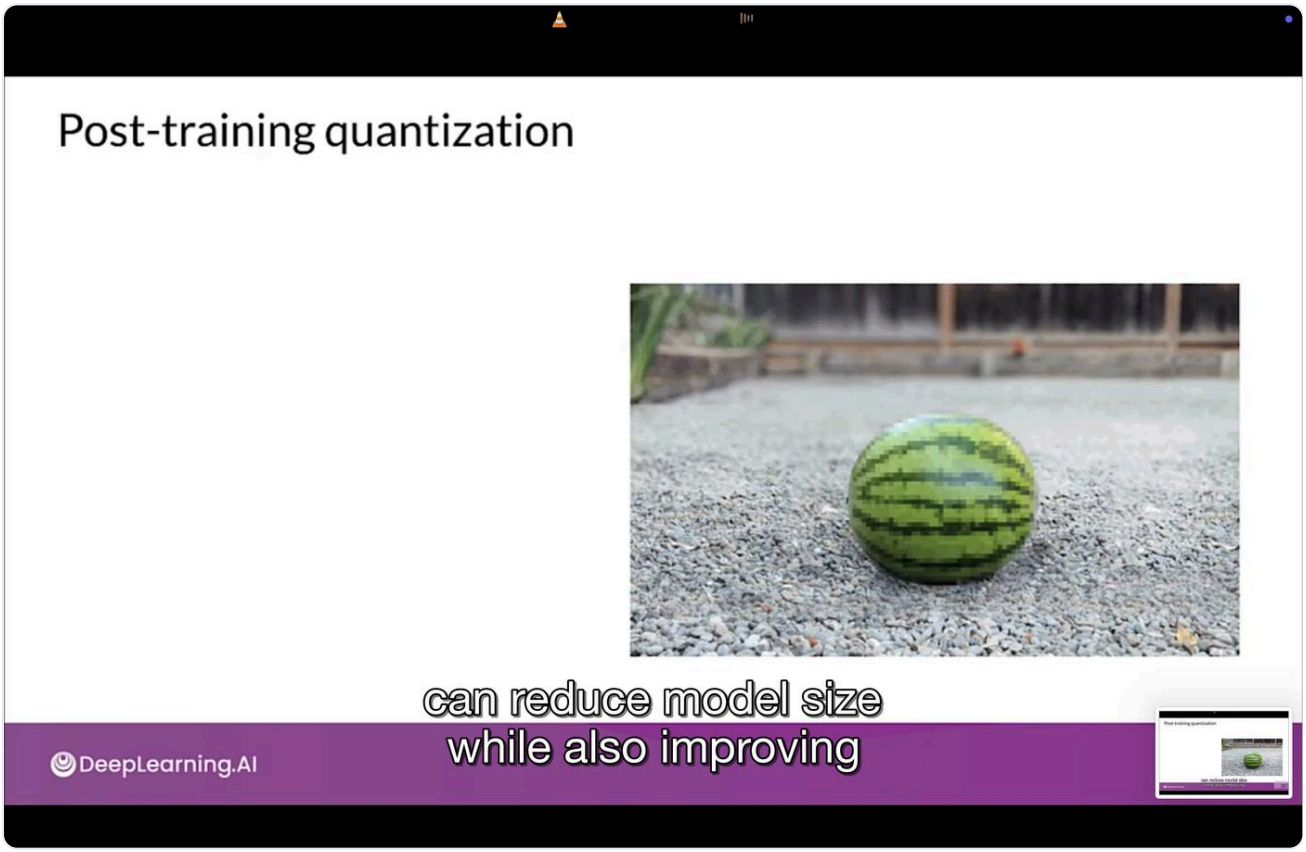

FIGURE 20.3

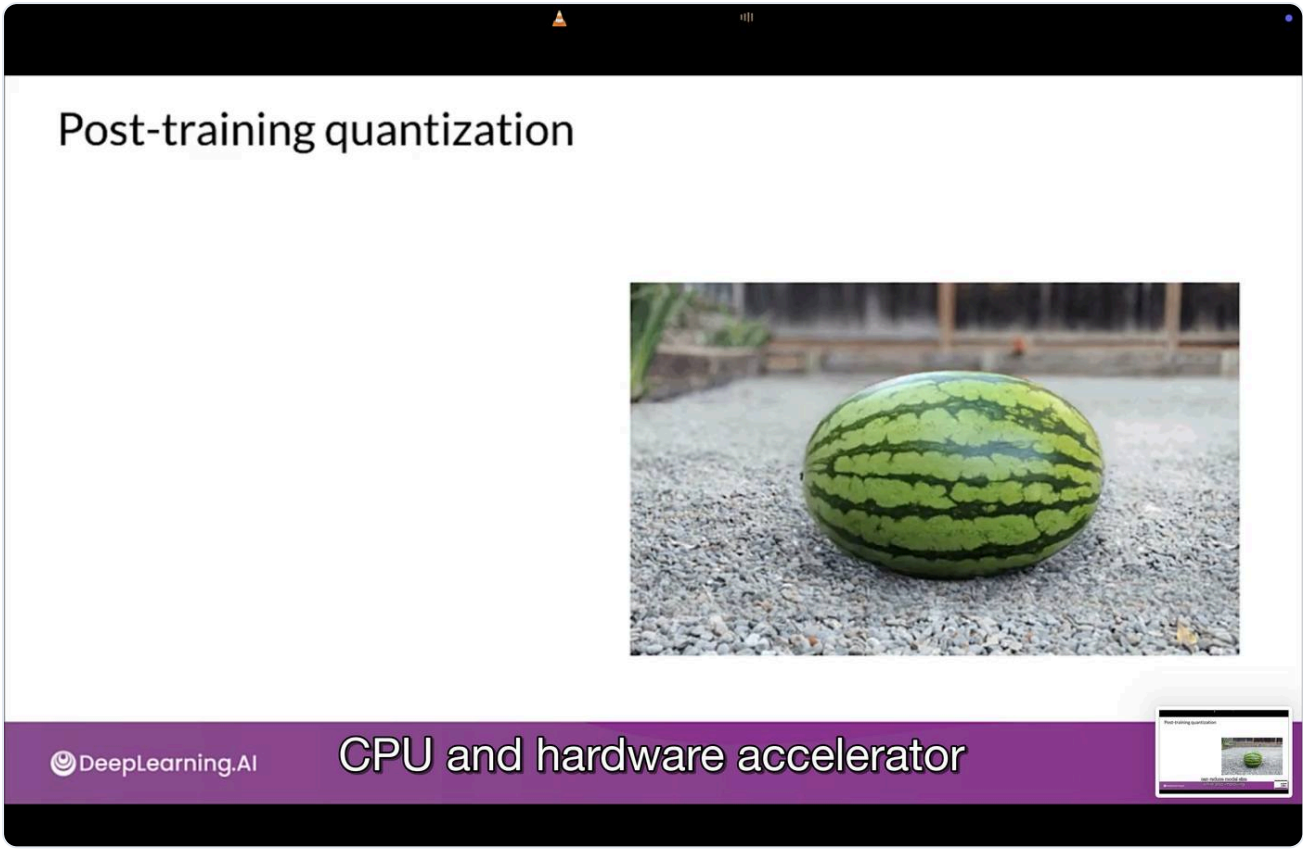

FIGURE 20.4

Post-training quantization is a conversion technique that can reduce model size while also improving CPU and hardware accelerator latency with little degradation in model accuracy.

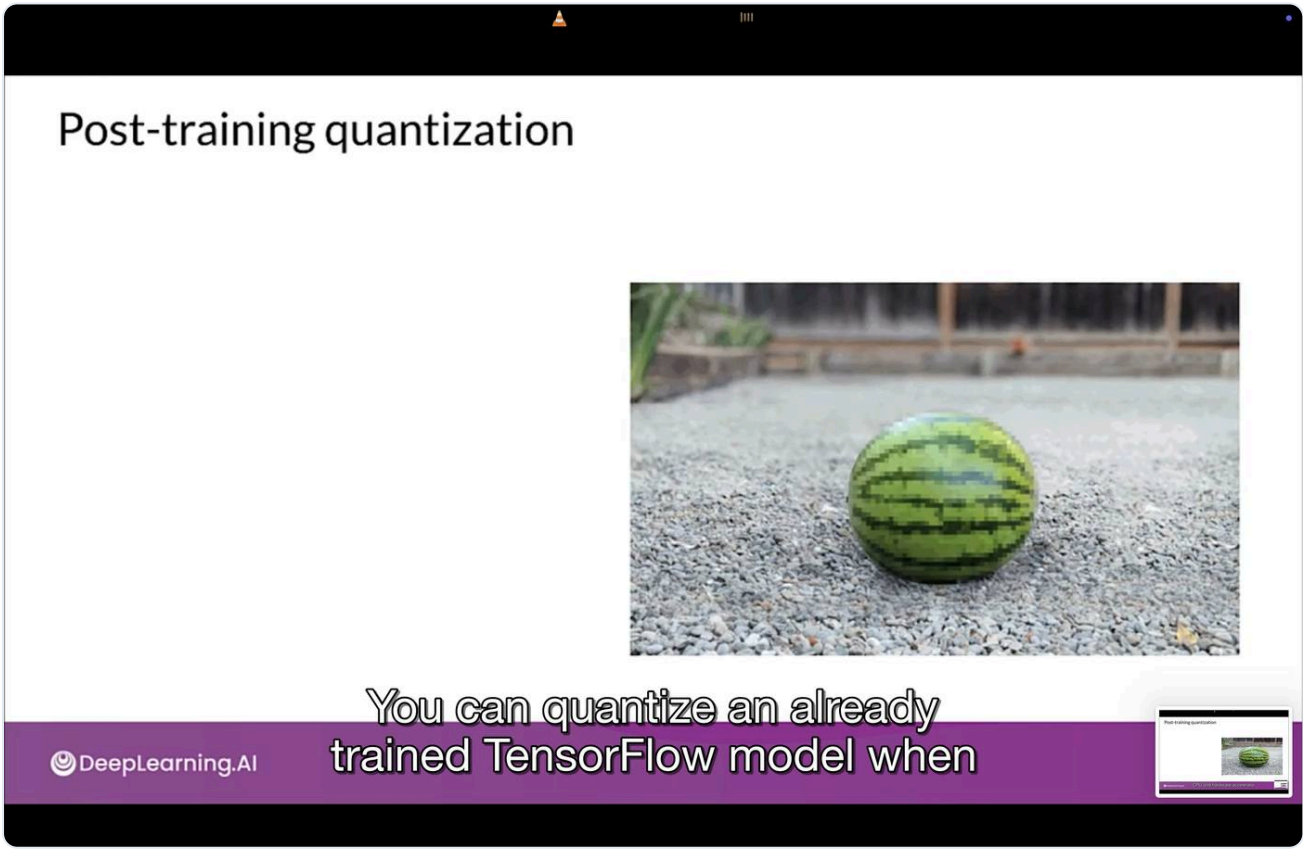

FIGURE 20.5

You can quantize an already trained TensorFlow model when you convert it to TensorFlow Lite format using the TensorFlow Lite converter. It's easy to use since it's integrated into the TFLite converter directly. What post-training quantization basically does is to convert, or more precisely, quantize the weights from floating point numbers to integers in an efficient way. By doing this, you can gain up to three times lower latency without taking a major hit on accuracy. With the default optimization strategy, the converter will do its best to apply a post-training quantization, trying to optimize the model for both size and latency.

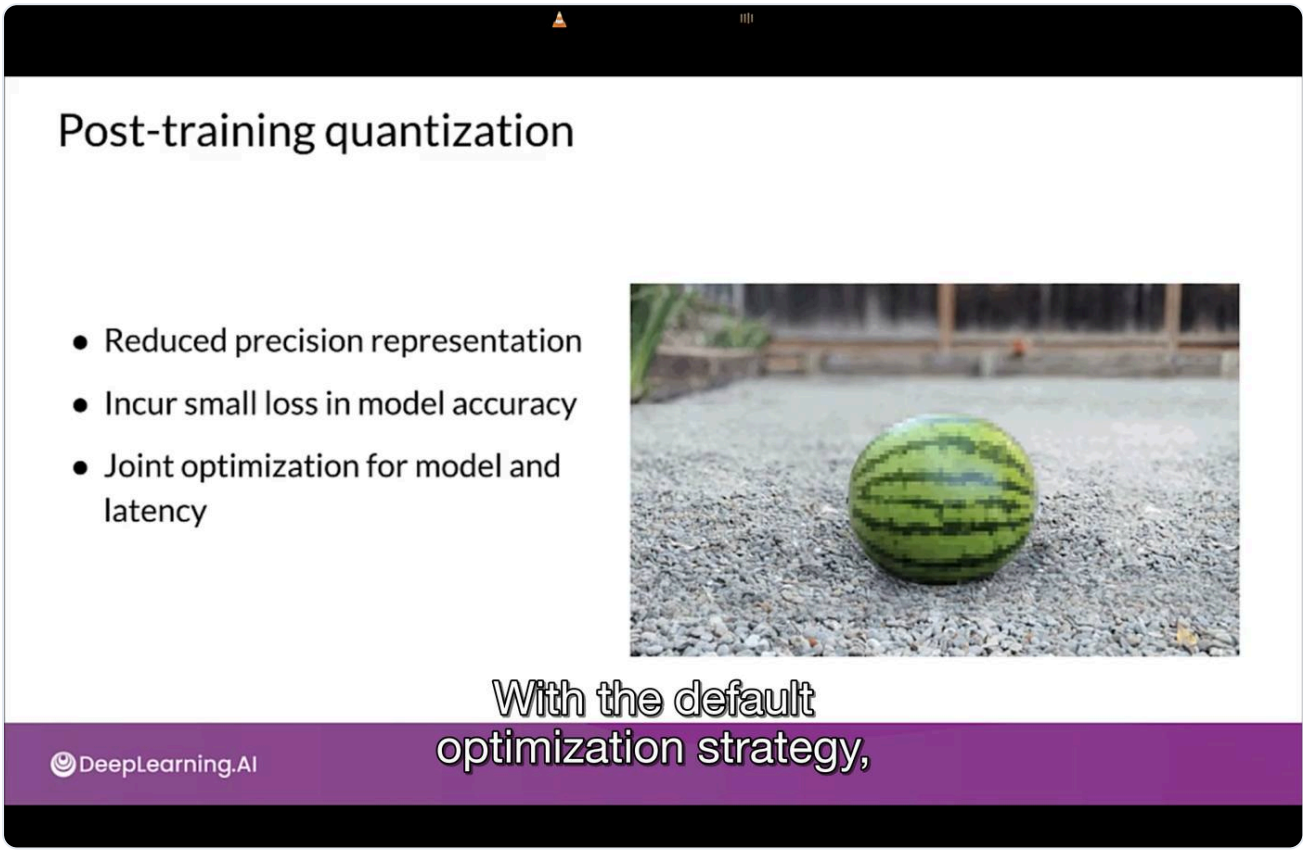

FIGURE 20.6

This is recommended, though you can always customize this behavior. There are several post-training quantization options to choose from. This is a summary of the choices and the benefits they provide. If you're looking for a decent speed-up, like 2-3 times faster, while being two times smaller, you can consider dynamic range quantization. On the other hand, if you want to squeeze out even more performance from your model, then full integer quantization or float16 quantization may result in faster performance. Float16 is especially useful when you plan to use a GPU.

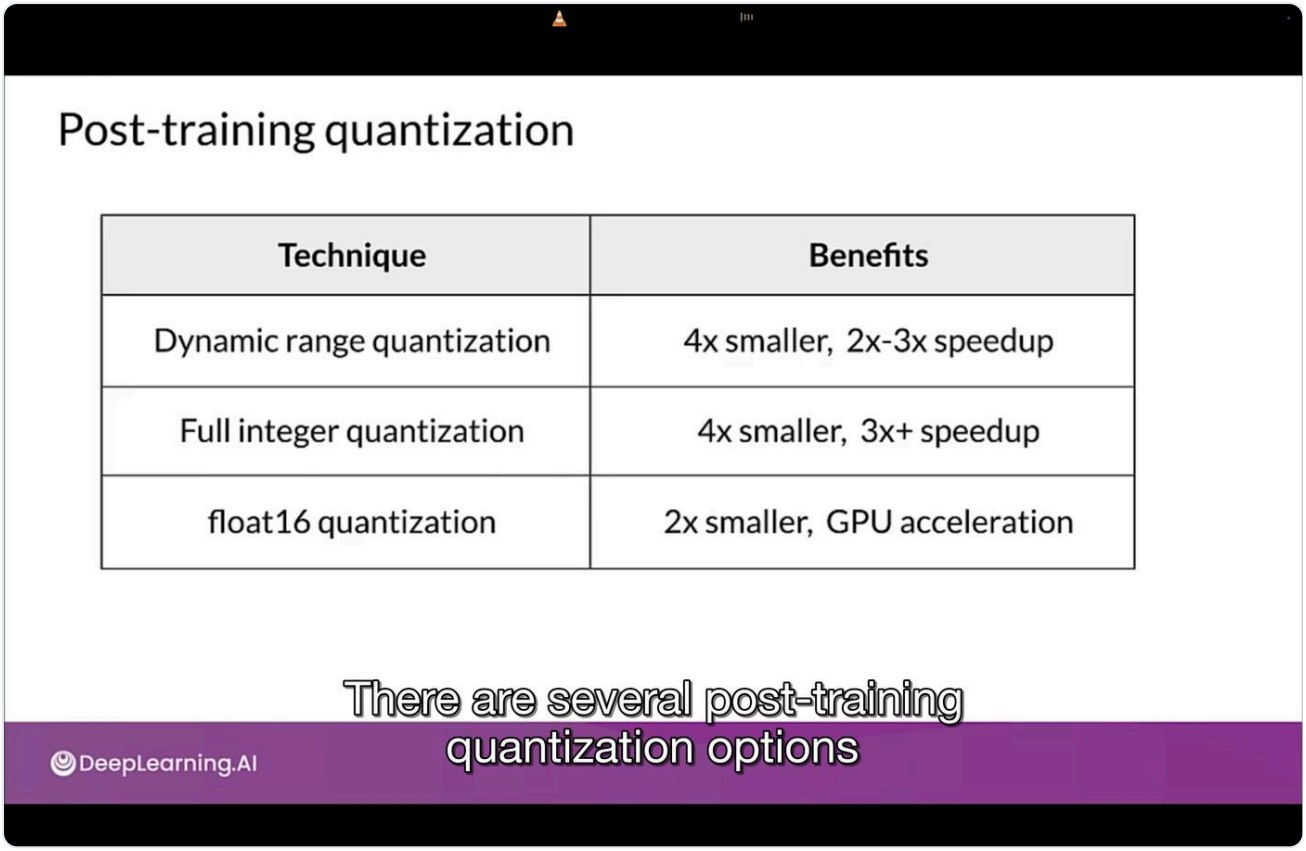

FIGURE 20.7 | Dynamic Range Quantization

With dynamic range quantization, during inference, the weights are converted from eight bits to floating point, and the activations are computed using floating point kernels. This conversion is done once and cached to reduce latency. This optimization provides latencies which are close to fully fixed point inference. Post-training quantization takes only two lines of code. Let's begin by importing TensorFlow and defining a converter with TFLite.

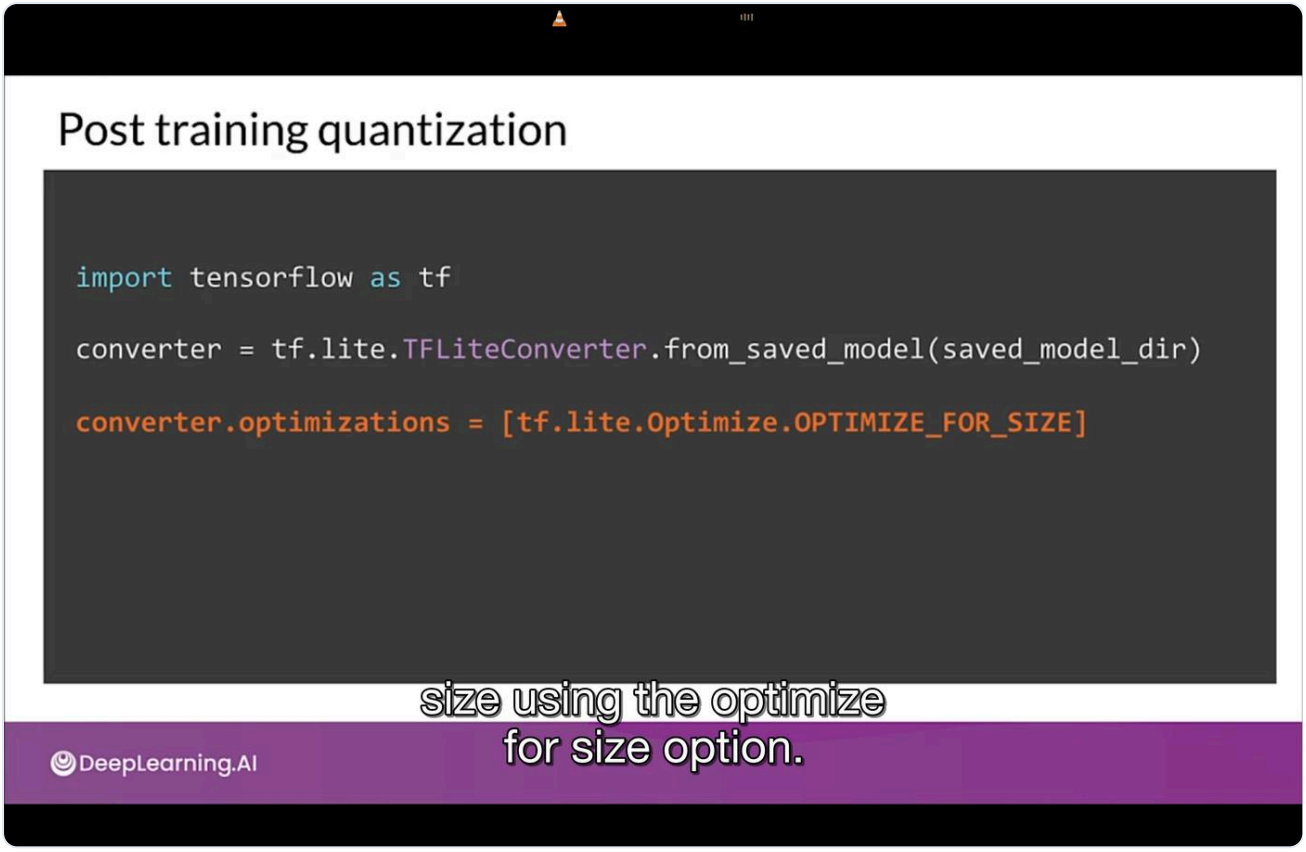

FIGURE 20.8 | Post Training Quantization

Then you set the converter to optimize the model for size using the optimize for size option.

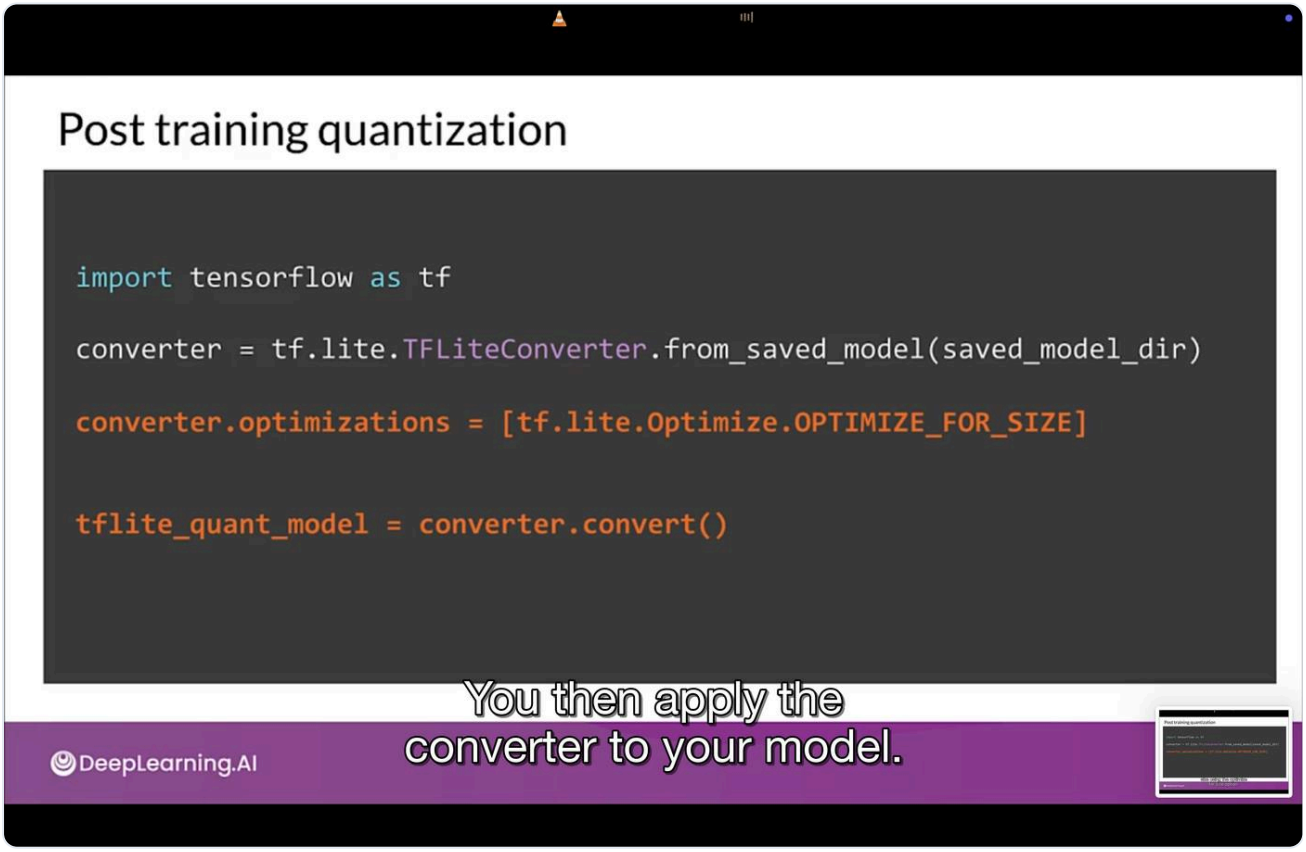

FIGURE 20.9 | Post Training Quantization

You then apply the converter to your model. The other available optimization modes include optimize for latency, which reduces a latency of your model, while default mode basically tries to optimize it for both speed and storage. Enhanced optimizations can be applied by providing a representative dataset.

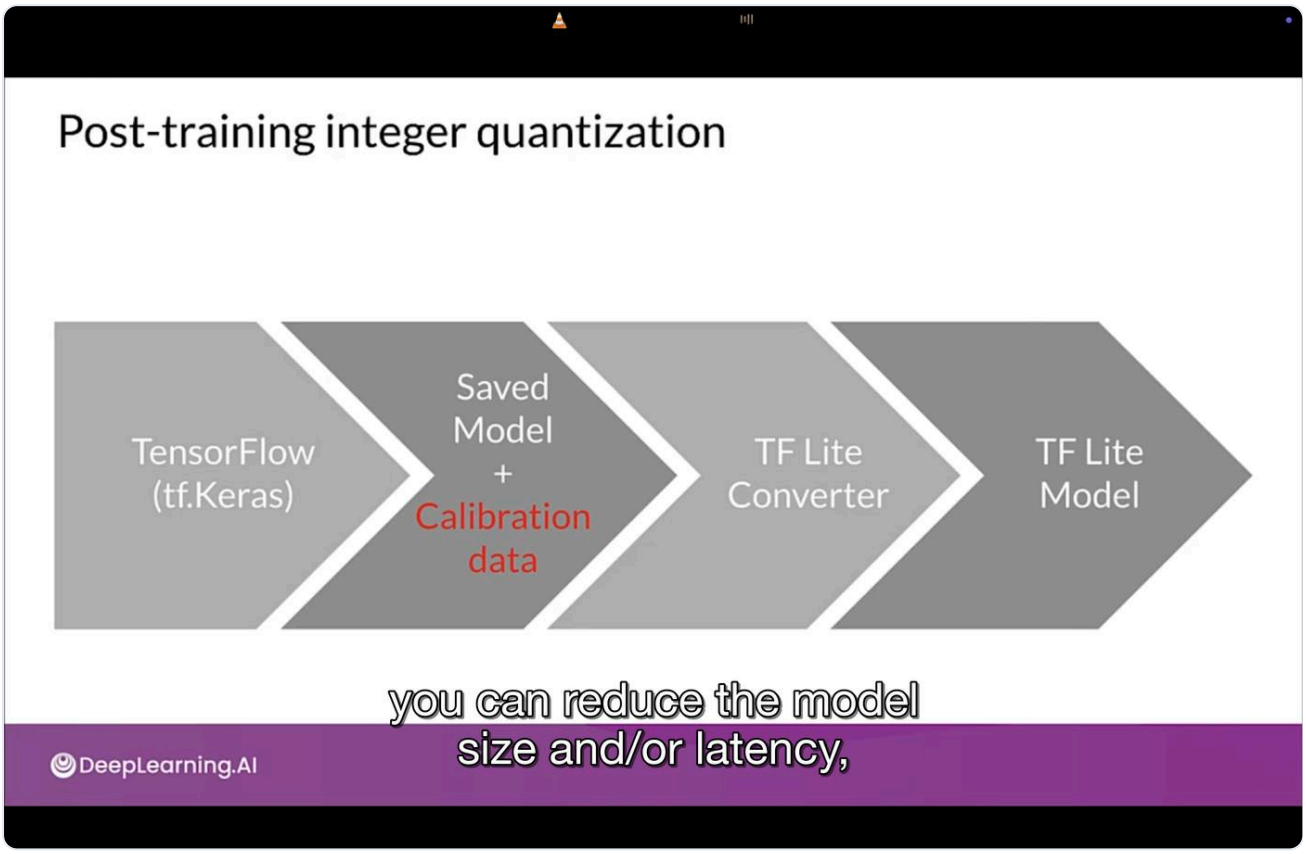

FIGURE 20.10

Using dynamic range quantization, you can reduce the model size and/or latency, but this comes with a limitation as it requires inference to be done with floating point numbers. This may not always be ideal since some hardware accelerators only support integer operations, for example, Edge TPUs. The optimization toolkit also supports post-training integer quantization. This enables users to take an already trained floating point model and fully quantize it to use only eight bits signed integer, which enables fixed point hardware accelerators to run these models. When targeting greater CPU improvements or fixed point accelerators, this is often a better option.

Post-training integer quantization works by gathering calibration data, which it does by running inferences on a small set of inputs so as to determine the right scaling parameters needed to convert the model to an integer quantized model. Post-training quantization can result in a loss of accuracy, particularly for smaller networks, but it is often fairly negligible. On the plus side, this will speed up execution of the heaviest computations by using lower precision and the most sensitive computations with higher precision, thus typically resulting in little or no final loss of accuracy. Pre-trained fully quantized models are also available for specific networks in the TensorFlow Lite model repository.

It's important to check the accuracy of the quantized model to verify that any degradation in accuracy is within acceptable limits.

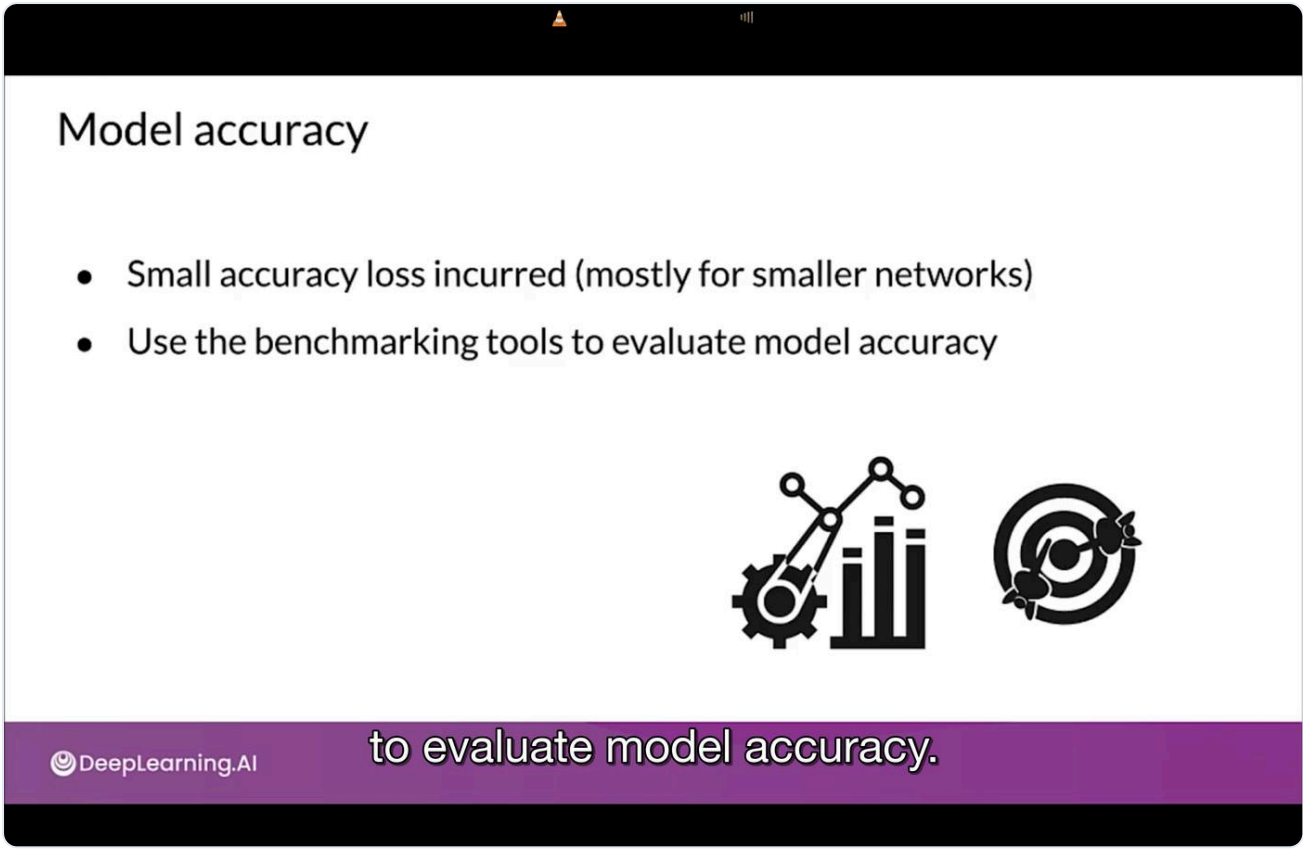

FIGURE 20.11

TensorFlow Lite includes a tool to evaluate model accuracy.

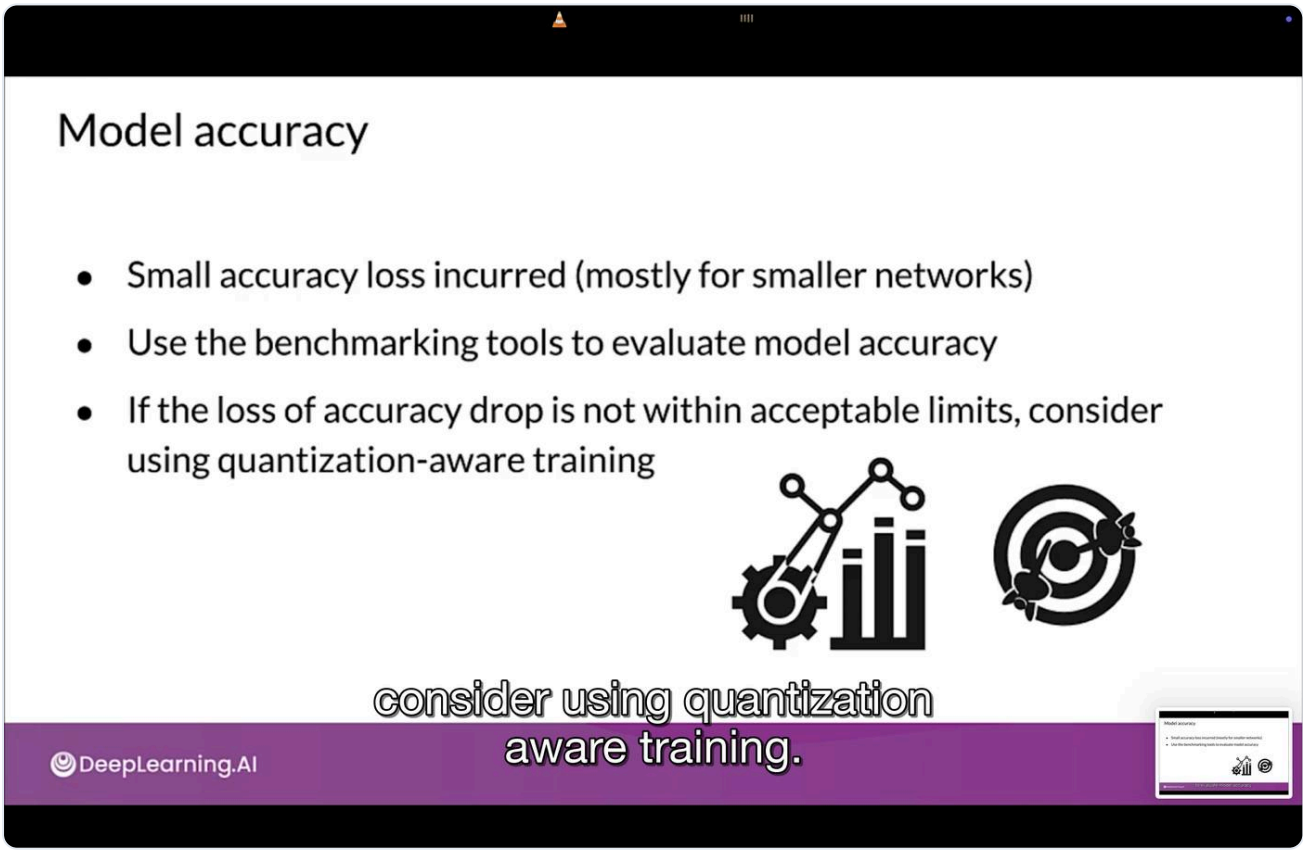

FIGURE 20.12 | Model Accuracy

Alternatively, if the loss of accuracy is too great, consider using quantization aware training. However, doing so requires modifications during model training to add fake quantization nodes, while post-training quantization techniques are fairly simple.

LECTURE 21

Quantize Custom Keras Layer

Now let's look at Quantization-aware training. The simplest approach to quantize a neural network is to first train it in full precision and then simply quantize the weights to fixed point. This is post-training quantization. By contrast, quantization aware training applies quantization to the model while it is being trained. The core idea is that quantization aware training simulates low precision inference time computation in the forward pass of the training process. By inserting fake quantization nodes, the rounding effects of quantization are assimilated in the forward pass, as it would normally occur in actual inference.

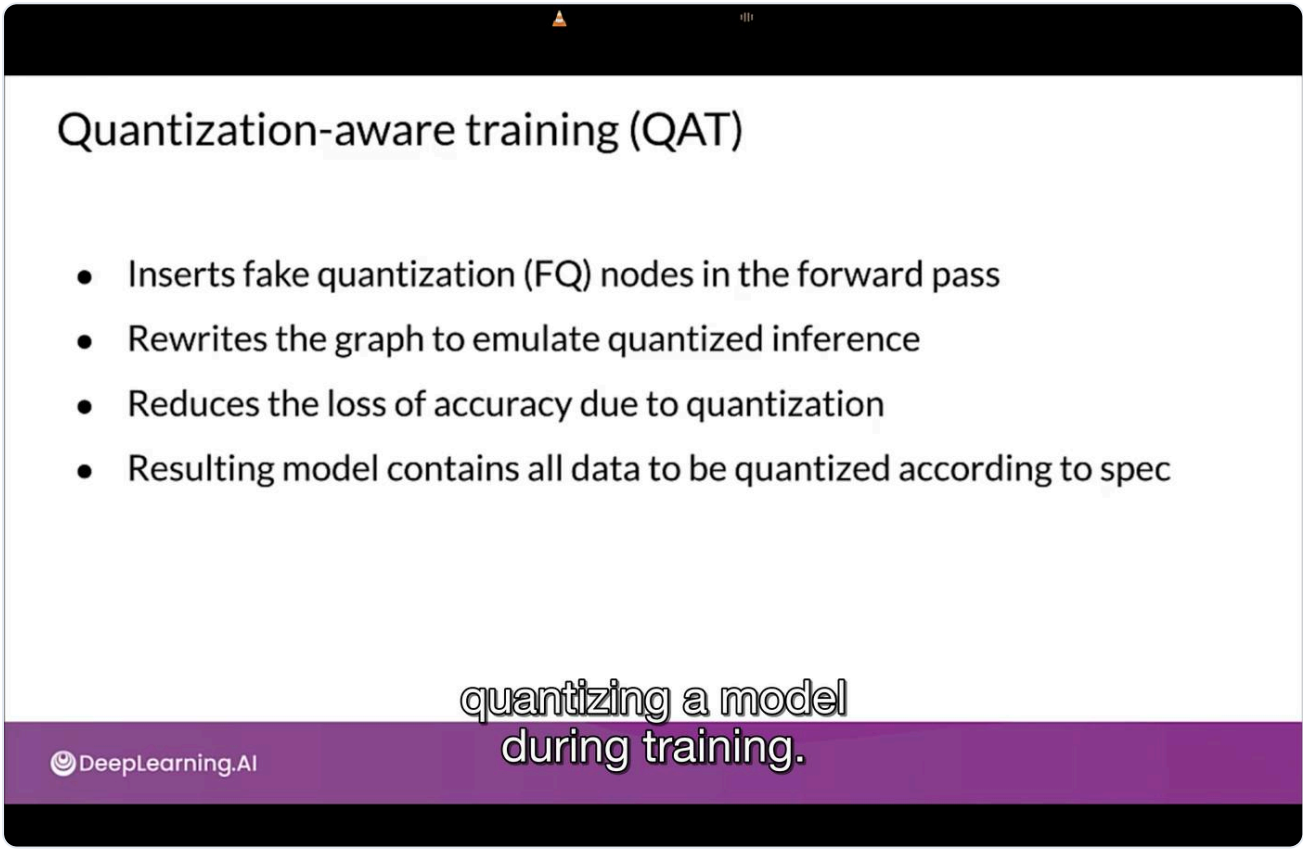

FIGURE 21.1

The goal is to fine-tune the weights to adjust for the precision loss. If fake quantization nodes are included in the model graph at the points where quantization is expected to occur, for example, convolutions. Then in the forward pass, the flood values will be rounded to the specified number of levels to simulate the effects of quantization. This introduces the quantization error as noise during training and is part of the overall loss which the optimization algorithm tries to minimize. Here, the model learns parameters that are more robust to quantization. Next, let's see the process of quantizing a model during training.

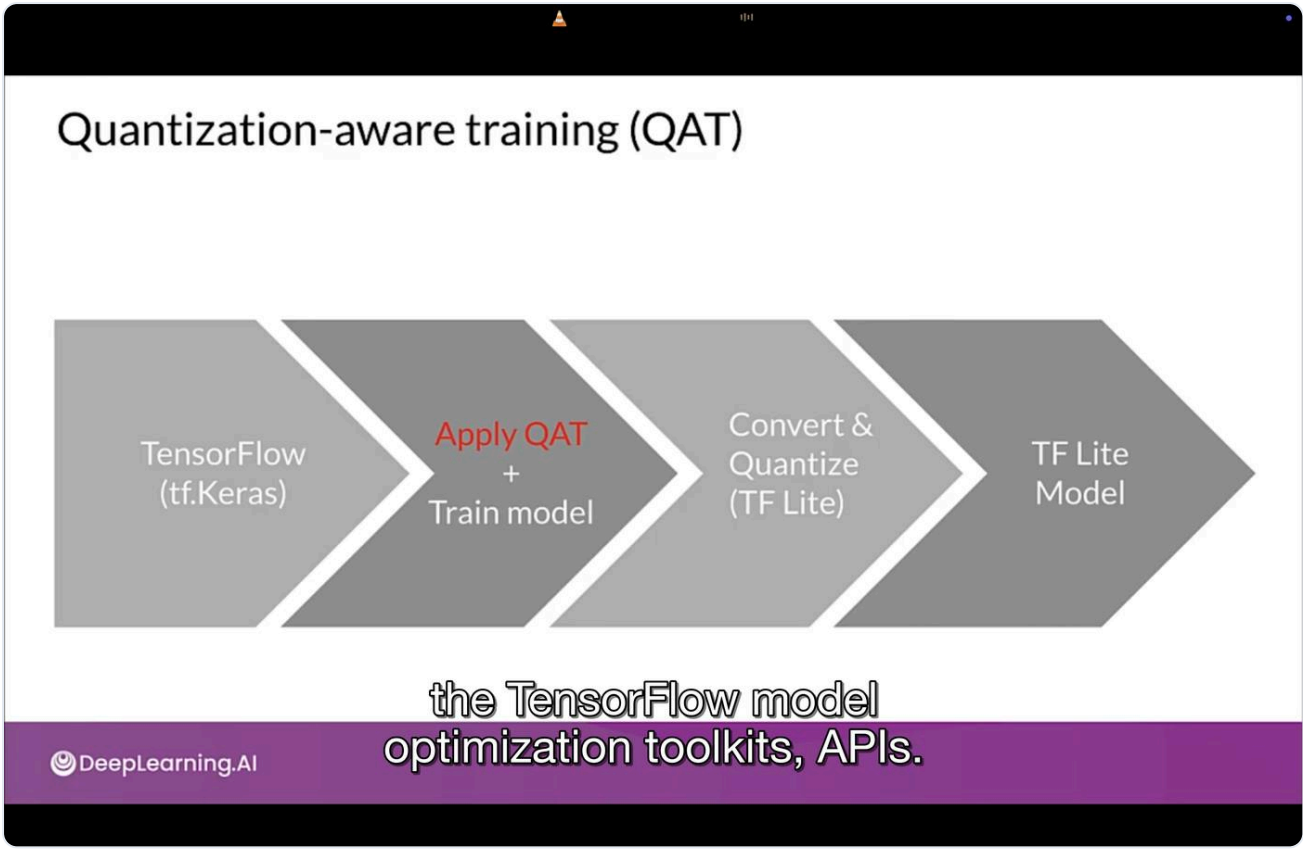

FIGURE 21.2

In quantization aware training, you first build a model like you usually would and make it quantization aware using the TensorFlow model optimization toolkits, APIs. Finally, you train this model with the quantization emulation operations to get our integer-only quantized model. Let's look at this simplified graph showing basic operations in a neural network. The next step is to add quantization emulation operations. The quantization emulation operations need to be placed in the training graph such that they're consistent with the way that the quantized graph will be computed.

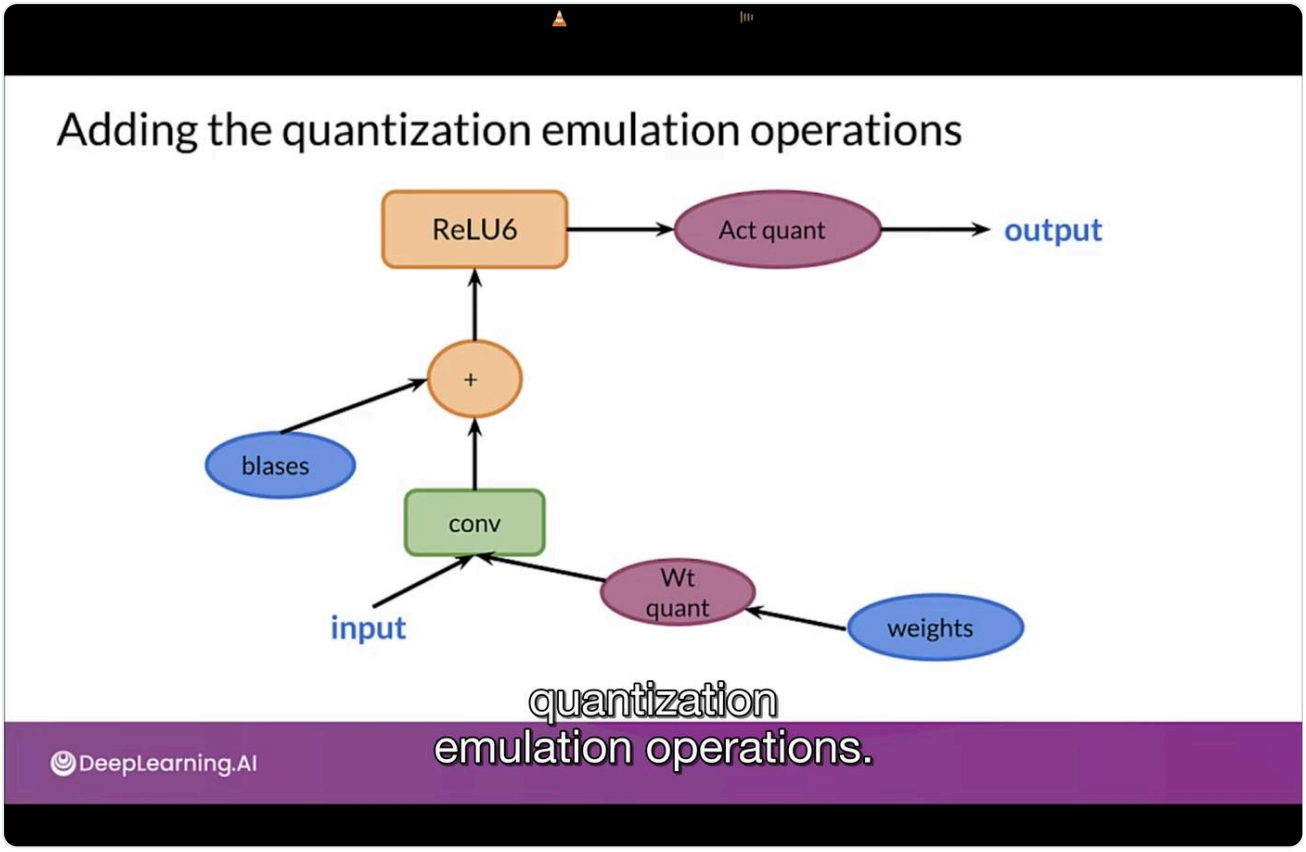

FIGURE 21.3

The weight quant and activation quant Ops introduce losses in the forward pass of the model to simulate actual quantization loss during inference. Note how there is no quant operation between convolution and Relu6. This is because Relu6 gets fused in TensorFlow Lite. The quantization where training API makes it easy to train with quantization awareness for an entire model or only parts of it. Then export it for deployment with TensorFlow Lite.

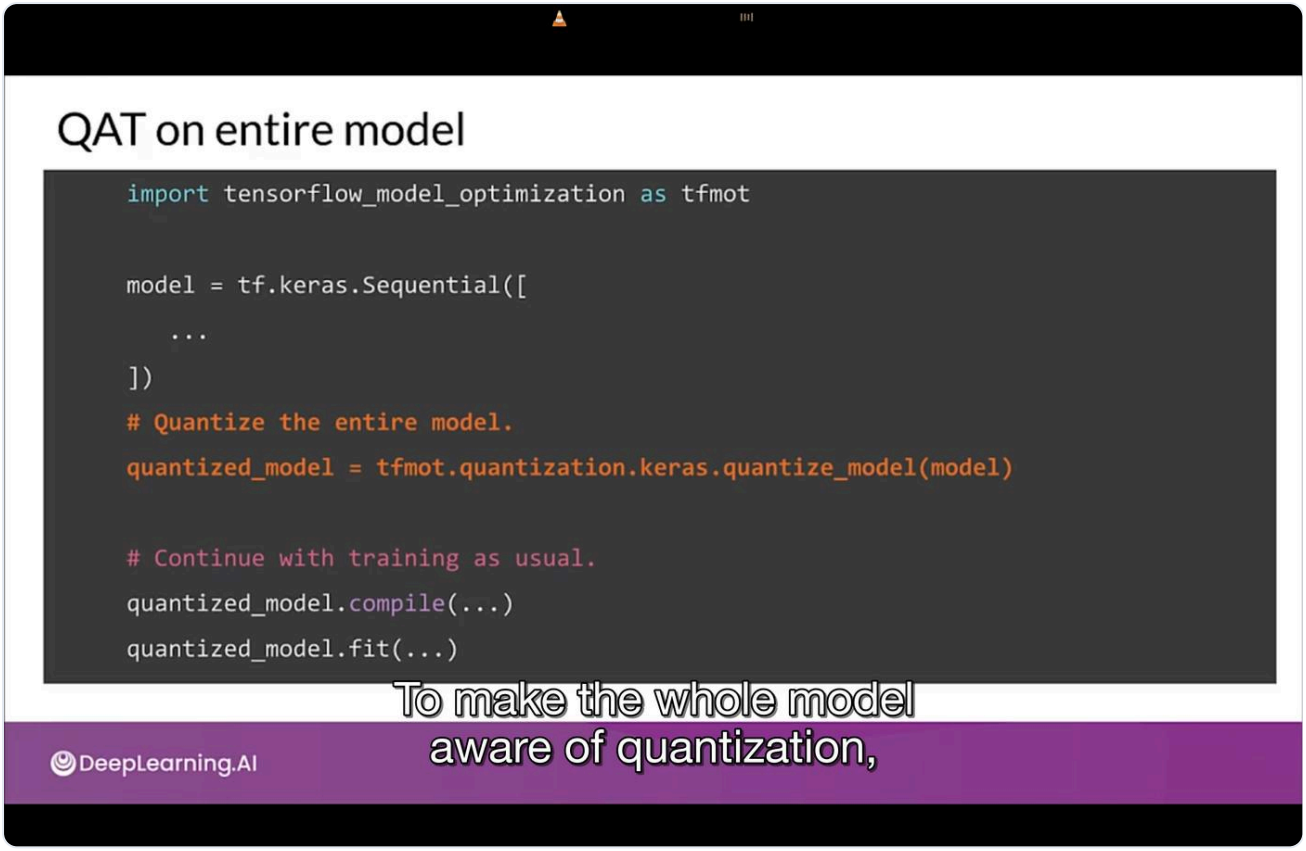

FIGURE 21.4 | QAT on Entire Model

To make the whole model aware of quantization, we apply tfmot.quantization.keras.quantize_model(model).

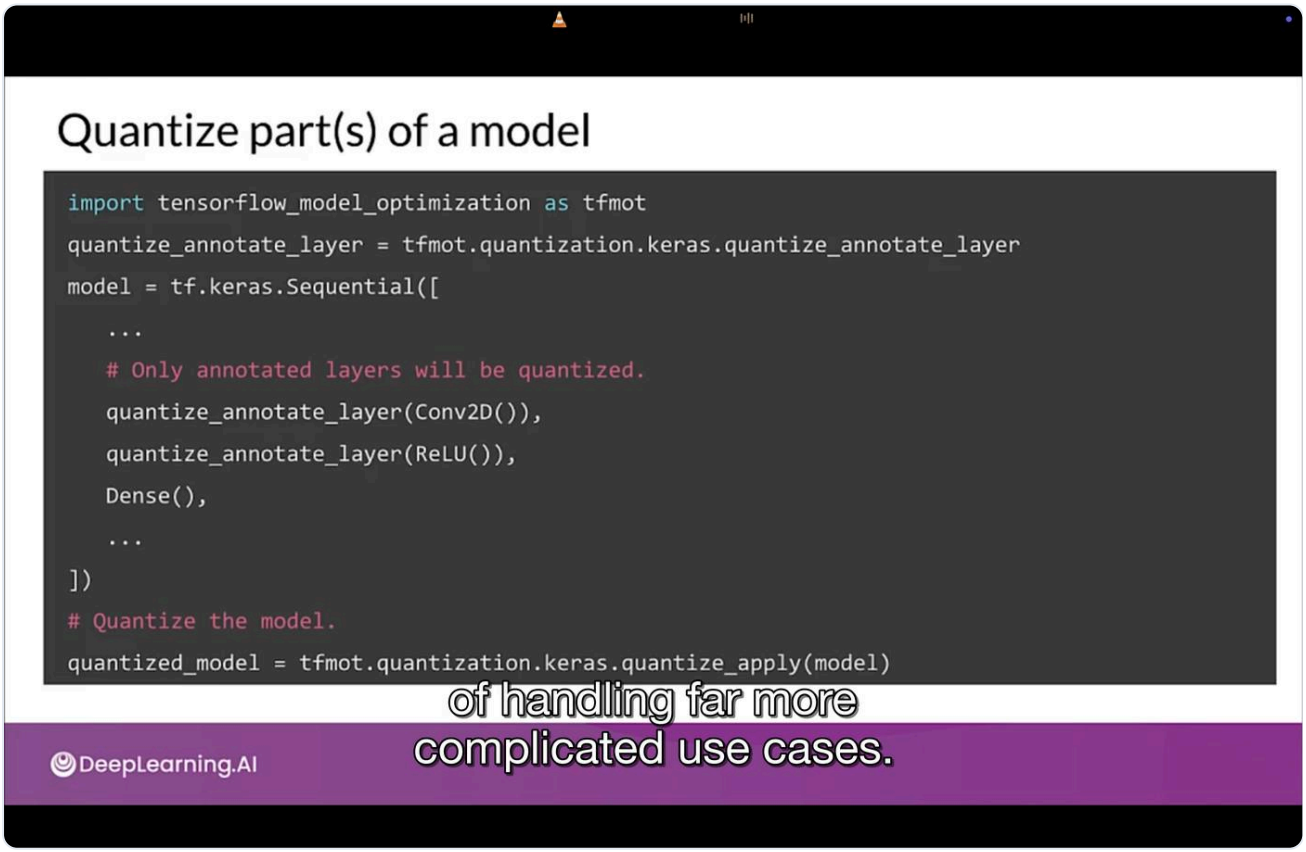

FIGURE 21.5 | Quantize Part(s) of a Model

The API is also quite flexible and capable of handling far more complicated use cases. For example, it allows you to control quantization precisely within a layer. Create custom quantization algorithms, and handle any custom layers that you may have written. You can selectively quantize layers of a model to explore the trade-off between accuracy, speed, and model size. For example, try quantizing the later layers instead of the first layers.

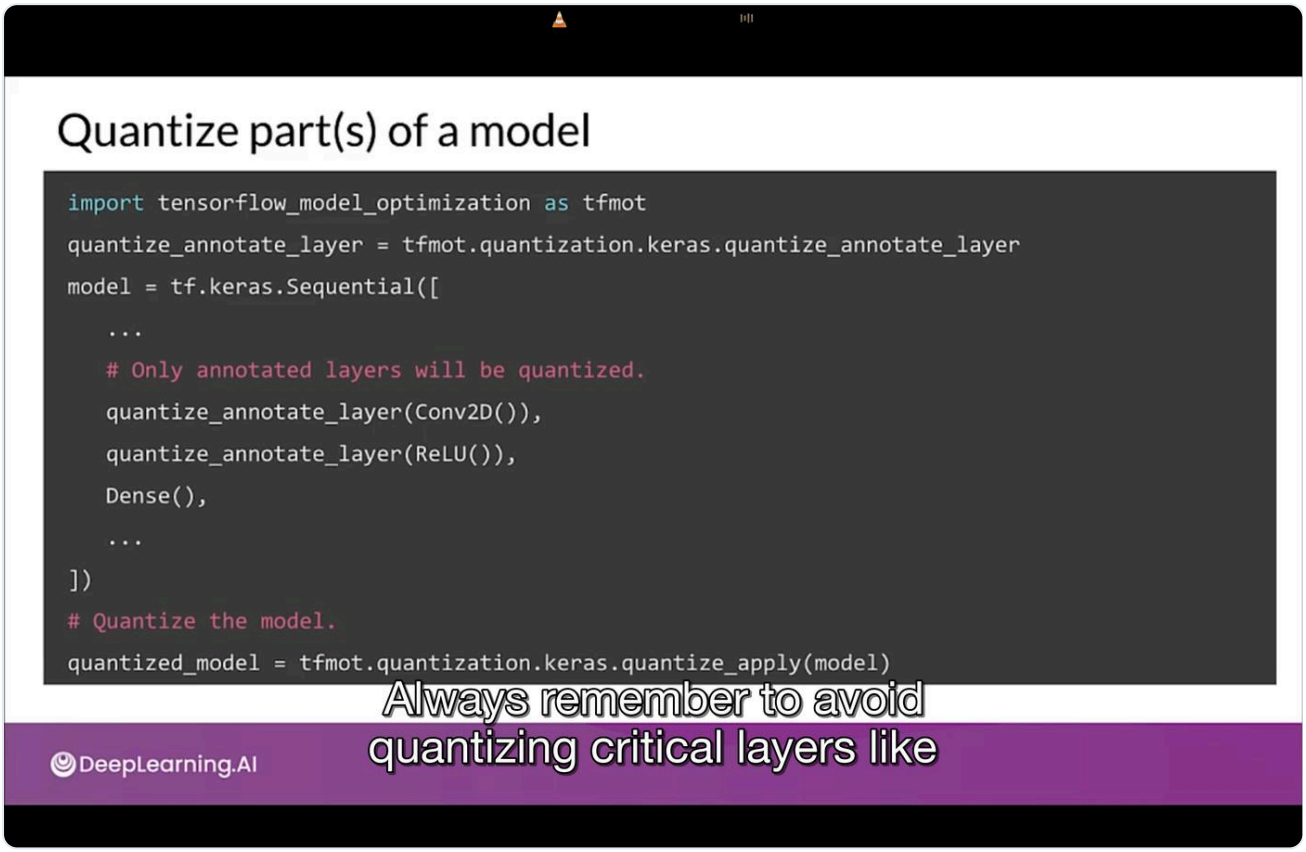

FIGURE 21.6 | Quantize Part(s) of a Model

Always remember to avoid quantizing critical layers like the attention mechanism in transformer architectures for example.

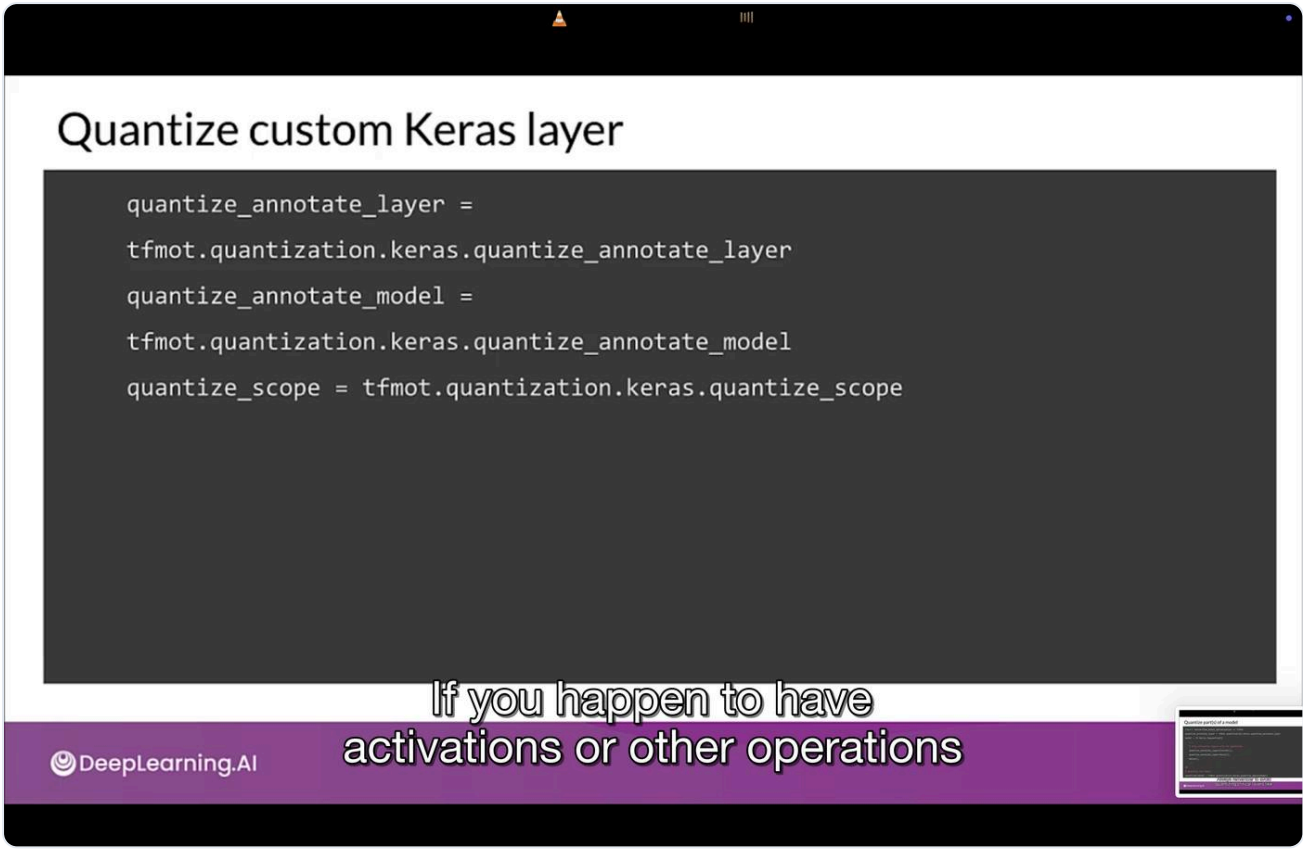

FIGURE 21.7 | Quantize Custom Keras Layer

If you happen to have activations or other operations that aren't yet supported by the quantization aware framework. You can use a quantization configuration to solve this.

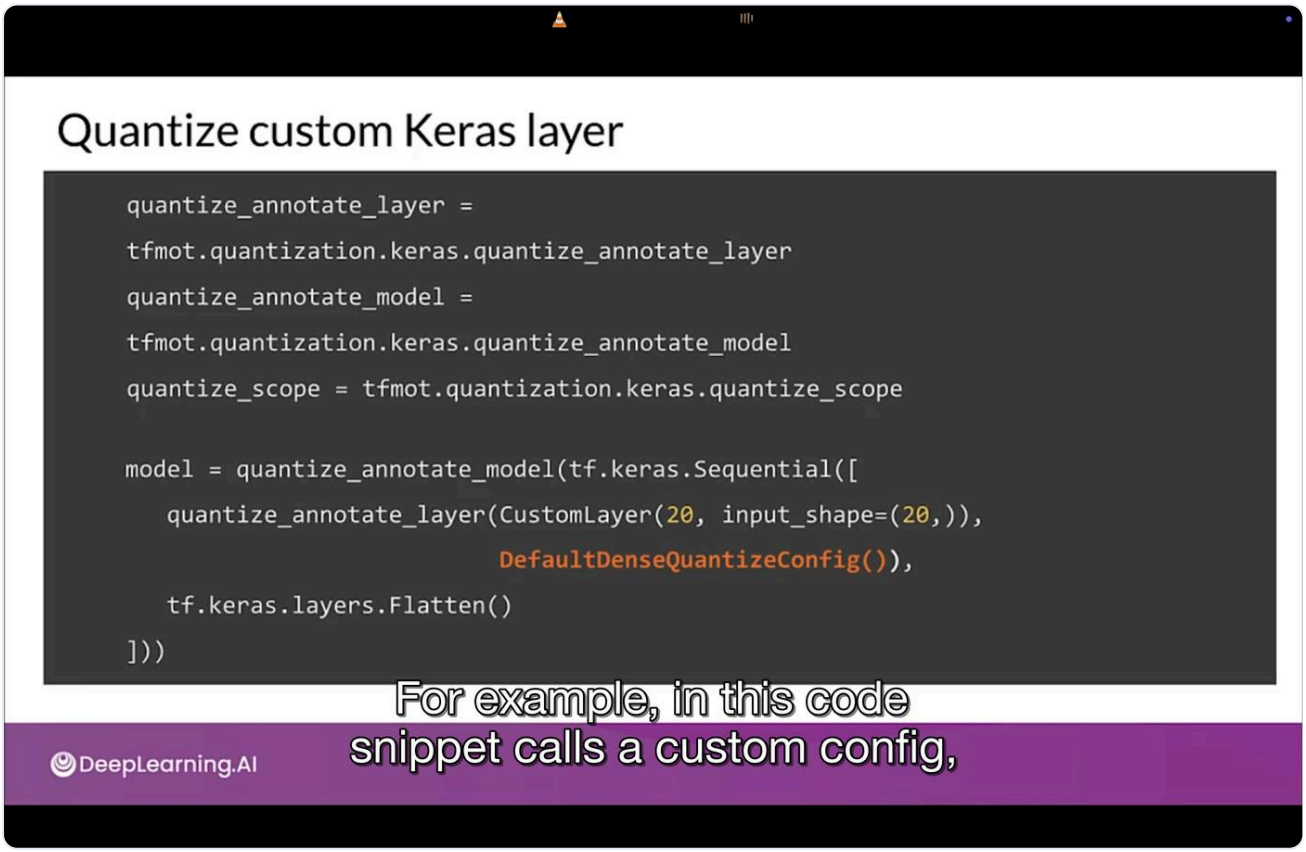

FIGURE 21.8 | Quantize Custom Keras Layer

For example, in this code snippet calls a custom config, like DefaultDenseQuantizeConfig() to quantize a custom layer.

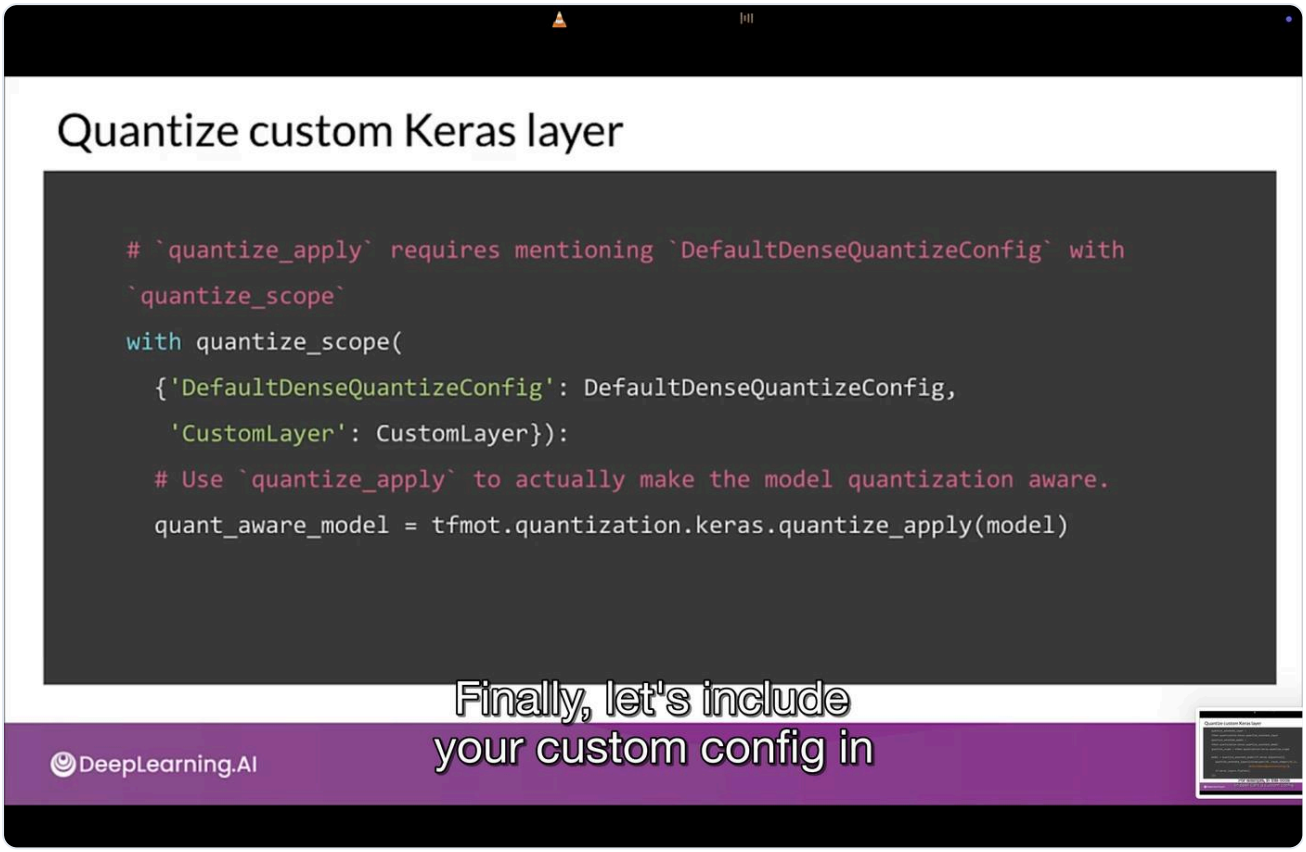

FIGURE 21.9 | Quantize Custom Keras Layer

Finally, let's include your custom config in your quantized scope before calling quantize apply.

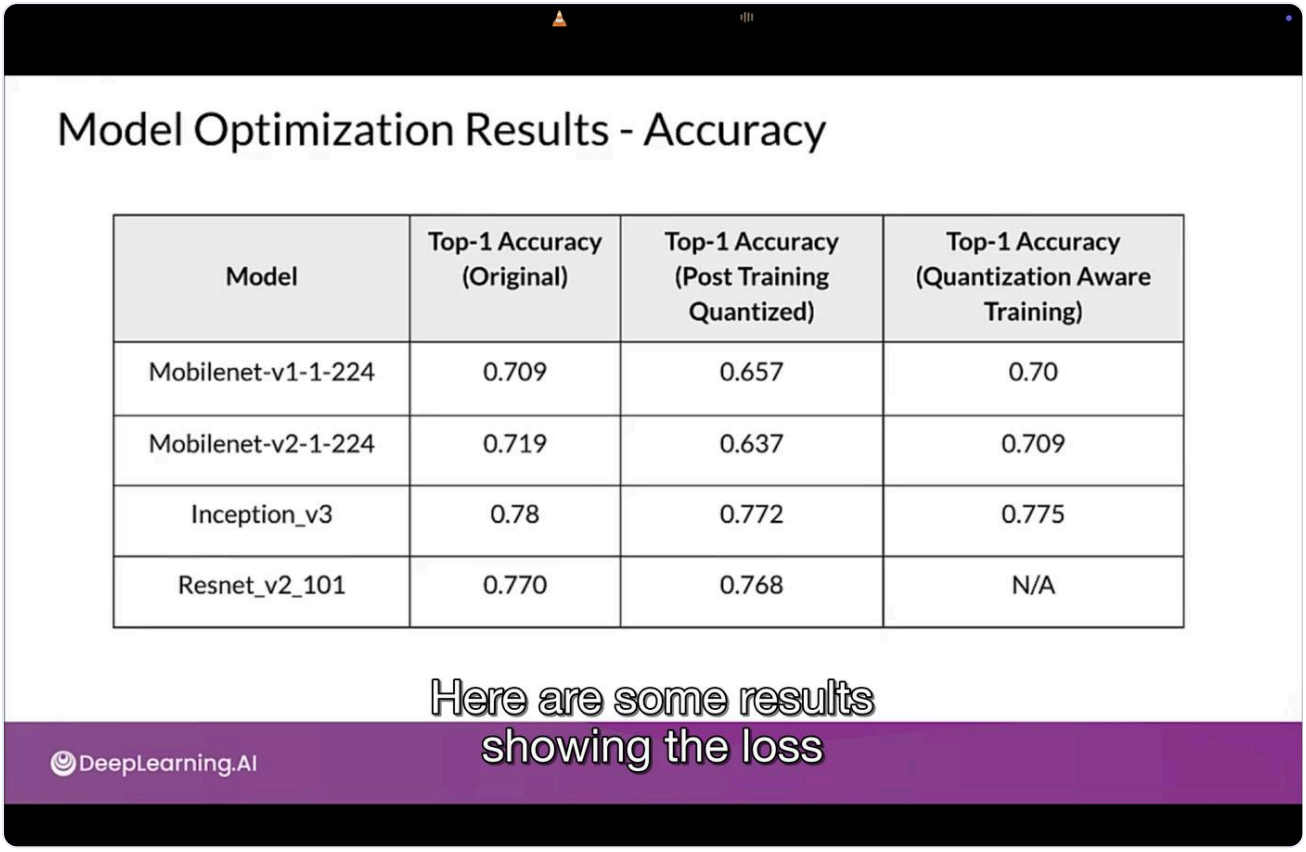

FIGURE 21.10 | Model Optimization Results - Accuracy

Here are some results showing the loss of accuracy on a few models. This should give you a feel for what to expect in your own models. Let's look at the latency for a few models.

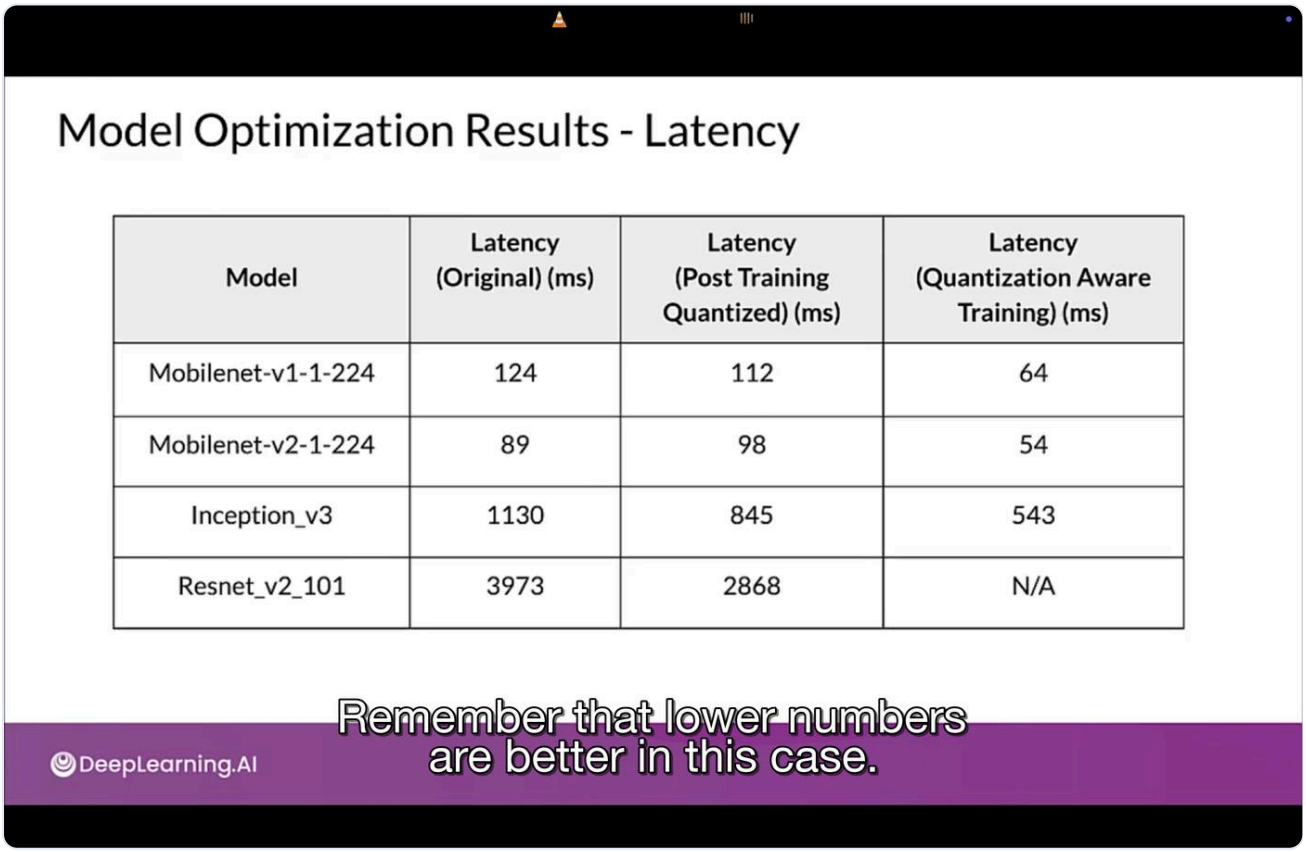

FIGURE 21.11 | Model Optimization Results - Latency

Remember that lower numbers are better in this case.

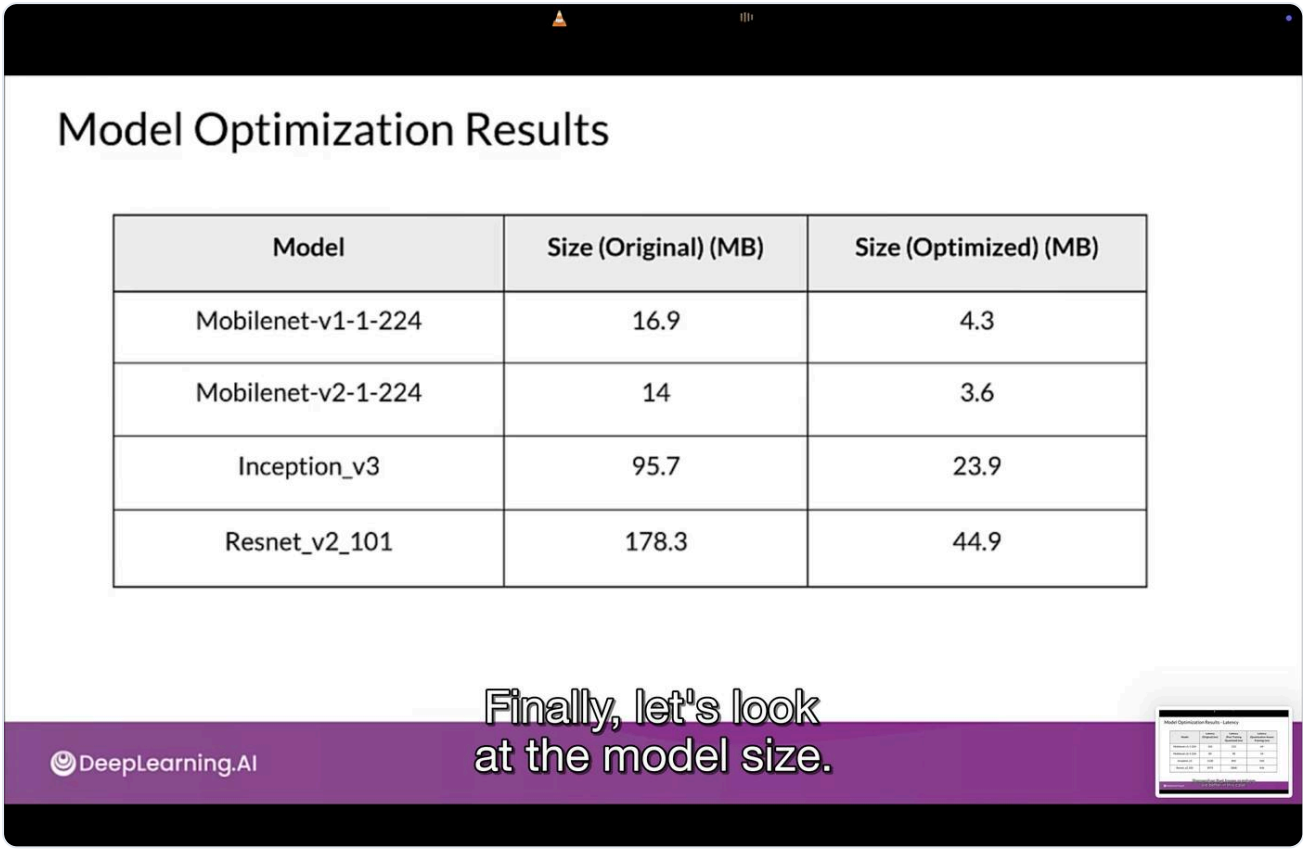

FIGURE 21.12 | Model Optimization Results

Finally, let's look at the model size. Lower numbers are better.

LECTURE 22

Sparsity Increases with Training

Another method to increase the efficiency of models is to remove parts of the model that did not contribute substantially to producing accurate results. This is referred to as pruning. Let's discuss that now. Optimizing machine learning programs can take several different forms. Fortunately, neural networks have proven resilient to various transformations aimed to the skull. When you consider more extensive neural networks with more layers and nodes, reducing their storage and computational cost becomes critical, especially for some real time applications.

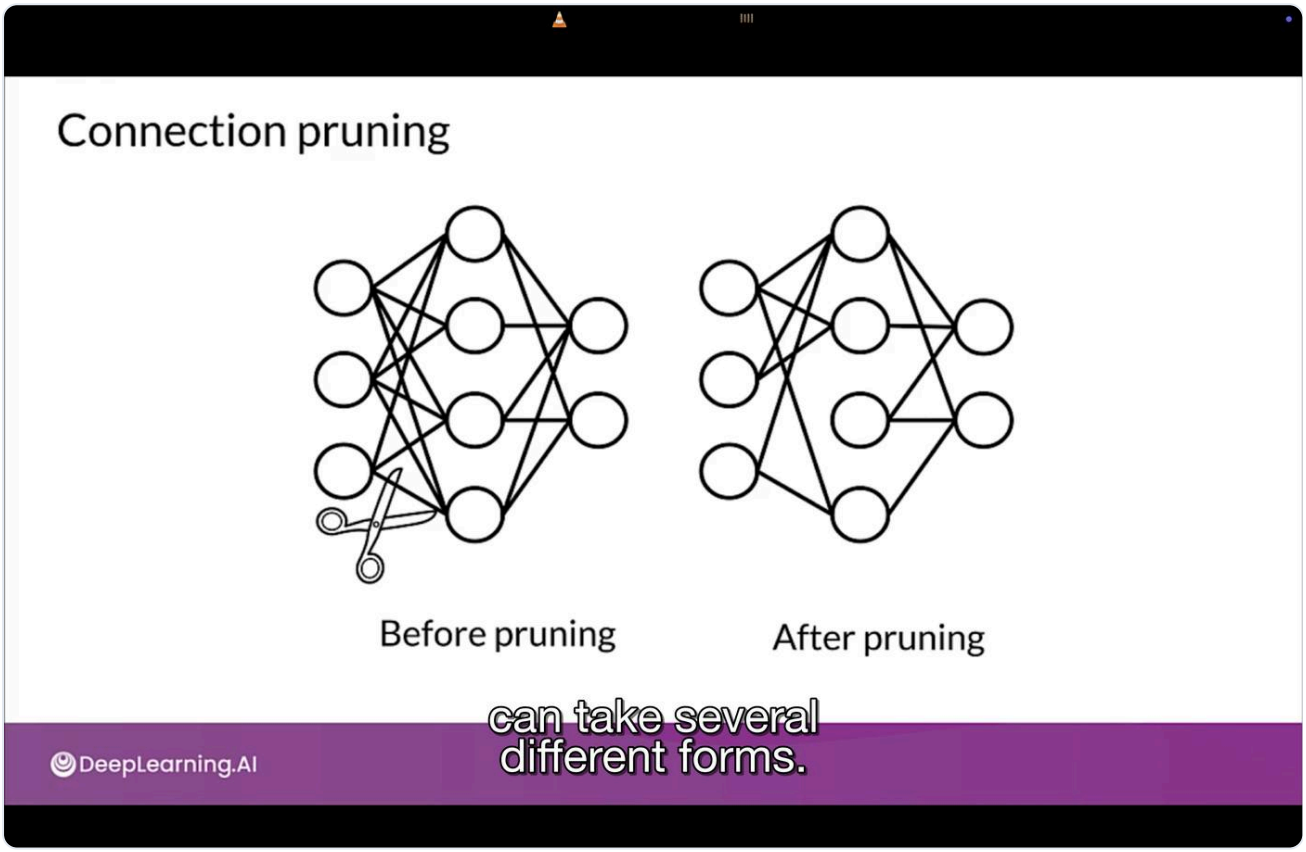

FIGURE 22.1 | Connection Pruning

Model compression can be used to address this problem. As machine learning models were pushed into imbedded devices like mobile phones, compressing neural networks grew in importance. Pruning in deep learning is a biologically inspired concept that we'll discuss next. Pruning aims to reduce the number of parameters and operations involved in generating a prediction by removing network connections. With pruning, you can lower the overall parameter count in the network. Networks generally look like the one on the left.

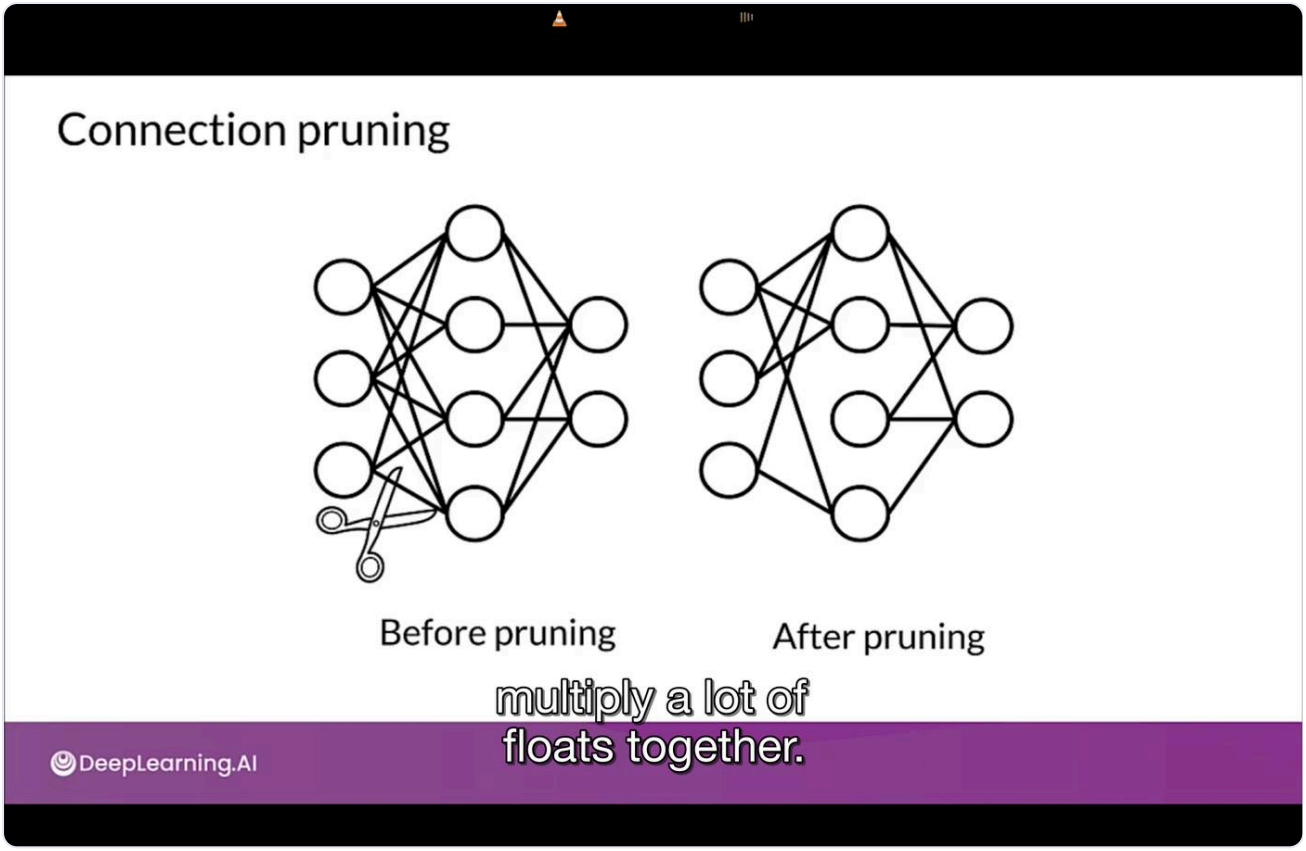

FIGURE 22.2 | Connection Pruning

Here every neuron in a layer has a connection to the layer before it, but this means we have to multiply a lot of floats together. Ideally, we'd only connect each neuron to a few others and save on doing some of the multiplications, if we can find a way to do that without too much loss of accuracy. That's the motivation behind pruning. Connection sparsity has long been a foundational principle in neuroscience research, as it is one of the critical observations about the neocortex. Everywhere you look in the brain, the activity of neurons is always sparse. But common neural network architectures have a lot of parameters which generally aren't sparse.

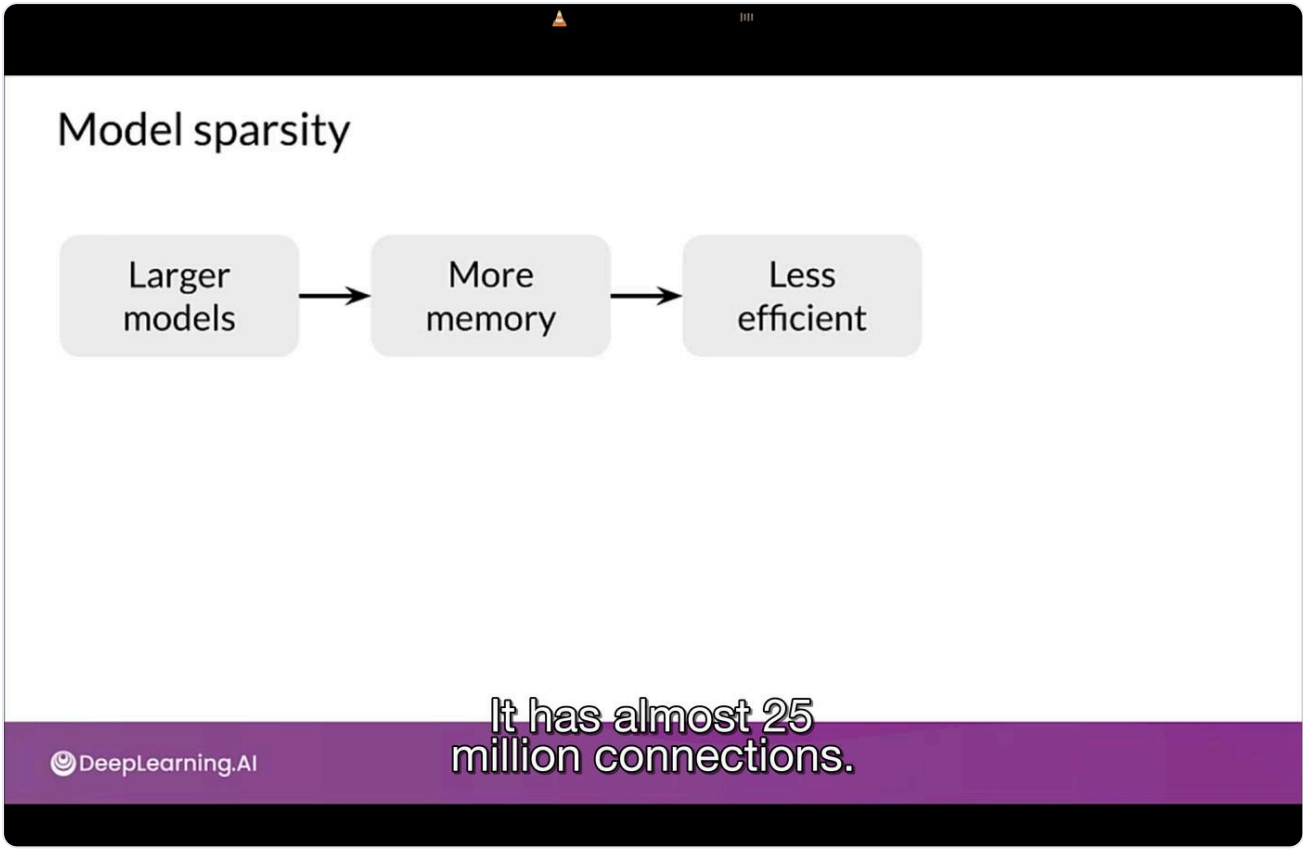

FIGURE 22.3 | Model Sparsity

Take for example ResNet-50. It has almost 25 million connections. This means that during training we need to adjust 25 million weights. Doing that is relatively costly to say the least, so there is a need to fix this somehow. The story of sparsity in neural network starts with pruning, which is a way to reduce the size of the neural network through compression. Reducing the number of parameters would have several benefits. A sparse network is not only smaller, but it is also faster to train and use.

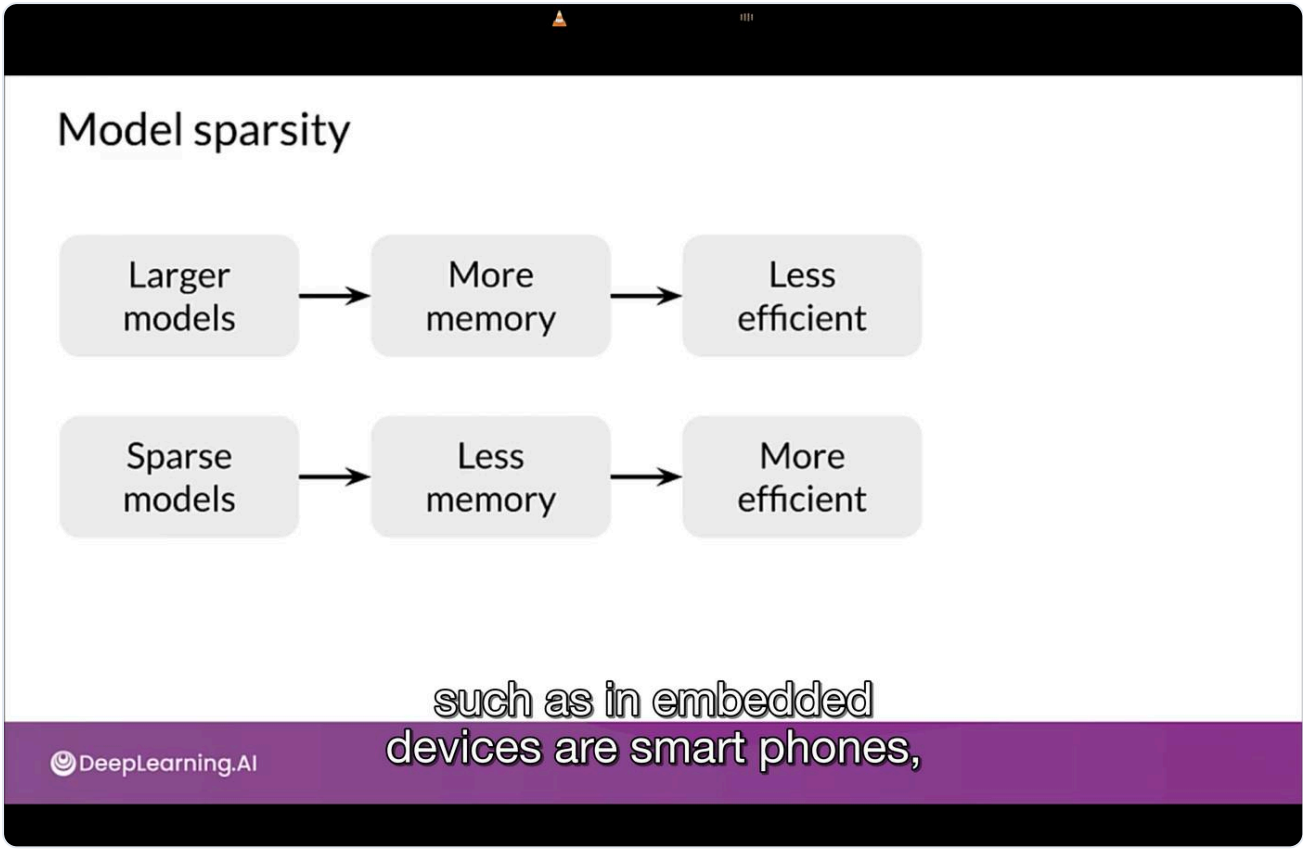

FIGURE 22.4 | Model Sparsity

Where hardware is limited, such as in embedded devices are smart phones, speed and size can make or break a model. Also, more complex models are more prone to overfitting. In some sense, restricting the search space can also act as a regularizer. However, even when all that said, it's not a simple task since reducing the model's capacity can also lead to a loss of accuracy. As in many other areas, there is a delicate balance between complexity and performance. Now let's take a more in-depth look at some of the challenges and potential solutions. Let's start with a little history. The first major paper advocating sparsity and neural networks dates back from 1990, written by Yann LeCun, John S.

Denker, and Sara A.

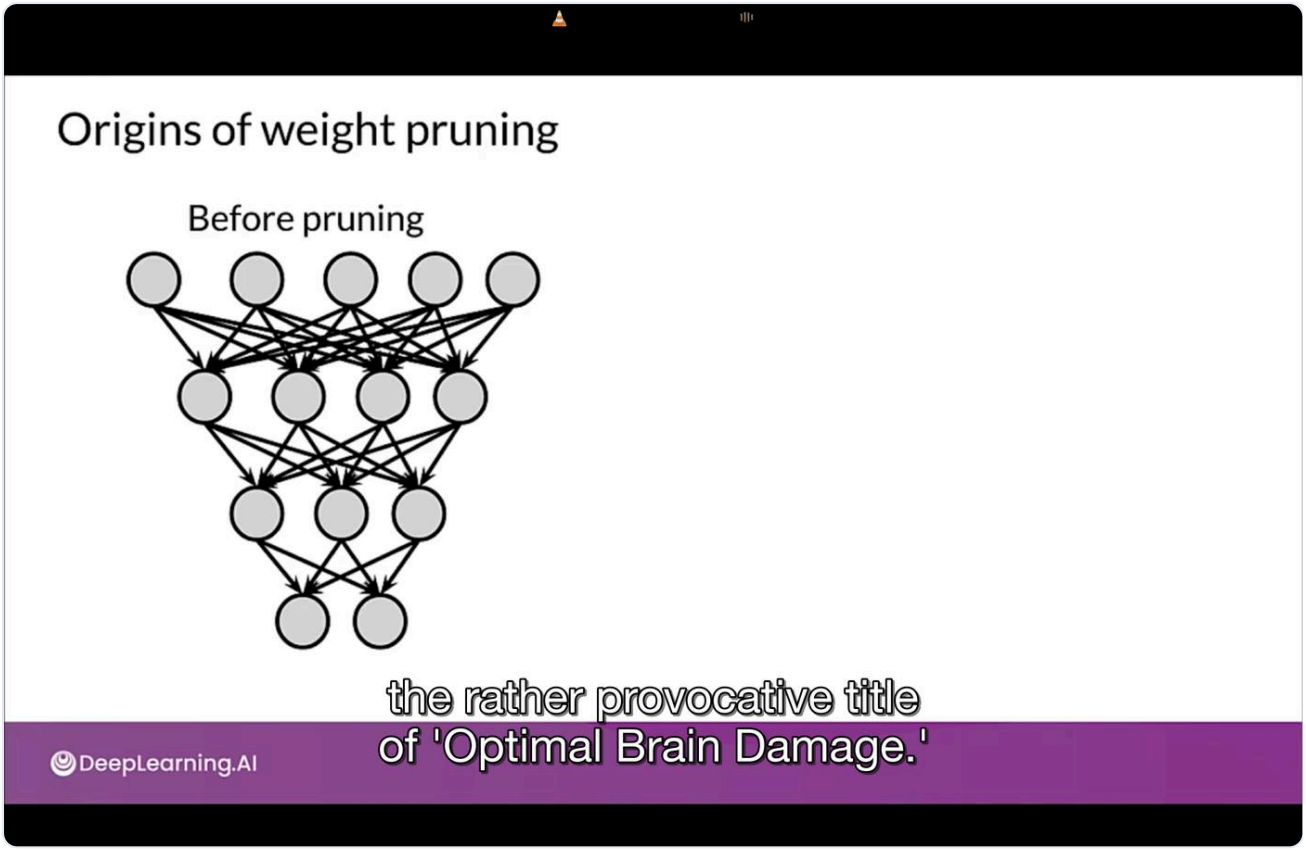

FIGURE 22.5 | Origins of Weight Pruning

Solla and was given the rather provocative title of 'Optimal Brain Damage. At the time, post-pruning neural networks to compress train models was already a popular approach. Pruning was mainly done by using magnitude as an approximation for saliency to determine less useful connections. The intuition being that smaller magnitude weights have a smaller effect in the output, and hence are less likely to have an impact in the model outcome if proven.

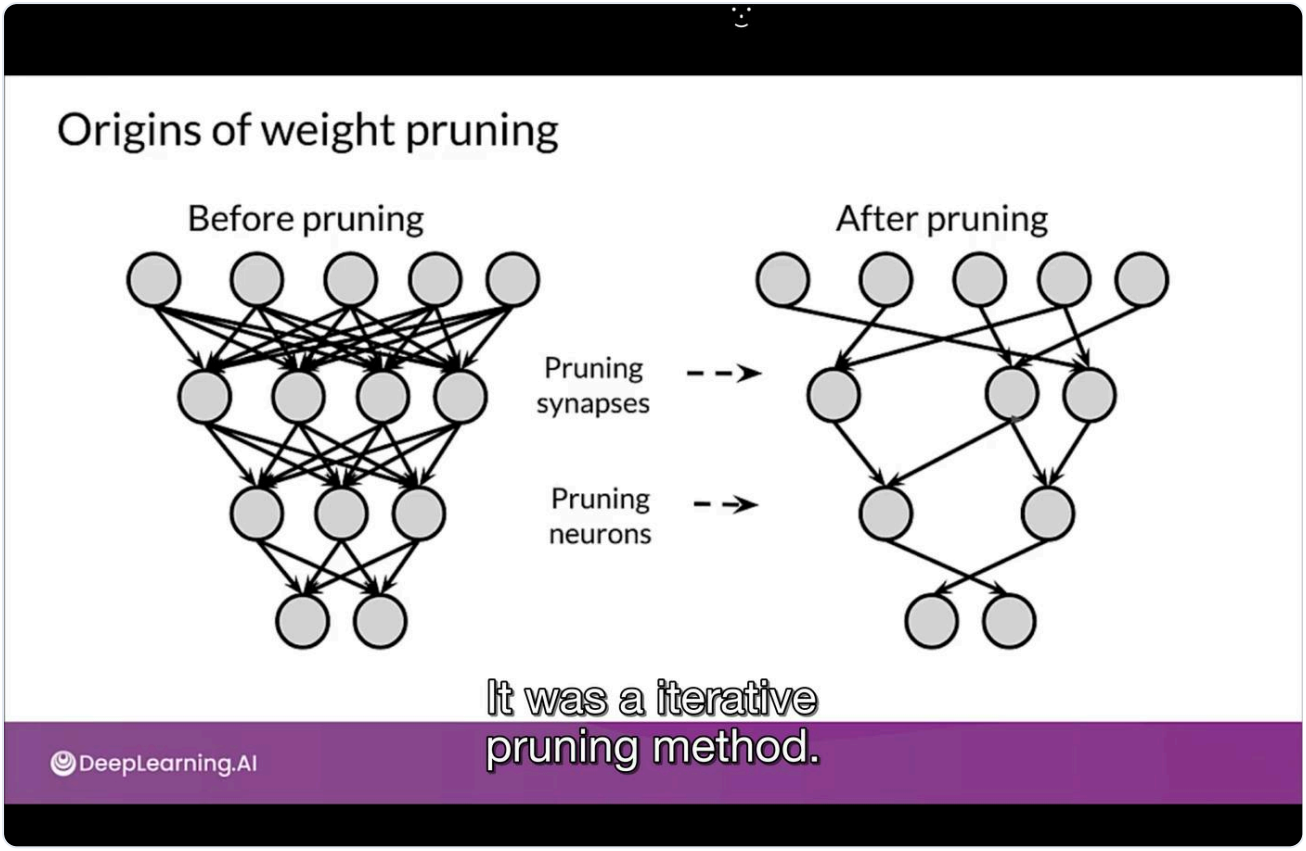

FIGURE 22.6 | Origins of Weight Pruning

It was a iterative pruning method. The first step was to train a model, and then the saliency of each weight was estimated, which was defined by the change in the loss-function upon applying a perturbation to the nodes in the network. The smaller the change, the less the effect the weight would have on the training. Finally, they eliminate the weights with the lowest saliency. This is equivalent to setting them to zero. Finally, this pruned model was retrained. One particular challenge arises with this method when the prune network is retrained. It turned out that due to its decreased capacity, retraining was much more difficult. The solution to this problem arrived later, along with an insight called the lottery ticket hypothesis.

The probability of winning the jackpot of a lottery is very low. For example, if you're playing Powerball, you have odds of exactly one in about 3 million for the winning ticket. What are your chances if you purchase n tickets?

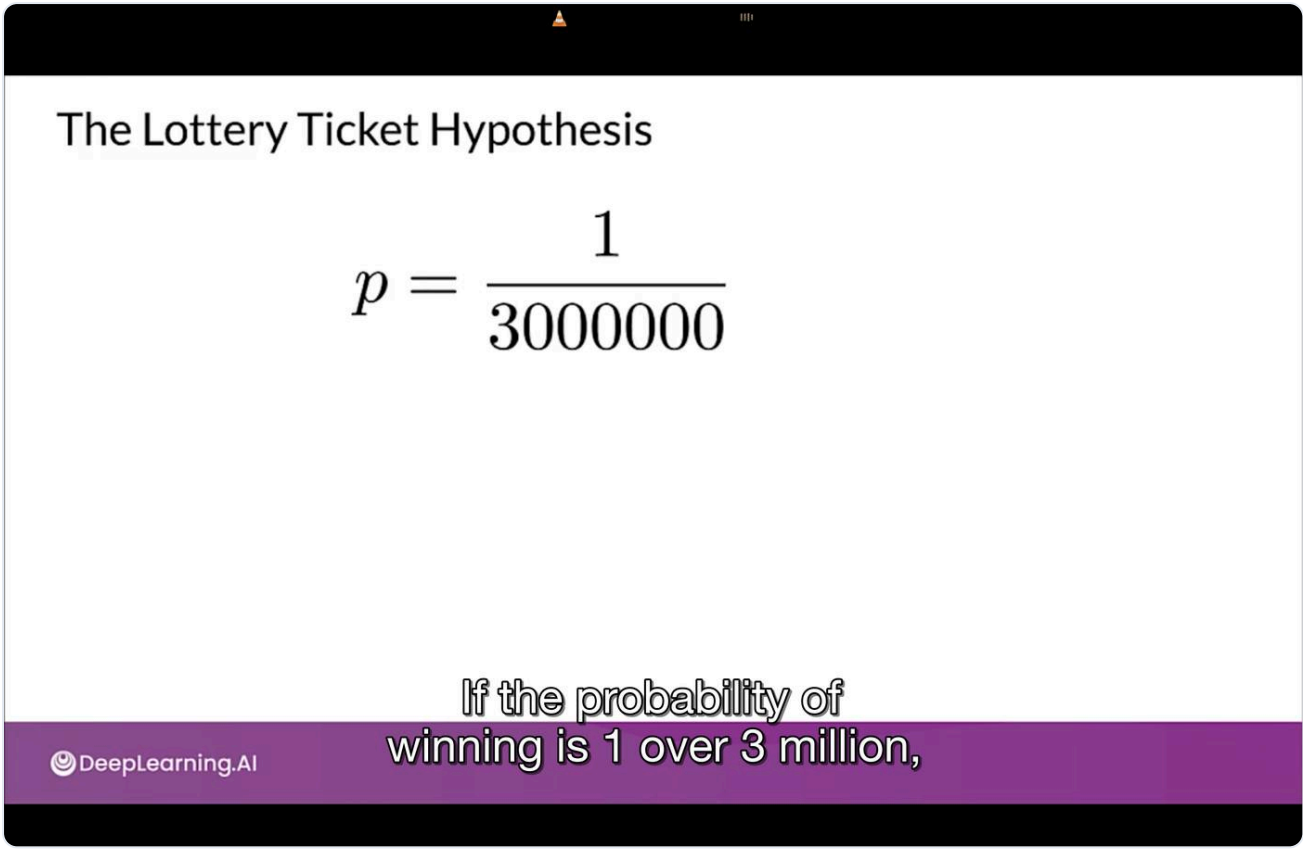

FIGURE 22.7 | The Lottery Ticket Hypothesis

If the probability of winning is 1 over 3 million, then what about the chances of not winning? It's the compliment of 1 minus p. Extend this when buying n tickets and we have the probability of 1 minus p to the power of n. From this, it follows that the probability of at least one of them winning is simply the complement again. What does this have to do with neural networks? Before training, the weights of a model are initialized randomly.

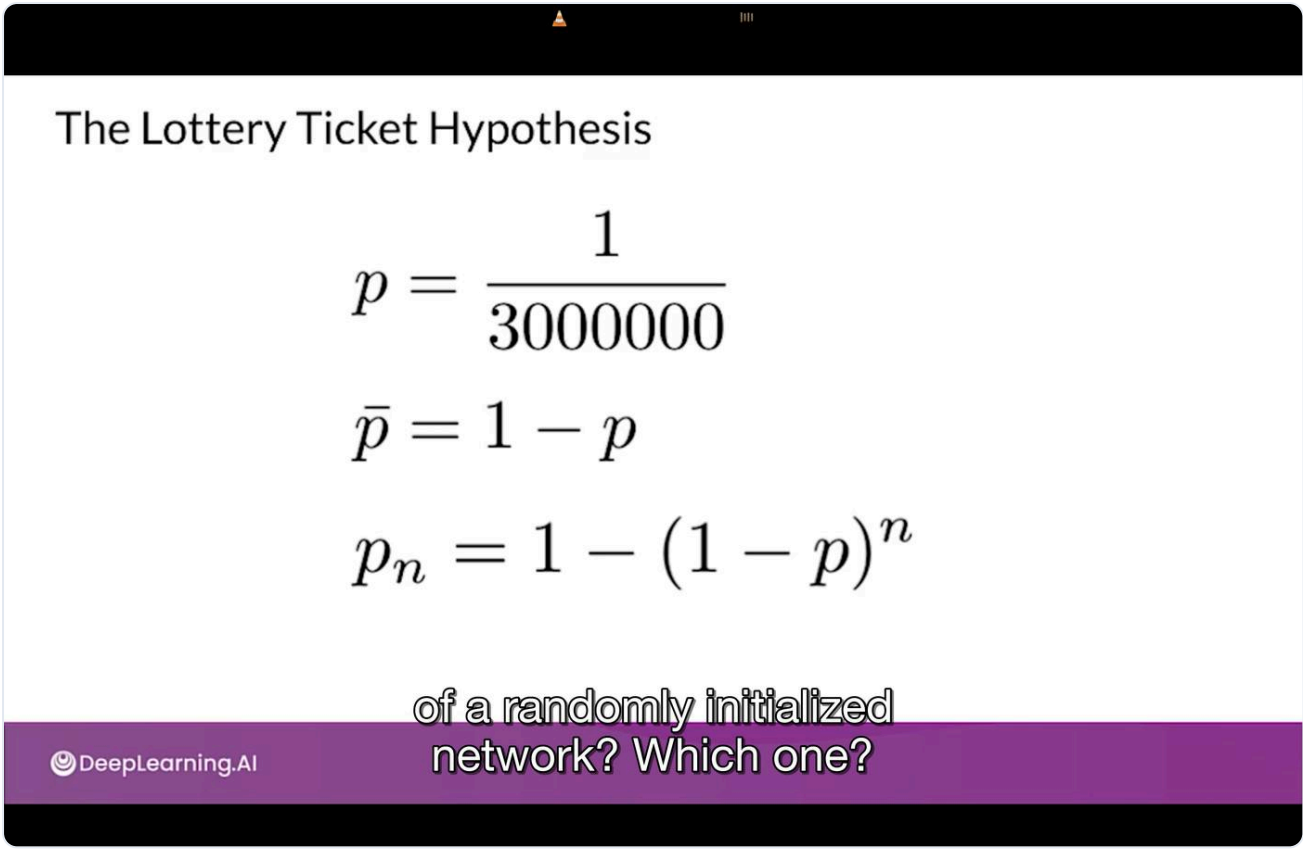

FIGURE 22.8 | The Lottery Ticket Hypothesis

Can it happen that there is a sub-network of a randomly initialized network? Which one? The initialization lottery? Some researchers set out to investigate the problem and answer the question.

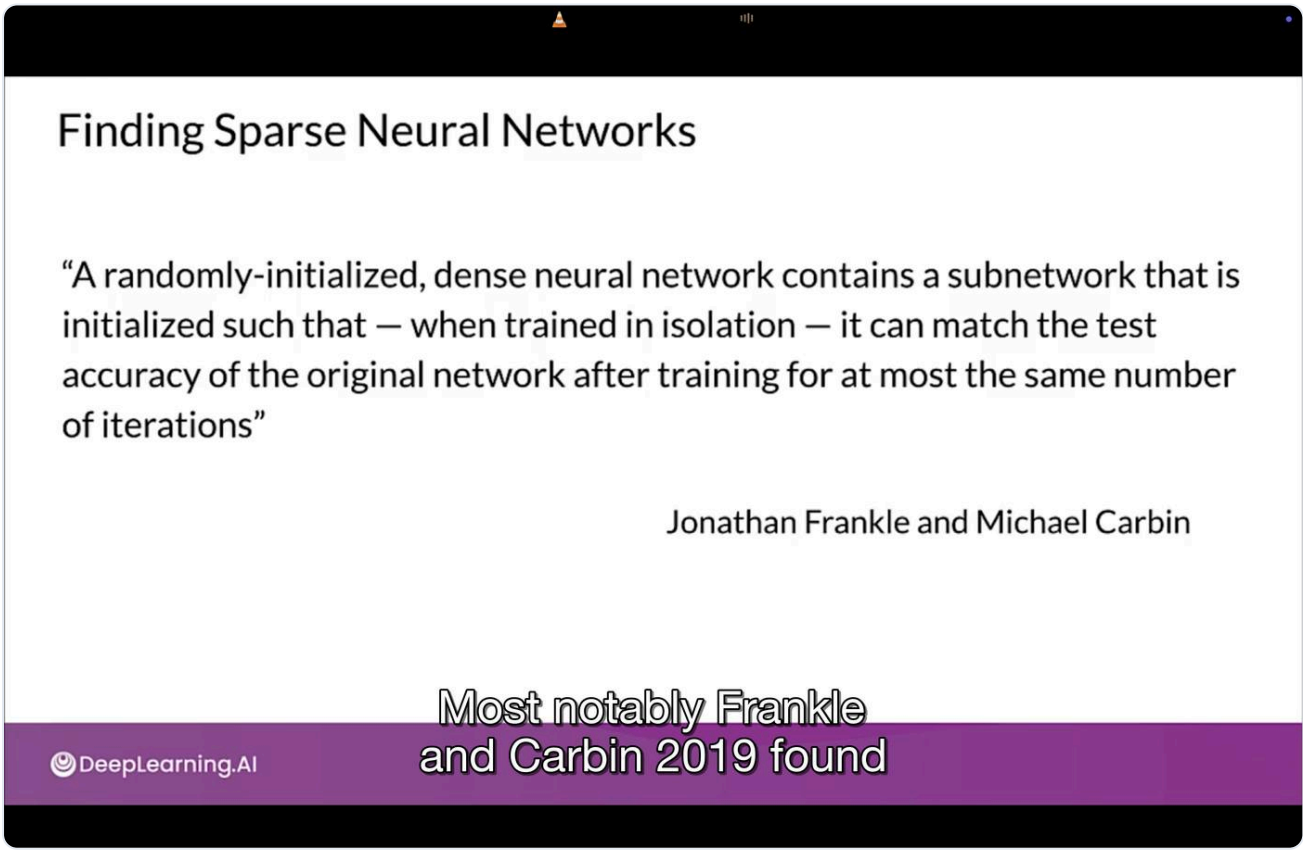

FIGURE 22.9 | Finding Sparse Neural Networks

Most notably Frankle and Carbin 2019 found that fine-tuning the weights after training was not required for these new prune networks. In fact, they showed that the best approach was to reset the weights to their original value and then retrain the entire network. This would lead to models with even higher accuracy compared to both the original dense model and the post-pruning plus fine-tuning approach proposed by Han and colleagues. This discovery led them to propose an idea considered wild at first, but now commonly accepted. That over parameterized dense networks containing several sparse subnetworks with varying performances, and one of these subnetworks is the winning ticket, which outperforms all others.

However, there were significant limitations to this method. For one, it does not perform well for larger-scale problems and architectures. The original paper, the authors stated that for more complex datasets like ImageNet and deeper architectures like ResNet, the method fails to identify the winners of the initialization lottery.

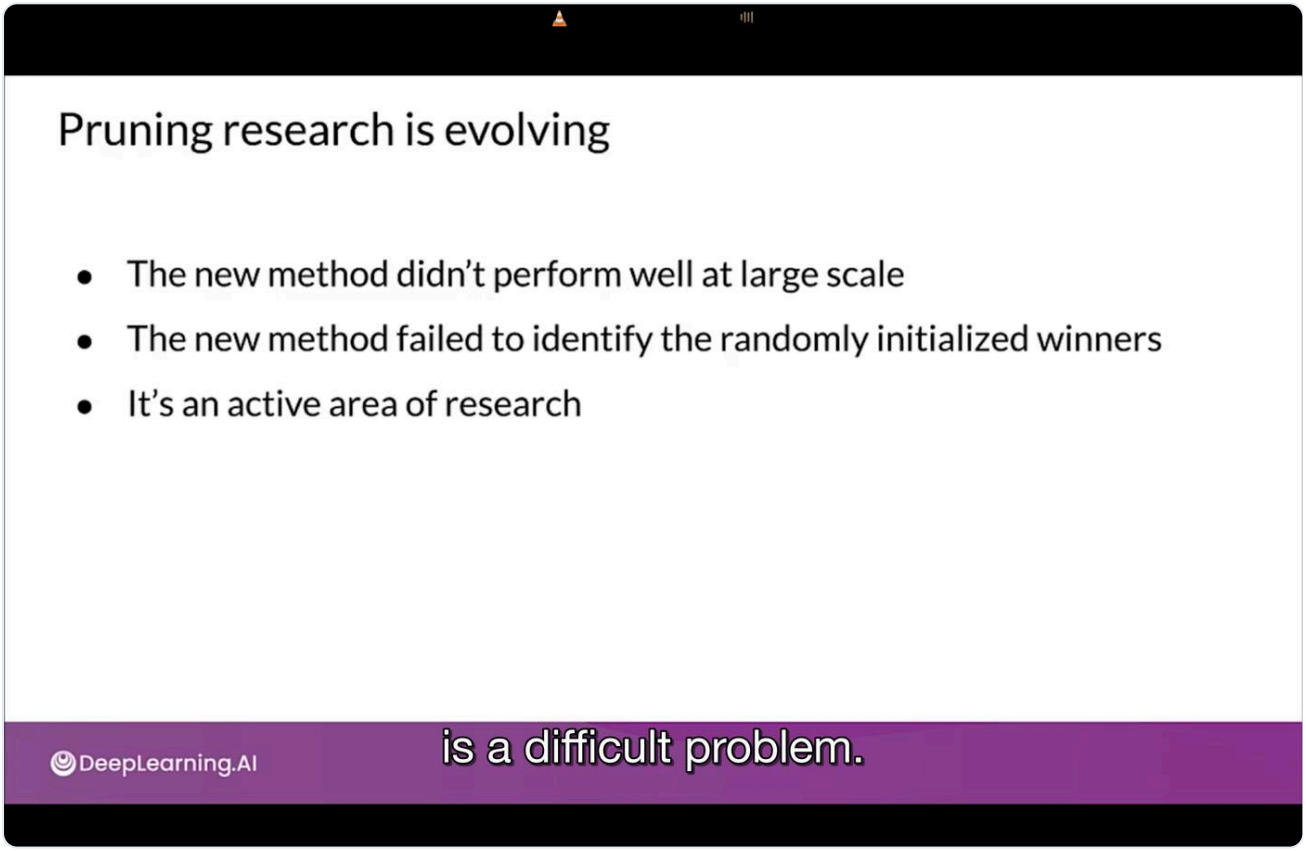

FIGURE 22.10 | Pruning Research Is Evolving

In general, achieving a good sparsity accuracy tradeoff is a difficult problem. Is a very active research field and the state of the art keeps improving.

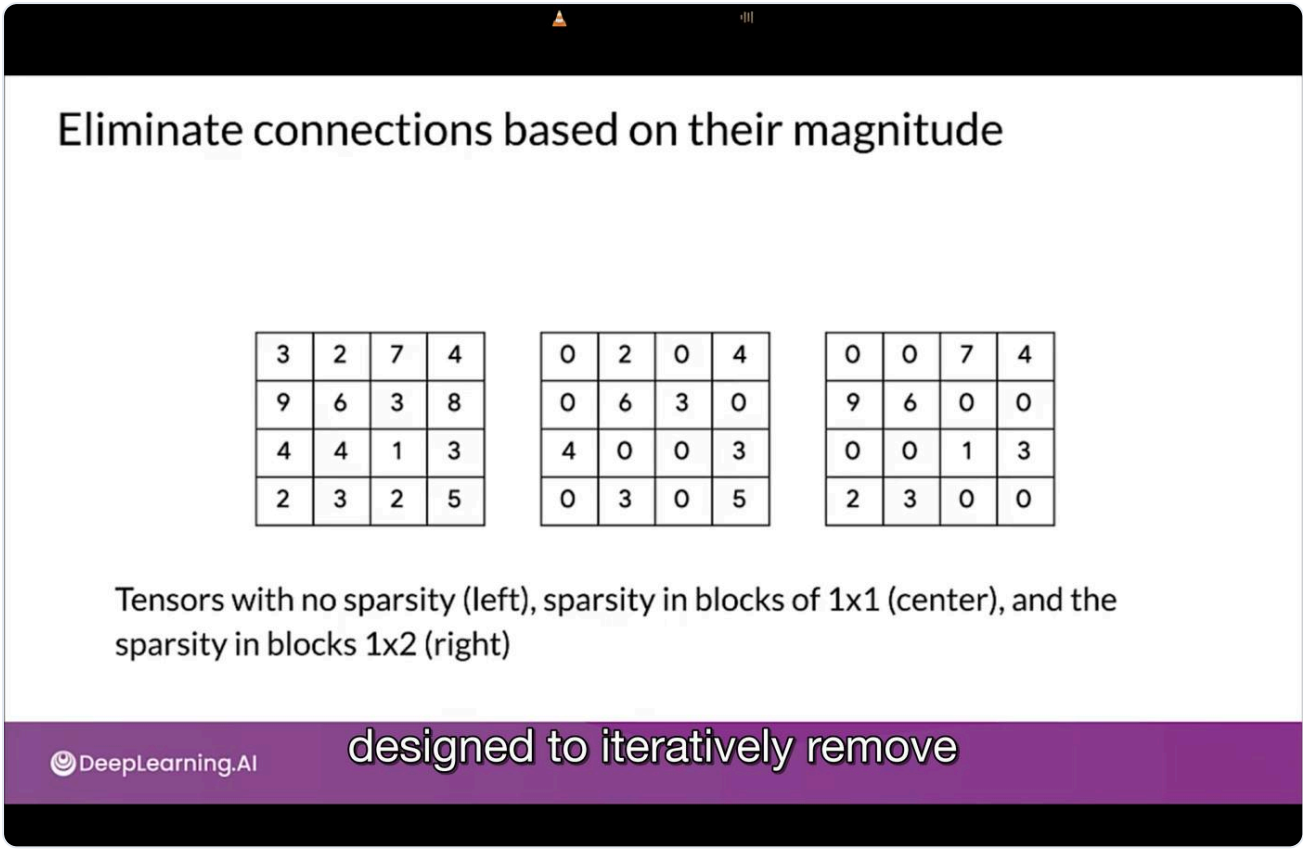

FIGURE 22.11 | Eliminate Connections Based on Their Magnitude

TensorFlow includes a Keras-based weight pruning API, which uses a straightforward yet broadly applicable algorithm designed to iteratively remove connections based on their magnitude during training.

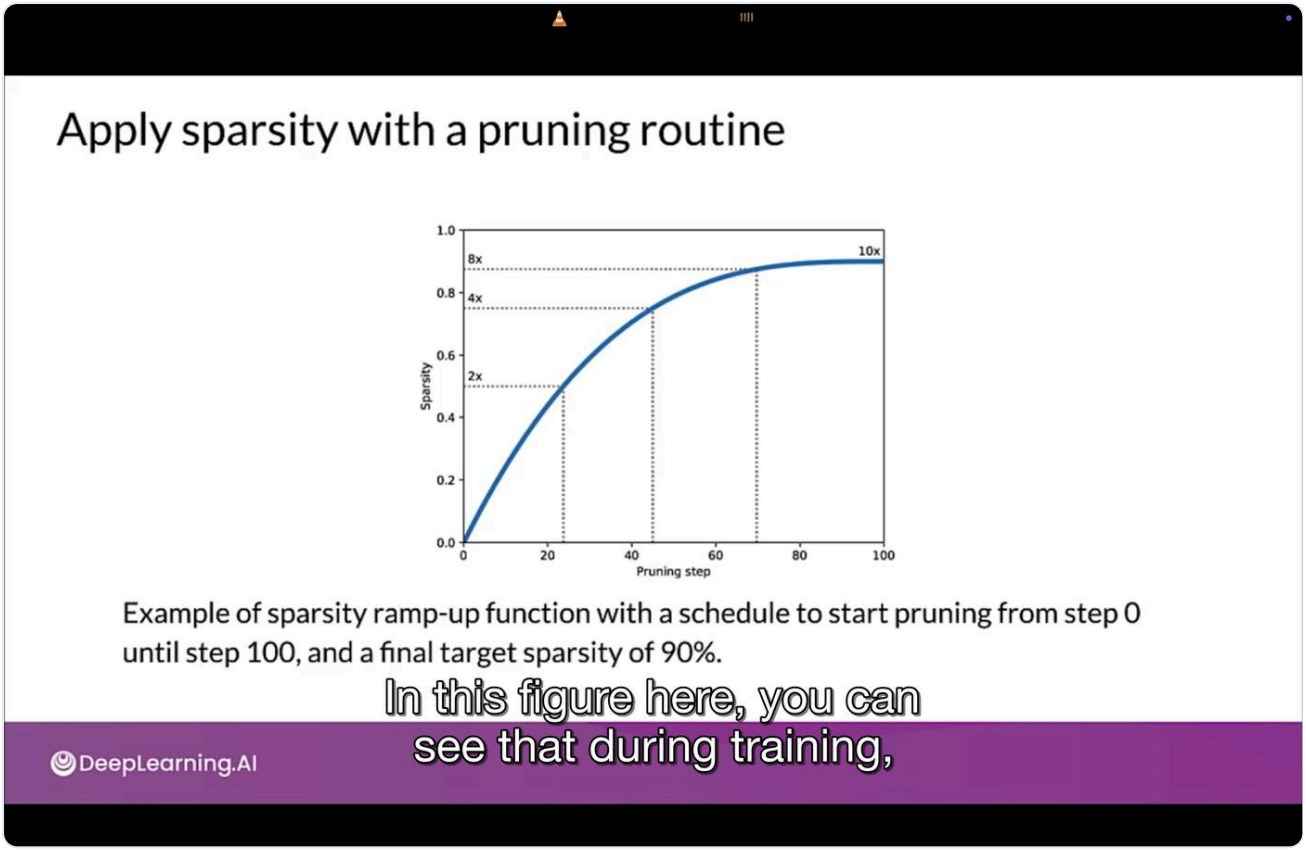

FIGURE 22.12

Fundamentally a final target sparsity is specified along with a schedule to perform the pruning. In this figure here, you can see that during training, a pruning routine will be scheduled to execute. Removing the weights with the lowest magnitude values that are closest to zero until the current sparsity target is reached. Every time the pruning routine is scheduled to execute, the current sparsity target is recalculated starting from zero percent until it reaches the final target sparsity at the end of the pruning schedule.

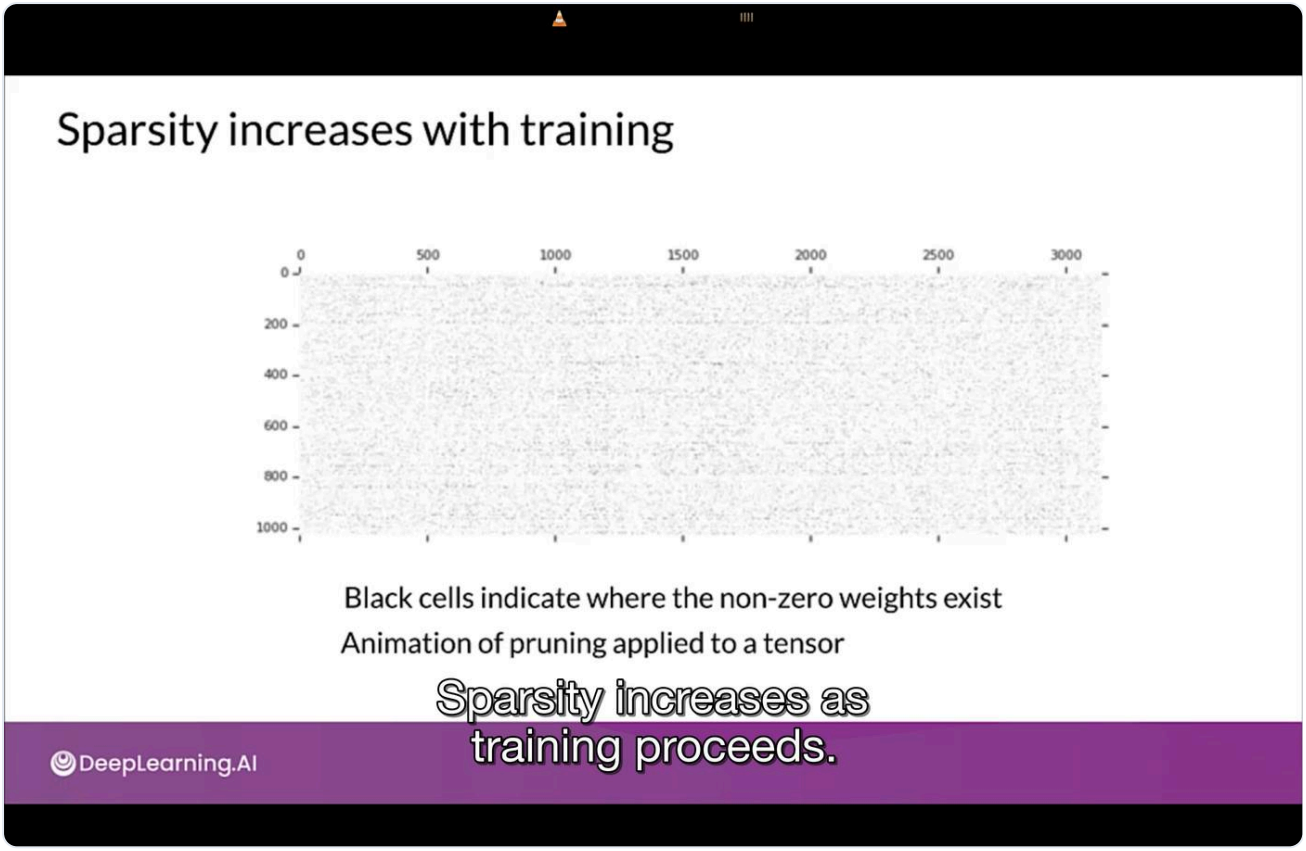

FIGURE 22.13 | Sparsity Increases with Training

By gradually increasing it according to a smooth ramp-up function. Just like the schedule, the ramp-up function can be tweaked as needed. For example, in certain cases, it may be convenient to schedule the training procedure to start after a certain step when some convergence level has been achieved. Or end pruning earlier than the total number of training steps in your training program to further fine-tune the system at the final target sparsity level. Sparsity increases as training proceeds.

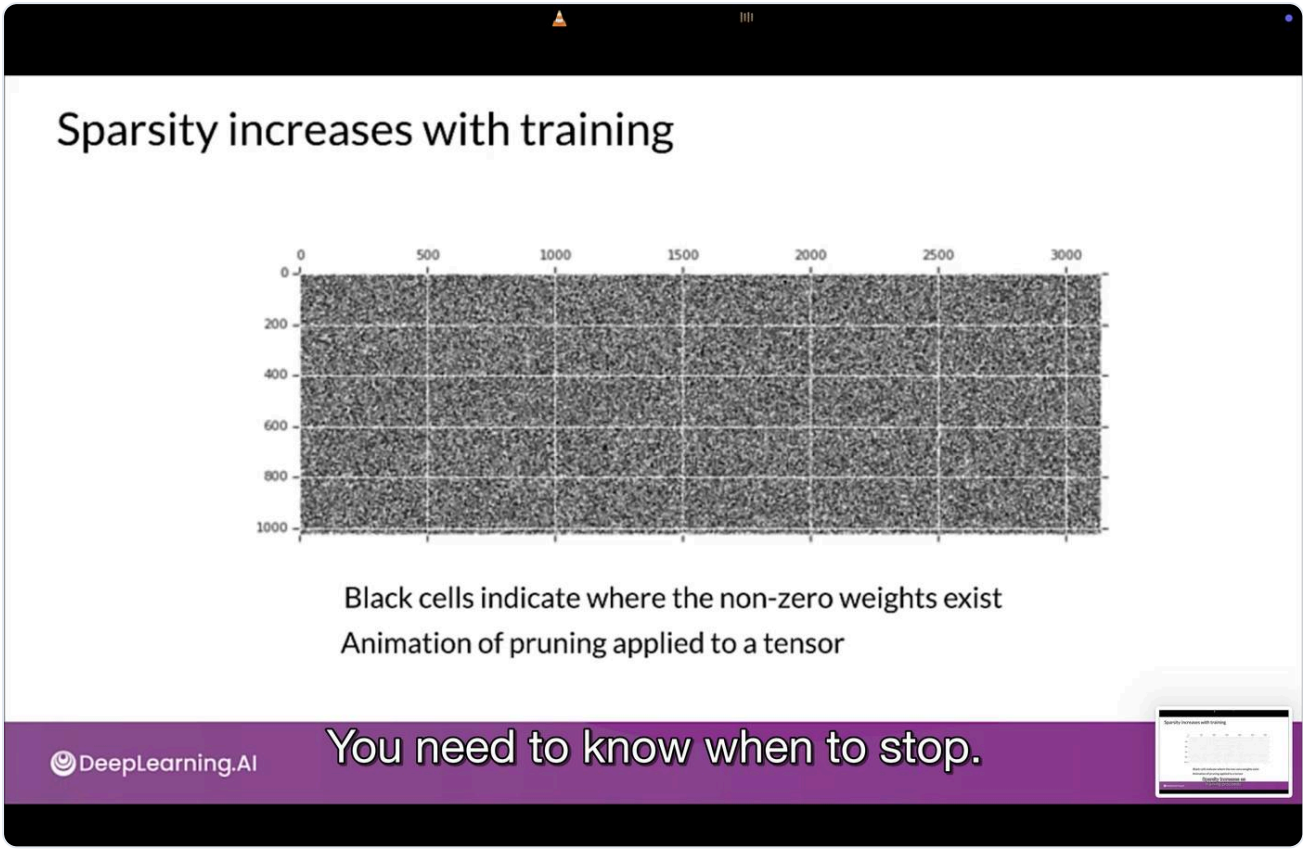

FIGURE 22.14 | Sparsity Increases with Training

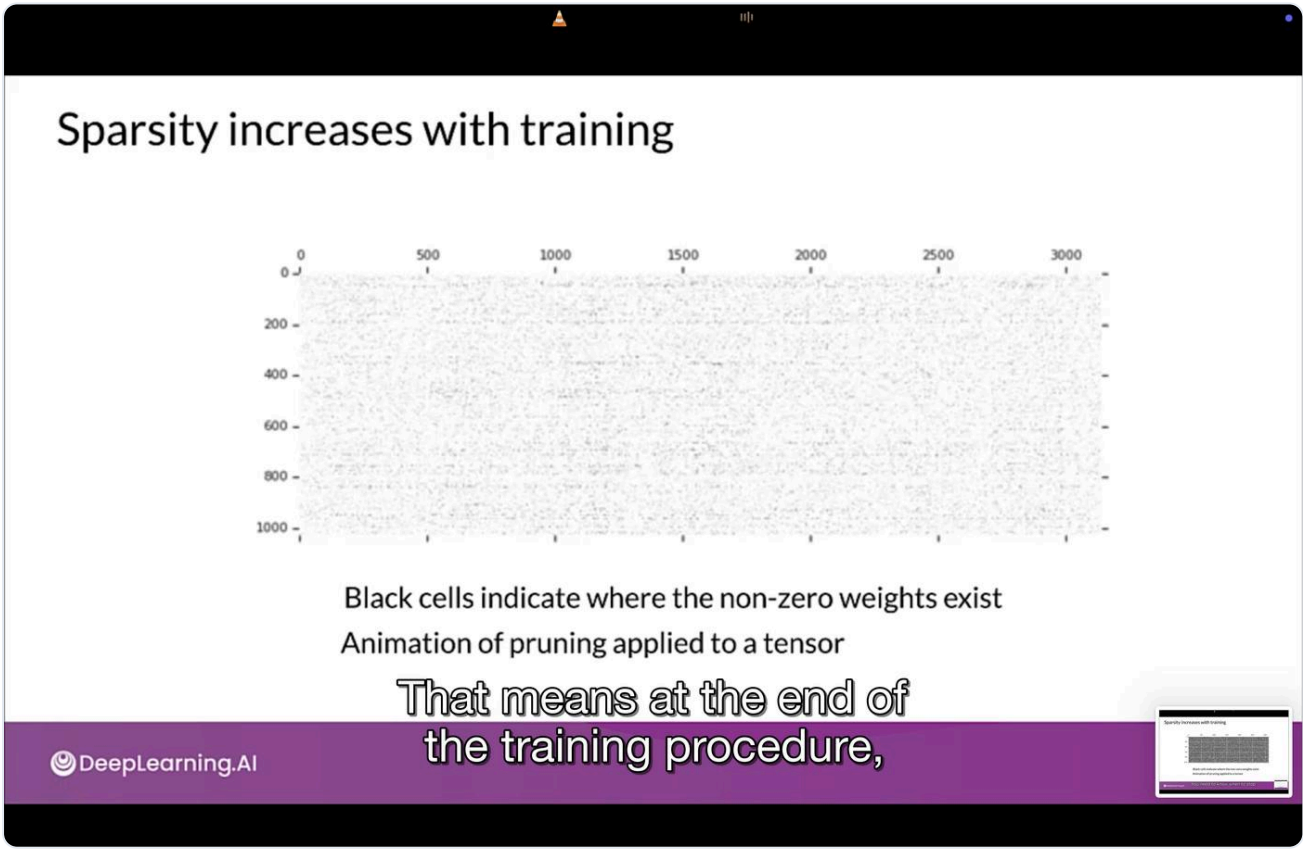

FIGURE 22.15 | Sparsity Increases with Training

You need to know when to stop.

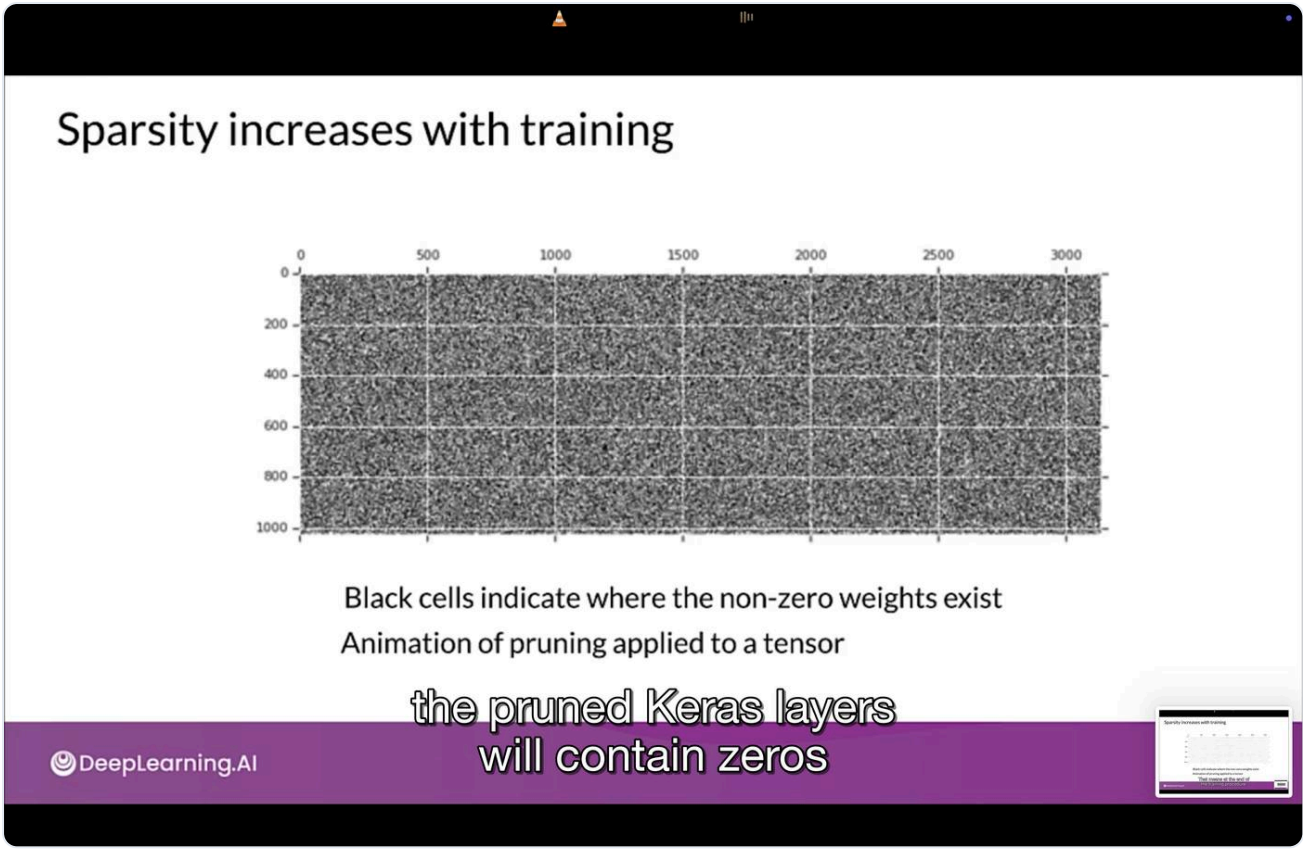

FIGURE 22.16 | Sparsity Increases with Training

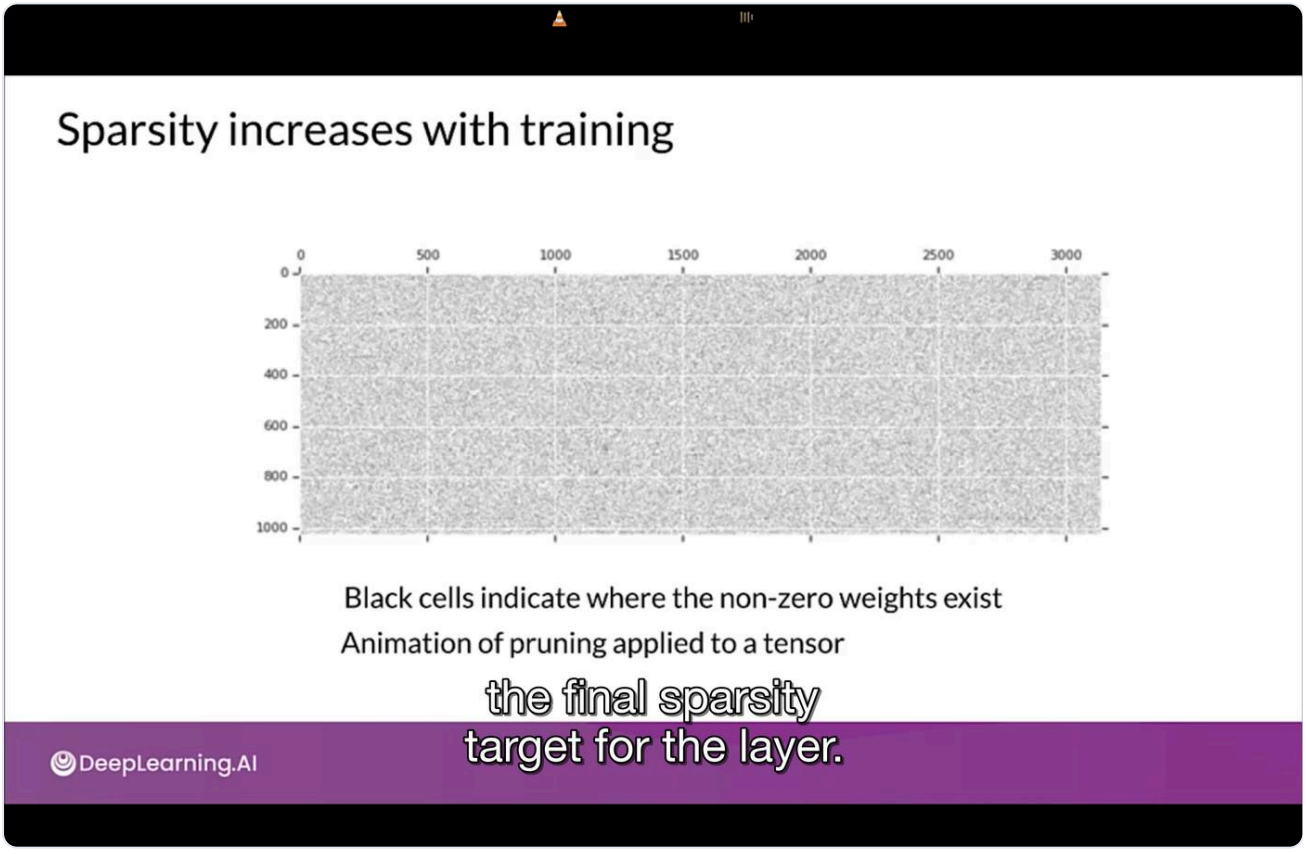

FIGURE 22.17 | Sparsity Increases with Training

That means at the end of the training procedure, the tensors corresponding to the pruned Keras layers will contain zeros where weights have been pruned according to the final sparsity target for the layer. An immediate benefit that you can get out of pruning is disk compression. That's because sparse tensors are compressible. Thus by applying simple file compression to the prune TensorFlow checkpoint or the converted TensorFlow Lite model. We can reduce the size of the model for storage and/or transmission. In some cases, you can even gain speed improvements in CPU and machine-learning accelerators that exploit integer precision efficiencies. Moreover, across several experiments, we found that weight pruning is compatible with quantization, resulting in compound benefits.

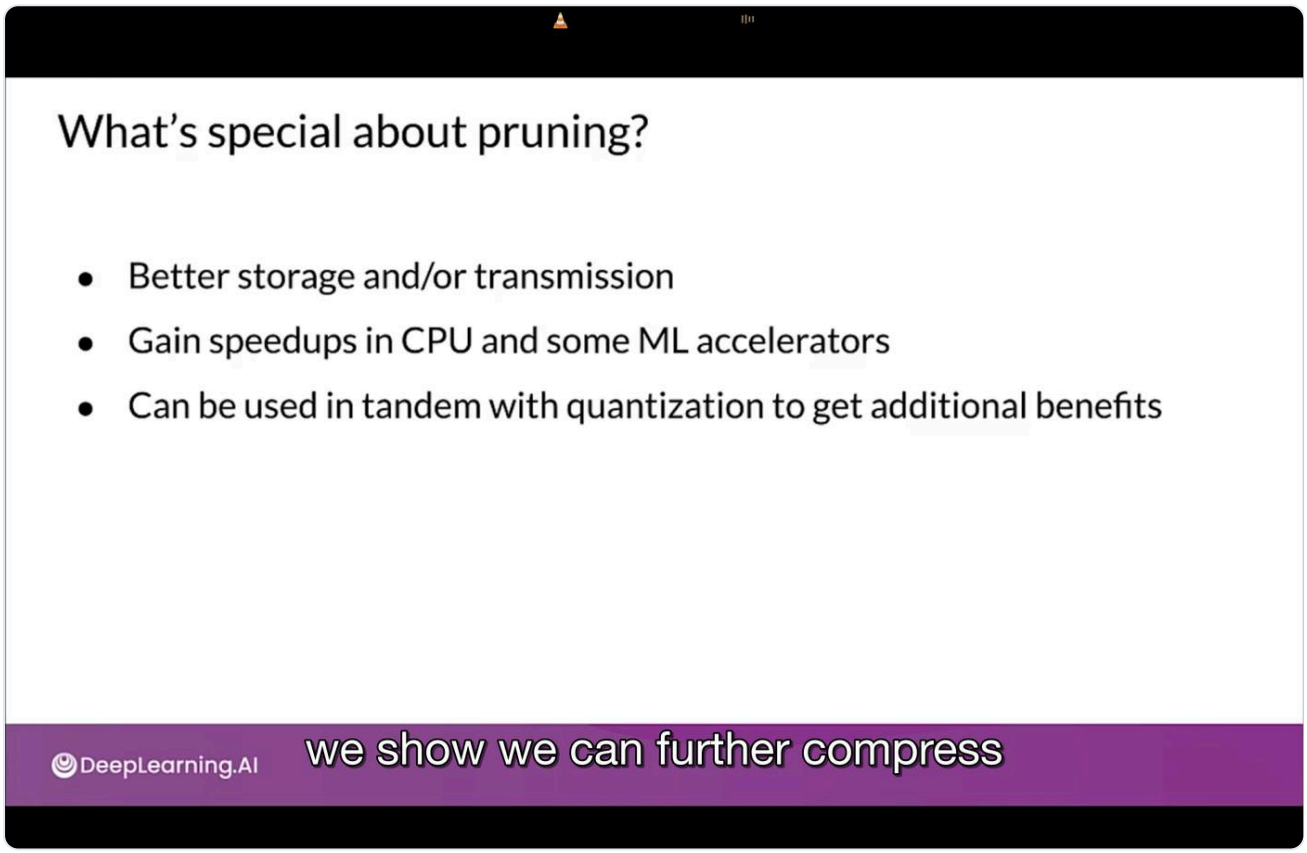

FIGURE 22.18

In the upcoming exercise, we show we can further compress the pruned model from two megabytes to just about half of a megabyte by applying post-training quantization.

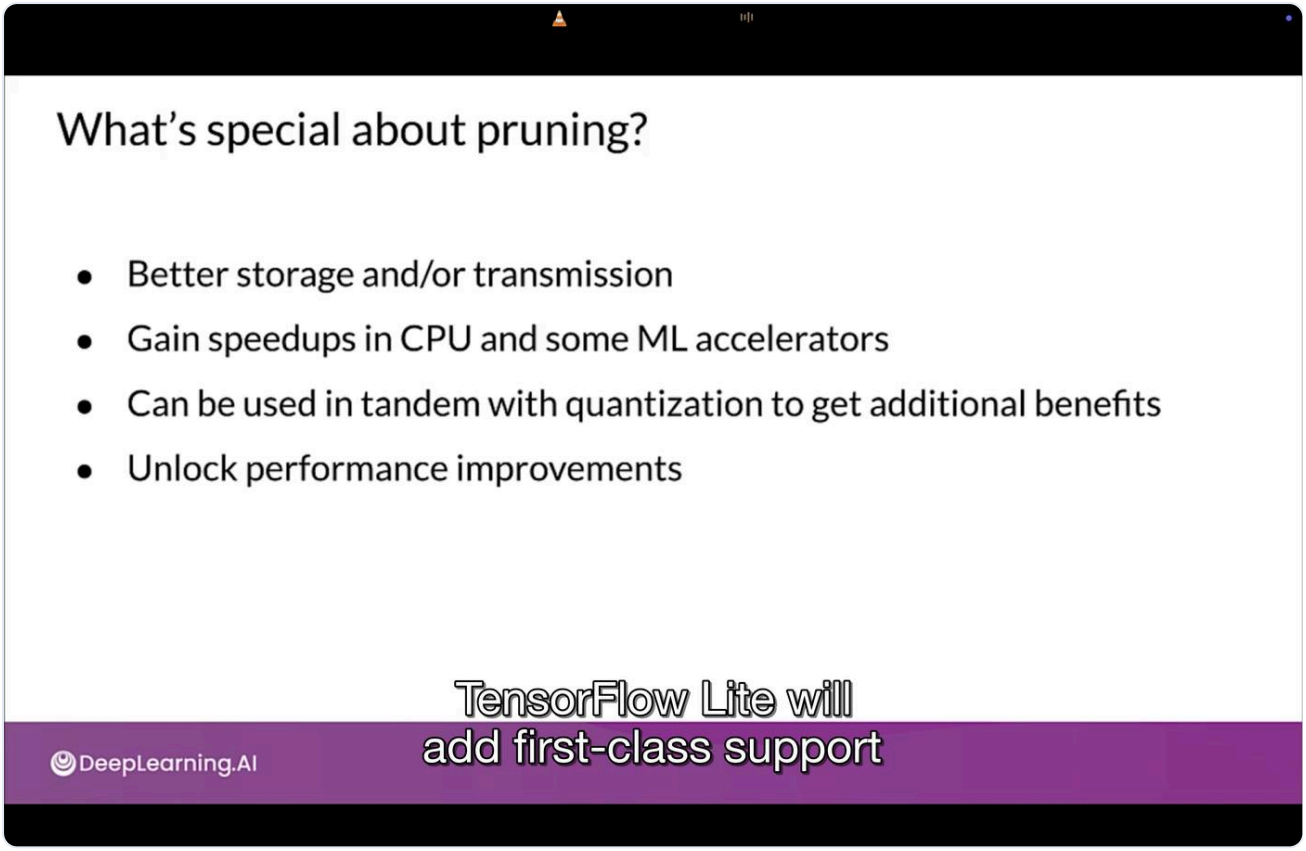

FIGURE 22.19

In the relatively near future, TensorFlow Lite will add first-class support for sparse representation in computation. Thus expanding the compression benefit to runtime memory and unlocking performance improvements. Sparse tensors allow you to skip otherwise unnecessary computations involving the zeroed values. Or depending on when you're watching this, it may already be included.

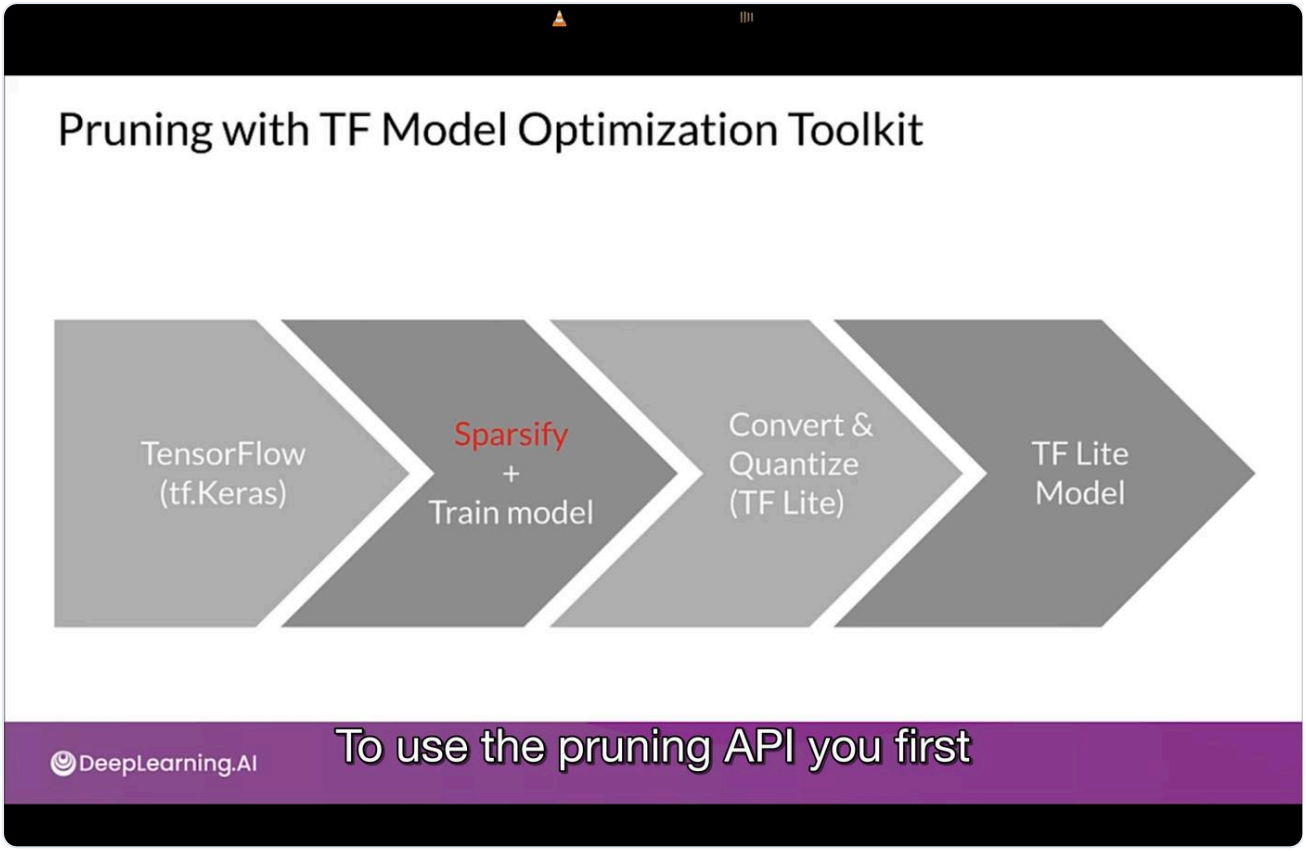

FIGURE 22.20 | Pruning with TF Model Optimization Toolkit

To use the pruning API you first create a TensorFlow Keras model. Then we add sparsity to some of the layers in the model and retrain it or train it. Finally, you can also read the benefits of quantization by converting the pruned model to TFLite.

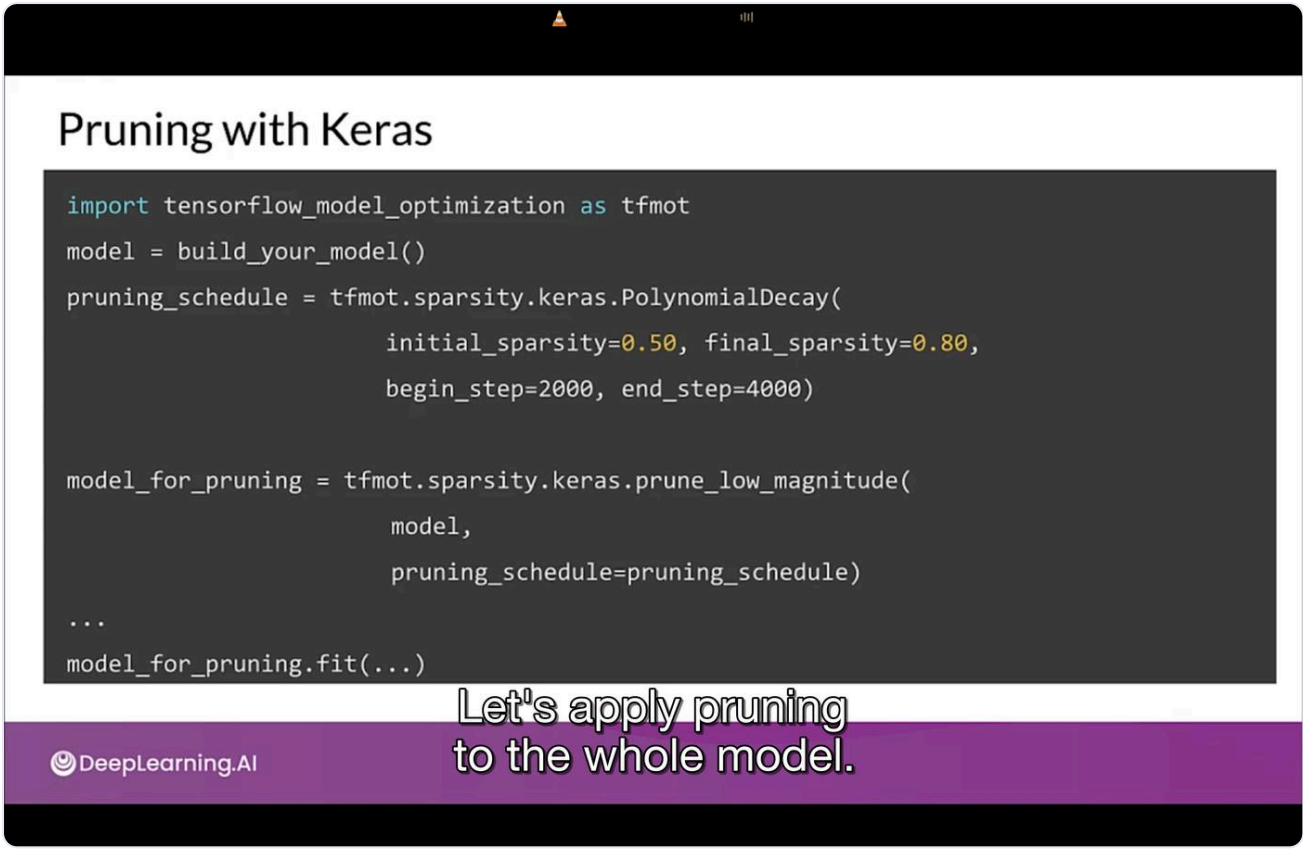

FIGURE 22.21 | Pruning with Keras

Let's apply pruning to the whole model. In this example, you start with 50 percent sparsity. 50 percent zeros and weights and end up with 80 percent sparsity. You can also prune only a part of the model or specific layers for model accuracy improvements. Later, you'll see how to create sparse models with the TensorFlow model optimization toolkit API for both TensorFlow and TFLite. You then combine pruning with post-training quantization for additional benefits.

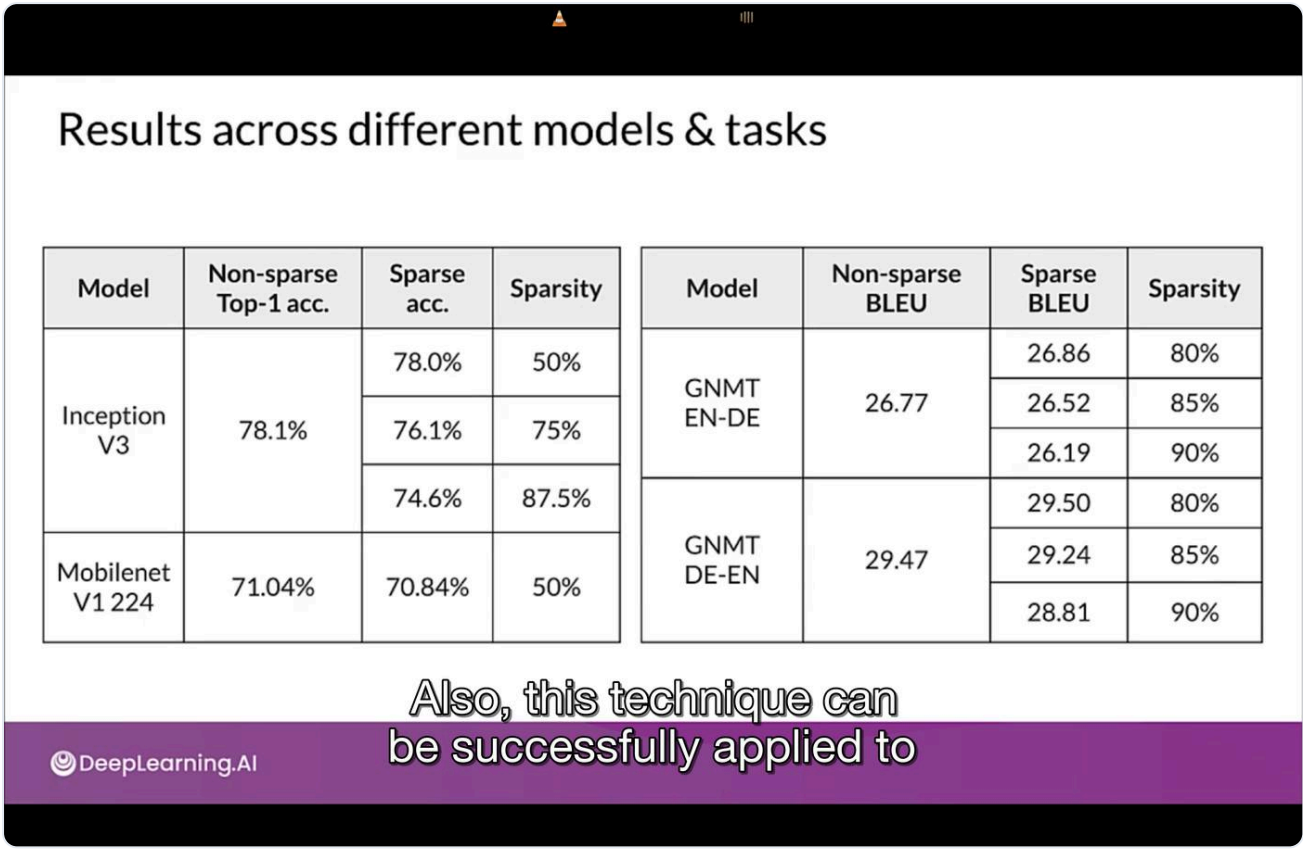

FIGURE 22.22 | Results Across Different Models & Tasks

Also, this technique can be successfully applied to different types of models across distinct tasks. From image processing convolutional-based neural networks to speech processing using recurrent neural networks. This table shows some of these experimental results. That was quite a week. We started with understanding dimensionality and looked at a few ways to control it and reduce it. Then we went into some resource constraints scenarios, including mobile and IoT applications. We looked at some of the issues around that and ways to deal with that as well. Including quantization of a couple of different kinds and pruning. I hope you enjoyed it. I thought it was a great week and we'll see you next time.

LECTURE 23

Types of Distributed Training

Welcome back. We've got a great week planned. We're going to start with the discussion of distributed training, including a couple of different kinds of parallelism. Then we'll turn to high performance modeling, including high performance ingestion. Then we're going to turn to one of my favorite topics, knowledge distillation. So I'm really looking forward to this, I hope you enjoy it. Let's get started. We'll start with the discussion of distributed training. When you start prototyping, training your model might be a fast and simple task. However, fully training a model can become a very time-consuming task. Datasets in many domains are getting larger and larger.

As the size of the datasets increase, models take longer and longer to train. The same logic applies for larger architectures, and it's not just training time. The number of epochs for a model also increases as a result.

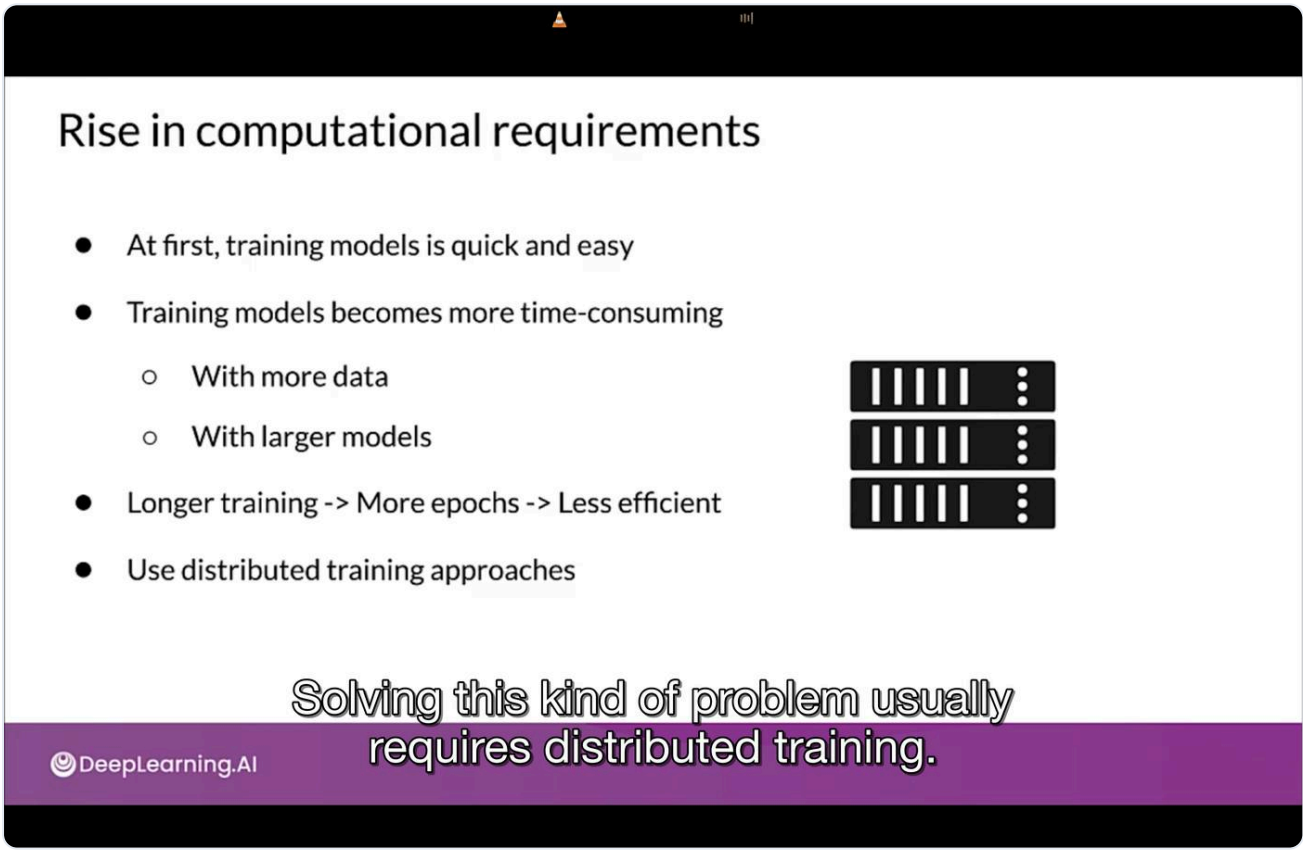

FIGURE 23.1

Solving this kind of problem usually requires distributed training. Distributed training allows us to train huge models, and at the same time, speed up training. In this lesson, you'll see how models can be accelerated using data and model parallelism. Also, you'll look at high performance modeling techniques like distribution strategies and high performance ingestion pipelines such as TF data. Now, let's look into basic ways to perform distributed training. These include data parallelism and model parallelism. Data parallelism is probably the easiest of the two to imple-

ment. In this approach, you divide the data into partitions. You copy the complete model to all of the workers, where each one operates on a different partition of the data and the model updates are synchronized across workers.

This type of parallelism is model agnostic and can be applied to any neural network architecture. Usually, the scale of data parallelism corresponds to the batch size.

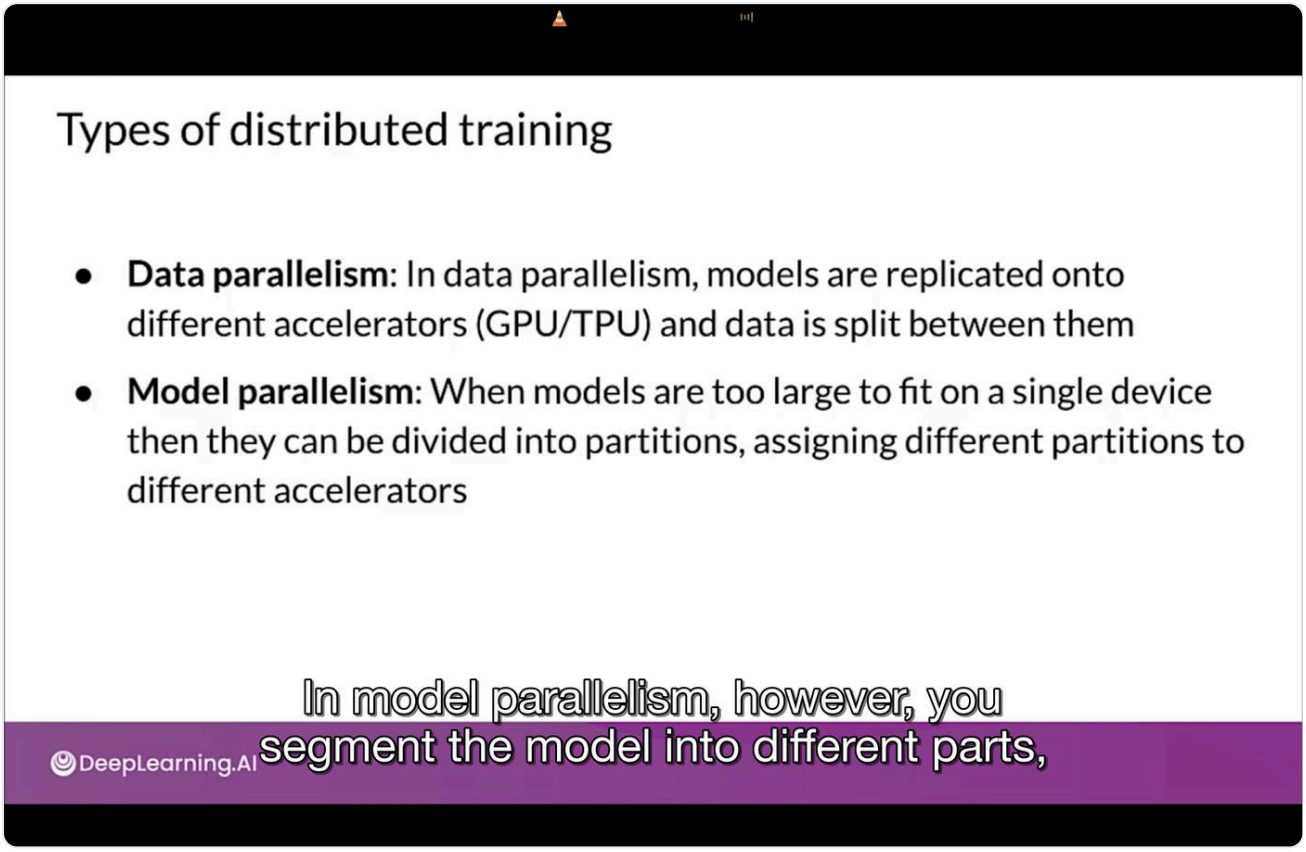

FIGURE 23.2 | Types of Distributed Training

In model parallelism, however, you segment the model into different parts, training concurrently on different workers. Each model will train on the same piece of data. Here workers only need to synchronize the shared parameters, usually once for each forward or back propagation step. You generally use model parallelism when you have a larger model which won't fit into memory on your accelerators. Implementation of model parallelism is relatively advanced compared to data parallelism. Model parallelism is a more complex topic. So for now, let's look at how data parallelism works with an illustration and then revisit model parallelism. In data parallelism, the data is split into partitions.

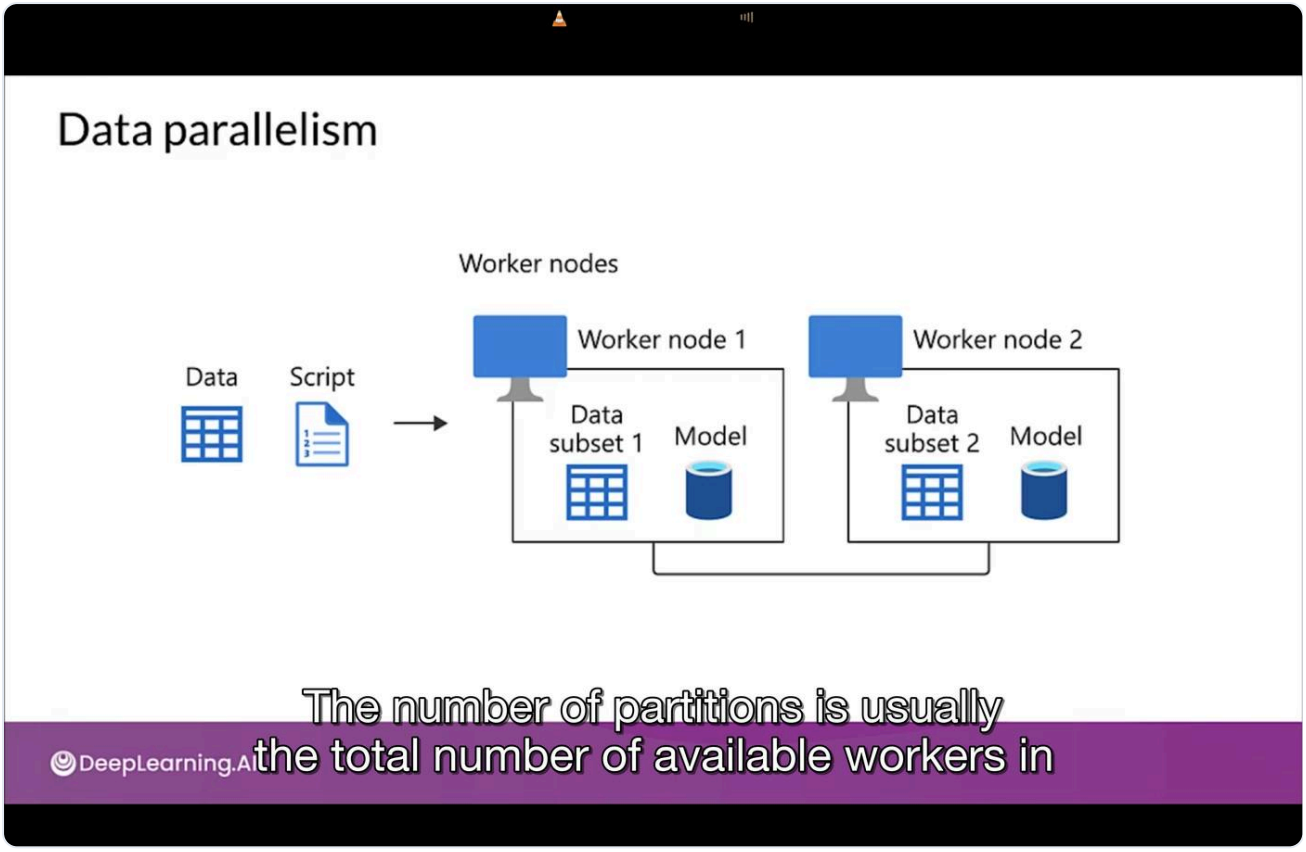

FIGURE 23.3 | Data Parallelism

The number of partitions is usually the total number of available workers in the compute cluster. To train, you copy the model onto each of these worker nodes, with each worker training on its own subset of the data. This requires each worker to have enough memory to load the entire model, which for larger models can be a problem. Each worker independently computes the errors between its predictions for its training samples and the labeled data. Then each worker performs backprop to update its model based on the errors, and communicates all of its changes to the other workers so that they can update their models. This means that the workers need to synchronize their gradients at the end of each batch to ensure that they are training a consistent model.

There are two basic styles of distributed training using data parallelism. In synchronous training, each worker trains on its current mini batch of data, applies its own updates, communicates out its updates to the other workers. And waits to receive and apply all of the updates from the other workers before proceeding to the next mini batch. And all-reduce algorithm is an example of this.

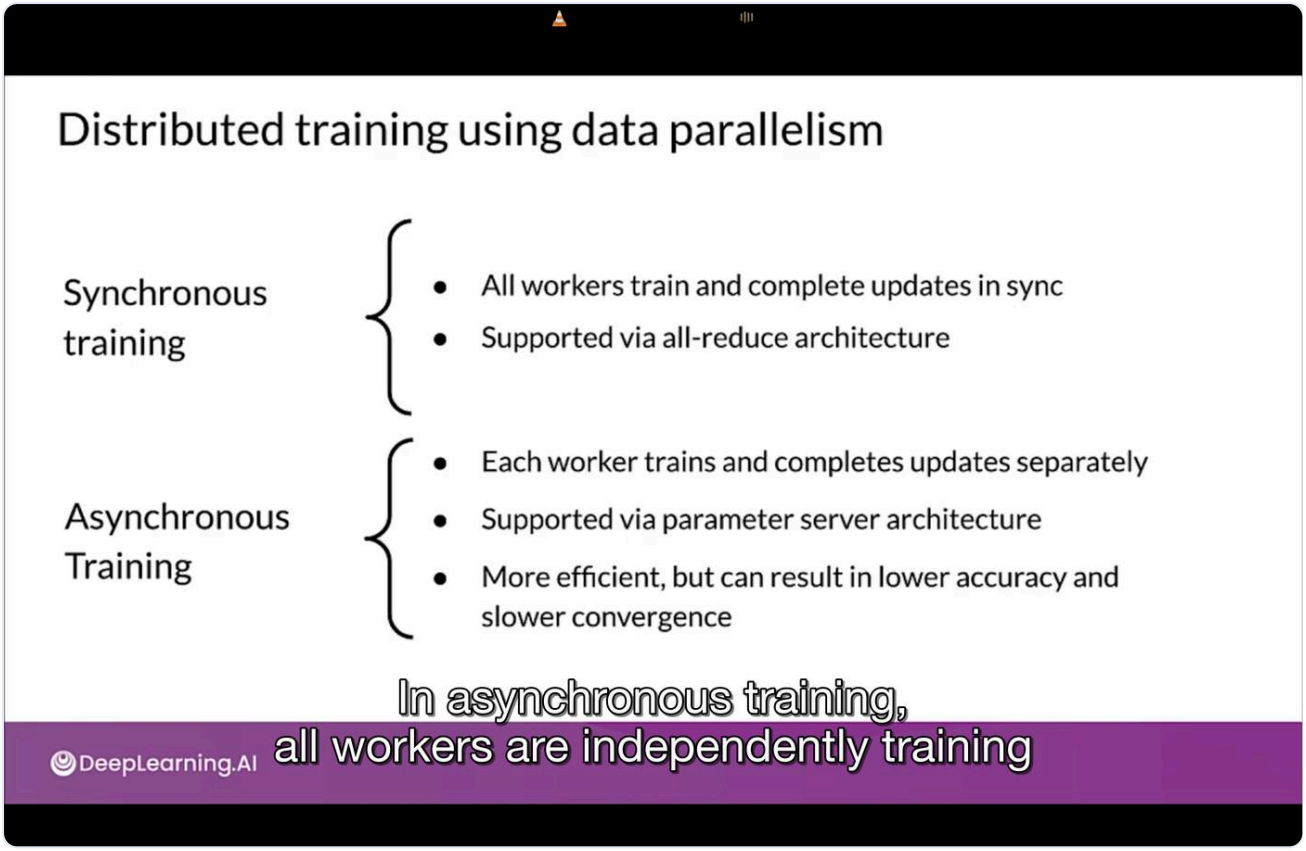

FIGURE 23.4 | Distributed Training Using Data Parallelism

In asynchronous training, all workers are independently training over their mini batch of data and updating variables asynchronously. Asynchronous training tends to be more efficient, but can be more difficult to implement. Parameter server algorithm is an example of this. One major disadvantage of asynchronous training is reduced accuracy and slower convergence, which means that more steps are required to converge. Slow convergence may not be a problem, since the speed up in asynchronous training may be enough to compensate. However, the accuracy loss may be an issue depending on how much accuracy is lost and the requirements of the application. To use distributed training, it's important that models become distribute-aware.

Fortunately, high-level APIs like Keras or Estimators support distributed training. You can even create custom training loops to provide more precise control. You'll see that to make these ordinary models capable of performing training or inference in a distributed manner, you need to make them distribute-aware with some small changes of code.

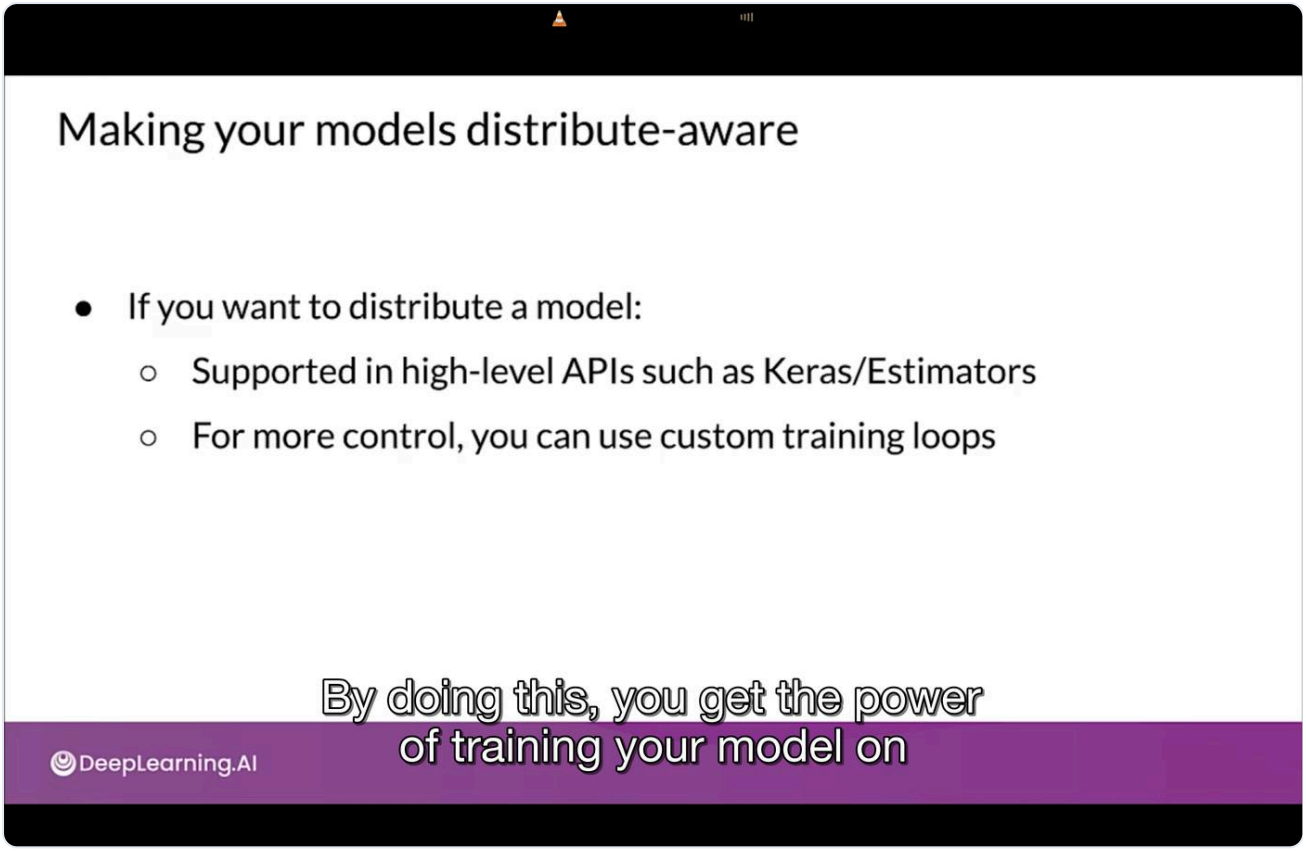

FIGURE 23.5

By doing this, you get the power of training your model on a large number of accelerators, like GPUs or TPUs. In particular, to perform distributed training in TensorFlow, you can make use of TensorFlow's distribute.Strategy class library. This class supports several distribution strategies for high-level APIs and also supports training using a custom training loop. The tf.distribute.Strategy class supports the execution of TensorFlow code not only in eager mode, but also in graph mode, as well as using tf function. In addition to training models, it's also possible to use this API to perform model evaluation and prediction in a distributed manner on different platforms.

It requires a minimal amount of extra code to adapt your models for distributed training.

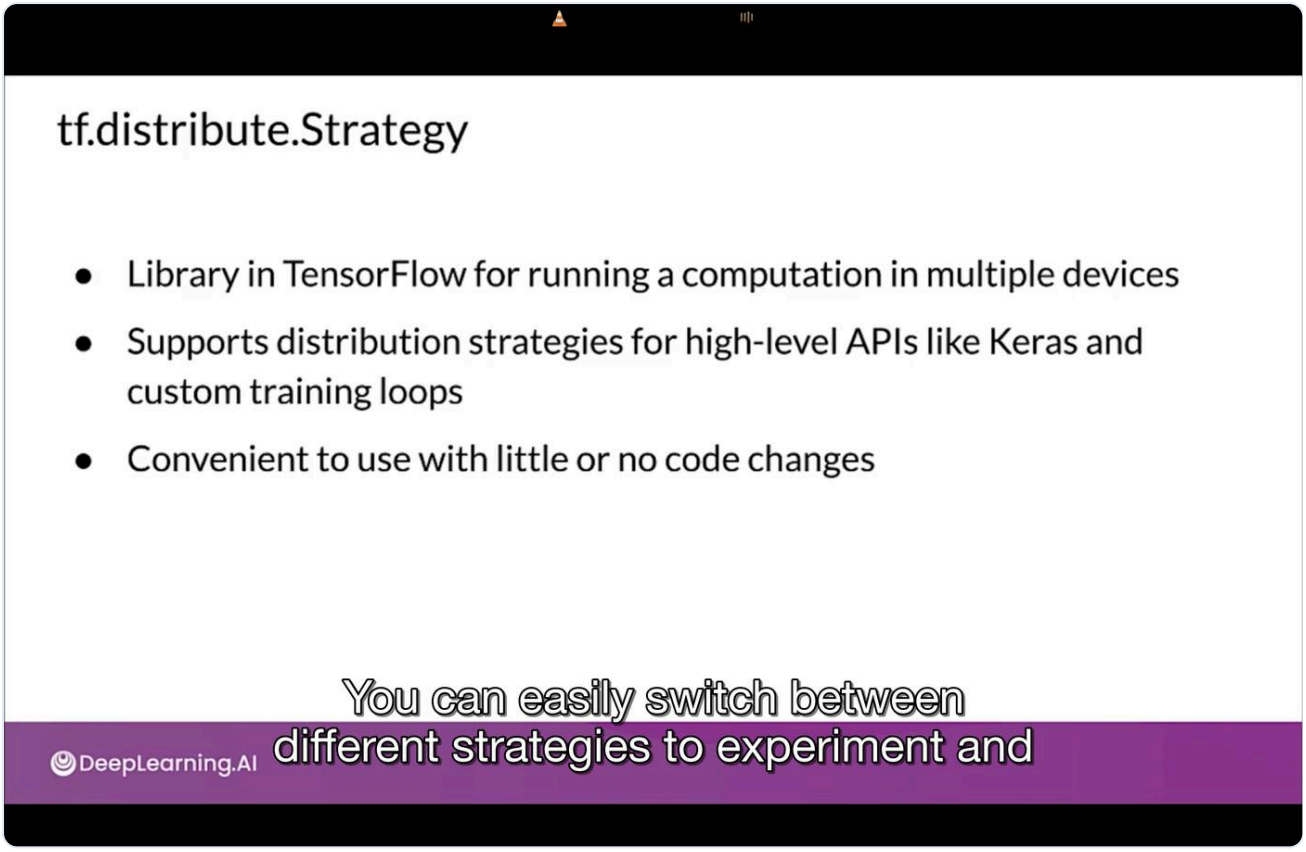

FIGURE 23.6

You can easily switch between different strategies to experiment and find the ones that best fit your needs.

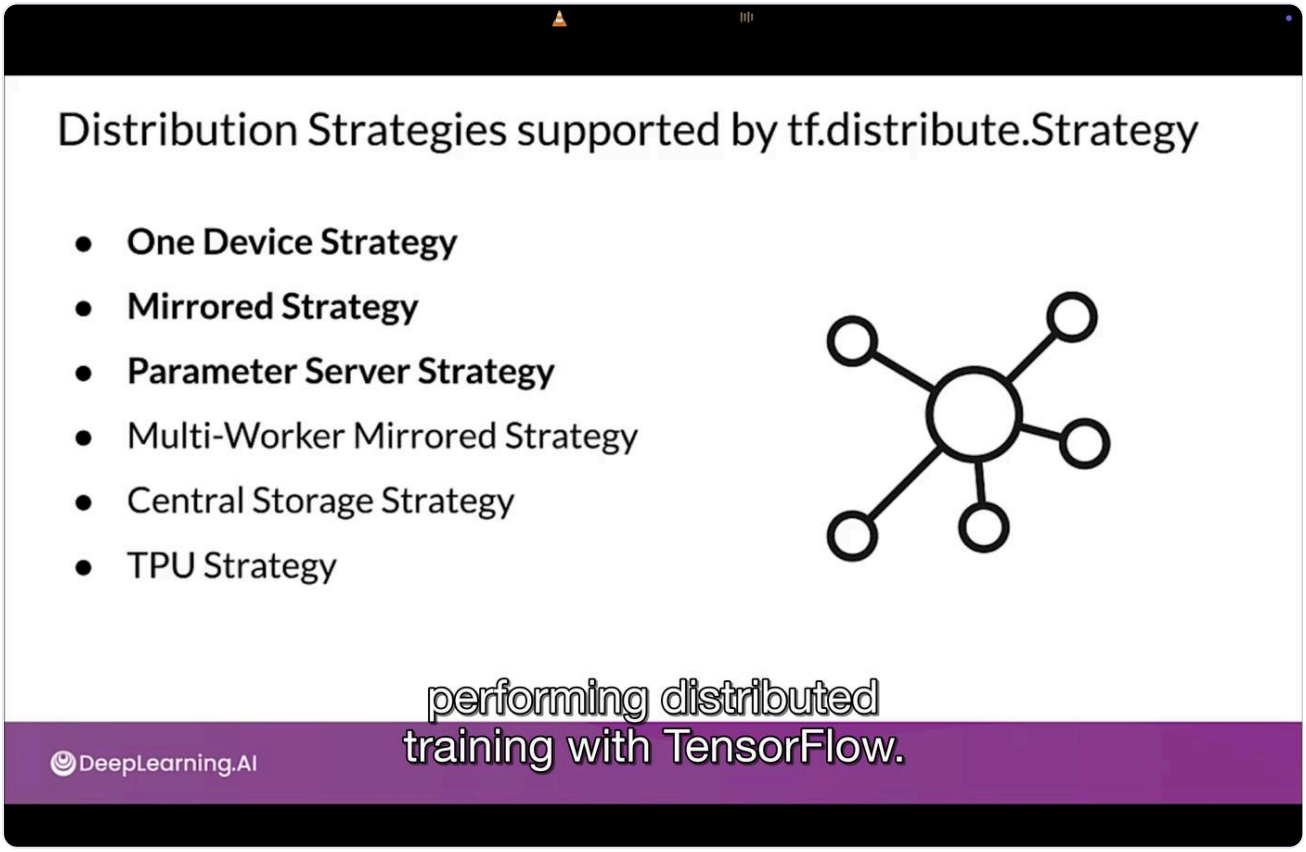

FIGURE 23.7 | Distribution Strategies Supported by tf.distribute.Strategy

There are many different strategies for performing distributed training with TensorFlow. The following ones are the most used, one device strategy, mirrored strategy, parameter service strategy, multi-worker mirrored strategy, central storage strategy, and TPU strategy. Let's focus on the first three options. For more information on this tf.distribute.Strategy, see the links in the resource section. One device strategy will place any variables created in its scope on the specified device. Input distributed through this strategy will be prefetched to the specified device. Moreover, any functions called via strategy.run will also be placed on the specified device as well.

Typical usage of this strategy could be testing your code with tf.distribute.Strategy API before switching to other strategies which actually distribute to multiple devices and machines.

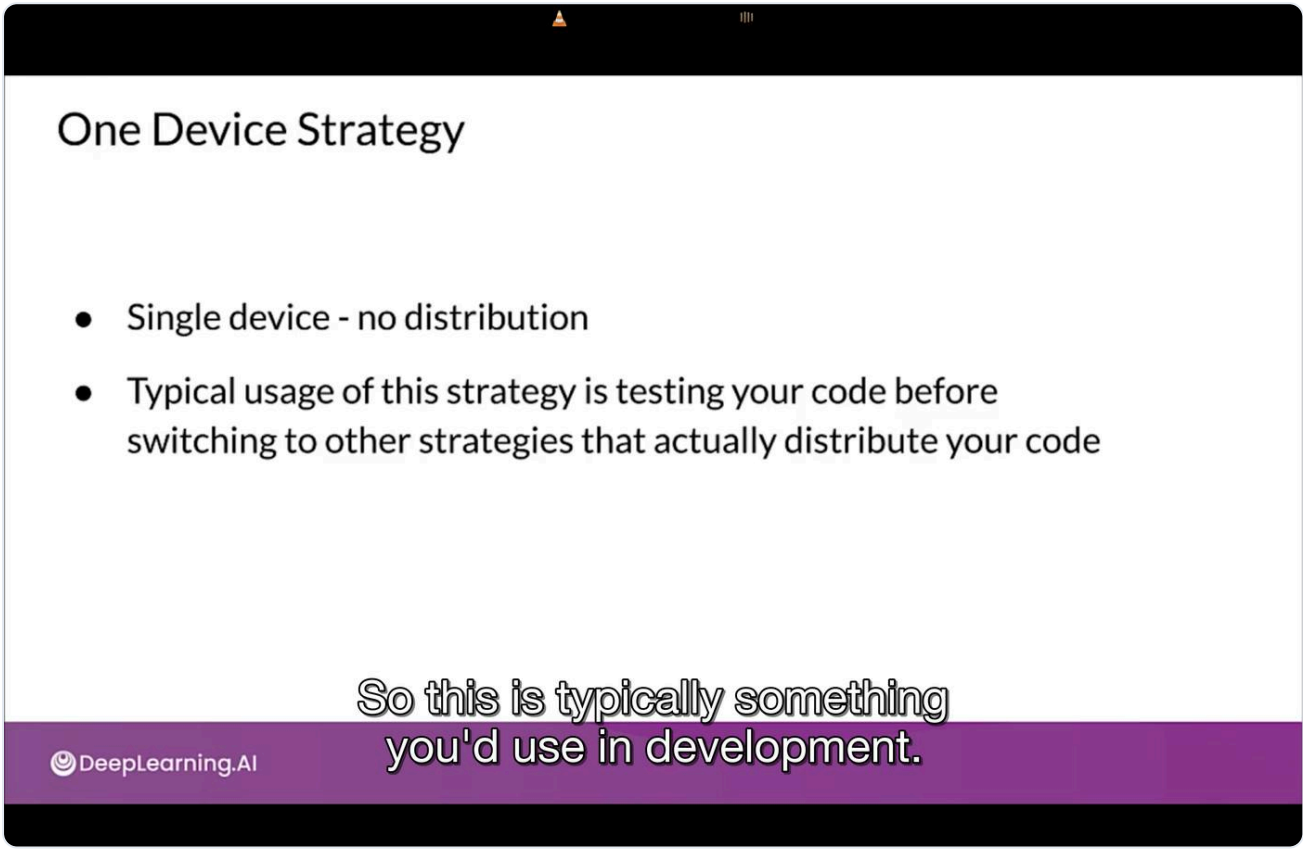

FIGURE 23.8 | One Device Strategy

So this is typically something you'd use in development. Mirrored strategy supports synchronous distributed training on multiple GPUs on one machine. It creates one replica per GPU device. Each variable in the model is mirrored across all the other replicas. Together these variables form a single conceptual variable called a mirrored variable. These variables are kept in sync with each other by applying identical updates.

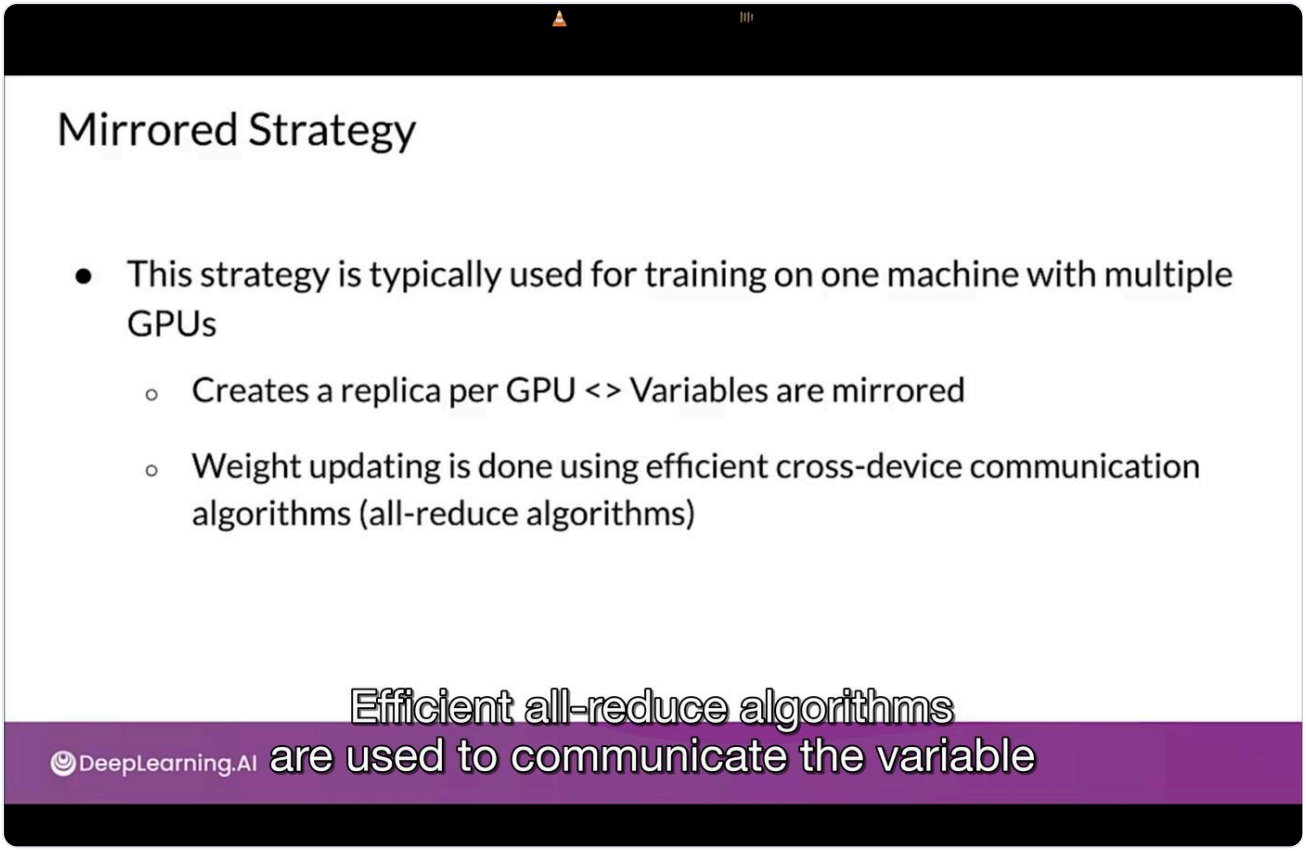

FIGURE 23.9 | Mirrored Strategy

Efficient all-reduce algorithms are used to communicate the variable updates across the devices. All-reduce aggregates tensors across all the devices by adding them up and makes them available on each device. It's a fused algorithm that is very efficient and can reduce the overhead of synchronization significantly. The parameter service strategy is a common asynchronous data parallel method to scale up model training on multiple machines. A parameter server training cluster consists of workers and parameter servers. Variables are created on parameter servers, and they are read and updated by workers in each step.

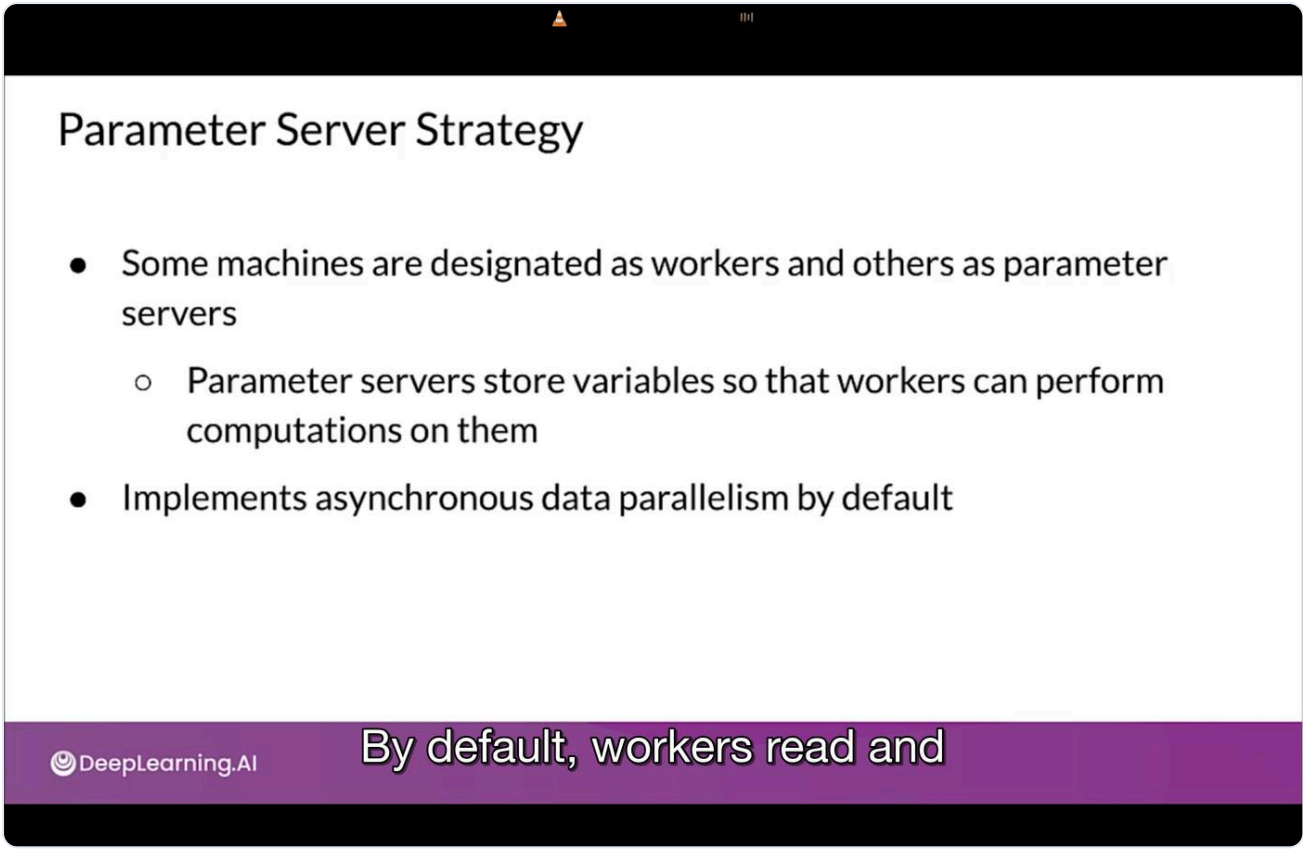

FIGURE 23.10 | Parameter Server Strategy

By default, workers read and update these variables independently without synchronizing with each other. This is why sometimes parameter server style training is also referred to as asynchronous training. Typically in synchronous training, the entire cluster of workers would fail if one or more of the workers were to fail. It's important to consider some form of fault tolerance in cases where workers die or become unstable. This allows you to recover from a failure incurred by preempting workers. This can be done by preserving the training state in the distributed file system. Since all the workers are kept in sync in terms of training epochs and steps, other workers would need to wait for the failed or preempted worker to restart in order to continue.

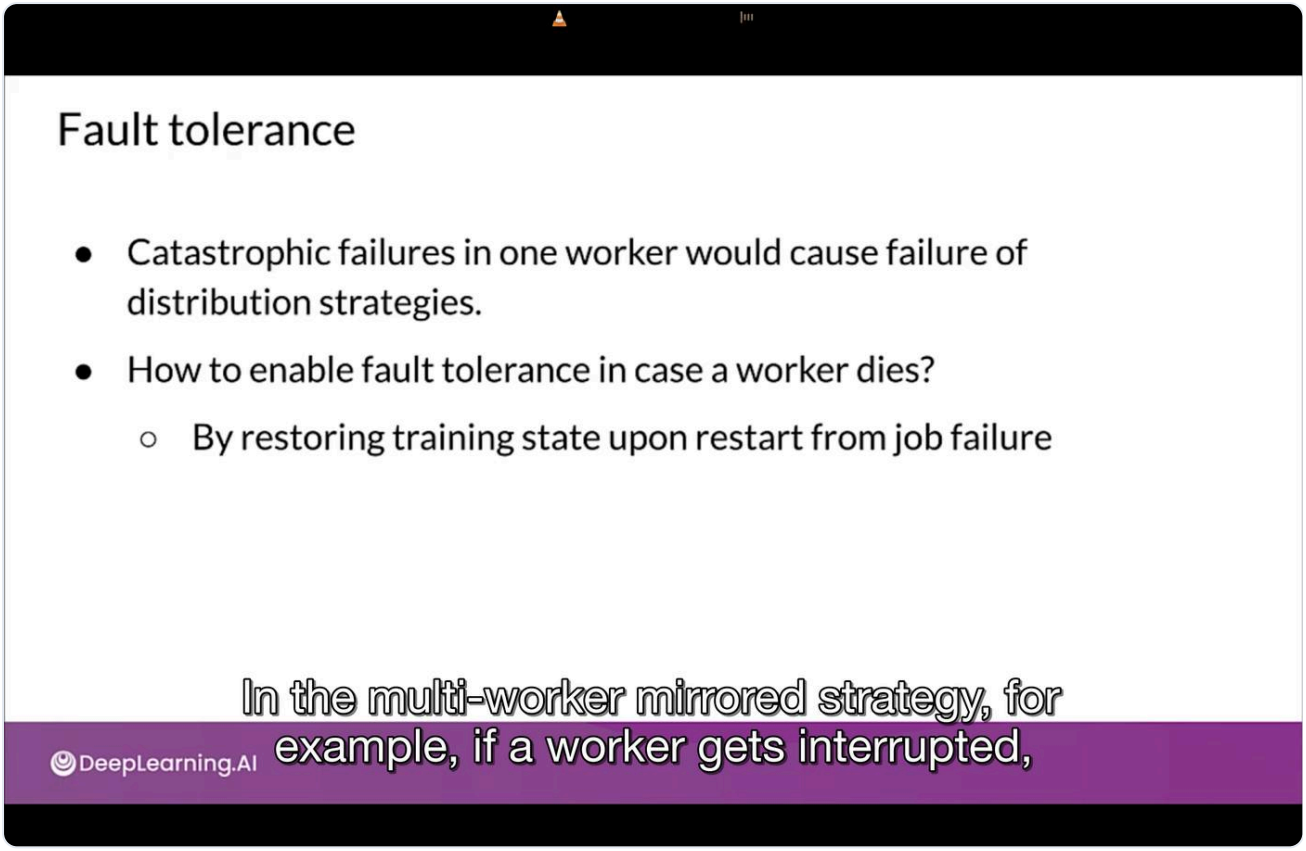

FIGURE 23.11 | Fault Tolerance

In the multi-worker mirrored strategy, for example, if a worker gets interrupted, the whole cluster pauses until the interrupted worker is restarted. Other workers will also restart, and the interrupted worker rejoins the cluster. Then there's needs to be a way so that every worker picks up its former state, thereby allowing the cluster to get back in sync to allow for training to proceed smoothly. For example, Keras provides this functionality in the BackupAndRestore callback.

LECTURE 24

Parallelize Data Transformation

Accelerators are a key part of high-performance modeling, training, and inference, but accelerators are also expensive, so it's important to use them efficiently. That means keeping them busy, which requires you to supply them with enough data fast enough. That's why high-performance ingestion is important in high-performance modeling. Let's discuss that now. Let's discuss why input pipelines are often needed for training models and the issues around applying transformations on data. Generally, transformations deal with pre-processing tasks that tend to add overhead to your training input pipeline. For example, when augmenting huge image classification datasets, many of these transformations are often applied element wise.

As a result, if it takes too long to apply transformations, this can lead to the CPU being underutilized while it waits for data and it's not just limited to the transformations. At times, you could have data that simply cannot fit into memory and it often becomes a challenge to build a scalable input data pipeline that will feed your data fast enough to keep your training processors busy. In the end, the goal should be to efficiently utilize the hardware available and reduce the time required to load the data from the disk, and also reduce the time required for pre-processing it. Input pipelines are important part of many training pipelines, but there are often similar requirements for inference pipelines as well.

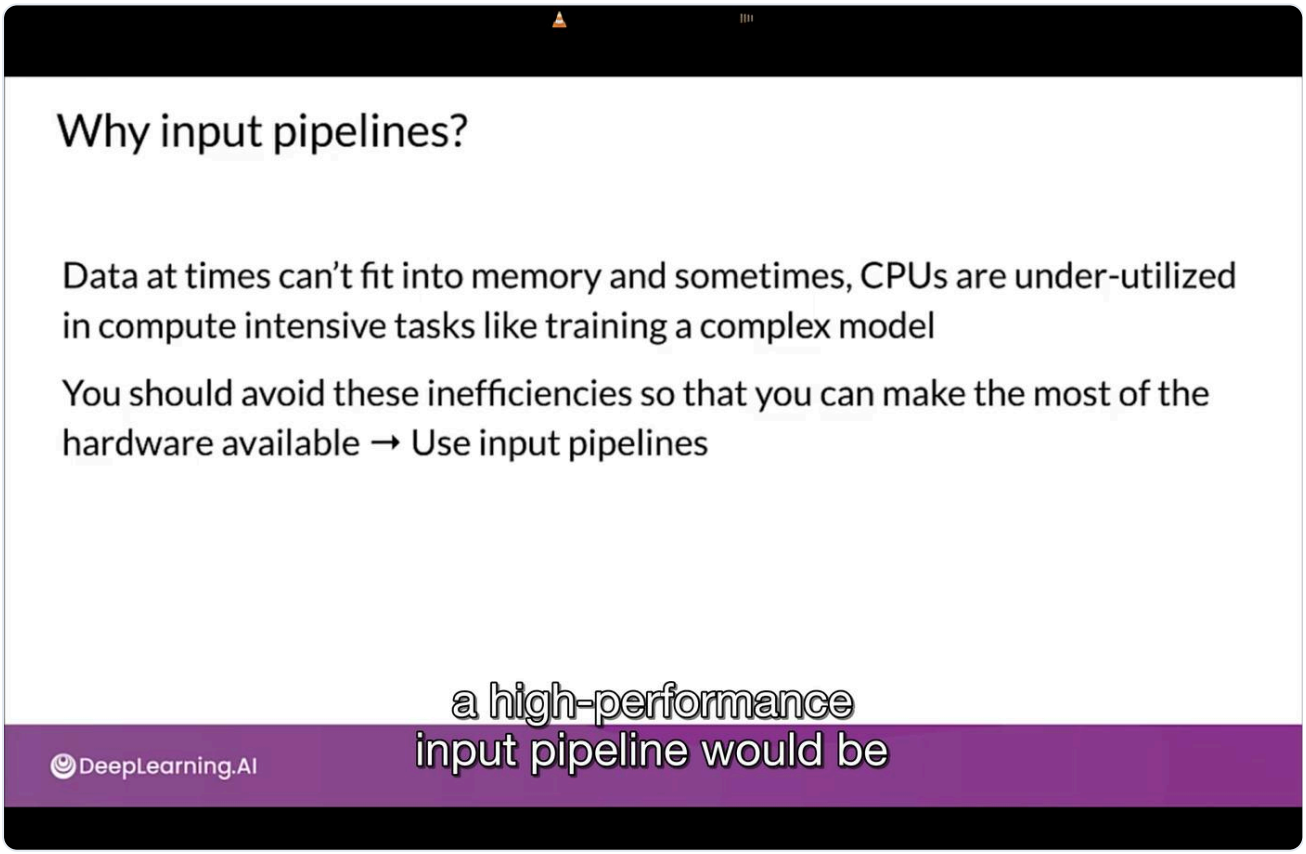

FIGURE 24.1 | Why Input Pipelines?

In the larger context of a training pipeline such as a TFX training pipeline, a high-performance input pipeline would be part of the trainer component and possibly other components like Transform that may need to do quite a bit of work on the data. There are many ways to design an efficient input pipeline. One framework that can help is TensorFlow data or tf.data. Let's consider TF data as an example of how to design an efficient input pipeline. You can view input pipelines as an ETL process, providing a framework to facilitate applying performance optimization. The 1st step involves extracting data from data stores that may be either local or remote, like hard drives, SSDs, cloud storage, and HDFS.

In the 2nd step, data often needs to be pre-processed. This includes shuffling, batching, and repeating data. The way you order these transformations may have an impact on your pipelines performance. This is something that you need to be aware of when using any data transformation like map or batch or shuffle or repeat, and so forth.

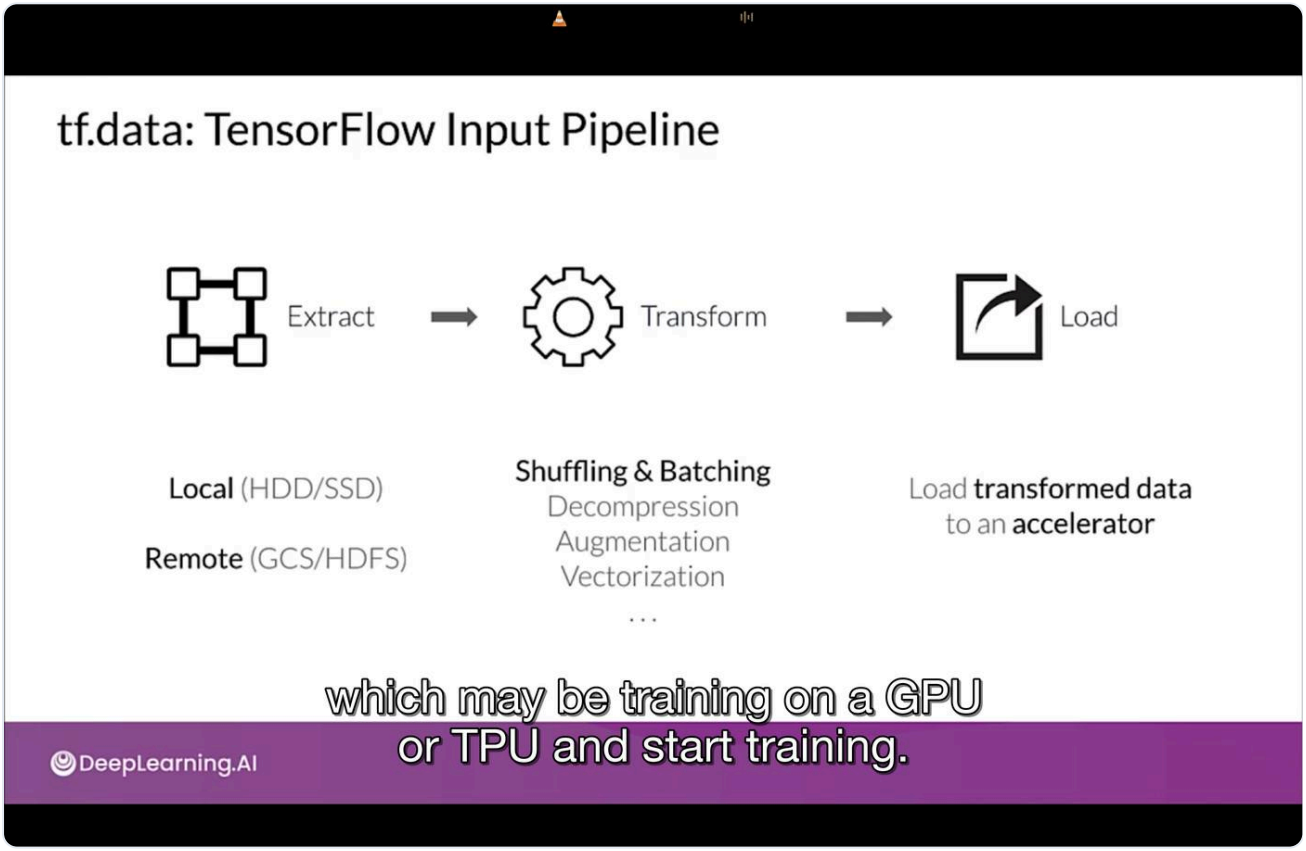

FIGURE 24.2 | Shuffling & Batching

You then load the pre-processed data into the model, which may be training on a GPU or TPU and start training. A key requirement for high-performance input Pipelines is to parallel processing of data across the various systems to try to make maximum efficient use of the available compute, IO, and network resources. Especially for more expensive components such as accelerators, you want to keep them busy as much as possible.

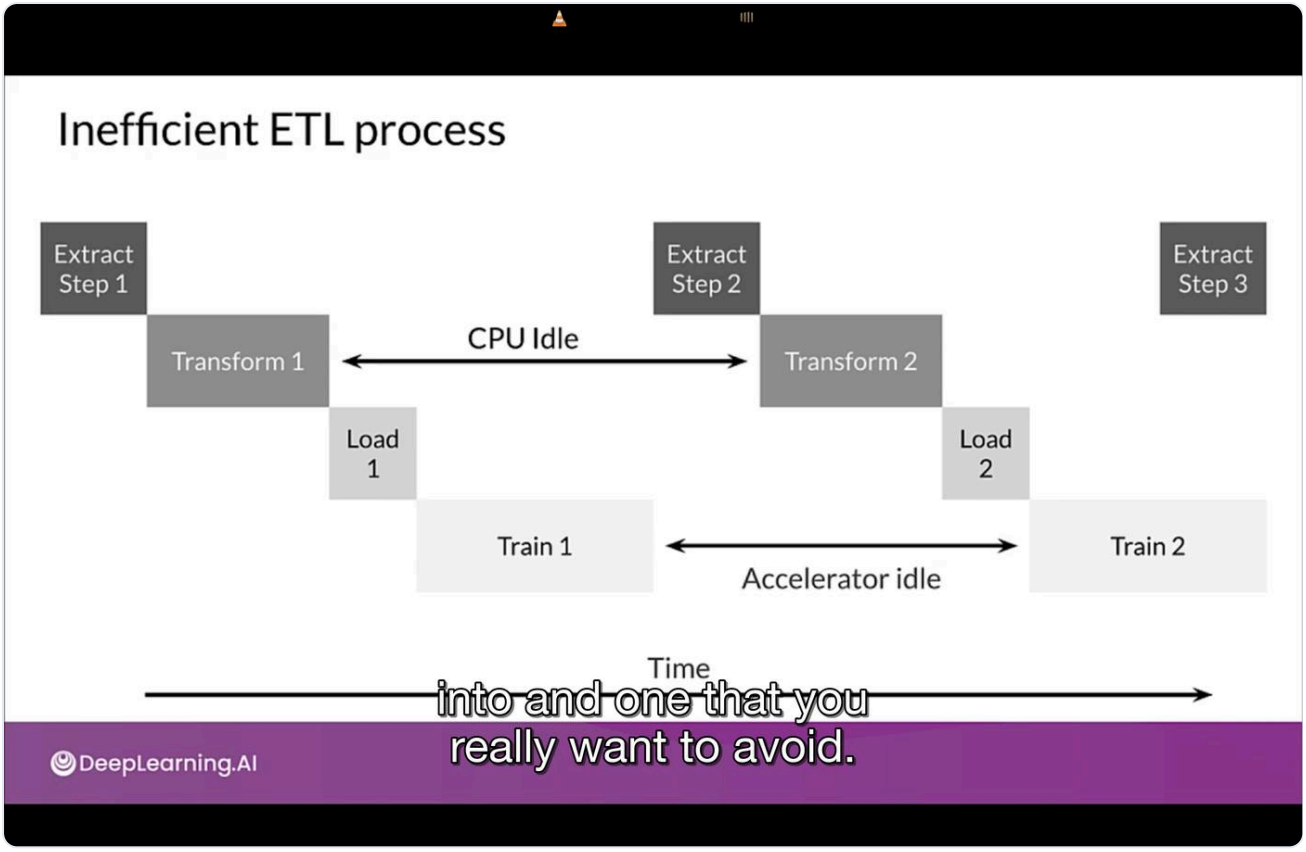

FIGURE 24.3

Let's look at a typical pattern that is easy to fall into and one that you really want to avoid. In this scenario, key hardware components, including CPUs and accelerators sit idle, waiting for the previous steps to complete. If you think about it, ETL, extract, transform, and load is a good mental model for data performance. Now, to give you some intuition on how pipelining can be carried out, each of the different phases of ETL use different hardware components in your system. Your extract phase is exercising your disk, your network for your loading from a remote system. Transform typically happens on the CPU and can be very CPU hungry. The load phase is exercising the DMA, or direct memory access subsystem and the connections to your accelerator, probably a GPU or TPU.

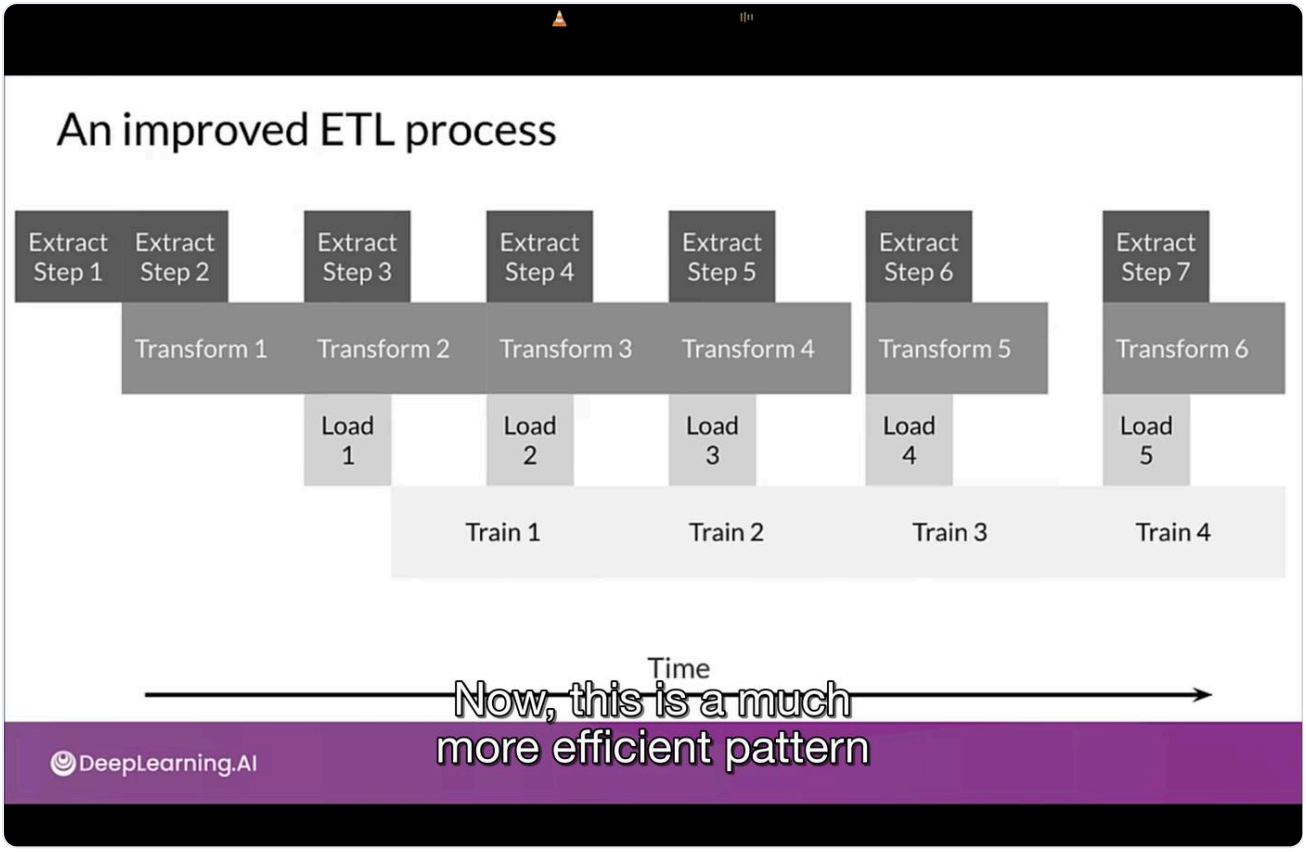

FIGURE 24.4 | An Improved ETL Process

Now, this is a much more efficient pattern than the previous one, although it's still not quite optimal. In practice, this pattern may be difficult to optimize further in many cases. As this diagram shows by parallelizing operations, you can overlap the different parts of ETL using a technique known as software pipelining. With software pipelining, you're extracting data for step 5, while you're transforming for step 4, and while you're loading data for step 3, and finally, your training for step 2 all at the same time. This results in a very efficient use of your compute resources. As a result, your training is much faster and your resource utilization is much higher.

Notice that now there are only a few instances where your hard drive and CPU are actually sitting idle.

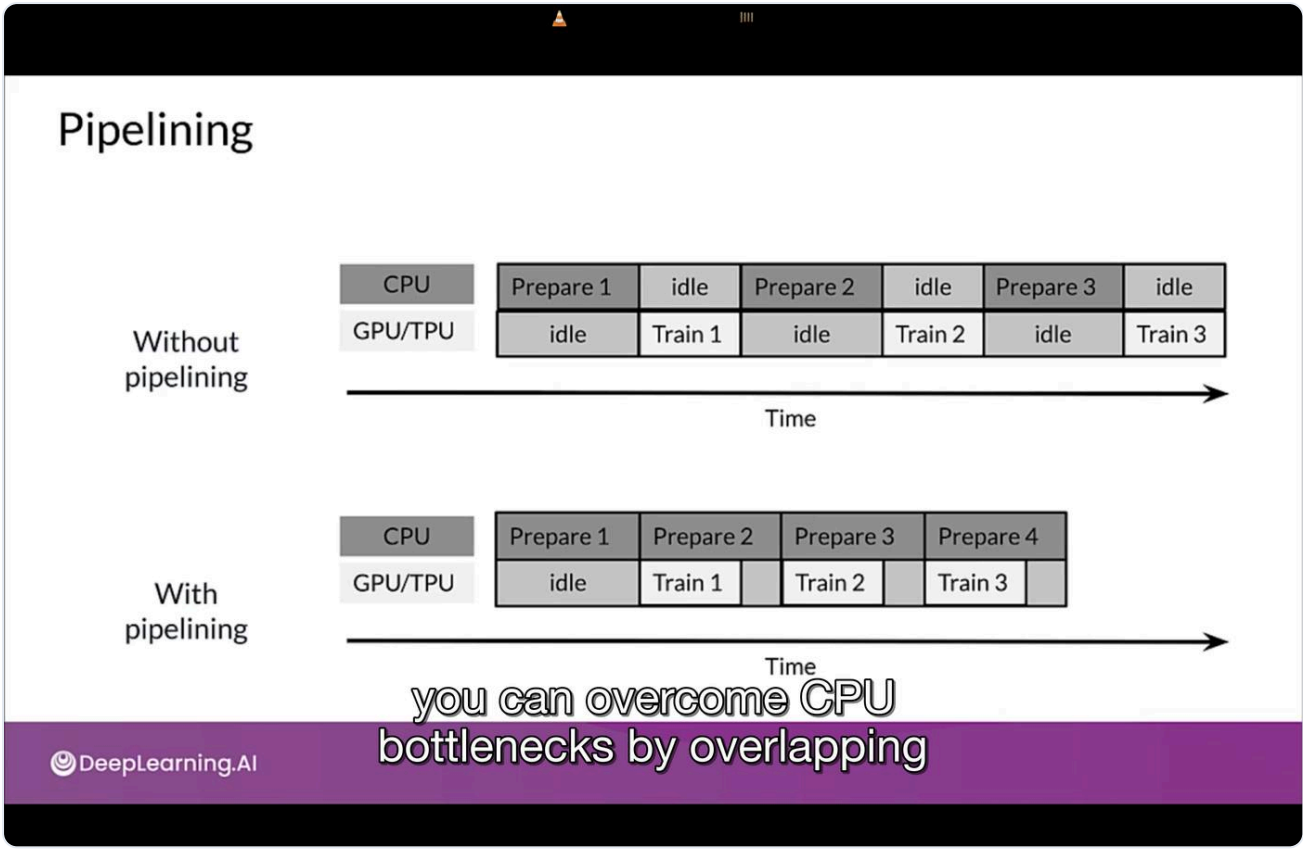

FIGURE 24.5

Through pipelining your training procedure, you can overcome CPU bottlenecks by overlapping the CPU pre-processing and model execution of accelerators. While the accelerator is busy with training the model in the background, the CPU starts preparing data for the next training step. You can see there's a significant improvement in model training time when using pipelining. While you can still expect some idle time, it's greatly reduced by using pipelining. How do you optimize your data pipeline in practice? There are a few basic approaches that could potentially be used to accelerate your pipeline. Prefetching, for example, where you begin loading data for the next step before the current step completes.

Other techniques involve parallelizing data extraction and transformation. Caching the dataset to get started with training immediately once a new epic begins, is also very effective when you have enough cash.

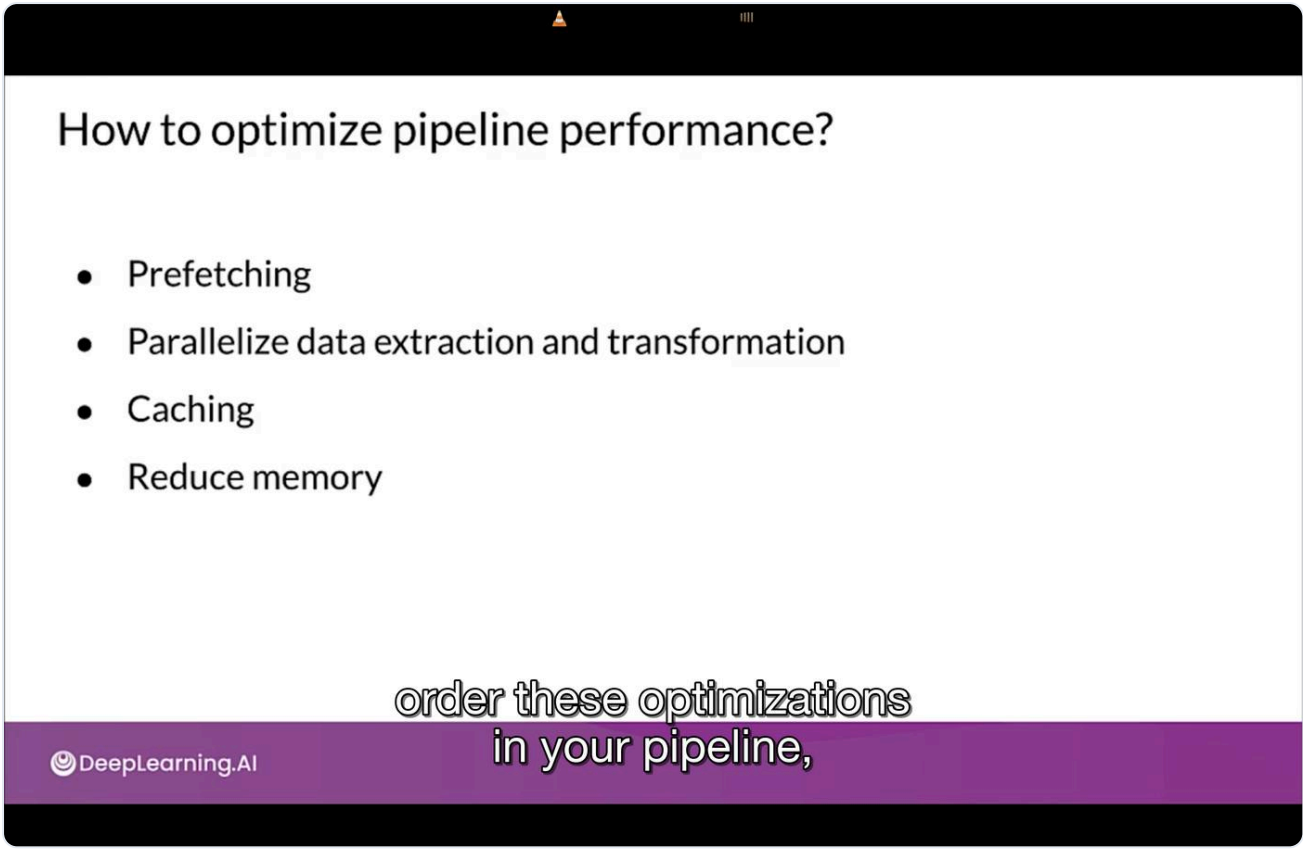

FIGURE 24.6 | How to Optimize Pipeline Performance?

Finally, you need to be aware of how you order these optimizations in your pipeline, to maximize the pipelines efficiency. With prefetching, you overlap the work of a producer with a work of a consumer, while the model is executing step as, the input pipeline is reading the data for step S plus 1. This reduces the total time it takes for a step to either train the model or extract data from disk, whichever is highest.

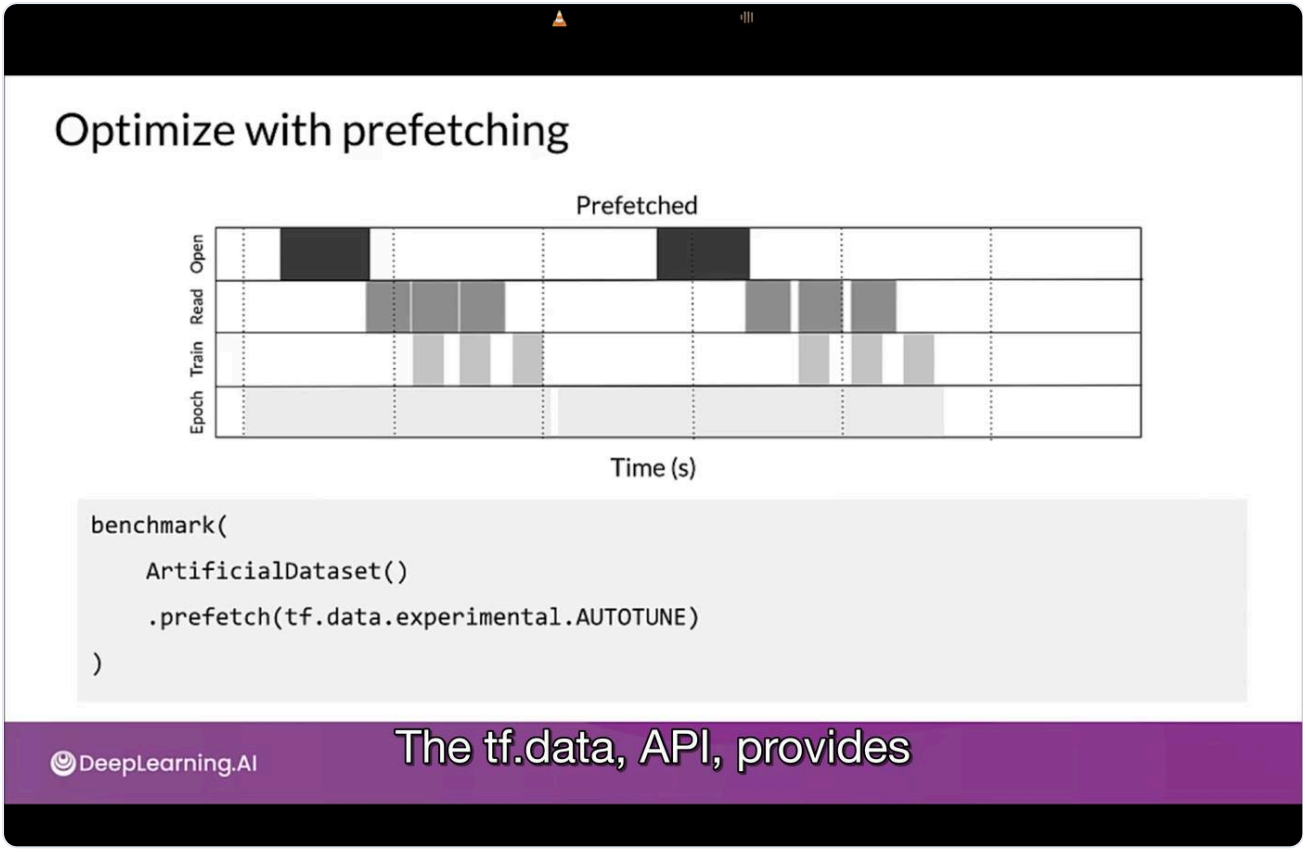

FIGURE 24.7 | Optimize with Prefetching

The tf.data, API, provides the tf dataset prefetch transformation. You can use this to decouple the time when data is produced, from the time when the data is consumed. This transformation uses a background thread and an internal buffer to prefetch elements from the input dataset ahead of time before they are requested. Ideally, the number of elements to prefetch should be equal to, or possibly greater than, the number of batches consumed by a single training step. You can either manually tune this value, or set it to tf data experimental autotune, as in this example, which will configure the tf data runtime, to optimize the value dynamically at runtime.

In a real-world setting, the input data may be stored remotely, for example, on GCS or HDFS. A dataset pipeline that works well when reading data locally, might become bottlenecked on Io when reading data remotely, because of the following differences between local and remote storage. Time=to-first-byte. Reading the first byte of a file from remote storage, can take orders of magnitude longer, than from local storage. Read throughput.

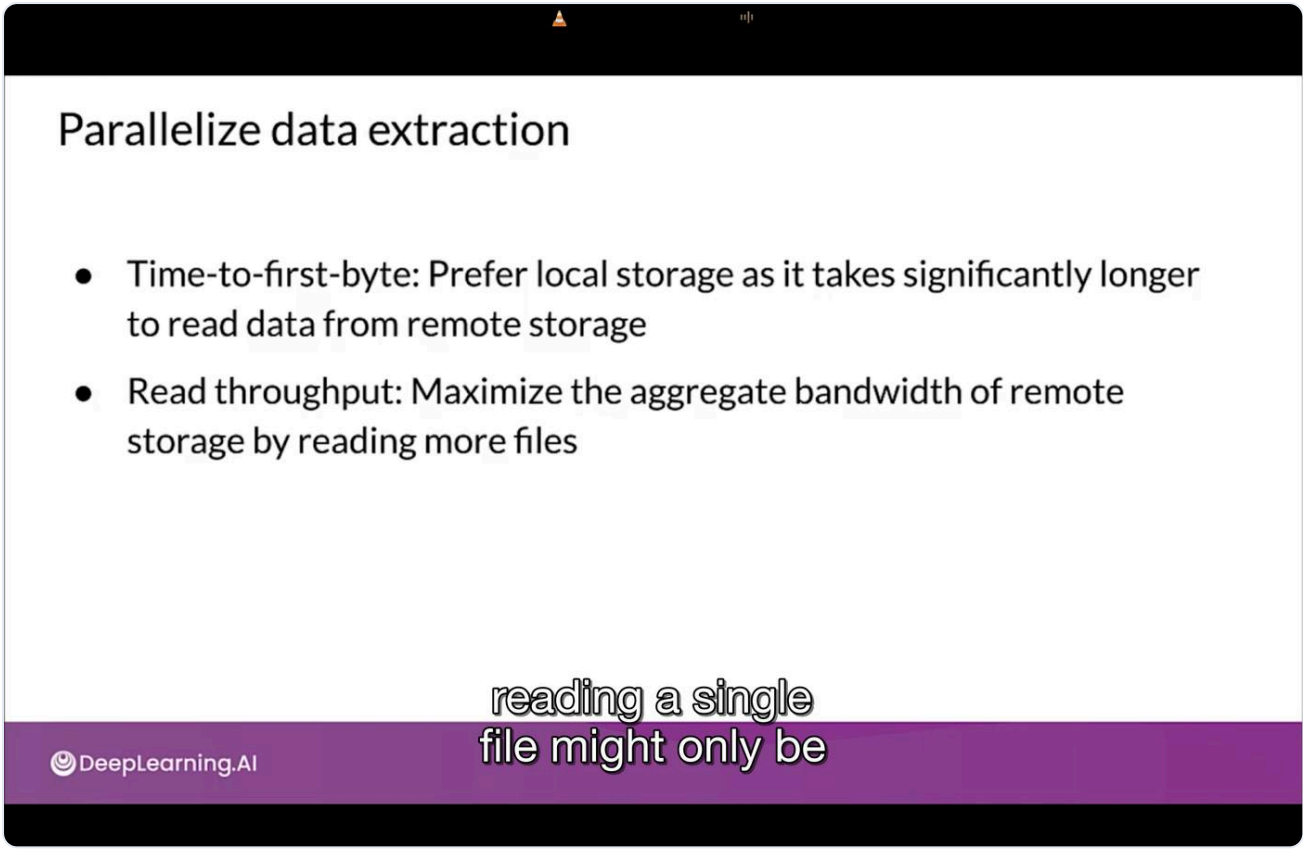

FIGURE 24.8 | Parallelize Data Extraction

Again, while remote storage typically offers large aggregate bandwidth, reading a single file might only be able to utilize a small fraction of this bandwidth.

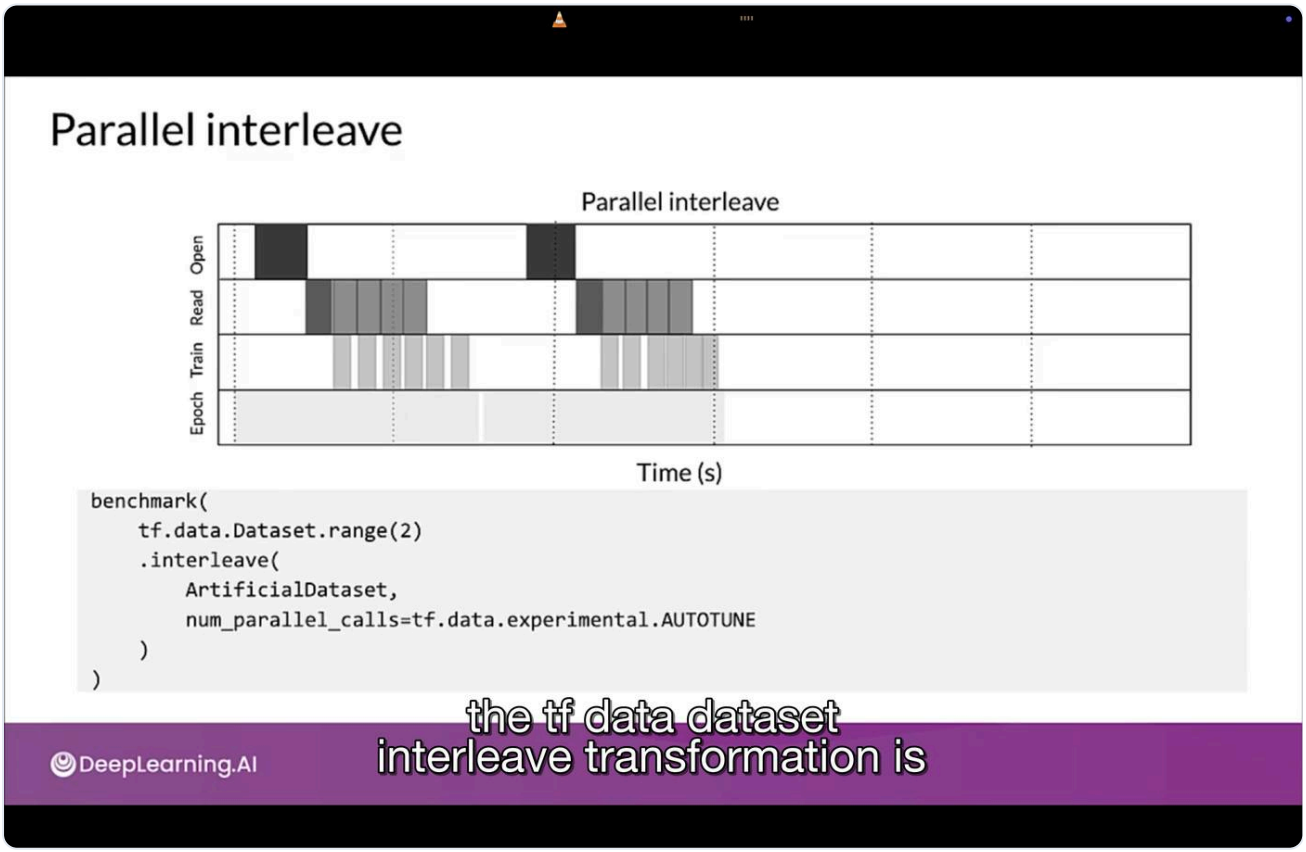

FIGURE 24.9

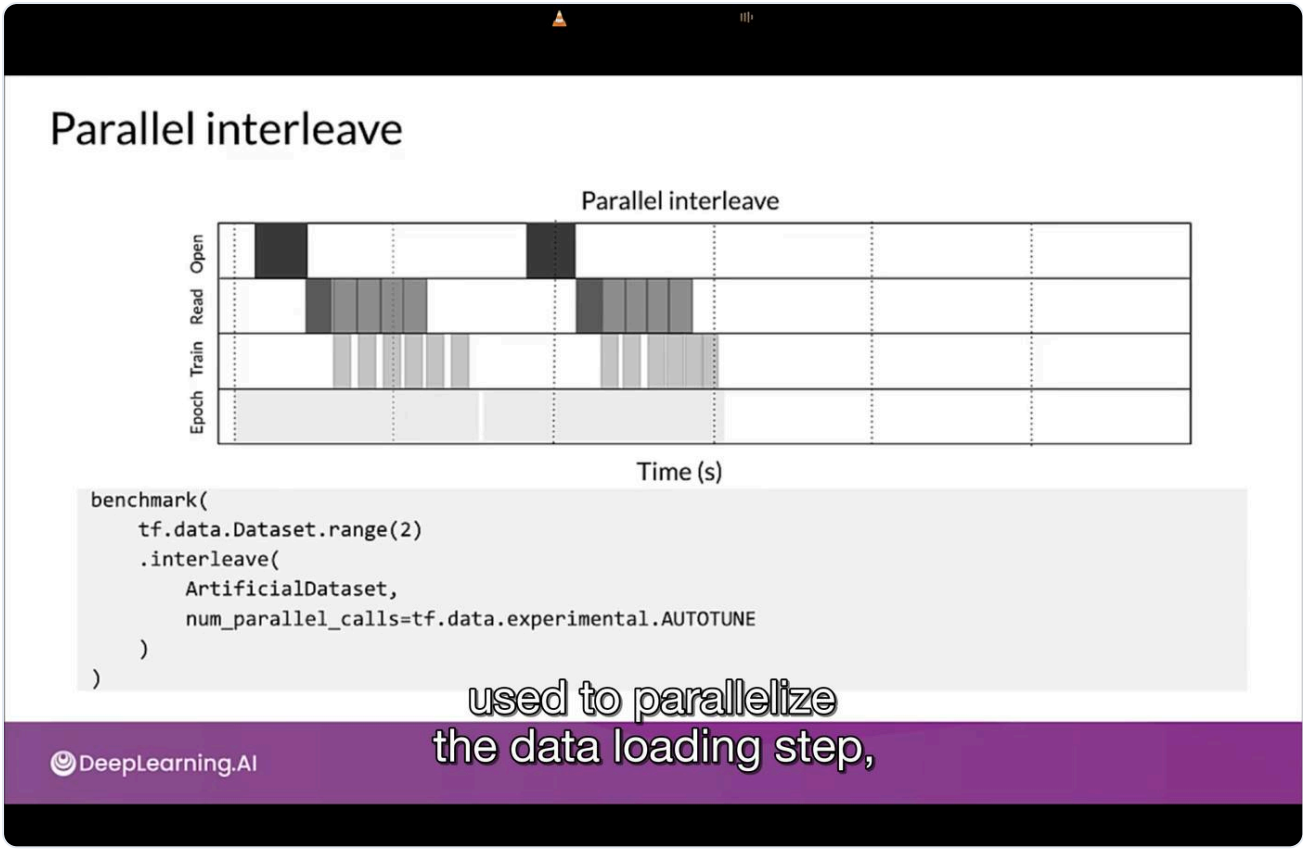

FIGURE 24.10 | Parallel Interleave

To reduce data extraction overhead, the tf data dataset interleave transformation is used to parallelize the data loading step, including interleaving contents of other datasets. The number of datasets to overlap as specified by the cycle length argument, while the level of parallelism is set with the num parallel calls argument. Similar to the prefetch transformation, the interleaved transformation supports tf data experimental auto-tune, which will delegate the decision about what level of parallelism to use for the tf data runtime. When preparing data, input elements may need to be preprocessed. For example, the tf data, API, offers the tf data dataset map transformation, which implies a user-defined function to each element of the input dataset.

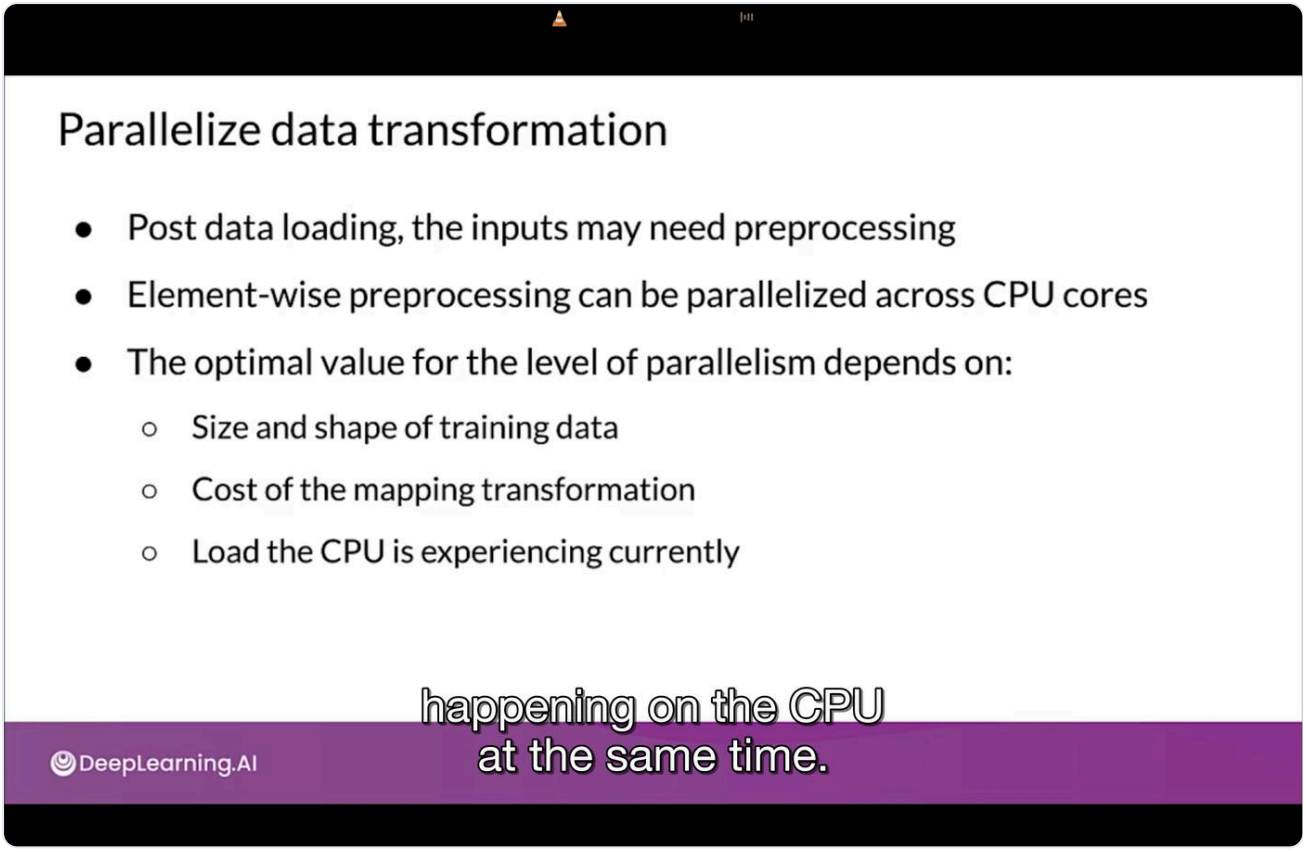

FIGURE 24.11 | Parallelize Data Transformation

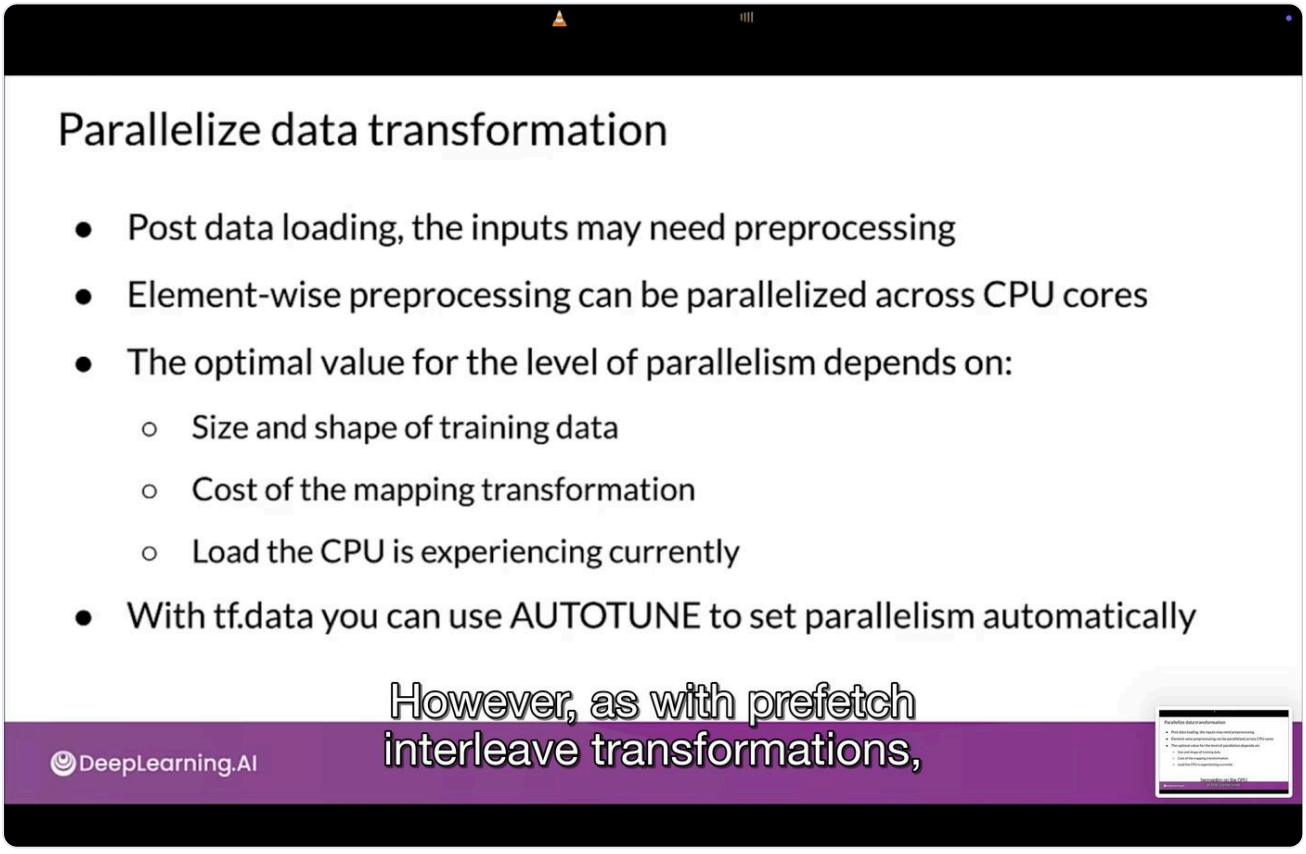

FIGURE 24.12 | Parallelize Data Transformation

Element-wise preprocessing can be parallelized across multiple CPU cores. Similar to the prefetch and interleaved transformation, the map transformation provides the num parallel calls argument to specify the level of parallelism. Choosing the best value for the num parallel calls argument depends on your hardware, the characteristics of your training data such as size and shape, the cost of your map function, and what other processing is happening on the CPU at the same time. A simple heuristic is to use the number of available CPU cores. However, as with prefetch interleave transformations, the map transformation in tf data supports the tf data experimental autotune parameter, which will delegate the decision about which level of parallelism to use for the tf data runtime.

Let's see what happens when you apply pre-processing in parallel on multiple samples.

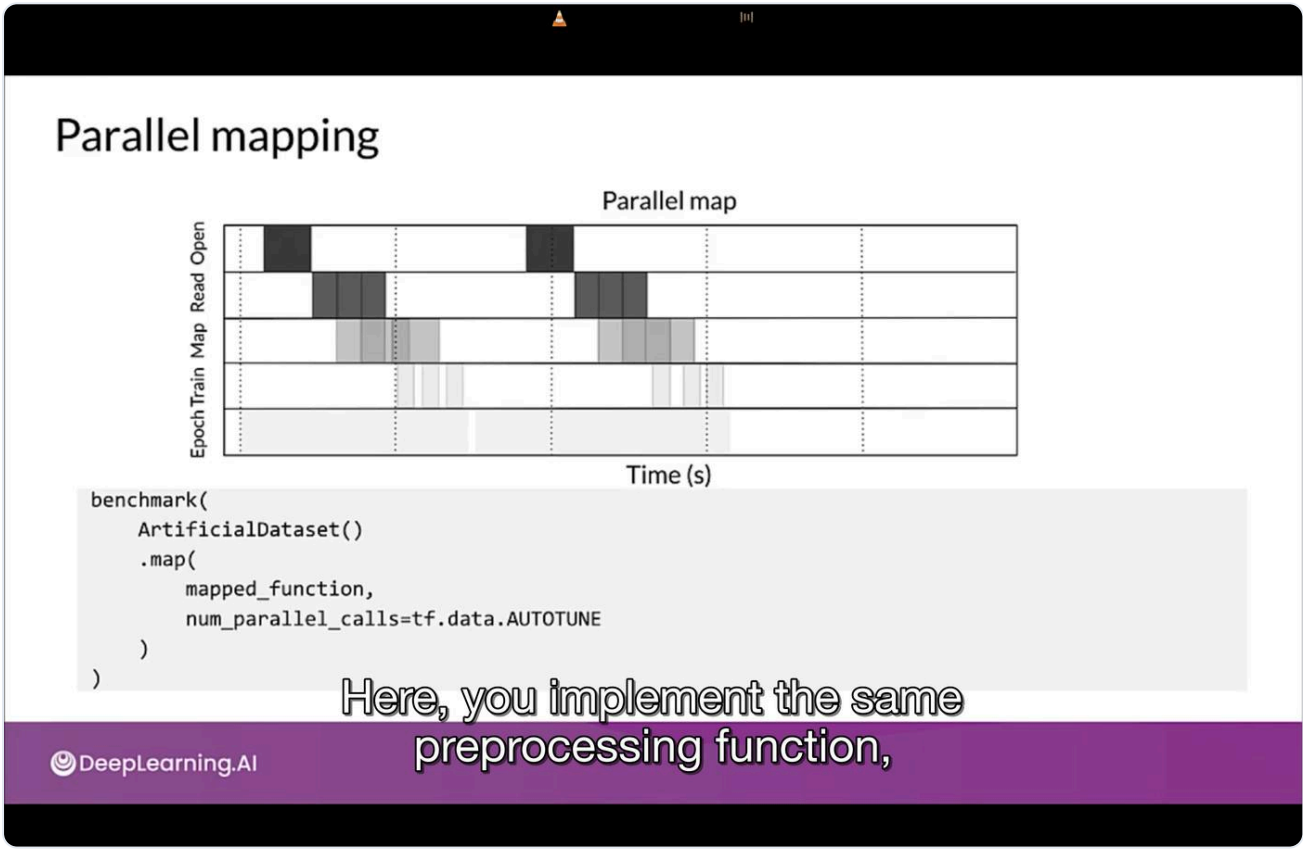

FIGURE 24.13 | Parallel Map

Here, the preprocessing steps overlap, reducing the overall time for a single iteration, this can potentially be a big win. Here, you implement the same preprocessing function, but apply it in parallel, on multiple samples. Tf data dataset includes the ability to cash a dataset, either in memory or on local storage. In many instances, caching is advantageous and leads to increased performance. This will save some operations, like file opening and data reading, from being executed during each epic.

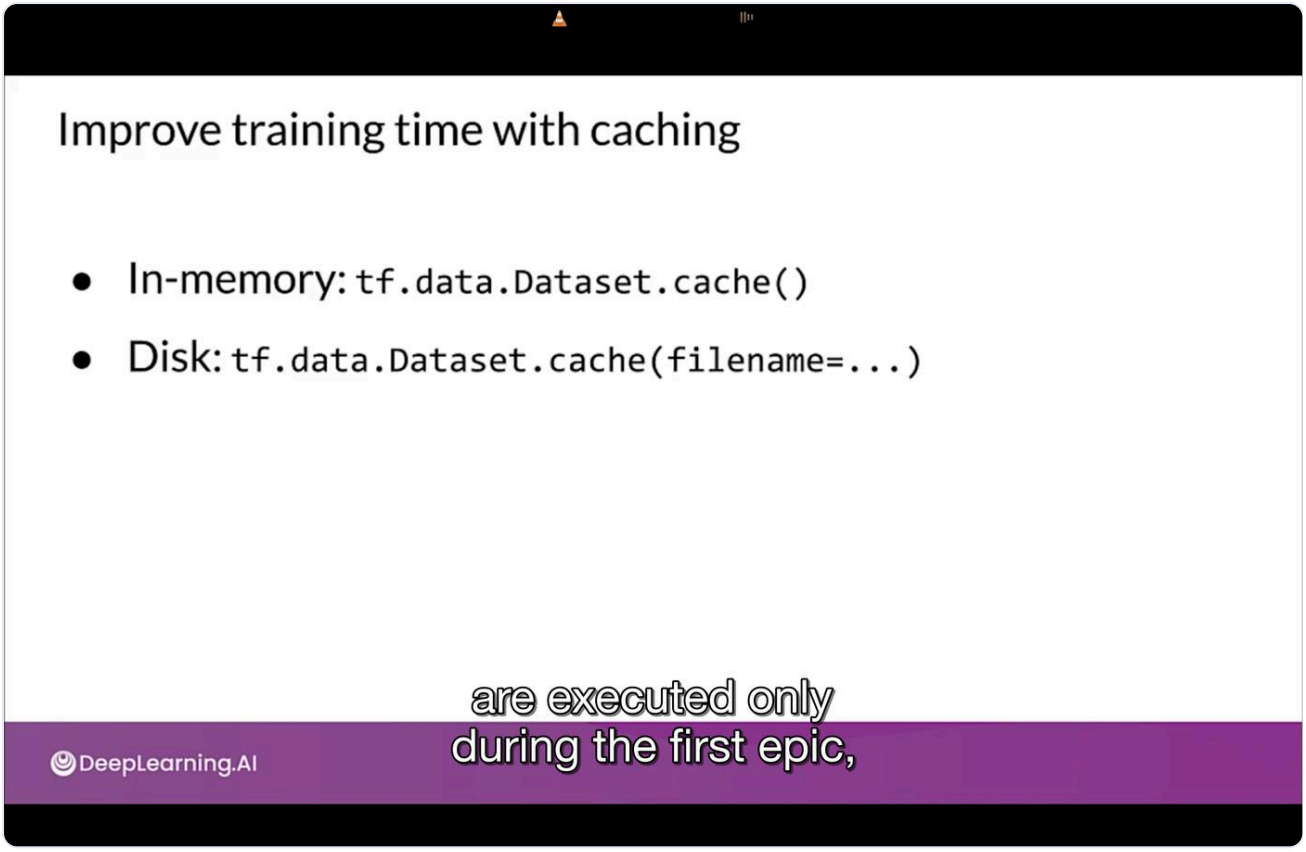

FIGURE 24.14 | Improve Training Time with Caching

When you cache a dataset, the transformations before caching, like file opening and data reading, are executed only during the first epic, the next epics, we'll re-use the cache data.

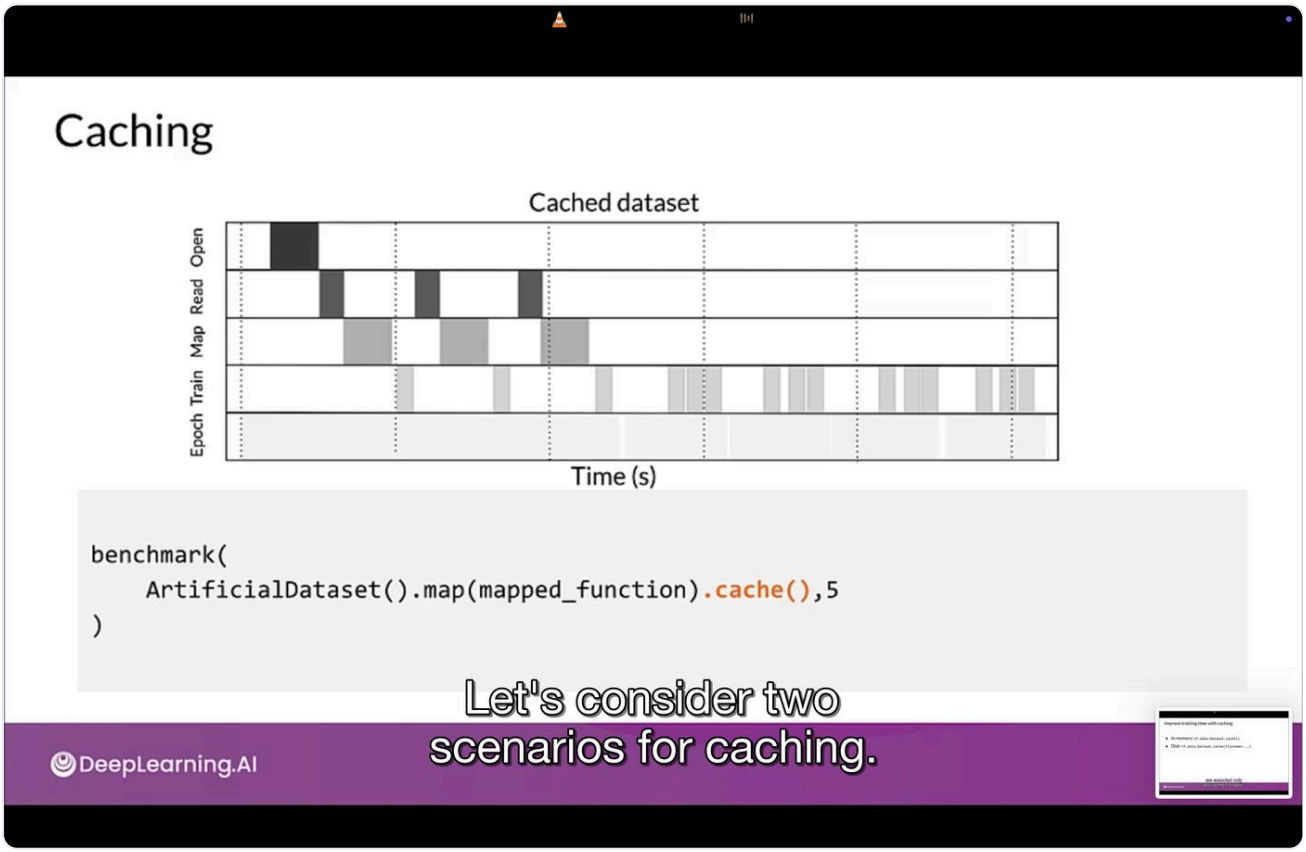

FIGURE 24.15

Let's consider two scenarios for caching. If the user-defined function passed into the map transformation is expensive, then it makes sense to apply the cache transformation after the map transformation, as long as the resulting dataset can fit into memory or local storage. On the contrary, if the user-defined function increases the space required to store the dataset beyond the cache capacity, either apply it after cash transformation, or consider pre-processing your data before your training job, to reduce resource requirement.

LECTURE 25

Pipeline Parallelism

Now let's talk about high-performance modeling and some of the issues with increasingly large models, along with some advanced techniques for training large models. In recent years, the size of machine learning datasets and models has been continuously increasing, allowing for improved results on a wide range of tasks, including speech recognition, visual recognition, and language processing. Recent advances by BigGAN, BERT, and GPT-3 have shown that ever-larger models lead to better task performance. At the same time, hardware accelerators like GPUs and TPUs have also been increasing in power, but at a significantly slower pace. The gap between model growth and hardware improvement has increased the importance of parallelism.

Parallelism in this context means training a single machine learning model on multiple hardware devices. Some model architectures, especially small models, are conducive to parallelism and can be divided quite easily between hardware devices. In enormous models, synchronization costs lead to degraded performance, preventing them from being used. There's also a strong correlation between model size and classification accuracy. For example, the winner of the 2014 ImageNet Visual Recognition Challenge was GoogleNet, which achieved 74.8 top-1 accuracy with four million parameters. While just three years later, the winner of the 2017 ImageNet Challenge went to Squeeze-and-Excitation Networks, which gained an 82.7 top-1 accuracy with 145.8 million parameters.

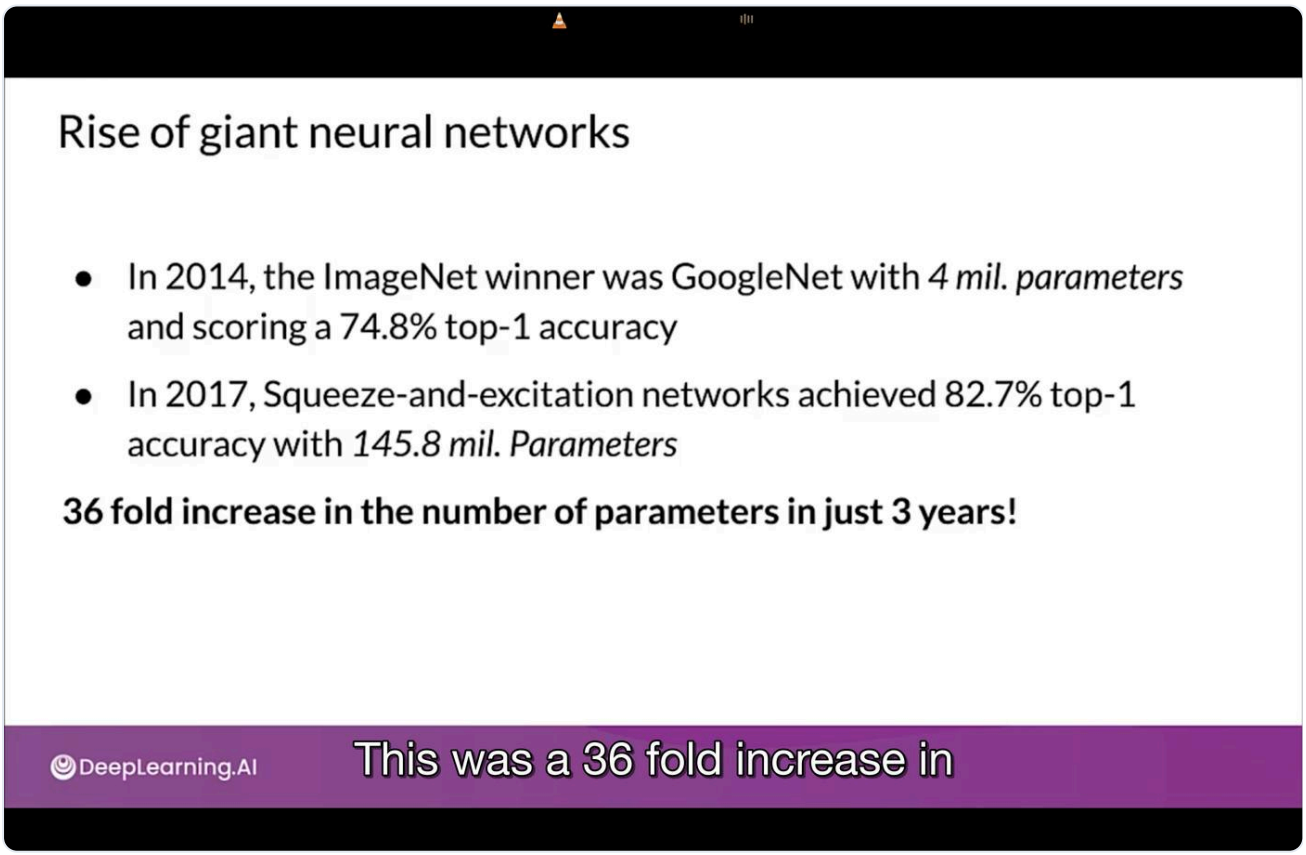

FIGURE 25.1

This was a 36 fold increase in the number of parameters in just three years. Massive numbers of weights and activation parameters require massive memory storage. The natural question is where to store them as the models continue to require more and more parameters? In the last few years, memory capacity made very modest increases compared to model parameters growth. More concretely, GPU memory has only increased by a factor of approximately three, and the current state-of-the-art image models have already reached the available memory found on Cloud TPU V2s. There is a strong and pressing need for an efficient, scalable infrastructure that enables large-scale deep learning and overcomes the memory limitations on current accelerators.

But in some ways, this is not a new problem. Let's look at how some older methods approach this problem of overcoming memory constraints and see if we could apply them today.

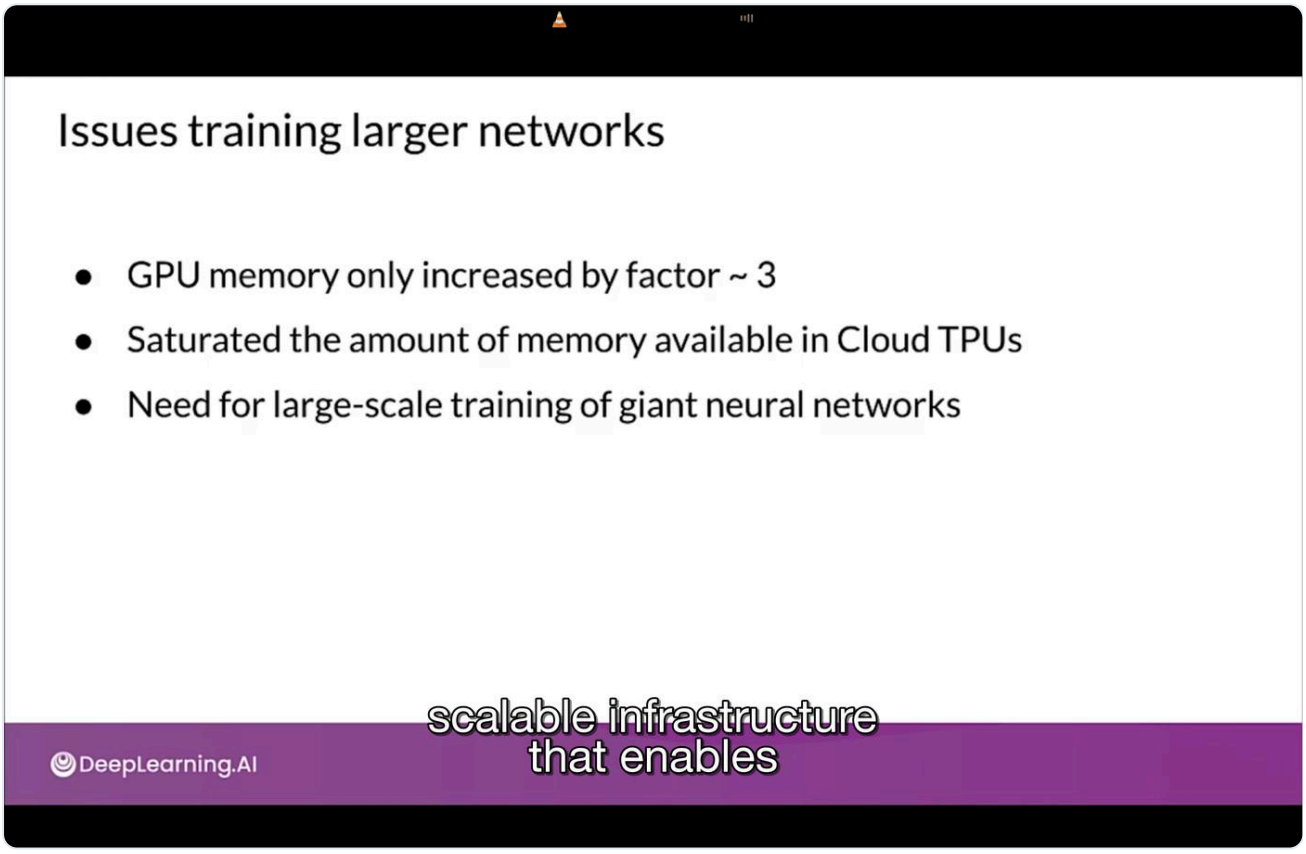

FIGURE 25.2 | Issues Training Larger Networks

Then let's look at some newer methods and some of the advances that they've been able to make. There is a need to strategize and implement solutions to overcome these important memory constraints. One strategy that can overcome problems with insufficient GPU memory is gradient accumulation. Gradient accumulation is a mechanism to split full batches into several mini-batches. During backprop, the model isn't updated with each mini-batch, and instead, the gradients are accumulated. When a full batch completes, the accumulated gradients of all the previous mini-batches are used for backprop to update the model. This process is as effective as using a full batch for training the network since model parameters are updated the same number of times.

The second approach is swapping. Here, since there isn't enough storage on the accelerator, you copy activations back to the CPU or memory and then back to the accelerator.

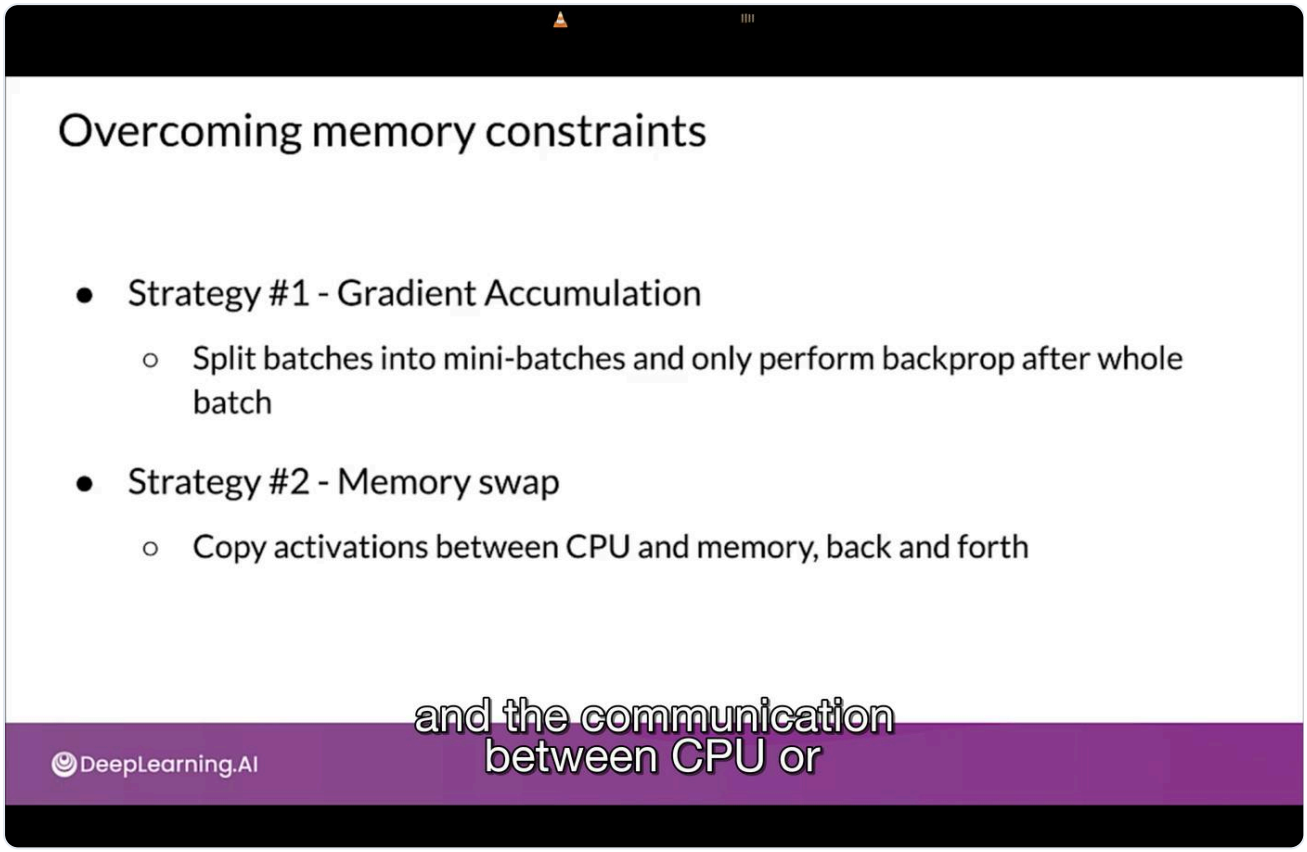

FIGURE 25.3 | Overcoming Memory Constraints

The problem here is that it's slow, and the communication between CPU or memory and the accelerator becomes the bottleneck. Returning to our discussion of parallelism, the basic idea is to split the computation between multiple workers. You've already seen two ways to parallelize, data parallelism and model parallelism. Data parallelism splits the input across workers. Model parallelism splits the model across workers. In data parallelism, different workers or GPUs work on the same model but deal with different data. The model is replicated across a number of workers and each worker performs the forward and backward pass. When it finishes the process, it synchronizes the updated model weights with the other devices and calculates the updated weights of the entire minibatch.

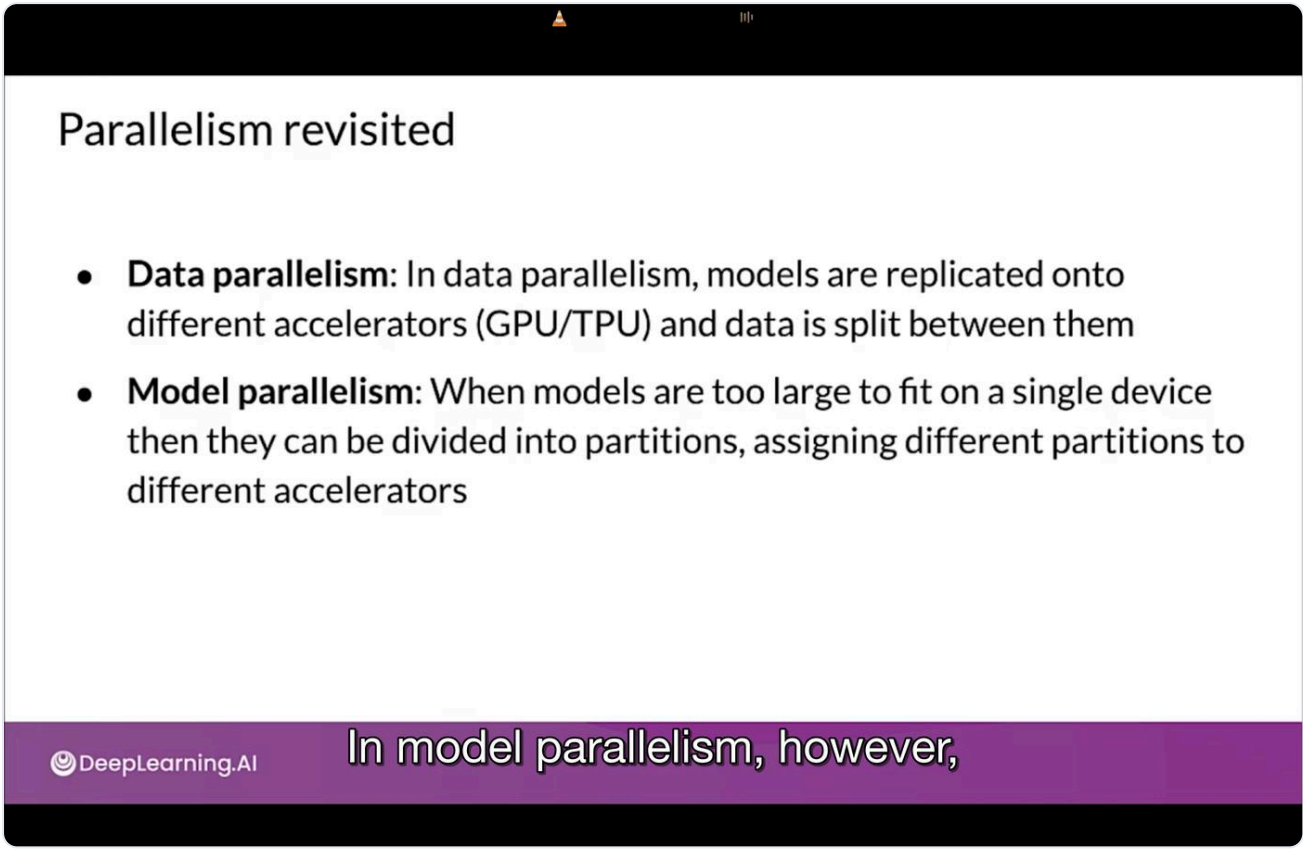

FIGURE 25.4

In model parallelism, however, workers only need to synchronize the shared parameters, usually once for each forward or backprop step. Also, larger models aren't a major concern since each worker operates on a subsection of the model using the same training data. When using model parallelism in training, the model is divided across k workers with each worker holding a part of the model. A naive approach to model parallelism is to divide an n-layered neural network into k workers by simply hosting n over k layers on each worker. More sophisticated methods make sure that each worker is similarly busy by analyzing the computational complexity of each layer.

Standard model parallelism enables training of larger neural networks but suffers from a large hit in performance since workers are constantly waiting for each other and only one can perform updates at a given time. Moving back to data parallelism, let's look at some benchmarks.

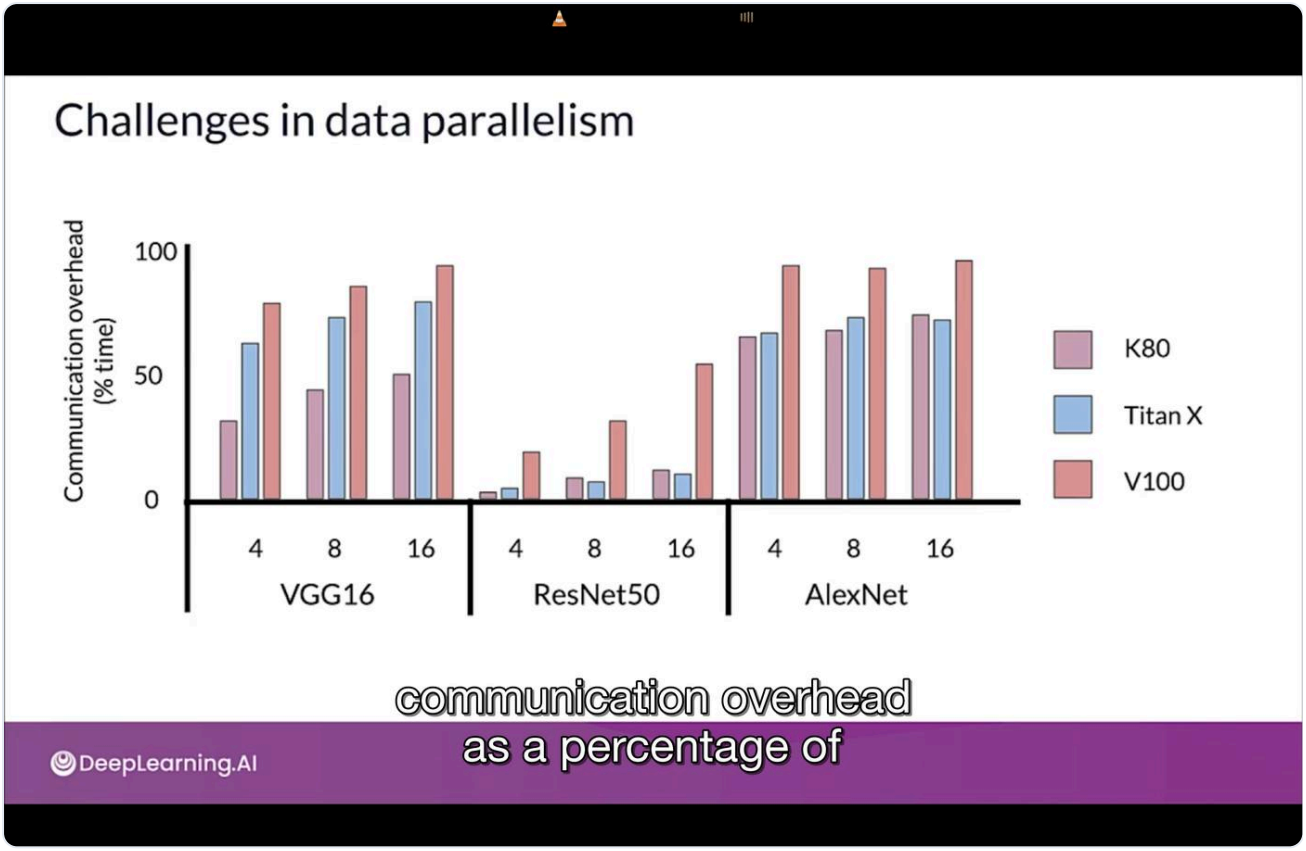

FIGURE 25.5 | Challenges in Data Parallelism

This chart shows communication overhead as a percentage of total training time for three different hardware configurations. Many models like AlexNet and VGG16 have a high communication overhead even on the relatively slow K80 accelerator. Two factors contribute to an increase in the communication overhead across all models, an increase in the number of data-parallel workers and an increase in GPU compute capacity. With data parallelism, the input dataset is partitioned across multiple GPUs. Each GPU maintains a full copy of the model and trains on its own partition of data while periodically synchronizing weights with other GPUs, using either collective communication primitives or parameter servers.

The frequency of parameter synchronization affects both statistical and hardware efficiency. Synchronizing at the end of every mini-batch reduces the staleness of weights used to compute gradients, ensuring good statistical efficiency. Unfortunately, this requires each GPU to wait for gradients from other GPUs, which significantly lowers hardware efficiency. Communication stalls are inevitable in data-parallel training due to the structure of neural networks, and the result is that communication can often dominate total execution time. Rapid increases in accelerator speeds further shift the training bottleneck towards communication. Then there's another problem.

Accelerators. They have limited memory and limited communication bandwidth on the host machine. That means that model parallelism is needed for training bigger models on accelerators by dividing the model into partitions and assigning different partitions to different accelerators. But due to the sequential nature of neural networks, this naive strategy may result in only one accelerator being active during computation, significantly underutilizing accelerator compute capacity.

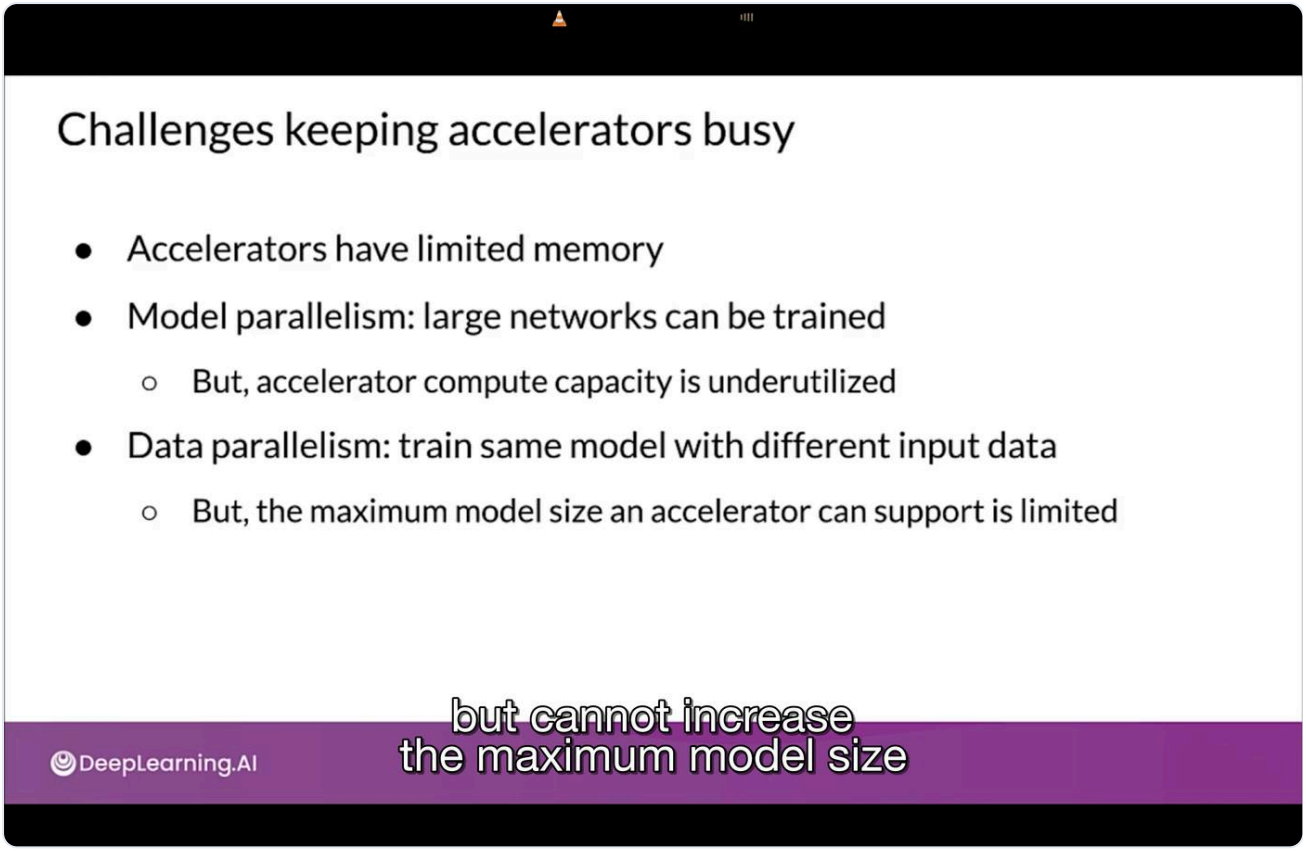

FIGURE 25.6

On the other hand, a standard data parallelism approach allows concurrent training of the same model with different input data on multiple accelerators, but cannot increase the maximum model size an accelerator can support. The issues with data and model parallelism have led to the development of pipeline parallelism. In this diagram.

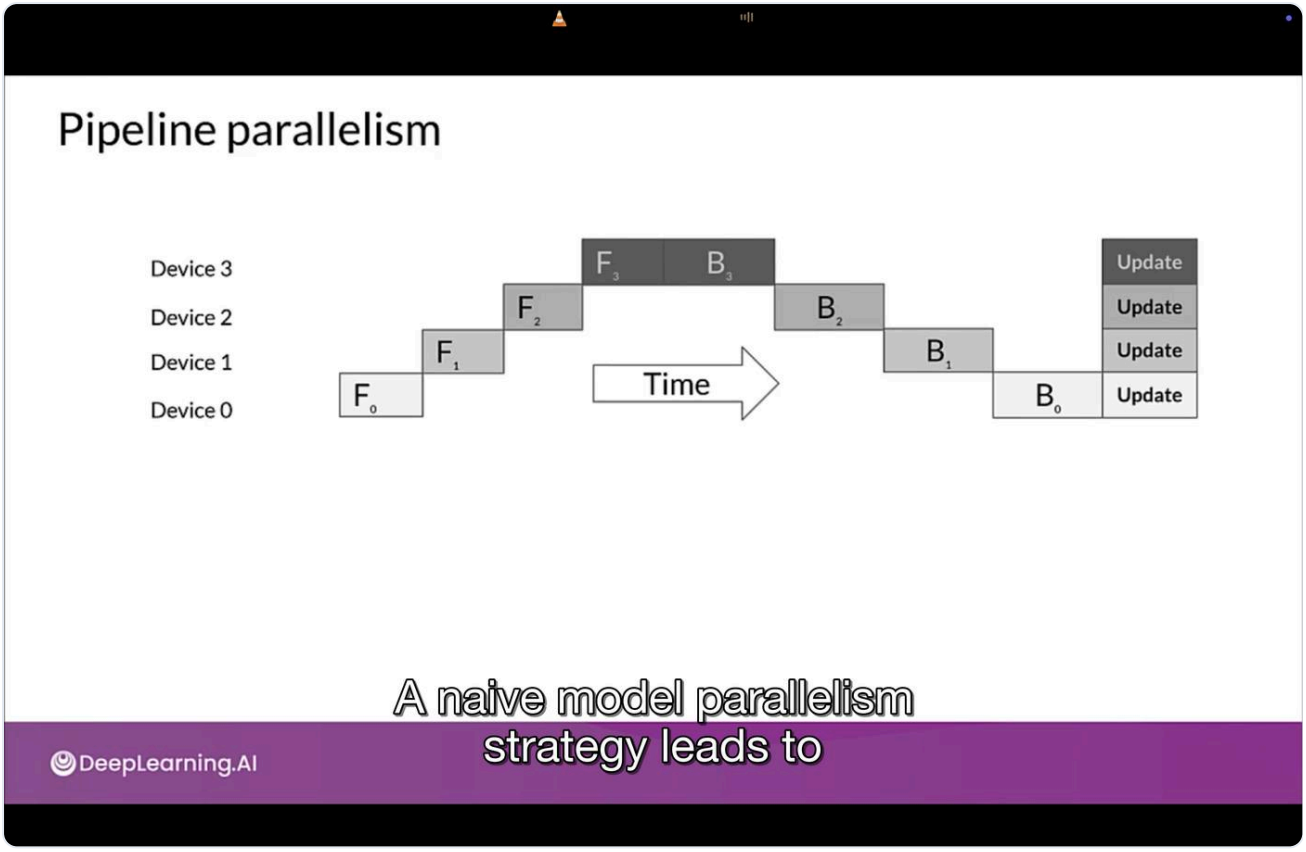

FIGURE 25.7 | Pipeline Parallelism

A naive model parallelism strategy leads to severe under utilization due to the sequential nature of the model, only one accelerator is active at a time. To enable efficient training across multiple accelerators, you need to find a way so that you can partition a model across different accelerators and automatically split a mini-batch of training examples into smaller micro-batches. By pipelining any execution across micro-batches, accelerators can operate in parallel. In addition, gradients are consistently accumulated across micro-batches so that the number of partitions does not affect model quality.

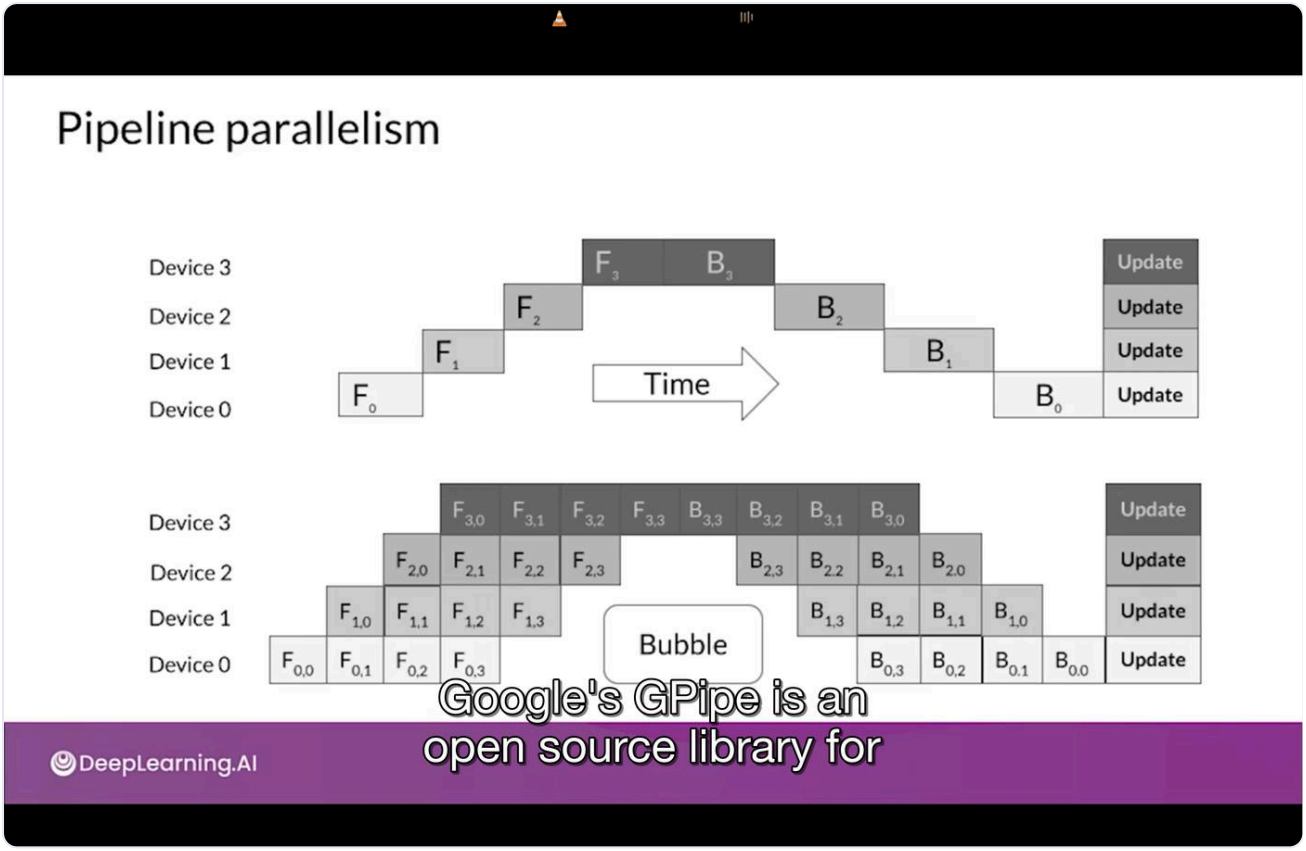

FIGURE 25.8 | Pipeline Parallelism

Google's GPipe is an open source library for efficiently training large-scale models using pipeline parallelism. In the diagram at the bottom, GPipe divides the input mini-batch into smaller micro-batches enabling different accelerators to work on separate micro-batches at the same time. GPipe essentially presents a new way to approach model parallelism, which allows training of large models on multiple hardware devices with an almost one-to-one improvement in performance. It also helps models include significantly more parameters, allowing for better results in training. There's also PipeDream from Microsoft, which also supports pipeline parallelism.

GPipe and PipeDream are similar in many ways. Pipeline parallelism frameworks such as GPipe and PipeDream integrate both data and model parallelism to achieve high efficiency and preserve model accuracy. They do that by dividing mini-batches into smaller micro-batches, by allowing different workers to work on different micro-batches in parallel.

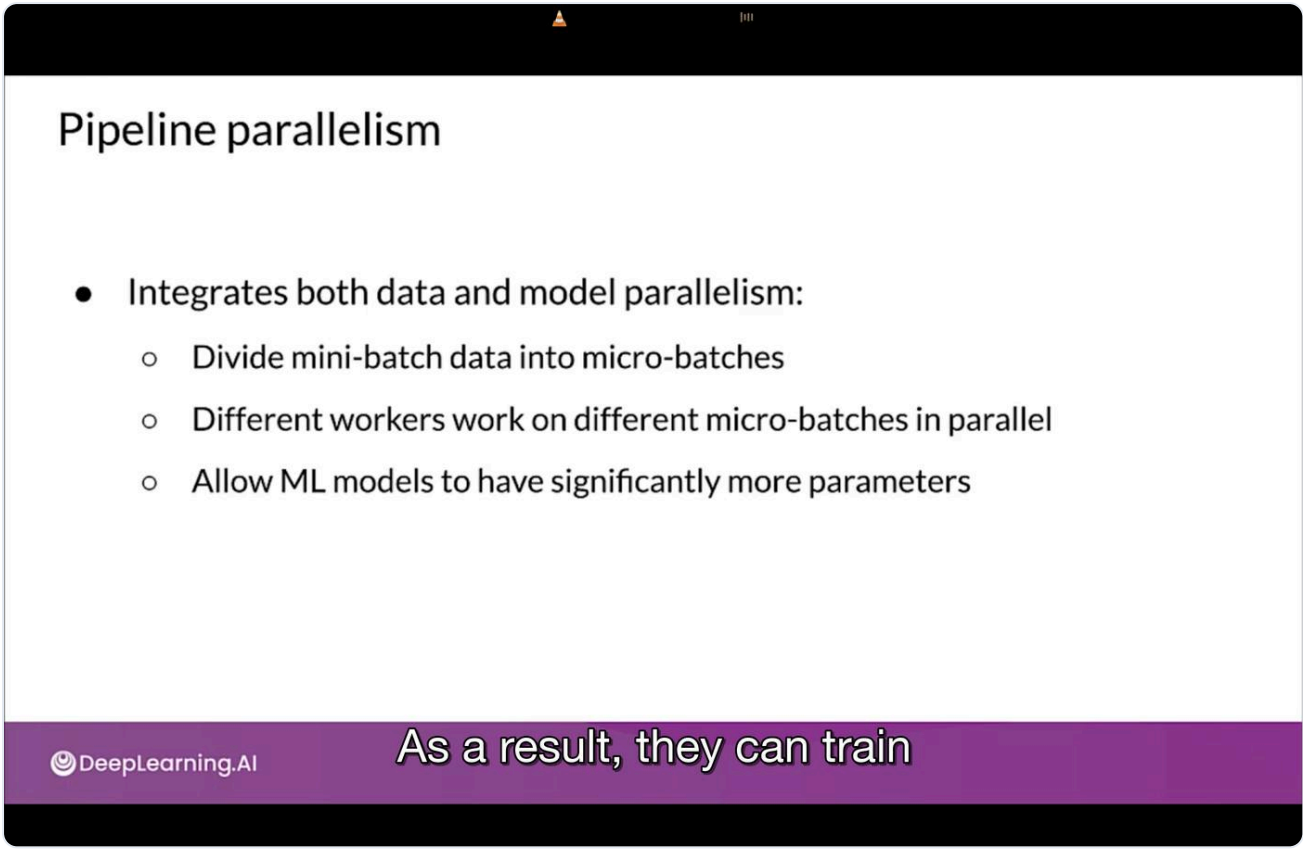

FIGURE 25.9 | Pipeline Parallelism

As a result, they can train models with significantly more parameters. GPipe is an open source distributed machine learning library that uses synchronous mini-batch gradient descent for training. It partitions a model across different accelerators and automatically splits a mini-batch of training examples into micro-batches. By pipelining the execution across micro-batches, accelerators can train in parallel.

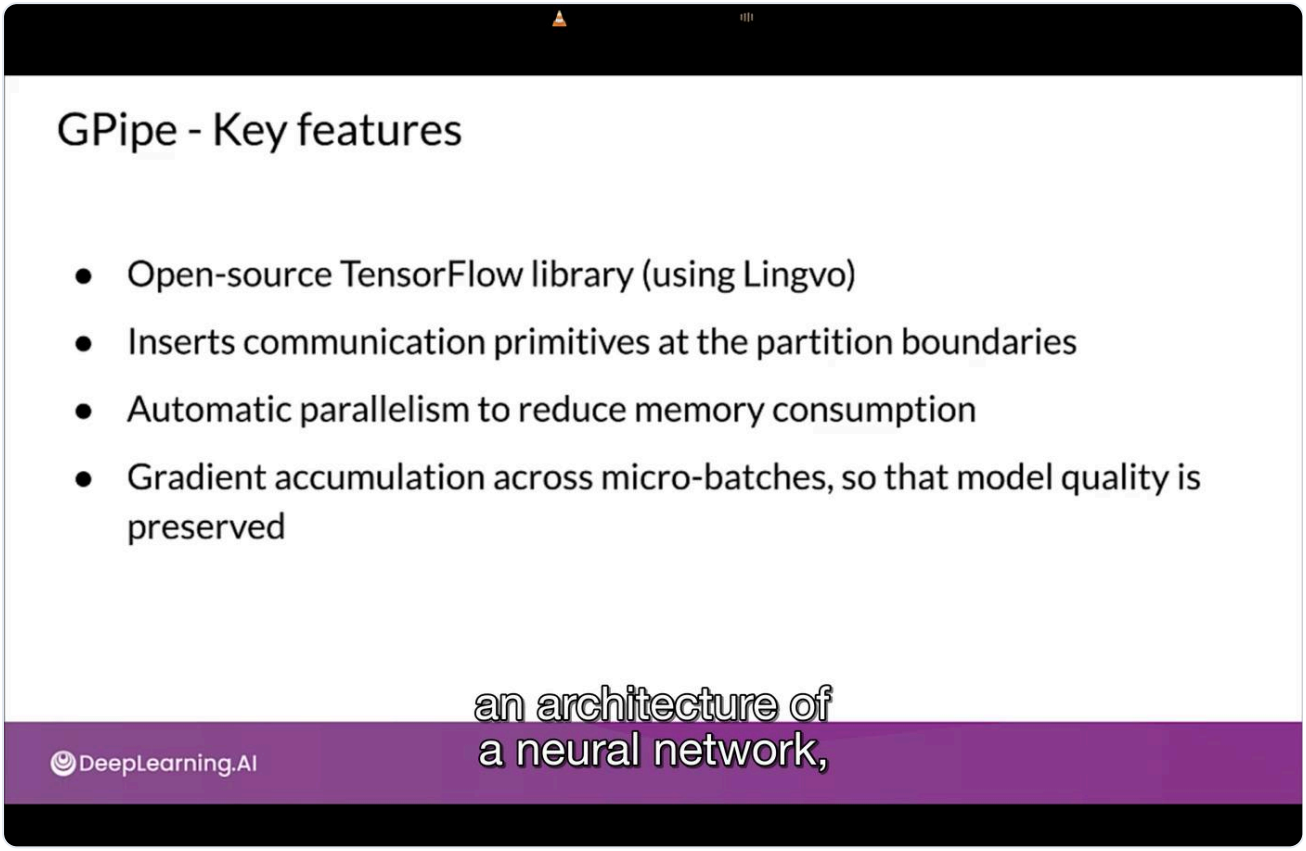

FIGURE 25.10

GPipe receives as input an architecture of a neural network, a mini-batch size and the number of hardware devices that will be available for the calculation. It then automatically divides the network layers into stages and the mini-batches into micro-batches, spreading them across the devices.

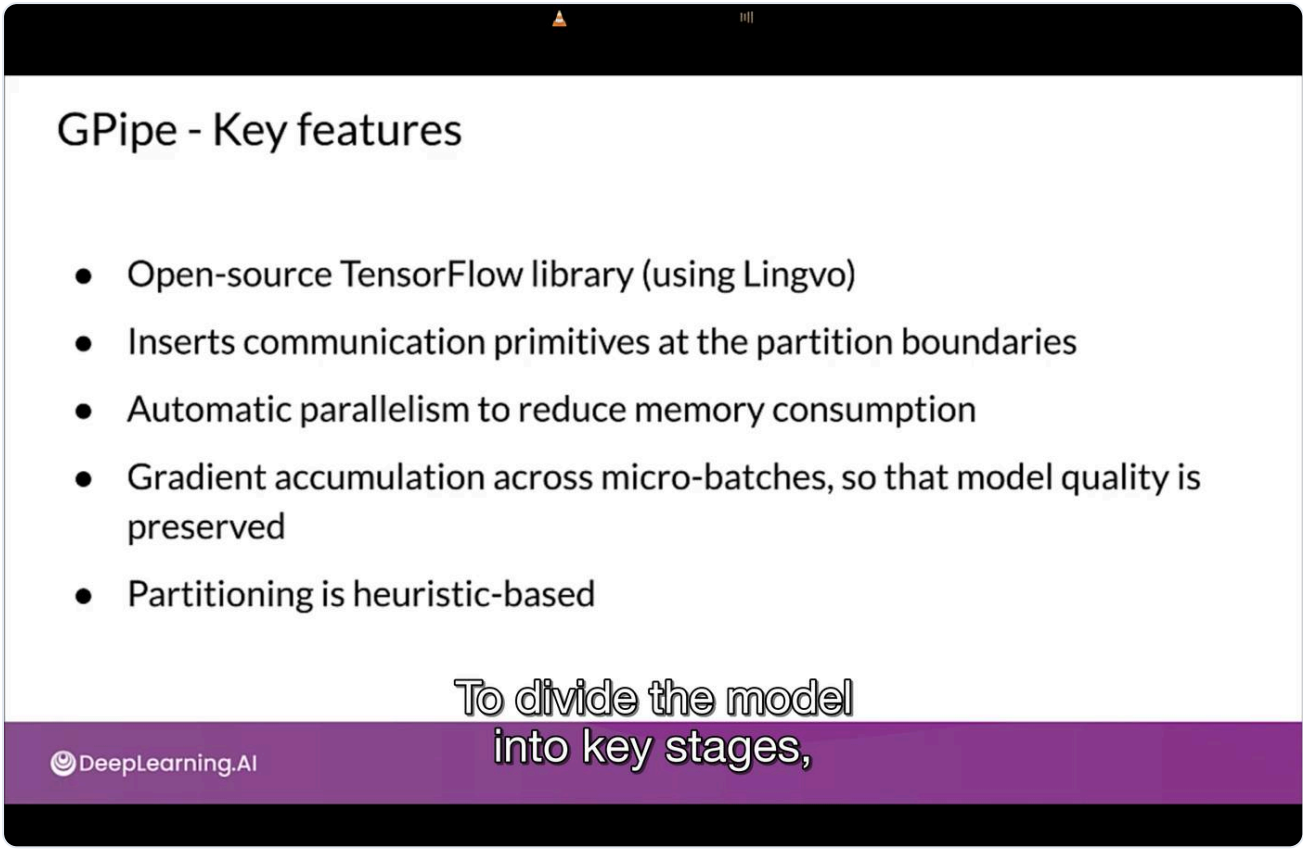

FIGURE 25.11

To divide the model into key stages, GPipe estimates the cost of each layer given its activation function and the content of the training data. GPipe attempts to maximize memory allocation for model parameters. The research team at Google ran experiments on Cloud TPU v2s, each of which has eight accelerator cores and 64 gigabytes of memory; eight gigabytes per accelerator. Without GPipe, a single accelerator can only train up to 82 million model parameters due to memory constraints. Thanks to re-computation in back propagation and batch splitting, GPipe reduce intermediate activation memory from 6.2 gigabytes to 3.4 gigabytes, enabling a single accelerator to train up to 318 million parameters.

They also saw that with pipeline parallelism, the maximum model size was proportional to the number of partitions as expected. With GPipe, testing with an AmoebaNet model was able to incorporate 1.8 billion parameters on the eight accelerators of the Cloud TPU, 25 times more than is possible without GPipe.

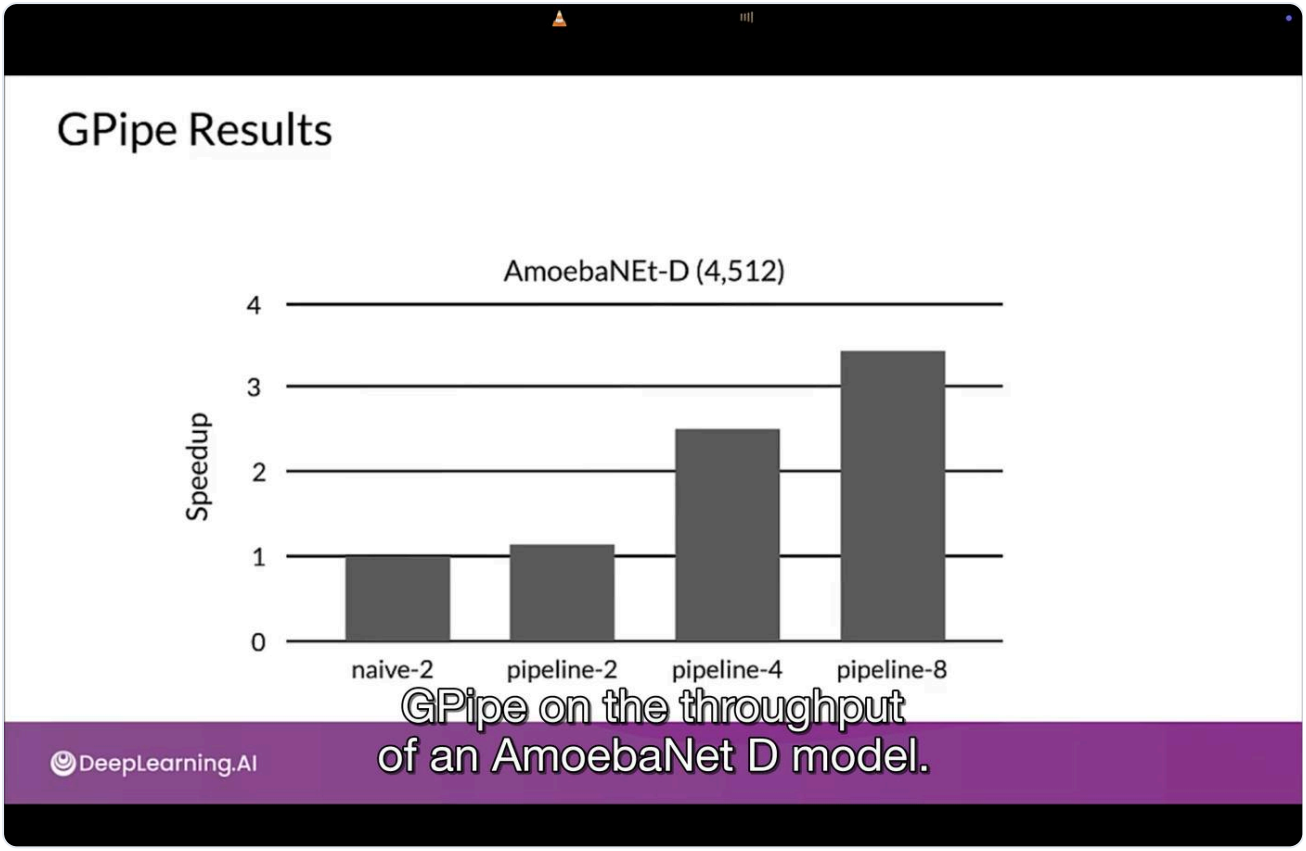

FIGURE 25.12 | GPipe Results

To test efficiency, they measured the effects of GPipe on the throughput of an AmoebaNet D model. Since training required at least two accelerators to fit the model size, they measured the speed up with respect to the naive case with two partitions, but no pipeline parallelism. They observed an almost linear speedup in training. Compared to the naive approach with two partitions, distributing the model across four times the accelerators achieved a speedup of 3.5 times.

LECTURE 26

Knowledge Distillation

So far, we've discussed ways to optimize the implementation of models to make them more efficient. But you can also try to capture or distill the knowledge that has been learned by a model, in a more of compact model by using a different style of training. This is known as knowledge distillation. Let's take a look at that now. By now you've seen a few different ways to optimize your models. But what happens if you want to deploy a model that is relatively complex and significantly large in size? Would merely reducing the number of parameters help with deployment? Or would reducing the model complexity help you deploy these larger, more powerful models?

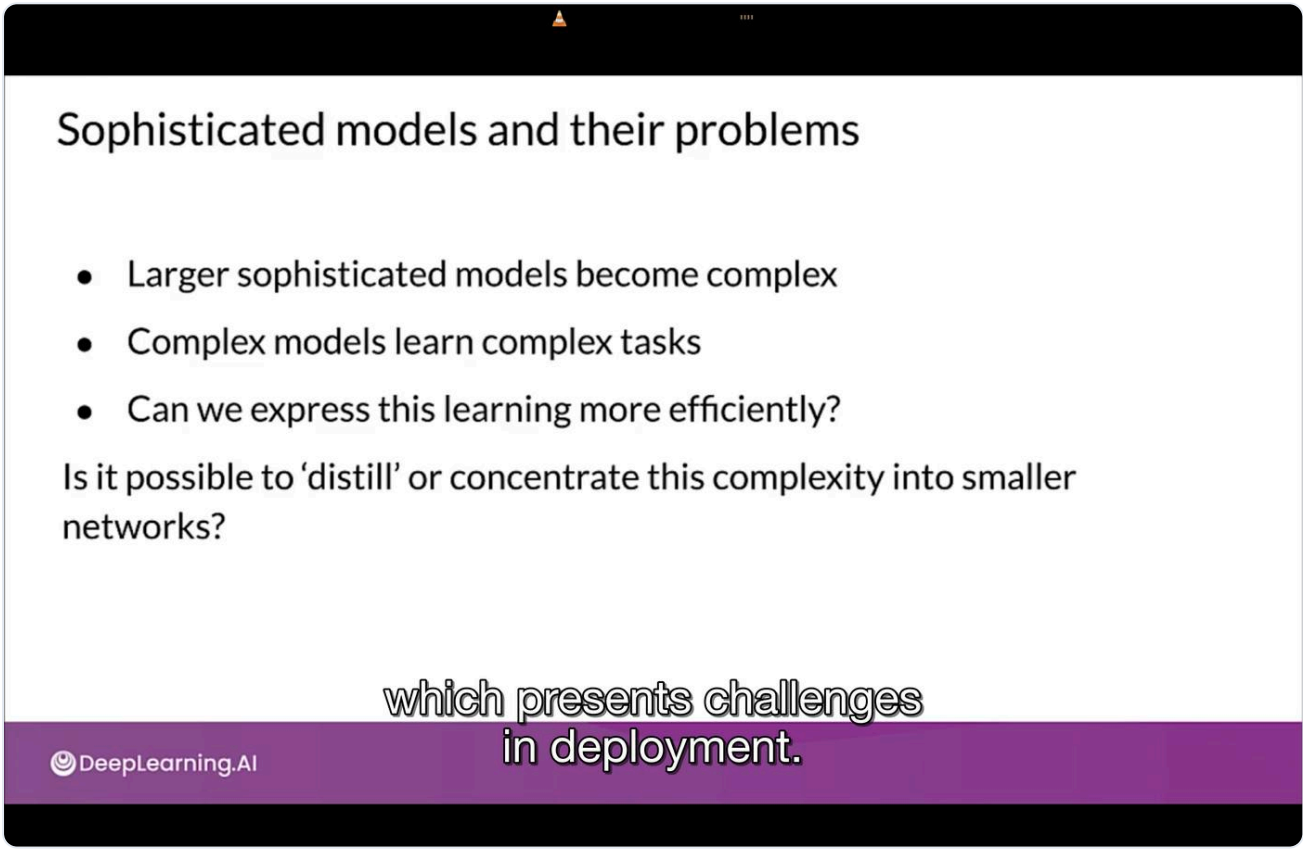

FIGURE 26.1

Let's try to answer those questions. For starters, let's look at how larger models become complex. Try to find out whether it's possible to have the same level of sophistication with smaller models. Models tend to become larger and more complex as they try to capture more information or knowledge in order to learn complex tasks. But if we can express or represent this learning more efficiently, we might be able to create smaller models that are equivalent to these larger, more complex models. As an example, let's take a look at a complex image classification model which presents challenges in deployment. For example, let's take a look at GoogLeNet.

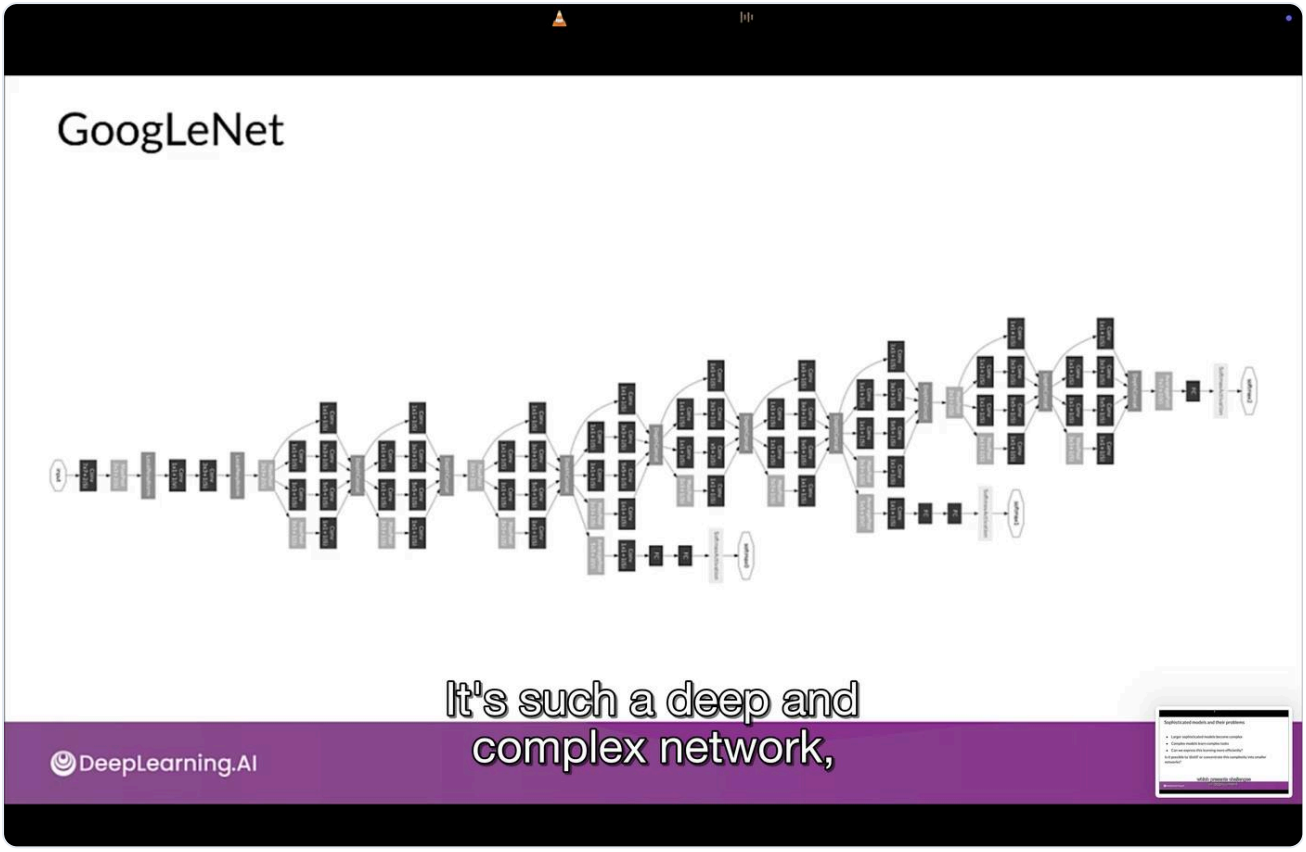

FIGURE 26.2

It's such a deep and complex network, that it doesn't even fit on this slide. The fact that it is so deep, gives it the ability to express complex relationships between features. But because it's so large, it's difficult or impossible to deploy it in many production environments, including mobile phones and edge devices. But can you have the best of both worlds, and capture the knowledge contained in a complex model like GoogLeNet. In a much smaller, more efficient model? That's the goal of knowledge distillation. Rather than optimizing the network implementation, as we saw with quantization and pruning. Knowledge distillation seeks to create a more efficient model which captures the same knowledge as a more complex model.

If needed, further optimization can then be applied to the result.

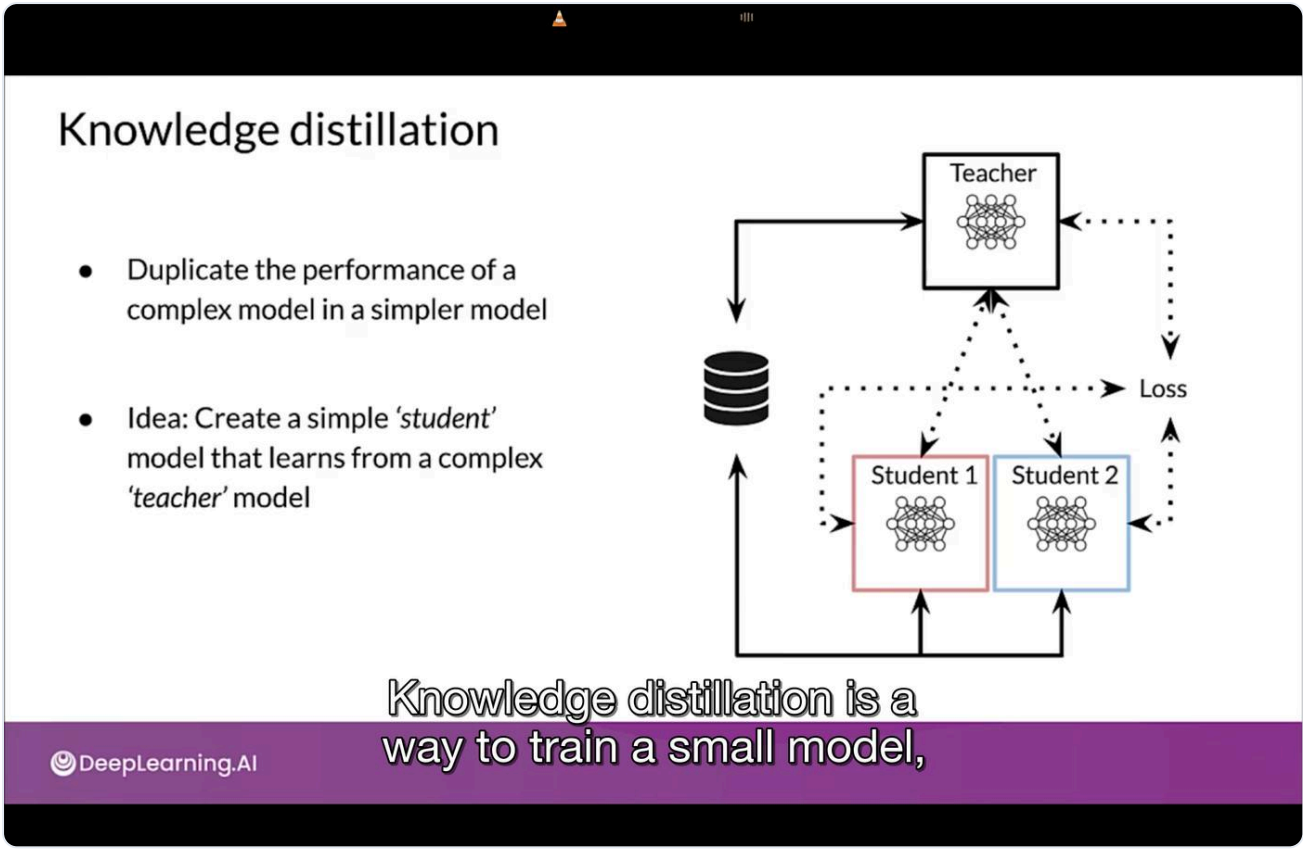

FIGURE 26.3 | Knowledge Distillation

Knowledge distillation is a way to train a small model, to mimic a larger model, or even an ensemble of models. It starts, by first training a complex model or model ensemble to achieve a high level of accuracy. It then uses that model as a teacher for a simpler student model. Which will then be the actual model that gets deployed to production. This teacher network can be either fixed or jointly optimized. Can even be used to train multiple student models of different sizes simultaneously.

LECTURE 27

First Quantitative Results of Distillation

Let's take a deeper look at how knowledge distillation works. Knowledge distillation introduces the idea of a teacher model and a student model. Let's discuss the roles of the teacher and student in knowledge distillation. In knowledge distillation, the training objective functions for the student and the teacher are different. The teacher will be trained first using a standard objective function that seeks to maximize the accuracy or a similar metric of the model. This is normal model training. The student then seeks transferable knowledge. It uses that objective function that seeks to match the probability distribution of the predictions of the teacher.

Notice that the student is not just learning the teacher's predictions, but the probabilities of the predictions.

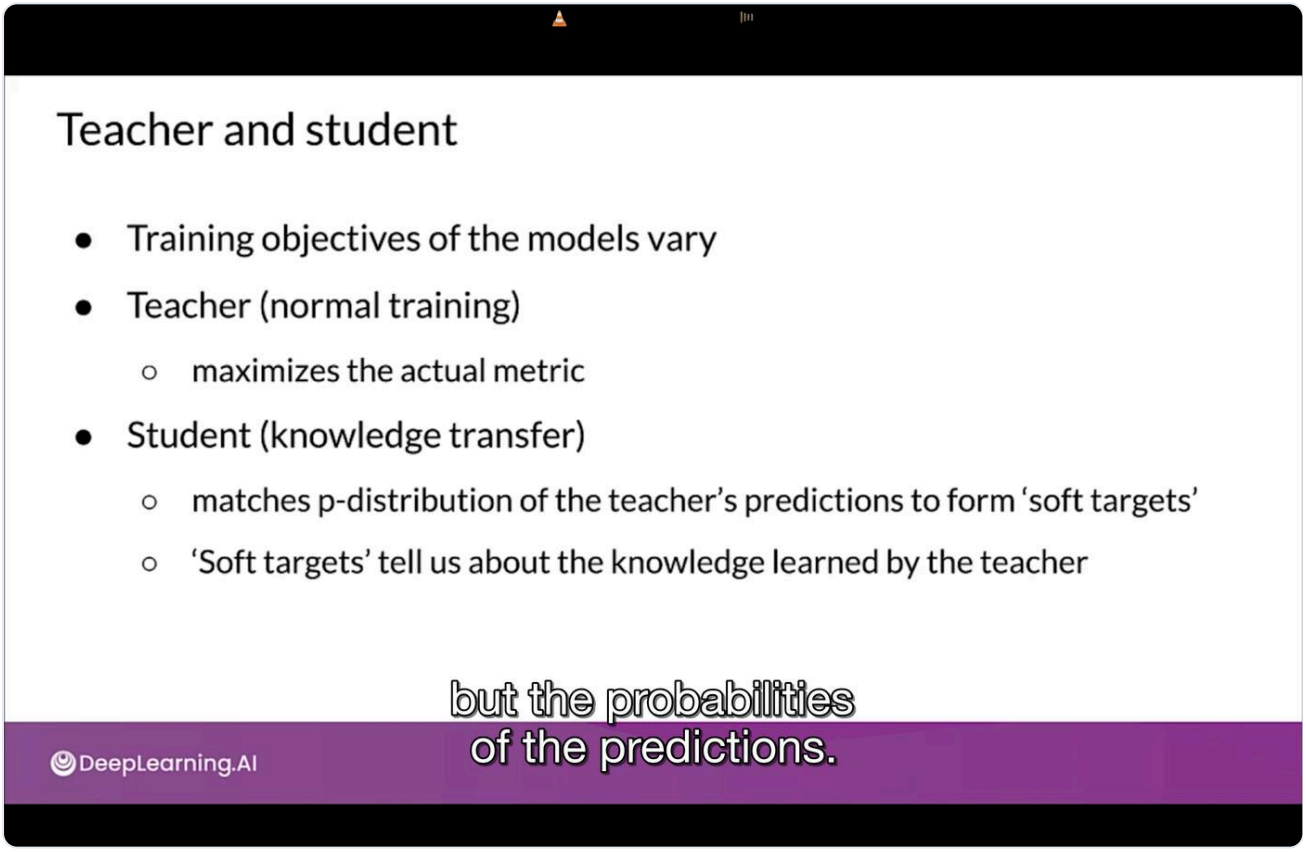

FIGURE 27.1 | Teacher and Student

The probabilities of the predictions of the teacher form soft targets, which provide more information about the knowledge learned by the teacher than the resulting predictions themselves. Don't worry. Knowledge distillation is not one of the dark arts, but it does involve dark knowledge. Let's discuss that now. The way knowledge distillation works is that you transfer knowledge from the teacher to the student by minimizing a loss function, in which the target is the distribution of class probabilities predicted by the teacher model.

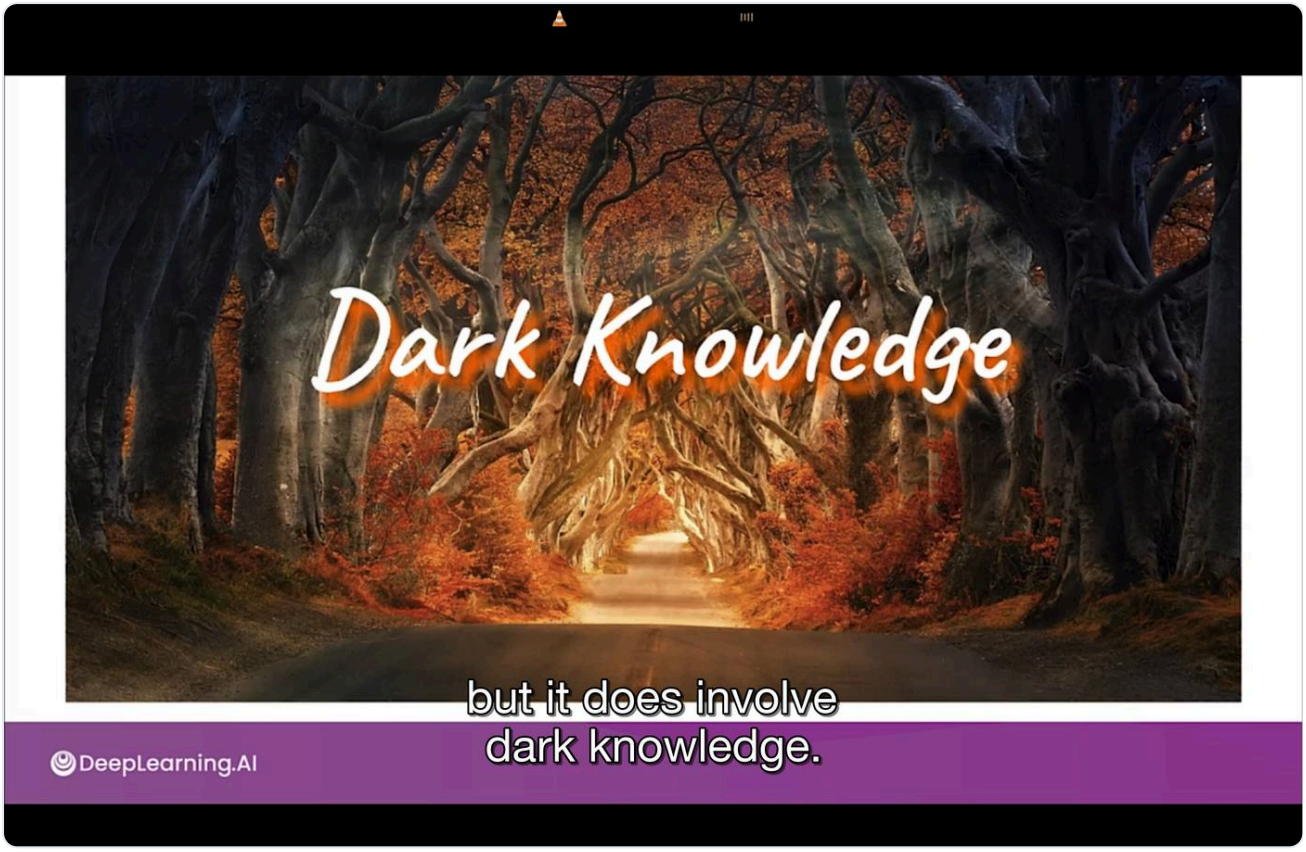

FIGURE 27.2

What happens here is that the teacher models logits form the input to the final softmax layer, which is often used since they provide more information about the probabilities of all target classes for each example. However, in many cases, this probability distribution has the correct class at a very high probability with all the other class probabilities very close to zero. Realistically, it sometimes doesn't provide much information beyond the ground truth labels already provided in the dataset. To tackle this issue, Hinton, Vinyals, and Dean, introduced the concept of a softmax temperature. By raising the temperature in the objective functions of the student and teacher, you can improve the softness of the teacher's distribution.

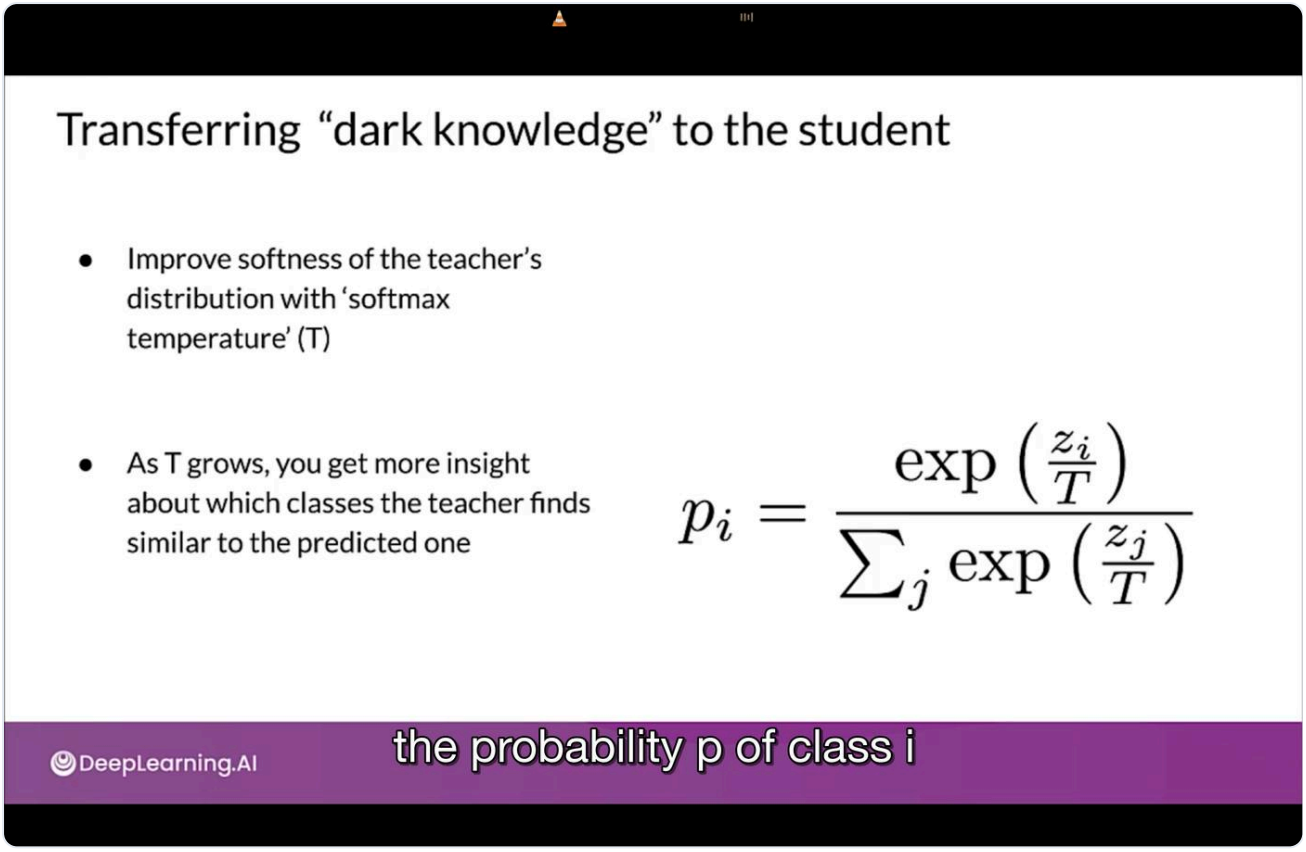

FIGURE 27.3

In the formula here, the probability p of class i is calculated from the logits z as shown. T simply refers to the temperature parameter. When t is 1, you get the standard softmax function. But as T starts growing, the probability distribution generated by the softmax function becomes softer, providing more information as to which classes the teacher found more similar to the predicted class. The authors call this the dark knowledge embedded in the teacher model. It is this dark knowledge that you are transferring to the student model in the distillation process. Various techniques are used to train the student to match the teacher's soft targets. In one approach, the student is trained on both the teacher's logits and the target labels using a normal objective function, and the two objective functions are weighted and combined in backpropagation.

In another similar approach, the distributions of the student's predictions and the teacher's predictions are compared using a metrics such as KL divergence.

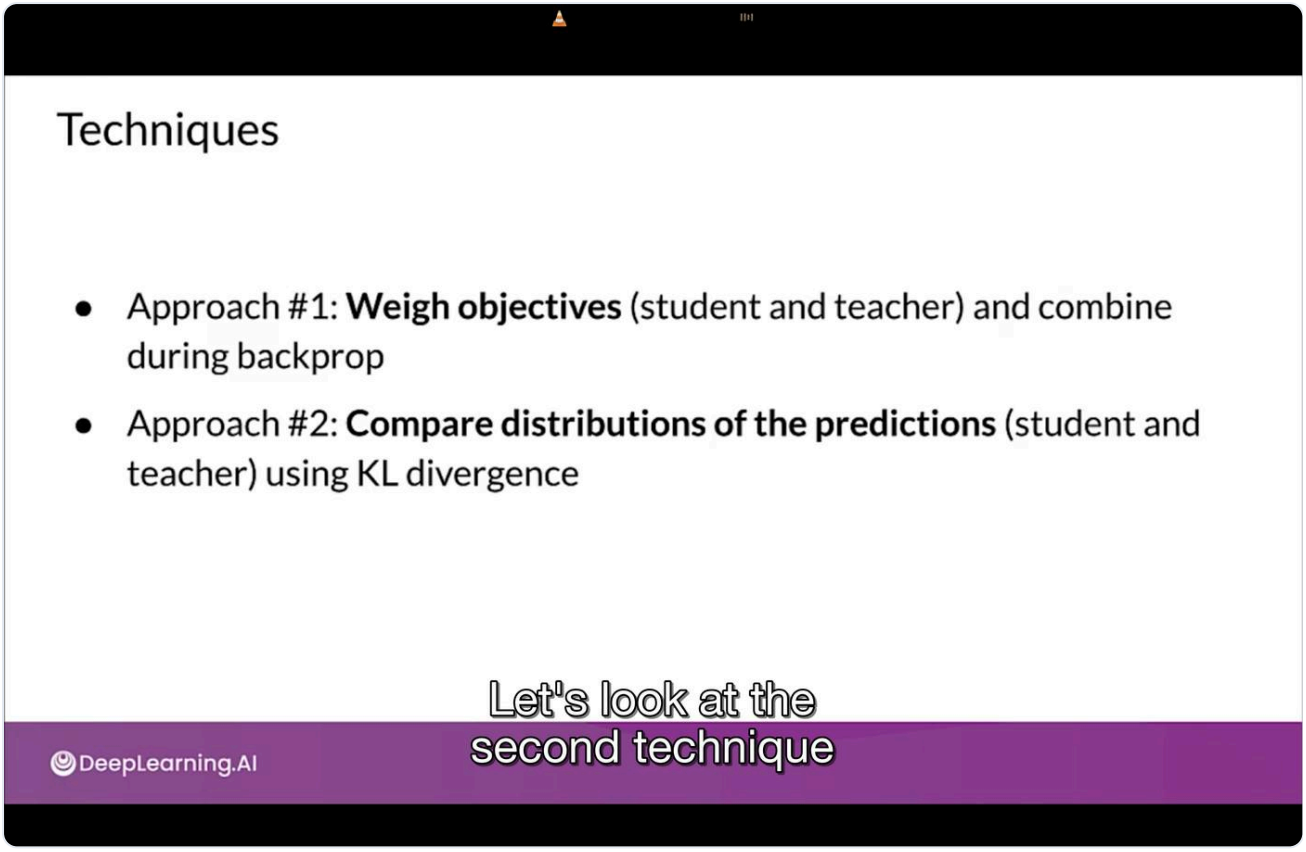

FIGURE 27.4

Let's look at the second technique now since it's more widely used.

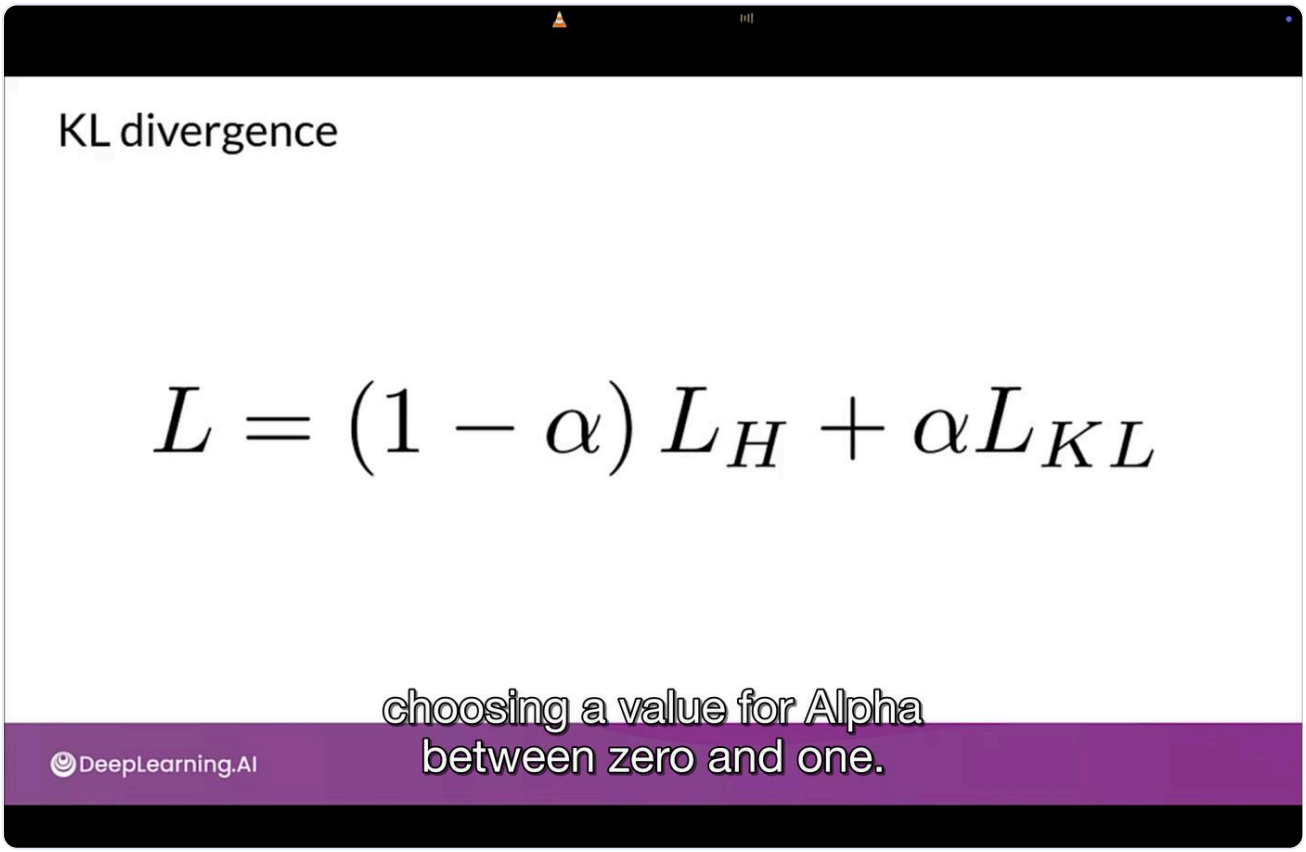

FIGURE 27.5

Generally, knowledge distillation is done by blending two loss functions, choosing a value for Alpha between zero and one. Here, L is the cross-entropy loss from the hard labels, and L_KL is the Kullback-Leibler divergence loss from the teacher's logits. In the case of heavy augmentation, you simply cannot trust the original hard labels due to the aggressive perturbations applied to data. The Kullback-Leibler divergence here is a metric of the difference between two probability distributions. You want those two probability distributions to be as close as possible. The objective here is to make the distribution over the classes predicted by the student as close as possible to the teacher.

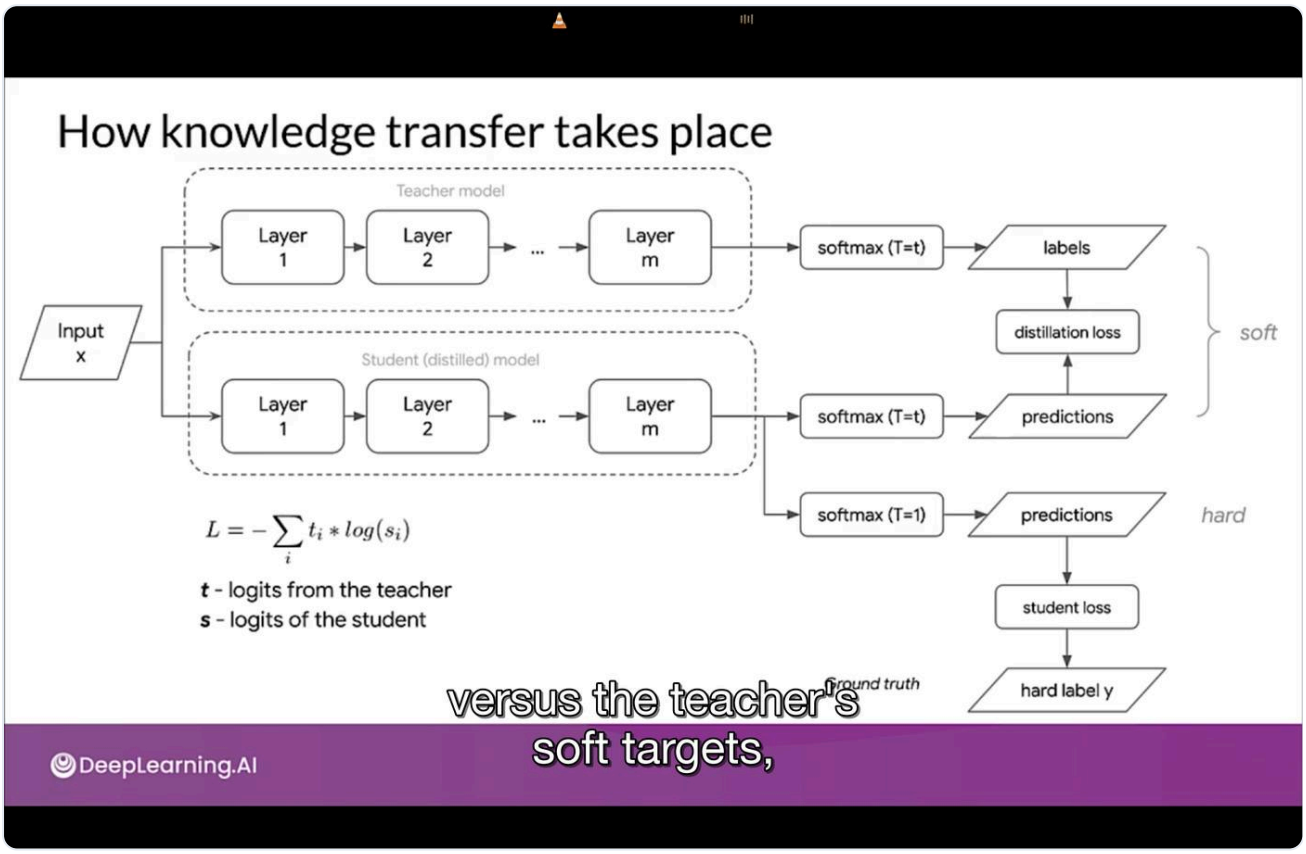

FIGURE 27.6

When computing the loss function versus the teacher's soft targets, we use the same value of T to compute the softmax on the student's logits. This loss is the distillation loss. The authors also found another interesting behavior. It turns out that distilled models are able to produce the correct labels in addition to the teacher's soft targets. That means that you can calculate the standard loss between the student's predicted class probabilities and the ground truth labels. These are known as hard labels or targets. This loss is the student loss. When you're calculating the probabilities for the student, you set the softmax temperature to one.

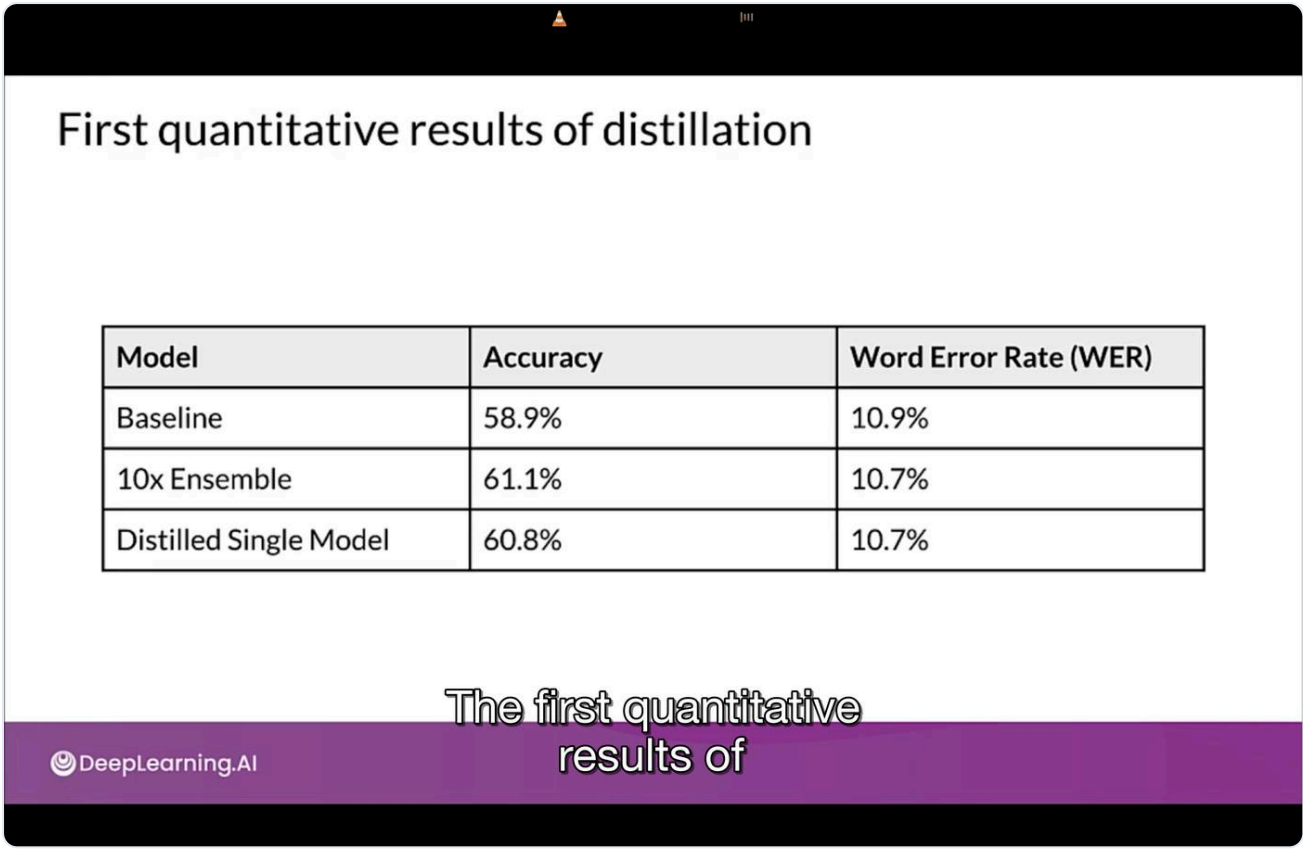

FIGURE 27.7 | First Quantitative Results of Distillation

The first quantitative results of applying knowledge distillation were promising. Hinton and his colleagues trained 10 separate models for an automatic speech recognition task using the same architecture and training procedure as the baseline. Automatic speech recognition tasks at the time relied on deep neural networks to map a short temporal context of features derived from the waveform to a probability distribution over the discrete states of a hidden Markov model. For the models, they randomly initialized their weights with different initial parameter values. This was essentially done so that there was enough diversity in the trained models. When averaging the ensemble's predictions, they would outperform the single models with ease.

They also considered varying the sets of data that each model sees. But they found that it wouldn't significantly impact the results so they decided to use this more straightforward strategy of comparing an ensemble of models against a single model. For the distillation process, they tried different values for the softmax temperature like 1, 2, 5, and 10. They also used a relative weight of 0.5 on the cross-entropy for the hard targets. This table shows that distillation can indeed extract more useful information from the training set than merely using the hard labels to train a single model. More than 80 percent of the improvement in accuracy achieved by an ensemble of 10 models is transferred to the distilled model.

The ensemble gives a smaller gain on the ultimate objective of word error rate on a 23K-word test set due to the mismatch in the objective function. Still, again, the increase in the word error rate achieved by the ensemble is transferred to the distilled model.

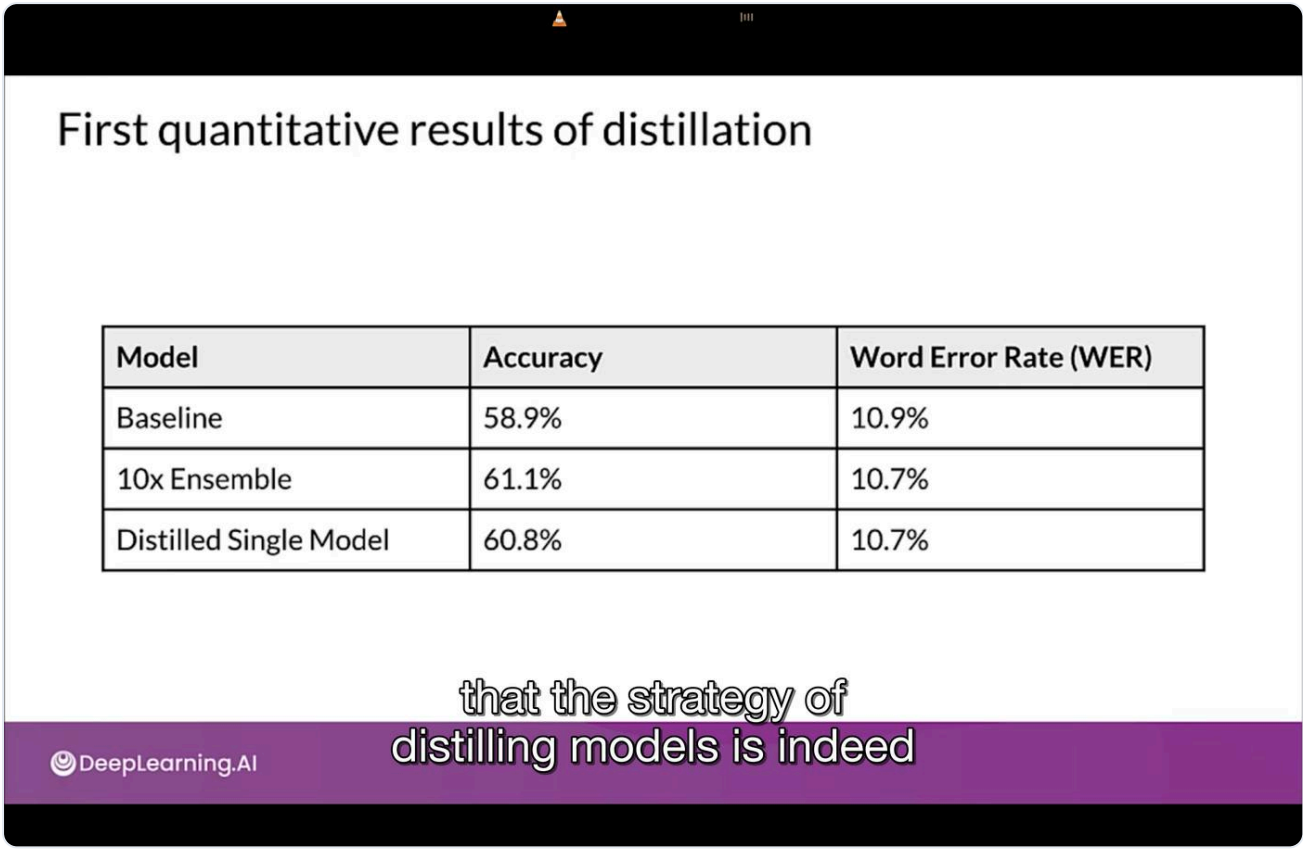

FIGURE 27.8 | First Quantitative Results of Distillation

With this, they were able to show that the strategy of distilling models is indeed beneficial and can be used to achieve the desired effect of distilling an ensemble of models into a single model that works significantly better than a model of the same size that has learned directly from the same training data.

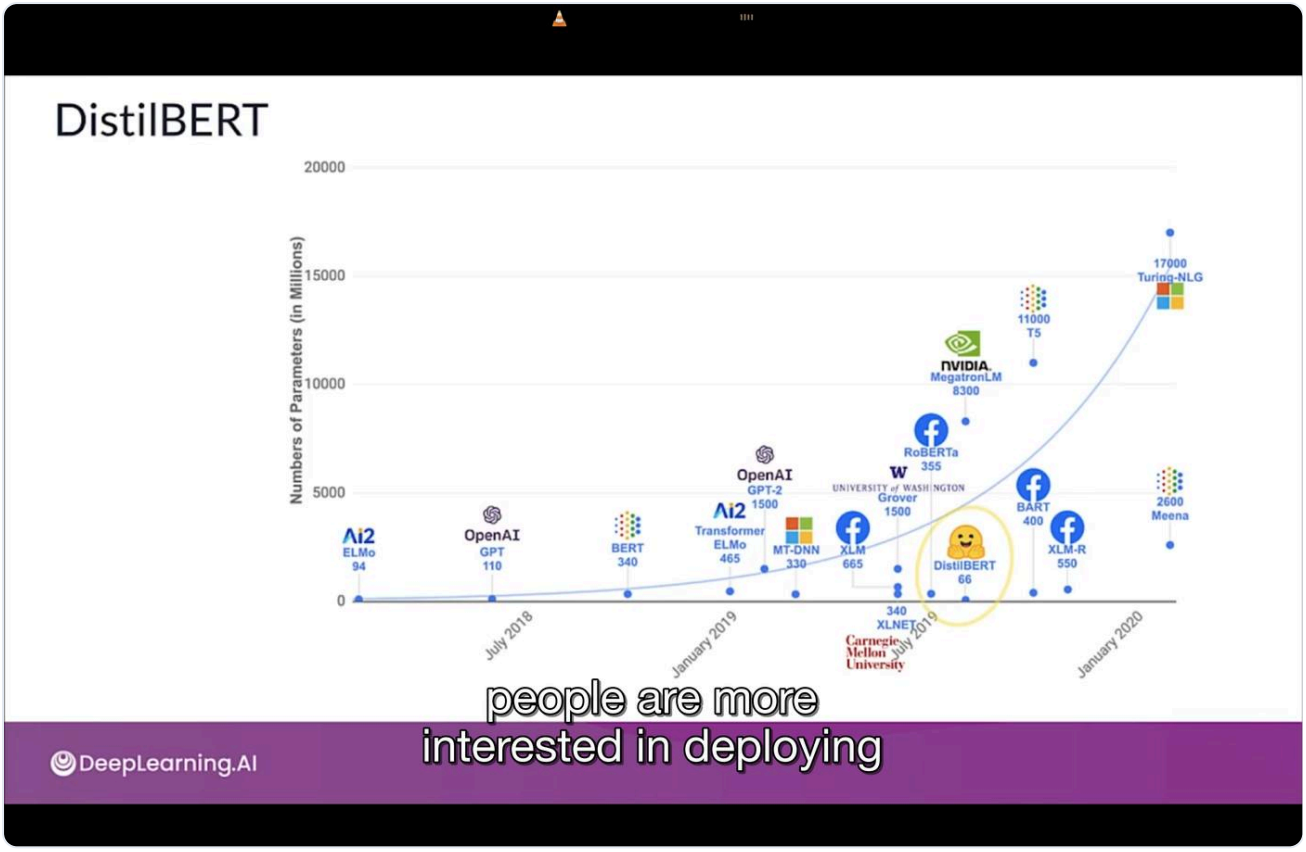

FIGURE 27.9

In the real world though, people are more interested in deploying a low-resource model with close to state of the art results, but a lot smaller and a lot faster. That's why Hugging Face created DistilBERT. A distilled version of BERT which uses 40 percent fewer parameters, runs 60 percent faster while preserving 97 percent of BERT's performance as measured on the GLUE language understanding benchmark. Basically, it's a smaller version of BERT where the token type embeddings and the polar layer typically used for the next sentence classification task are removed. To create DistilBERT, the researchers at Hugging Face applied knowledge distillation to BERT, and hence the name DistilBERT.

They kept the rest of the architecture identical while reducing the number of layers.

LECTURE 28

Train Teacher

Let's take a look at a specific implementation of knowledge distillation for a Q&A task. Let's look at how knowledge can be distilled for question answering.

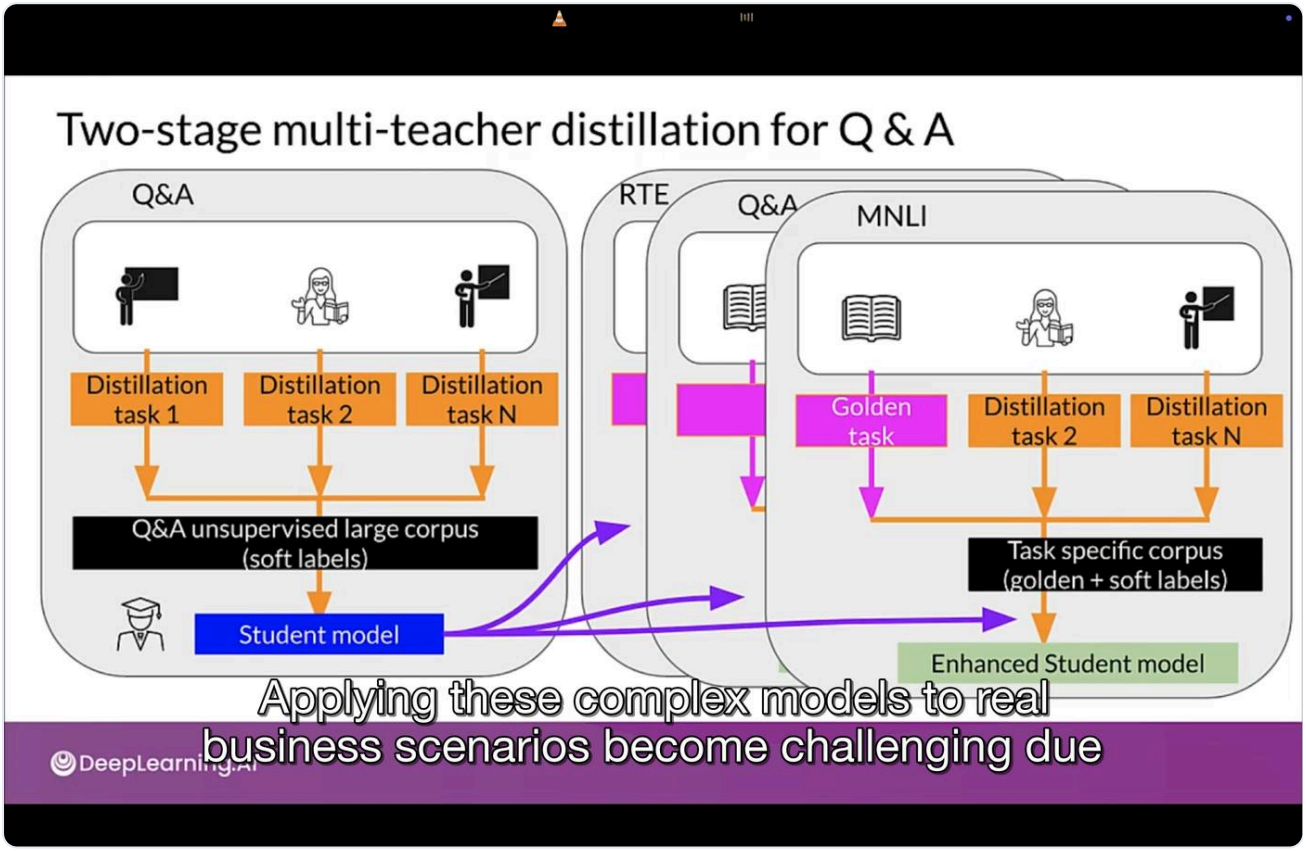

FIGURE 28.1

Applying these complex models to real business scenarios become challenging due to the vast amount of model parameters. Older model compression methods generally suffer from information loss during the model compression procedure. Leading to inferior models compared to the original one to tackle this challenge. Researchers at Microsoft proposed a two stage multi teacher knowledge distillation method, or TMKD for short, for a web question answering system. In this approach, they first developed a general Q&A distillation task for student model, pre training. And further fine tune this pre trained student model with a multi teacher knowledge distillation model.

The basic knowledge distillation approach presented so far, is known as a one on one model because one teacher transfers knowledge to one student. Although this approach can effectively reduce the number of parameters and the time for model inference. Due to the information loss during knowledge desolation, the performance of the student model is sometimes not on par with that of the teacher. This was the driving force for the authors to create a different approach

called M on M or many on many ensemble model. Combining both ensemble and knowledge distillation. This involves first training multiple teacher models. The models could be BERT or GPT or others similarly powerful models, each having different hyper parameters.

Then a student model for each teacher model is then trained. Finally, the student models trained from different teachers are ensemble to create the final result. Here, you prepare each teacher for a particular learning objective and then train them.

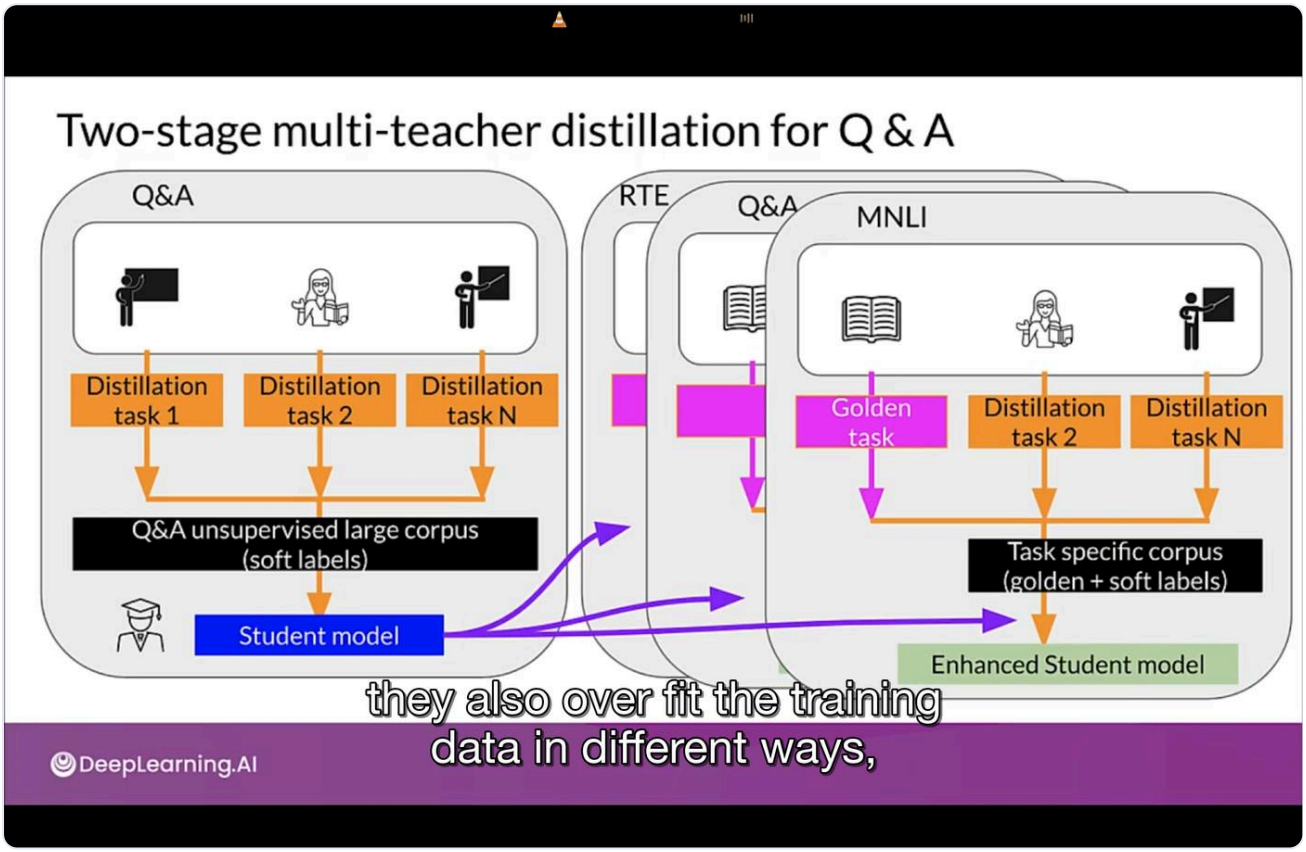

FIGURE 28.2

Different models have different generalization of capabilities and they also over fit the training data in different ways, achieving performance close to the teacher model.

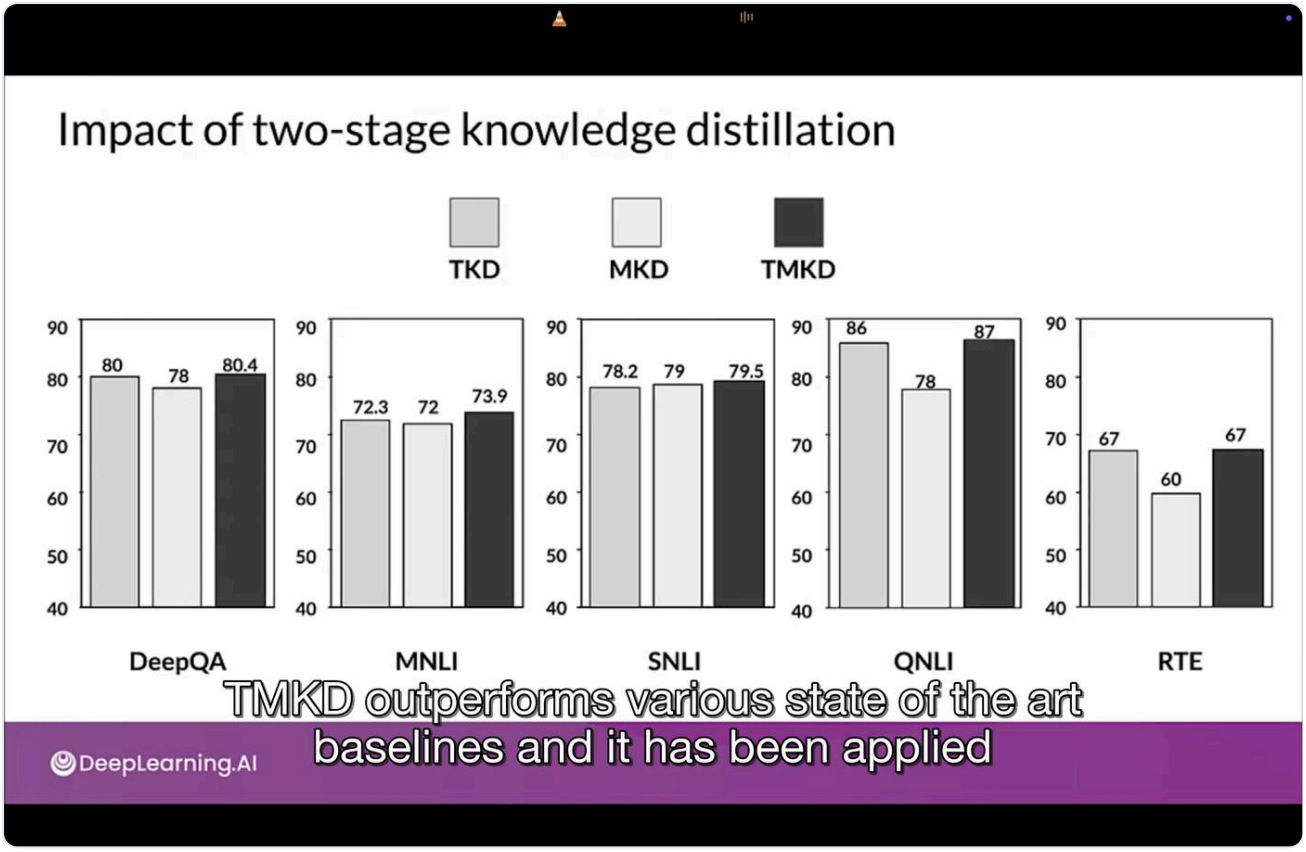

FIGURE 28.3 | Impact of Two-stage Knowledge Distillation

TMKD outperforms various state of the art baselines and it has been applied to real commercial scenarios since ensemble it is employed here. These compressed models benefit from large scale data and learned feature representations well. Results from experiments show that it can considerably outperformed baseline methods. And even chief comparable results to the original teacher models along with a substantial speed up of model inference. The others performed experiments on several datasets using benchmarks that are public. And even large scale to verify the method's effectiveness to support these claims, let's look at its advantages one by one. A unique aspect of TMKD is that it uses a multi teacher distillation task for student model pre training to boost model performance.

To analyze the impact of pre training the authors evaluated two models. The first one TKD is a three layer bert based model which is first trained using basic knowledge distillation pre training on the com Q A dataset. And then fine tune on a task specific corpus by using only one teacher for each task. The second model is a traditional knowledge distillation model which is again the same model but without the distillation pre training stage. TKD showed significant gains by leveraging large scale unsupervised Q&A pairs for distillation pre training. Another benefit of TMKD is its unified framework to learn from multiple teachers jointly. For this, the authors were able to compare the impact of multi teacher versus single teacher knowledge distillation using two models.

MKD a three layer bert based model trained by multi teacher distillation without a pre training stage. And KD, a three layer bert based model trained by single teacher distillation without a pre training stage. Whose aim is to learn from the average score of the teacher models. MKD outperforms KD on a majority of tasks demonstrating that a multi teacher distillation approach. Can

help the student model learn more generalized knowledge, fusing knowledge from different teachers. Finally, they compared TKD, MKD and TMKD with each other. As you can see here, TMKD significantly outperforms TKD at MKD in all datasets. Which verifies the complementary impact of the two stages distillation: pre training and multi teacher fine tuning.

In another example, researchers from google brain and Carnegie Mellon University trained models with a semi supervised learning method called noisy student. In this approach, the knowledge distillation process is iterative. It uses a variation of the classic teacher student paradigm. But here the student is purposely kept larger in terms of the number of parameters than the teacher. This is done so that the model can attain robustness, too noisy labels as opposed to traditional knowledge distillation patterns.

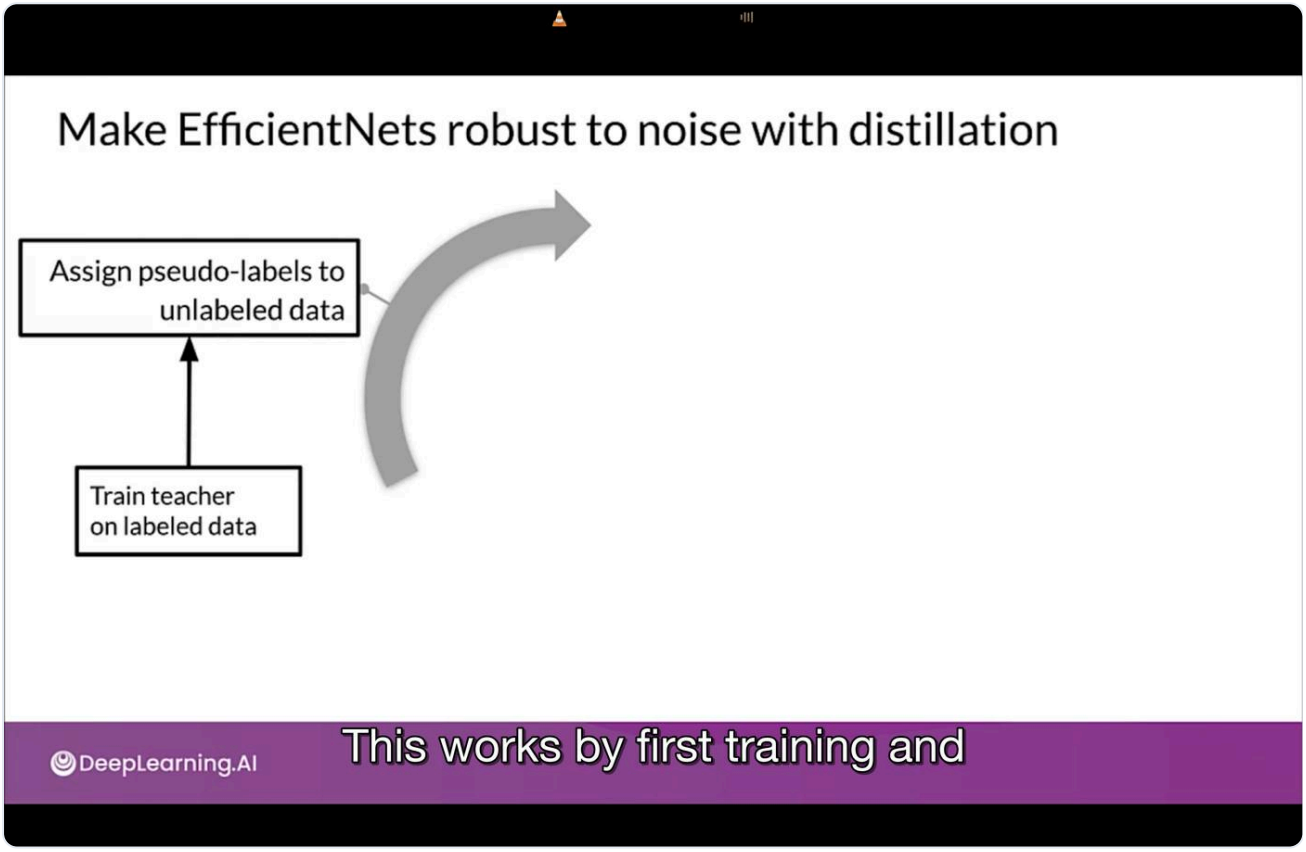

FIGURE 28.4 | Train Teacher

This works by first training and efficient net as the teacher model using labeled images. And then using the teacher to generate pseudo labels on a larger set of unlabeled images.

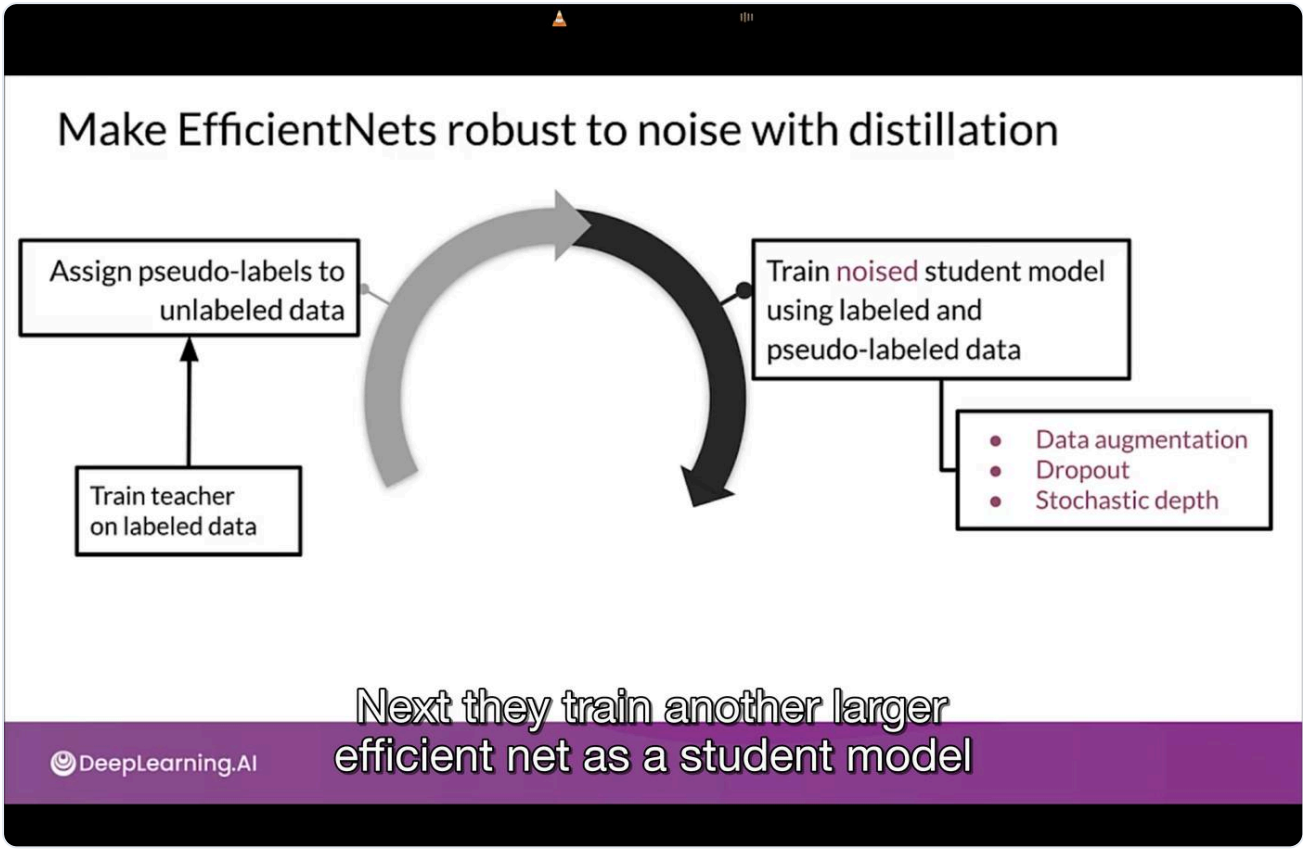

FIGURE 28.5 | Train Teacher

Next they train another larger efficient net as a student model on the combination of labeled and pseudo labeled images. This algorithm was reiterated a number of times by treating the student model as a teacher model to relabel the unlabeled data and train a new student model. An important element of his approach was to ensure that noise was added to the student model using dropout, stochastic depth, data augmentation, and so on. Via rand augment during its training. This noising pushed it to learn harder from pseudo labels. Adding noise to a student ensures that the task is much harder for the student and hence the name noisy student. And it doesn't merely learn the teacher's knowledge.

In an side note, the teacher model is not noise during the generation of pseudo labels to ensure its accuracy isn't altered in any way.

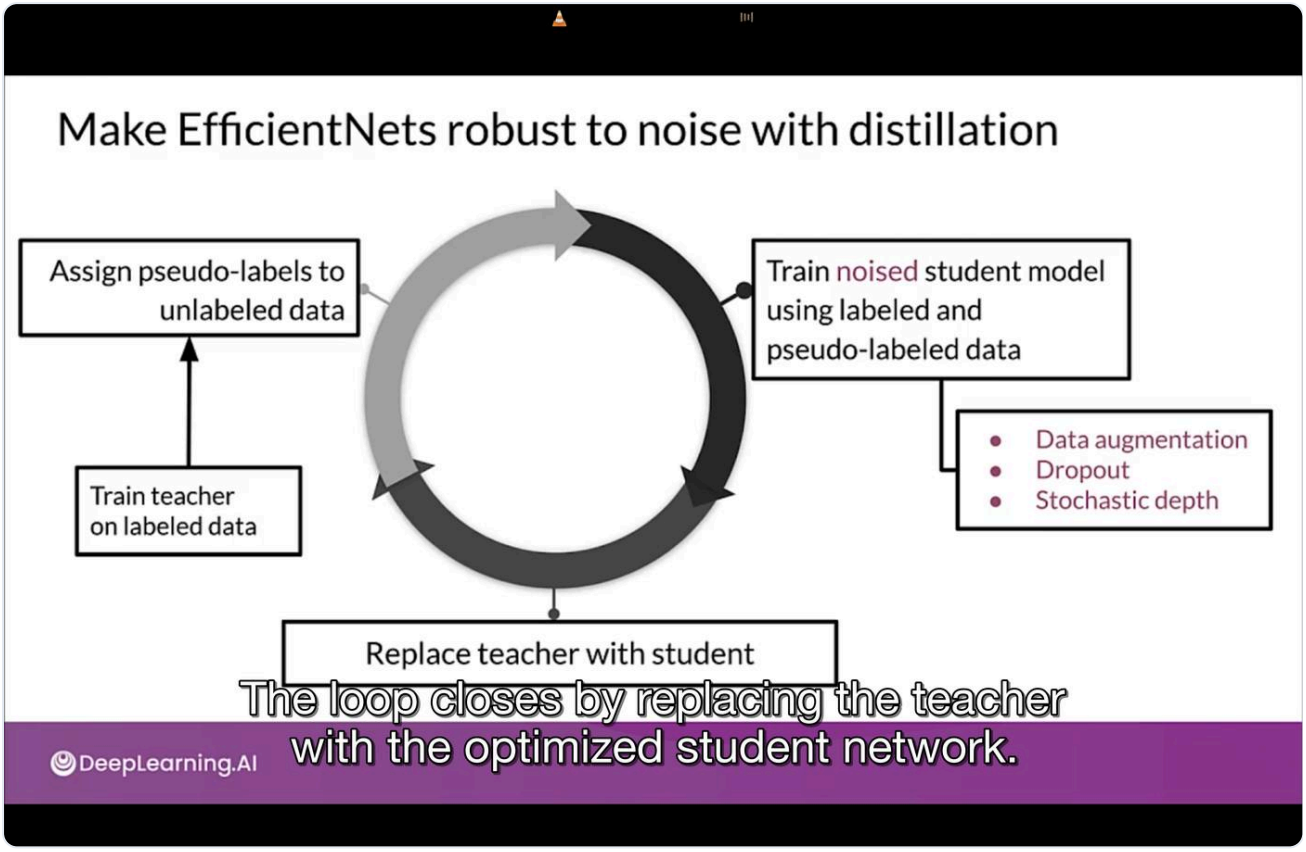

FIGURE 28.6 | Train Noised Student Model

The loop closes by replacing the teacher with the optimized student network.

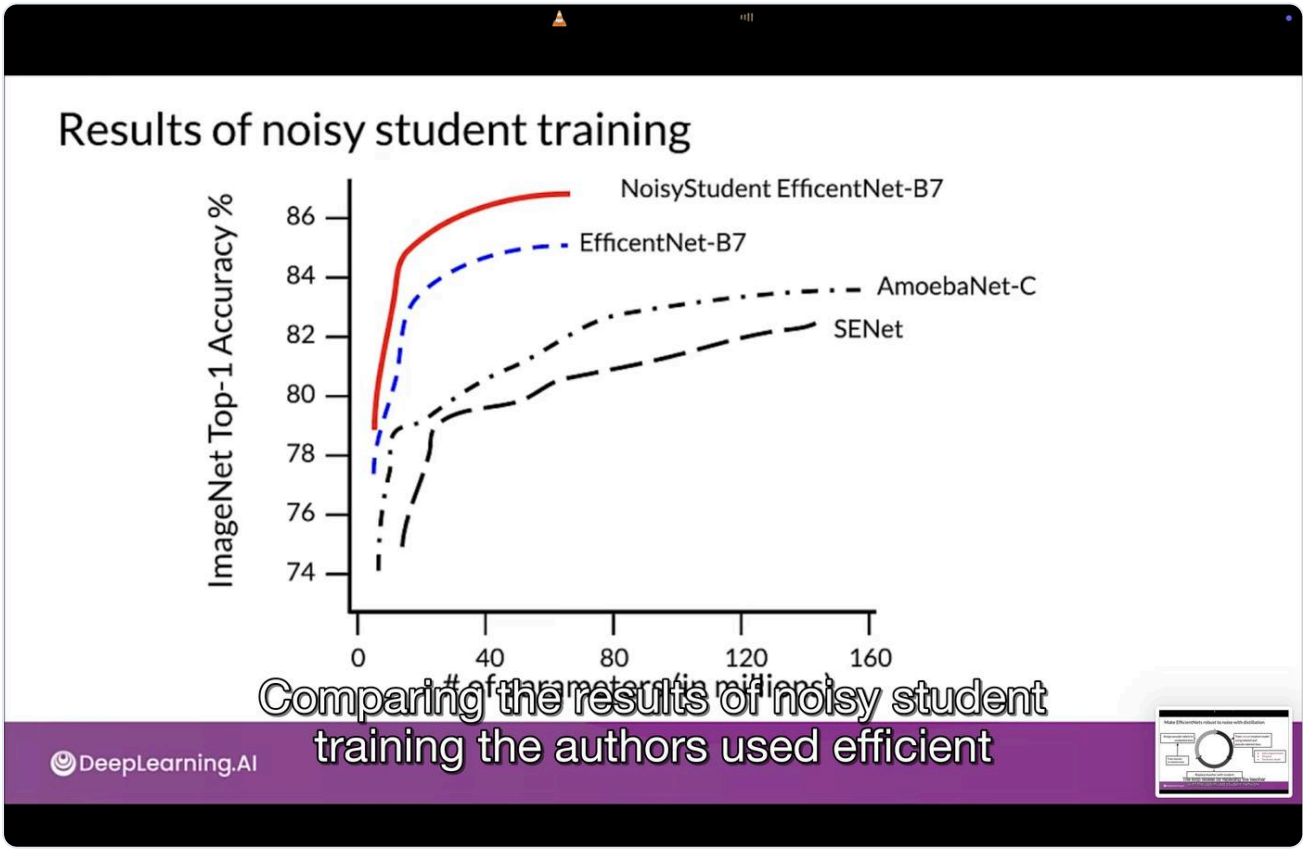

FIGURE 28.7 | Results of Noisy Student Training

Comparing the results of noisy student training the authors used efficient nets as their baseline models. This graph shows different sizes of efficient models along with some well known state of the art models for comparison. Note the results of the noisy student in red. One key factor is that the data sets were balanced across different classes, which improved training, especially for smaller models.

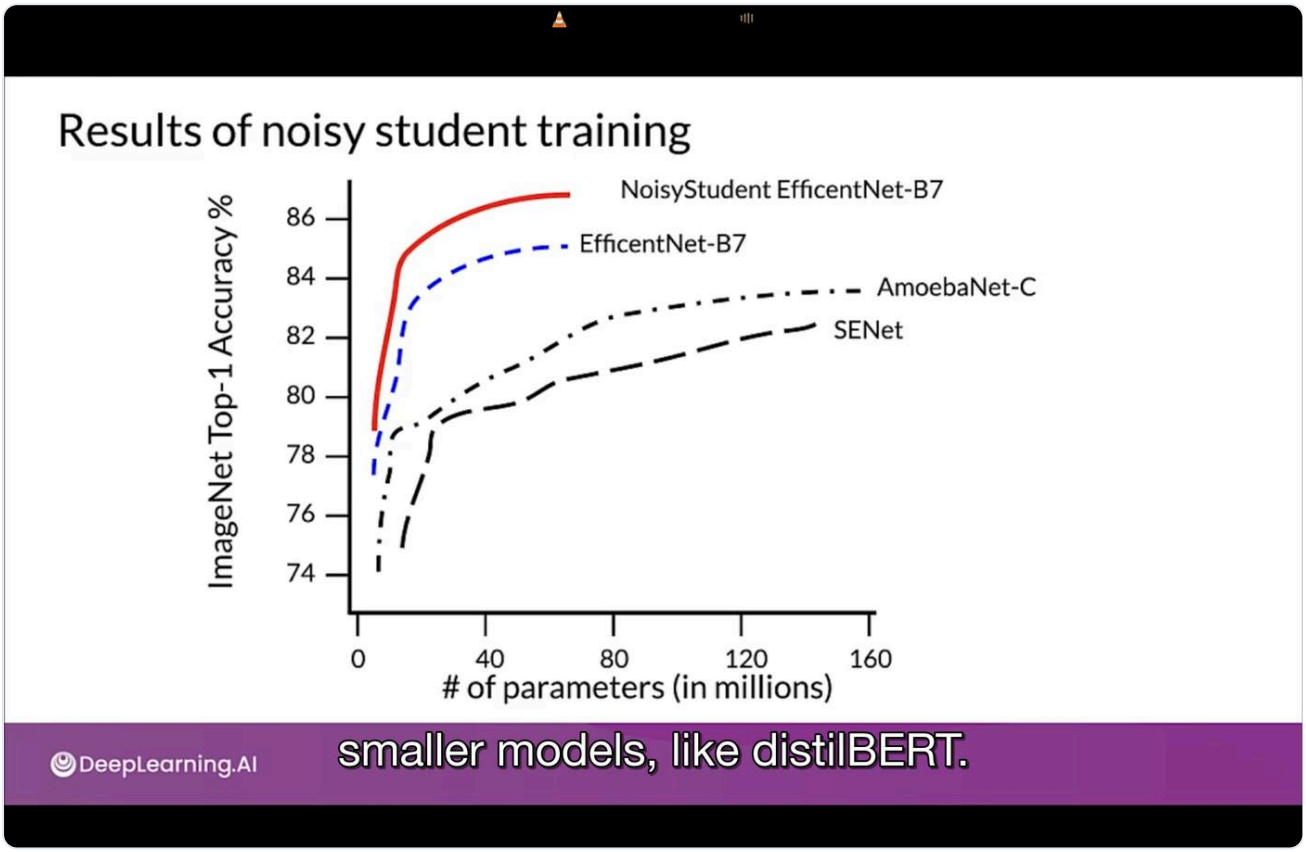

FIGURE 28.8 | Results of Noisy Student Training

These results show that knowledge distillation isn't just limited to smaller models, like distilBERT. But can also be used to increase the robustness of an already great model using noisy student training. Well, we covered a lot, didn't we first, we started with a discussion of distributed training. And then we turn to high performance modeling, including high performance ingestion, which turns out to be very important in high performance modeling. And then we turn to one of my favorite topics knowledge distillation. So I enjoyed it. I hope you did. And I'll see you next time.

LECTURE 29

Performance Metrics vs Optimization Objectives

Welcome back. We've got a big week planned. We're going to start with a deep look at model performance analysis. Including diving into a key tool, TensorFlow model analysis or TFMA. Then we'll take a look at model debugging and understanding model robustness. Which are important, especially understanding sensitivity analysis and how adversarial attacks, work with robustness and sensitivity. Then we'll look at some ways to do model remediation as well. Then we'll be talking about fairness and how important fairness is, and ways to deal with it. Finally, we'll take a look at continuous evaluation and monitoring, and understanding when to retrain your model.

It's a big week. Let's get started. Successfully training a model and getting it to converge feels good. It often feels like you're done. If you're training a model for a class project or a paper that you're writing, then you are done. But for production ML, you now need to enter into a new phase of your development which involves a much deeper level of analysis of your model's performance. From a few different directions, and that's what we'll be learning about this week. Let's start with a little bit of review of model analysis. Looking at some of the different ways of analyzing models and determining performance. In much of this week, you'll be using the tools of TensorFlow model analysis or TFMA, and some related tools and technologies, but first, let's review some of the basics of model analysis.

Here it's important to emphasize that you want to look at your model performance, not just on your entire dataset, but on individual slices of your data. After training and or deployment, you might notice a decay in performance. It's natural to inquire about possible ways to improve model performance.

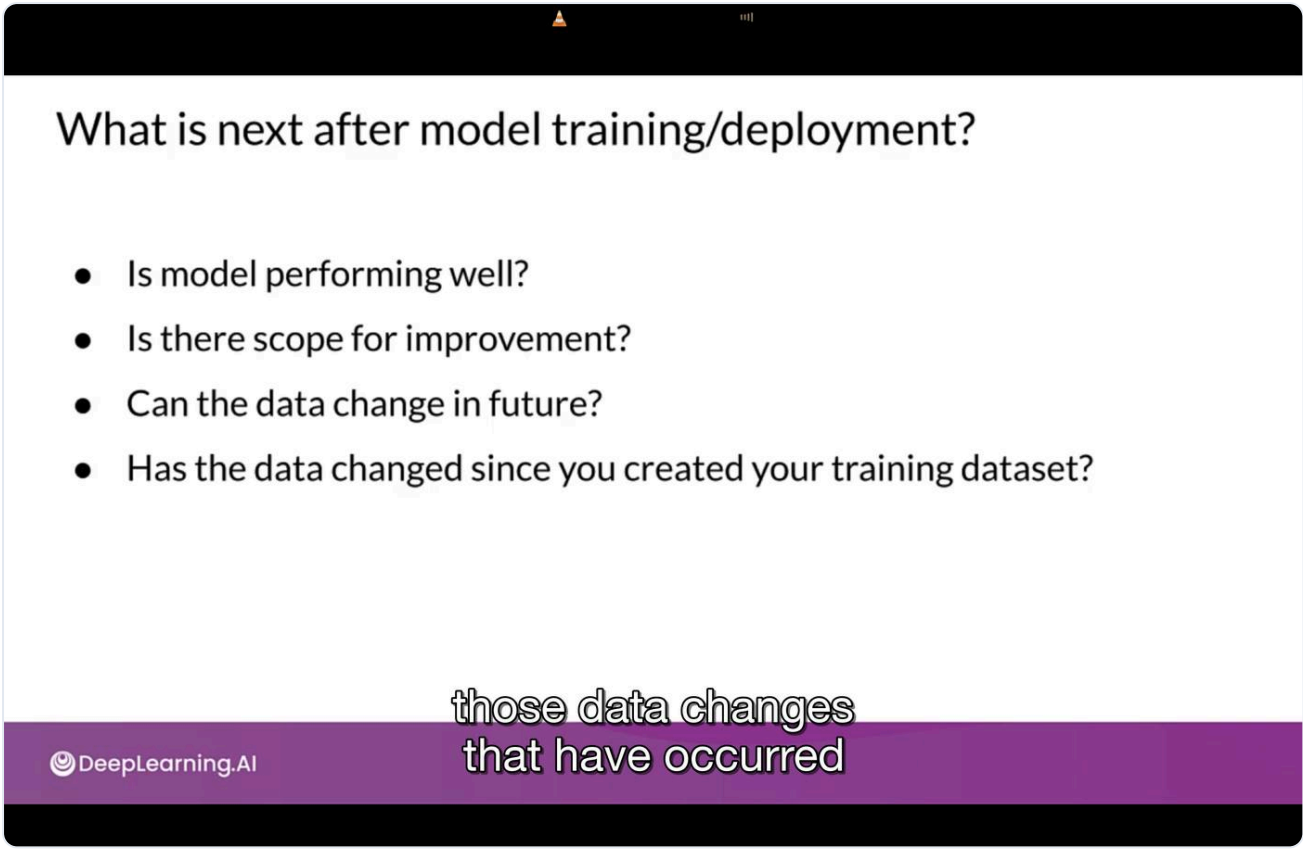

FIGURE 29.1 | What Is Next After Model Training/deployment?

Additionally, you need to anticipate data changes that you might expect to see in the future and act on those data changes that have occurred since you originally trained your model. Choosing the slices that are important to analyze is usually based on domain knowledge. This will allow you to determine, if there is room for improvement in your model, and perhaps address data changes that have happened since you originally trained your model. For example, if your model is designed to predict demand for different kinds of shoes. Looking at the performance of your model on individual types of shoes, perhaps different colors or styles. That'll be important.

That will require slicing your data for different types of shoes. At a high level, there are two main ways to analyze the performance of your model. Black-box evaluation and model introspection. In black-box evaluation, you generally don't consider the internal structure of the model. You're just interested in quantifying the performance of the model through metrics and losses. This is often sufficient within the normal course of development. But if you're interested in how the model is working internally, maybe looking for ways to improve it. You can apply various model introspection methods.

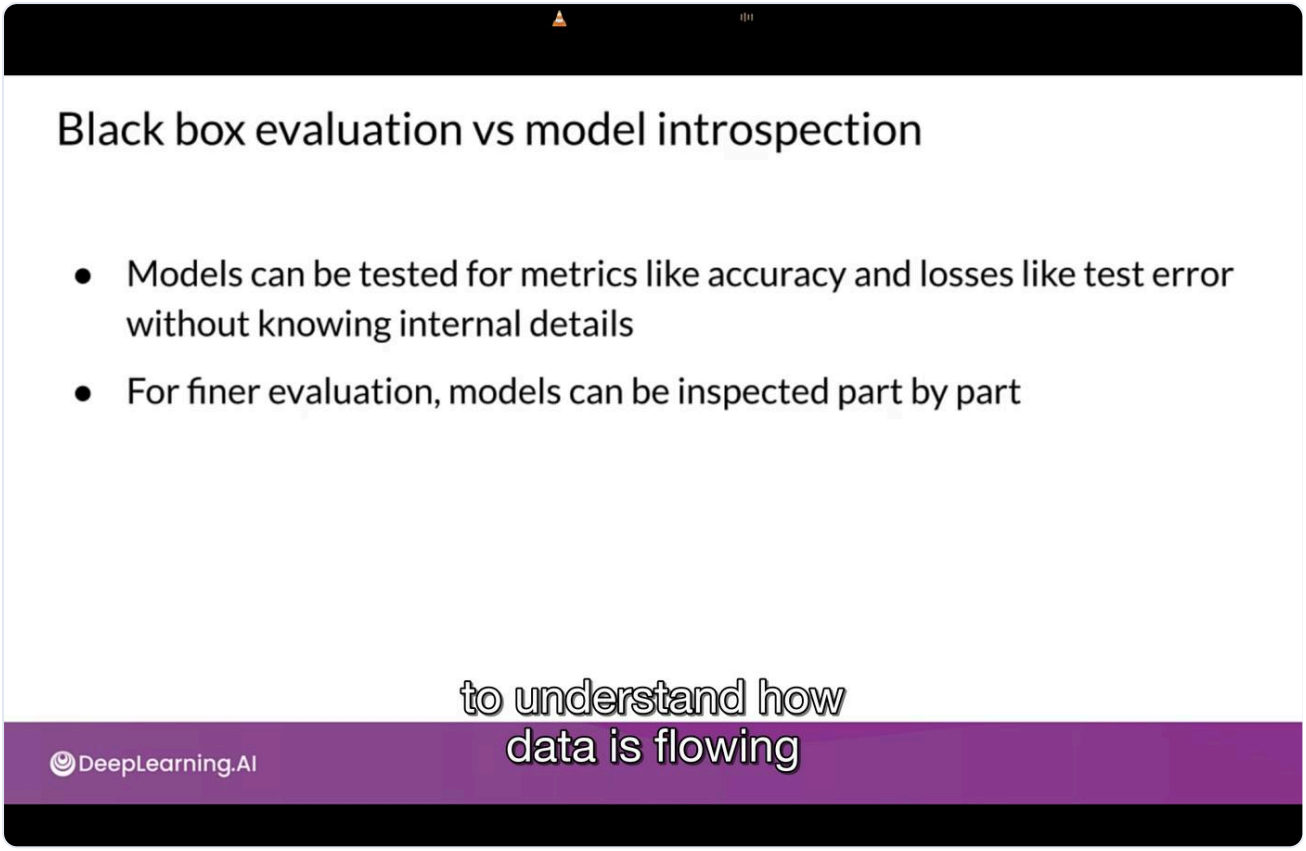

FIGURE 29.2 | Black Box Evaluation vs Model Introspection

Model introspection methods are very useful when you're experimenting with new architectures, to understand how data is flowing internally within each layer of the model. This can help you adjust and iterate your model architecture to improve performance and efficiency.

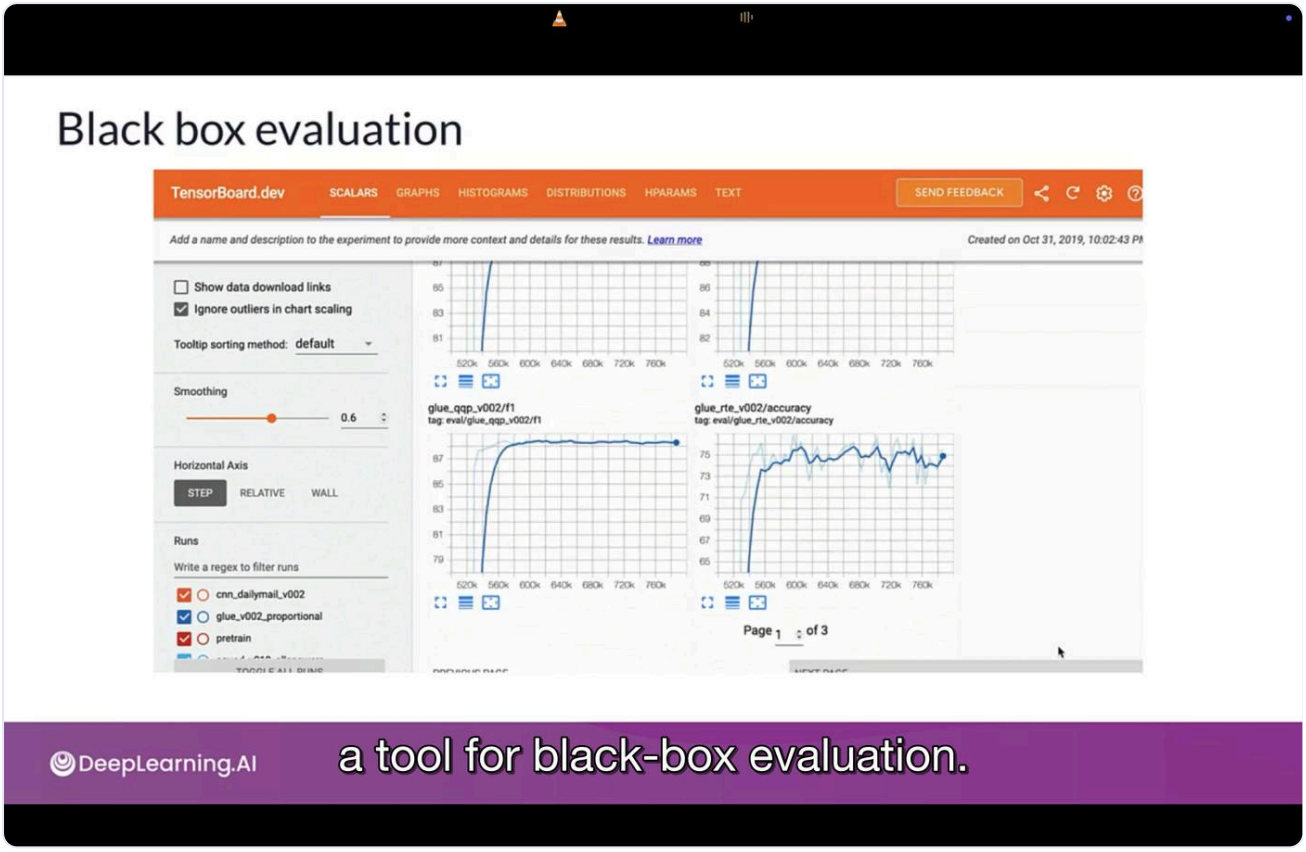

FIGURE 29.3

TensorBoard is an example of a tool for black-box evaluation. Using TensorBoard, we can monitor the loss and accuracy of every iteration of the model. You can also closely monitor the training process itself. Please check the reading list to learn more about TensorBoard. In model introspection, the goals change completely. You're not just interested in the model's final results. You're also interested in the details of each layer. On the left side, we have the maximally activated patches of the various filters in the convolutional layers of the model - this is a CNN - when we run through a series of images belonging to a particular class. Using those patterns, you can inspect at which layer the model is learning a particular structure of your data.

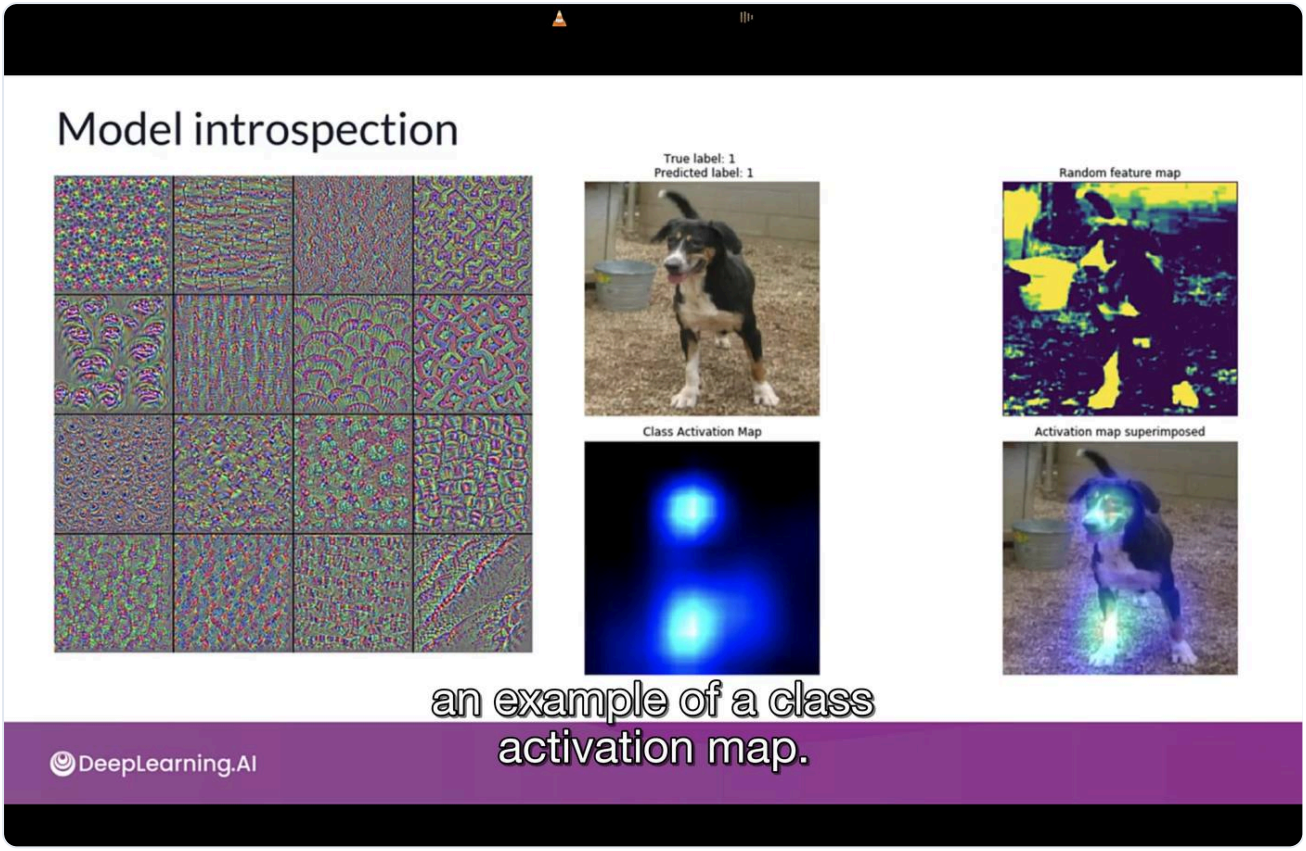

FIGURE 29.4 | Model Introspection

On the right side is an example of a class activation map. Here, you're interested in knowing which parts of the image are primarily responsible for making the desired prediction of that class. Using this information, you can try to improve the performance of your model by tuning or including more features related to those highlighted areas. Next, let's look at the difference between performance metrics and optimization. First, performance metrics. Based on the problem you're trying to solve, you need to quantify the success of your model using some measurement, and for this, you use various performance metrics. Performance metrics will be different for different types of tasks like classification and regression and so forth.

You're familiar with these from designing and training models. Now let's focus on the optimization part.

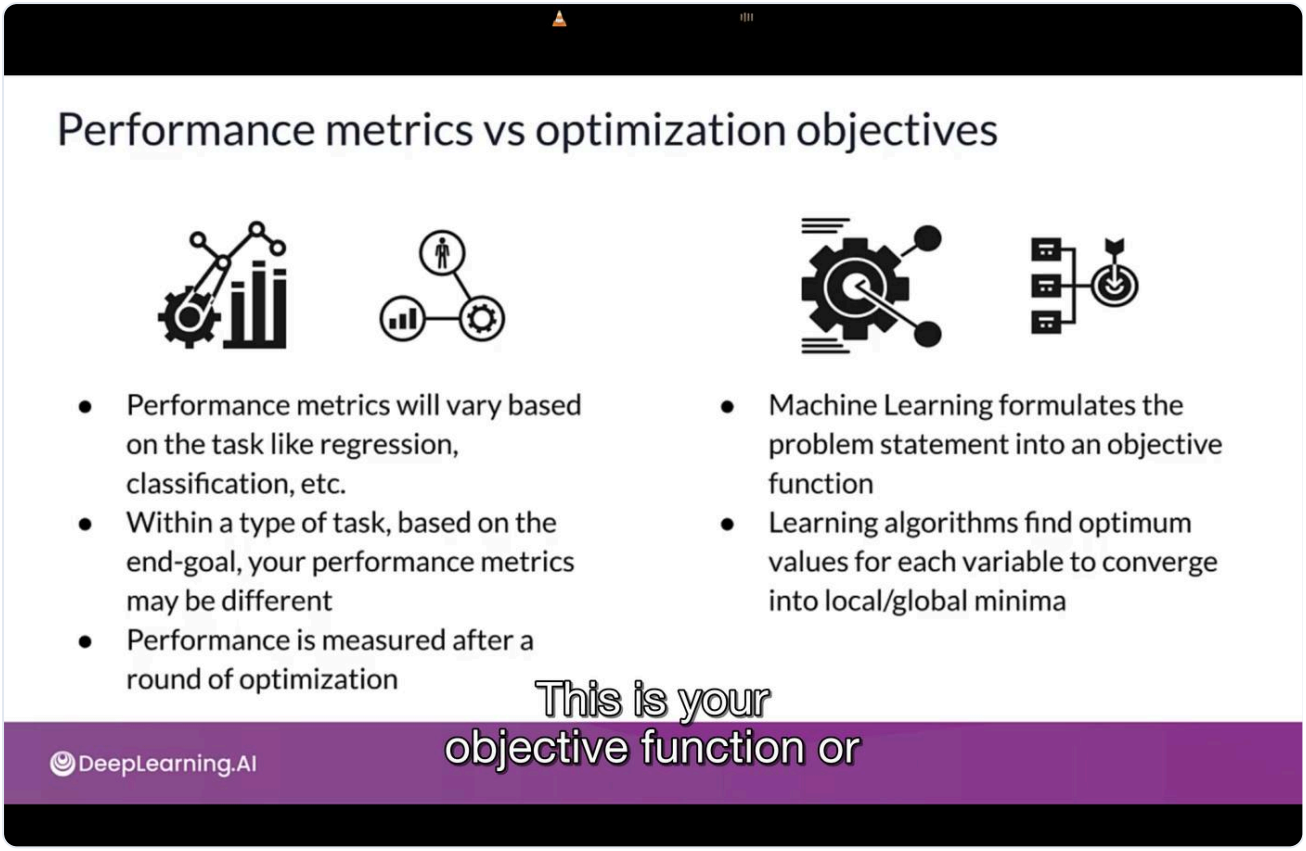

FIGURE 29.5 | Performance Metrics vs Optimization Objectives

This is your objective function or cost function or loss function. People use different names for it. When you train your model, you try to minimize the value of this function in order to find an optima, hopefully, a global optima in your loss surface. If you check out TensorBoard again, you'll notice options for tracking performance metrics like accuracy and optimization objectives such as loss after each epic of training and validation.

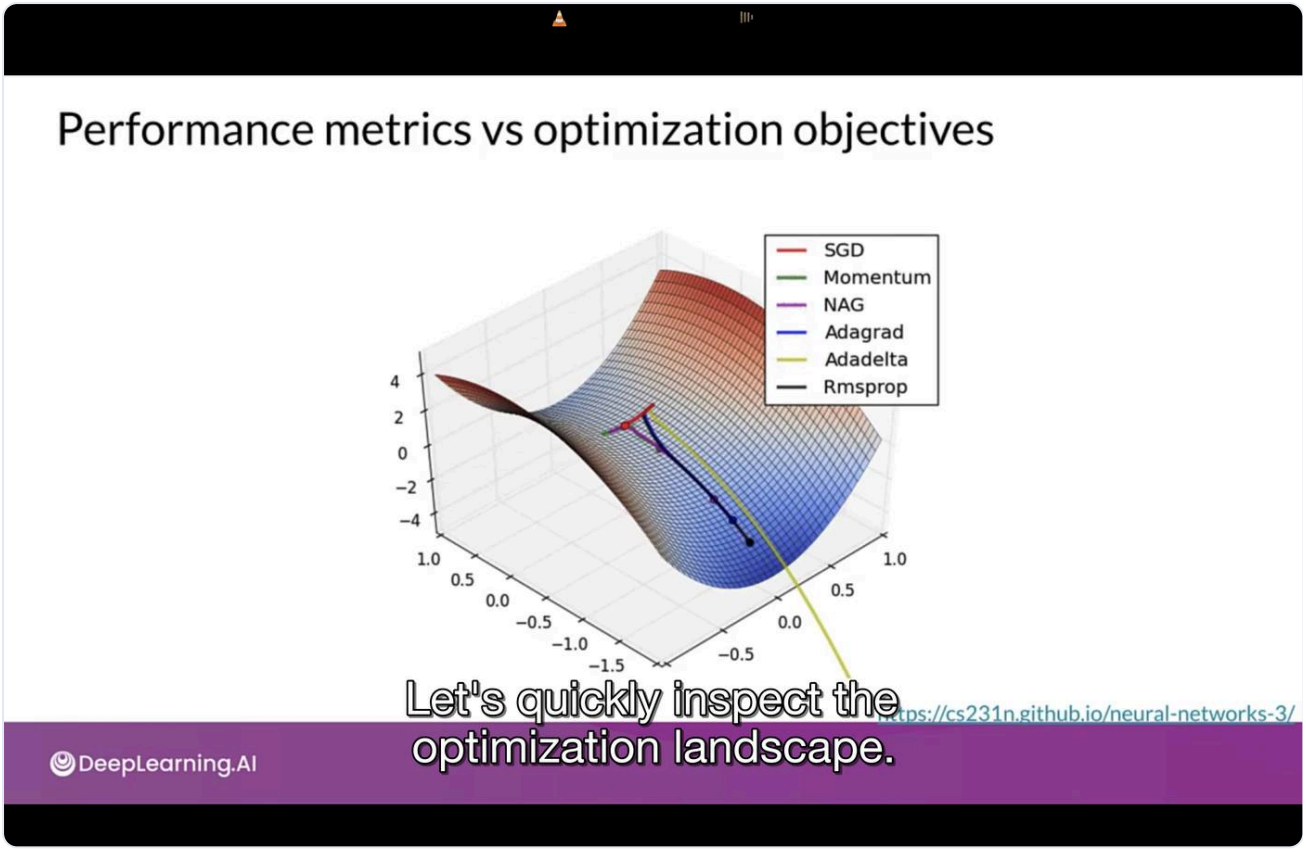

FIGURE 29.6 | Performance Metrics vs Optimization Objectives

Let's quickly inspect the optimization landscape. This is also referred to as a loss surface. Here, the losses represented as a function of two parameters. Depicted here is a saddle point, where the curvature along different dimensions has different signs. One dimension curves up and another down. This is typical of many optimization problems. The animation shows trajectories for different optimization algorithms. Each of these optimizers traverses the landscape differently, according to their update method, and may take different amounts of time to converge to an optimum value.

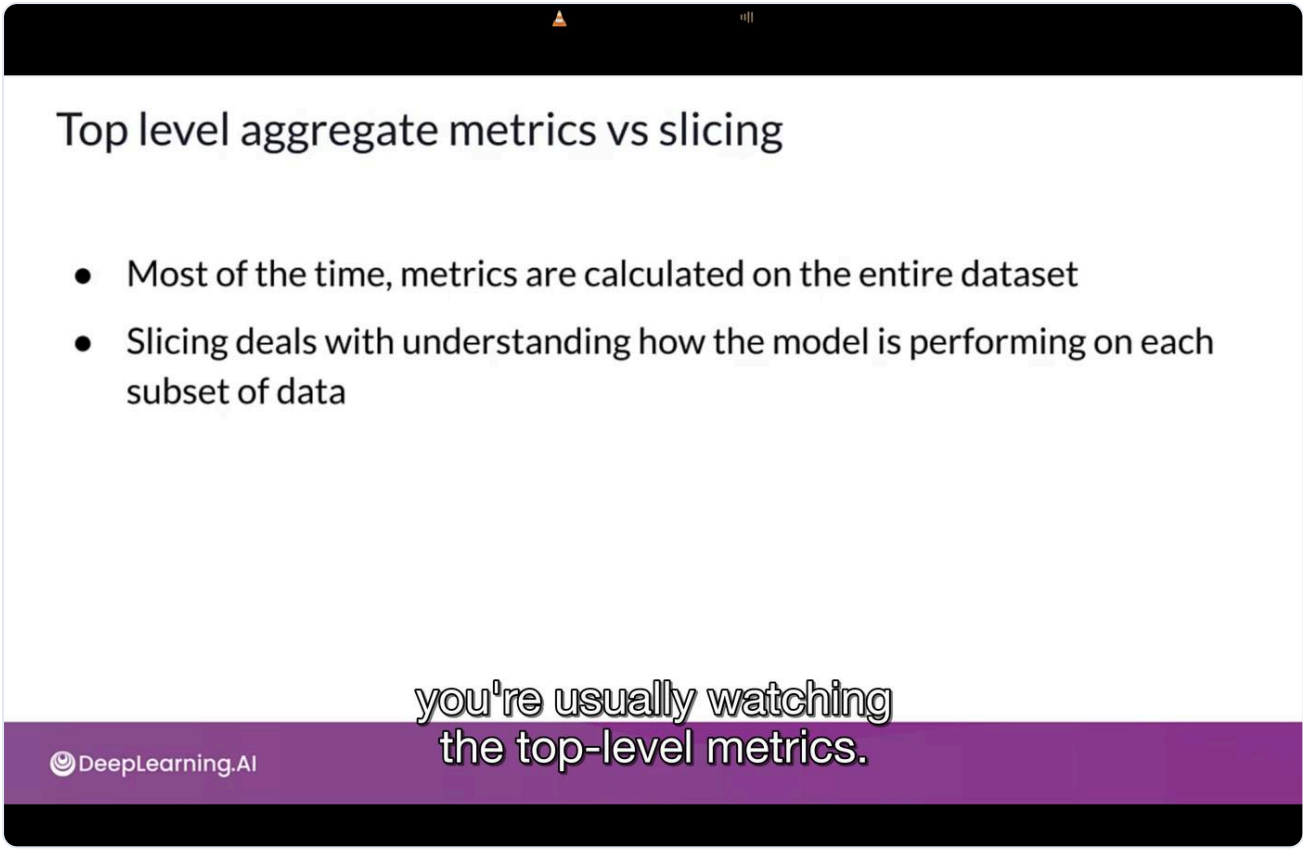

FIGURE 29.7 | Top Level Aggregate Metrics vs Slicing

When you're evaluating your training performance, you're usually watching the top-level metrics. This is to decide whether your model is doing well or not. But it doesn't tell you about how well it does, on individual parts of the data. For example, different customers in different stores may experience your model very differently. If their experience isn't good, it reflects badly on your model.

LECTURE 30

One Model vs Multiple Models Over Time

TensorFlow Model Analysis, or TFMA is a versatile tool for doing deep analysis of your model's performance. Let's discuss that now. Your top-level metrics can easily hide problems with particular parts of your data.

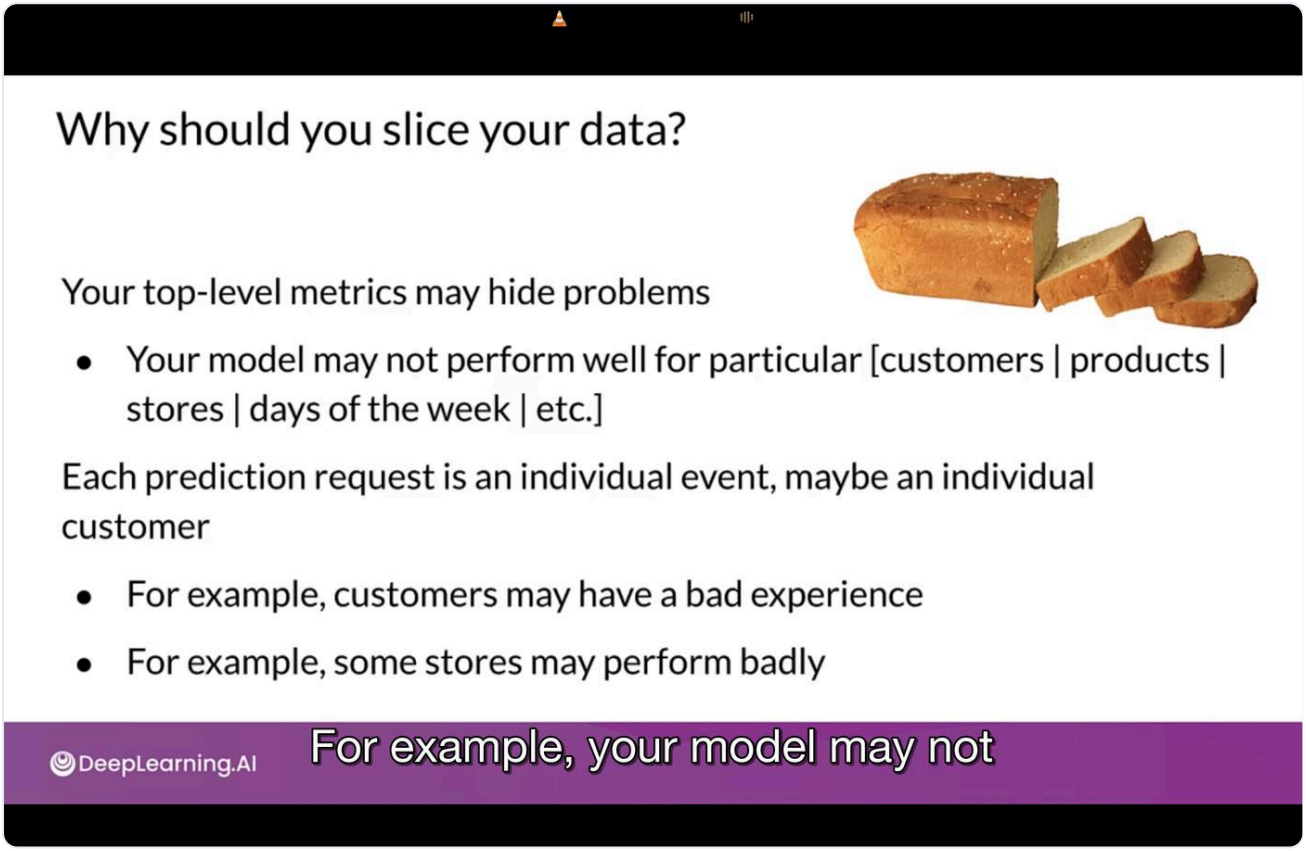

FIGURE 30.1 | Why Should You Slice Your Data?

For example, your model may not perform well for particular customers, or particular products or stores or days of the week, or different subsets of your data that makes sense for your domain or problem. Thinking about customers who are requesting a prediction from your model. If you're model produces a bad prediction, then that customer's experience of your model is bad. Regardless of how well it may do in top-level metrics. To enable developers to look at their performance of their model at a deeper level Google created TensorFlow Model Analysis or TFMA. TFMA is an open source, scalable framework for doing deep analysis of model performance, including analyzing performance on slices of data.

TFMA is also a key part of a machine-learning pipelines, such as a TFX pipeline, to perform deep analysis before you deploy a newly trained version of your model.

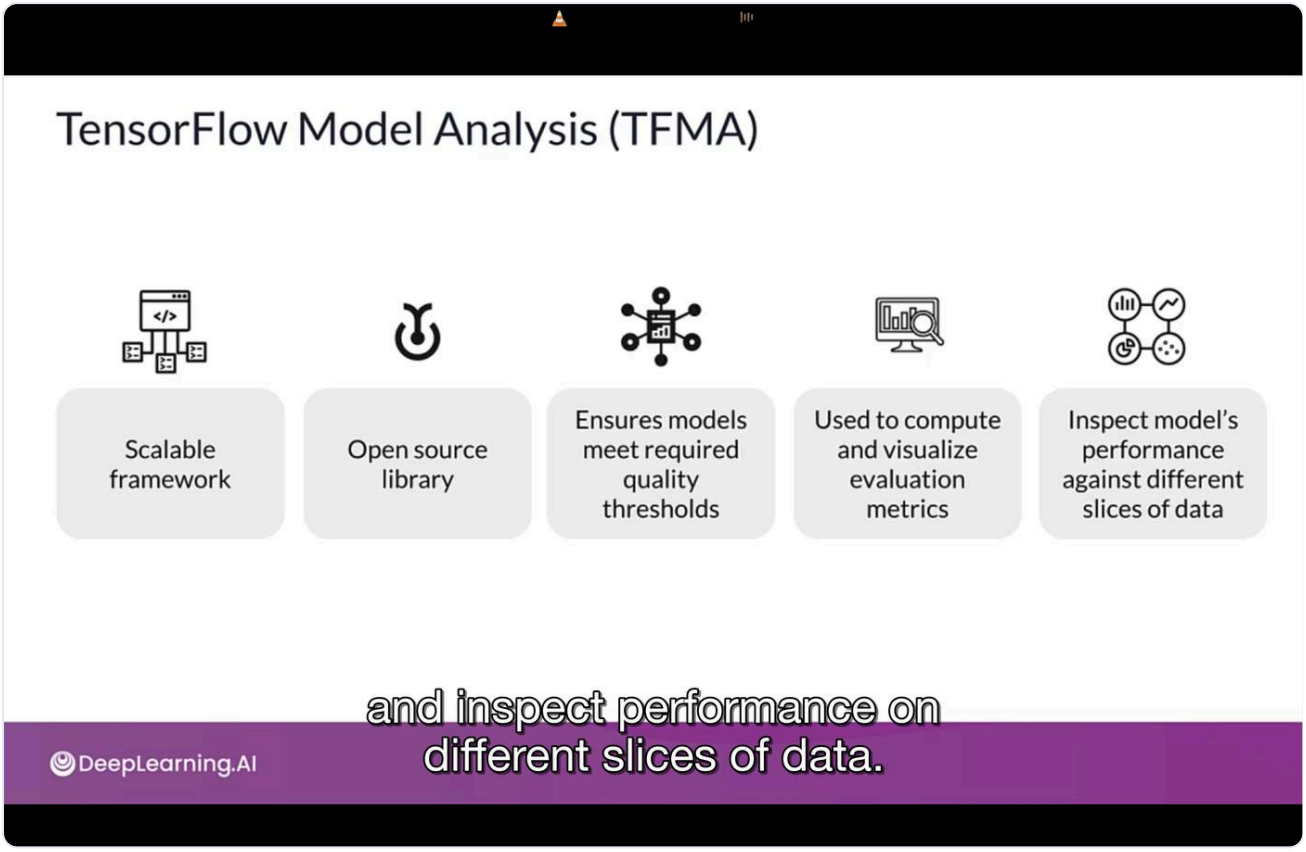

FIGURE 30.2 | TensorFlow Model Analysis (TFMA)

TFMA has built-in capabilities to check that your models meet certain quality standards and to visualize evaluation metrics and inspect performance on different slices of data. TFMA can be used by itself or as part of another framework such as TFX. Let's take a look at the high level architecture of TFMA. The TFMA pipeline is made up of four main components. Read inputs, extractors, evaluators, and write results. The first stage read input is made up of a transform that takes raw inputs such as CSV or TF records and so forth and it converts it into a dictionary format which is understandable by the next stage, extraction. Across all the stages, the output is kept in this dictionary format, which is at the datatype tfma.extracts.

The next stage, extraction performs distributed processing using Apache Beam.

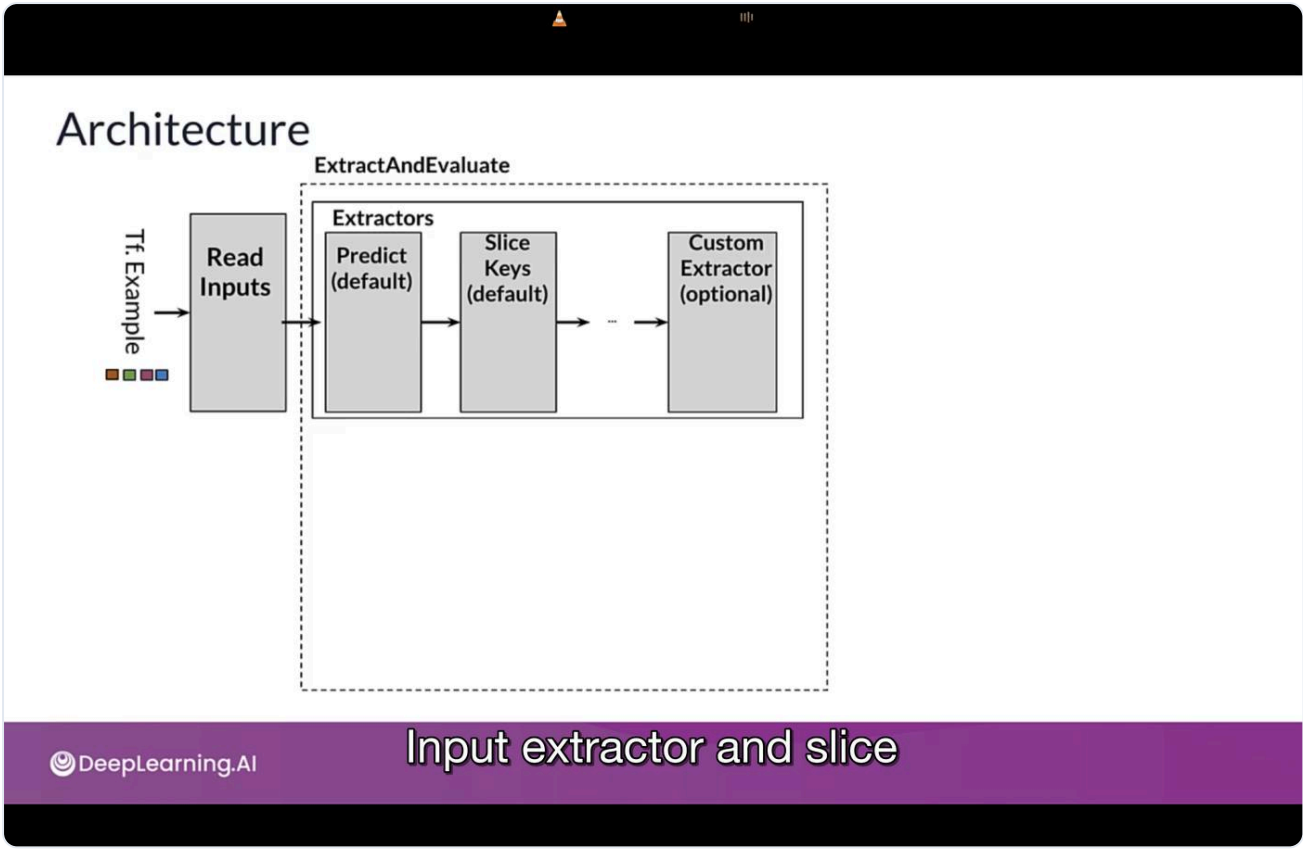

FIGURE 30.3

Input extractor and slice key extractor for slices of the original dataset, which will be used by predict extractor to run predictions on each slice. The results are sent to the next stage, again, as a tfma.extracts dictionary.

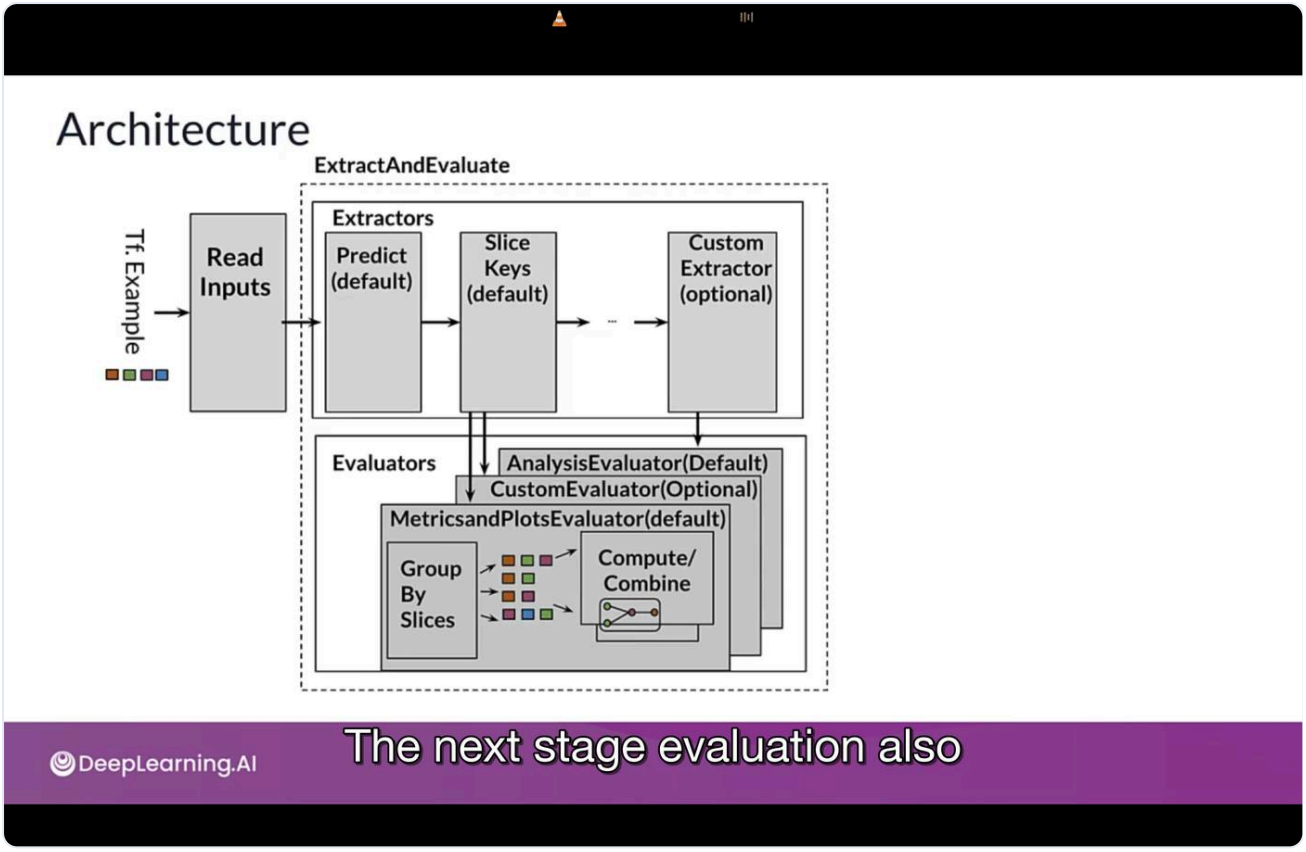

FIGURE 30.4

The next stage evaluation also performs distributed processing using Apache Beam. There are several evaluators and you can create custom evaluators as well. For example, the metrics and plots evaluator extracts the required fields from the data to evaluate the performance of the model against the received predictions from the previous stage.

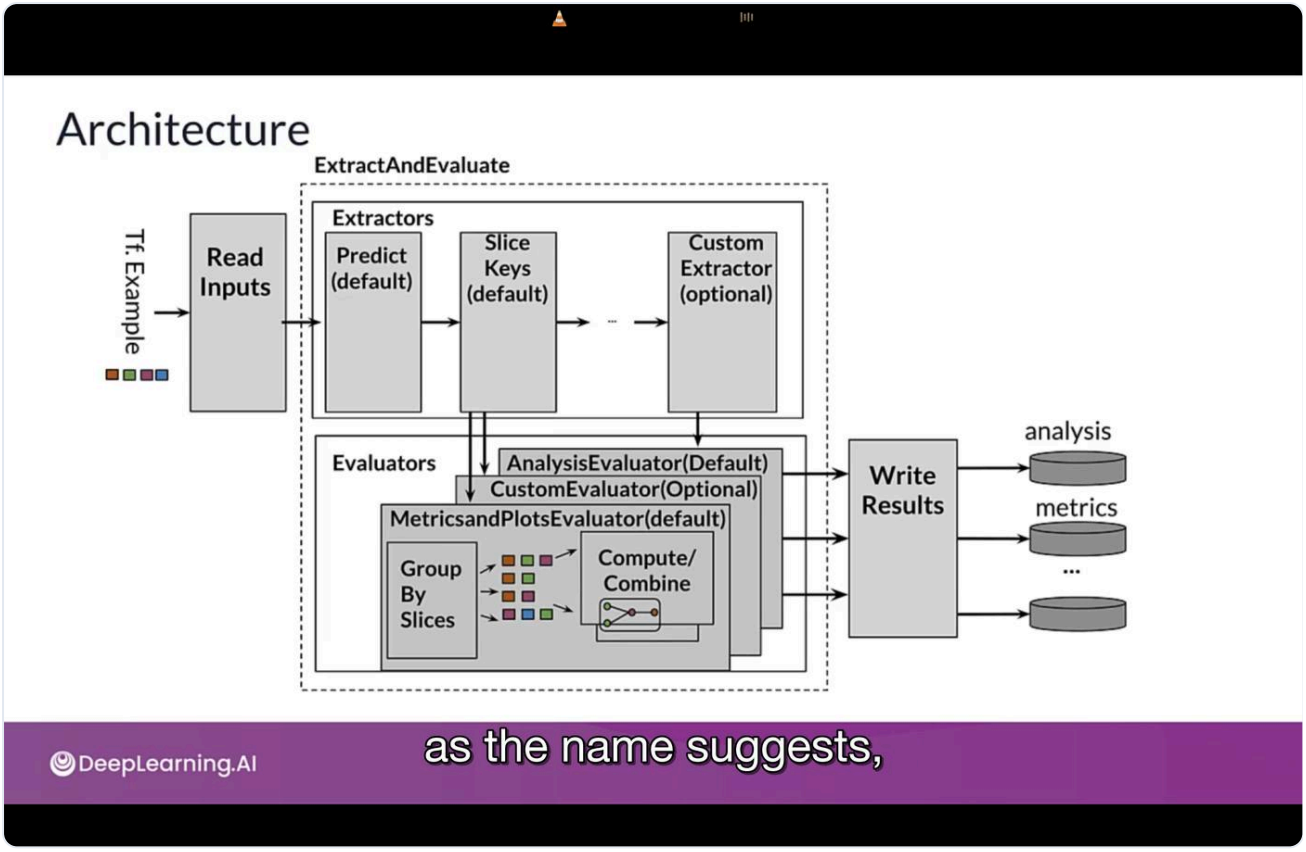

FIGURE 30.5

The final stage, write results, as the name suggests, writes the results to disk. TensorBoard and TFMA are used in different stages of the development process. At a high level, TensorBoard is used to analyze the training process itself while TFMA a is used to do deep analysis of the finished trained model. TensorBoard is used to inspect the training process of a single model often as you're monitoring your progress during training.

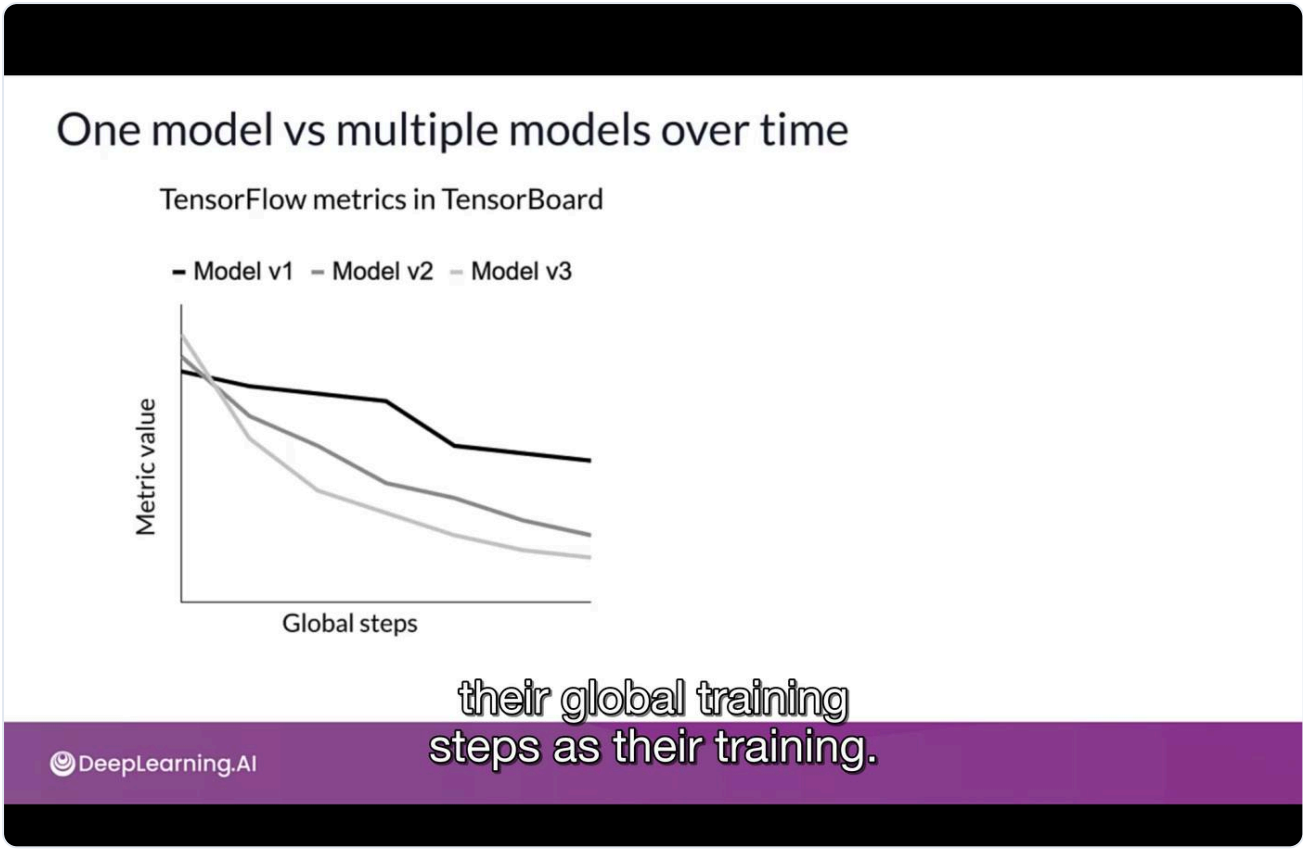

FIGURE 30.6 | One Model vs Multiple Models Over Time

It can also be used to visualize the training progress for more than one model with performance for each plotted against their global training steps as their training.

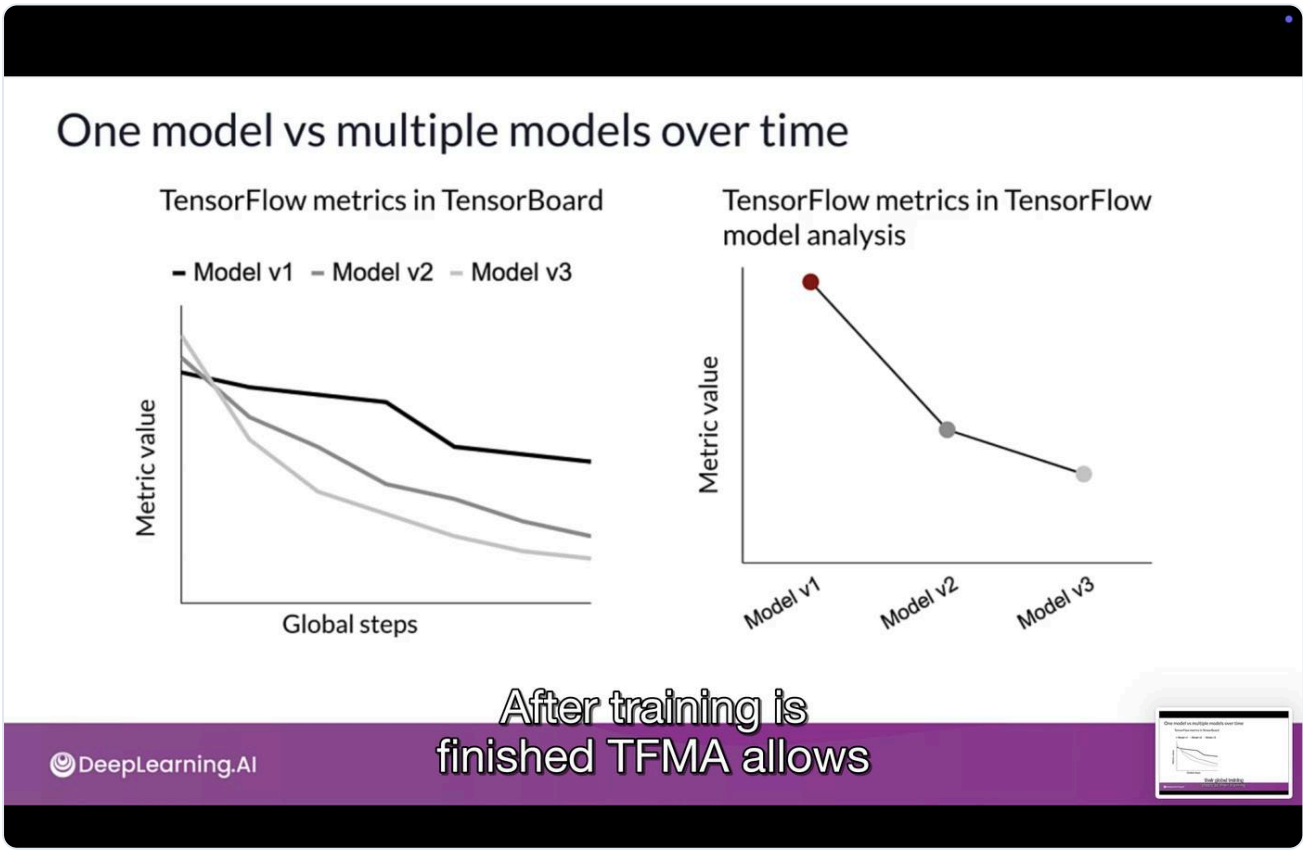

FIGURE 30.7 | One Model vs Multiple Models Over Time

After training is finished TFMA allows developers to compare different versions of their train models. While, TensorBoard visualizes streaming metrics of multiple models over global training sets TFMA visualizes the metrics computed for a single model over multiple versions of the exported saved model. Most model evaluation results look at aggregate or a top-level metrics on the entire training dataset. This aggregation often hides problems with model performance. For example, a model may have an acceptable AUC over the entire EVAL dataset, but under performs on specific slices. In general, a model with good performance on average may exhibit failure modes that are not apparent by looking at an aggregate metric.

Slicing metrics allow you to analyze the performance of a model on a more granular level.

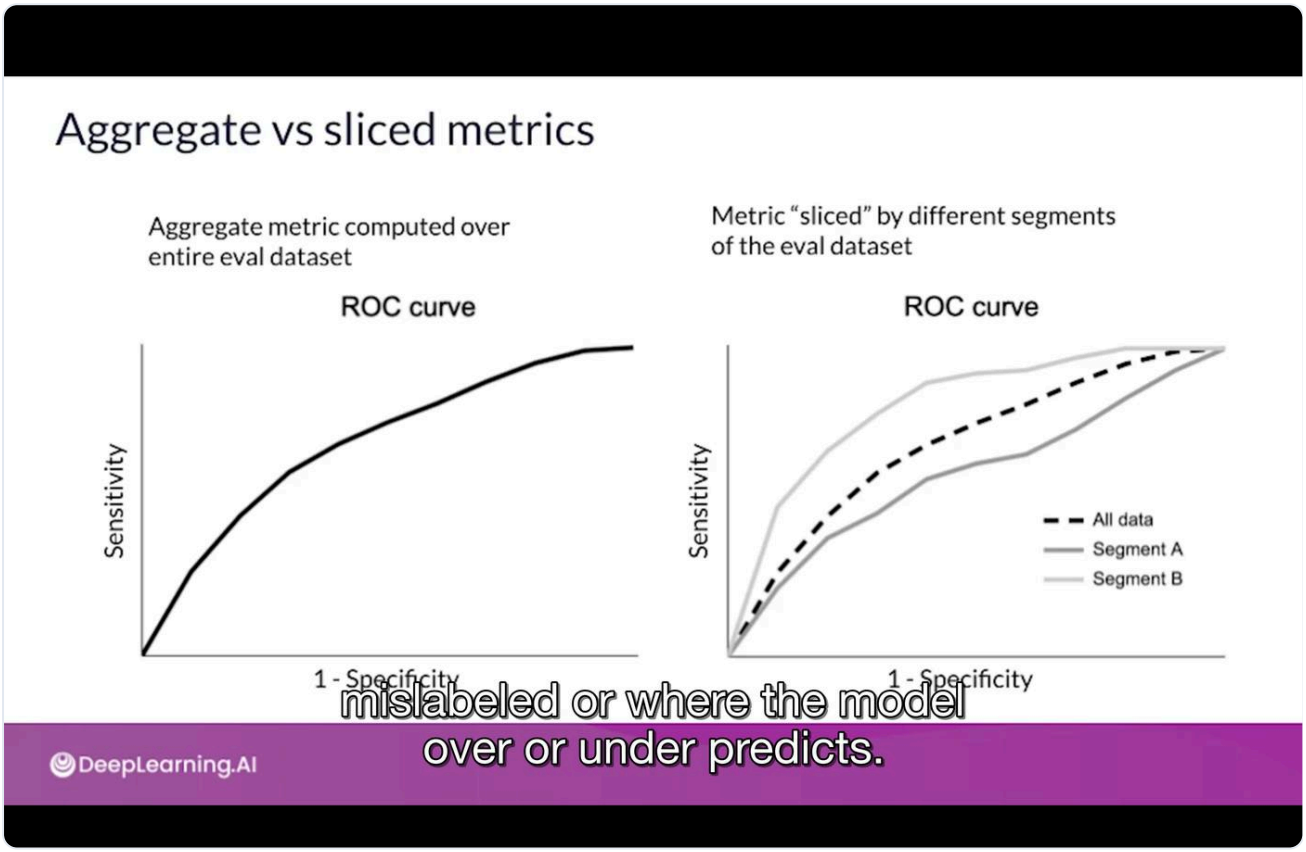

FIGURE 30.8 | Aggregate vs Sliced Metrics

This functionality enables developers to identify slices were examples may be mislabeled or where the model over or under predicts. For example, TFMA could be used to analyze whether a model that predicts the generosity of a taxi tip works equally well for riders that take the taxi during the day versus at night by slicing the data by the hour. In general, determining which slices are important to analyze requires domain knowledge about your data and application. TensorBoard confuse metrics on a mini-batch basis during training.

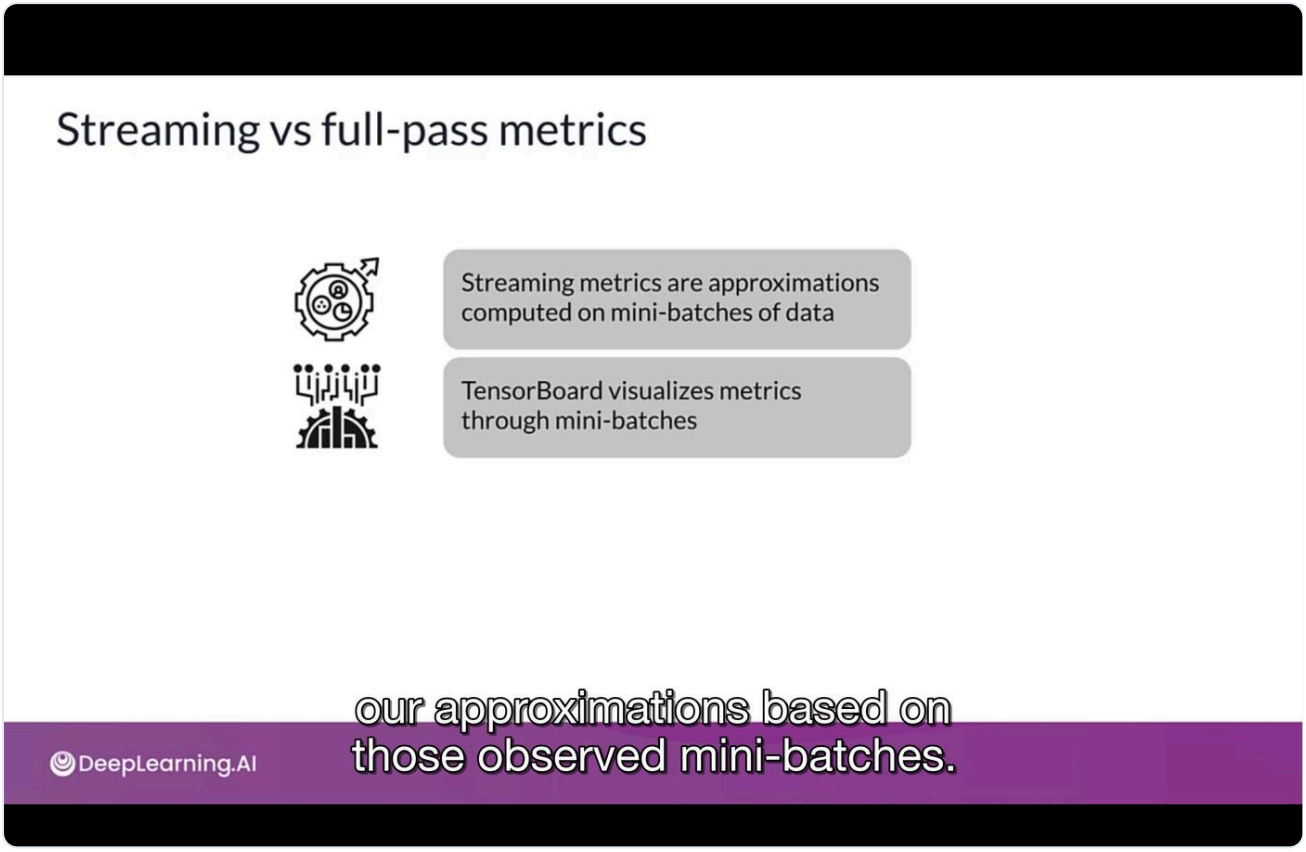

FIGURE 30.9 | Streaming vs Full-pass Metrics

They're called Streaming metrics and are our approximations based on those observed mini-batches.

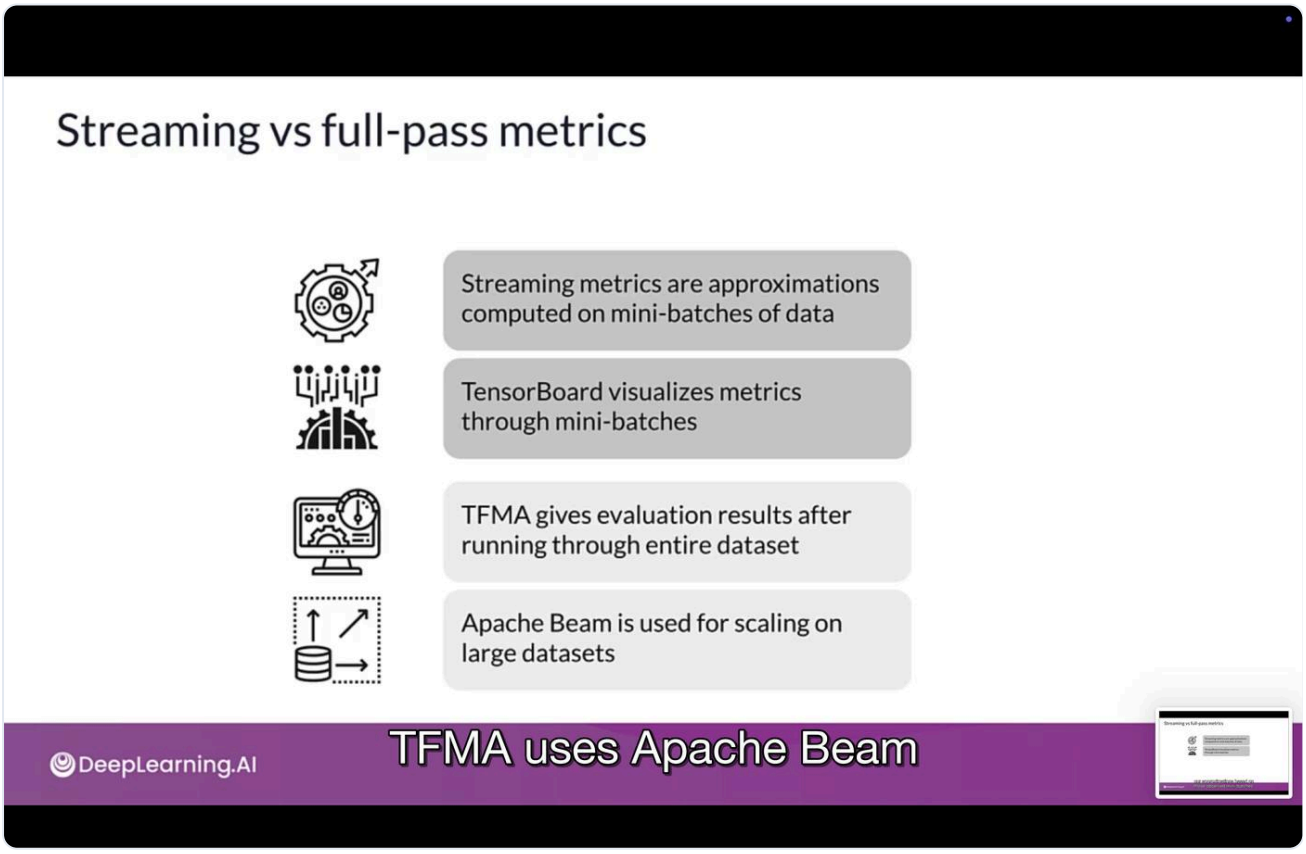

FIGURE 30.10 | Streaming vs Full-pass Metrics

TFMA uses Apache Beam to do a full pass over the evil dataset. This not only allows for accurate calculation of metrics, but also scales up to massive evaluation dataset since Beam pipelines can be run using distributed processing back ends. Note that TFMA may computes the same tensor flow metrics that are computed by the TensorFlow EVAL worker just more accurately by doing a full pass over the specified dataset. TFMA can also be configured to compute additional metrics that were not identified or not defined in the model. Furthermore, if evaluation datasets are slice to compute metrics for specific segments, each of those segments may only contain a small number of examples.

To accurately compute metrics, a deterministic full pass over those examples is important.

LECTURE 31

EVAL Config and TFMA

Let's take a quick look at how to set up and use TFMA to perform model analysis. Over the next set of slides, you'll be learning how TFMA helps you evaluate your model performance in different ways. You'll learn how TFMA evaluates your model against different slices of data and how TFMA keeps track of metrics over time.

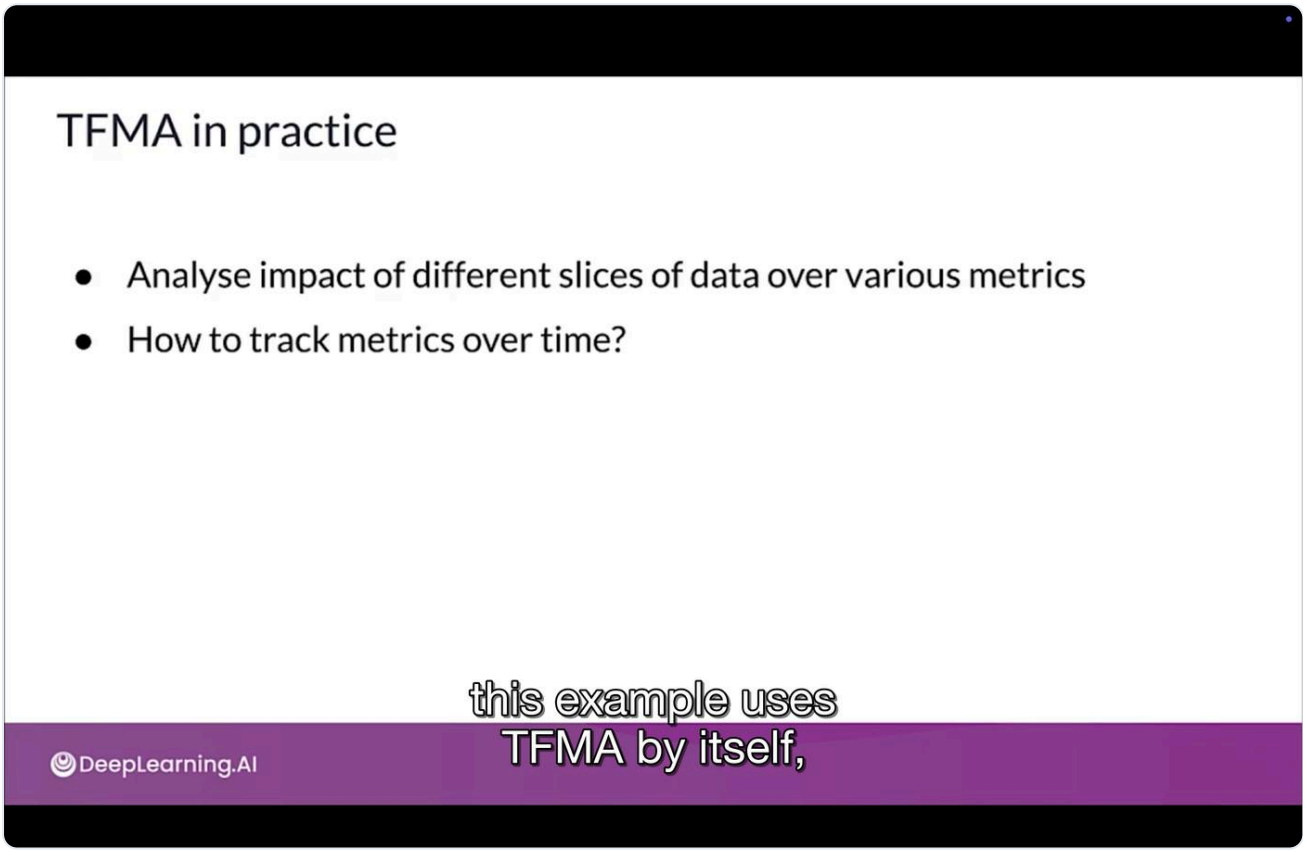

FIGURE 31.1

Before getting started, note that this example uses TFMA by itself, outside of a TFX pipeline. When you use TFMA in a TFX pipeline, the evaluator component already includes many of the steps performed in this example so you won't have to do them yourself. Instead, you would usually only provide the eval_config to the evaluator component, which we'll see later. As usual, let's begin by importing all the necessary libraries. In this case, we need TensorFlow, TensorFlow Transform, and TensorFlow Model Analysis.

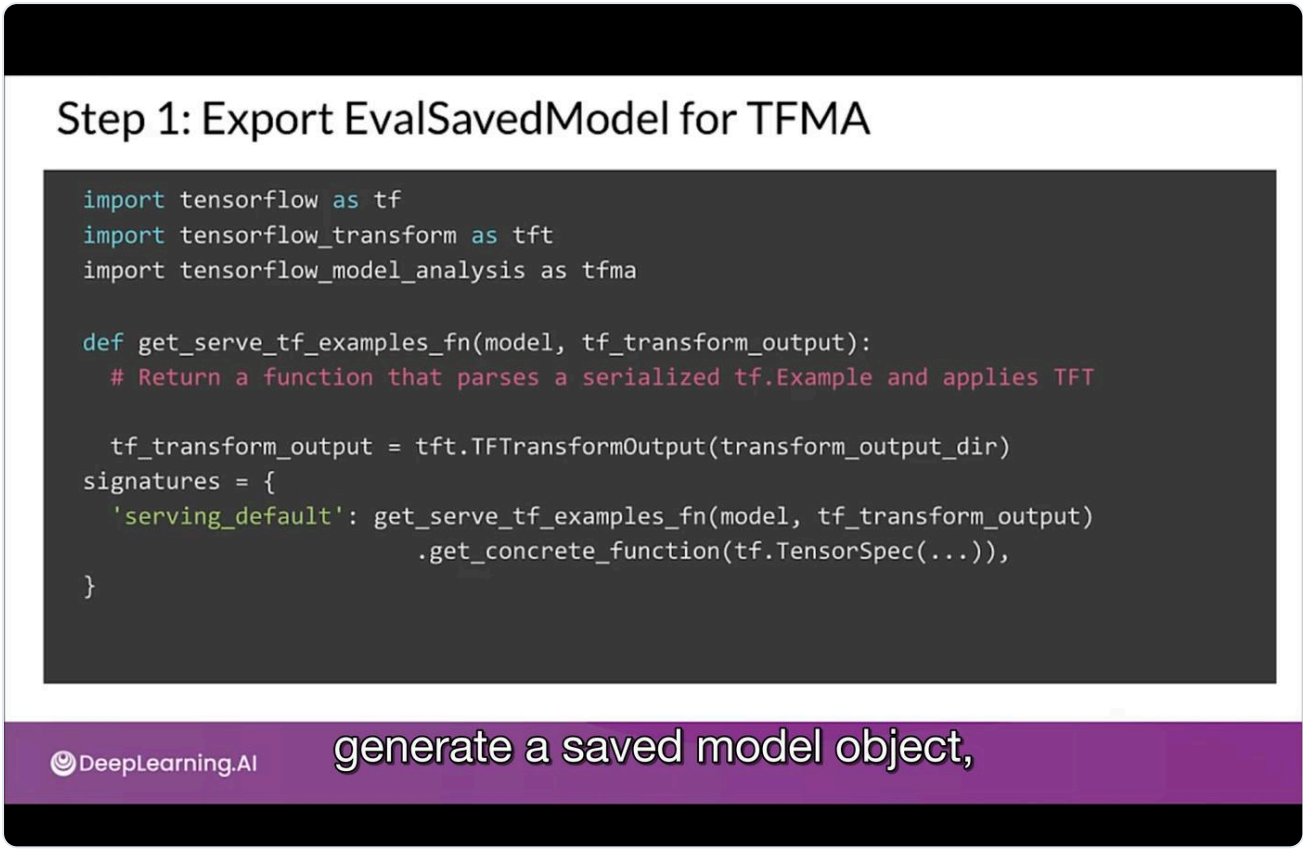

FIGURE 31.2

Next, to use TFMA, the first step is to train your model and generate a saved model object, which you will normally do anyway so that you can serve your model. The raw input examples need to be pre-processed in the same way during training while performing evaluation using TFMA. For example, if a feature was normalized before training, it will also need to be normalized before evaluation. If you train your model in a TFX pipeline and include a transform component, then the same pre-processing and feature engineering done during training will already be included in your saved model. However, if you did not train in TFX, you will need to apply the same pre-processing manually, which is done here in the get_serve_tf_examples function, in this case, using TensorFlow Transform.

Next, we generate a saved model.

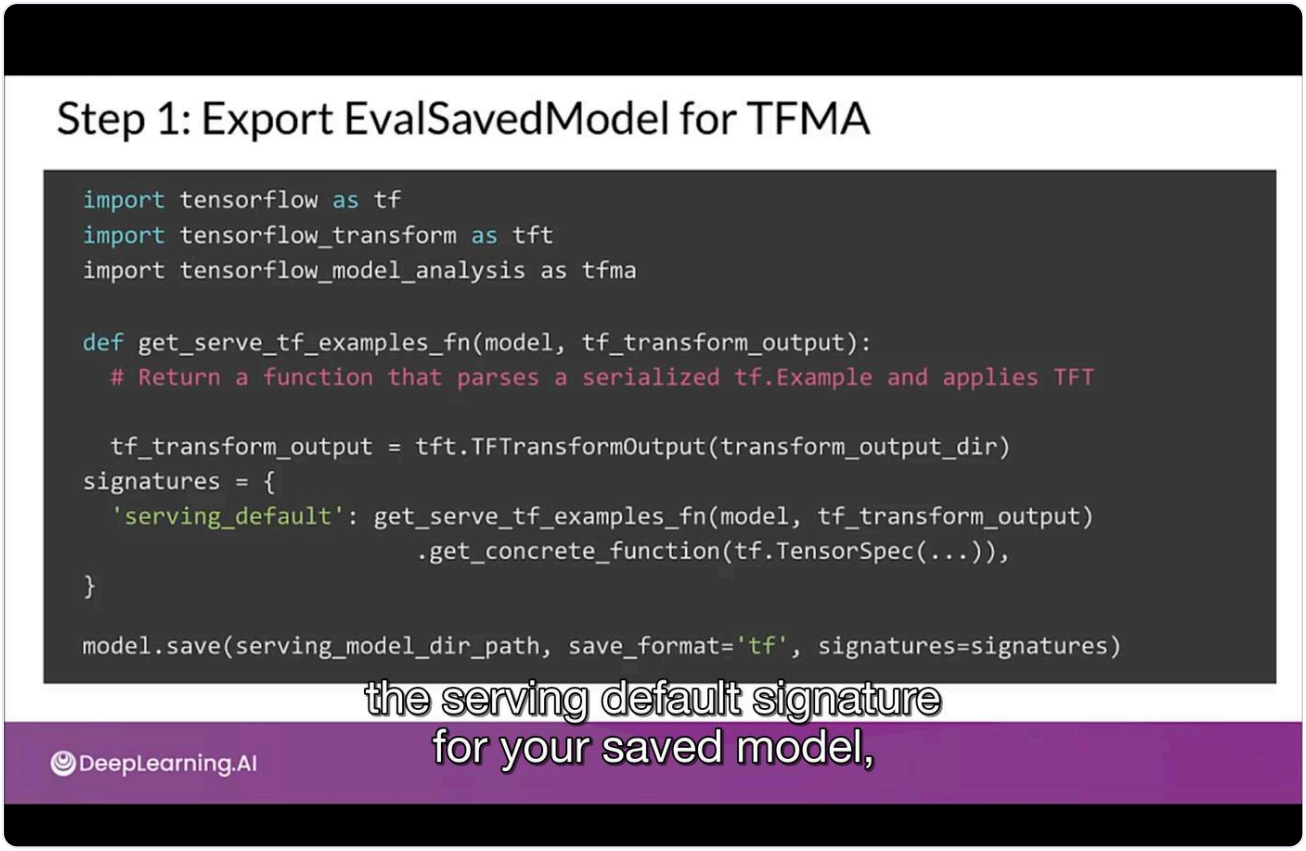

FIGURE 31.3

In most cases, you'll want to use the serving default signature for your saved model, but you can also explicitly specify a different signature for the model in the model specification configuration. The next step involves creating an eval_config object that encapsulates the requirements for TFMA. More concretely, first, you have to define the slices of your dataset and the metrics that you want to use for analyzing your model.

FIGURE 31.4

Then you wrap the model, slices, metrics, and other configurations in an eval_config object. This will determine what TFMA does when it's run. This is the same eval_config that you would give to the evaluator component in a TFX pipeline. With the eval_config object set, you're now ready to start analyzing your model. These examples use only one machine to perform the analysis. Using an orchestrator is not necessary. Here, the code sets the path to an evaluation graph and a directory to store the analysis.

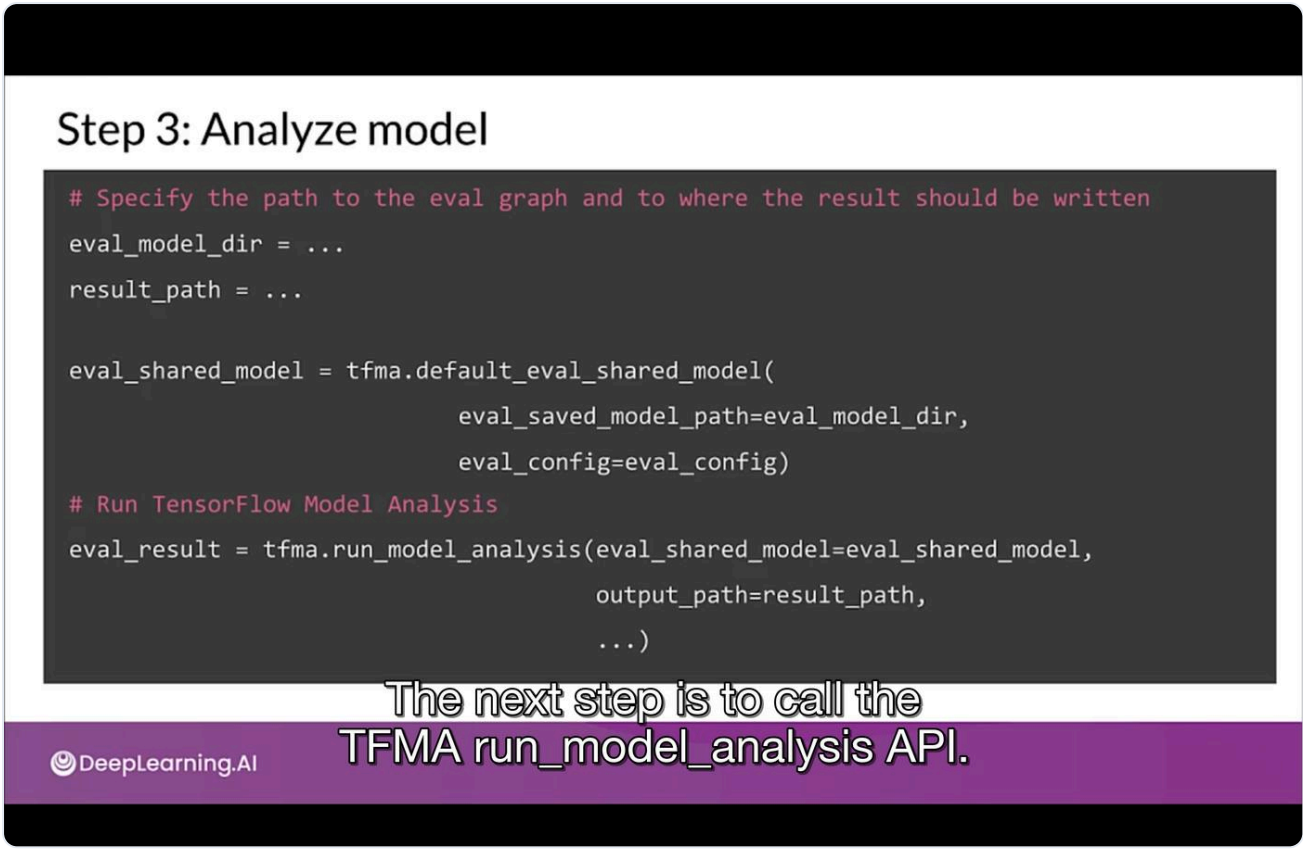

FIGURE 31.5

The next step is to call the TFMA run_model_analysis API. This will get your model analysis started.

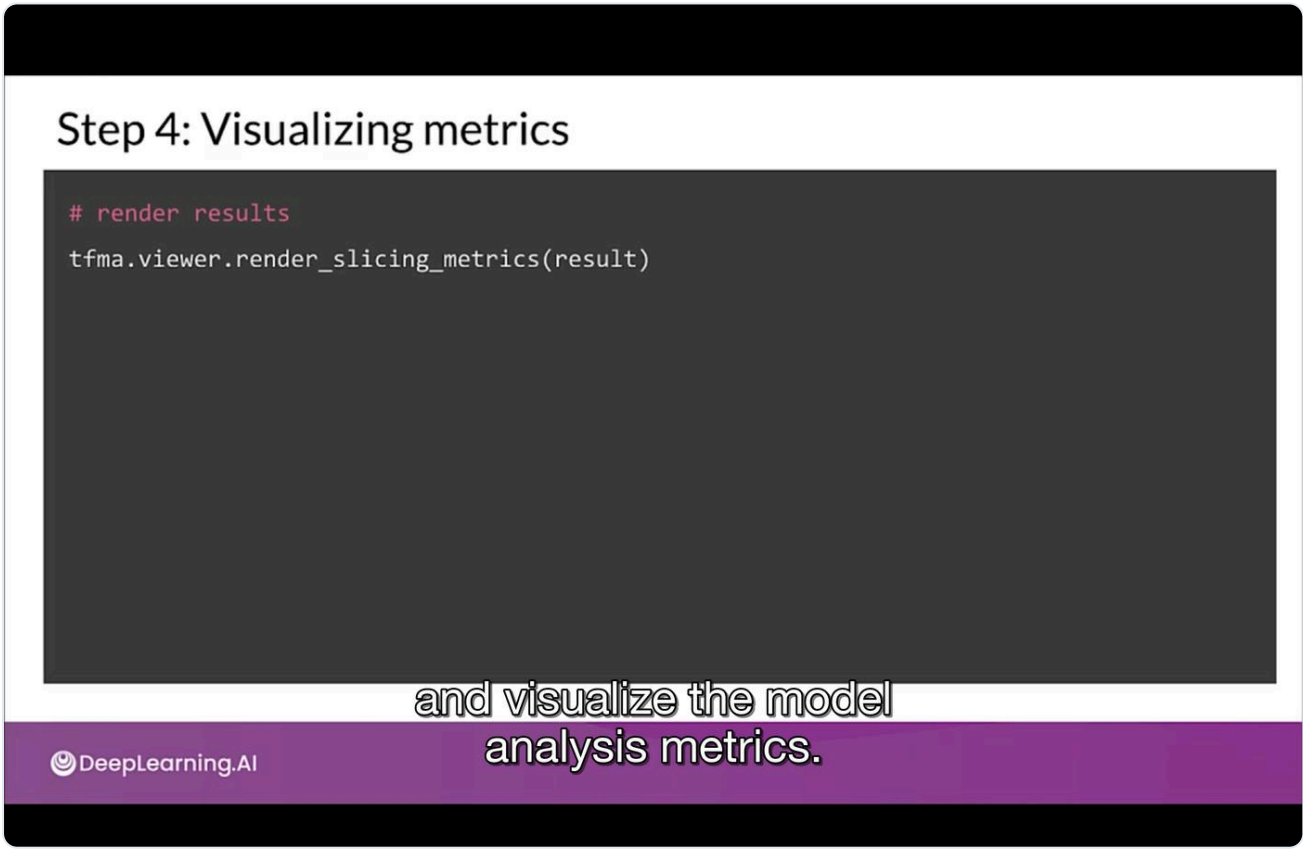

FIGURE 31.6

This API can accept slicing specifications directly as well. Finally, let's render the results and visualize the model analysis metrics. TFMA provides interactive widgets that can be run in notebooks, and this is a code that you would include in a notebook cell to generate the TFMA visualization widget.

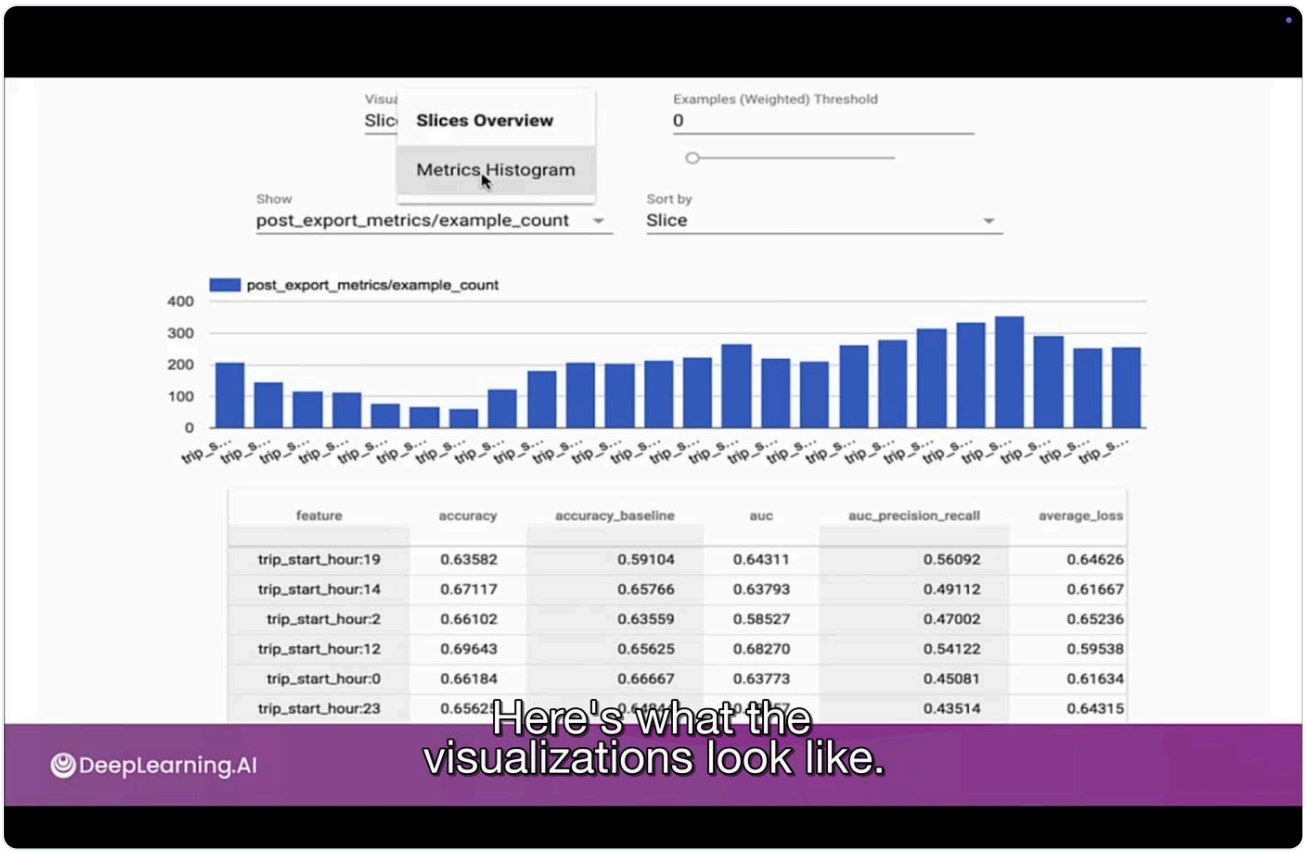

FIGURE 31.7

Here's what the visualizations look like. This tool enables you to really dig into your model performance and understand how it varies on different slices of data.

LECTURE 32

Robustness Metrics

Now let's discuss model performance beyond simple metrics and introduce some ways to analyze it and improve it. To talk about model debugging, let me first focus on model robustness. Checking the robustness of the model is a step beyond the simple measurement of model performance or generalization. A model is considered to be robust if its results are consistently accurate, even if one or more of the features change fairly drastically. Of course there are limits to robustness and all models are sensitive to changes in the data.

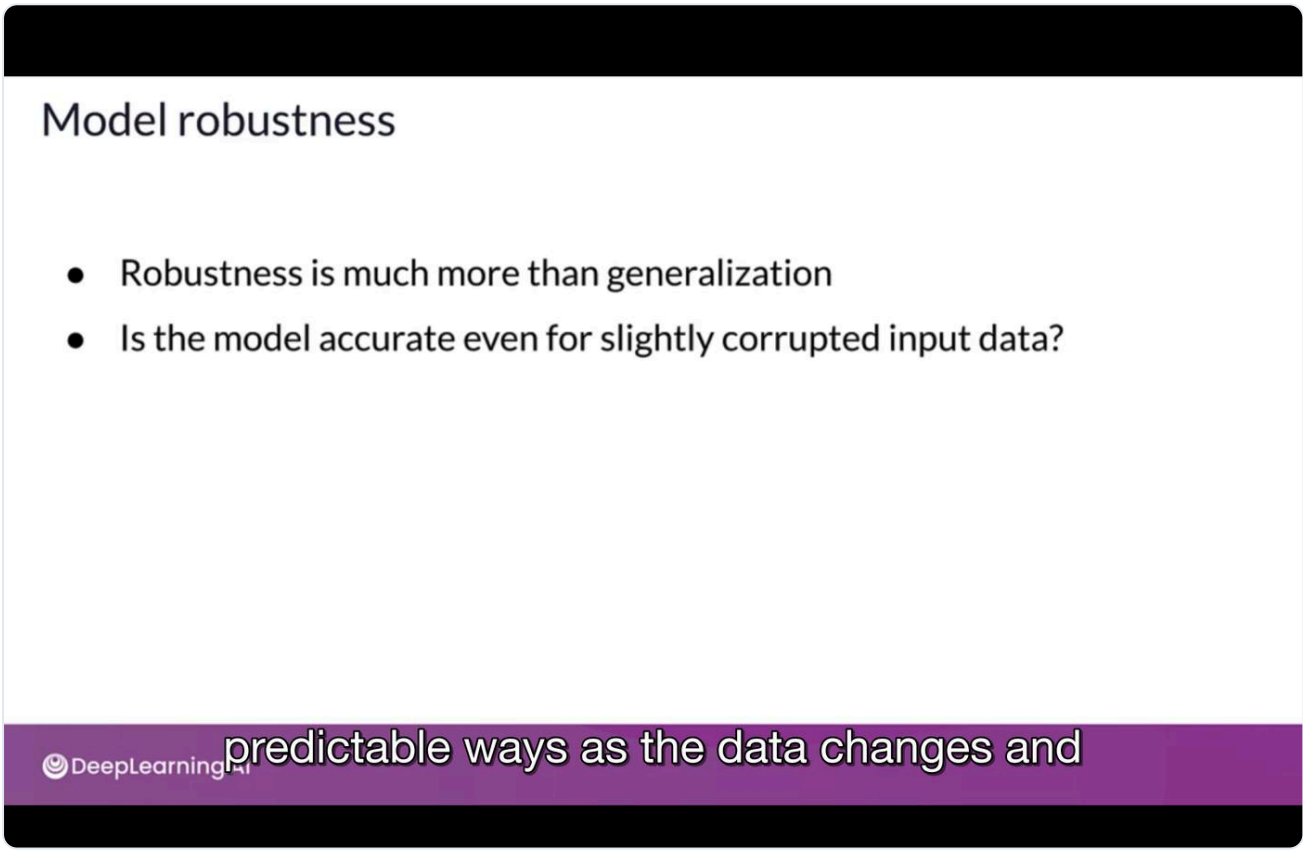

FIGURE 32.1 | Model Robustness

But there is a clear difference between a model that changes in gradual, predictable ways as the data changes and a model that suddenly produces wildly different results. So how do you measure the robustness of a model? The first and foremost thing is you shouldn't be measuring the robustness of a model during its training. Also you shouldn't be using the same data set that you used during training. So before you start the training process, you should split the data set into train, validation, and test splits.

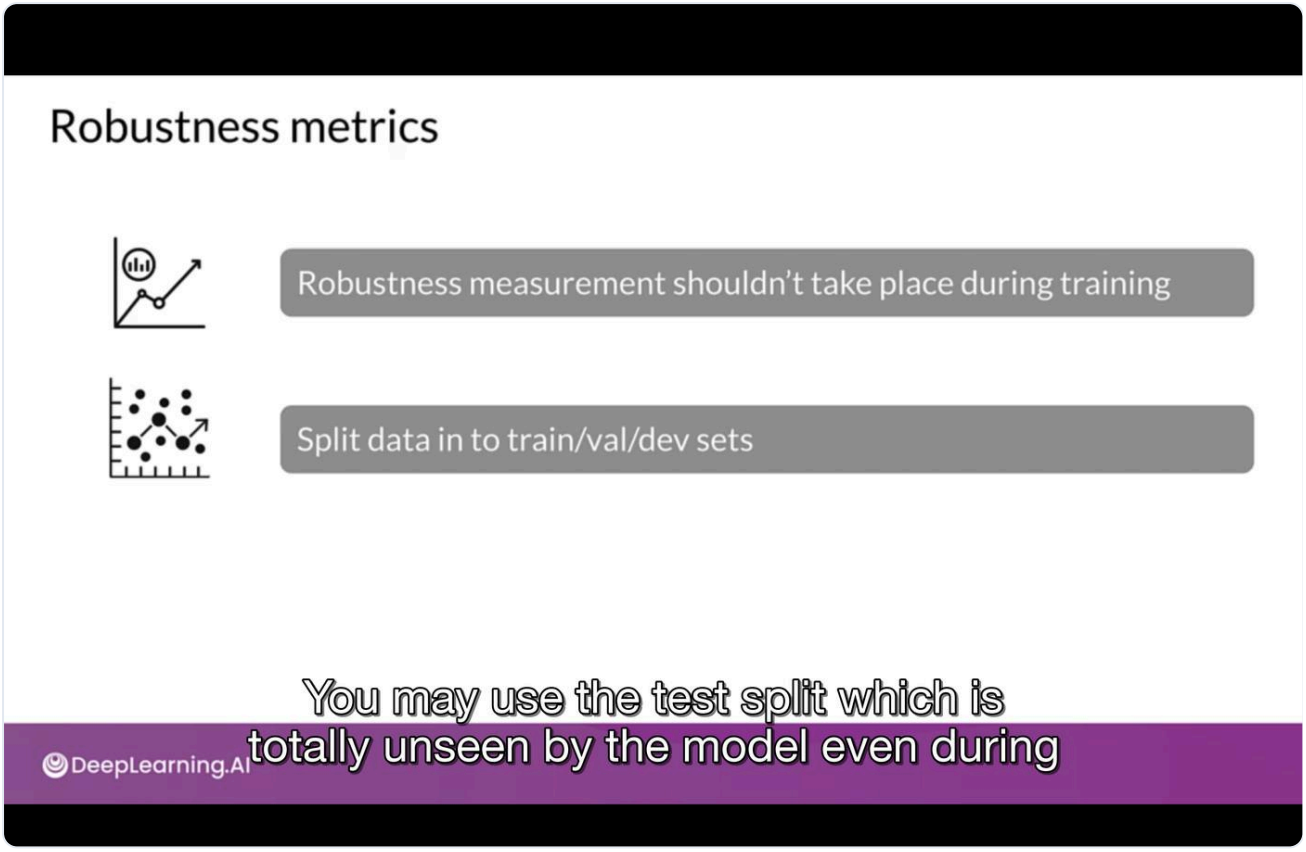

FIGURE 32.2 | Robustness Metrics

You may use the test split which is totally unseen by the model even during the validation stage for testing the model robustness. Or otherwise, the best choices to generate a variety of new types of data, and we'll discuss some of the methods to generate this later on.

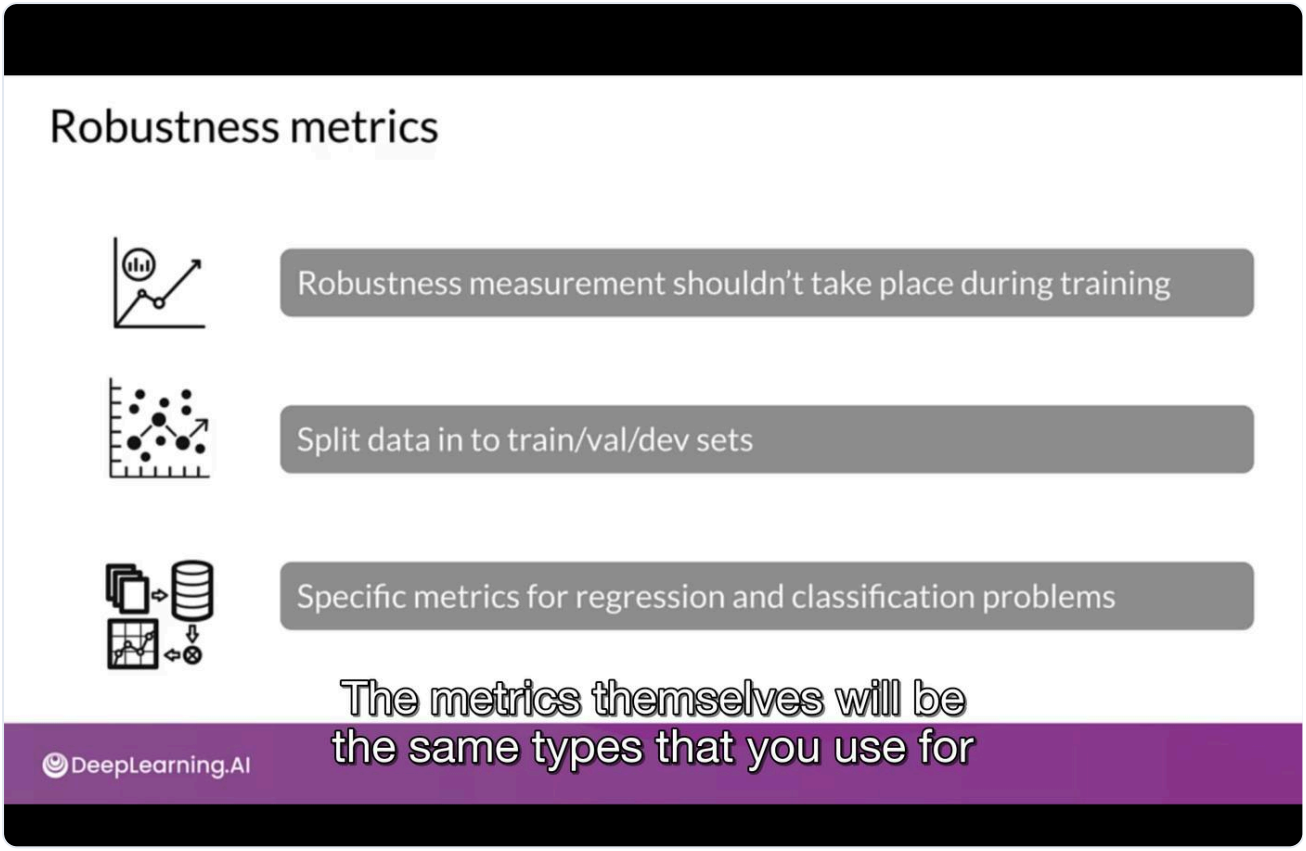

FIGURE 32.3 | Robustness Metrics

The metrics themselves will be the same types that you use for training depending on the model type. So things like RMSC for regression models and AUC for classification. Let's take a look at ways to increase model robustness. Model debugging is an emerging discipline focusing on finding and fixing problems in models and improving model robustness. Model debugging borrows various practices from model risk management, traditional model diagnostics, and software testing. Model debugging attempts to test ML models just like code, very similar to how you attest in software development.

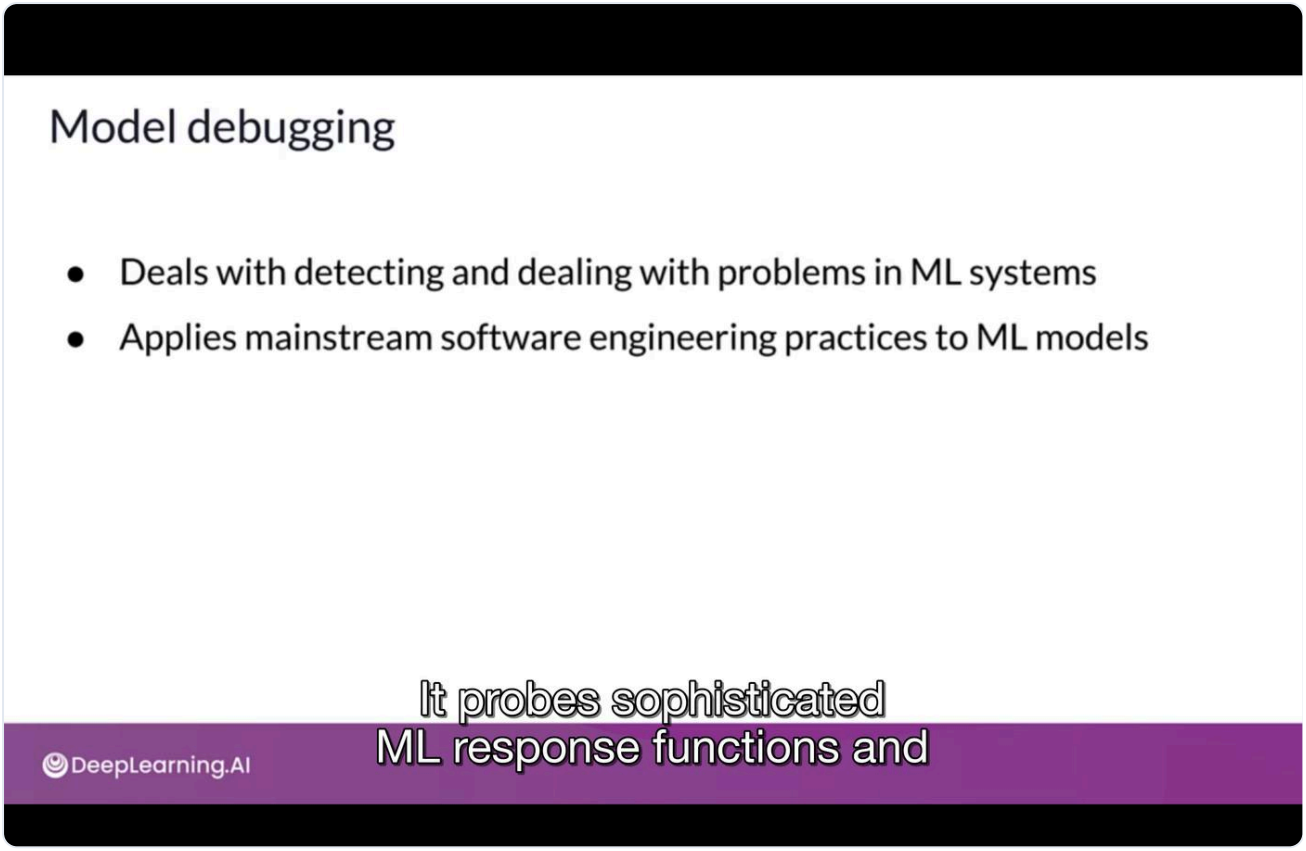

FIGURE 32.4 | Model Debugging

It probes sophisticated ML response functions and decision boundaries to detect and correct accuracy, fairness, security, and other problems in ML systems. We'll discuss this more in a bit. Model debugging has several objectives. One of the big problems with ML models is that they can be quite opaque and become black boxes. Model debugging tries to improve the transparency of models by highlighting how data is flowing inside. Another problem is social discrimination. Does your model work poorly for certain groups of people? Model debugging also aims to reduce the vulnerability of your model to attacks. For example, certain requests can be made once the model is in production that may be aimed at extracting data out of your model in order to understand how your model has been built.

This is especially a problem when data has a private information and it's been used for training. So was the data anonymized before it was used for training?

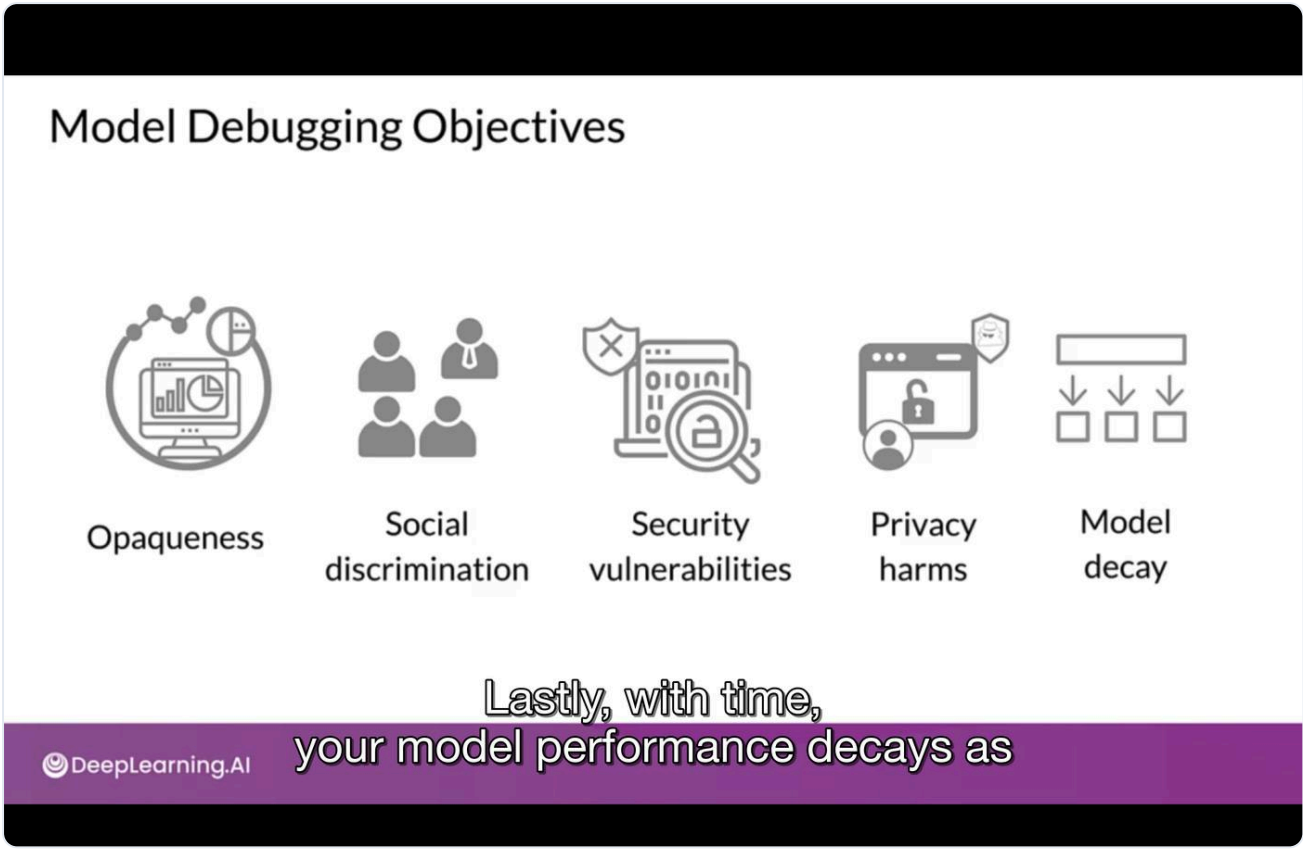

FIGURE 32.5 | Model Debugging Objectives

Lastly, with time, your model performance decays as the distribution of incoming data changes.

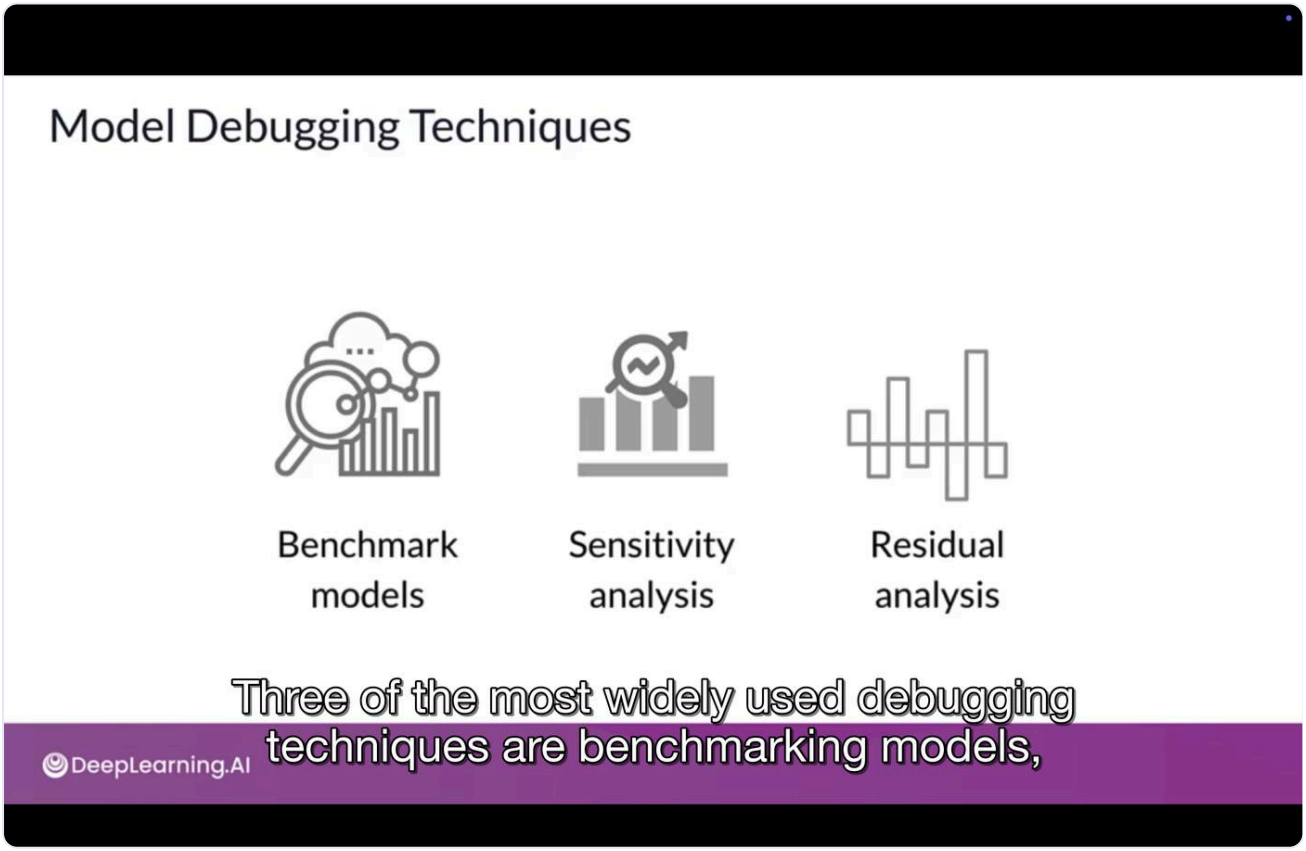

FIGURE 32.6 | Model Debugging Techniques

Three of the most widely used debugging techniques are benchmarking models, sensitivity analysis, and residual analysis. We'll discuss each of these individually.

LECTURE 33

Benchmark Models

We'll start with a brief discussion of using benchmarking models, which is always a good model debugging technique. A good place to start is by comparing your model to benchmark models. Which are a simple and powerful way to check your model's performance. Benchmarking models are small, simple models that are used before you start development for baselining your problem. They're generally not state-of-the-art, but instead are linear or other simple models with very consistent performance. You compare your model to see if it actually is performing better than the simpler benchmarking model as a sanity test.

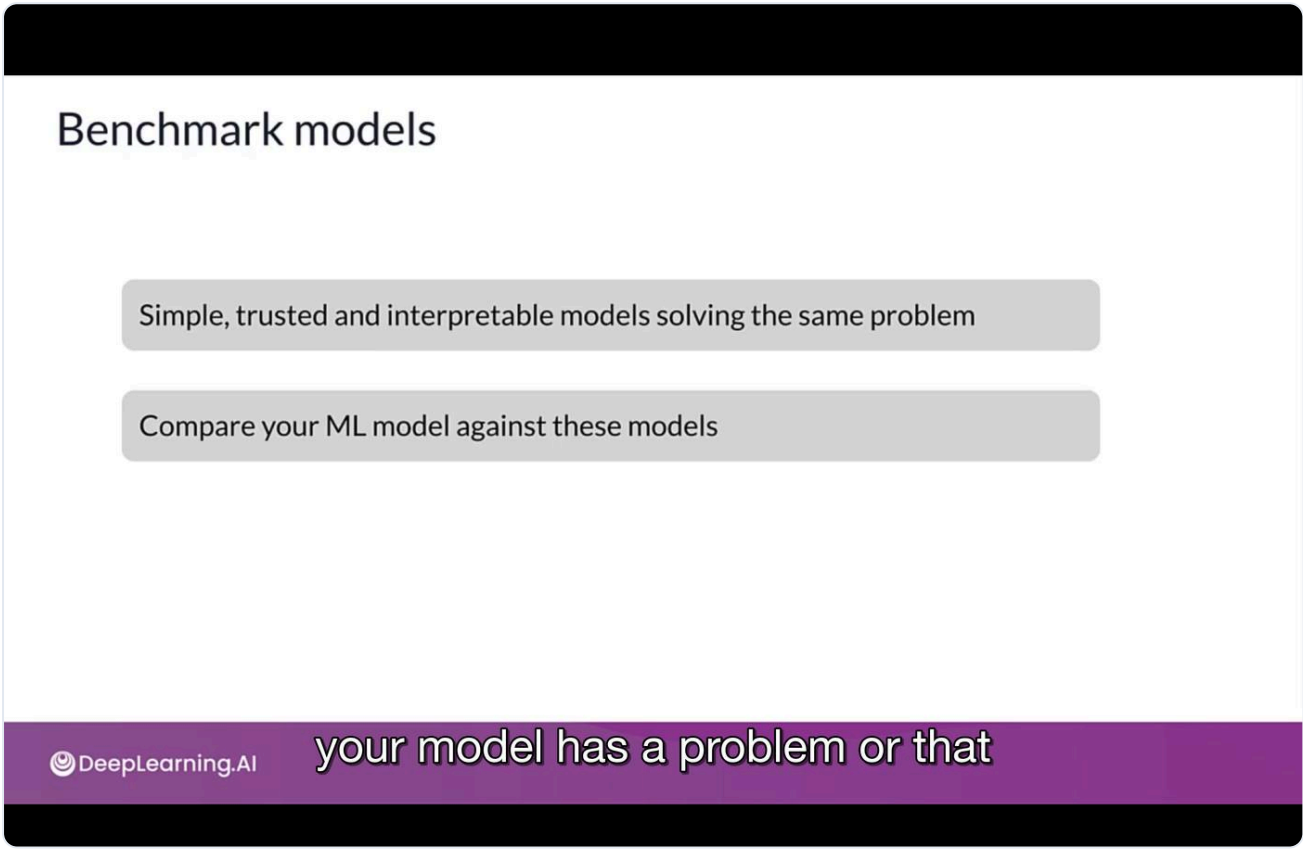

FIGURE 33.1 | Benchmark Models

If it doesn't, it could be that your model has a problem or that a simple model accurately models the data and is really all that you need for your application.

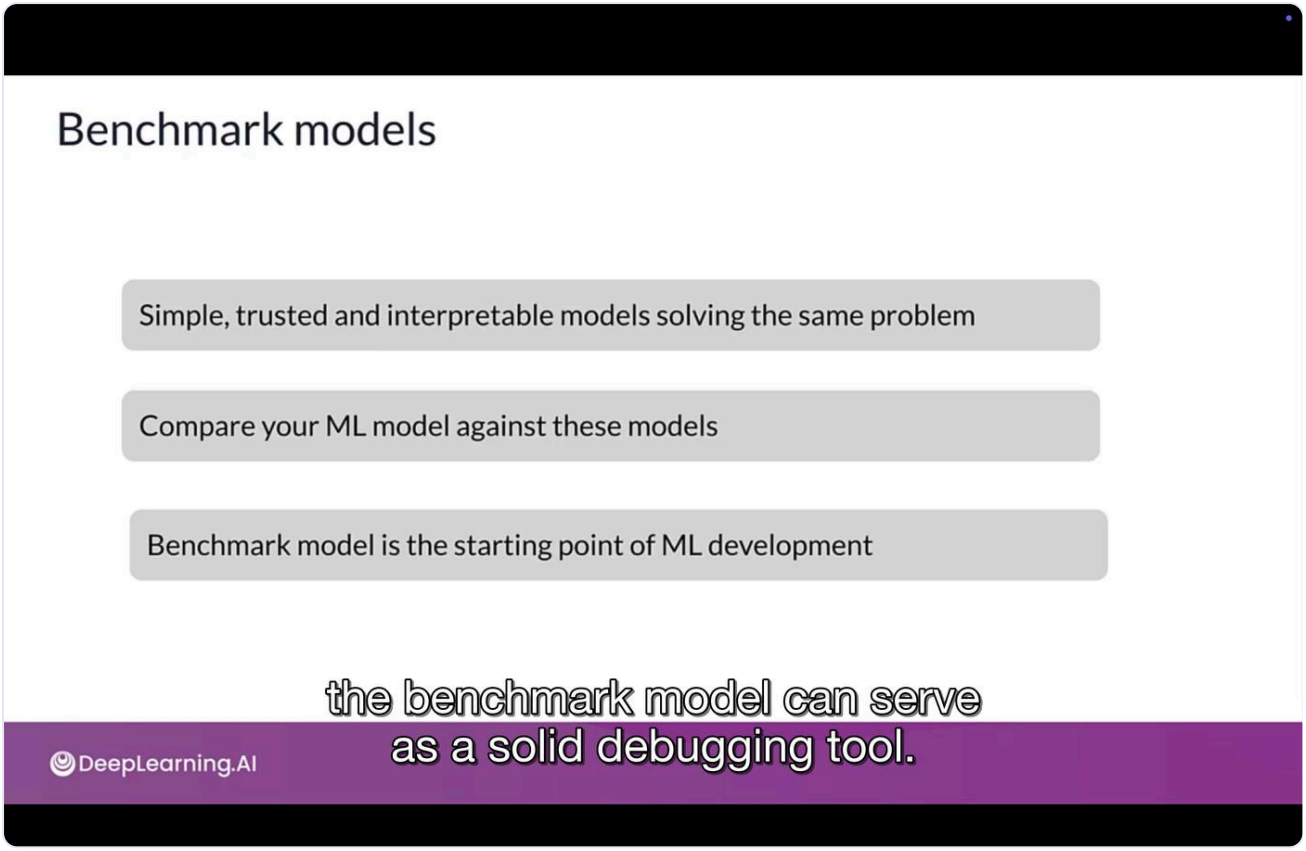

FIGURE 33.2 | Benchmark Models

Once a model passes the benchmark test, the benchmark model can serve as a solid debugging tool. Even if you're performing better than the benchmark model, you can still evaluate which test samples your model is failing. But the benchmark model predicts correctly. Then you need to study your model to find out why that's happening.

LECTURE 34

Random Attacks

Sensitivity analysis is an important way to evaluate your model's performance, including its vulnerability to adversarial attacks. Let's see what this is all about. Sensitivity analysis helps you understand your model by examining the impact that each feature has on your model's prediction. In sensitivity analysis you experiment by changing a single features value while holding the other features constant and observe the model results. If changing the features value causes the models result to be drastically different. It means that this feature has a big impact on the prediction.

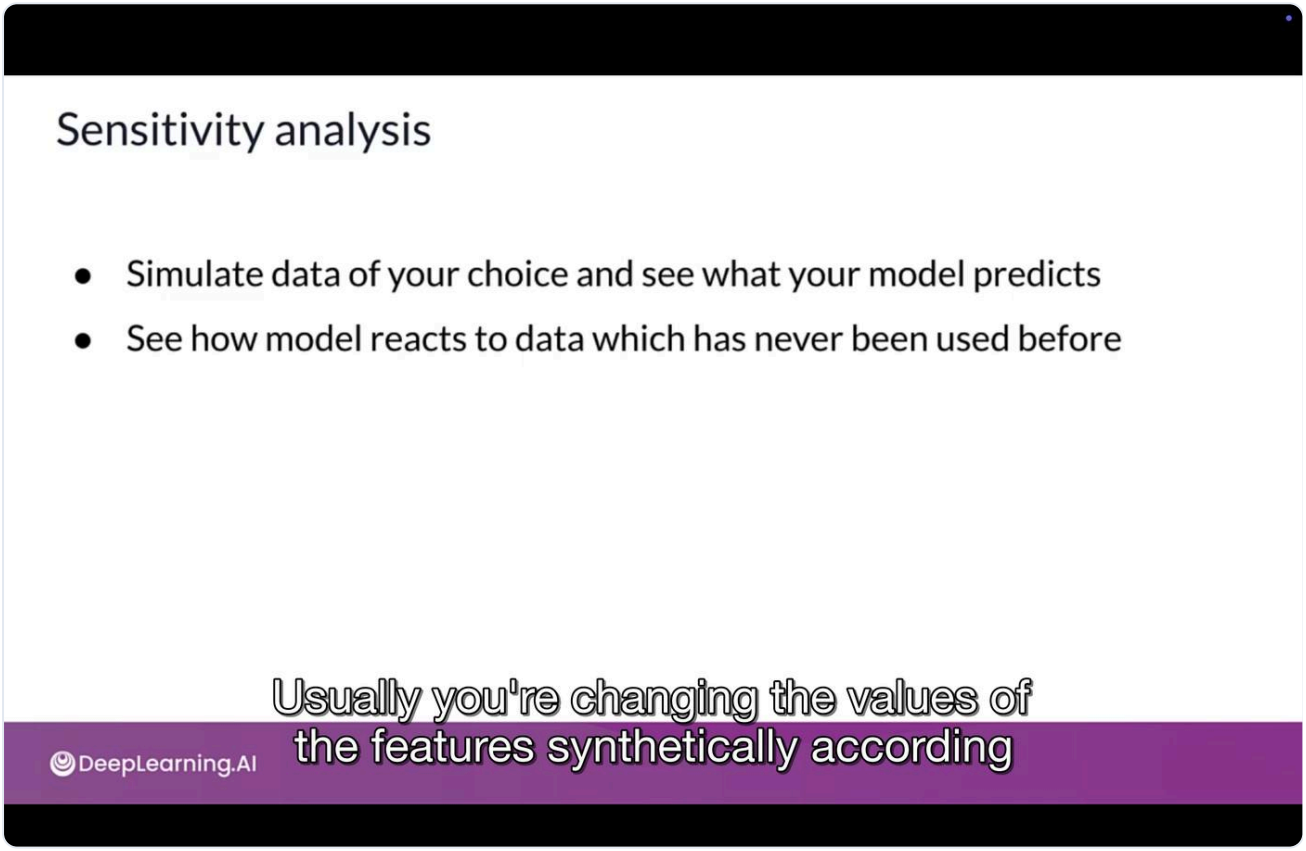

FIGURE 34.1 | Sensitivity Analysis

Usually you're changing the values of the features synthetically according to some distribution or process and ignoring the labels for the data. You're really not looking to see if the prediction is correct or not, but instead how much it changes different ways of doing sensitivity analysis. Use different techniques for changing the feature value. Let's explore a few different approaches.

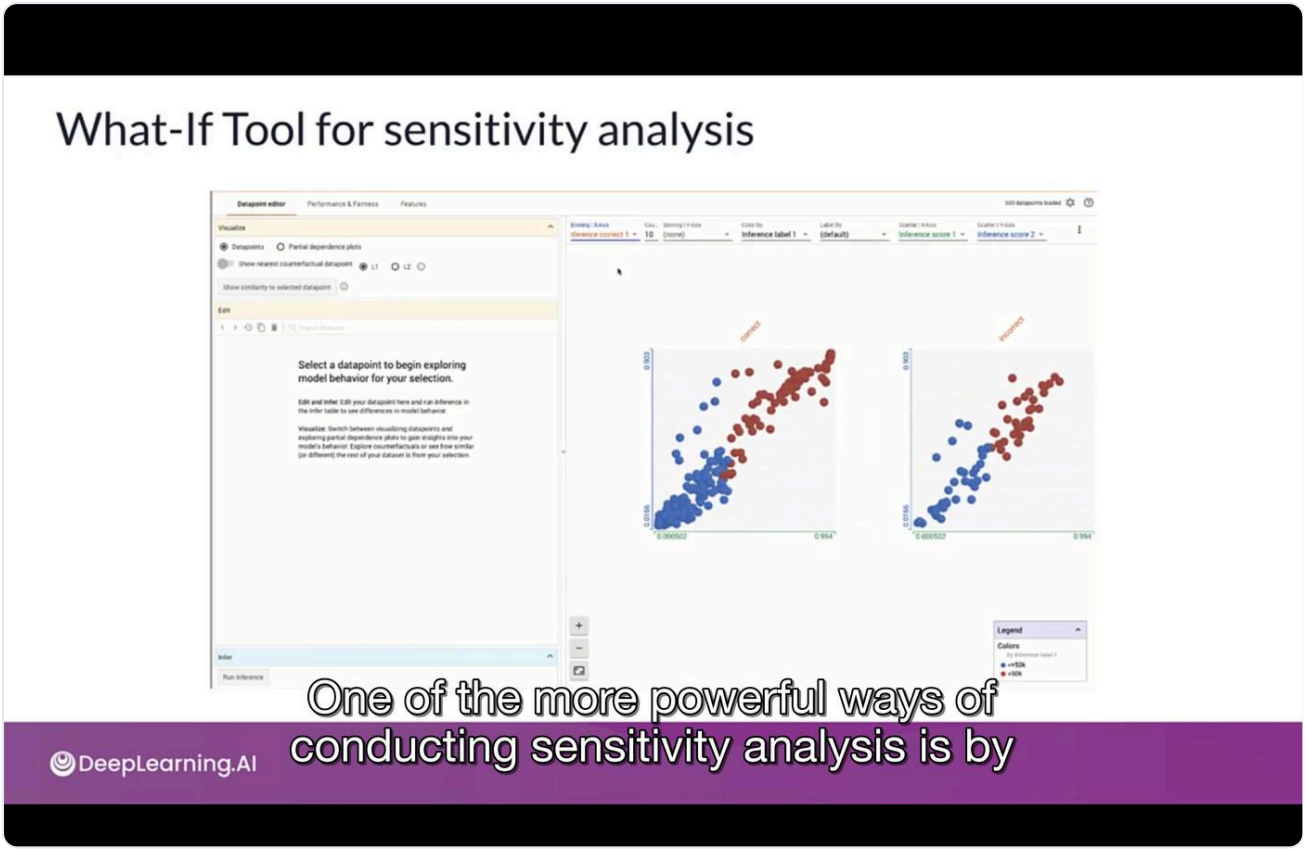

FIGURE 34.2 | What-If Tool for Sensitivity Analysis

One of the more powerful ways of conducting sensitivity analysis is by using the What-if tool which was created by the tensor flow team. The What-if tool allows you to visualize the results of sensitivity analysis to understand and debug your performance.

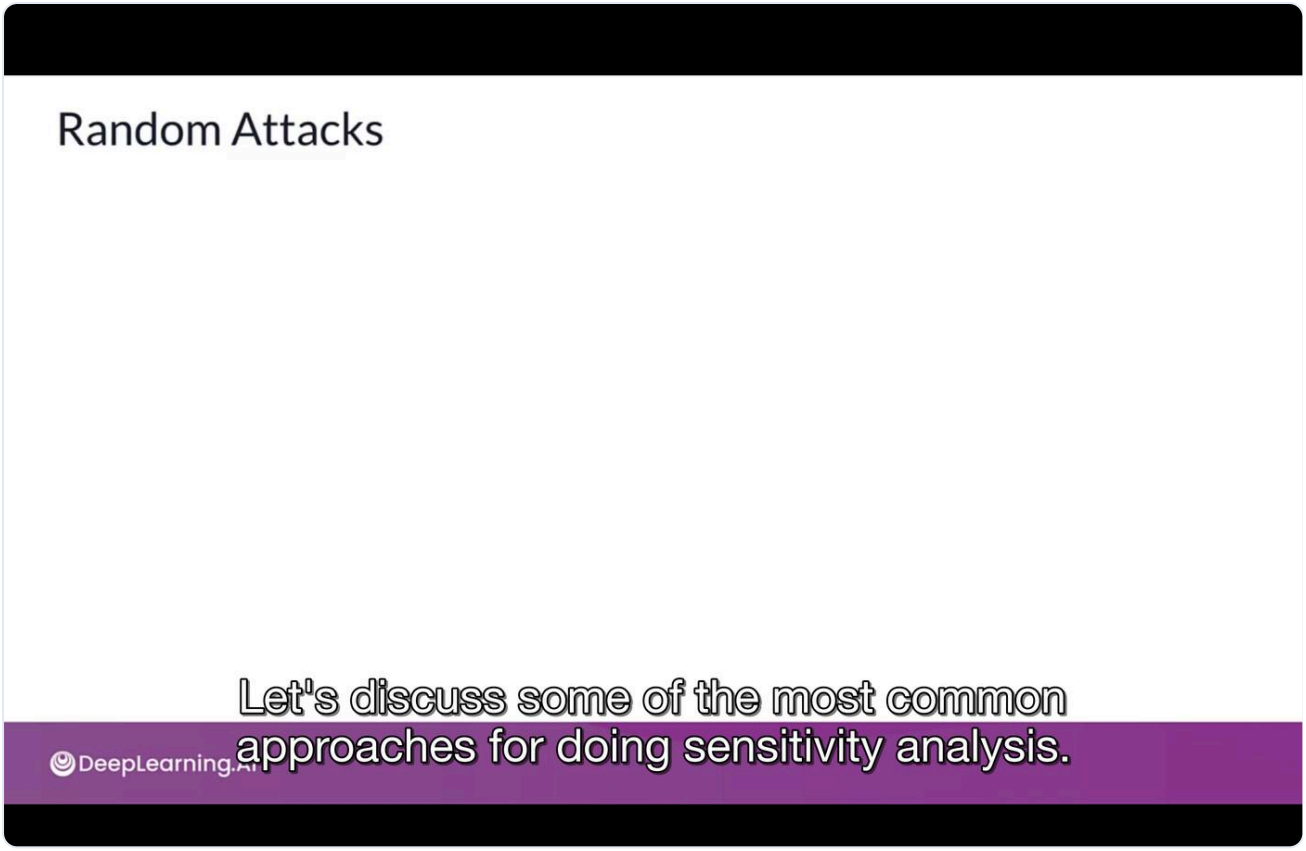

FIGURE 34.3 | Random Attacks

Let's discuss some of the most common approaches for doing sensitivity analysis. In random attacks you generate lots of random input data and test the models outputs.

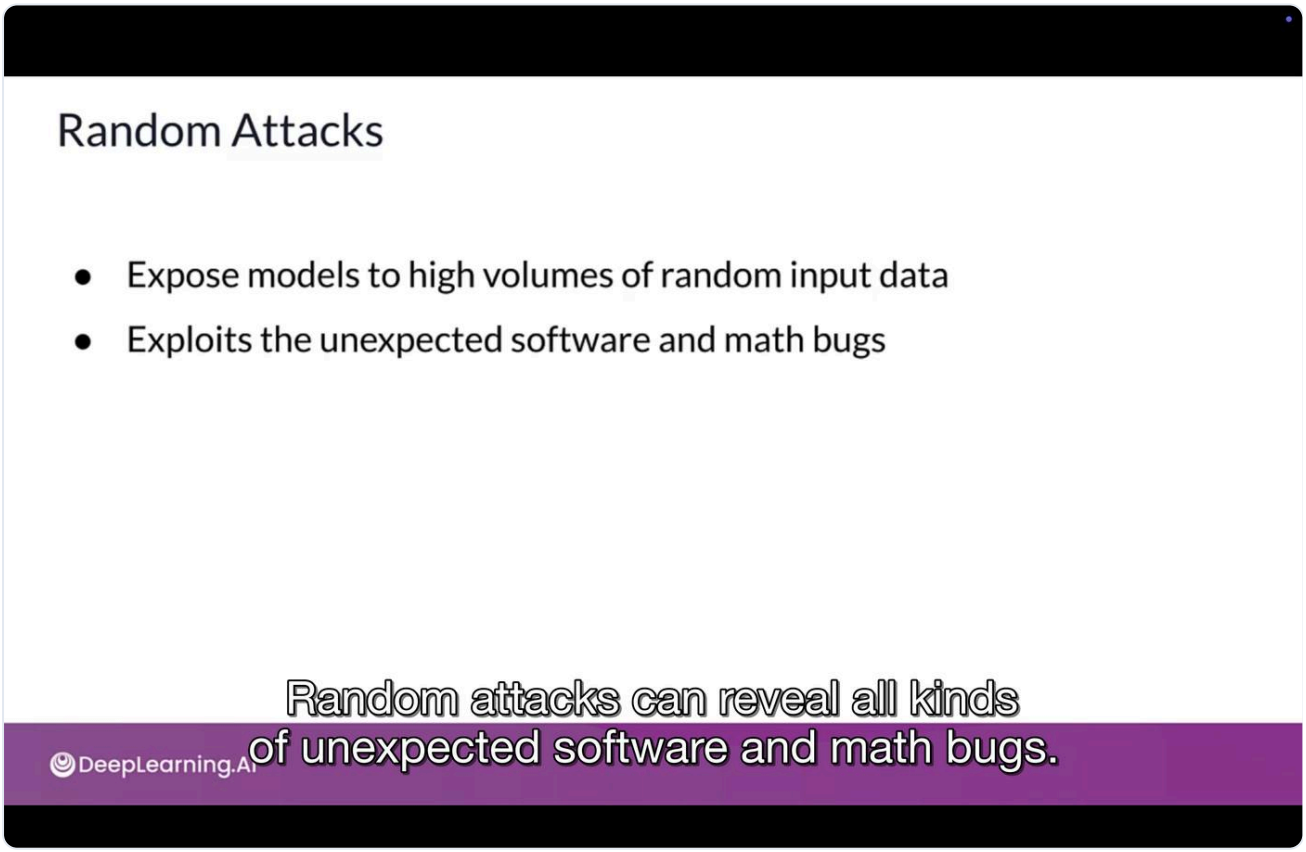

FIGURE 34.4 | Random Attacks

Random attacks can reveal all kinds of unexpected software and math bugs.

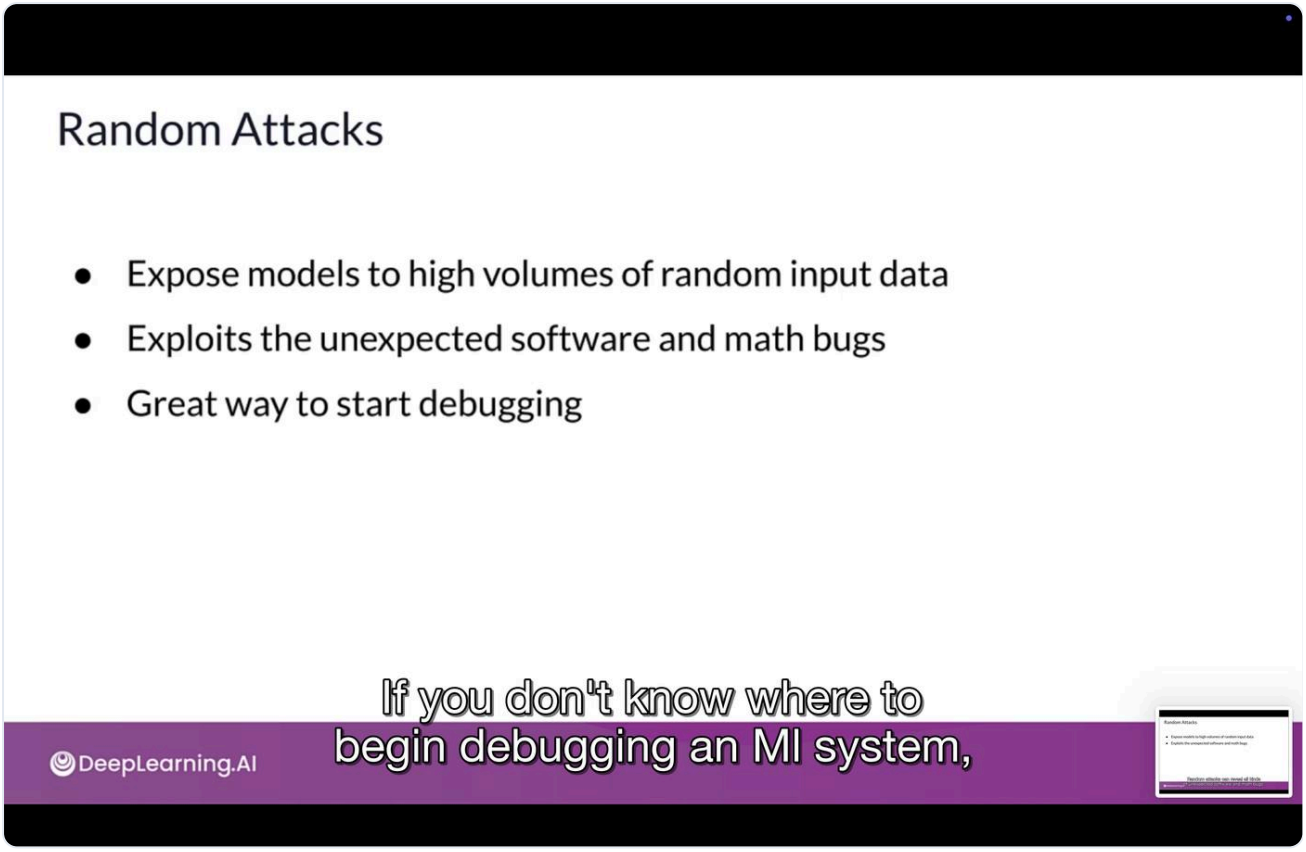

FIGURE 34.5 | Random Attacks

If you don't know where to begin debugging an Ml system, a random attack is a great place to get started. Another approach is to use a partial dependence plot. Partial dependence plots show the marginal effect of one or two features and the effect they have on the model results. A partial dependence plot can show whether the relationship between the label and a particular feature is linear monotonic, or more complex. For example, when applied to a linear regression model, partial dependence plots always show a linear relationship.

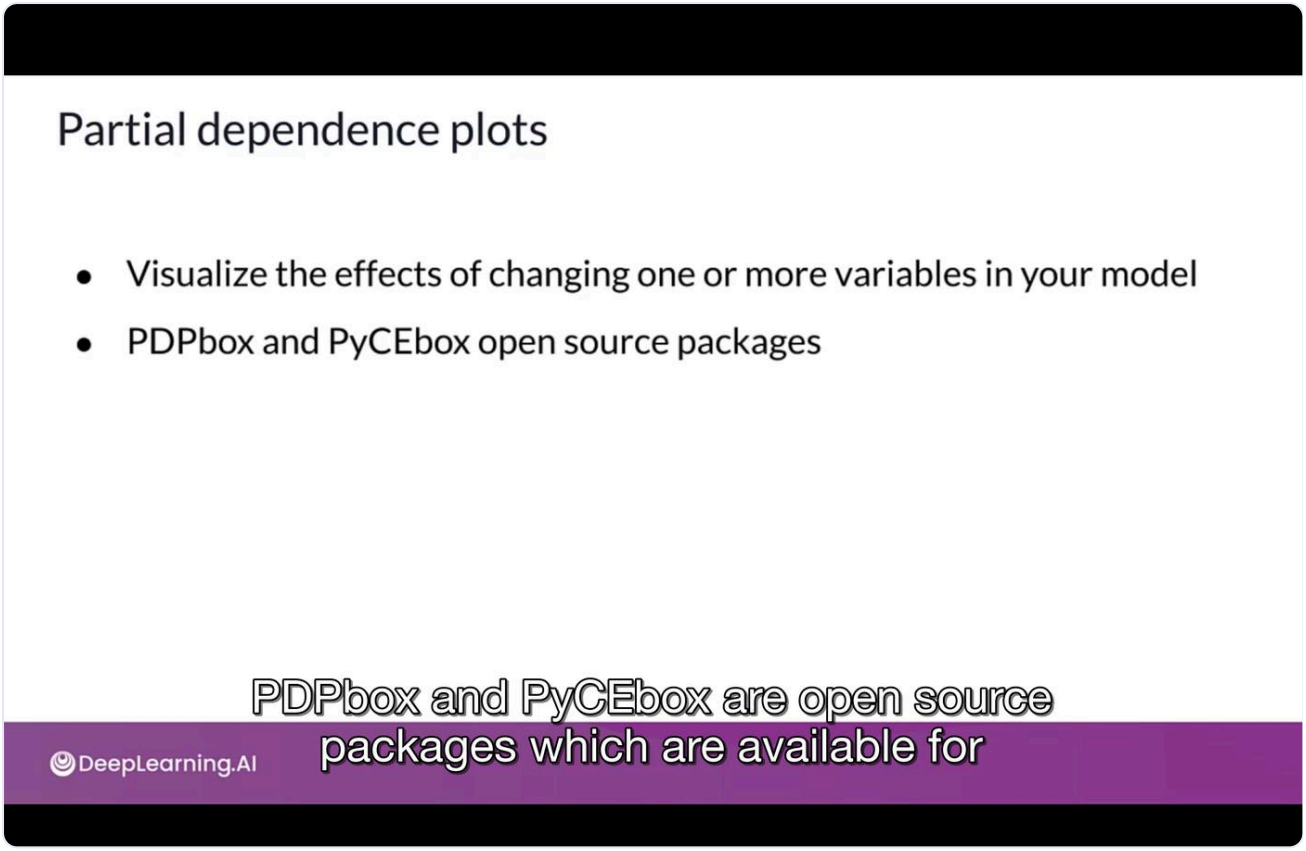

FIGURE 34.6 | Partial Dependence Plots

PDPbox and PyCEbox are open source packages which are available for creating partial dependence plots. Please check the reading list for more information, now that we've introduced two sensitivity analysis techniques, it's worth asking how vulnerable your model is to attacks. Several machine learning problems, including neural networks can be fooled into misclassifying adversarial examples which are formed by making small but carefully designed changes to the data so that the model returns an incorrect answer with high confidence. This could have daunting implications.

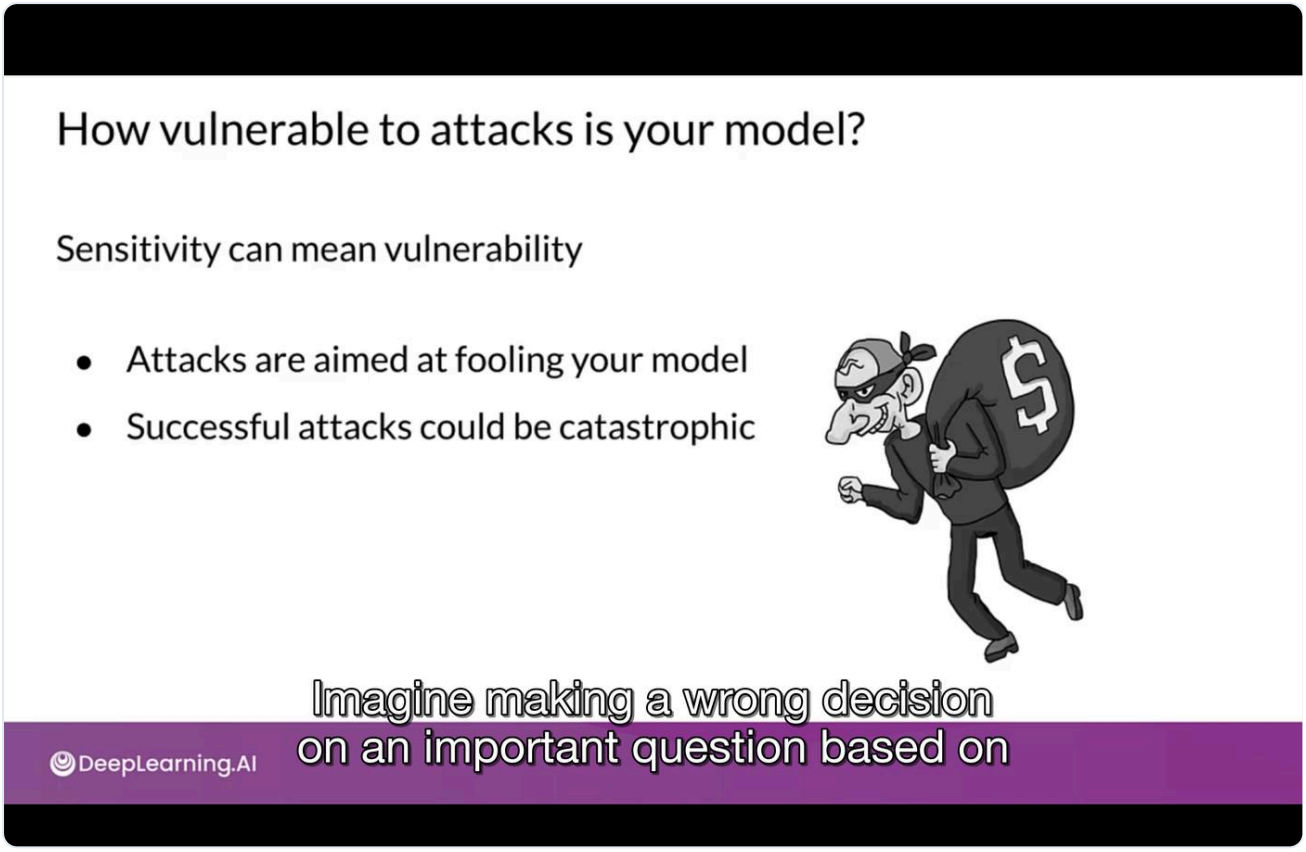

FIGURE 34.7 | How Vulnerable to Attacks Is Your Model?

Imagine making a wrong decision on an important question based on only slightly corrupted data.

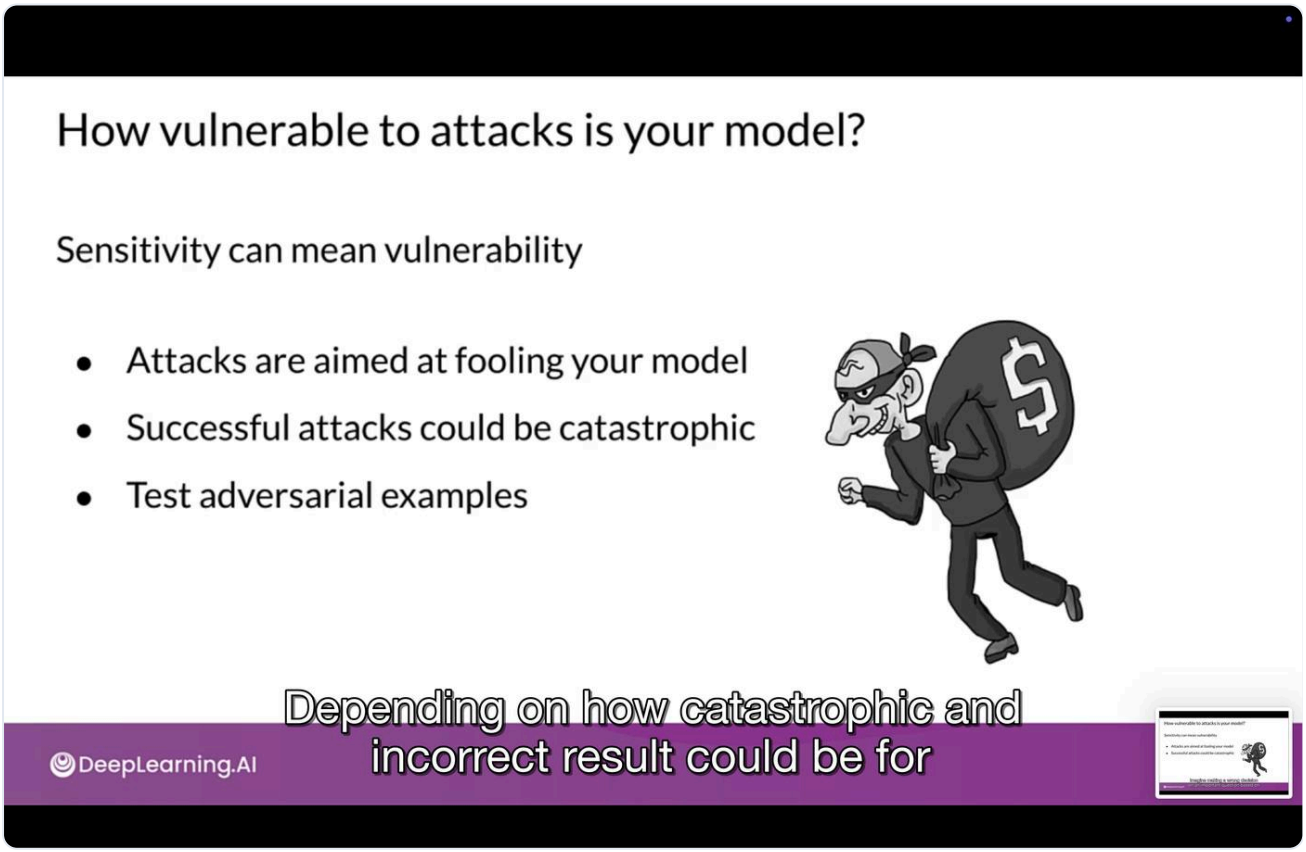

FIGURE 34.8 | How Vulnerable to Attacks Is Your Model?

Depending on how catastrophic and incorrect result could be for your application.

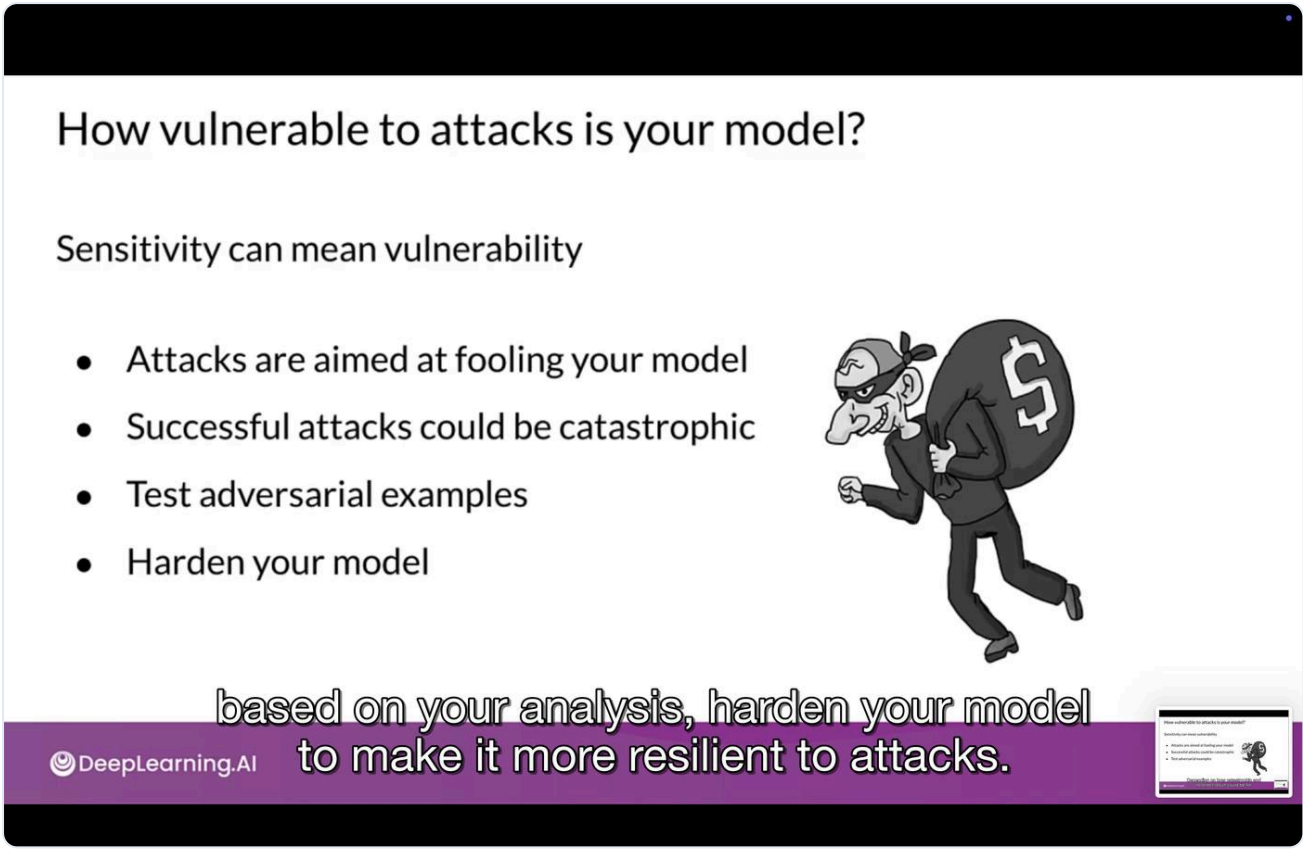

FIGURE 34.9 | How Vulnerable to Attacks Is Your Model?

You may need to test your model for vulnerabilities and based on your analysis, harden your model to make it more resilient to attacks. So what do these attacks look like?

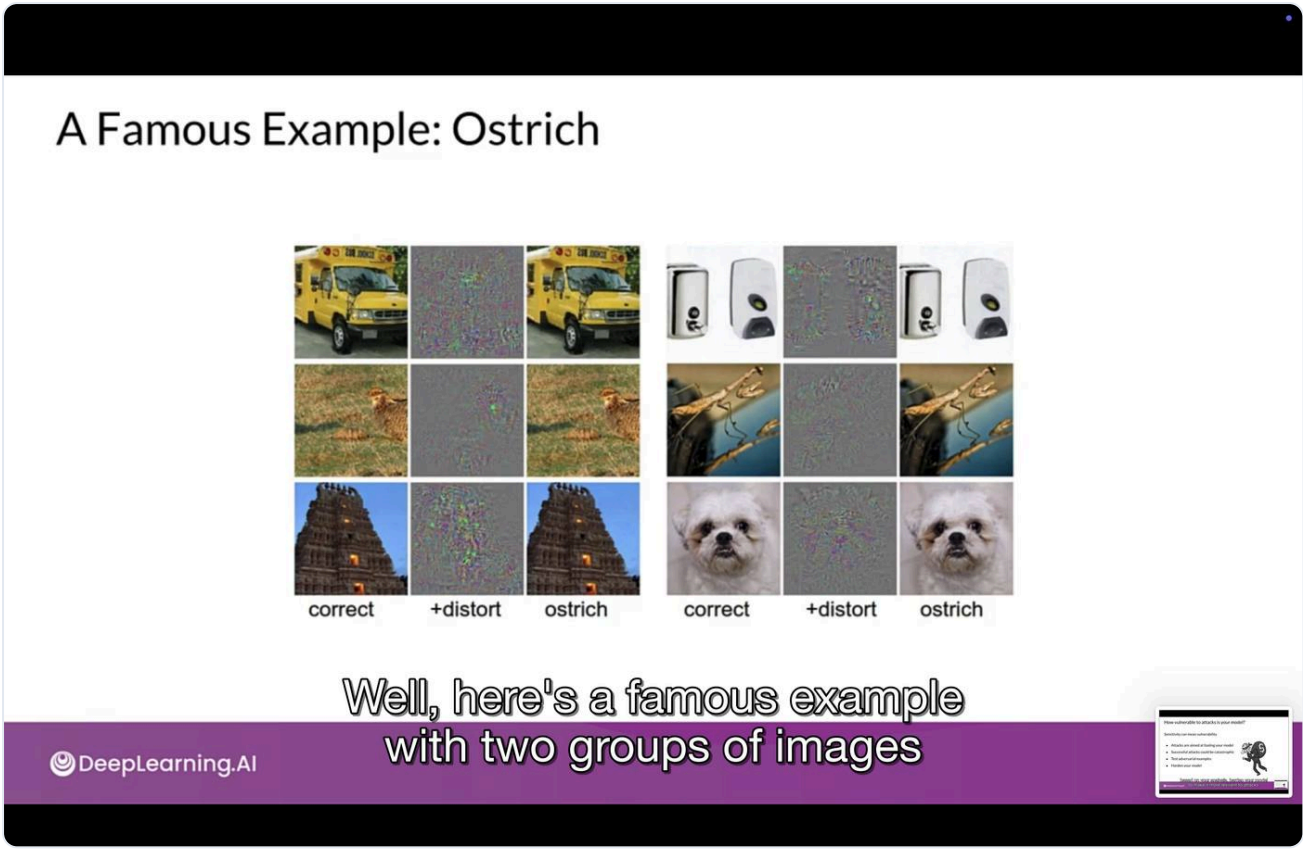

FIGURE 34.10 | A Famous Example: Ostrich

Well, here's a famous example with two groups of images applying only a tiny distortion to the images in the left columns, results in the images in the right columns, which a model train on image that classifies as an ostrich. So how serious of a problem is it?

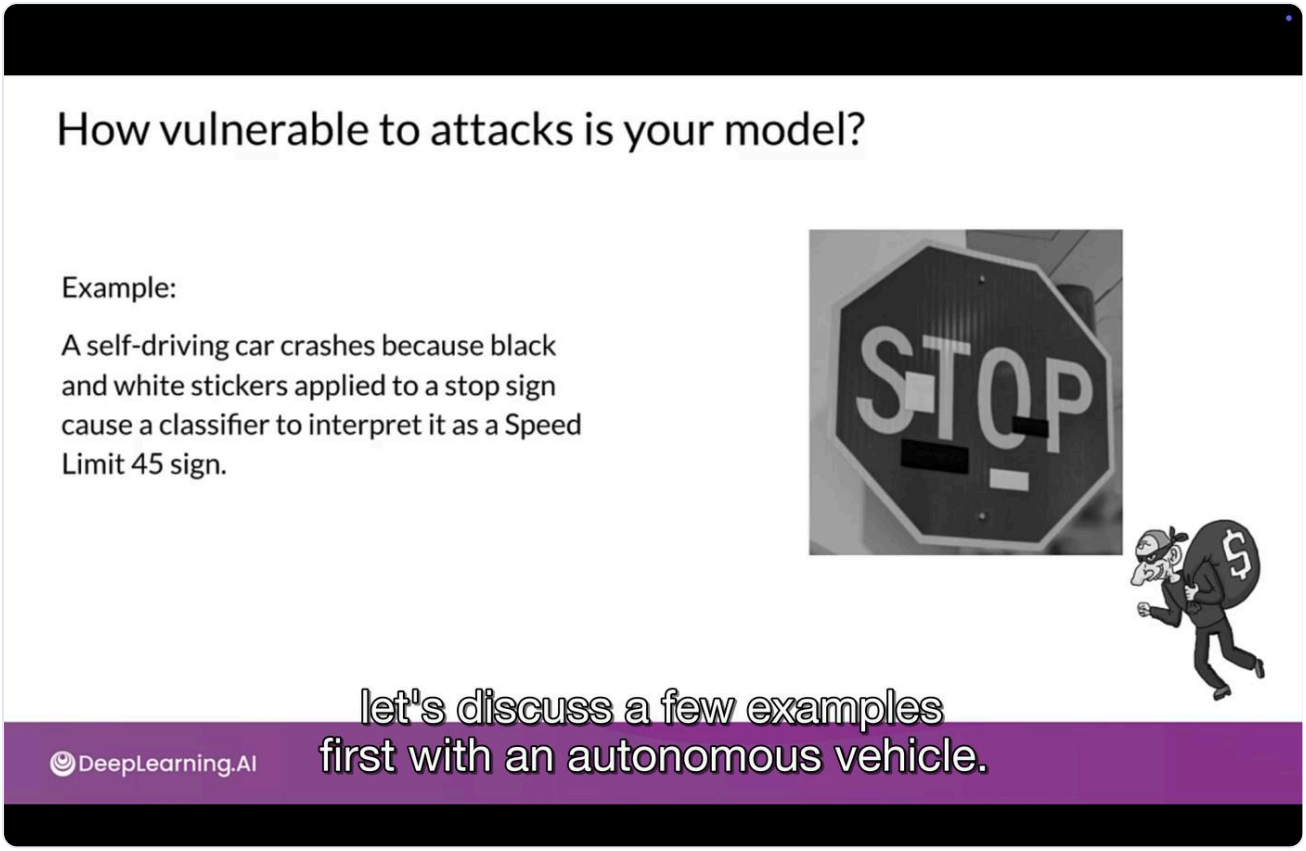

FIGURE 34.11 | How Vulnerable to Attacks Is Your Model?

Well, it depends on your application, but let's discuss a few examples first with an autonomous vehicle. It's important recognize traffic signs, other vehicles, people, etc.

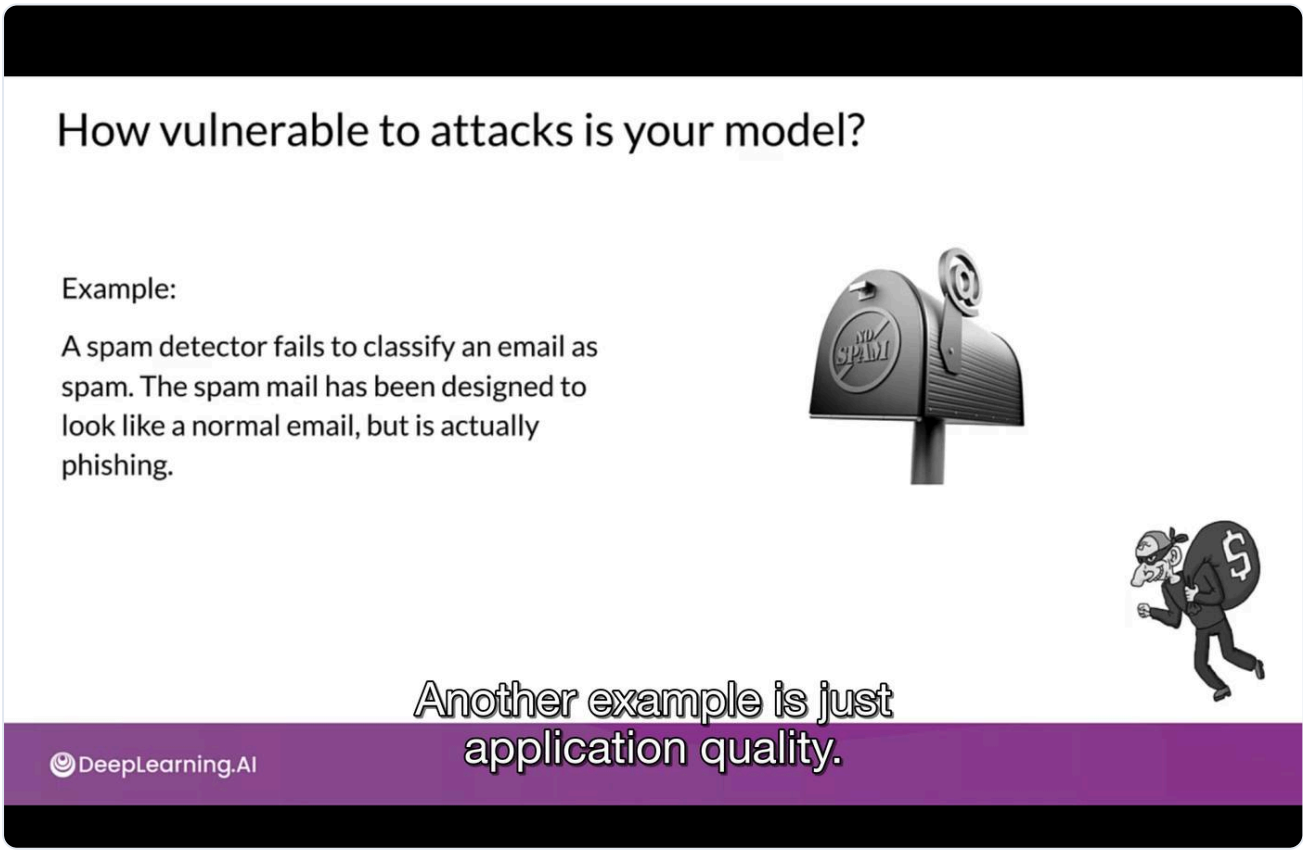

FIGURE 34.12 | How Vulnerable to Attacks Is Your Model?

Unfortunately, we've seen examples of this in real life, but as in this example, if a sign is altered in just the right way, it can fool the model and the results can be catastrophic. Another example is just application quality. If your business sells software to detect spam and phishing emails can get through, it reflects badly on your product.

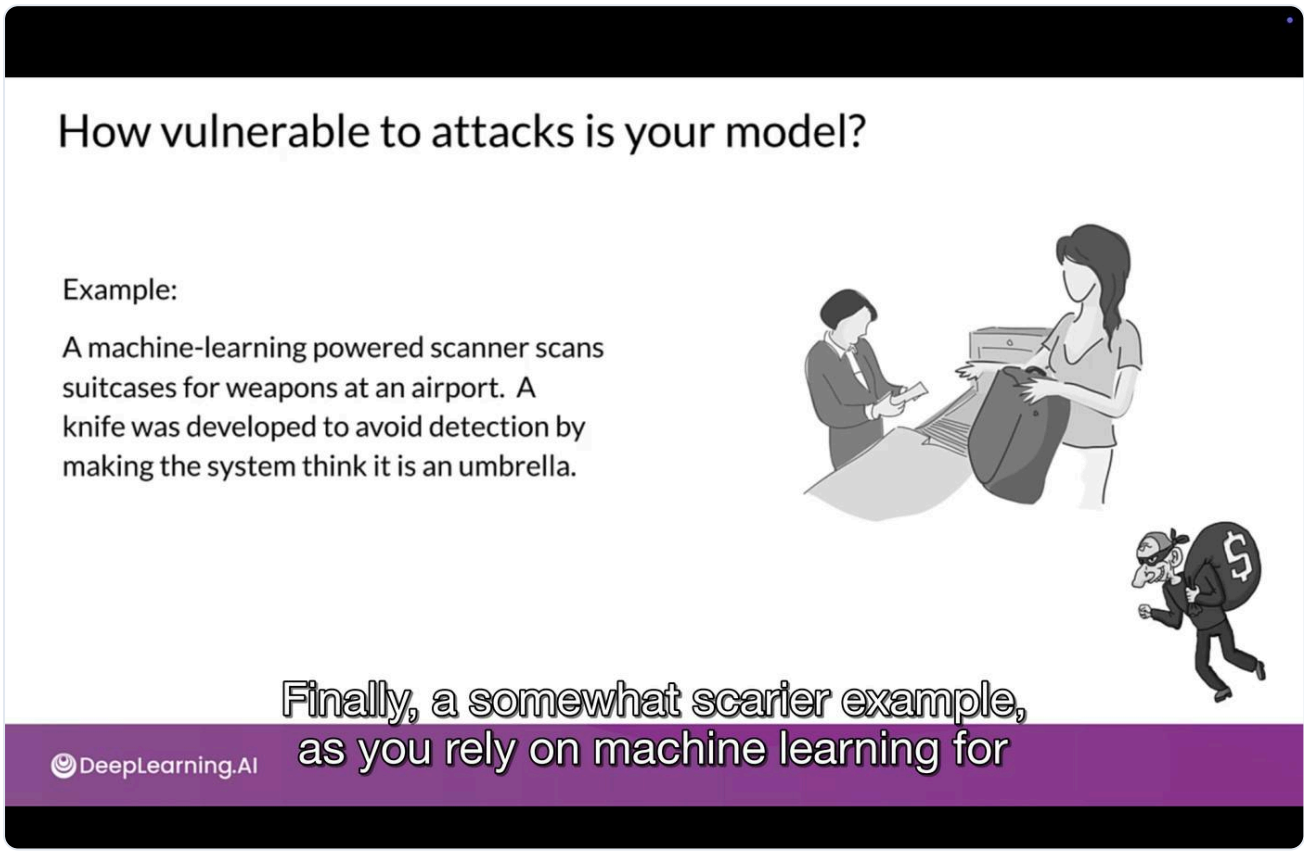

FIGURE 34.13 | How Vulnerable to Attacks Is Your Model?

Finally, a somewhat scarier example, as you rely on machine learning for more and more mission critical applications, you need to consider the security implications. A suitcase scanner is basically just an object classifier, but if it's vulnerable to attack, the results can be dangerous. The future of privacy forum, an industry group that studies privacy and security, suggest that security and privacy harms enabled by machine learning fall into roughly two categories. Informational and behavioral.

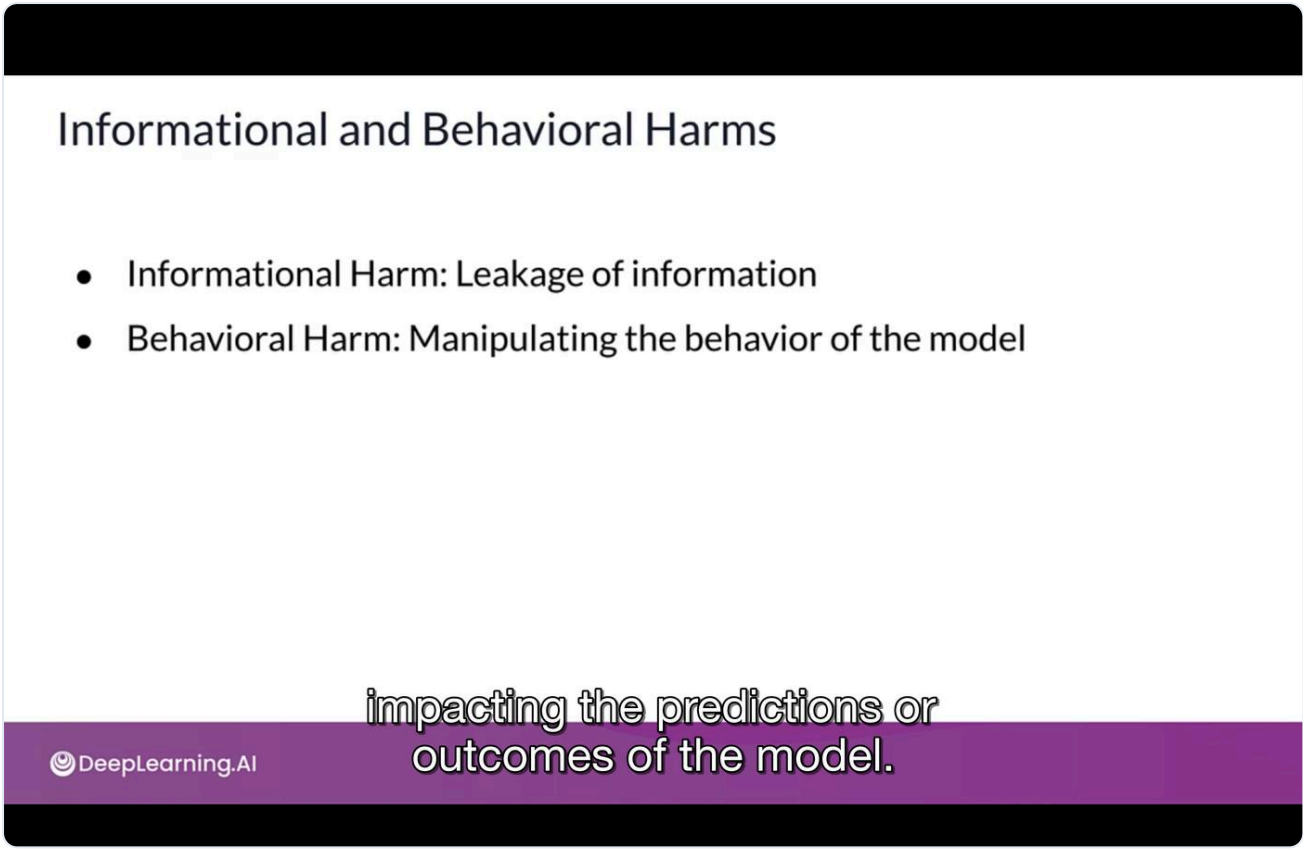

FIGURE 34.14 | Informational and Behavioral Harms

Informational harms relate to the unintended or unanticipated leakage of information, behavioral harms, on the other hand, relate to manipulating the behavior of the model itself, impacting the predictions or outcomes of the model. Let's look at informational harm, membership inference attacks are aimed at inferring whether or not an individual's data was used to train the model based on a sample of the model's output. While seemingly complex studies have shown that these attacks require much less sophistication than is frequently assumed.

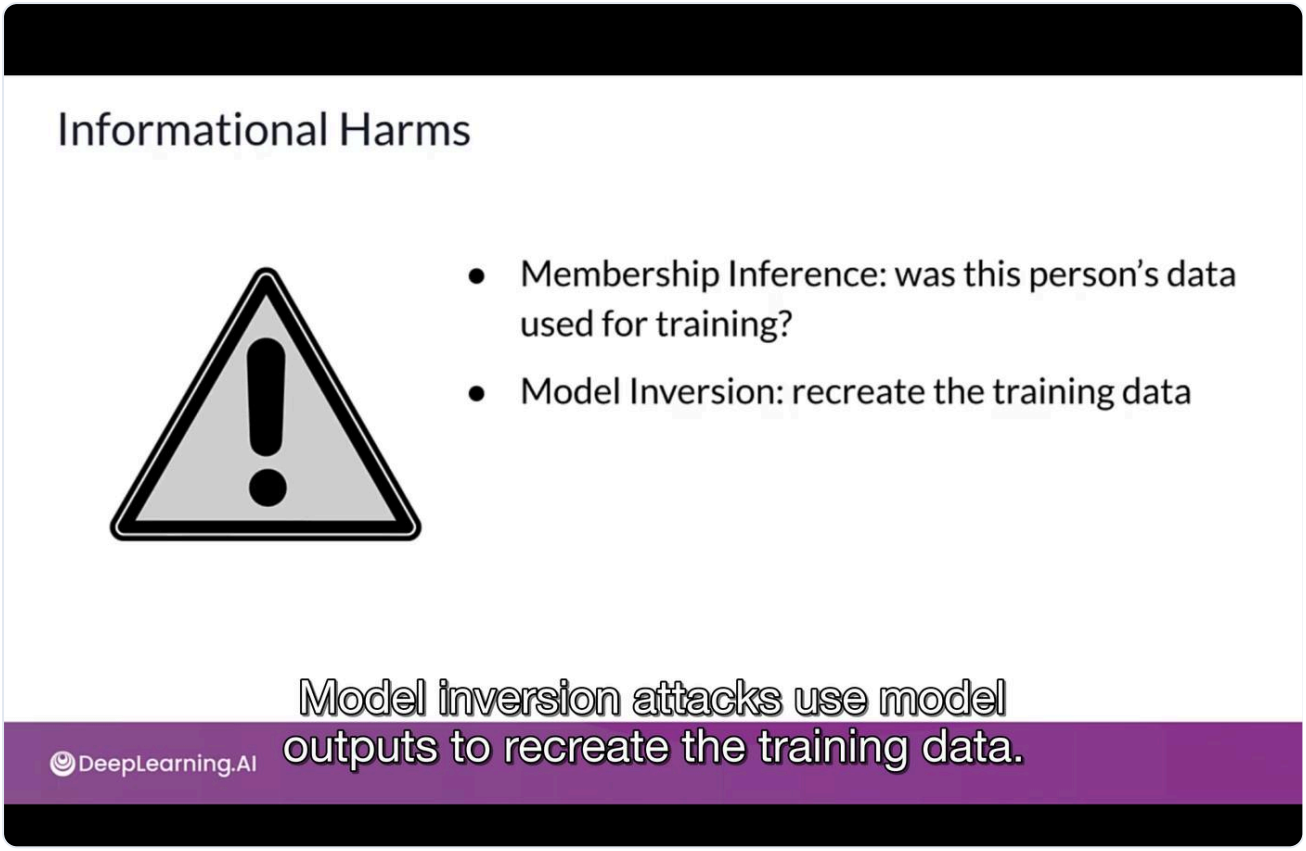

FIGURE 34.15 | Informational Harms

Model inversion attacks use model outputs to recreate the training data. In one well known example, researchers were able to reconstruct an image of an individual's face. Another study focused on ML systems that use genetic information to recommend dozing of specific medications and it was able to directly predict individual patient's genetic markers.

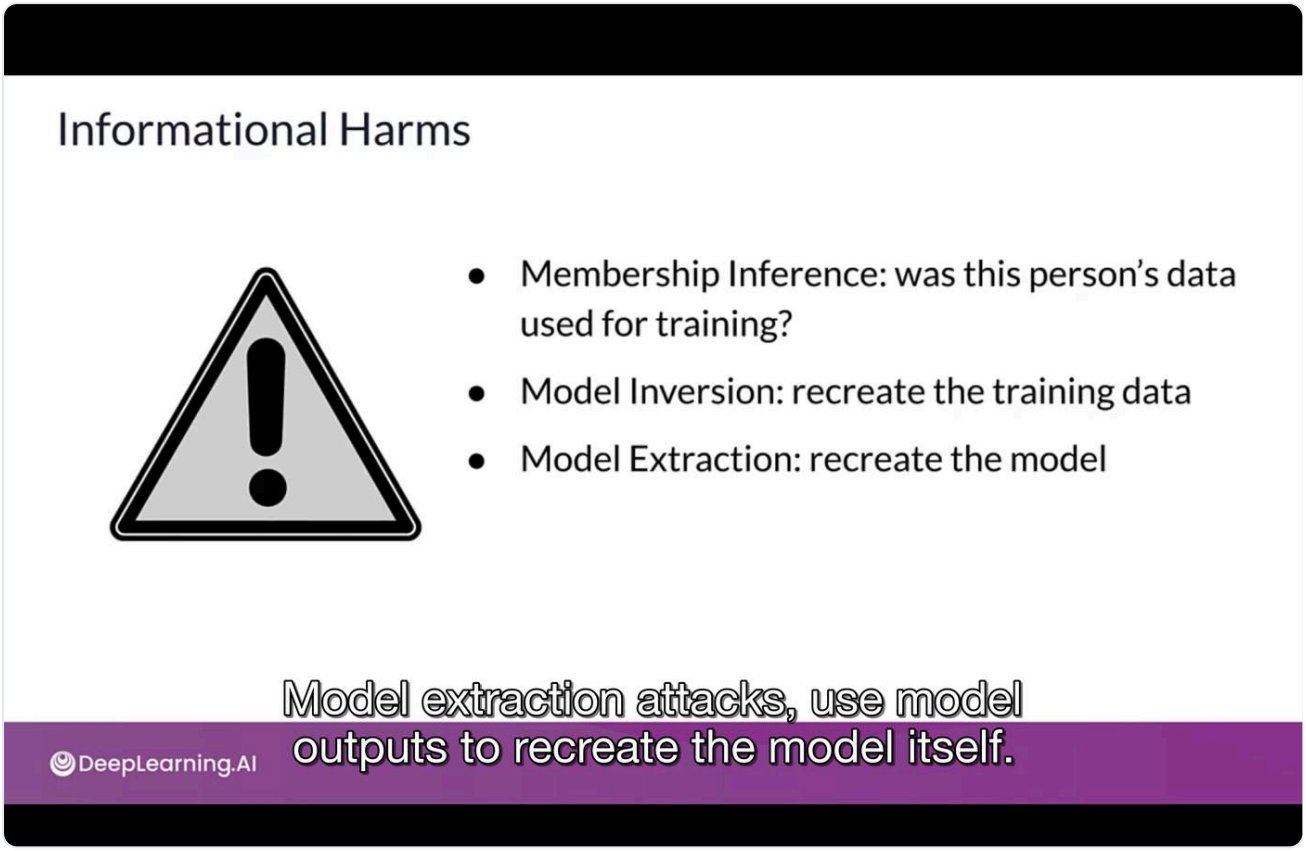

FIGURE 34.16 | Informational Harms

Model extraction attacks, use model outputs to recreate the model itself. This has been demonstrated against model as a service providers like big Ml and Amazon machine learning and can compromise privacy and security, as well as the intellectual property of the underlying model itself. And what about behavioral harms, model poisoning attacks occur when an adversary inserts malicious data into training data in order to alter the behavior of the model, for example, creating an artificially low insurance premium for particular individuals.

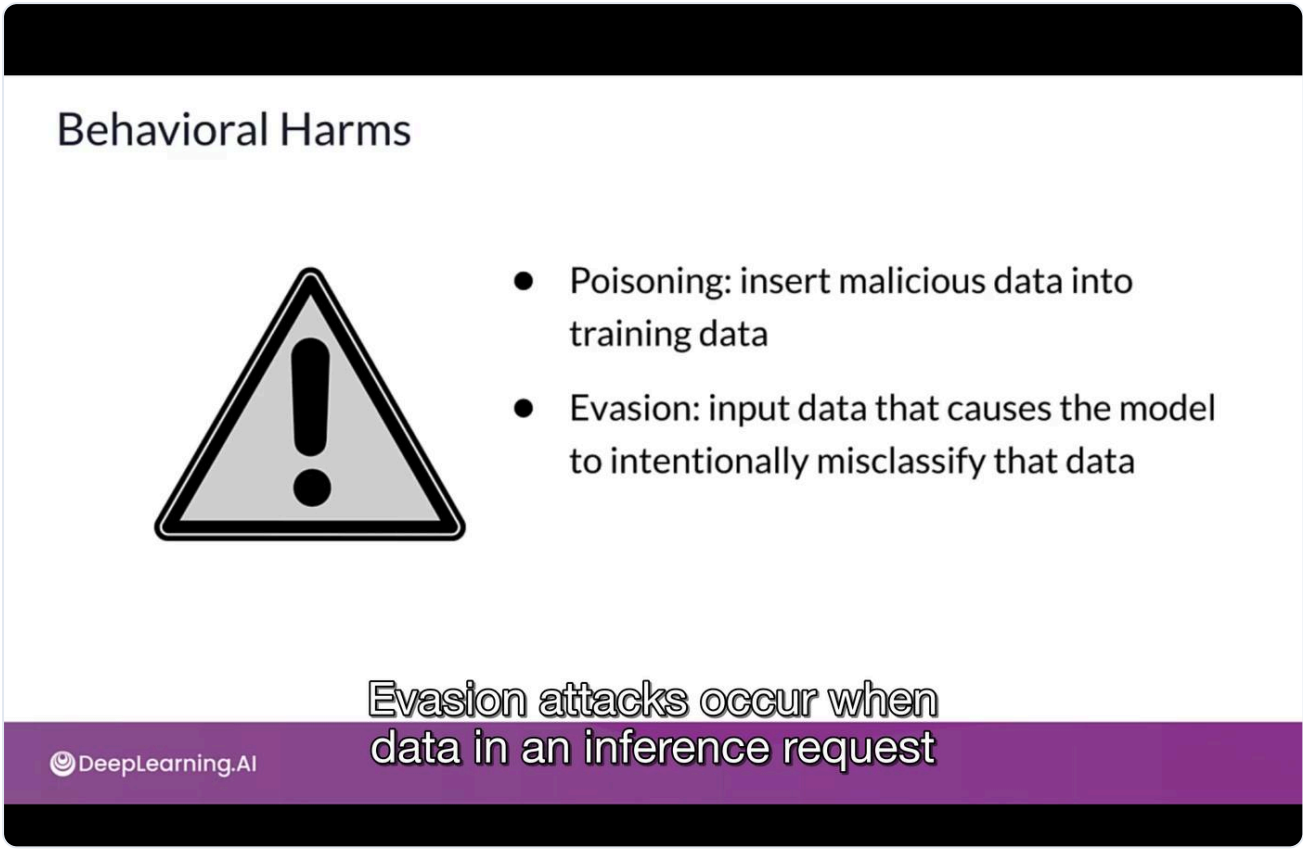

FIGURE 34.17 | Behavioral Harms

Evasion attacks occur when data in an inference request intentionally causes the model to miss classify that data. These attacks occur in a range of scenarios and the changes in the data may not be noticeable by humans. Our earlier example of an altered stop sign is one example of an evasion attack. Before hardening your models, you need to have some way of measuring the vulnerability to attack, clever hands is an open source python library that you can use to benchmark your models to measure their vulnerability to adversarial examples. To harden your model to adversarial attacks, what approach is to include sets of adversarial images in your training data so that the classifier is able to understand the various distributions of noise and your model learns to recognize the correct class.

This is known as adversarial training, examples created by tools such as clever hands can be added to your data set, but doing so limits your ability to use them to measure your models vulnerability.

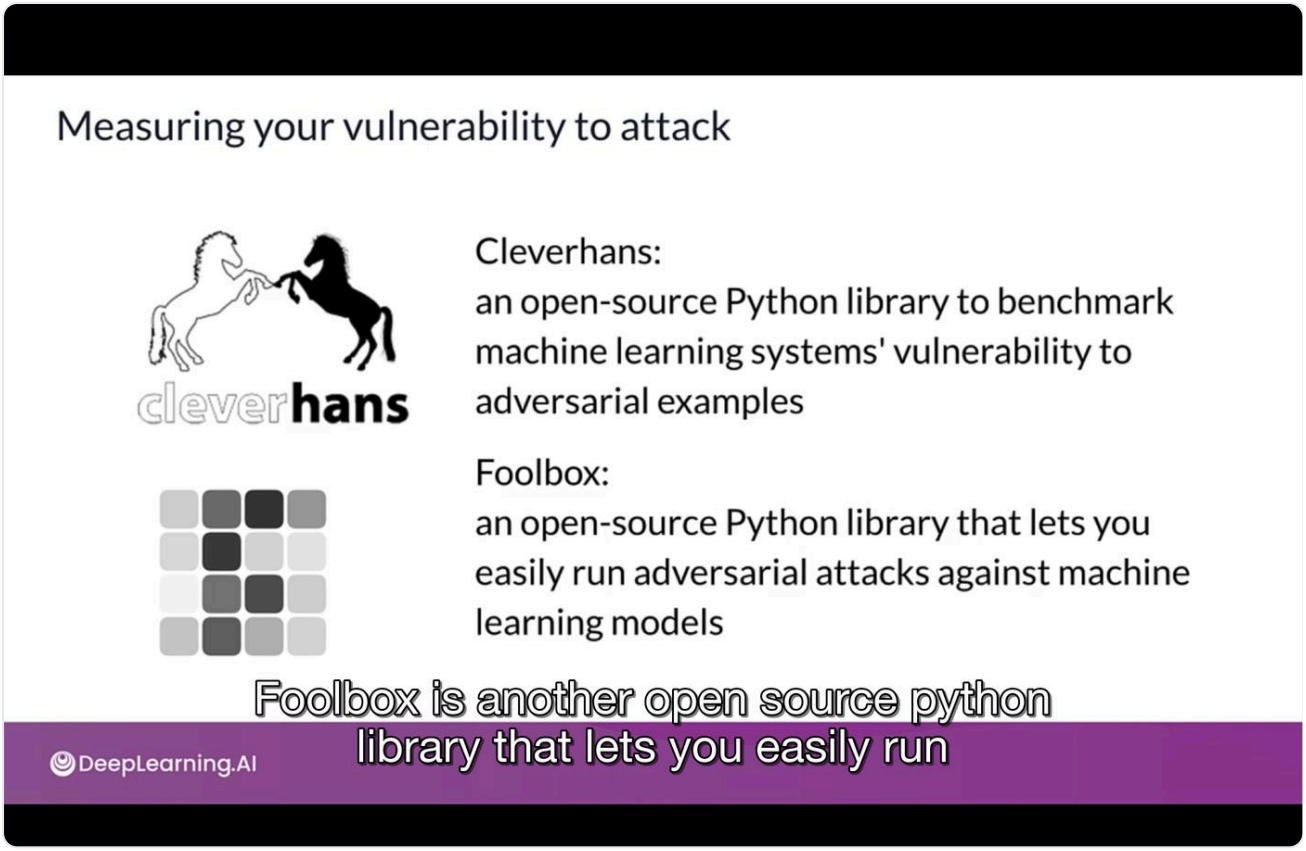

FIGURE 34.18 | Measuring Your Vulnerability to Attack

Since you are now almost testing with your training data, Foolbox is another open source python library that lets you easily run adversarial attacks against machine learning models like deep neural networks. It's built on top of eager pie and works natively with models in pytorch tensorflow and jacks. Unfortunately, detecting vulnerability is easier than fixing it. This is an emerging field and like many things in security, there is an arms race between attackers and defenders. One fairly advanced approach is defensive distillation.

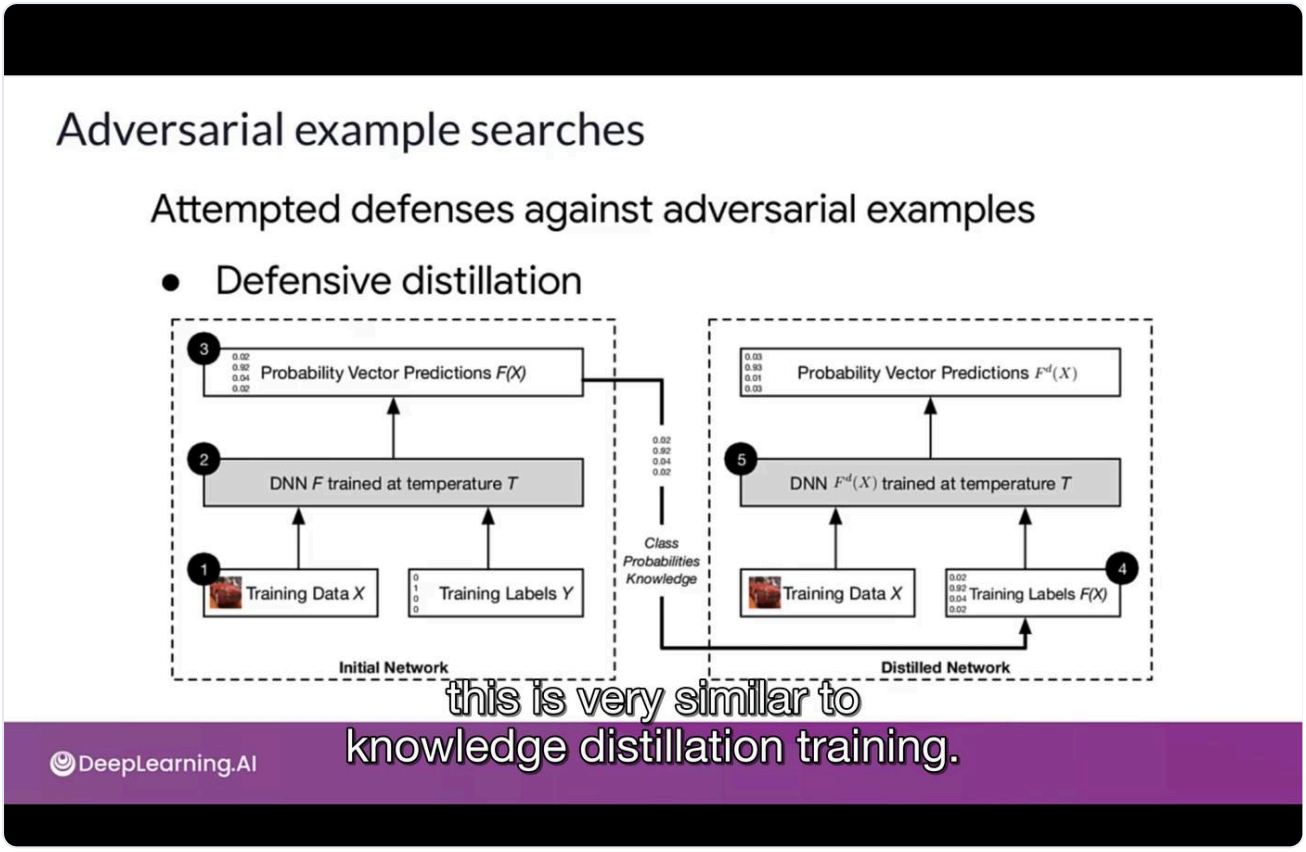

FIGURE 34.19

Since it does not use specific adversarial examples, it may provide more general hardening to new attacks, as the name suggests, this is very similar to knowledge distillation training. The goal is to increase model robustness and decrease sensitivity in order to decrease vulnerability to attacks. Defensive distillation reduced the effectiveness of a sample creation from 95% to less than 0.5% in one study. The main difference between defensive distillation and the original distillation proposed by Hinton and others that we discussed earlier in our model optimization section, is that we keep the same network architecture to train both the original network as well as the distilled network.

In other words, instead of transferring knowledge between different architectures, the authors proposed using knowledge distillation to improve a model's own resilience to attacks.

LECTURE 35

Create the Adversarial Image

Let's take a look at an attack on a model using an adversarial example, which uses the fast gradient sign method, which was described by Goodfellow and others in a kind of a landmark paper. So what is an adversarial example? Adversarial examples are specialized inputs created with the purpose of confusing and neural network, resulting in the misclassification of a given input. These notorious inputs are indistinguishable to the human eye, but cause the model to fail to identify the contents of the image. There are several types of such attacks.

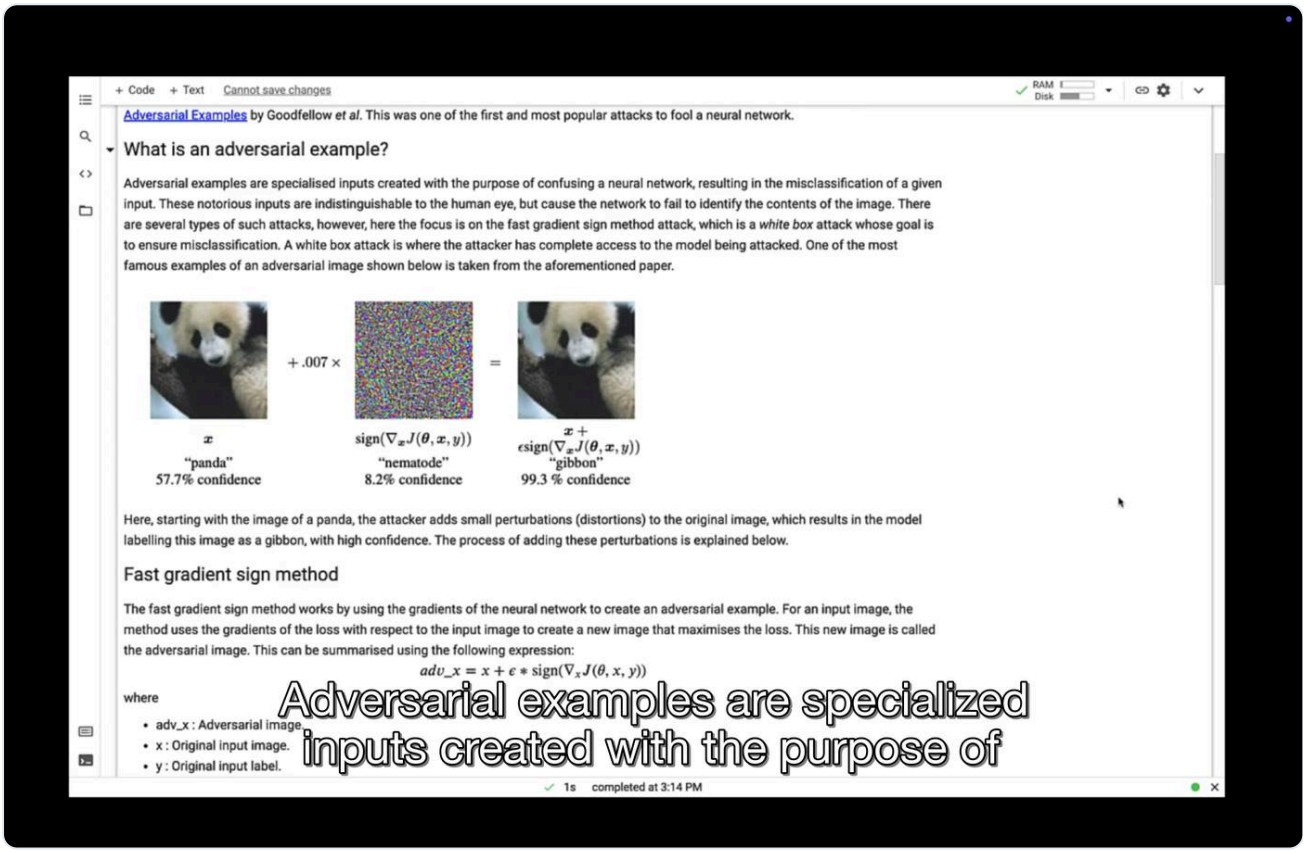

FIGURE 35.1

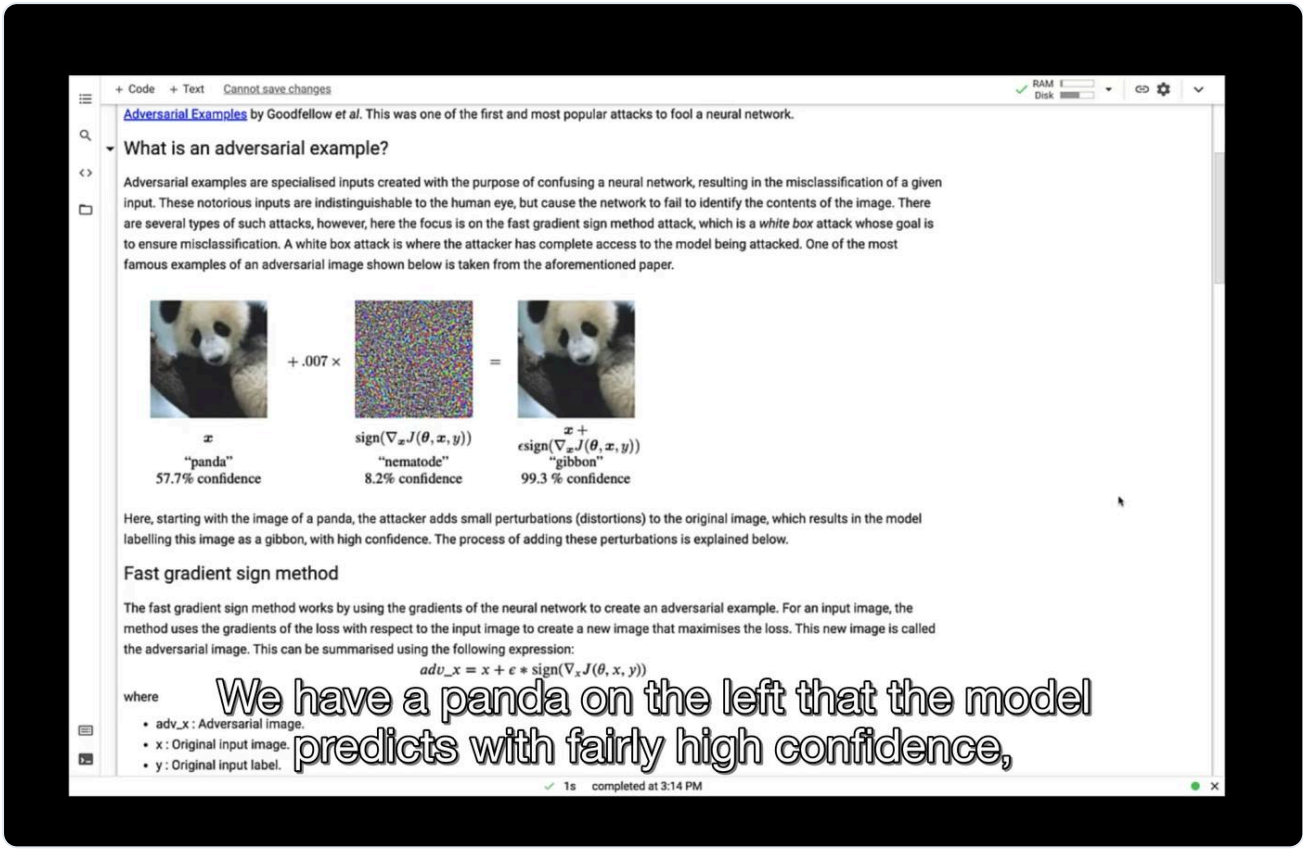

FIGURE 35.2

However, here the focus is on the fast gradient sign method attack, which is a white box attack whose goal is to ensure misclassification. A white box attack is where the attacker has complete access to the bottle being attacked. One of the most famous examples of an adversarial image is shown here, and it's taken from that paper. We have a panda on the left that the model predicts with fairly high confidence, some noise is added, which is fairly opaque to a human. And the result is that the model now thinks that that panda is a gibbon with a very high confidence.

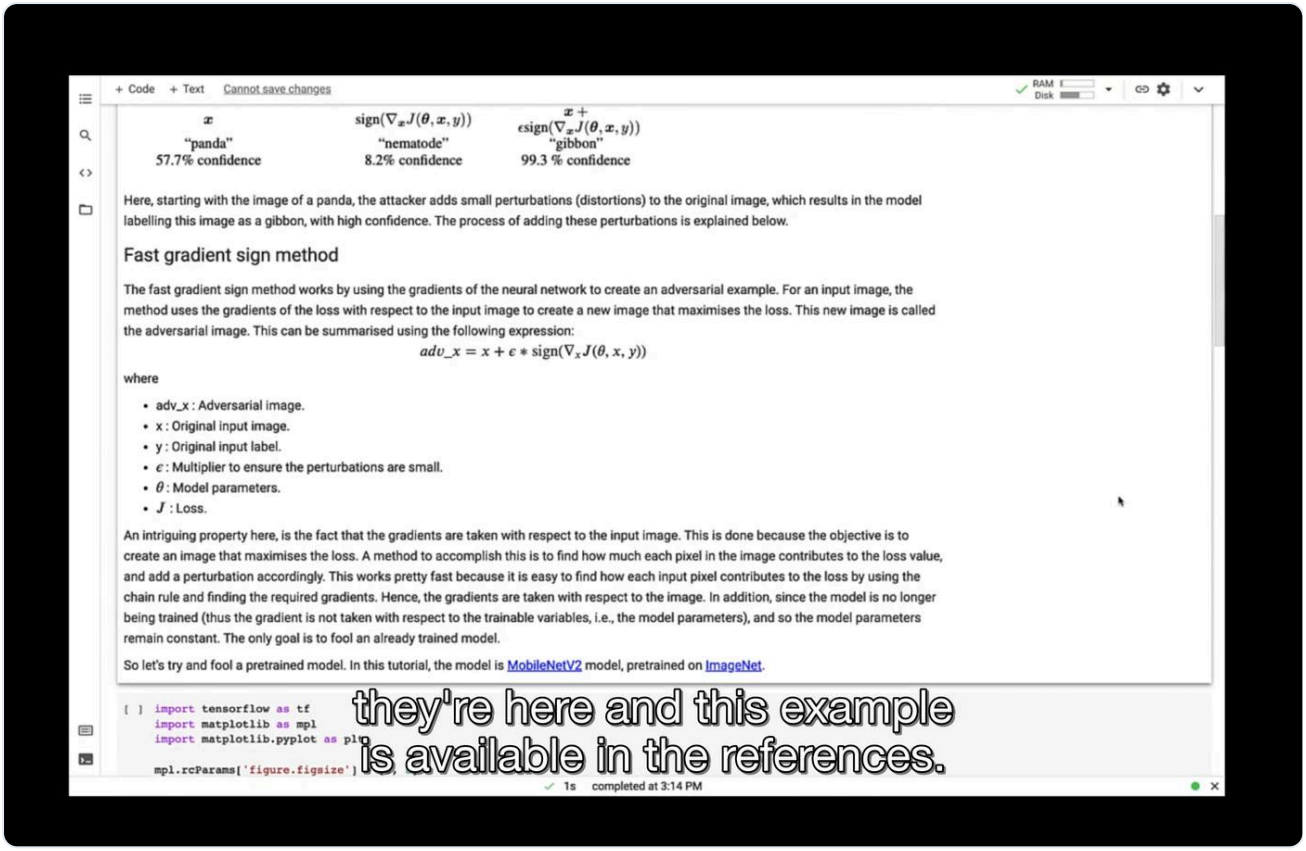

FIGURE 35.3

So I'm not going to go into the details of the attack method, they're here and this example is available in the references. You can go through it yourself. But what we will do is just walk through this example and show really how easy this kind of attack is.

FIGURE 35.4

So we're running into co lab here, we'll start by doing some imports of tensorflow and matplotlib, and then we'll load a pre-trained MobileNetV2 and the ImageNet class names. So this will take a second, there we go. And then, we have this helper function here to do pre-processing. And then, we'll take a look at the original image, which in this case is a Labrador Retriever. So we're going to download that data. And now let's look at the image, we'll do a plot here, and there it is a Labrador Retriever.

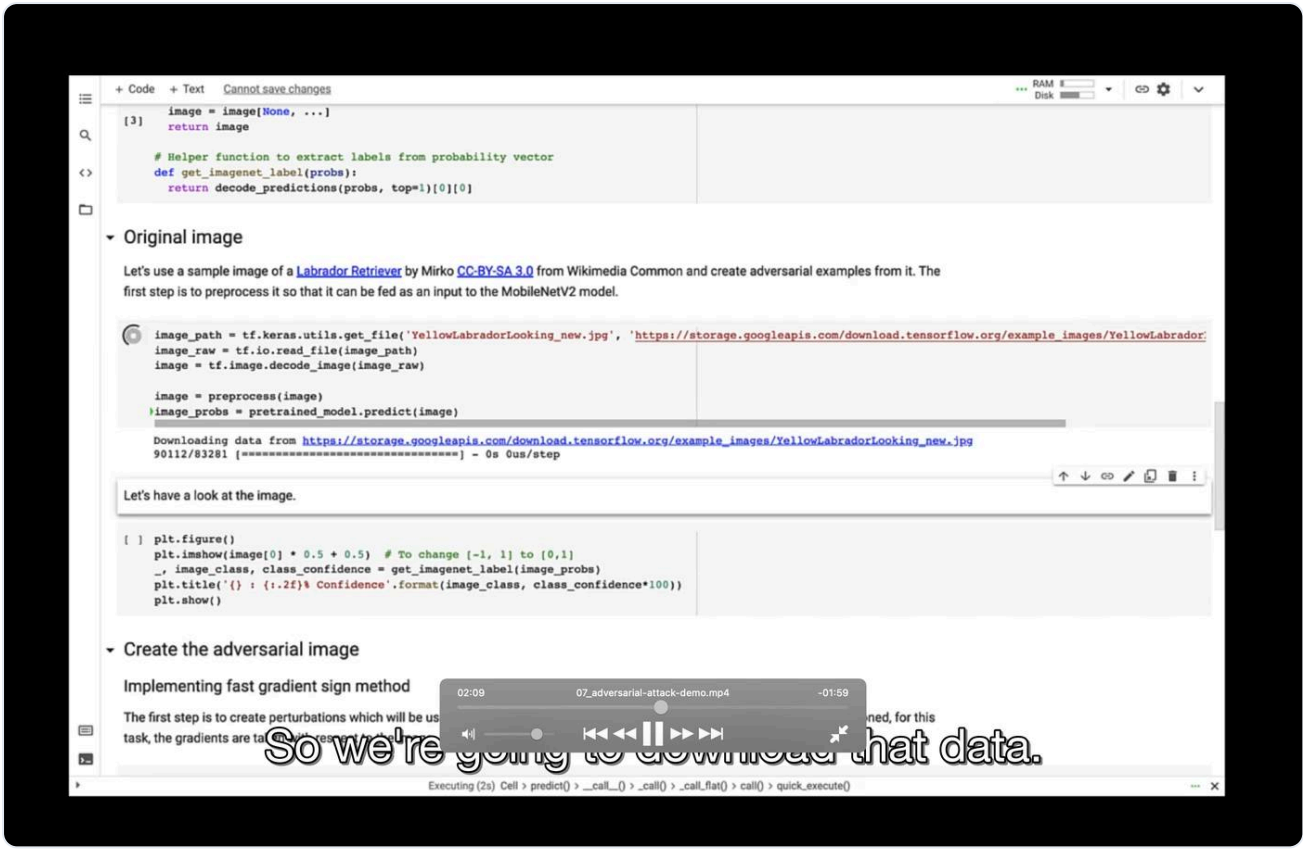

FIGURE 35.5

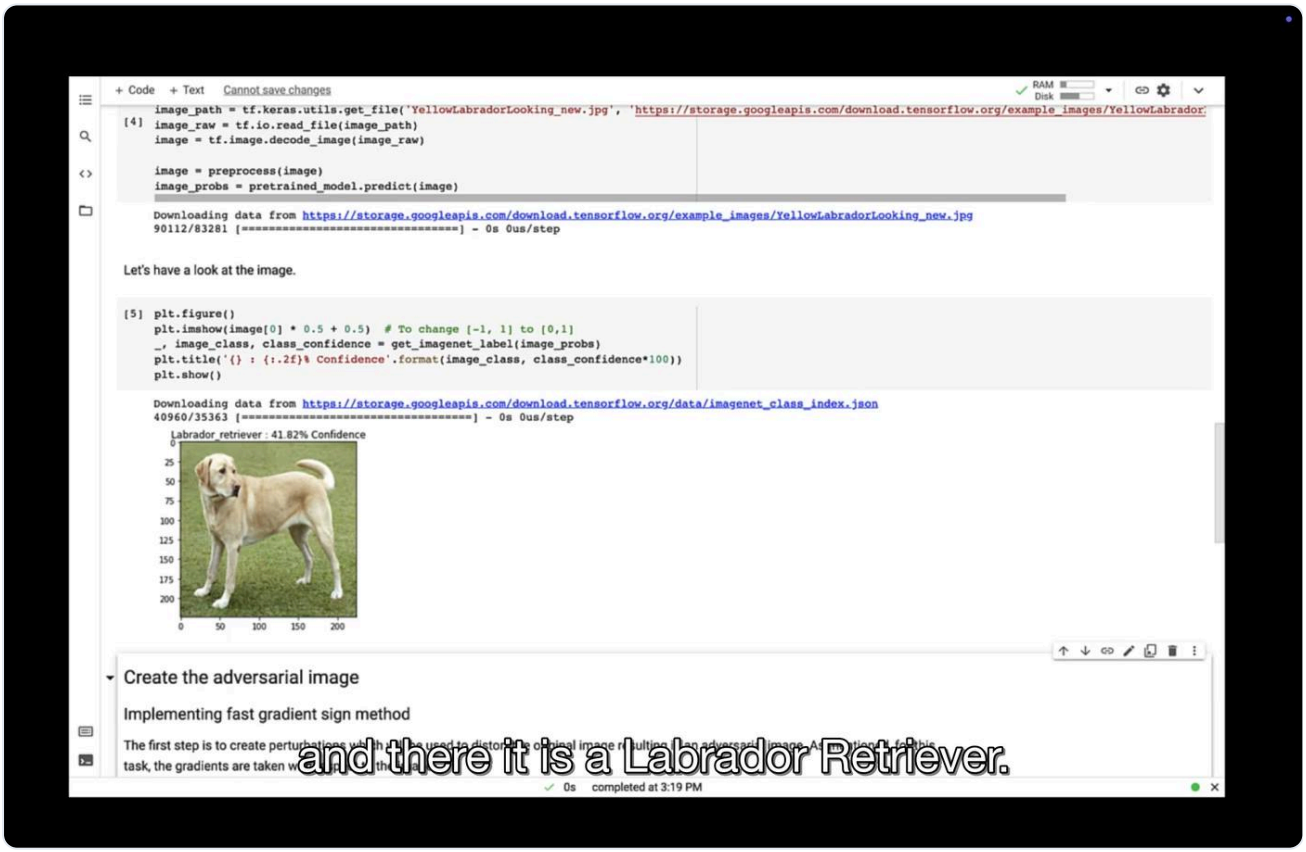

FIGURE 35.6

Okay, so we'll create an adversarial image using the fast gradient sign method.

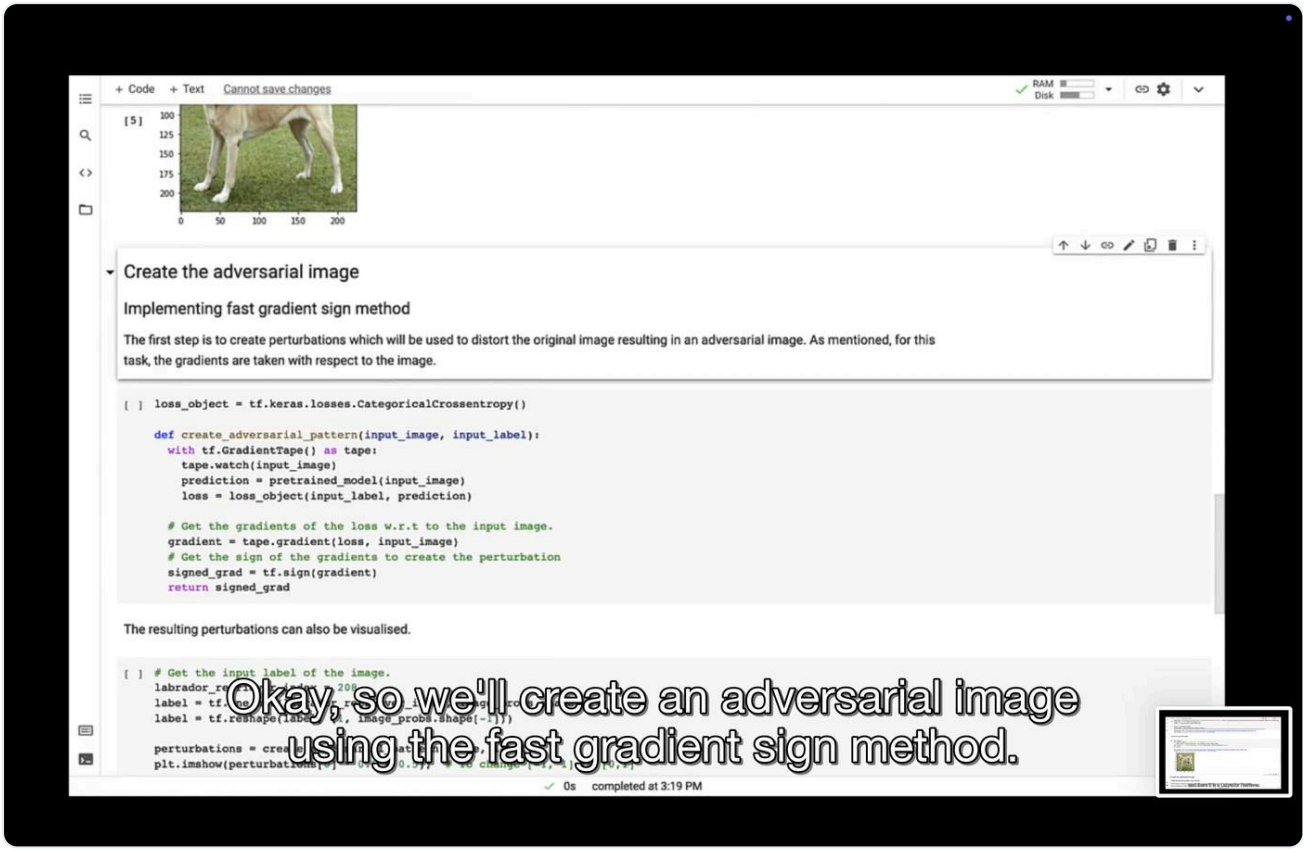

FIGURE 35.7 | Create the Adversarial Image

And there actually is the code that we'll use, you can see it's very simple.

FIGURE 35.8 | The Resulting Perturbations Can Also Be Visualised

And we can visualize the perturbations that that method makes to give you an idea. So looking at that, most people wouldn't expect it to have really any change in the model based on applying that, but as we'll see it does.

FIGURE 35.9

So one of the things we do is we're going to try different values for epsilon, which is one of the parameters of the method. And we'll see how different values of epsilon have different predictions that the model makes. So we have a helper method there to display the image, and then here's some different values of epsilon and the resulting model predictions.

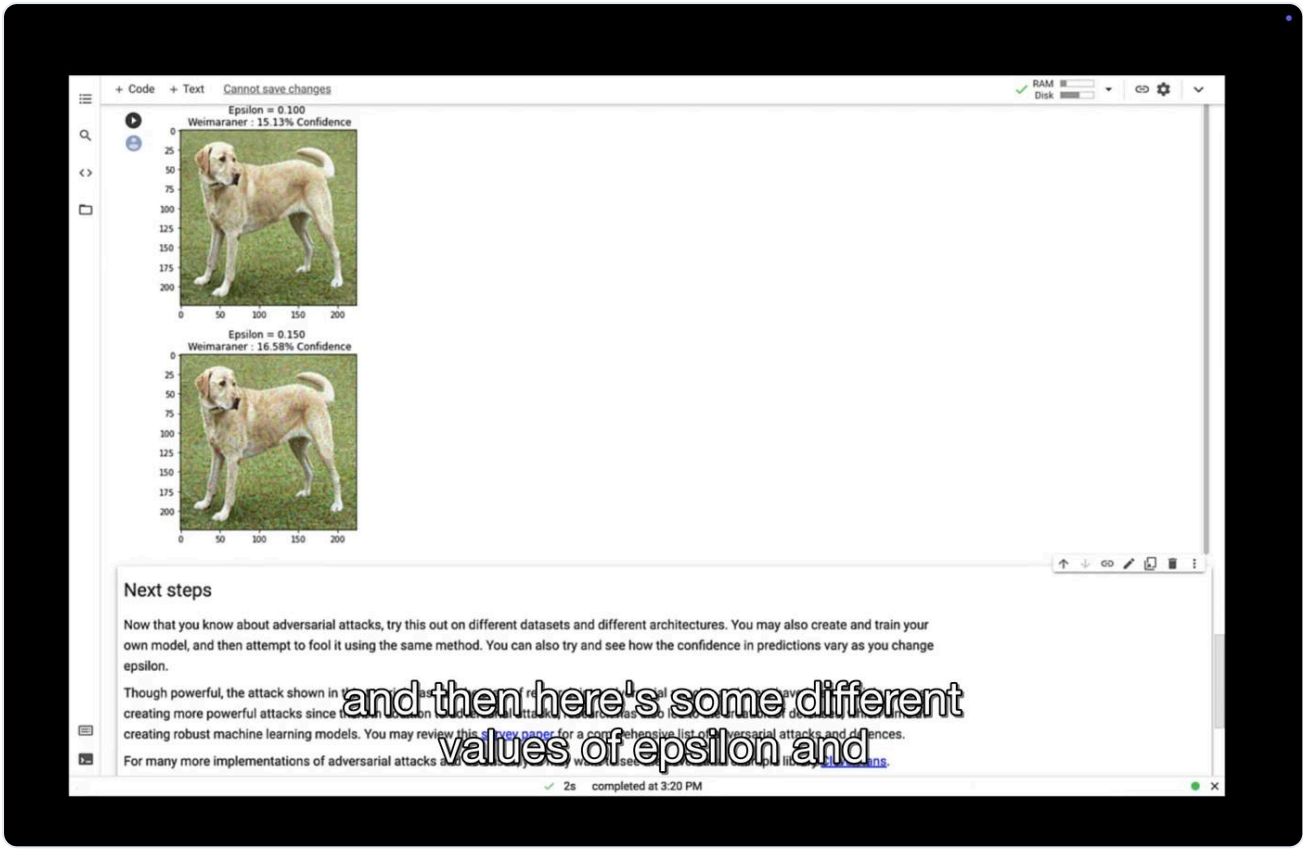

FIGURE 35.10

So there's our original Labrador Retriever confidences. This is actually, yeah, with the input and then with epsilon 0.01 and now thinks it's a Saluki which is a thin dog. And then a little higher value of epsilon.

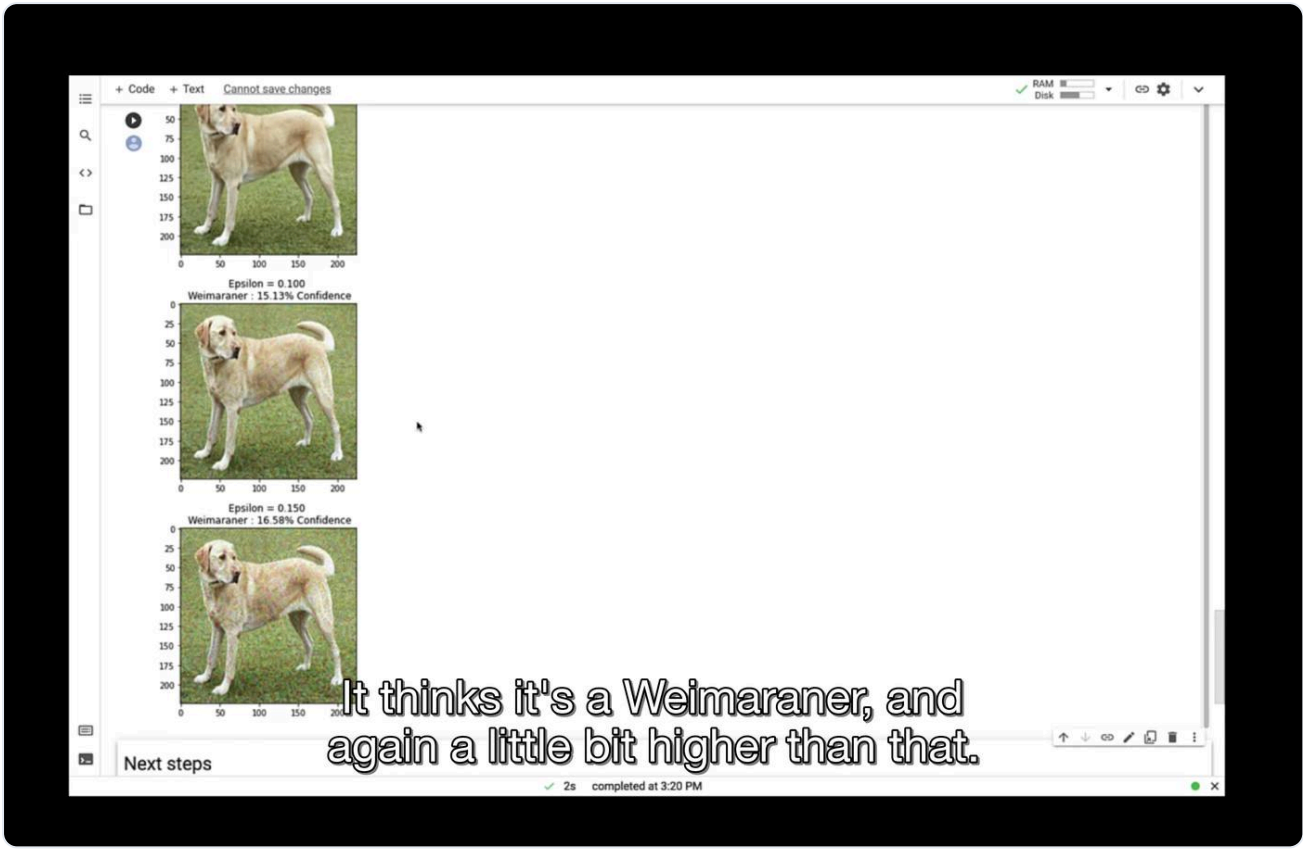

FIGURE 35.11

It thinks it's a Weimaraner, and again a little bit higher than that. It still thinks it's a Weimaraner with a little more confidence.

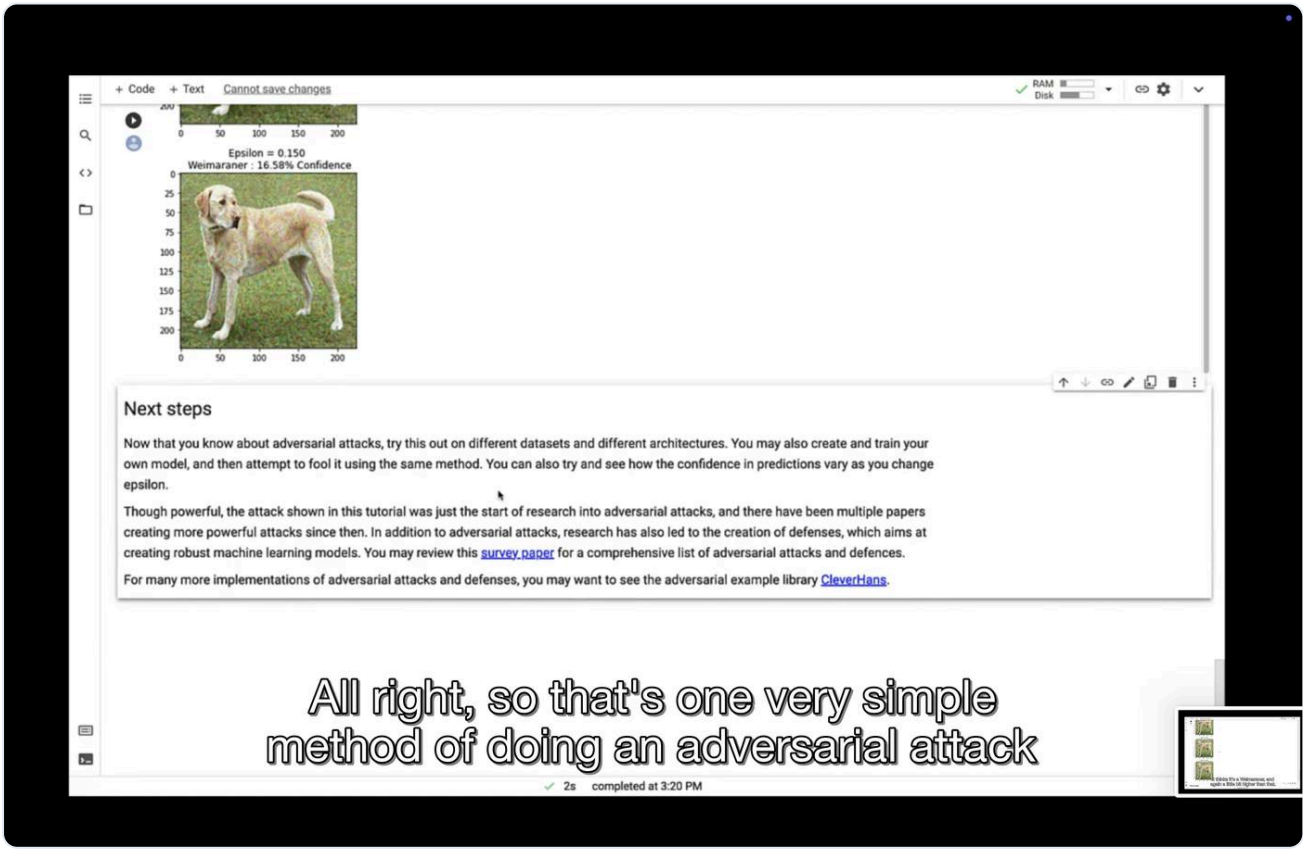

FIGURE 35.12

All right, so that's one very simple method of doing an adversarial attack that can fool the model into making an incorrect prediction based on image data.

LECTURE 36

Residual Analysis

Now let's discuss residual analysis, which is the last model debugging technique that will cover in this course. Another debugging technique is Residual Analysis. In most cases this can be used for regression models, since it requires measuring the distance between predictions and the ground truth. However, it requires having ground truth values for comparison, which can be difficult in many online or real time scenarios. In general, you want the residuals to follow a random distribution.

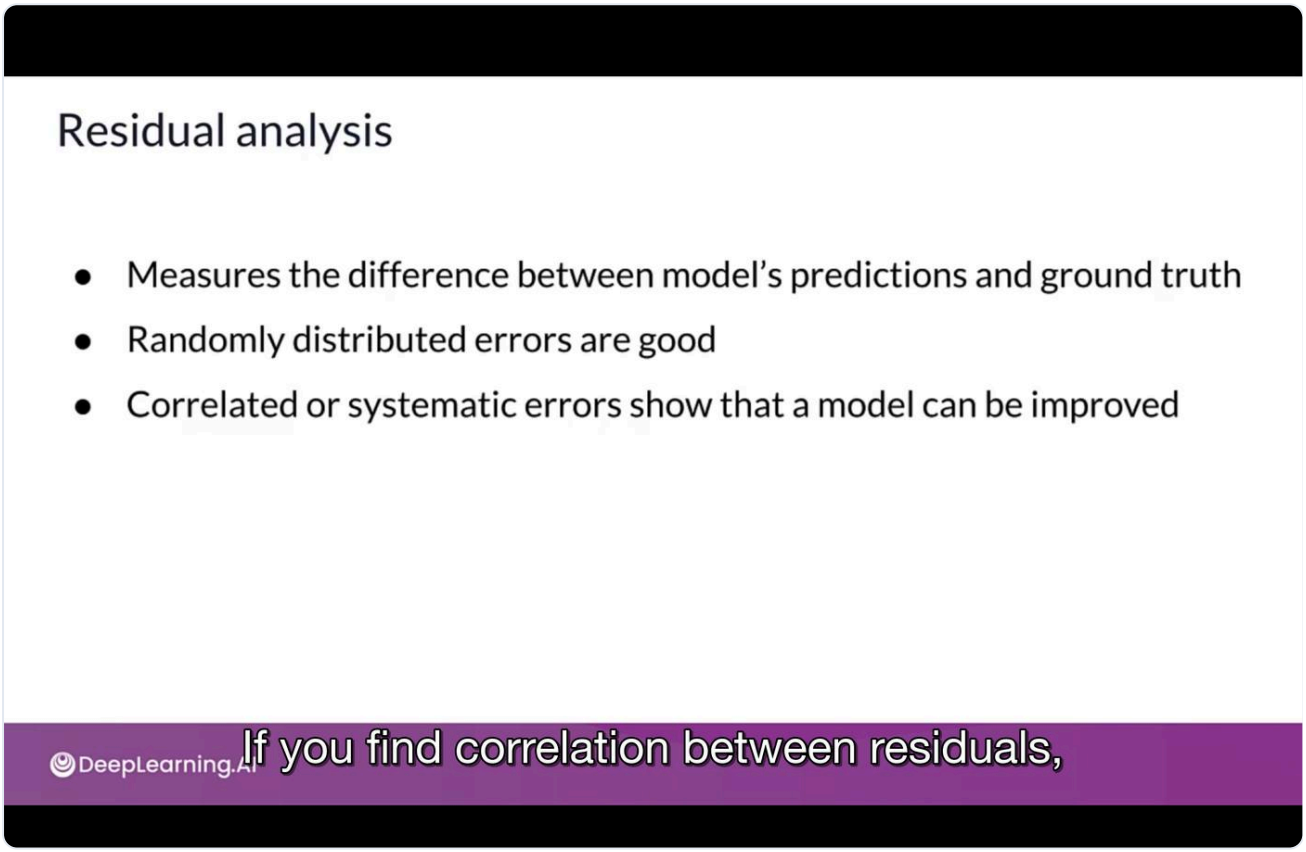

FIGURE 36.1 | Residual Analysis

If you find correlation between residuals, it's usually a sign that your model can be improved. If you've trained your aggression model, then you're probably familiar with metrics like root, mean, square, error or RMSE. And similar metrics, residual analysis uses those kinds of metrics but it primarily looks at the distribution of the errors. Often using residual plots to visually inspect the distribution of residuals. If your model is well trained and has captured the predictive information in the data, then the residuals should be randomly distributed.

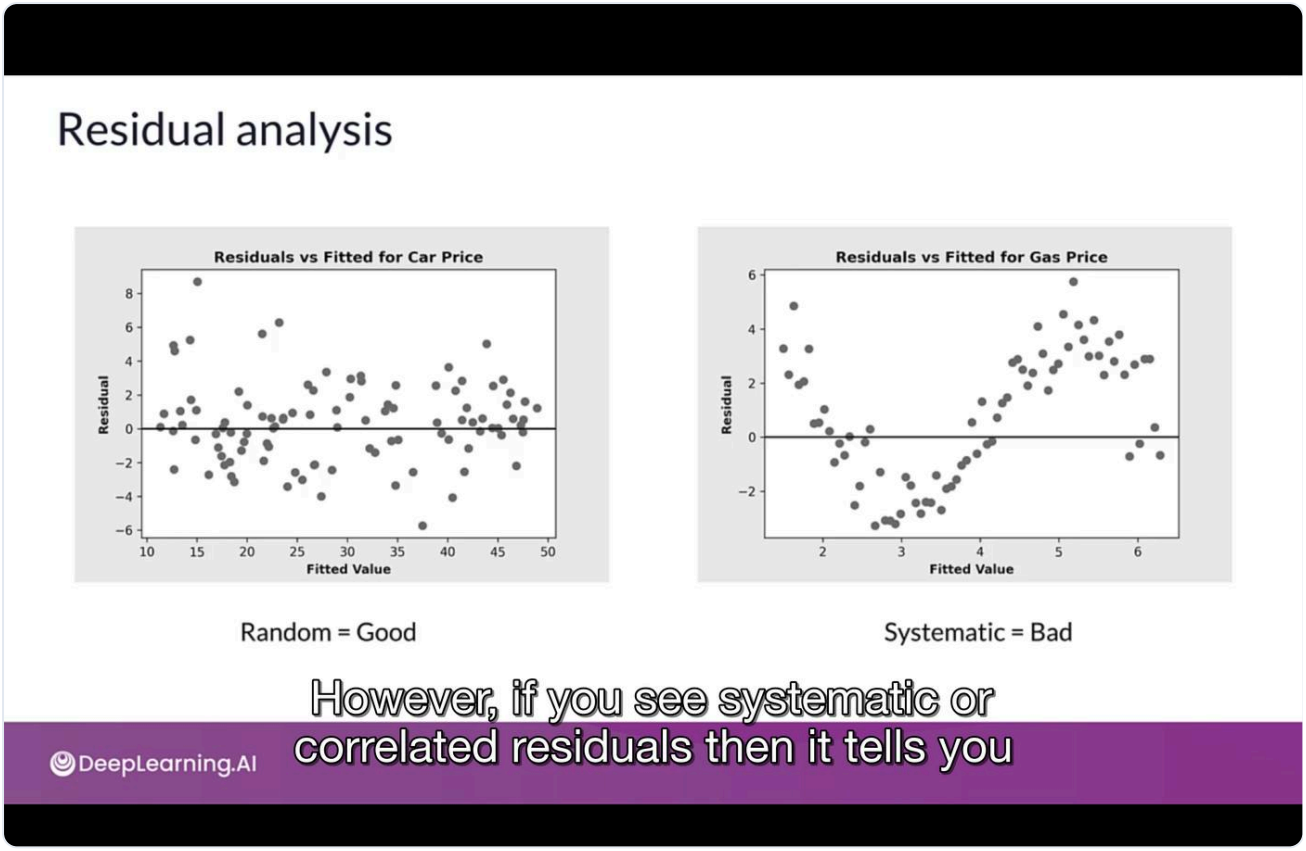

FIGURE 36.2 | Residual Analysis

However, if you see systematic or correlated residuals then it tells you that there is predictive information that the model has not captured. You can then look for ways to improve your model. So what should you aim for when performing Residual Analysis? The residuals should not be correlated with another feature that was available but was left out of the feature vector. If you can predict the residuals with another feature, that feature should be included in the feature vector. This requires checking the unused features for correlation with the residuals. Also, adjacent residuals should not be correlated with each other, in other words, they should not be autocorrelated.

If you can use one residual to predict the next residual, then there's some predictive information that is not being captured by the model.

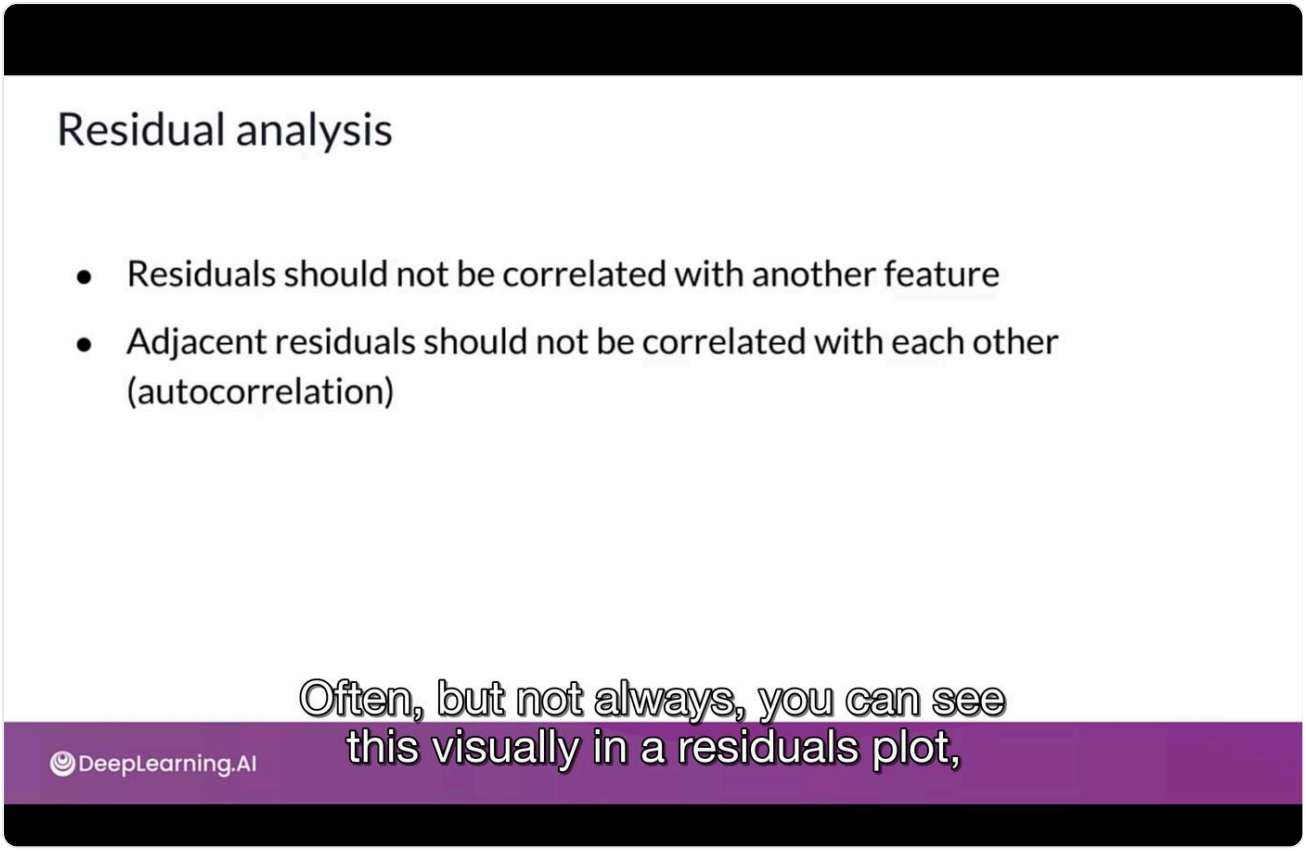

FIGURE 36.3 | Residual Analysis

Often, but not always, you can see this visually in a residuals plot, ordering can be important for understanding this. For example, if a residual is more likely to be followed by another residual that has the same sign, adjacent residuals are positively correlated. Performing a Durbin Watson test is also useful for detecting autocorrelation.

LECTURE 37

Remediation Techniques

so far we've discussed ways to analyze model robustness but we haven't talked about ways to improve it. Let's do that now with a discussion of model remediation. So what can you do to improve robustness? You should make sure that your training data accurately mirrors a request that you'll receive for your trained model. But data augmentation can also help your model generalize, which typically reduces sensitivity. You can generate data in many ways, including generative techniques, interpretive techniques or simply adding noise to your data. Data augmentation is also a great way to help correct for unbalanced data.

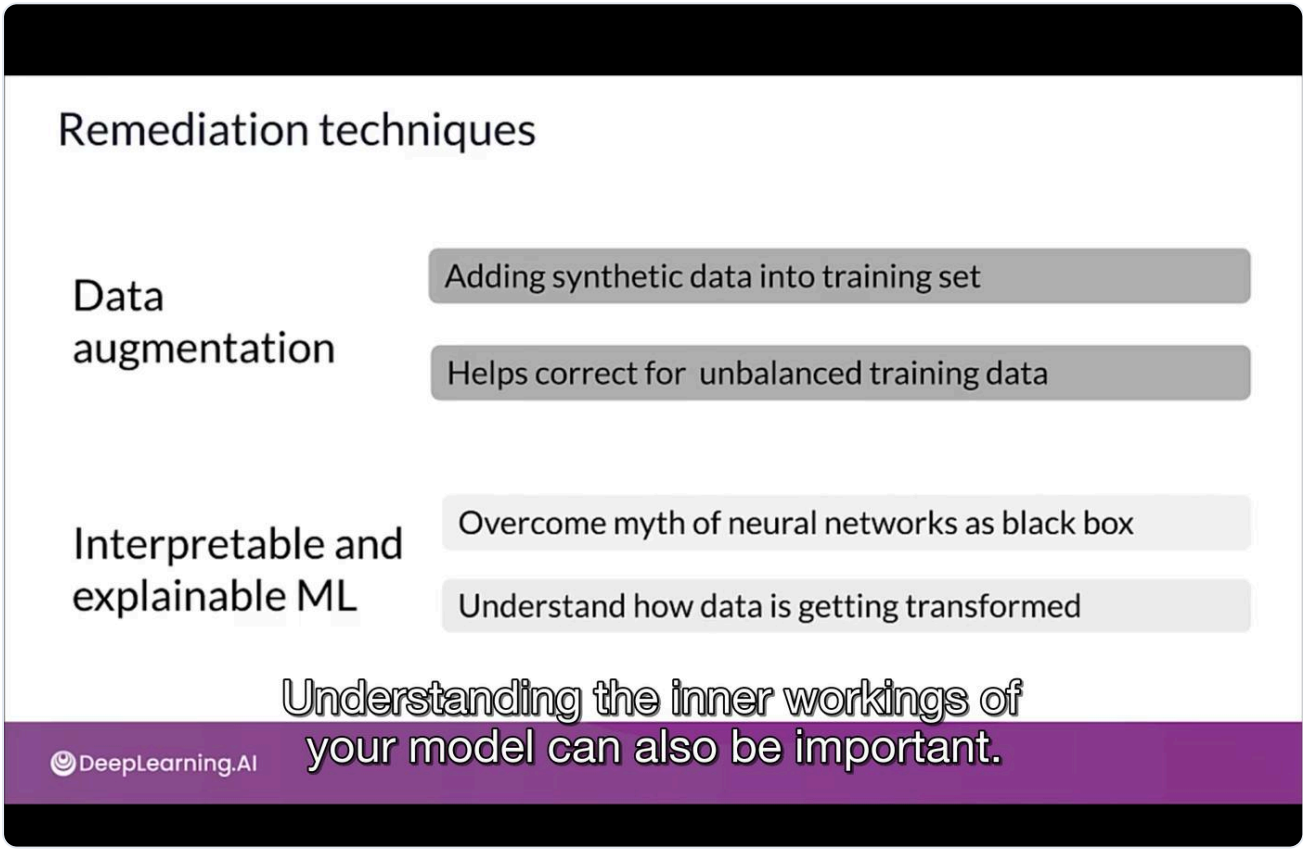

FIGURE 37.1 | Remediation Techniques

Understanding the inner workings of your model can also be important. Often more complex models can be black boxes and we sometimes don't make much effort to understand what is happening internally. However, there are tools and techniques which can help with model interpret ability and this can also help with improving model robustness. There are also model architectures which are more easily interpreted. Including tree based models as well as neural network models which are specifically designed for interpret ability. Another remediation technique is model editing. Some models such as decision trees are so directly interpret double that the learned parameters can be understood easily.

If you find that something is going wrong, you can tweak the model to improve its performance and robustness. Another technique is model assertions.

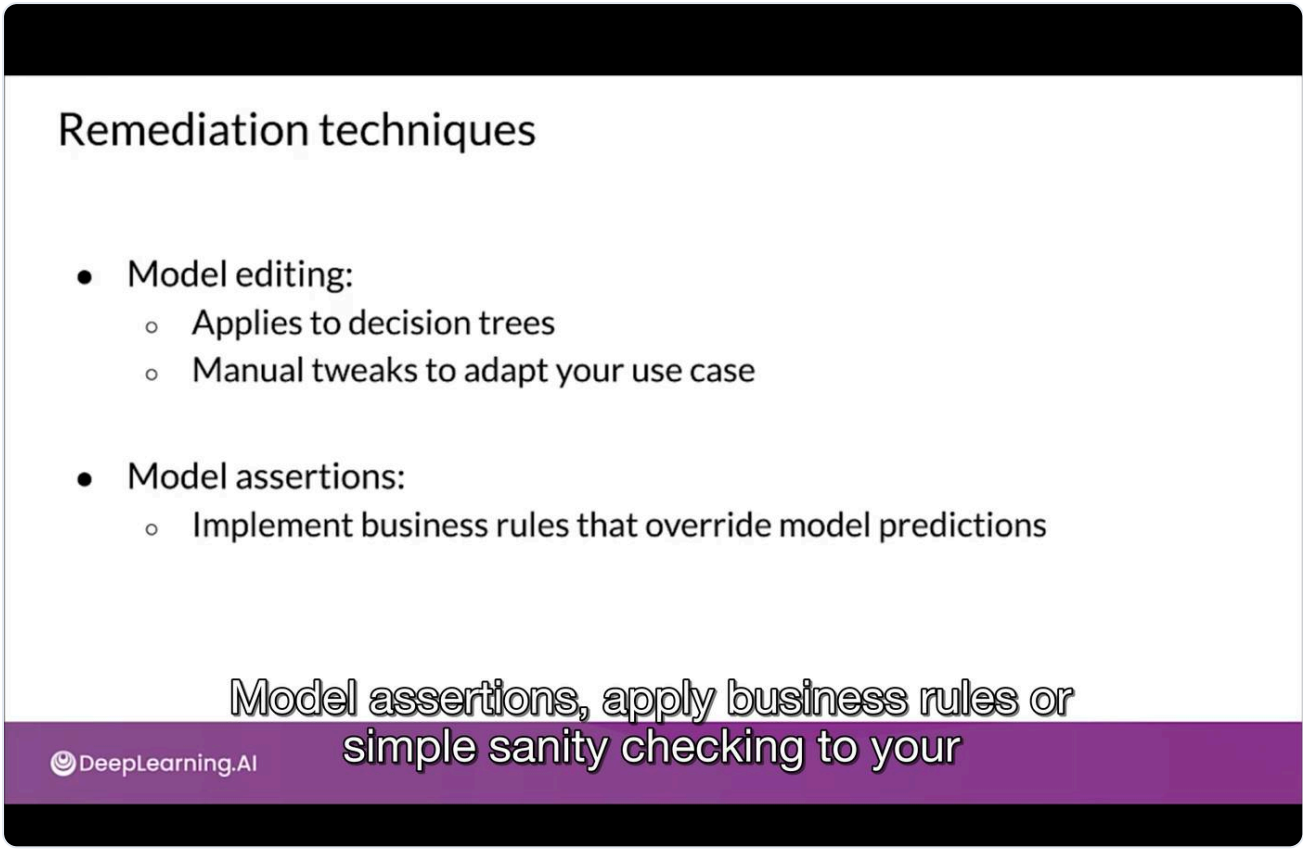

FIGURE 37.2 | Remediation Techniques

Model assertions, apply business rules or simple sanity checking to your models results and either alter or bypass the results before delivering them. For example, if predicting someone's age numbers should never be negative or if predicting a credit limit. Then it should never be more than a maximum amount. Next let's look at how model bias can be reduced or eliminated. This is discrimination remediation. The best solution for this is to have a diverse data set which is representative of the people who will be using your model. It also helps to have people on the development team from diverse backgrounds with expertise in ethics, privacy, social sciences and other related disciplines.

Careful feature selection, including sampling and reweighting rose to minimize discrimination in training data can also be helpful when training. You should consider fairness metrics when selecting hyper parameters and decision cut off thresholds.

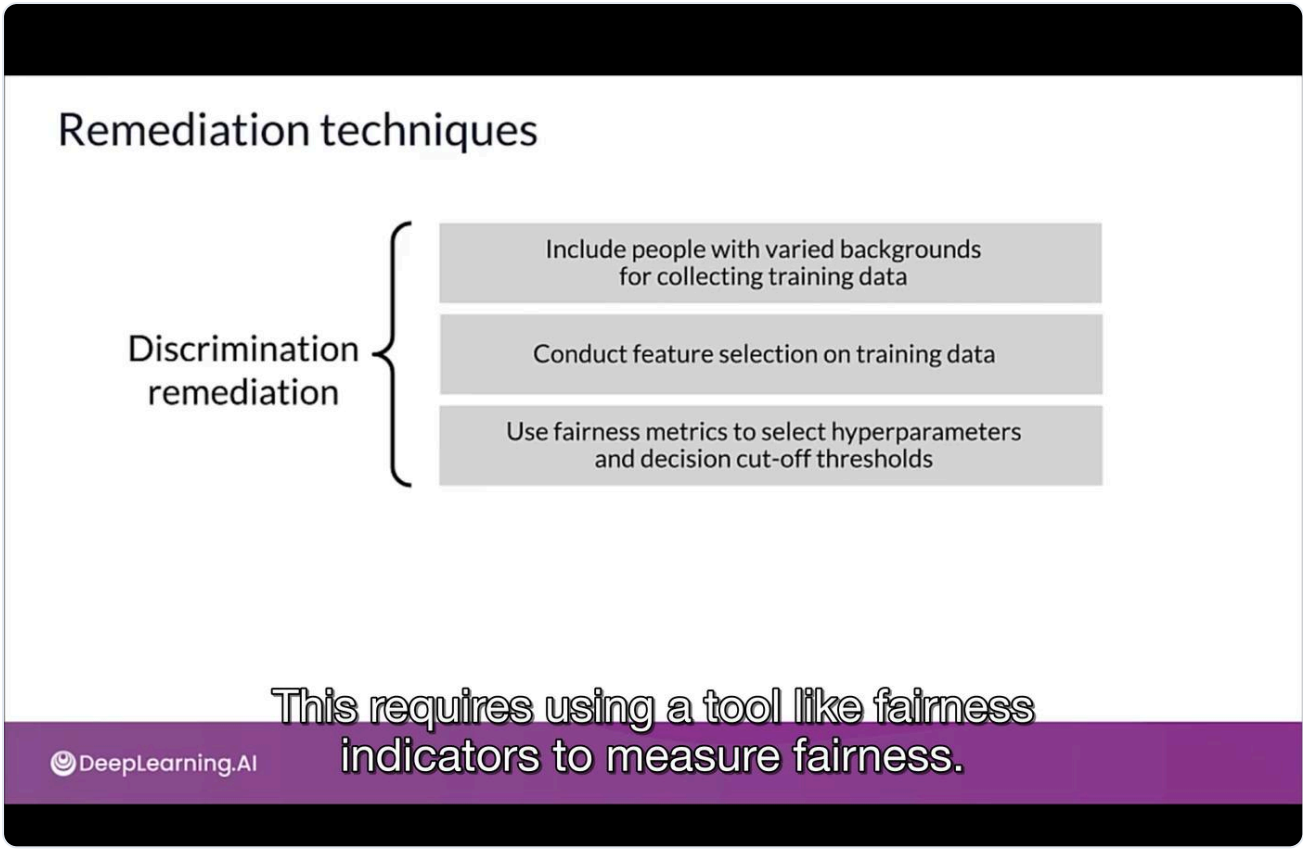

FIGURE 37.3 | Remediation Techniques

This requires using a tool like fairness indicators to measure fairness. This may also involve training fair models directly by learning fair representations or LFR and adversarial de biasing using tools such as AIF360. Or using dual objective functions that consider both accuracy and fairness metrics for a prediction post processing. Changing model predictions after training, reject option classification tools such as AIF 360. Or theme SML can also help to reduce unwanted bias tools like google model remediation library can help with improving model fairness as well. Check the references at the end of the lesson for links to libraries and resources.

Model debugging is not something that you only do during development. This requires constant monitoring of your model. The accuracy fairness or security characteristics of MM models will change during the lifetime of a model as the data and the world changes and as new attacks emerge. You should build monitoring into your process so that your models are checked for accuracy fairness and security problems on a regular basis.

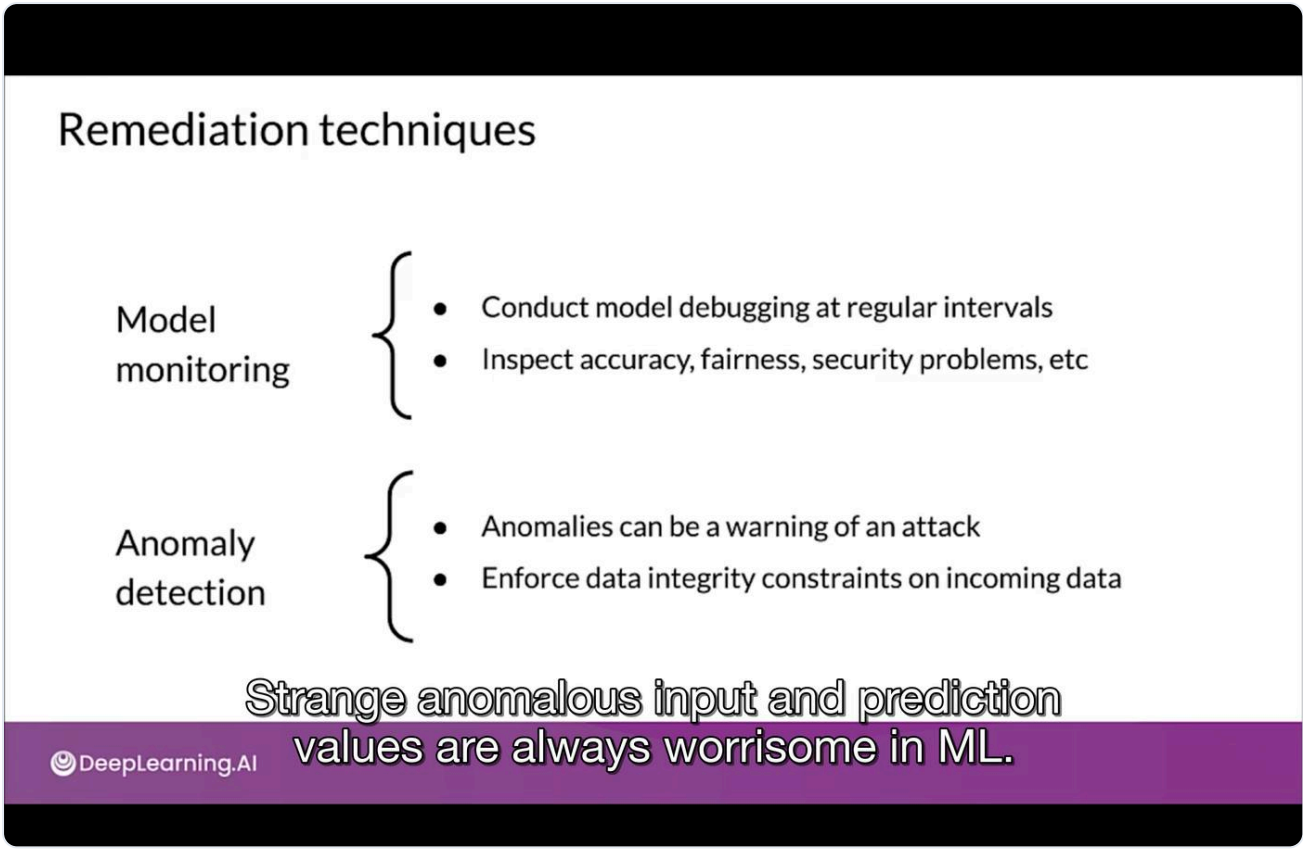

FIGURE 37.4 | Remediation Techniques

Strange anomalous input and prediction values are always worrisome in ML. And can be indicative of an attack, luckily an almost inputs. And predictions can be caught and corrected in real time using a variety of tools and techniques including data integrity constraints on input streams. Statistical process control methodologies on inputs and predictions anomaly detection through auto encoders and isolation for us. And also by comparing your model's prediction to benchmark model predictions

LECTURE 38

Fairness Indicators

In this section, we'll focus on how to make models fair and look at using fairness indicators library to assess fairness. Remember that in addition to serving your community well, focusing on fairness helps you serve different types of customers or situations well. In addition to analyzing and improving your model performance, you want to introduce checks and controls to ensure that your model behaves fairly in different scenarios. Accounting for fairness and reducing bias towards any group of people is an important part of that. You need to make sure that your model is not causing harm to the people who use it. Fairness indicators is an open-source library built by the TensorFlow team to easily compute commonly identified fairness metrics for binary and multiclass classifiers.

Fairness indicators scales.

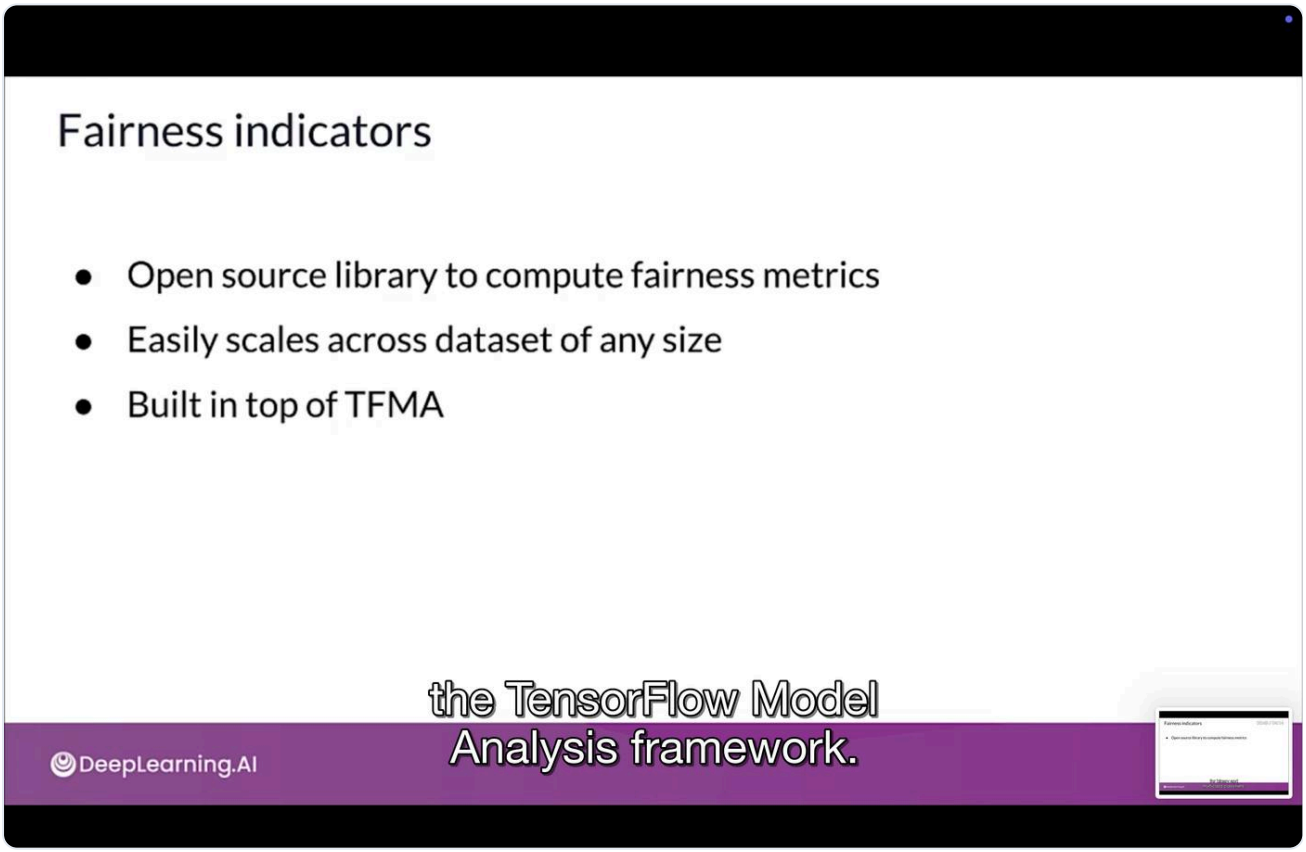

FIGURE 38.1 | Fairness Indicators

Has been built on top of the TensorFlow Model Analysis framework. What are the main features of the fairness indicators library? Fairness indicators is the library that enables easy measurement of commonly identified fairness metrics for binary and multiclass classifiers. With the fairness indicators tool suite, you can also compare model performance across subgroups to a baseline or to other models. This includes using confidence intervals to surface statistically significant disparities and performing evaluation over multiple thresholds.

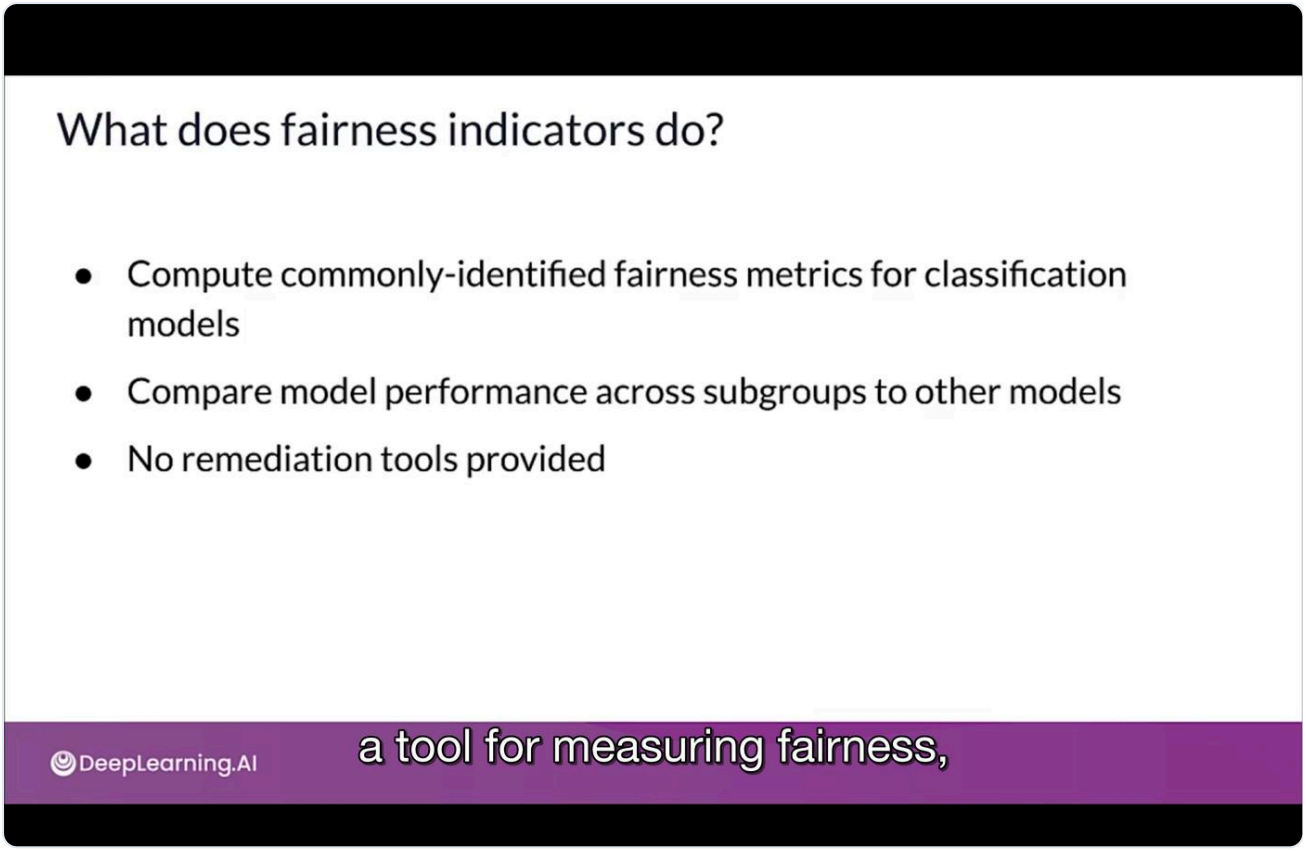

FIGURE 38.2

Fairness indicators is primarily a tool for measuring fairness, not for doing remediation to improve fairness. Looking at slices of data is actually quite informative when you're trying to mitigate bias and check for fairness. When measuring fairness, it's important to identify slices of data which are sensitive to fairness and measure your model's performance on those slices. If you only measure fairness using the entire dataset, it can easily hide fairness problems with particular groups of people. It's also important to consider and select the right metrics to measure for your dataset and users.

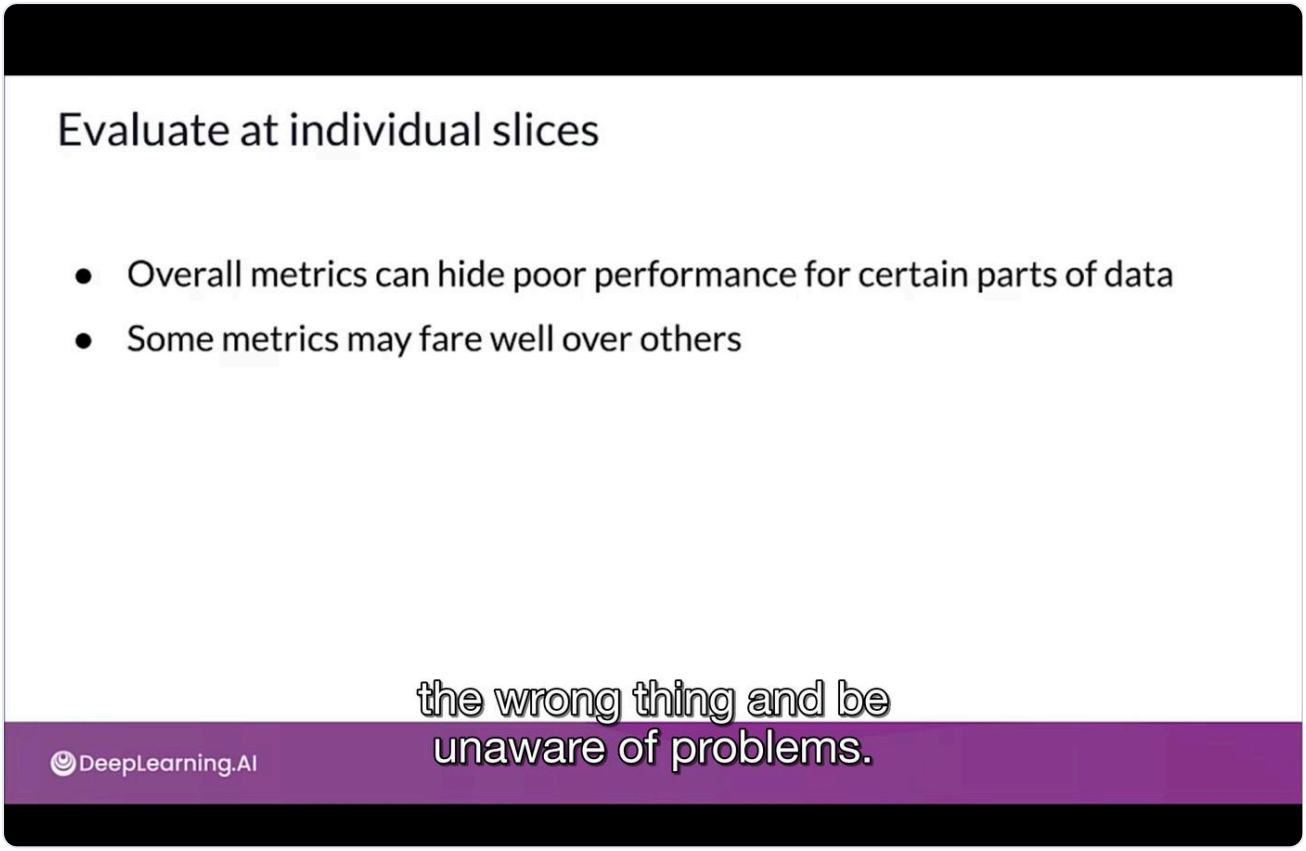

FIGURE 38.3 | Evaluate at Individual Slices

Because otherwise, you may measure the wrong thing and be unaware of problems. This is often based on domain knowledge. Keep in mind that measuring fairness is only one part of evaluating a broader user experience. Start by thinking about the different contexts through which a user may experience your application. Who are the different types of users for your application? Who else may be affected by the experience? It's important to remember that human societies are extremely complex. Understanding people and their social identities, social structures, and cultural systems are each huge fields of open research.

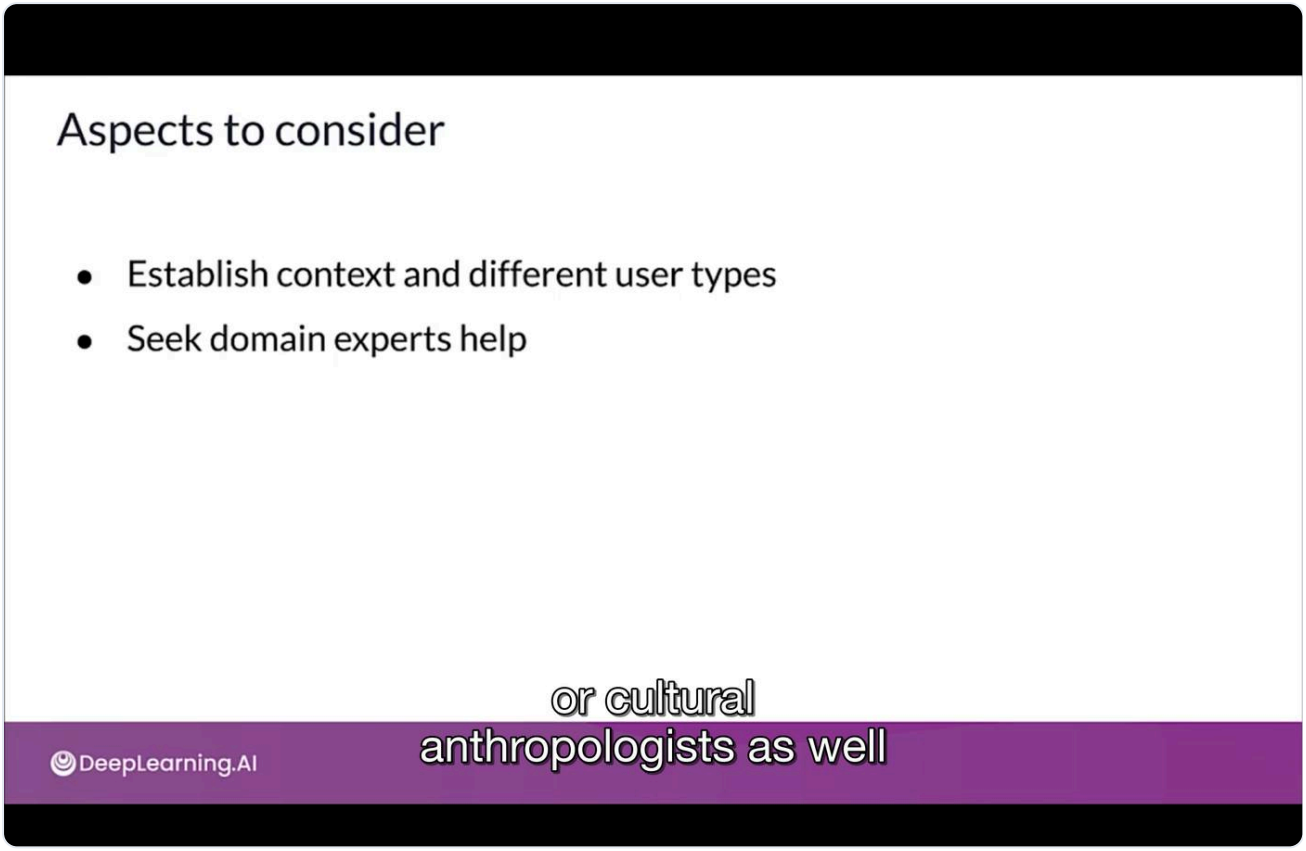

FIGURE 38.4

Whenever possible, we recommend talking to appropriate domain experts, which may include social scientists, sociolinguists, or cultural anthropologists as well as members of the communities that will be using your application.

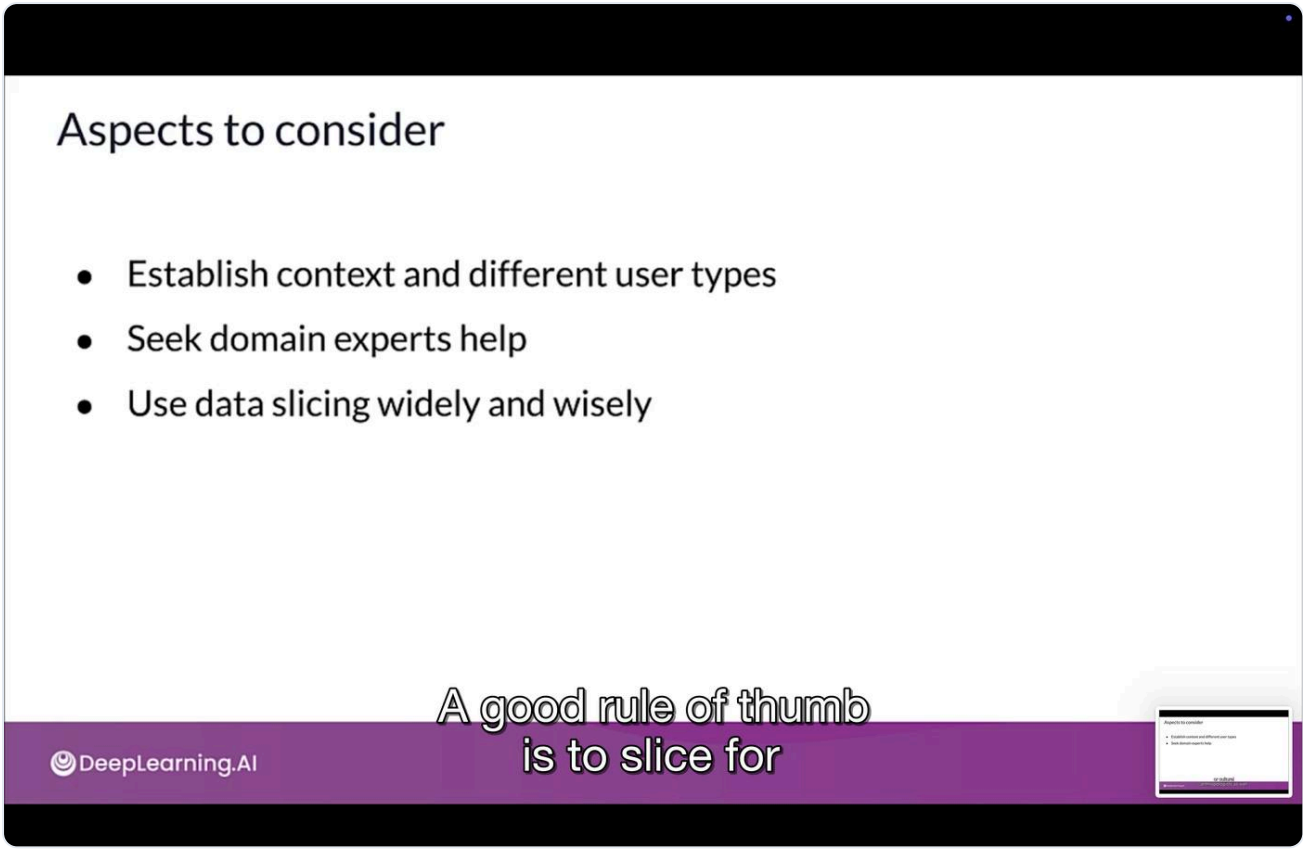

FIGURE 38.5

A good rule of thumb is to slice for as many groups of data as possible. Pay special attention to slices of data that deal with sensitive characteristics such as race, ethnicity, gender, nationality, incomes, sexual orientation, and disability status. Ideally, you should be working with labeled data. But if not, then you can apply statistics to look at the distributions of the outcomes with some assumptions around any expected differences. Let's look at some important guidelines to avoid common pitfalls when working with fairness indicators. When you're just getting started with fairness indicators, you should conduct various fairness tests on all the available slices of data.

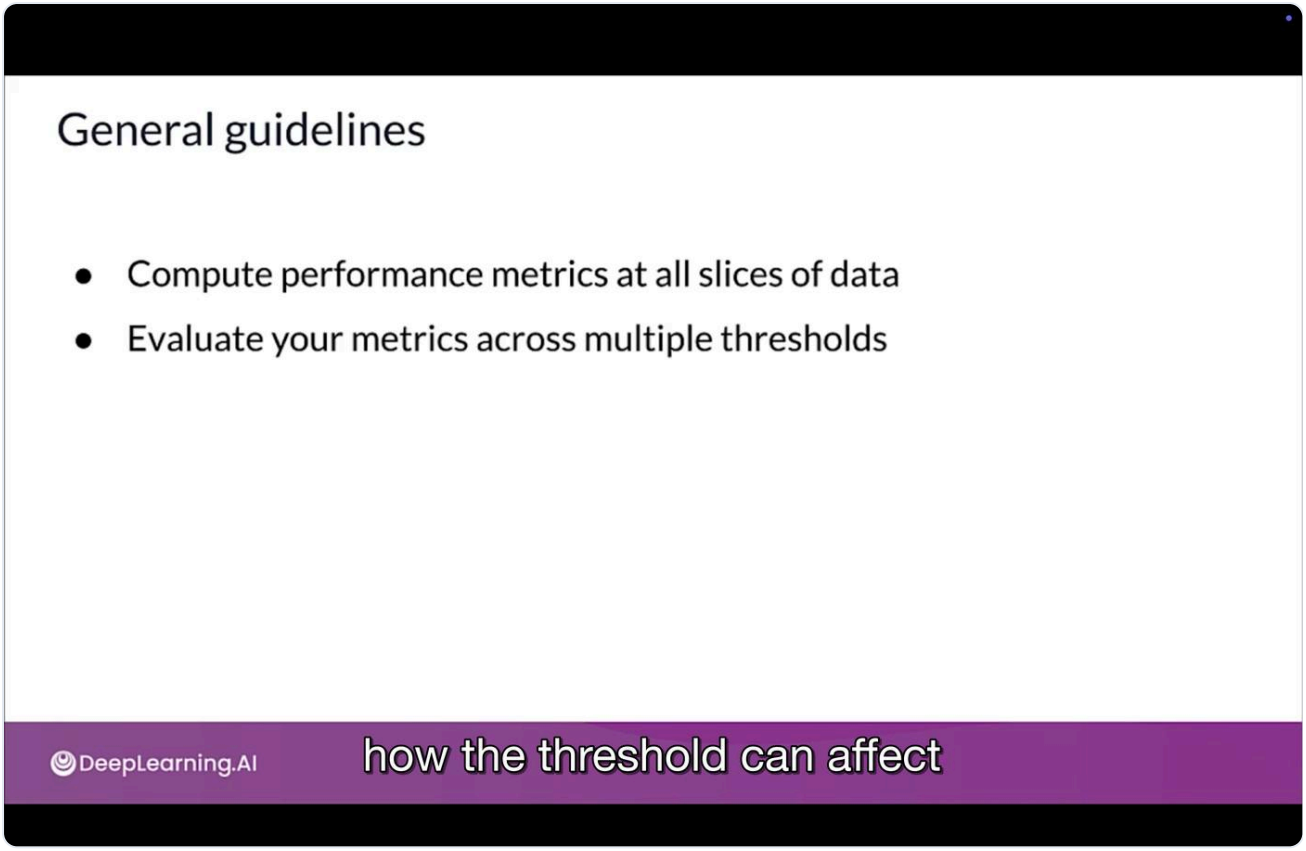

FIGURE 38.6

Next, you should evaluate the fairness metrics across multiple thresholds to understand how the threshold can affect the performance of different groups.

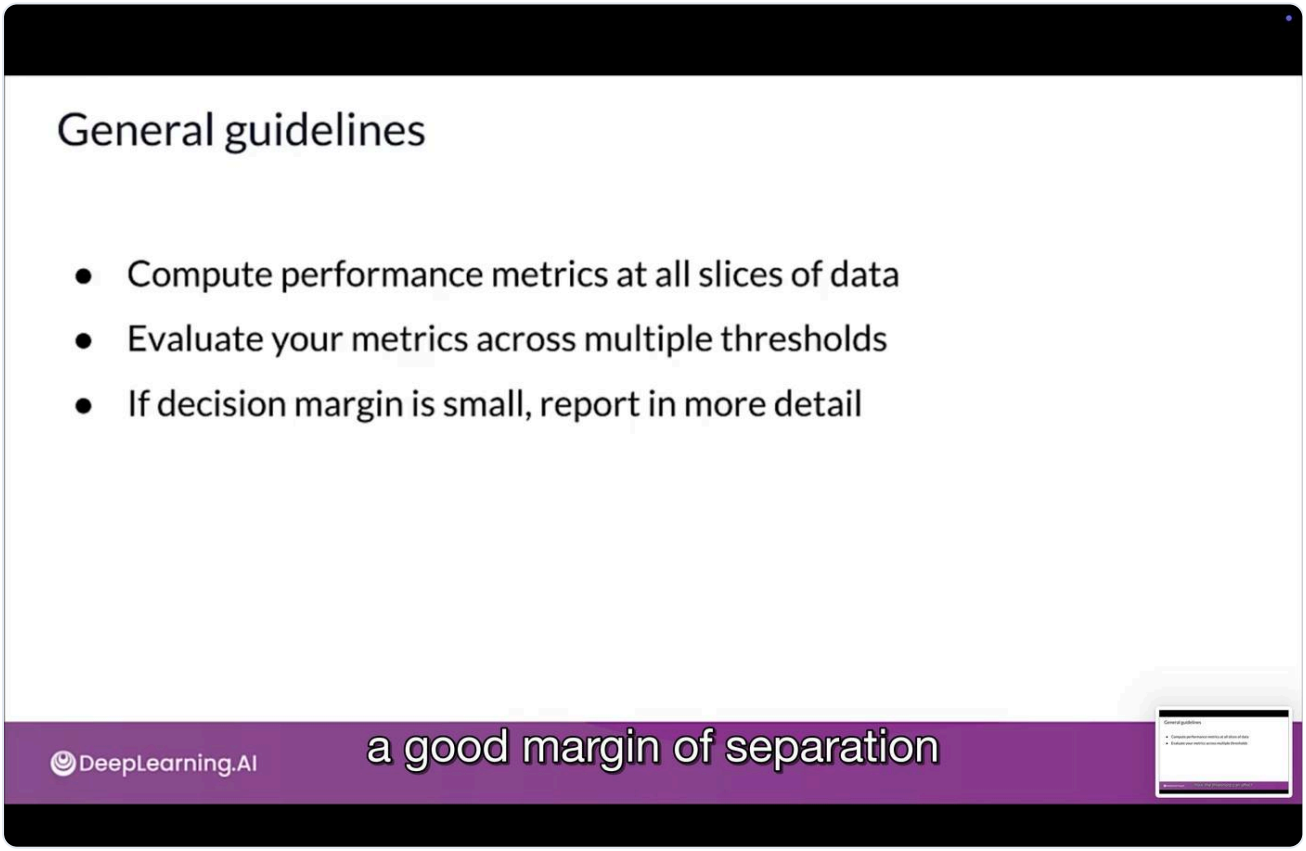

FIGURE 38.7

Finally, for predictions which don't have a good margin of separation from their decision boundaries, you should consider reporting the rate at which the label is predicted.

LECTURE 39

About the CelebA Dataset

Now let's look at the various fairness metrics which are available in fairness indicators. This includes the positive, negative rate, the ratios of true and false positives, accuracy and area under the curve. Will start with a quick overview of these metrics. Let's first consider the basic positive and negative rates. The basic positive and negative rates show the percentage of data points that are classified as positive or negative. These are independent of ground truth labels. These metrics help with understanding demographic parody, the equality of outcomes, which should be equal across subgroups.

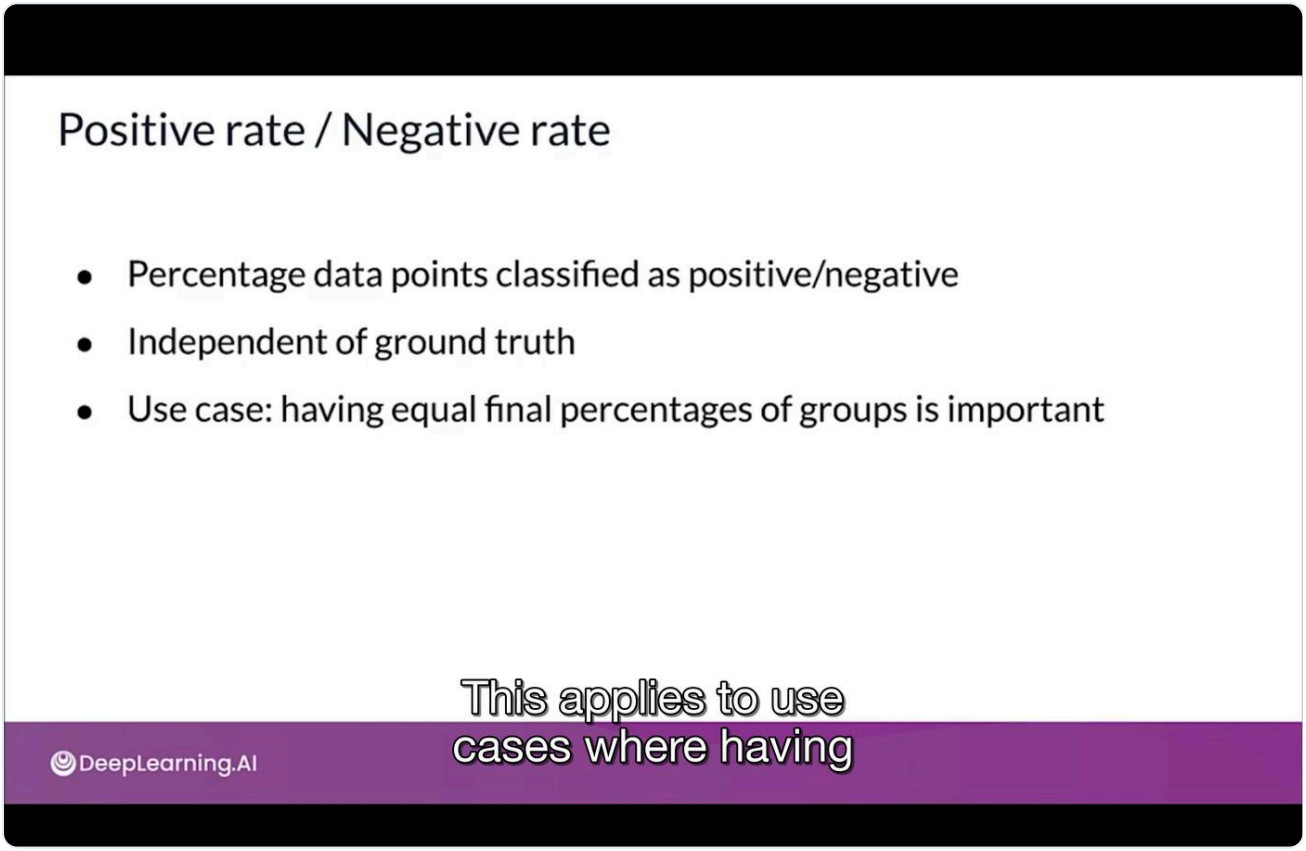

FIGURE 39.1 | Positive Rate / Negative Rate

This applies to use cases where having equal percentages of outcomes for different groups is important. Let's talk about the true positive rate, also known as TPR, and the false negative rate, also referred to as FNR. The true positive rate, measures the percentage of positive data points as labeled in the ground truth that are correctly predicted positive.

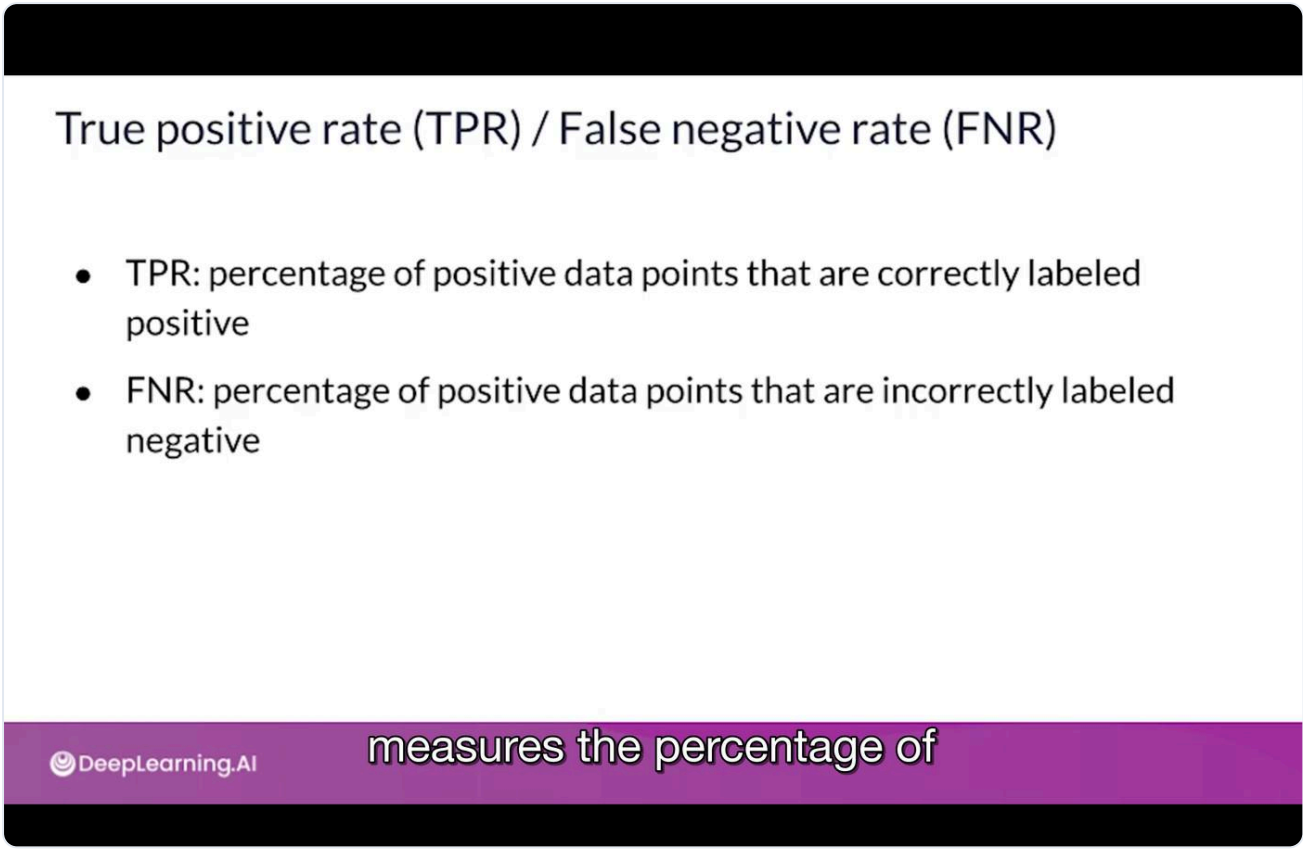

FIGURE 39.2 | True Positive Rate (TPR) / False Negative Rate (FNR)

Similarly, the false negative rate measures the percentage of positive data points that are incorrectly predicted negative.

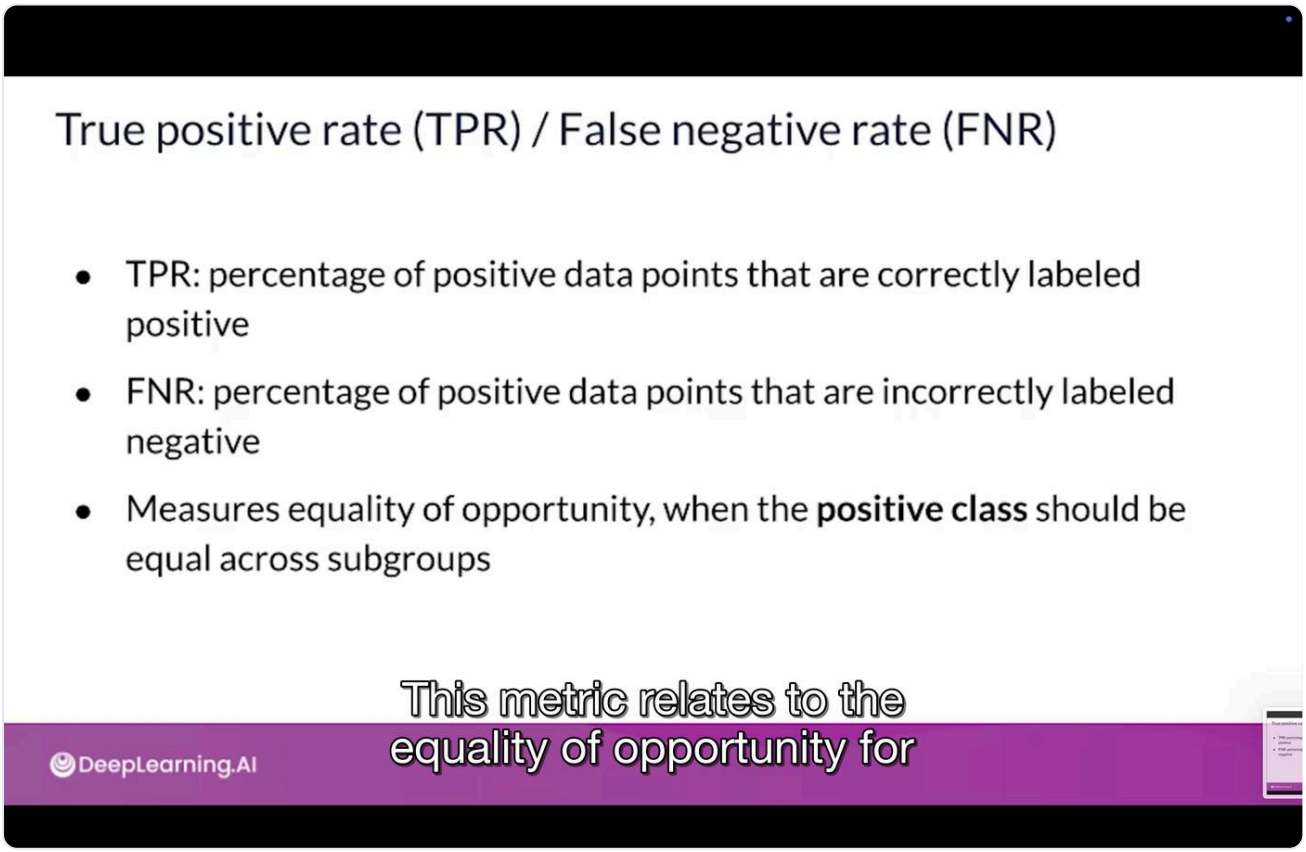

FIGURE 39.3 | True Positive Rate (TPR) / False Negative Rate (FNR)

This metric relates to the equality of opportunity for the positive class when it should be equal across subgroups.

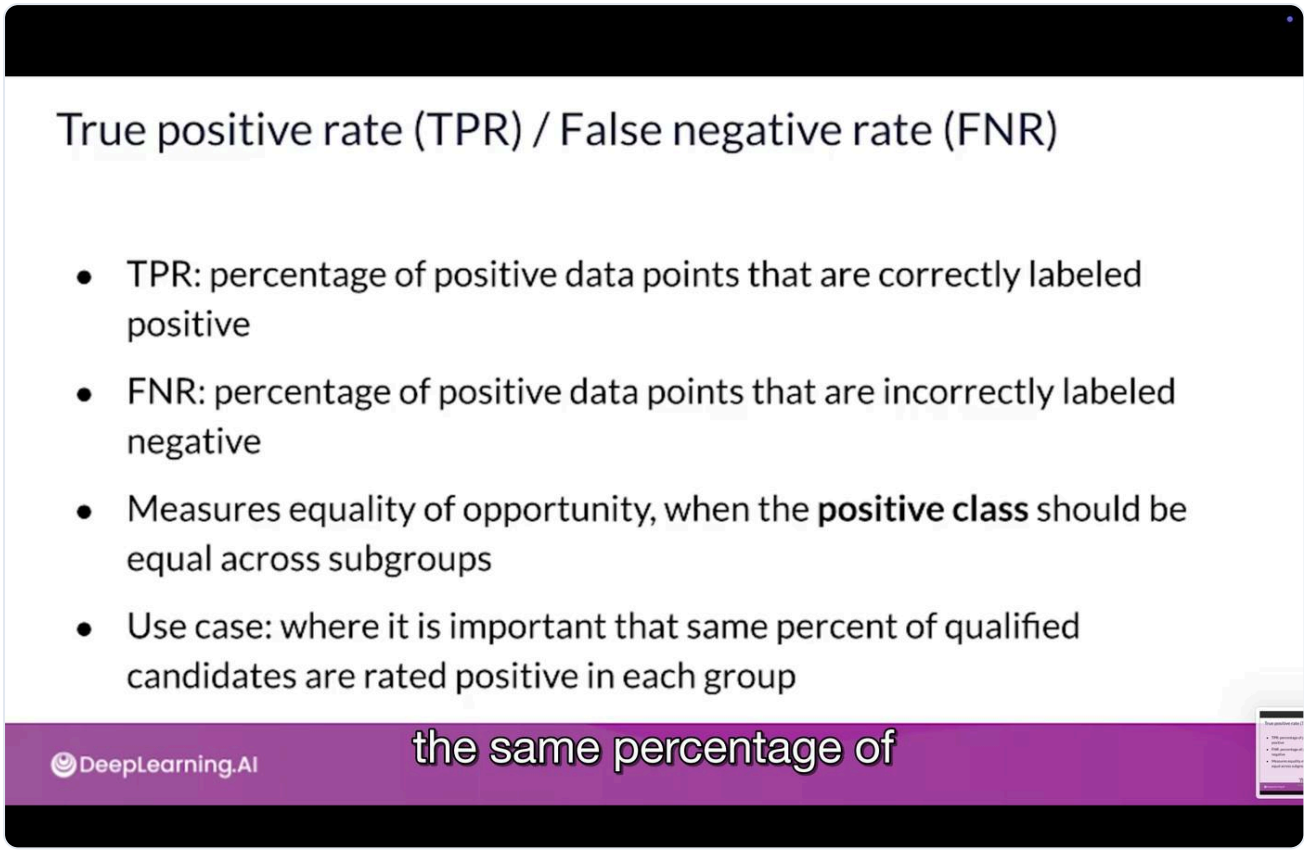

FIGURE 39.4 | True Positive Rate (TPR) / False Negative Rate (FNR)

This often applies to use cases where it's important that the same percentage of qualified candidates are rated positively in each group, such as for loan applications or school admissions. Now, let's talk about the true negative rate, also known as TNR, and the false positive rate, also referred to as FPR. The true negative rate measures the percentage of negative data points has labeled in the ground truths that are correctly predicted negative. Similarly, the false positive rate is the percentage of negative data points that are incorrectly predicted positive. This metric relates to quality of opportunity when the negative class should be equal across subgroups.

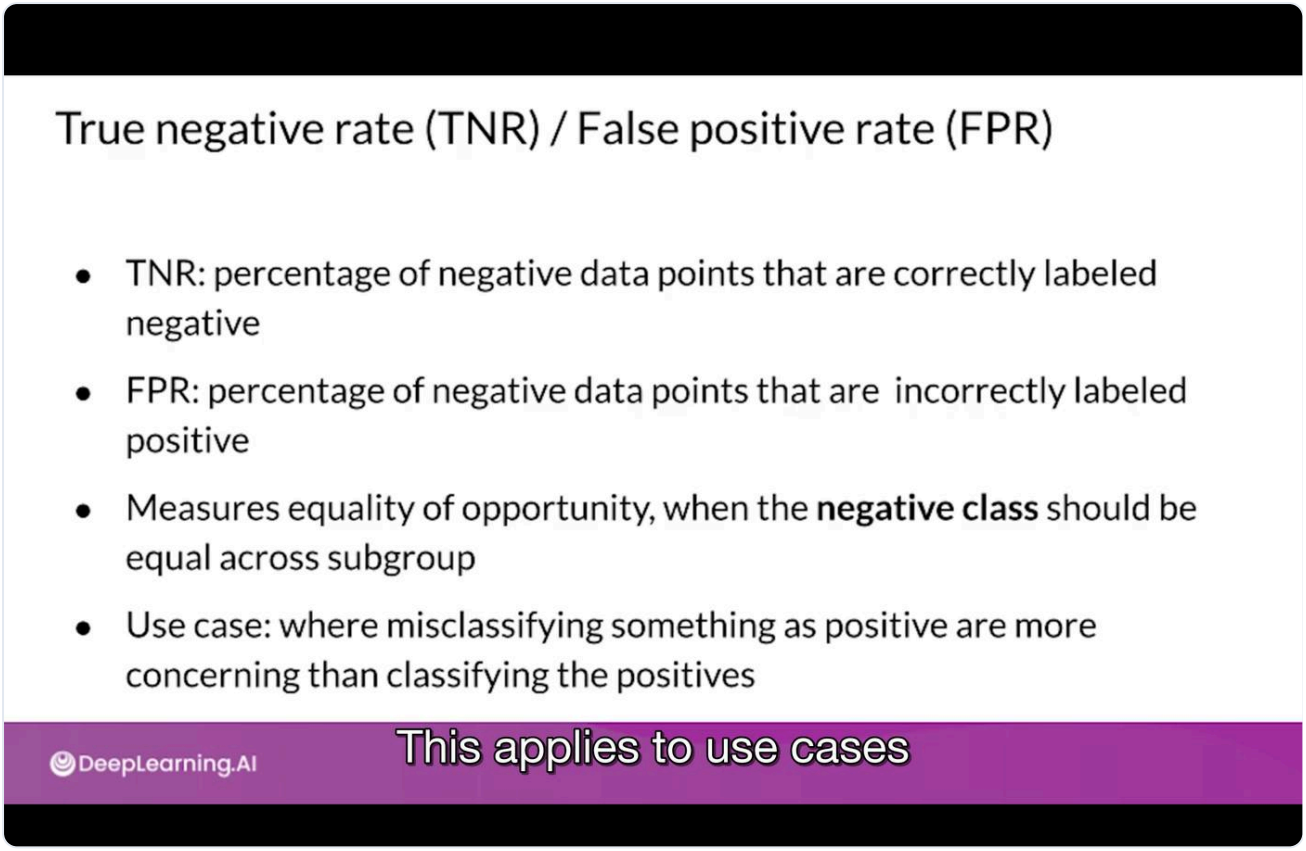

FIGURE 39.5 | True Negative Rate (TNR) / False Positive Rate (FPR)

This applies to use cases where misclassifying something is positive is more concerning than classifying the positives. This is most common in abuse cases where positives often lead to negative actions. These are also important for facial analysis techniques such as face detection or face attributes. The last set of fairness metrics will go through our accuracy and area under the curve, also known as AUC.

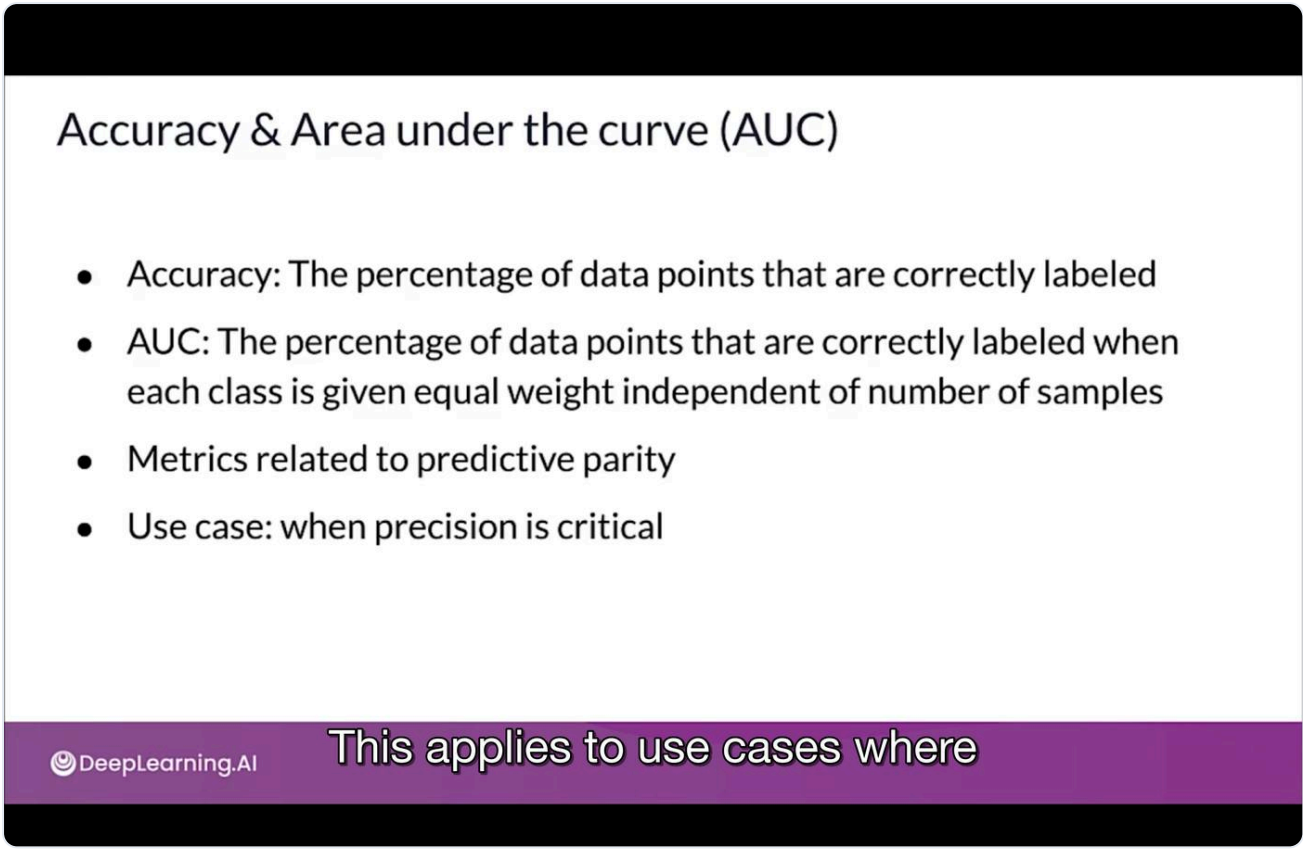

FIGURE 39.6 | Accuracy & Area Under the Curve (AUC)

Accuracy is the percentage of data points that are correctly labeled, and AUC is the percentage of data points that are correctly labeled when each class is given equal weight independent of the number of samples. Both of these metrics relate to predictive parody when equal across subgroups. This applies to use cases where the precision of the task is critical, but not necessarily in a given direction, such as face identification or phase clustering. Here's some thoughts to consider. A significant difference in a metric between two groups can be a sign that your model may have fairness issues. You should interpret your results according to your use case.

Next, achieving equality across groups with fairness indicators doesn't guarantee that your model is fair. Systems are highly complex and achieving equality on one or even all of the provided metrics can't guarantee fairness.

FIGURE 39.7

Fairness evaluation should be run throughout the development process and post-launch as well. Just like improving your product is an ongoing process and subject to adjustment based on user and market feedback, making your product fair and equitable requires ongoing attention. As different aspects of the model change, such as training data, inputs from other models or the design itself, fairness metrics are likely to change. Lastly, adversarial testing should be performed for rare and malicious examples.

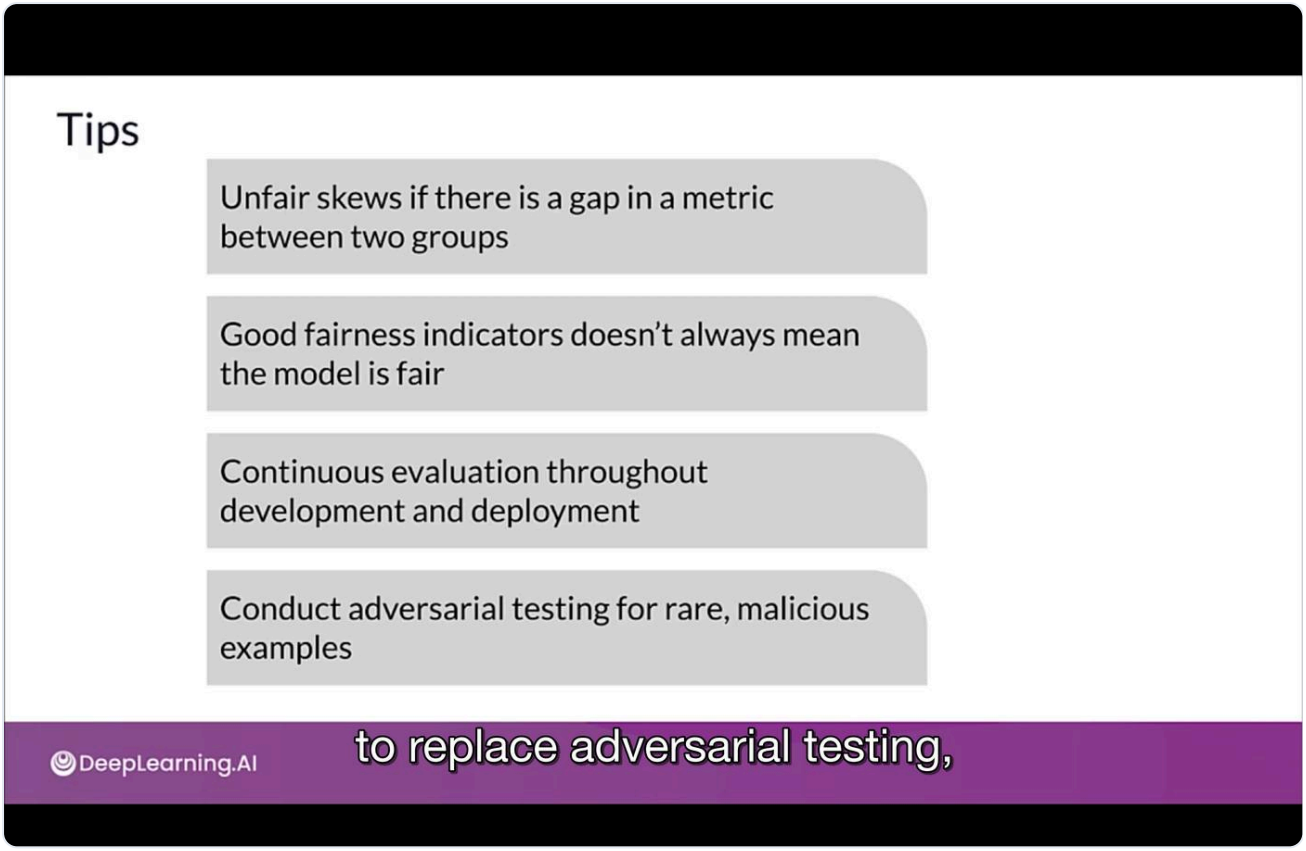

FIGURE 39.8

Fairness evaluations aren't meant to replace adversarial testing, but to provide an additional defense against rare targeted examples. This is crucial as these examples probably will not be included in training or evaluation data. Now let's look at a concrete example of using fairness Indicators. Let's consider the CelebA dataset which contains 200,000 images of celebrities. In addition to an image of a celebrity, each example has 40 attribute annotations. Some of these attributes are hair type, facial features, fashion accessories, etc. Every image also has five facial landmark locations, which mark the positions of the eyes, nose, and mouth.

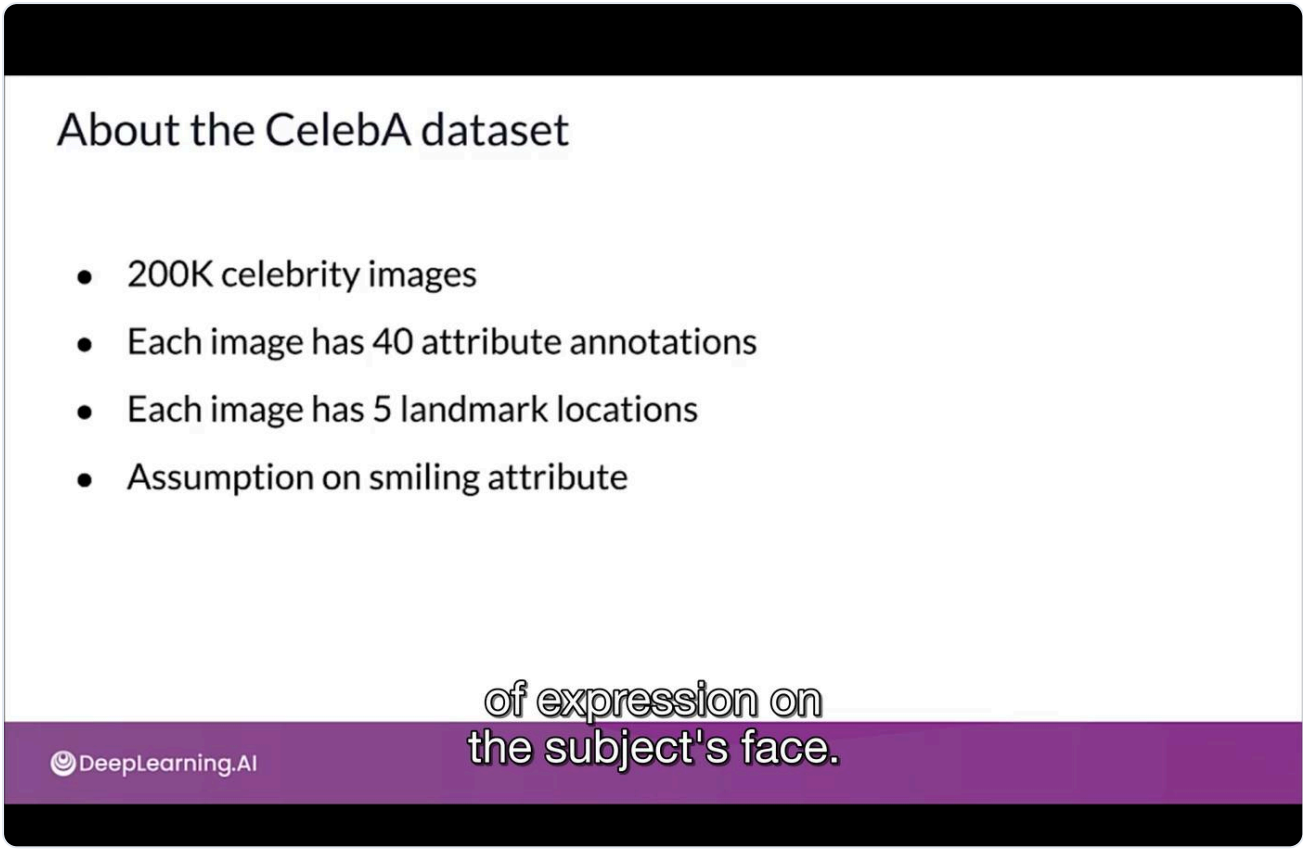

FIGURE 39.9 | About the CelebA Dataset

It's customary to assume that during the labeling of this dataset, the smiling attribute was determined through a pleasing, kind, or amusing type of expression on the subject's face. So you practice using fairness indicators to explore the smile attribute in this dataset. First, you start by building a classification model to identify whether the person in the image is smiling or not. Once the model is ready, you evaluate model performance across various age groups using fairness indicates.

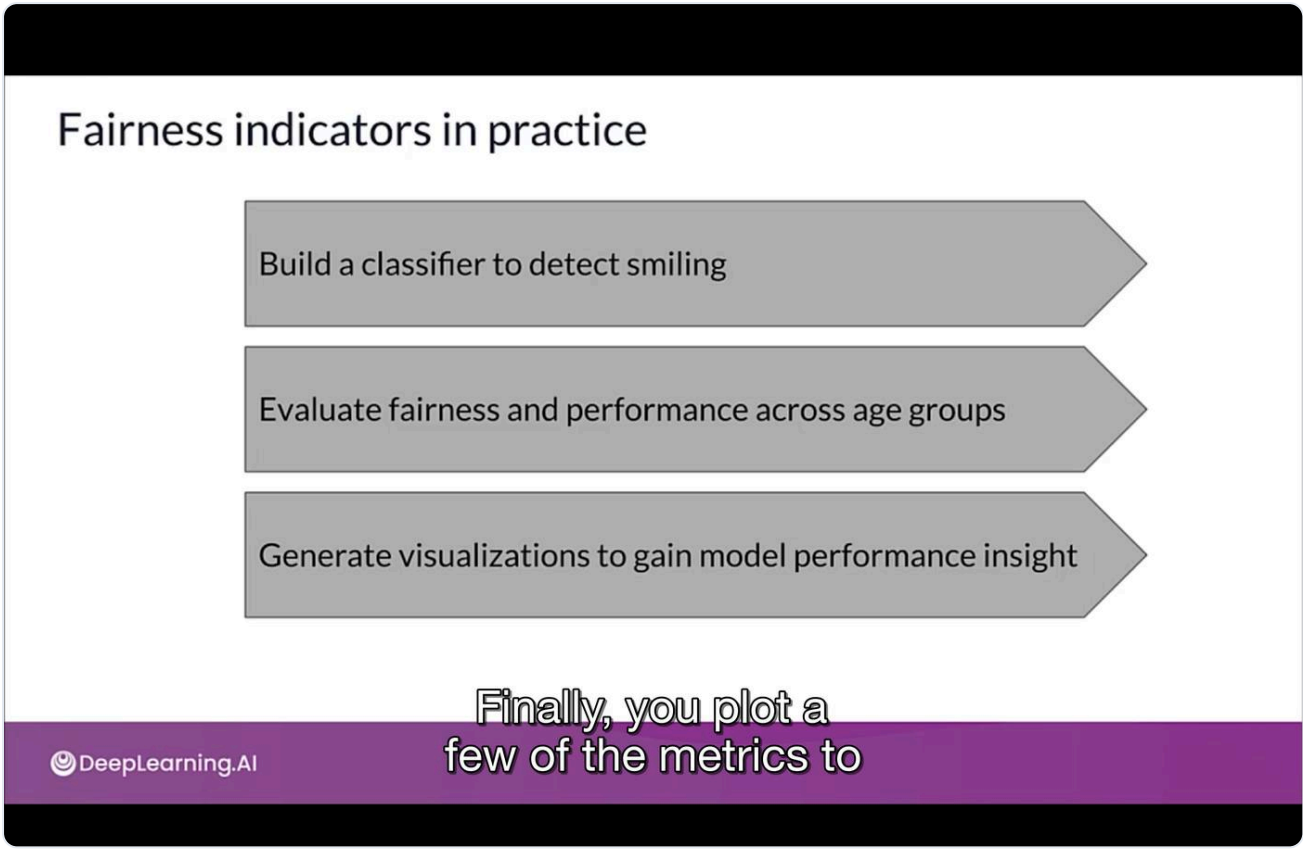

FIGURE 39.10 | Fairness Indicators in Practice

Finally, you plot a few of the metrics to understand how well our model is performing.

LECTURE 40

Statistical Process Control

We've mentioned a few times that keeping a healthy model running and operational, requires continuous evaluation and monitoring. Let's take a quick look at these important topics. Let's look at some of the ways of monitoring your model once it has been deployed to production. When you train your model, you use the training data that's available at that time. The training data represents a snapshot of the world at the time that it was collected and labeled. But the world changes, and for many domains, the data changes too. Sometime later when you're model's being used to generate predictions, it may or may not know enough about the current state of the world to make accurate predictions.

For example, if your model to predict movie sales was trained on data collected in the 90s, it might predict that customers would buy VHS tapes. Is that still a good prediction today? My guess is no. When the model goes bad, your application and your customers will suffer. Before it becomes a fire drill to collect new training data and fix the model, you want an early warning that your model performance is changing.

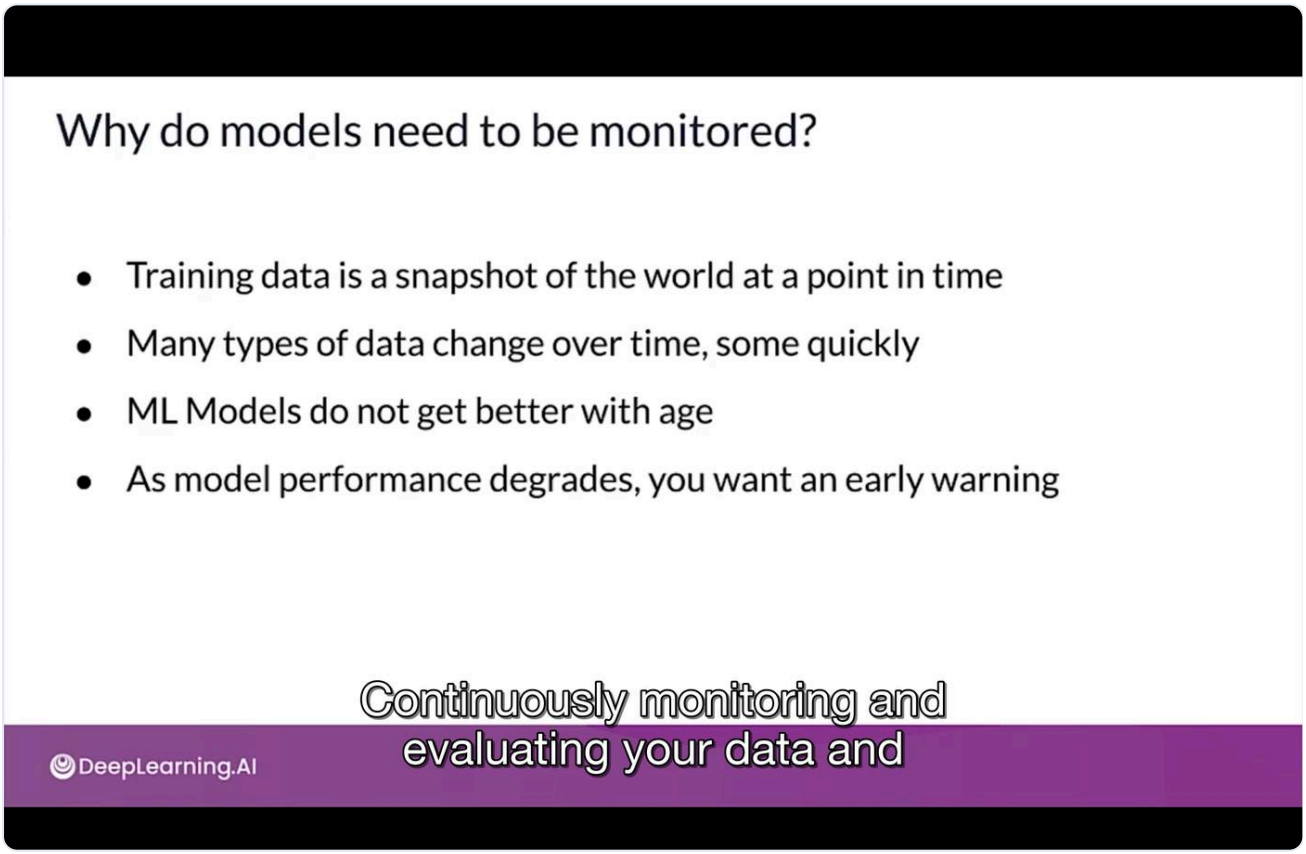

FIGURE 40.1 | Why Do Models Need to Be Monitored?

Continuously monitoring and evaluating your data and your model performance will help give you that early warning. Let's take a look at data drift and shift. The most extreme form of this is concept drift, which is a change in the relationship between your input data and your labels. Concept drift can happen even when the rest of your data doesn't change. For example, the exact

same customer who bought red boots six months ago may instead want white sneakers today, if the fashions have changed. The input to your model might be exactly the same, depending on which features you capture, but the correct prediction has changed. There's also the idea of an emerging concept.

An emerging concept refers to new patterns in the data distribution that weren't previously present in your dataset. This can happen in several ways: labels may have become obsolete and new labels may need to be added, as in our VHS example earlier. Based on the type of distribution change, the dataset shift can be classified into two types. The first is covariate shift. In covariate shift, the distribution of your input data changes, but the conditional probability of your output over the input remains the same. The distribution of your labels doesn't change.

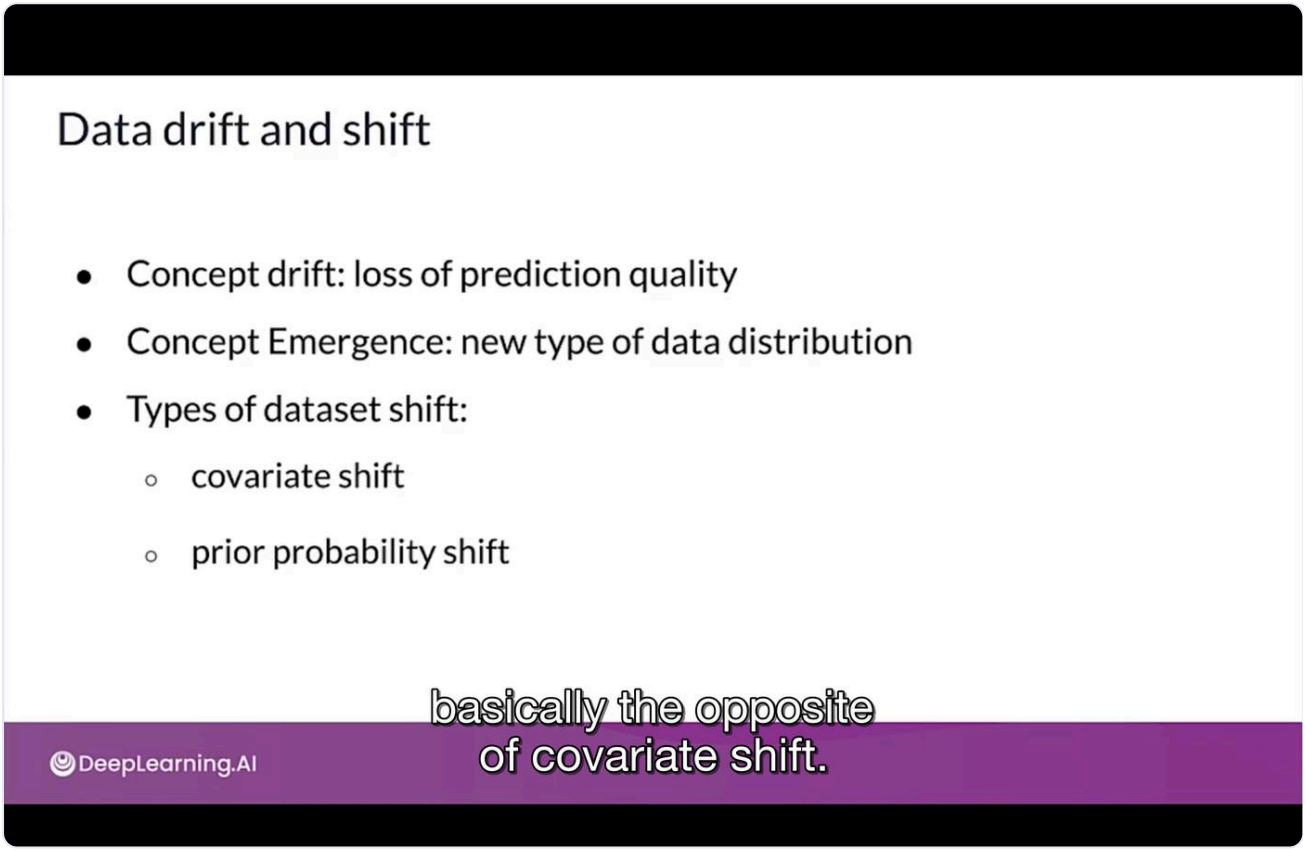

FIGURE 40.2 | Data Drift and Shift

Prior probability shift is basically the opposite of covariate shift. The distribution of your labels changes, but your input data stays the same. Concept drift can be thought of as a type of prior probability shift. Now let's think about how to monitor your data and model performance. First, think of your trained model in a production setting. Let's start with the raw data that you'll use to make a prediction.

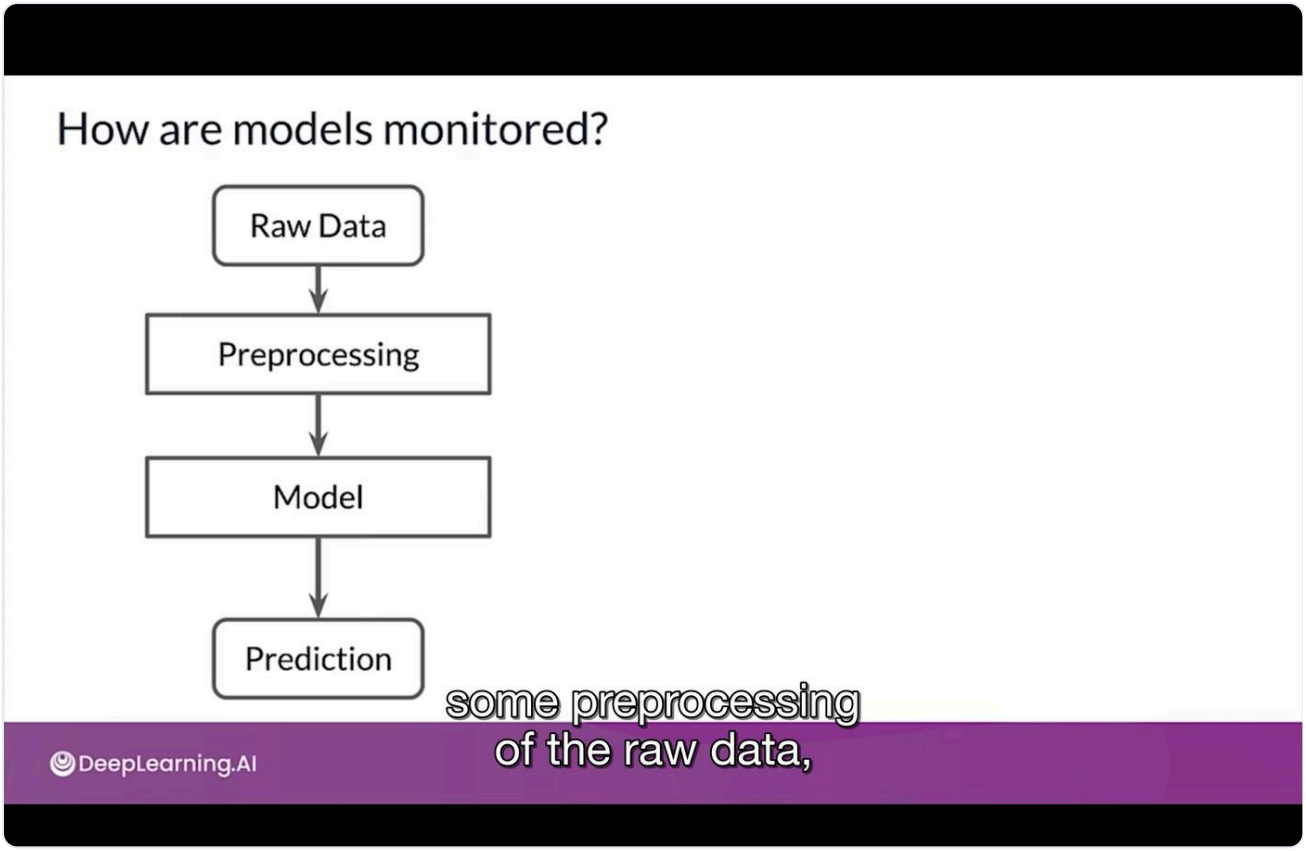

FIGURE 40.3 | How Are Models Monitored?

Before making a prediction, you probably need to do some preprocessing of the raw data, and then you use your model to generate a prediction given the input data. Now, let's add monitoring and evaluation.

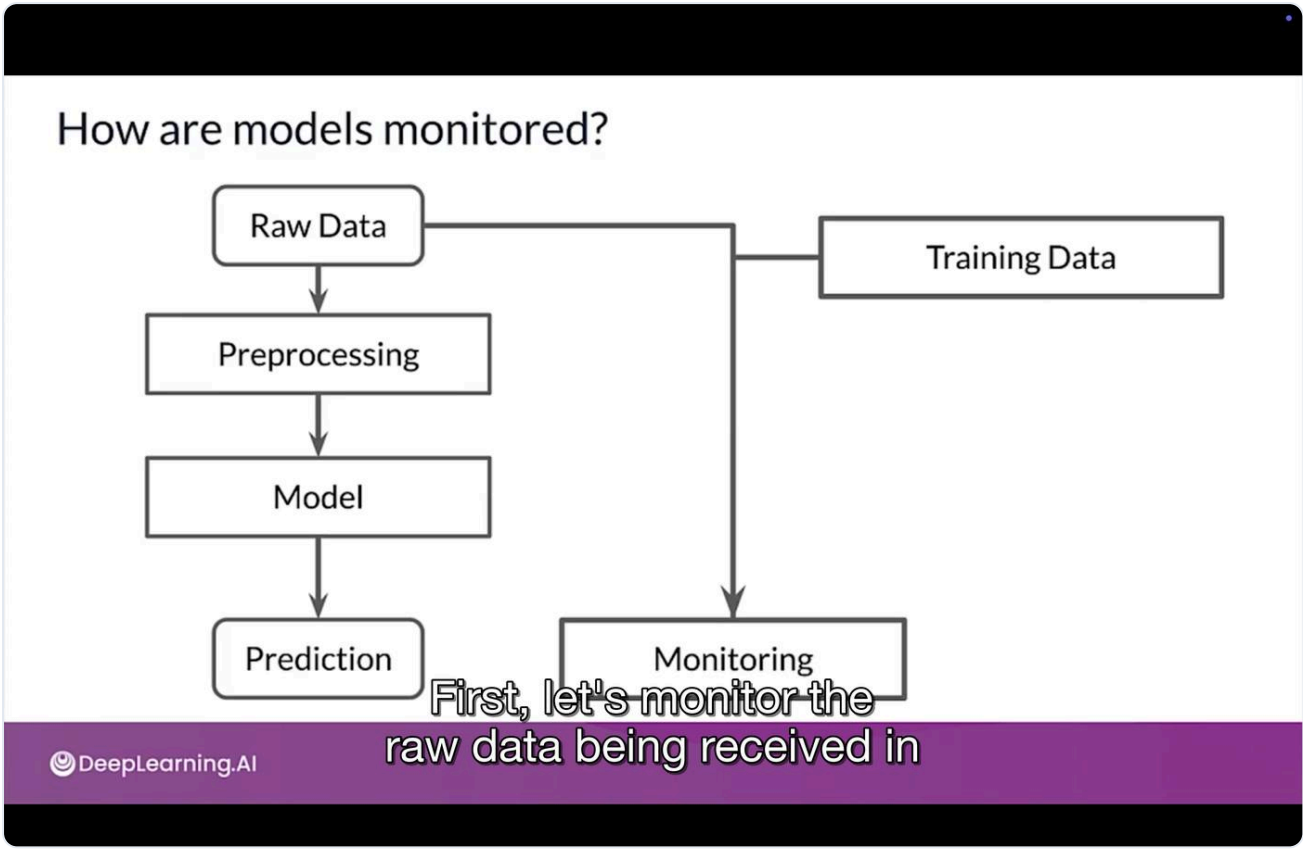

FIGURE 40.4 | How Are Models Monitored?

First, let's monitor the raw data being received in production and compare it to the data that you trained your model with.

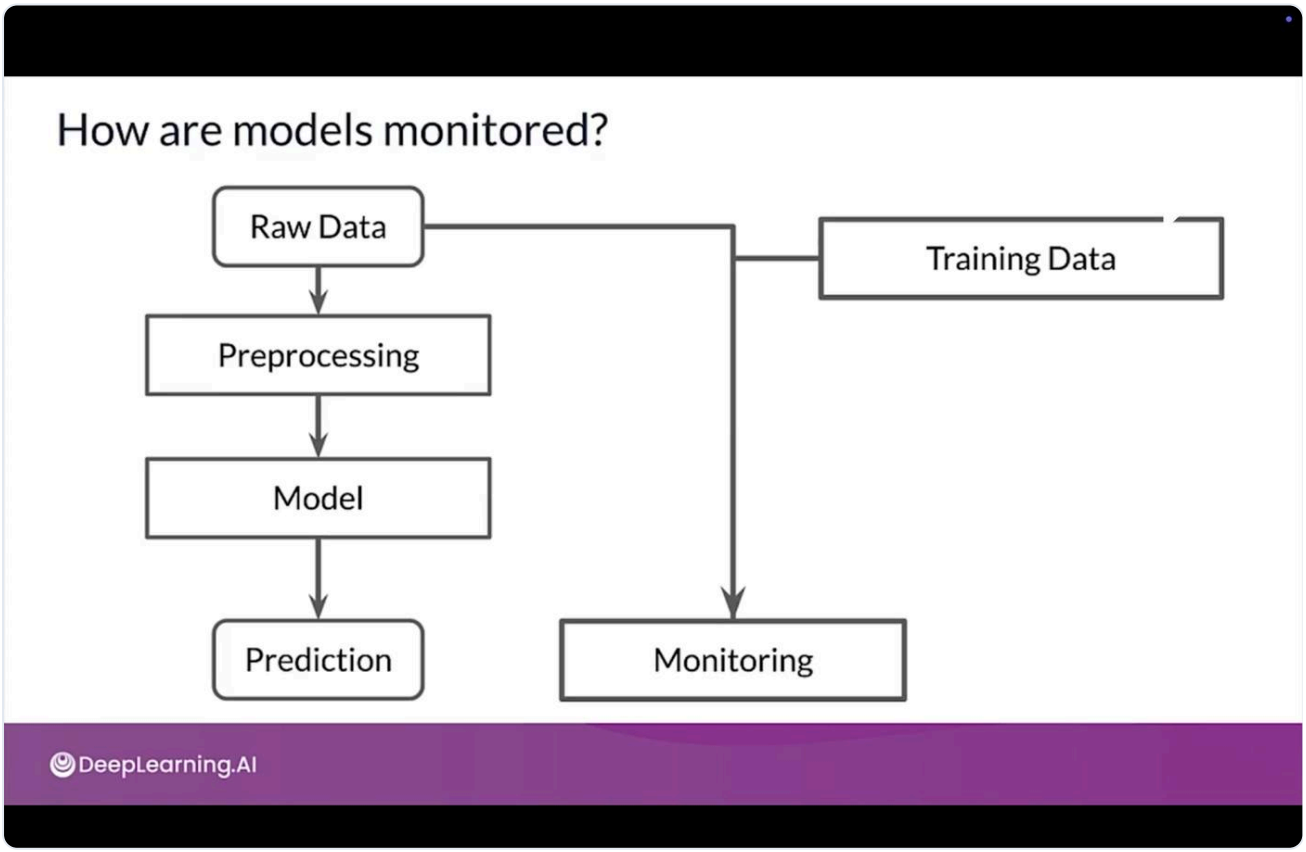

FIGURE 40.5 | How Are Models Monitored?

This lets you look for changes in the input data over time or covariate shift. Next, let's add monitoring for the predictions which you're generating.

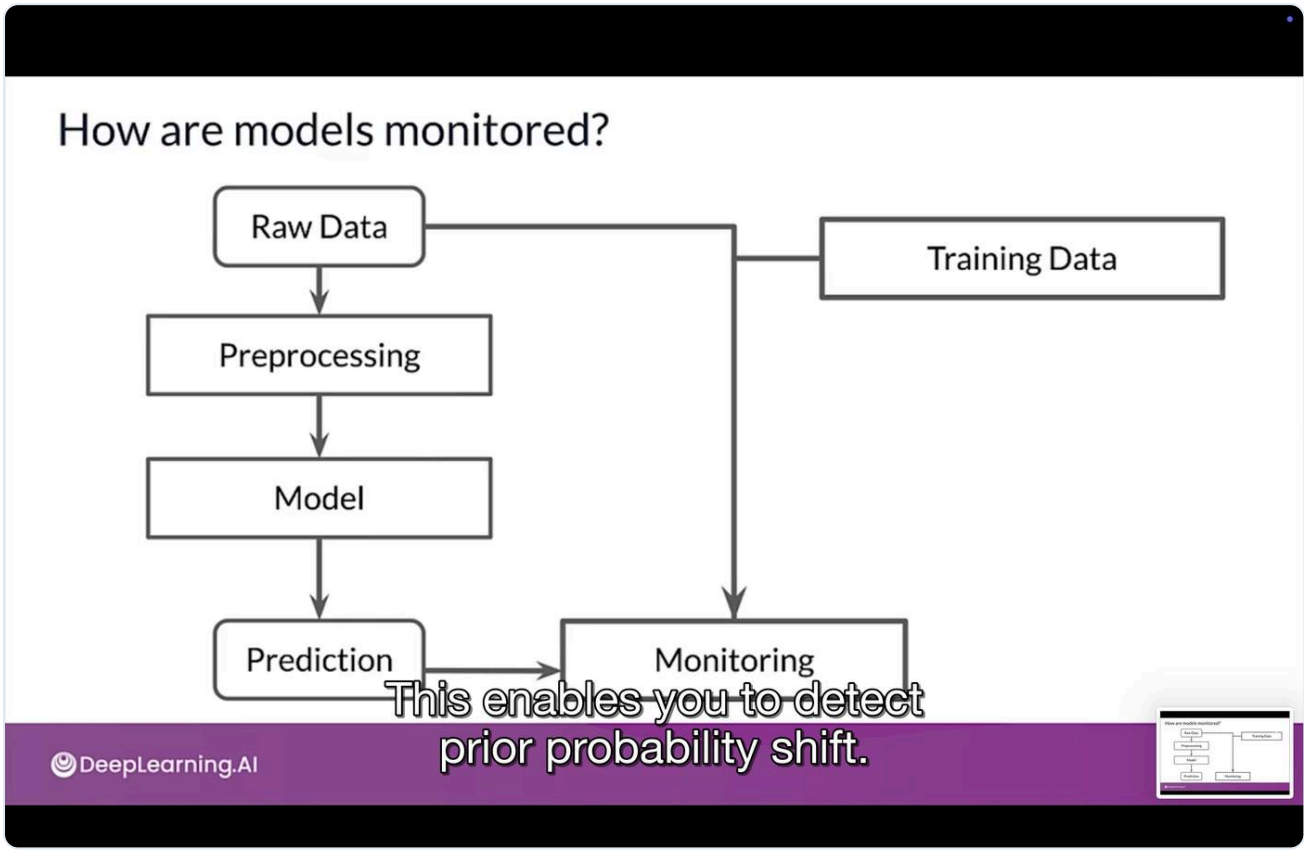

FIGURE 40.6 | How Are Models Monitored?

This enables you to detect prior probability shift.

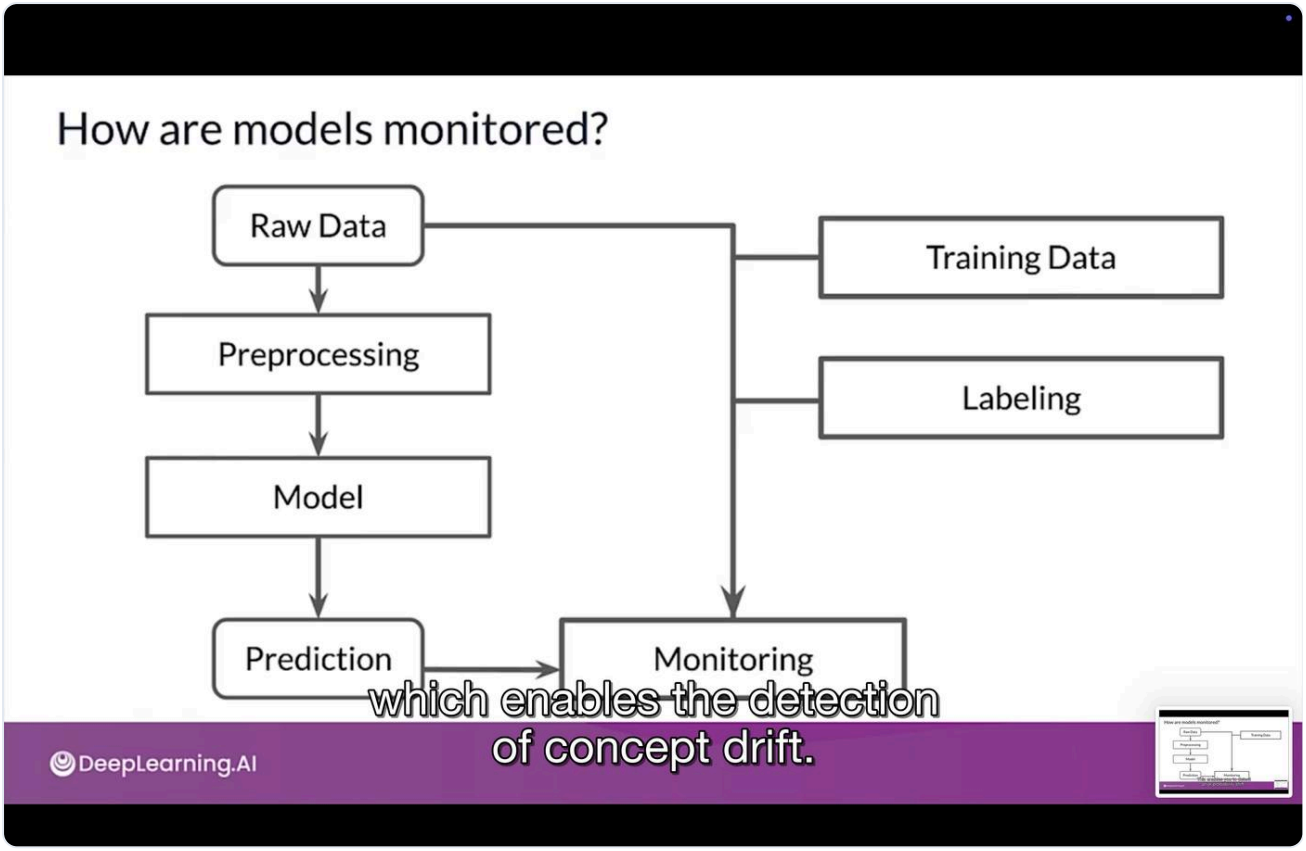

FIGURE 40.7 | How Are Models Monitored?

Finally, let's add a labeling process which enables the detection of concept drift. Note, that in many cases, there will be a significant delay in generating labels for a sample of the new input data. Now that you've added monitoring, how do you look for drift and shift? There are both supervised and unsupervised methods based on whether or not you have labels for the incoming stream of data and the kind of drift or shift that you're looking for. Let's discuss each of these next. Our first supervised technique is statistical process control. Statistical process control has been primarily used in manufacturing for quality control since the 1920s. It uses statistical methods to monitor and control process, which in the case of your deployed model is the incoming stream of raw data for prediction requests.

This is useful to detect drift.

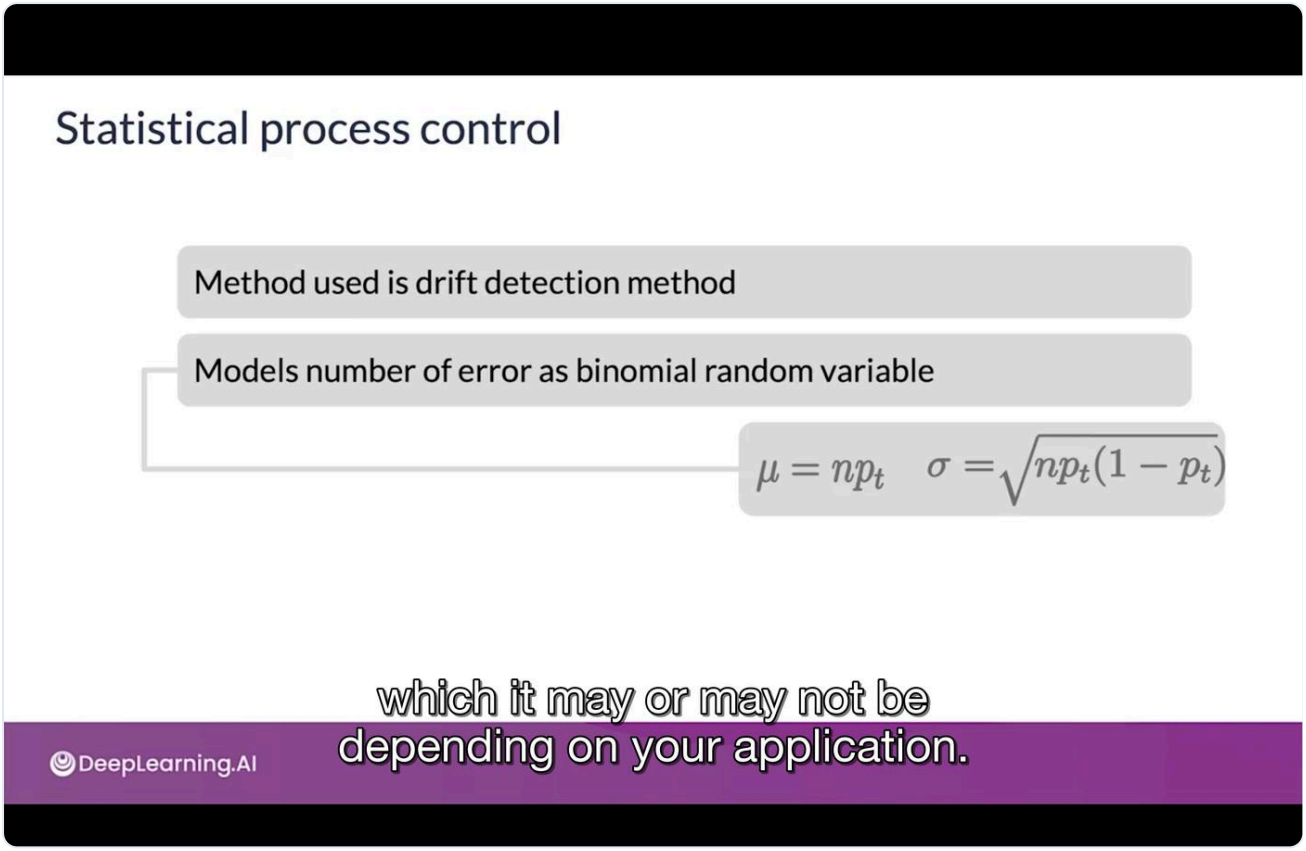

FIGURE 40.8 | Statistical Process Control

It assumes that the stream of data will be stationary, which it may or may not be depending on your application. That the errors follow a binomial distribution.

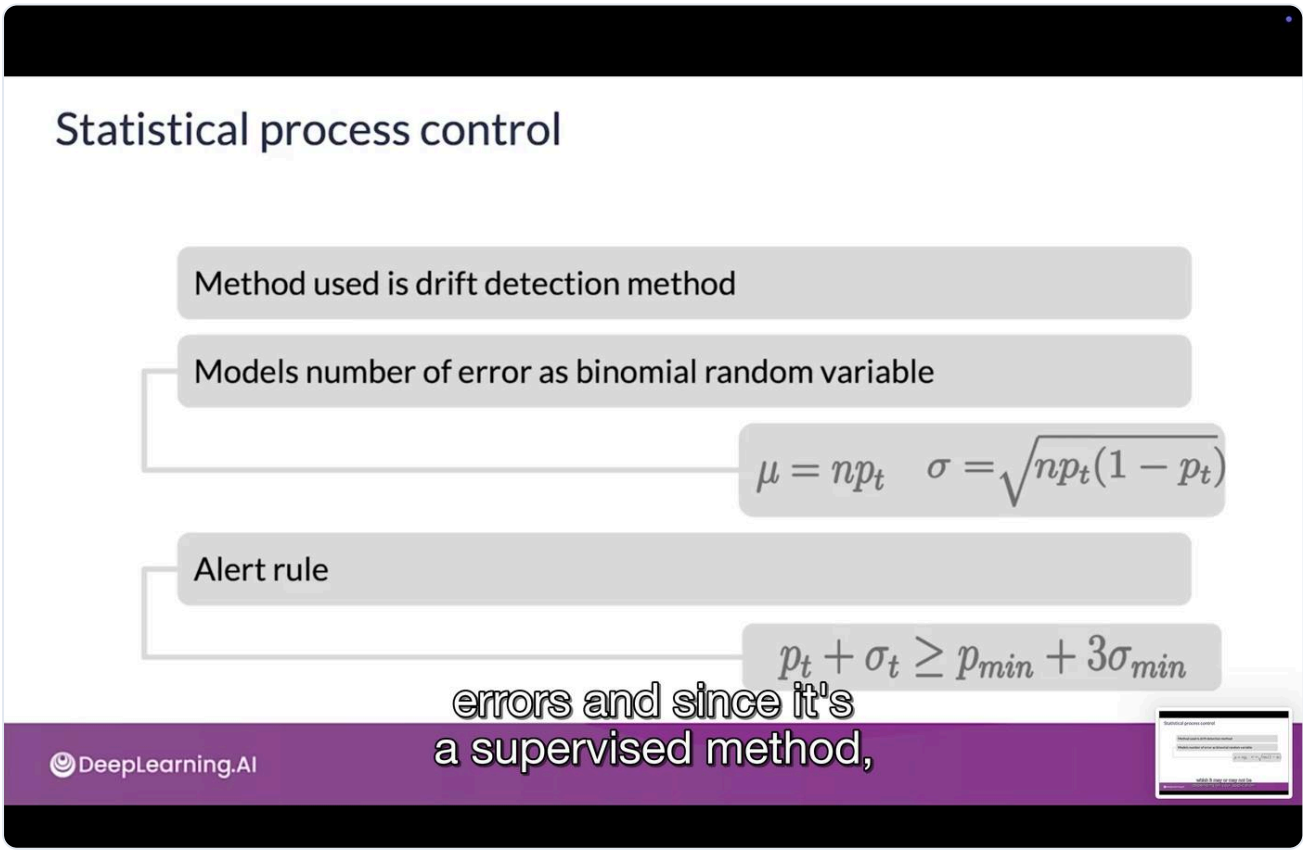

FIGURE 40.9 | Statistical Process Control

It analyzes the rate of errors and since it's a supervised method, it requires us to have labels for incoming stream of data. Essentially, this method triggers a drift alert if the parameters of the distribution go beyond a certain threshold rule. The second supervised technique is sequential analysis. In sequential analysis, we use a method called linear four rates. The basic idea is that if data is stationary, the contingency table should remain constant.

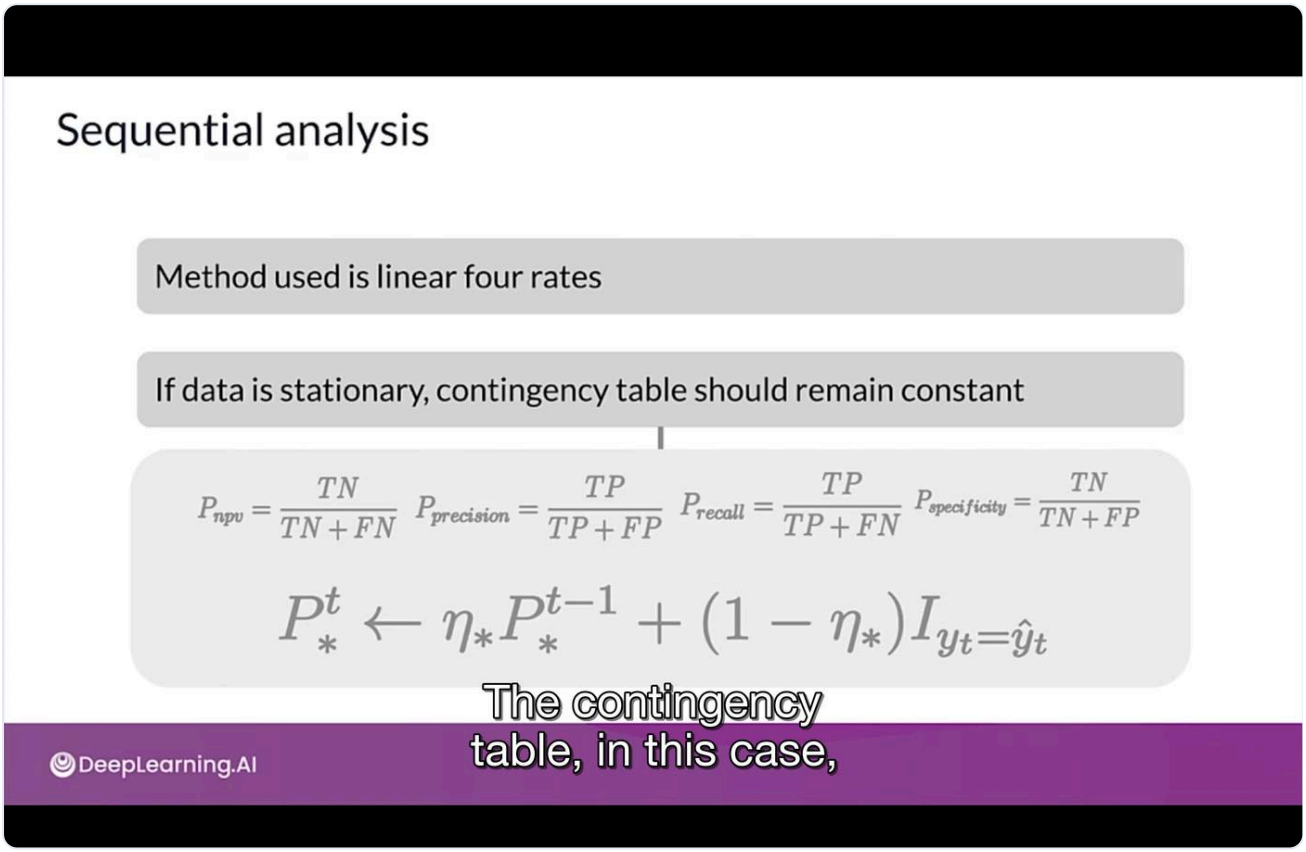

FIGURE 40.10 | Sequential Analysis

The contingency table, in this case, corresponds to the truth table for a classifier that you're probably familiar with: true positive, false positive, false negative, and true negative. You use those to calculate the four rates: net predictive value, precision, recall, and specificity. If the model is predicting correctly, these four value should continue to remain constant. The last supervised technique to review is air distribution monitoring. The method of choice here is known as adaptive windowing. In this method, you divide the incoming data into windows, the size of which adapts to the data. Then you calculate the mean error rate at every window of data.

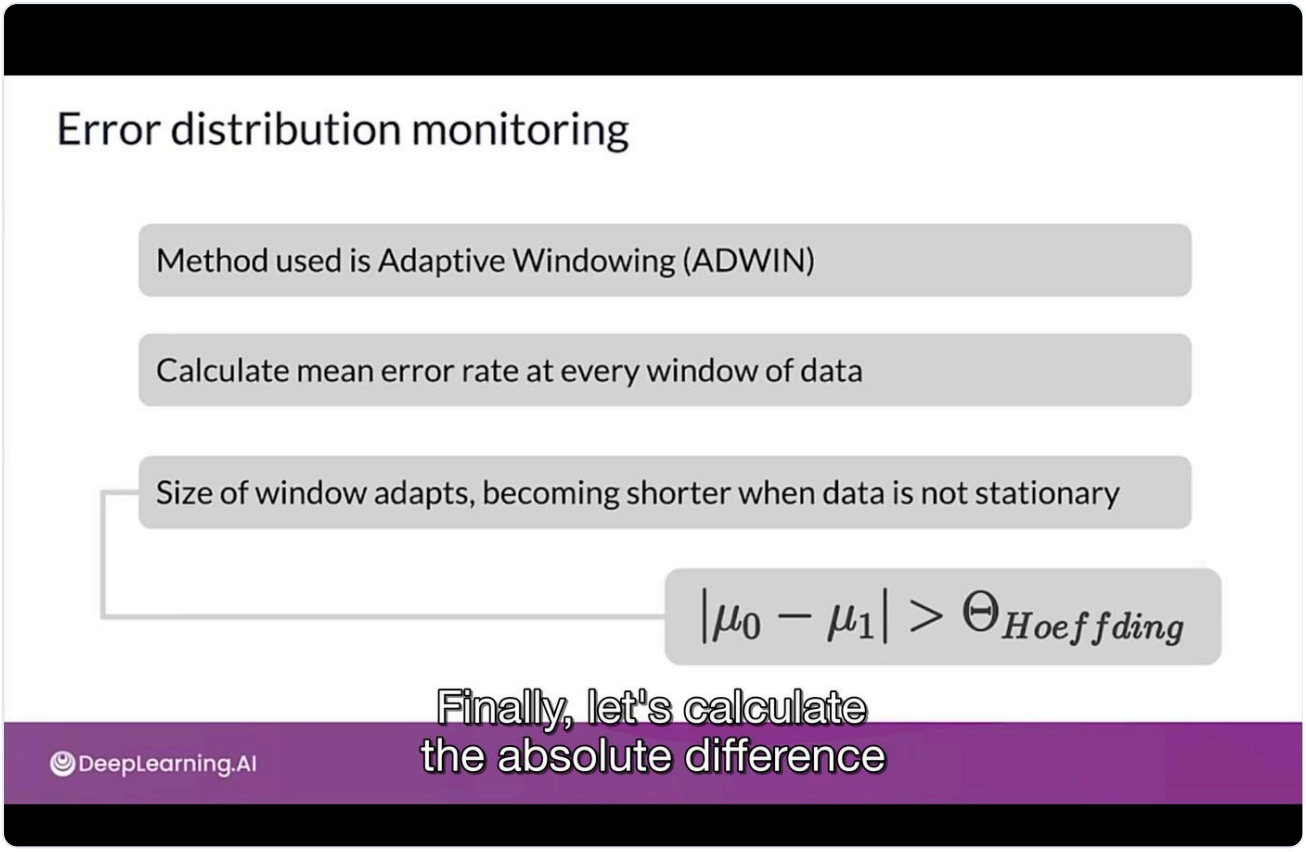

FIGURE 40.11 | Error Distribution Monitoring

Finally, let's calculate the absolute difference of the mean error rate at every successive window and compare it with a threshold based on the Hoeffding bound. The Hoeffding bound is used for testing the difference between the means of two populations. Now, let's move on to unsupervised techniques. The main problem with supervised techniques is that you need to have labels, and generating labels can be expensive and slow. In unsupervised techniques, you don't need labels. Note that you can also use unsupervised techniques in addition to supervised techniques when you do have labels. Let's start with clustering or novelty detection. In this method, you cluster the incoming data into one of the known classes.

If you see that the features of the new data are far away from the features of known classes, then you know that you're seeing an emerging concept. Based on the type of clustering you choose, there are multiple algorithms available. We've listed a few, including OLINDDA, MINAS, ECSMiner, and GC3. But for this course, we won't be discussing the details of these algorithms.

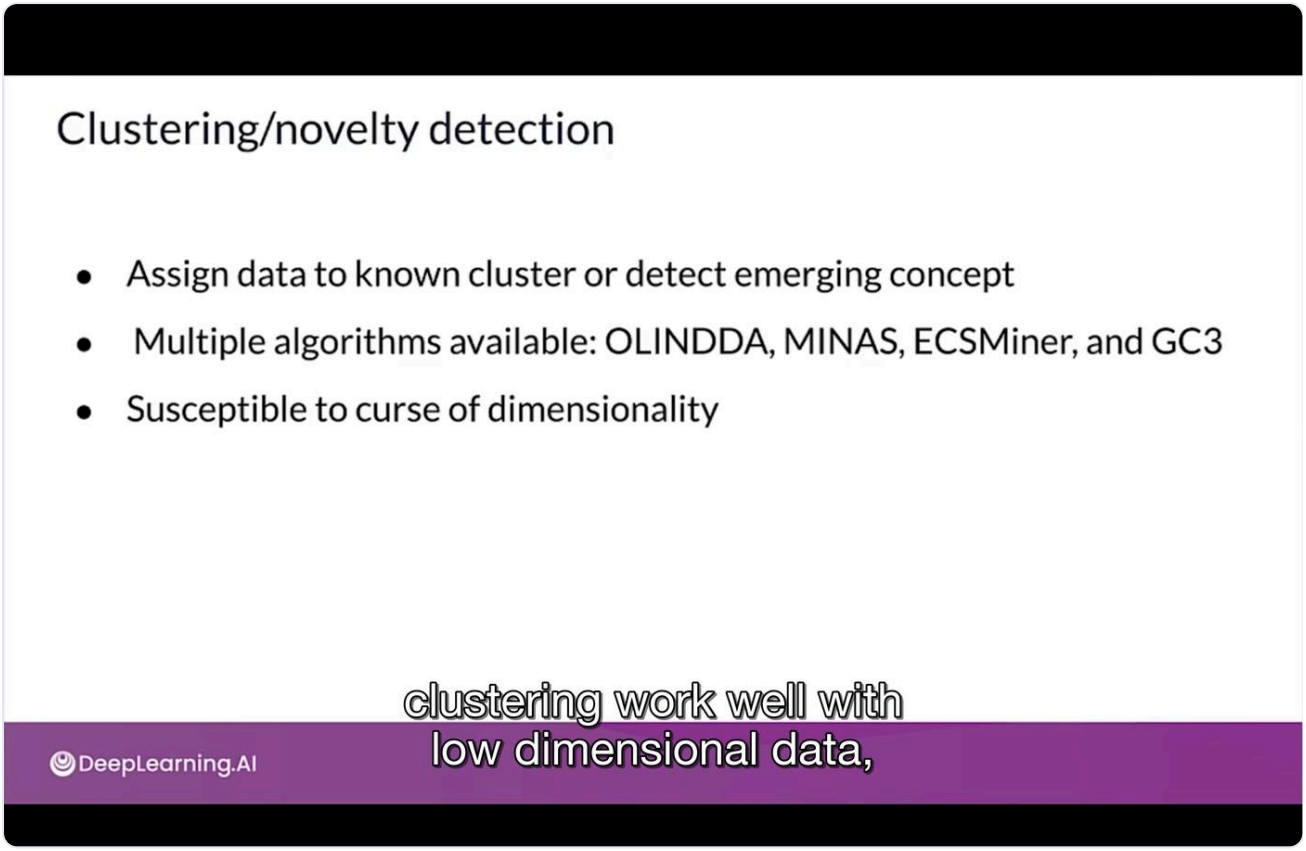

FIGURE 40.12 | Clustering/novelty Detection

While the visualization and ease of working with clustering work well with low dimensional data, the curse of dimensionality kicks in once the number of dimensions grow significantly. Eventually, these methods start getting inefficient, but you can use dimensionality reduction techniques like PCA to help make it manageable.

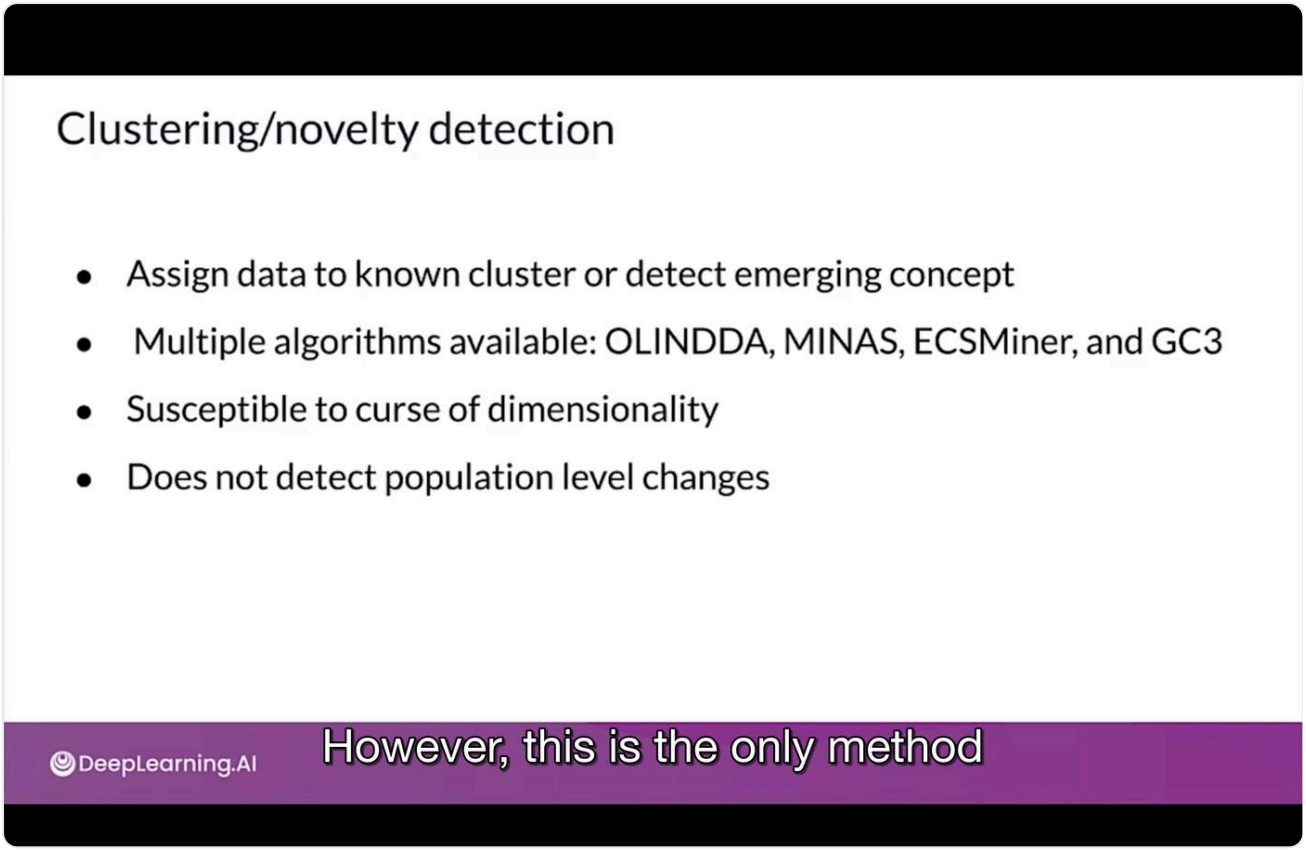

FIGURE 40.13 | Clustering/novelty Detection

However, this is the only method that helps you in detecting emerging concepts. The downside of this method is that the text only cluster-based drift, not population-based changes. The next unsupervised technique is feature distribution monitoring. In feature distribution monitoring, we monitor each feature of the dataset separately. You split the incoming dataset into uniformly size windows, and then compare the individual features against each window of data. There are multiple algorithms available to do the comparison, and here are two of the most popular ones. The first is Pearson correlation, which is using the change of concept technique.

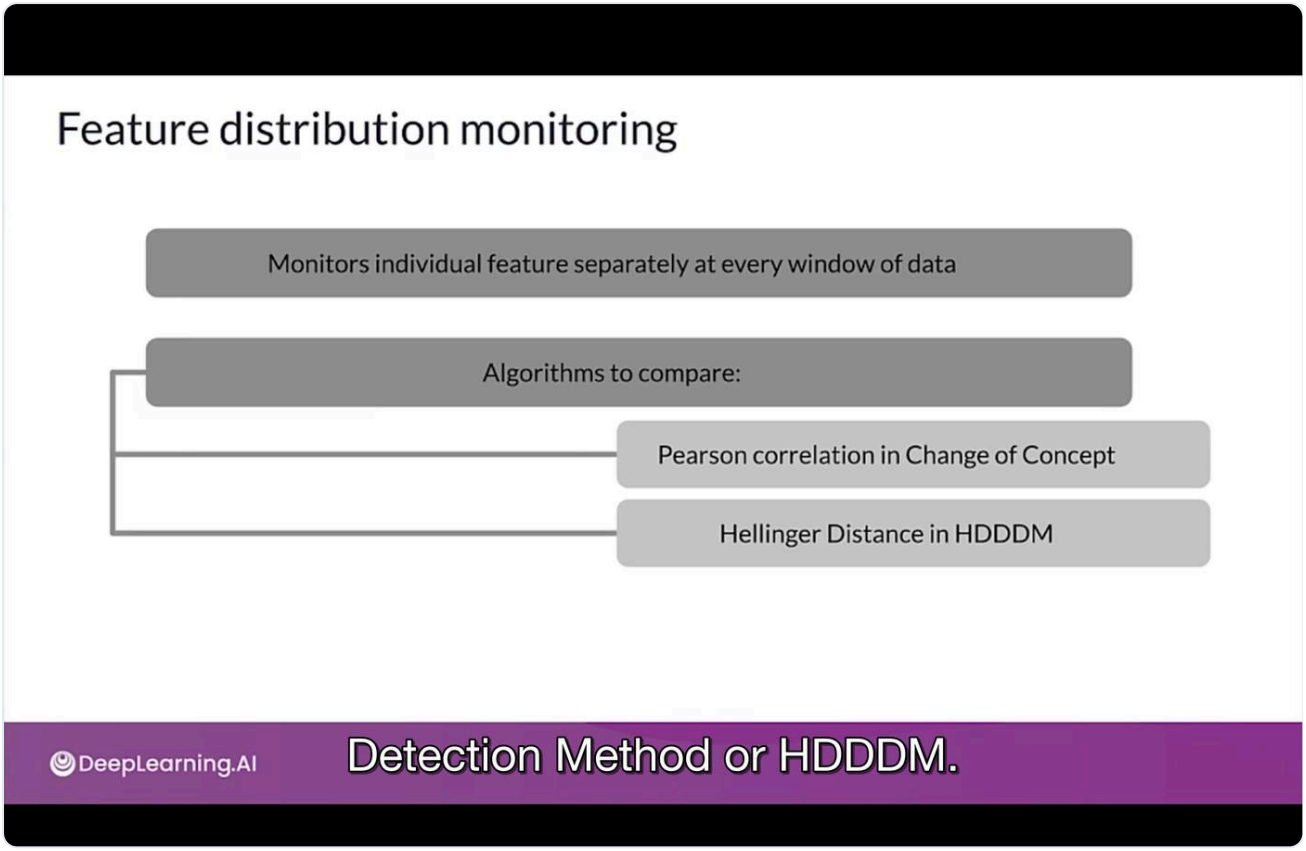

FIGURE 40.14 | Feature Distribution Monitoring

There's also the Hellinger distance, which is used in the Hellinger Distance Drift Detection Method or HDDDM. The Hellinger distance is used to quantify the similarity between two probability distributions. See the reference list at the end of the week for more information. Similar to the case of novelty detection, if the curse of dimensionality kicks in, you can make use of dimensionality reduction techniques like PCA to reduce the number of features. The downside of this method is that it is not able to detect population drift since it only looks at individual features.

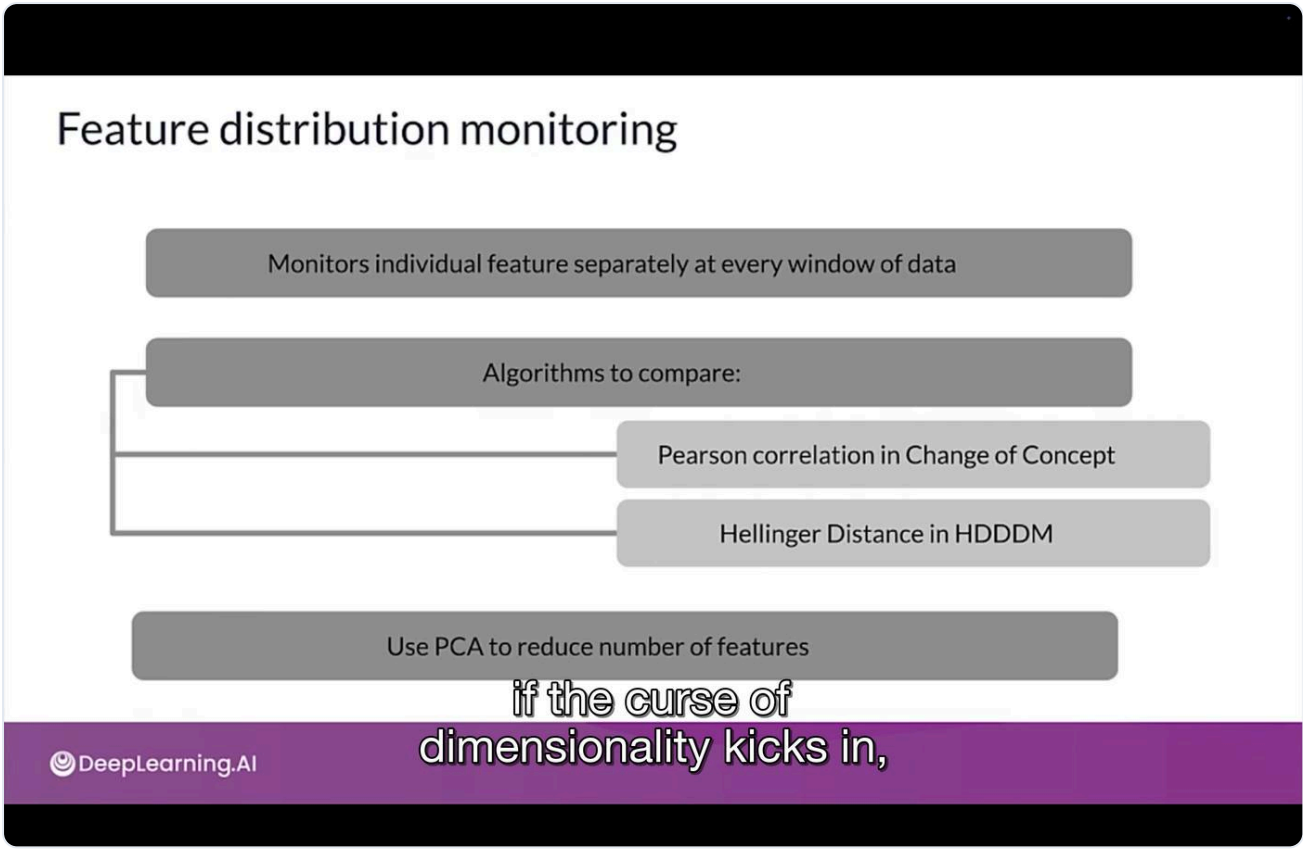

FIGURE 40.15 | Feature Distribution Monitoring

The last unsupervised technique to cover is model-dependent monitoring. This method monitors the space near the decision boundaries or margins in the latent feature space of your model. One of the algorithms used is Margin Density Drift Detection or MD3. Space near the margins where the model has low confidence matters more than in other places, and this method looks for incoming data that falls into the margins. A change in the number of samples in the margin, the margin density, indicates drift.

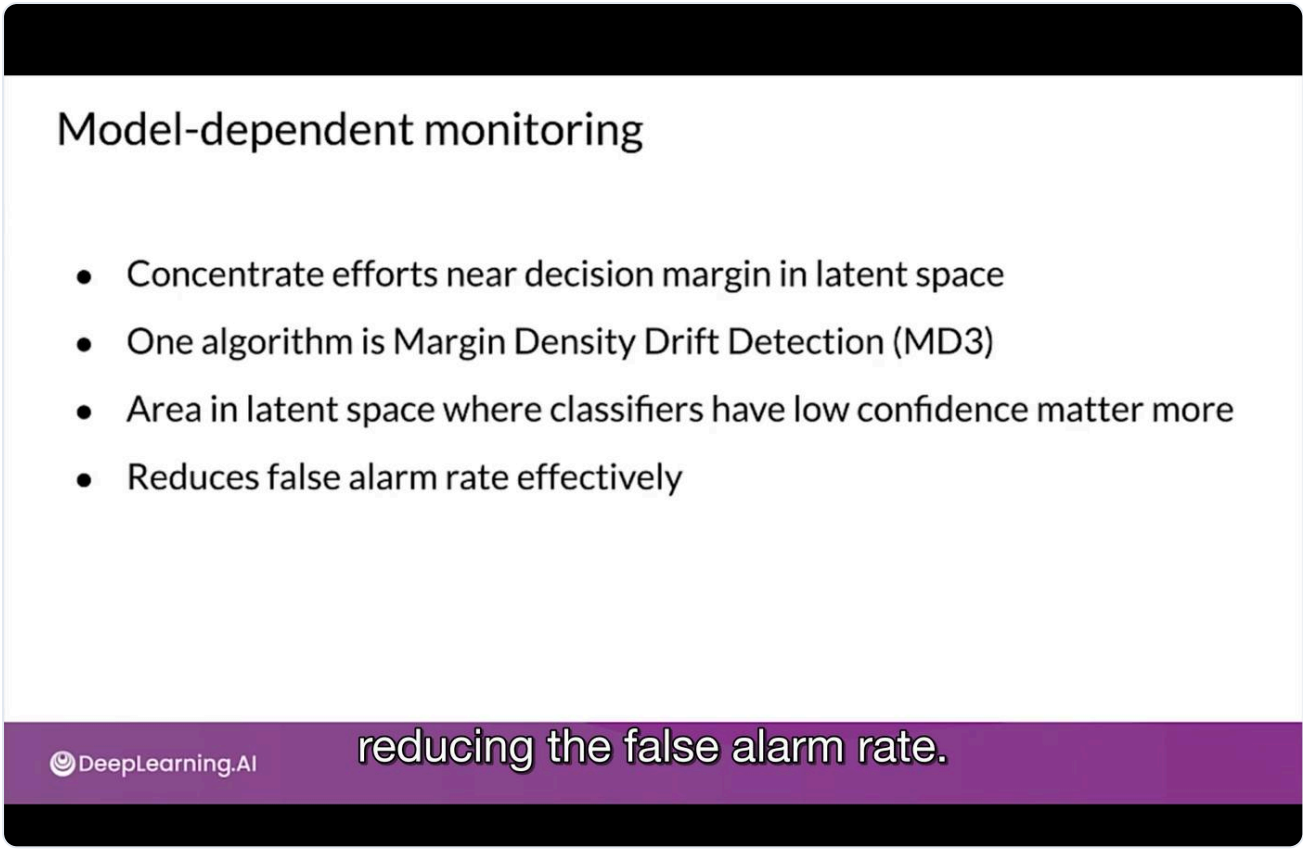

FIGURE 40.16

This method is very good at reducing the false alarm rate. There are many ways to evaluate different aspects of model performance, looking at both the incoming stream of requests and the predictions generated by the model. Cloud hosting providers, including Google, offer services which can be used for continuous evaluation of your data and your model. One such service is Microsoft Azure Machine Learning datasets which focuses on data drift. Another is Amazon SageMaker Model Monitor which focuses on concept drift. Here, let's take a look at Google Cloud AI continuous evaluation which also focuses primarily on concept drift, as an example of the services which are available.

Google Cloud AI continuous evaluation regularly samples prediction input and output from trained models that you've deployed to AI Platform Prediction. AI Platform Data Labeling service then assigns human reviewers to provide ground truth labels for a sample of the prediction requests that you receive. Or alternatively you can provide your own ground truth labels using a different technique. We discussed several labeling techniques previously. The data labeling service then compares your model's predictions with the ground truth labels to provide continual feedback on how your model is performing over time. You start by deploying a trained model to AI Platform Prediction as a model version, then you can create an evaluation job for the model version.

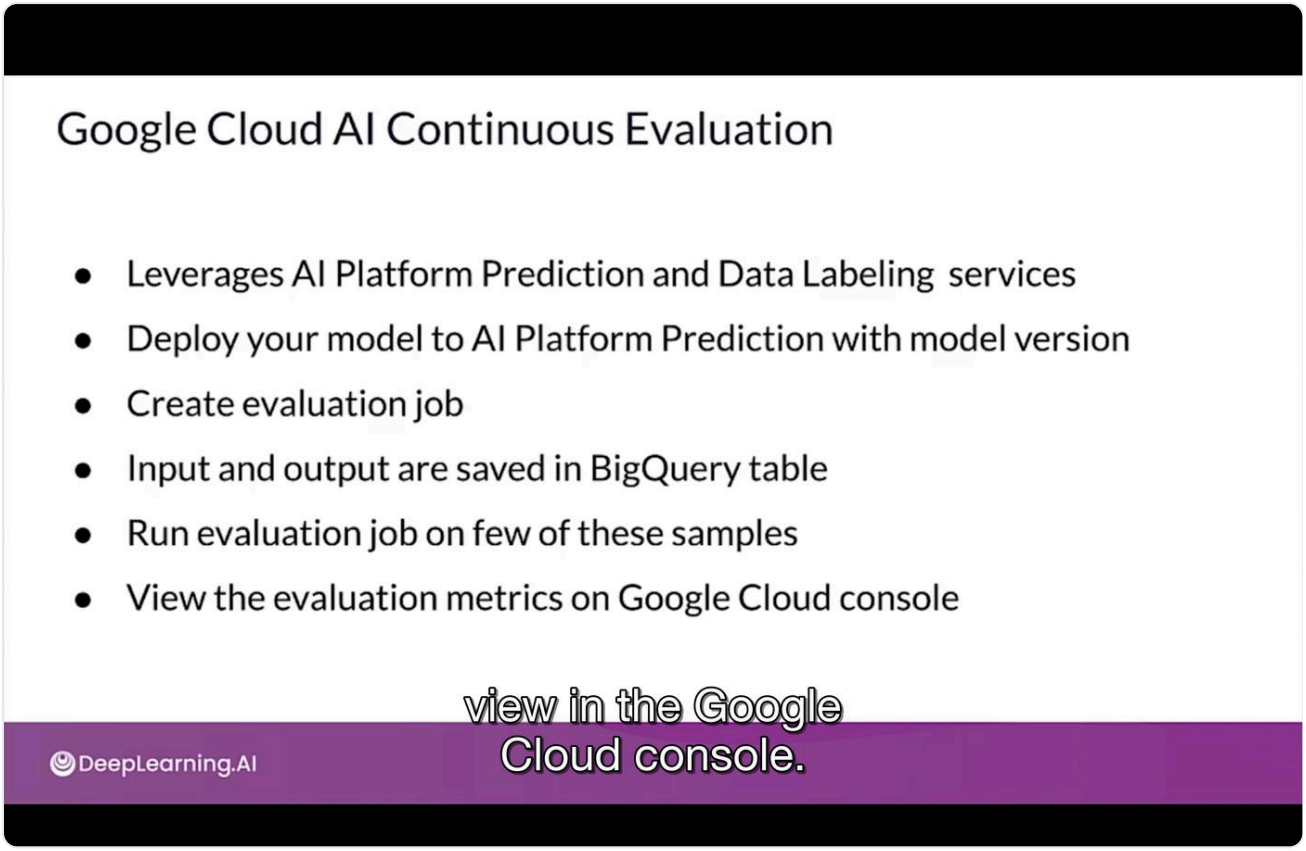

FIGURE 40.17 | Google Cloud AI Continuous Evaluation

As your model serves online predictions, the input and output for some of these predictions is saved in a BigQuery table. You can customize how much data gets sampled. Intermittently, the evaluation job runs generating evaluation metrics which you can view in the Google Cloud console. How often should you retrain your model? There's no fixed answer to this question. It largely depends on the data and the world. The rate of change of the data and the rate of change of the world that you are modeling will determine how often you need to retrain your model to adapt to change. Of course, you can retrain your model whenever you wish including when you've made improvements to your model's design.

Retraining too often is okay, but it can result in higher costs for computer resources. Not retraining often enough can lead to degraded model performance. Ideally, you should monitor your data and model well enough to be able to use your evaluation results to trigger retraining automatically.

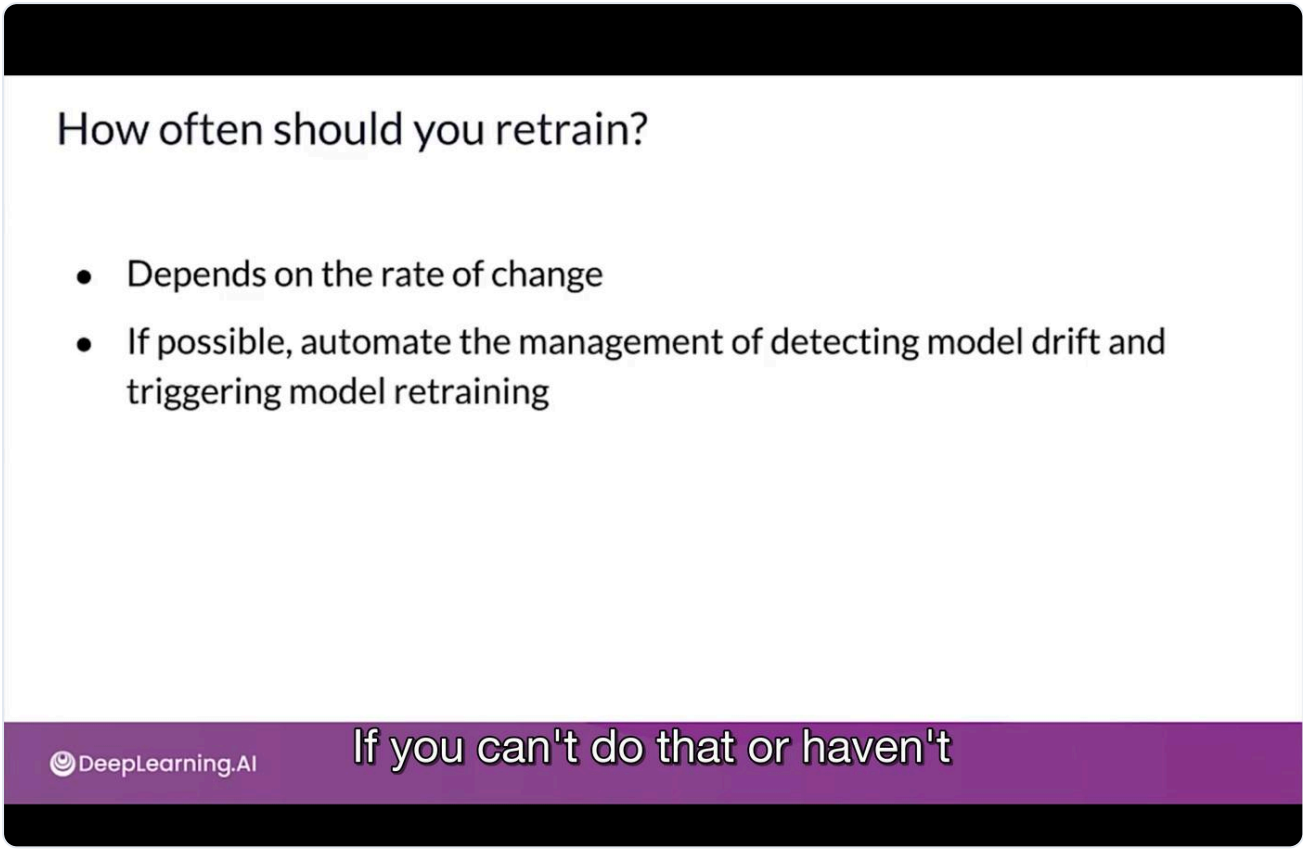

FIGURE 40.18 | How Often Should You Retrain?

If you can't do that or haven't reached the level of maturity in your deployment, then you can also just retrain on a schedule.

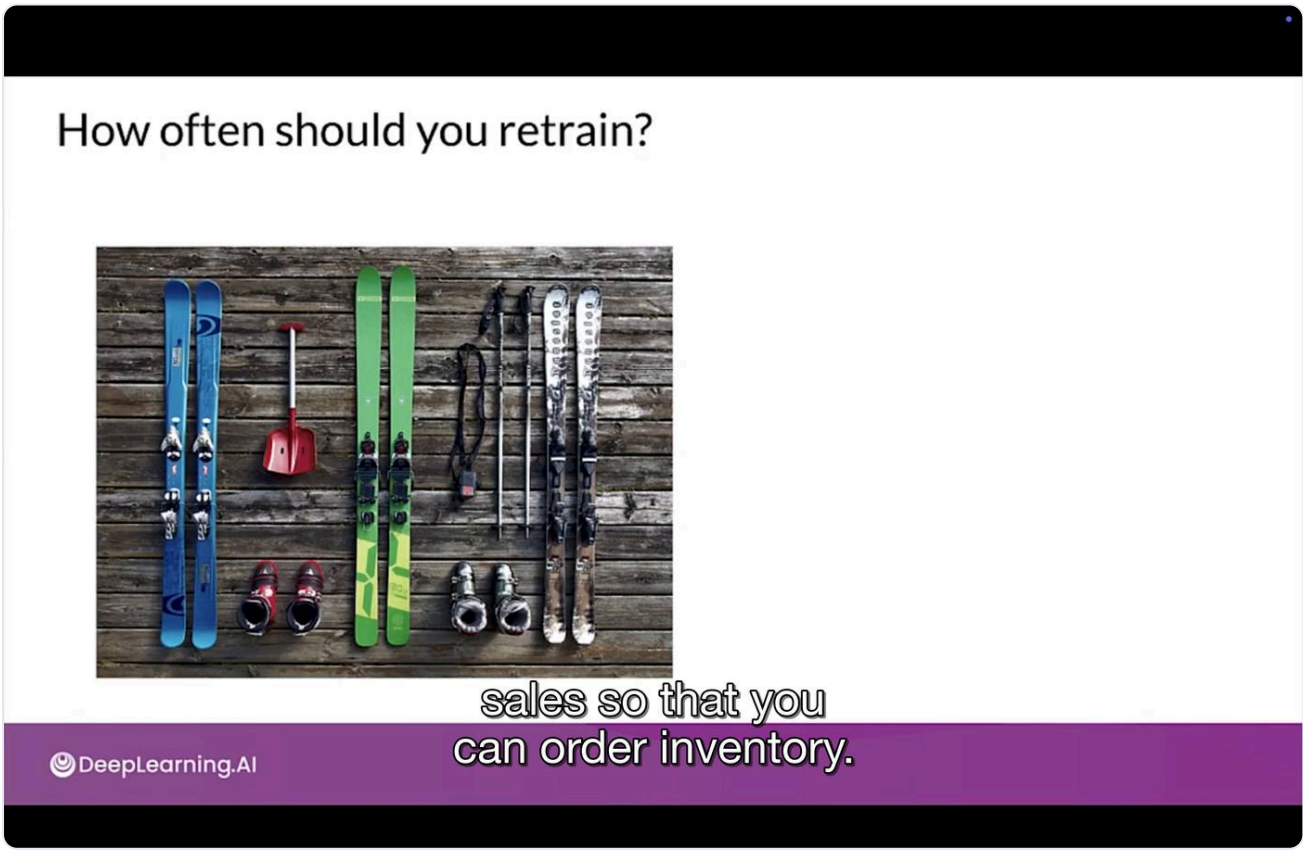

FIGURE 40.19 | How Often Should You Retrain?

For example, imagine that you're selling sporting goods and using your model to predict sales so that you can order inventory. In the winter, your model is performing well and your inventory ordering is nearly perfect. Your customers are buying winter clothes and winter sports equipment like skis.

FIGURE 40.20 | How Often Should You Retrain?

However, as spring approaches, you start to see changes in customer behavior. The same customers that we're buying skis before start to buy tennis rackets. But your model is still predicting skis and you're in danger of ordering too many. Because you had continuous monitoring in place, you catch the problem early and retrain your model with new data from recent customer interactions and sales. Now, your inventory is back to nearly perfect again. Well, it was quite a week. We started with model performance analysis at a fairly deep level, including a deep dive into TensorFlow Model Analysis or TFMA, which is a great tool for understanding your model performance.

Then we took a look at model debugging and robustness, including sensitivity analysis and understanding adversarial attacks as well. Then we looked at model remediation, which is of course very important, and fairness as well, which we looked at ways to measure fairness and ways to try to deal with it as well. Finally, we finished up with continuous evaluation and monitoring, which are of course very important for understanding when you need to retrain your model and how to measure your model performance throughout its life. It was a lot to go over, I hope you enjoyed it, I did, and I'll see you next time.

LECTURE 41

Need for Explainability in AI

Welcome back. This week is all about interpretability, explaining to yourself and probably to other people why your model did what it did. It's important increasingly in a lot of production ML for a number of different reasons including fairness, regulatory requirements, and legal requirements, as well as just being able to understand your model better so that you can either fix problems with it or improve it. Interpretability, let's get started. We'll start by discussing explainable AI. Interpretability is becoming increasingly important at the same time that it's becoming increasingly difficult with more and more complex models. But the good news is that the techniques for achieving interpretability are improving as well.

Concretely, you could interpret that to mean that we're going to study interpretability.

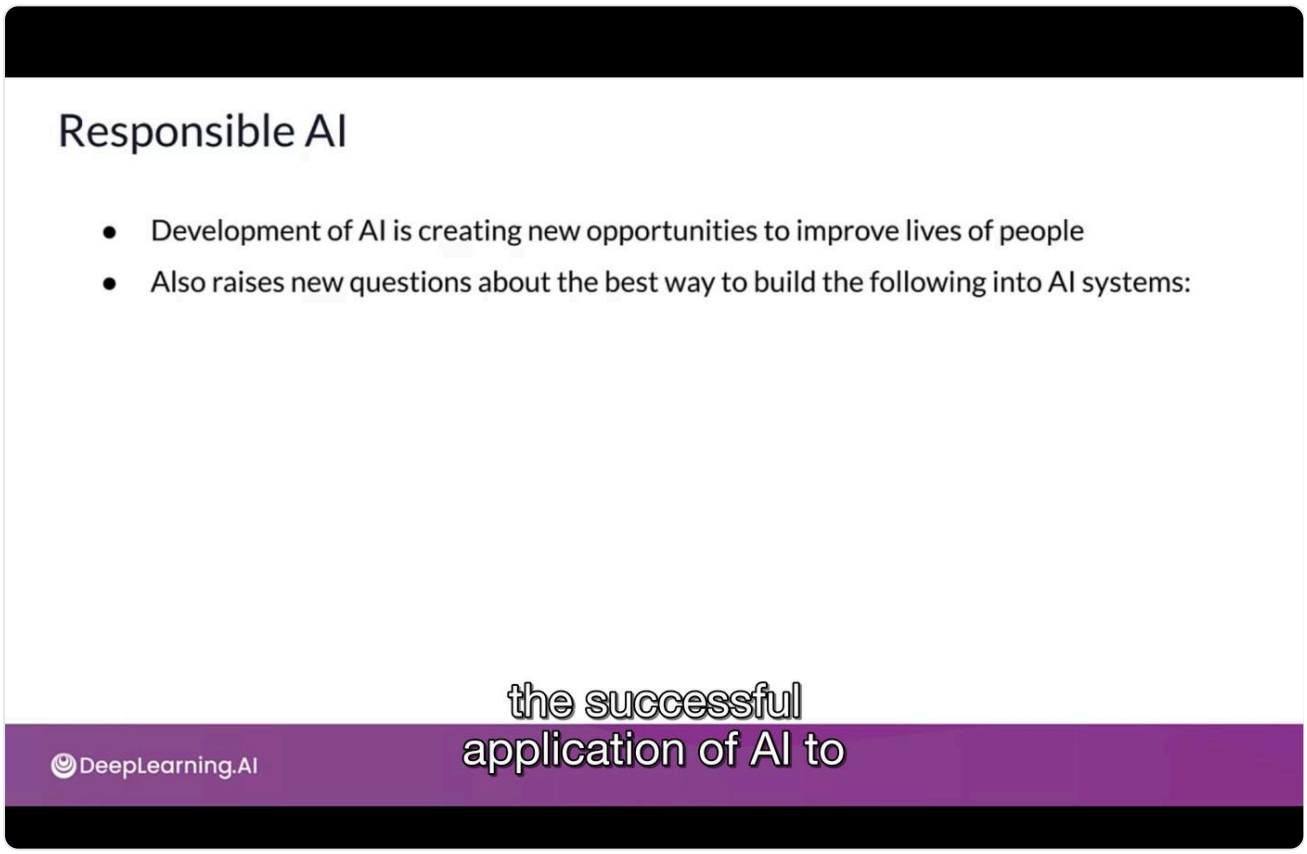

FIGURE 41.1

Interpretability and explainability are parts of a larger field known as responsible AI. The development of AI and the successful application of AI to more and more problems has resulted in the rapid growth of the ability to perform tasks which were previously not possible. This has created many great new opportunities. But at the same time, it also puts a lot of power into the results of models. There are sometimes questions about models and how responsibly they handle a number of factors which influence people and can cause harm. Issues of fairness, which we've talked

about previously, are central to Responsible AI. Explainability in a smaller sense, interpretability are key to being able to do Responsible AI because we need to understand how models generated their results.

Privacy is also part of responsible AI, since models often operate with Personally Identifiable Information or PII, and they are often trained with PII.

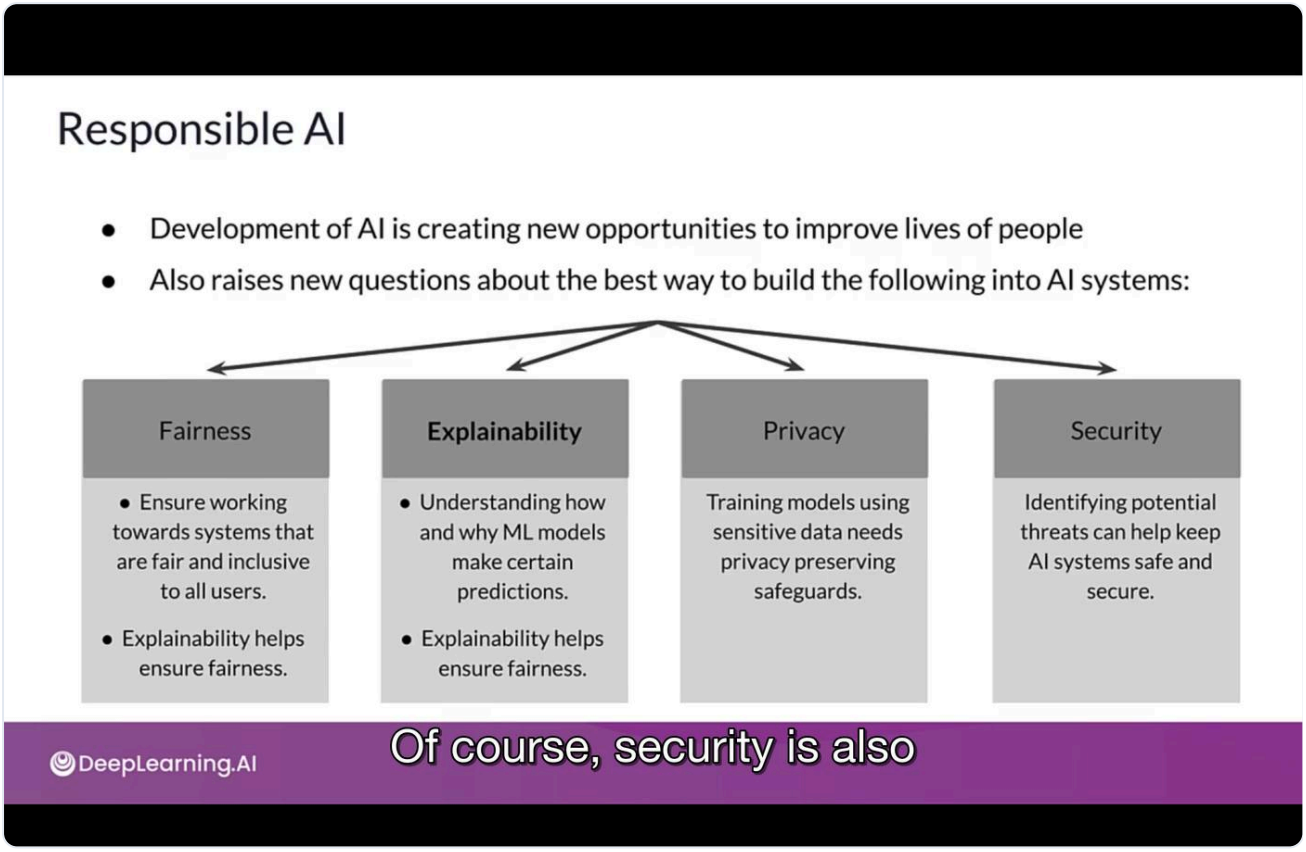

FIGURE 41.2

Of course, security is also an issue and related to privacy. Since one of the attacks that we've talked about is to pull the training data out of a model, which could mean pulling private information from a model. The results generated by a model can be explained in different ways. One of the most advanced techniques is to create a model architecture that is inherently explainable. A simple example of this is a decision tree-based model, which by its nature is explainable. But they're increasingly more advanced and complex model architectures that are also designed to be inherently explainable. This is the field of Explainable Artificial Intelligence or XAI.

Explainability is important in many ways.

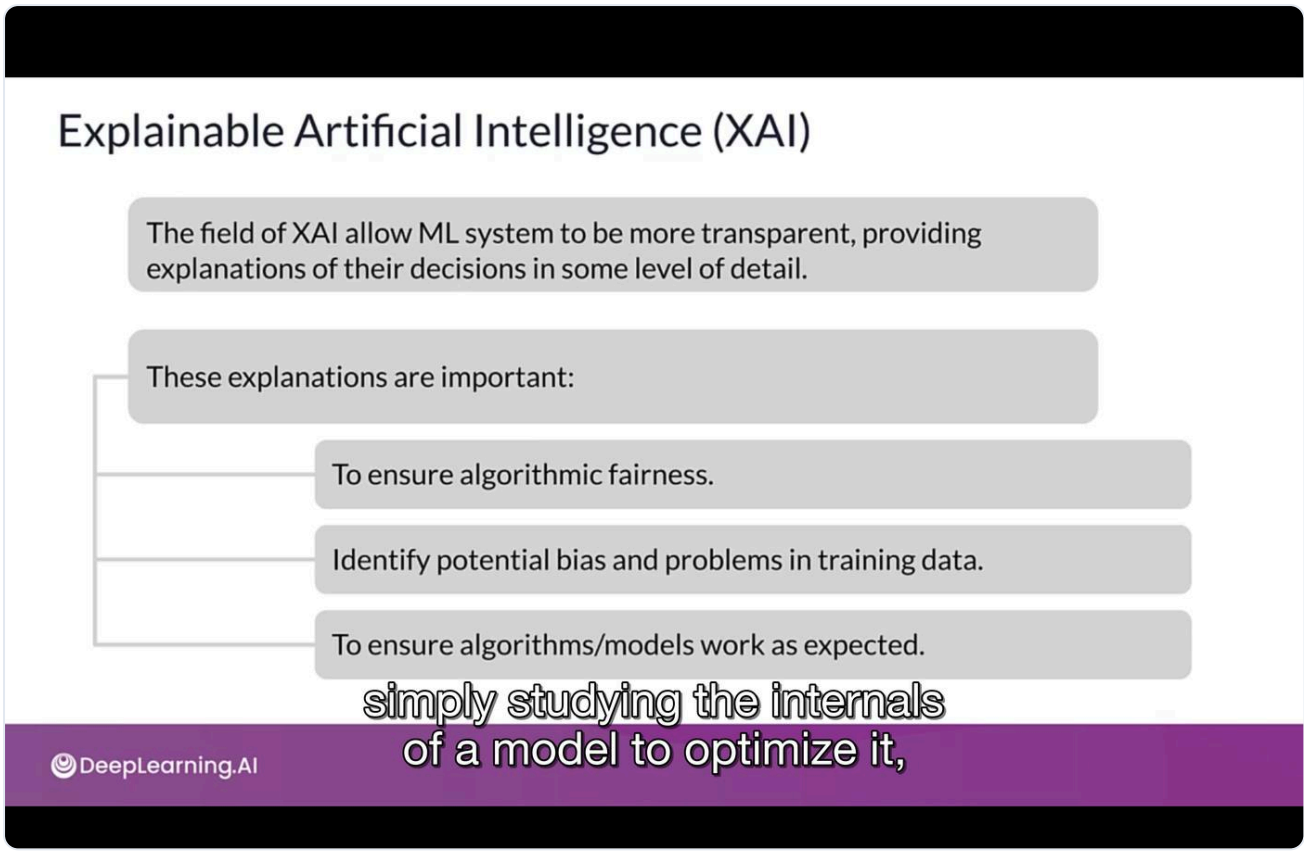

FIGURE 41.3 | Explainable Artificial Intelligence (XAII)

These include: ensuring fairness, looking for issues with bias in the training data, regulatory, legal, and branding concerns, and simply studying the internals of a model to optimize it, to produce the best results. Why is explainability in AI so important? Well, fundamentally, it's because we need to explain the results and the decisions that are made by our models. This is especially true for models with high sensitivity, including natural language models, which when confronted with certain examples, can generate wildly wrong results. It also includes vulnerability to attacks which we need to evaluate on an ongoing basis and not just after an attack has already happened.

Of course, fairness is a key issue.

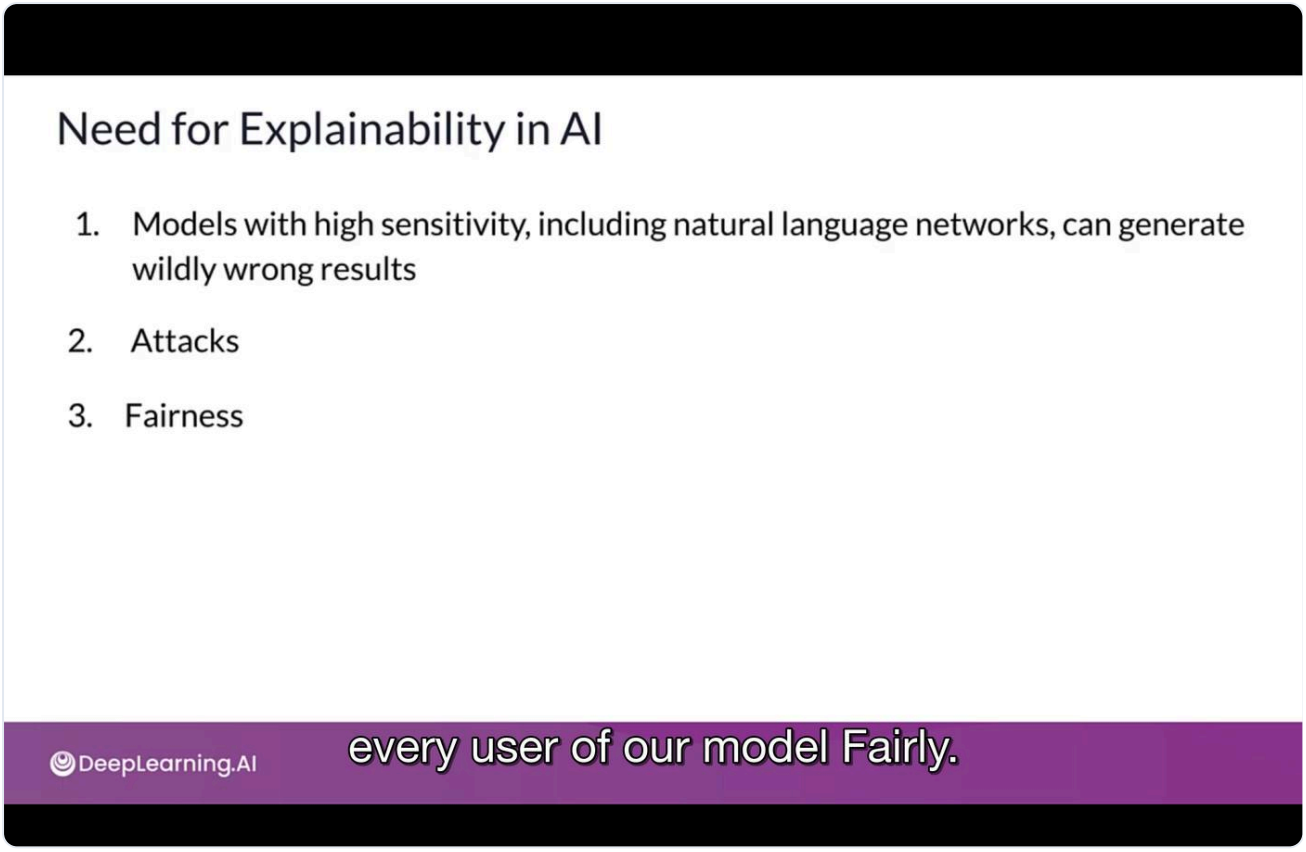

FIGURE 41.4 | Need for Explainability in AI

Since we want to make sure that we're treating every user of our model Fairly.

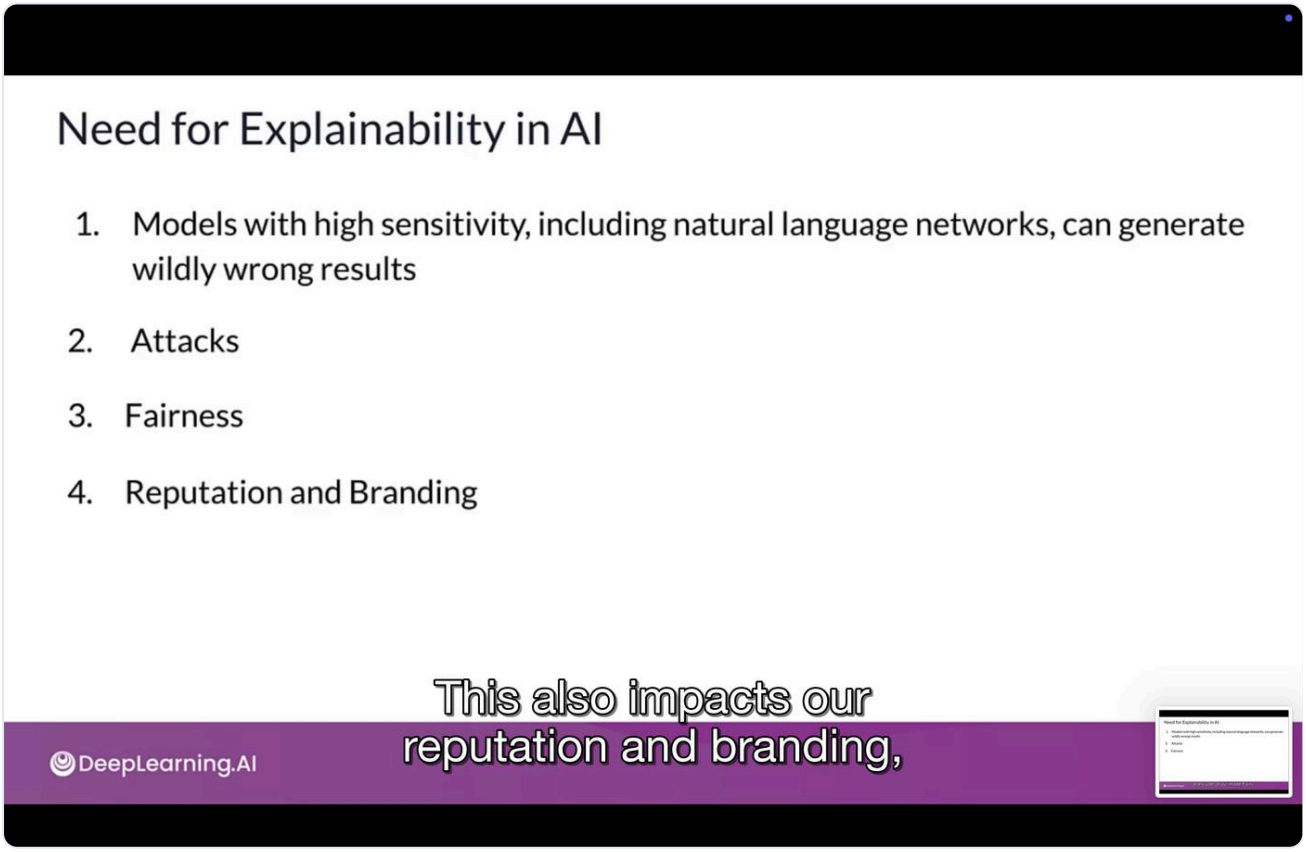

FIGURE 41.5 | Need for Explainability in AI

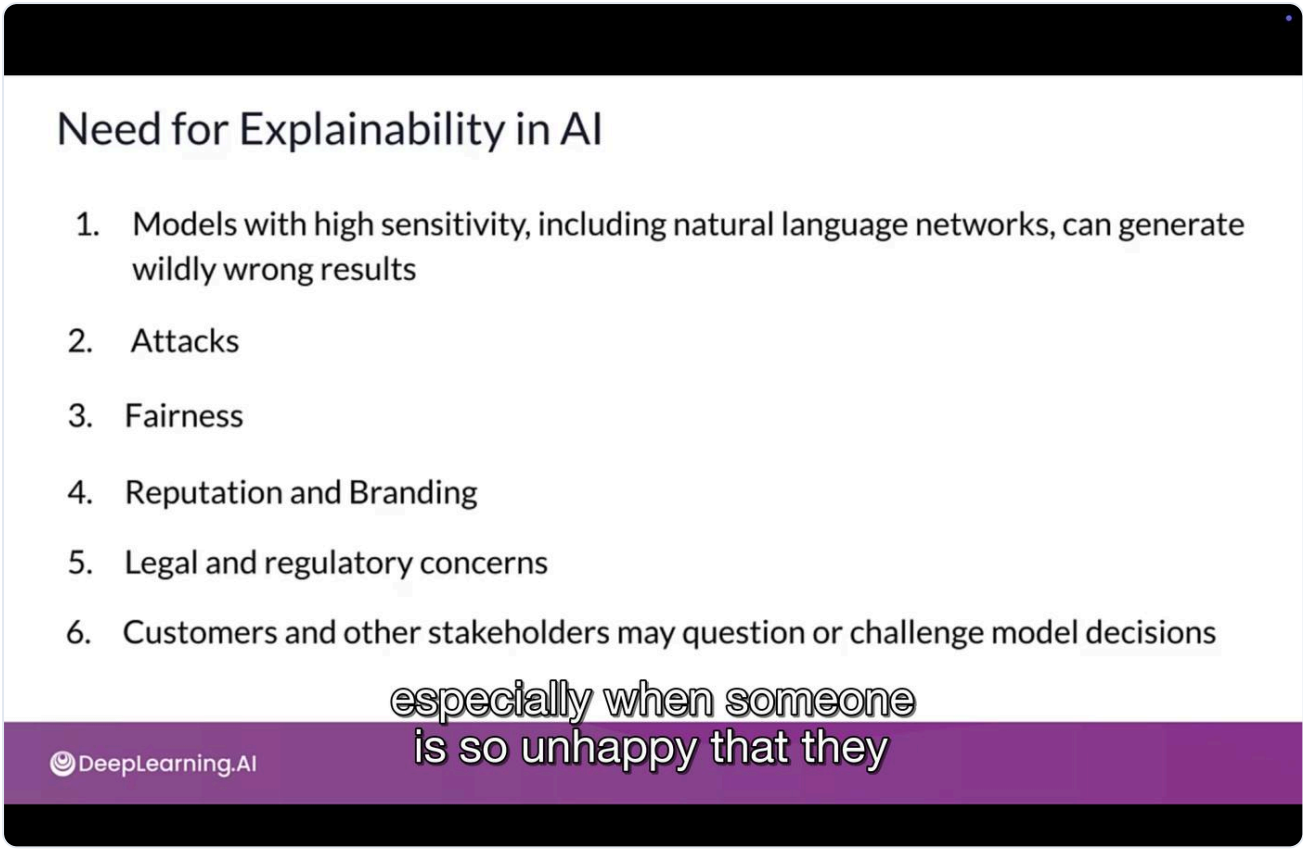

FIGURE 41.6 | Need for Explainability in AI

This also impacts our reputation and branding, especially in cases where customers or other stakeholders may question or challenge our models decision. But really in any case, where we generate a prediction, and of course, there are legal and regulatory concerns, especially when someone is so unhappy that they challenge us and our model in court, or when our models result in an action that causes harm.

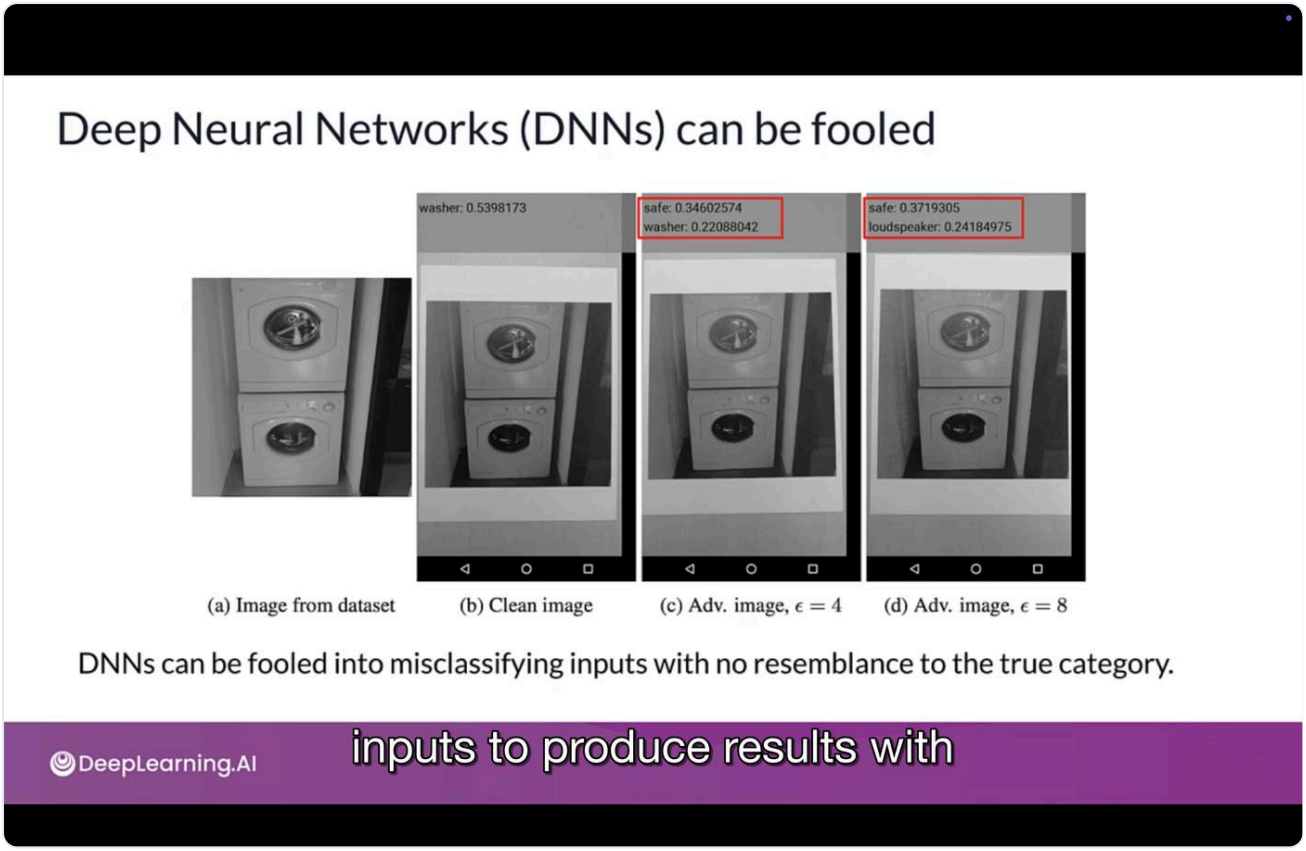

FIGURE 41.7

Deep Neural Networks can be fooled into misclassifying inputs to produce results with no resemblance to the true category. This is easiest to see in examples of image classification, but fundamentally it can occur with any model architecture. The example on this slide demonstrates a black-box attack in which the attack is constructed without access to the model. The example is based on a phone app for image classification. Using physical adversarial examples. What you see is a clean image of a washer from the dataset, the image a on the left. That's used to generate adversarial images with various degrees of perturbation. The next was printing out the clean and adversarial images and then using the TensorFlow camera demo app to classify them.

The clean image b is recognized correctly as a washer when perceived through the camera while increasing the adversarial perturbation in images c and d results in greater misclassification. The key result that is in image d, where the model thinks that the washer is either a safe or a loudspeaker, but definitely not a washer. Looking at the image, would you agree with the model? Maybe not. Can you see the adversarial perturbation that was applied? It's not that easy to see.

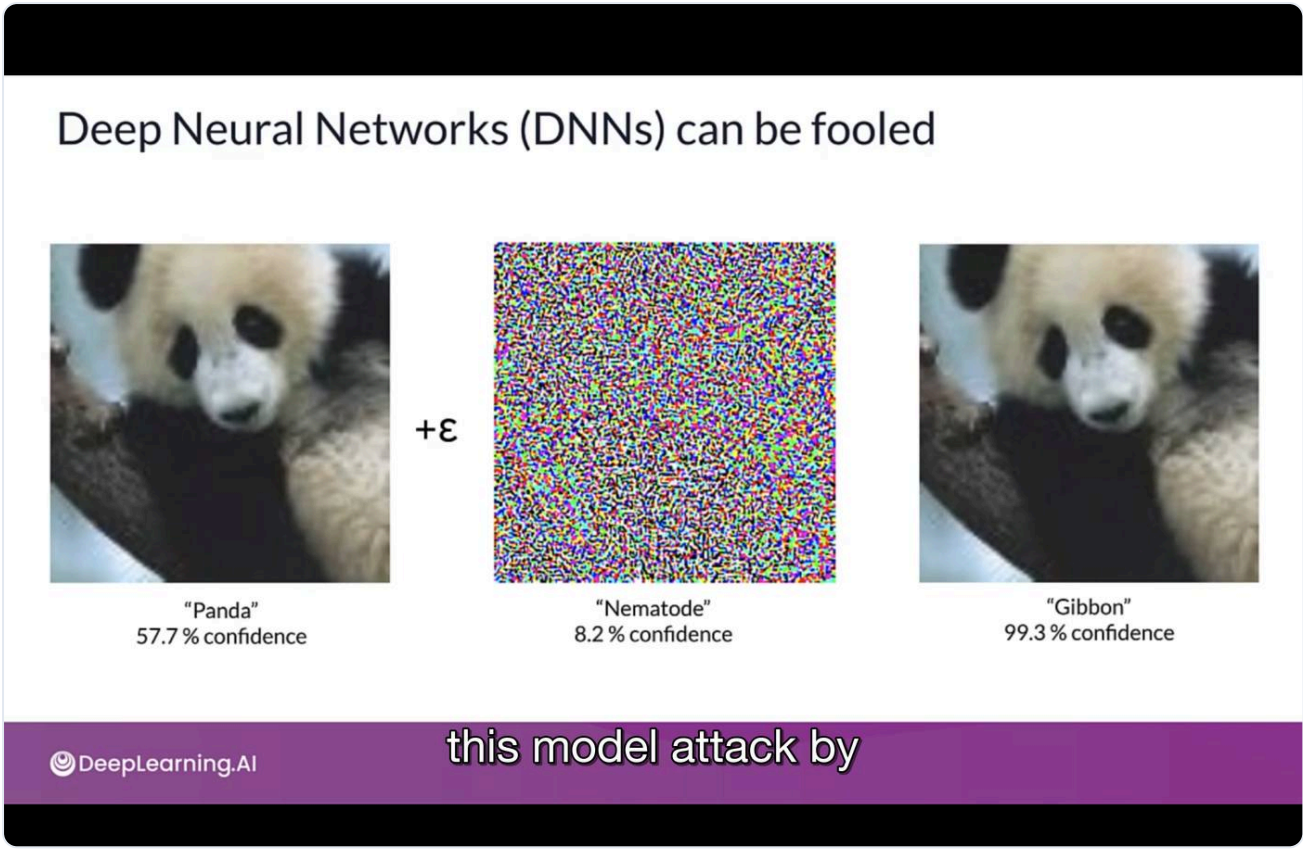

FIGURE 41.8 | Deep Neural Networks (DNNs) Can Be Fooled

This is perhaps the most famous example of this model attack by adding an imperceptibly small amount of well-crafted noise, an image of a panda can be misclassified as a gibbon, with 99.3 percent confidence. That's much higher than the original confidence that the model had, that it was a panda.

LECTURE 42

Interpretability of ML Models

Let's look now at some of the basic ways of interpreting models. There are two broad overlapping categories; techniques which can be applied to models in general and model architectures which are inherently interpretable. Interpretability itself does not have a mathematical definition. Biran and Cotton provided one good definition of interpretability.

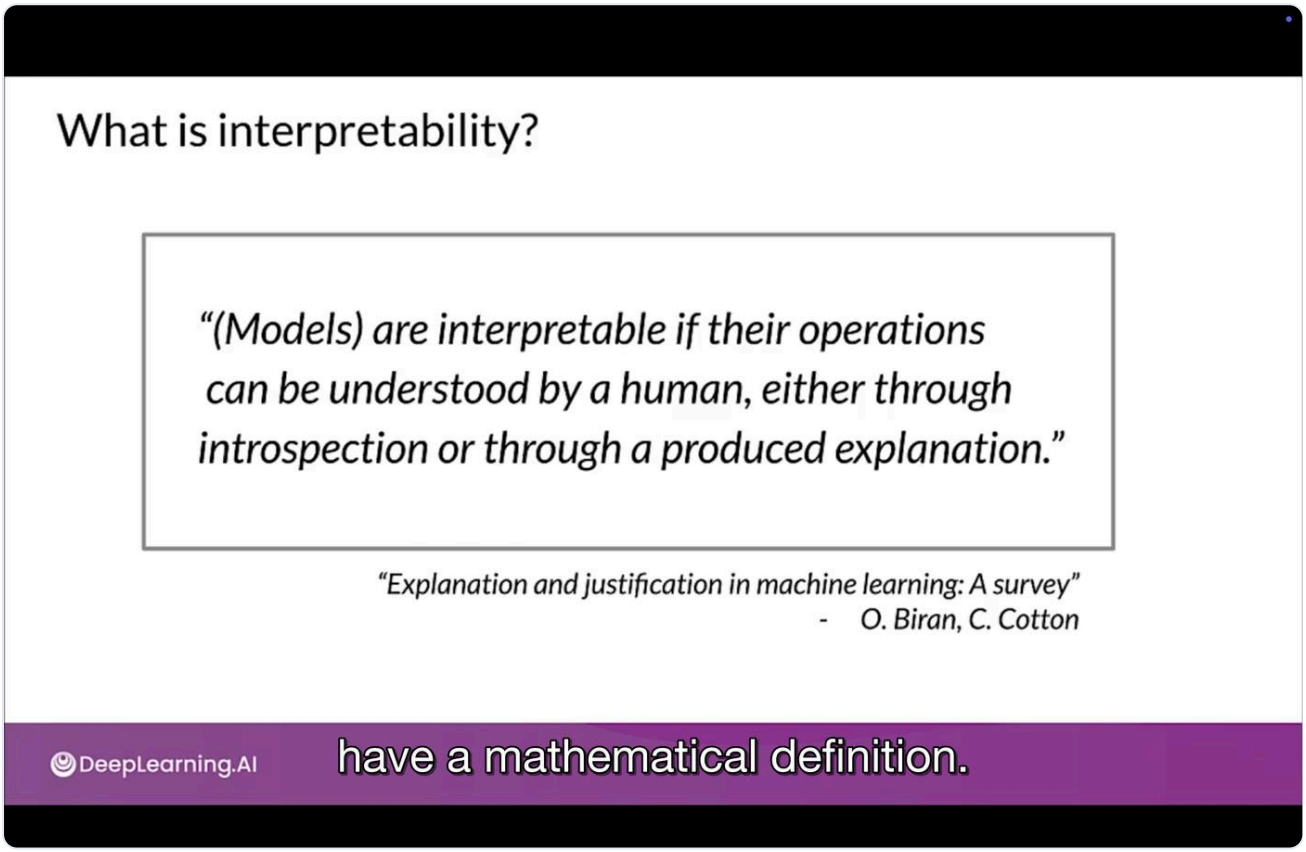

FIGURE 42.1 | What Is Interpretability?

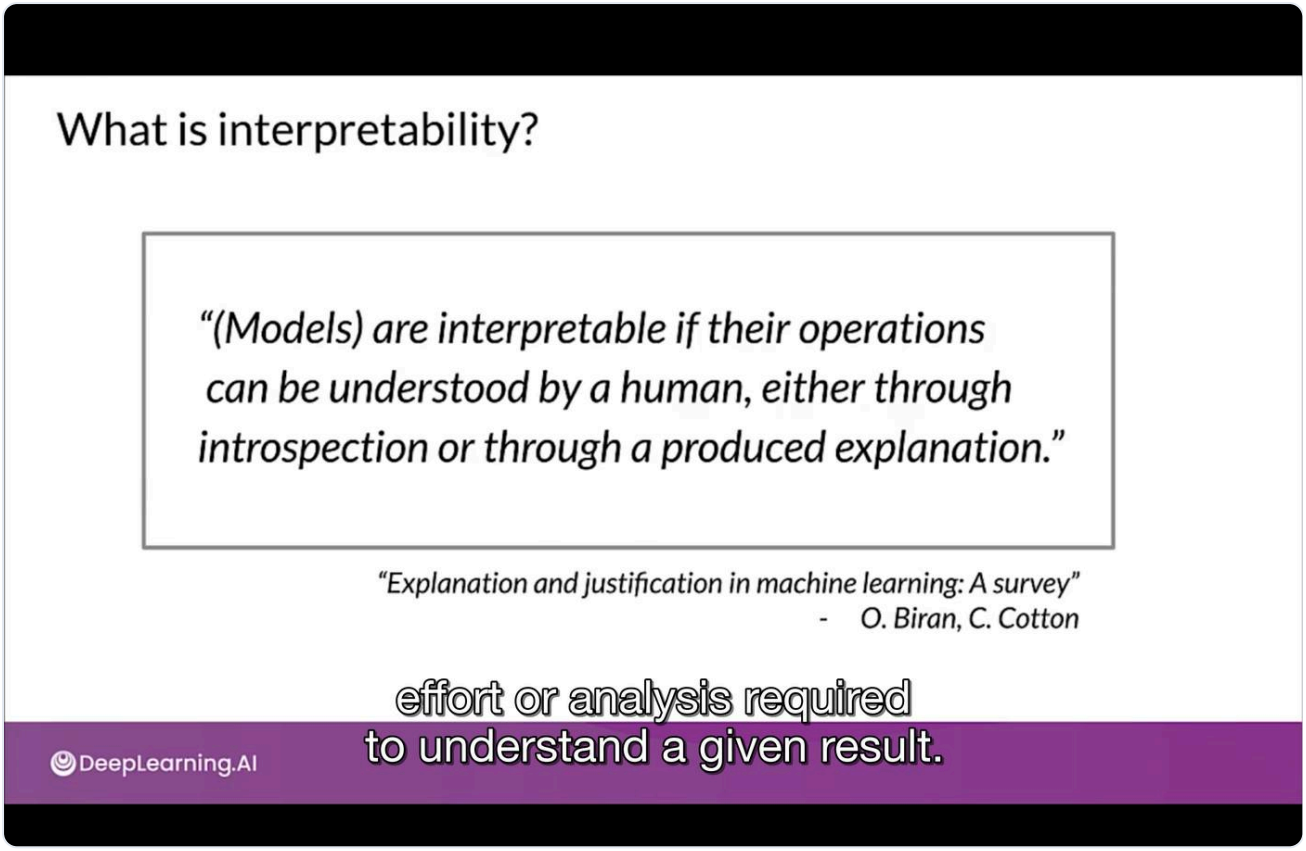

FIGURE 42.2 | What Is Interpretability?

They wrote that in systems, or in this case models, they are interpretable if their operations can be understood by a human either through introspection or through a produced explanation. In other words, if there's some way for a human to figure out why a model produced a certain result, then the model is interpretable. Practically speaking, however, the level of effort required needs to be feasible as well. One measure of interpretability of models is the amount of effort or analysis required to understand a given result. Ideally, you should be able to query the model to understand the what, why, and how of its algorithmic decisions. Why did the model behave in a certain way?

You should be able to identify and validate the relevant variables driving the model's outputs. Doing so will help you develop trust in the reliability of the predictive system even in unforeseen circumstances. This diagnosis will help ensure accountability and confidence in the safety of the model. How can we trust the predictions made by the model? Well, you should be able to validate any given data point to demonstrate to business stakeholders and peers that the model works as expected. This will help ensure the transparency of the model. What information can the model provide to avoid prediction errors?

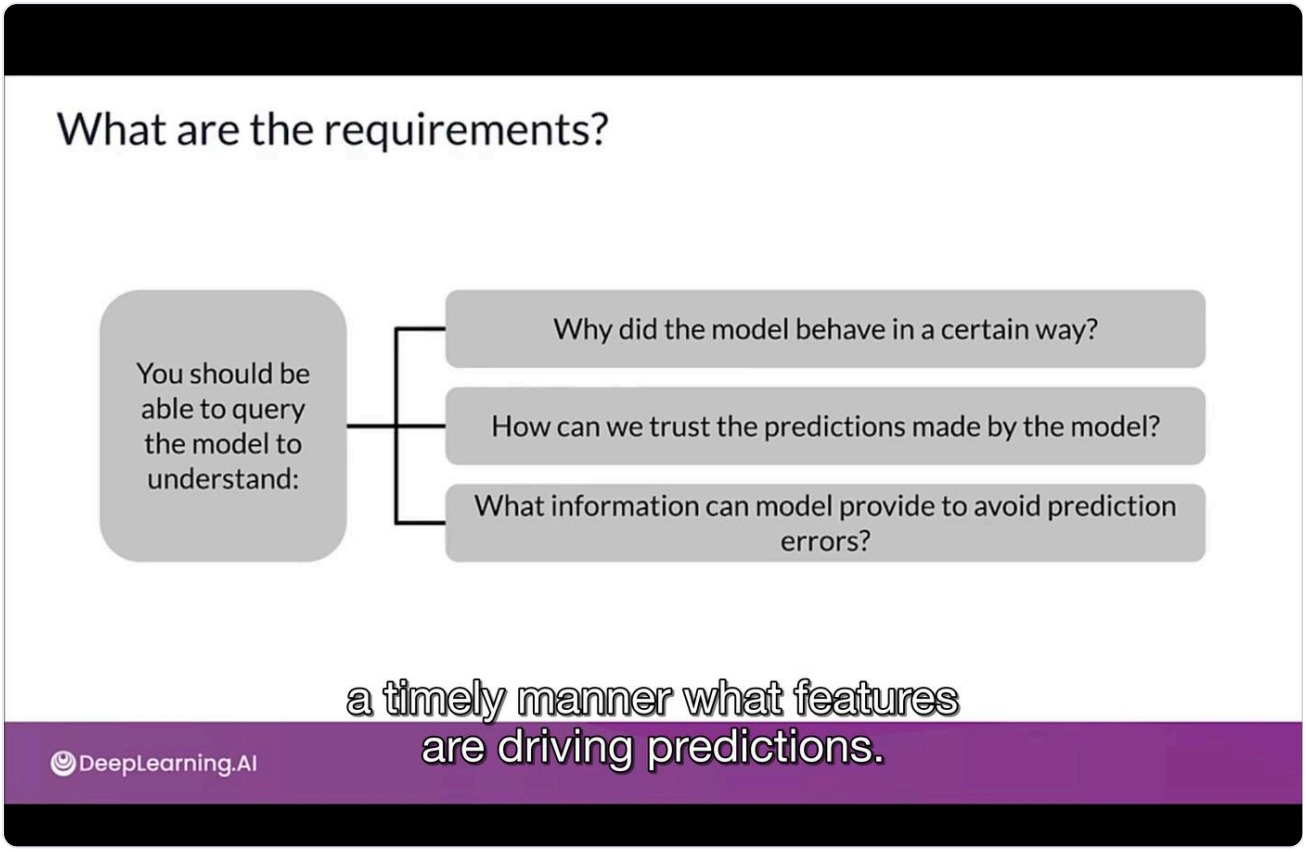

FIGURE 42.3 | What Are the Requirements?

You should be able to query and understand latent variable interactions in order to evaluate and understand in a timely manner what features are driving predictions. This will help you to ensure the fairness of the model. There are some criteria that can be used for categorizing model interpretation methods. For example, interpretability methods can be grouped based on whether they're intrinsic or Post-Hoc. They could also be model-specific or model agnostic.

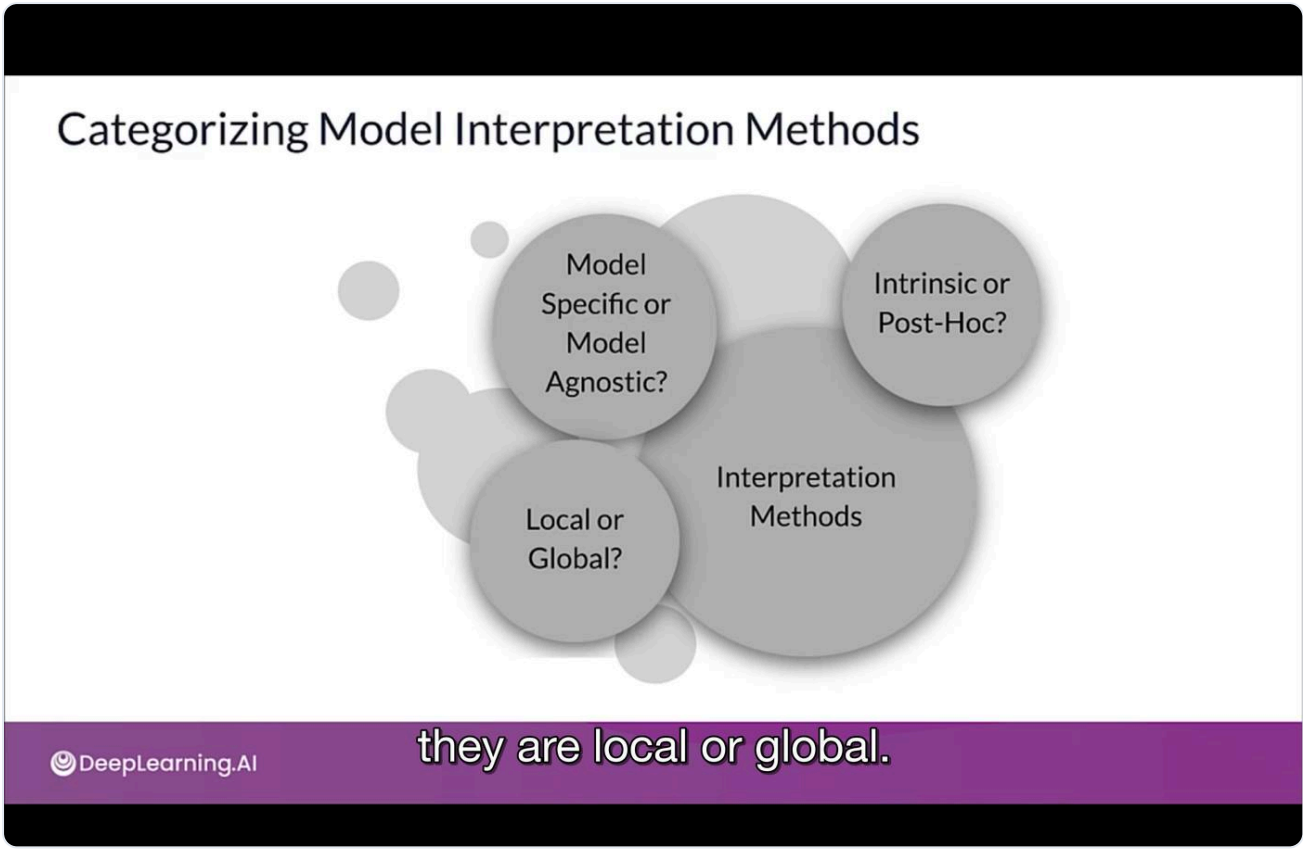

FIGURE 42.4 | Categorizing Model Interpretation Methods

They can also be grouped according to whether they are local or global. Let's discuss each of these criteria. One way of grouping model interpretability methods is by whether the model itself is intrinsically interpretable. Model architectures that are intrinsically interpretable have been around for a long time. The classic examples of this are linear models and tree-based models. More recently, however, more advanced model architectures such as lattice models have been developed to enable both interpretability and a high degree of accuracy on complex modeling problems.

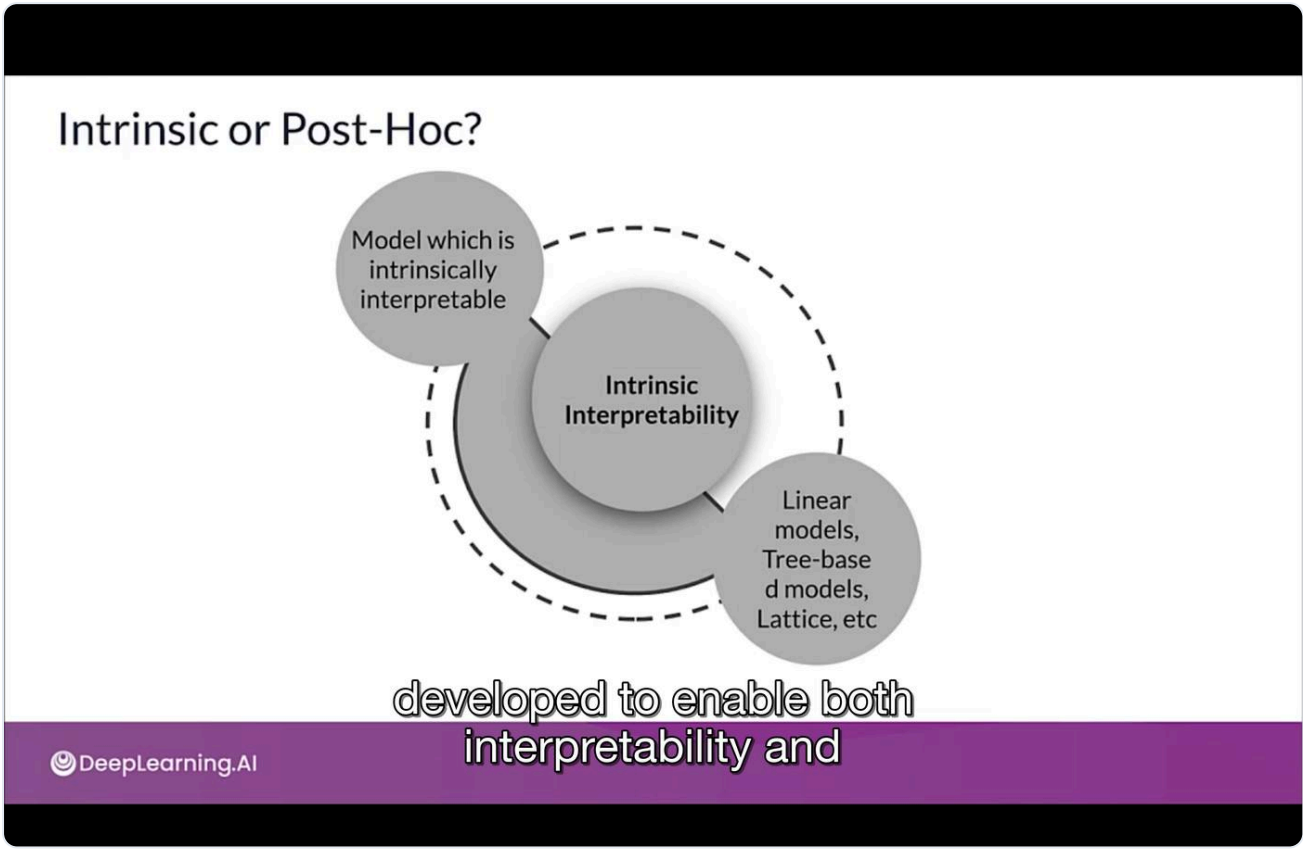

FIGURE 42.5

Lattice models, for example, can match or in some cases exceed the accuracy of neural networks. Let's consider Post-Hoc methods. Post-hoc methods treat models as black boxes and often don't distinguish between different model architectures. They tend to treat all models the same and are applied after training to try to examine particular results to understand what caused the model to generate them. There are some methods, however, especially for convolutional networks, that do inspect the layers within the network to try to understand how results were generated. There is, however, always some level of uncertainty about whether the interpretation of the reasons for certain results is correct or not since Post-hoc methods don't evaluate the actual sequence of operations that led to the generation of the results.

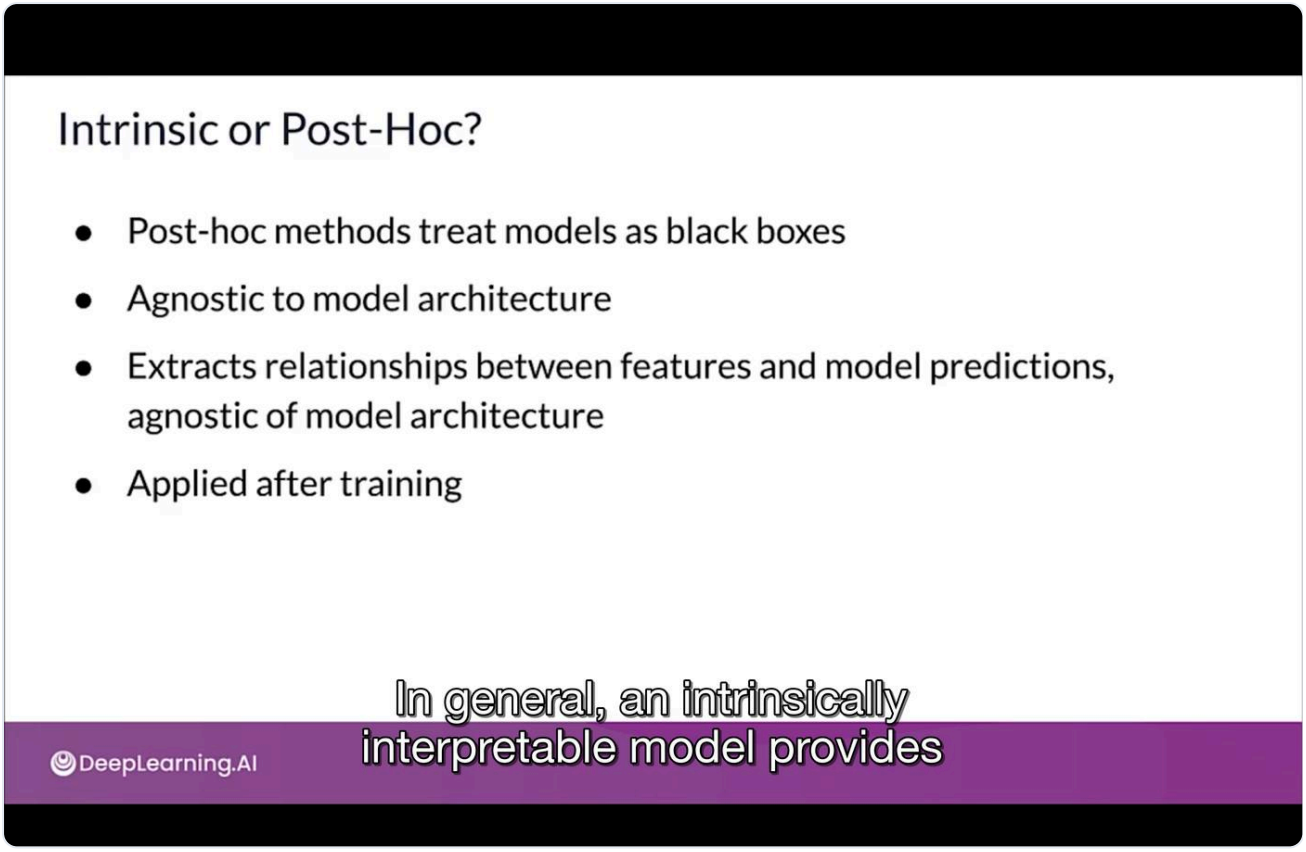

FIGURE 42.6

In general, an intrinsically interpretable model provides a higher degree of certainty as to why it generated a particular result. Examples of these analyses include feature importance and partial dependency plots. The various interpretation methods can also be roughly classified according to the type of results they produce. Some methods create a summary of feature statistics. Some methods return a single value for a feature. For example, the feature importance returns a single number per feature. A more complex example would be pairwise feature interaction strength, which associates a number with each pair of features. Some methods rely on visualization to summarize features.

For example, partial dependence plots. Partial dependence plots are curves that show a feature and its average predicted output. In this case, drawing the curve is more meaningful and intuitive than simply representing the values in a table. Some model-specific methods look at model internals. The interpretation of intrinsically interpretable models falls into this category. For example, in case of less complex models such as linear models, you can look at the learned weights to produce an interpretation. Similarly, the learned tree structure in a tree-based model serves as an interpretation. In lattice models, the parameters of each layer are the output of that layer, which makes it relatively easy to analyze, understand, and debug each part of the model.

Some methods examine particular data points.

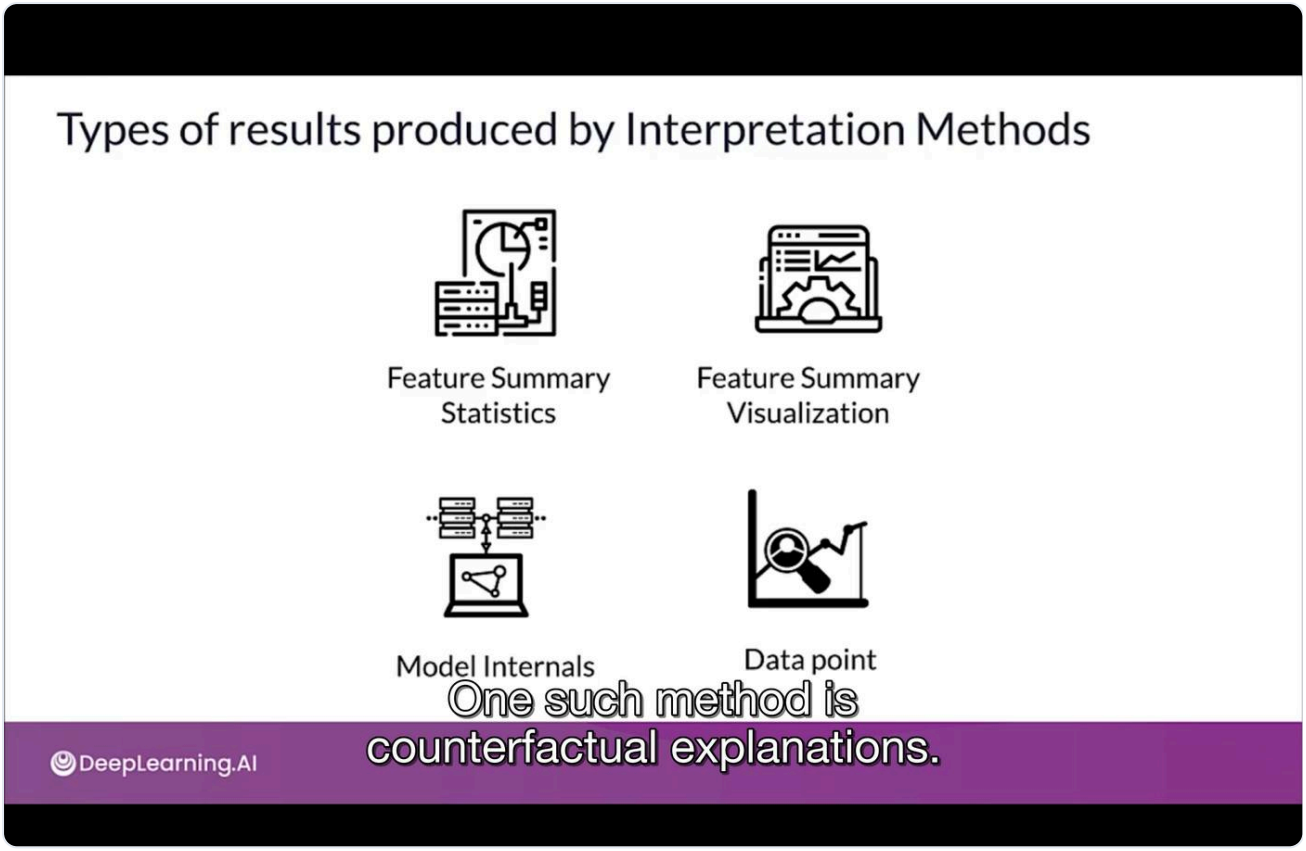

FIGURE 42.7 | Types of Results Produced by Interpretation Methods

One such method is counterfactual explanations. Counterfactual explanations are used to explain the prediction of a data point. In order to do so, it finds another data point by changing some features so that the predicted output changes in a relevant way. The change should be significant. For example, the new data point should be predicting a different class. Model-specific methods are limited to specific model types.

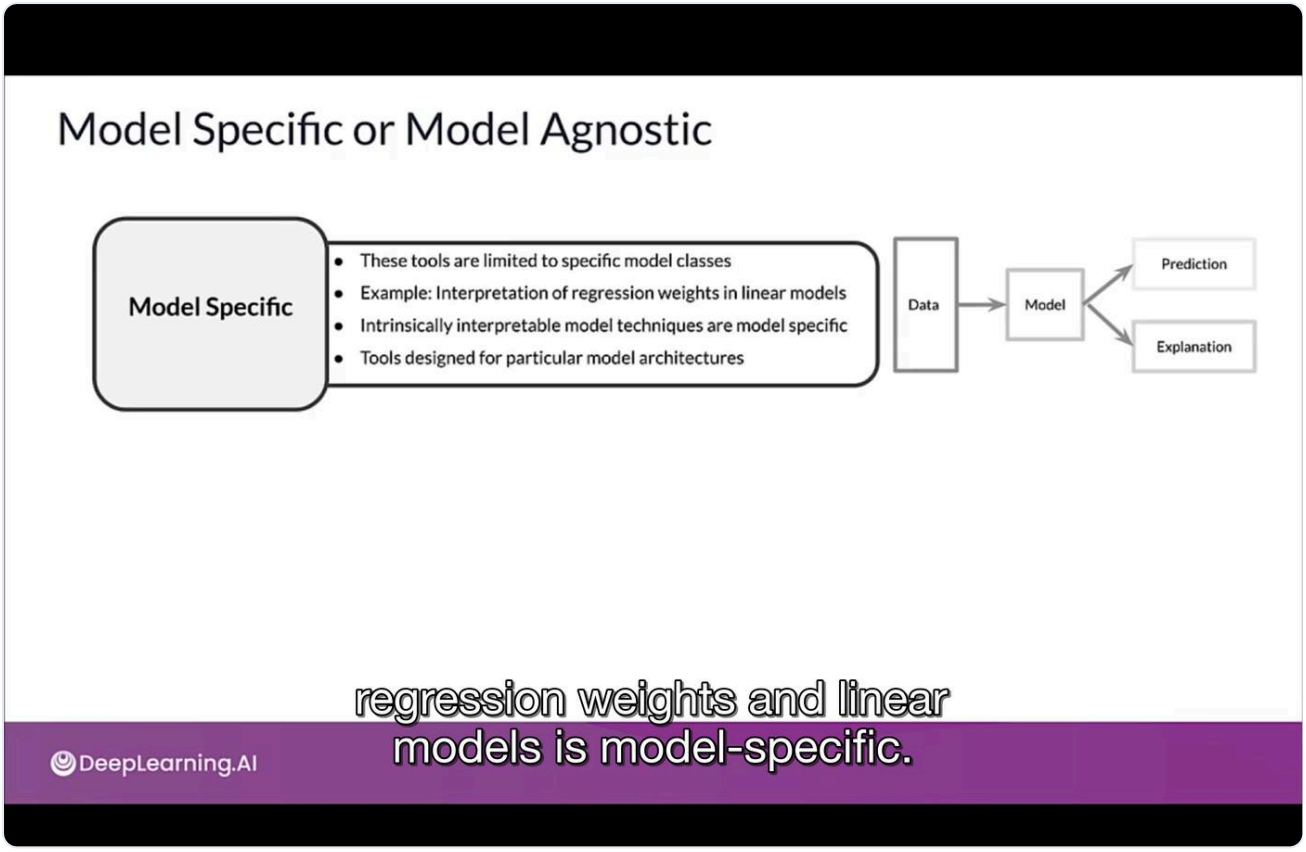

FIGURE 42.8 | These Tools Are Limited to Specific Model Classes

For example, the interpretation of regression weights and linear models is model-specific. By definition, the techniques for interpreting intrinsically interpretable models are model-specific, but model-specific methods are not limited to intrinsically interpretable models. There are also tools that specifically focus on neural network interpretation. Model agnostic methods are not specific to any particular model. They can be applied to any model after it's trained. Essentially, they are Post-hoc methods.

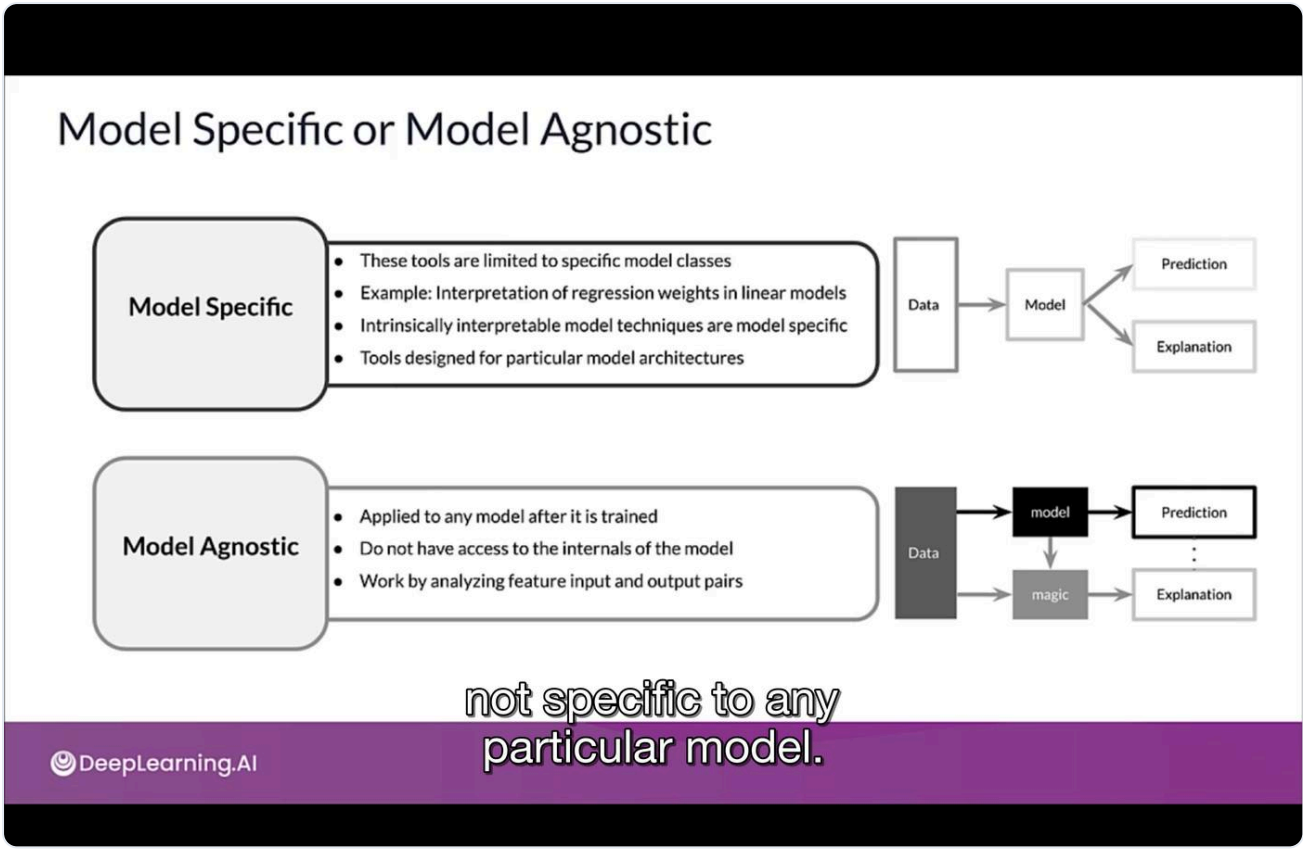

FIGURE 42.9 | Model Specific or Model Agnostic

These methods do not have access to the internals of the model such as weights and parameters, etc. They usually work by analyzing feature input and output pairs and trying to infer relationships.

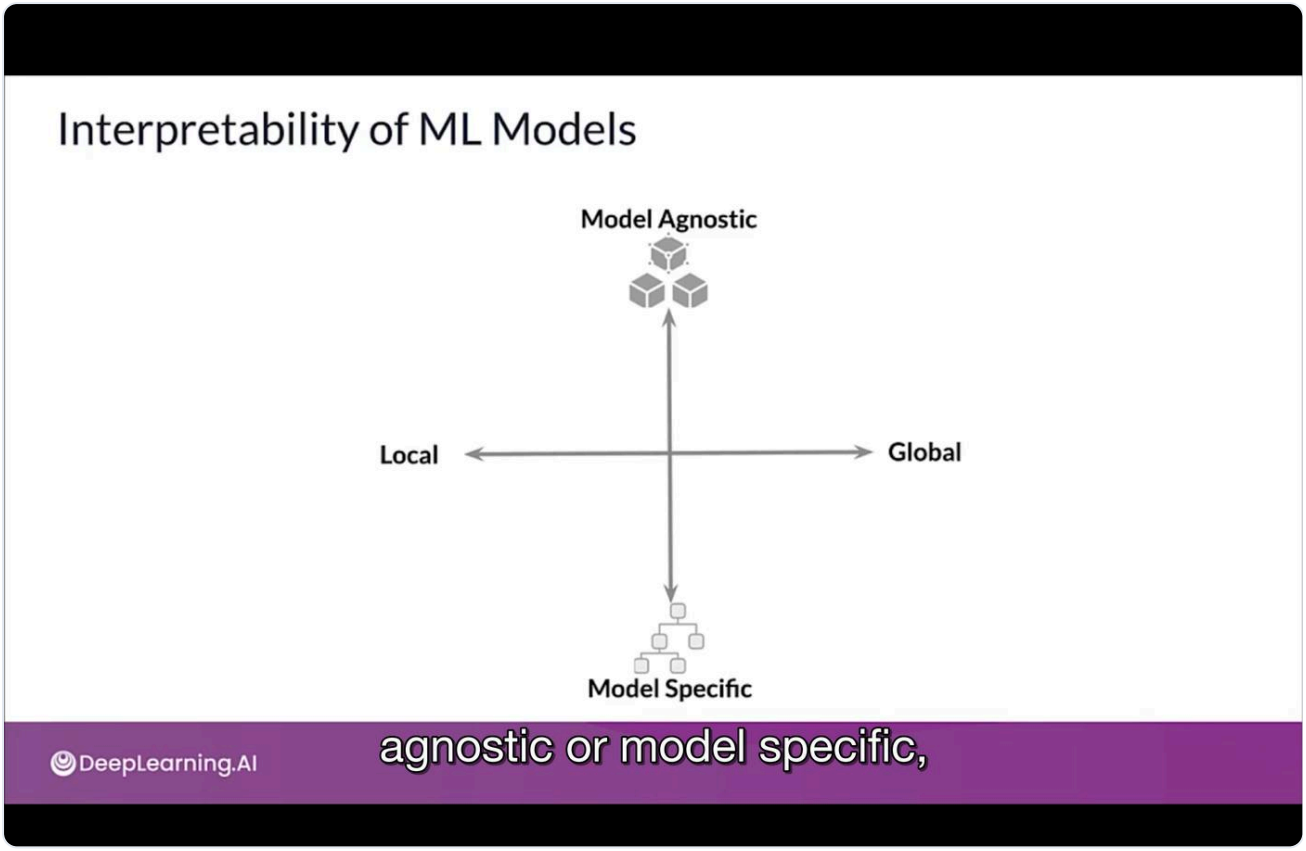

FIGURE 42.10 | Interpretability of ML Models

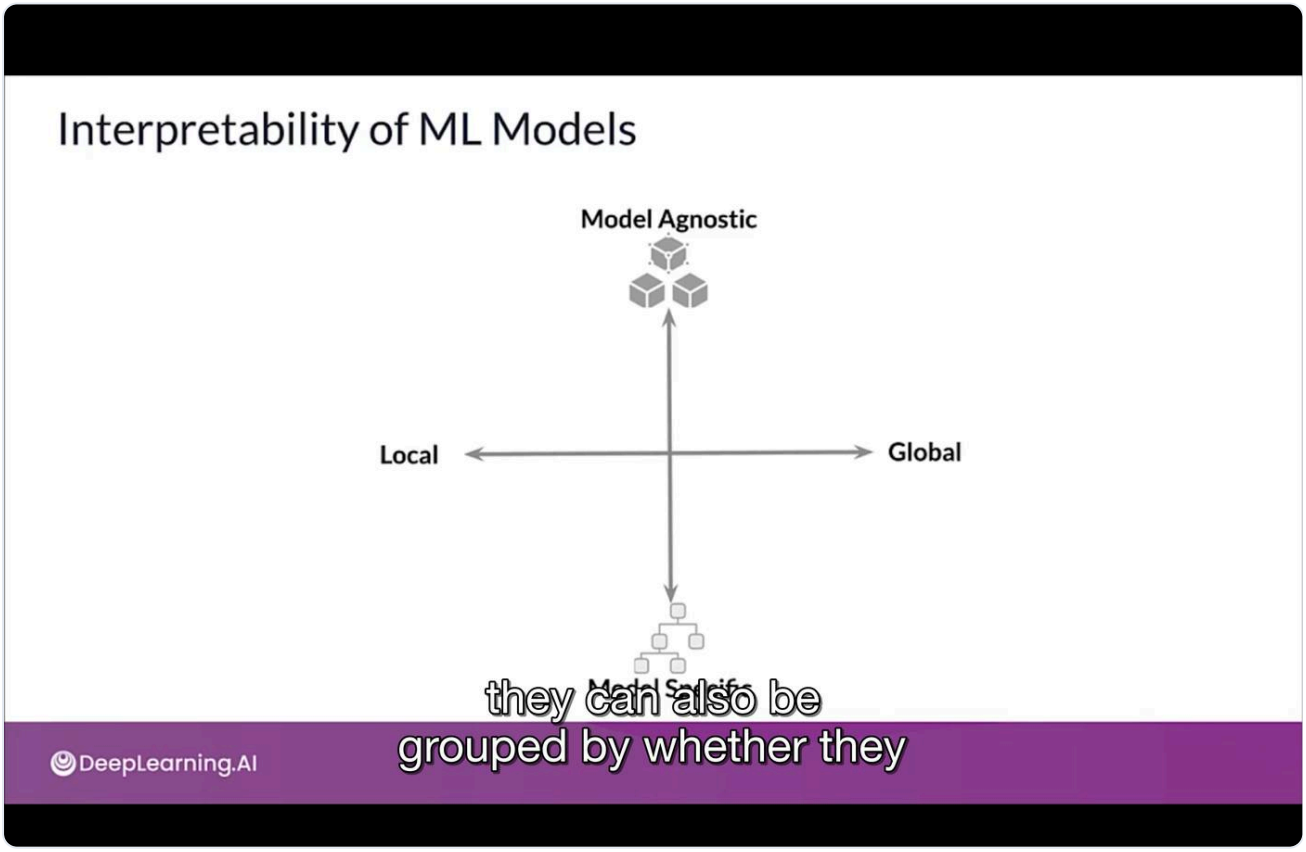

FIGURE 42.11 | Interpretability of ML Models

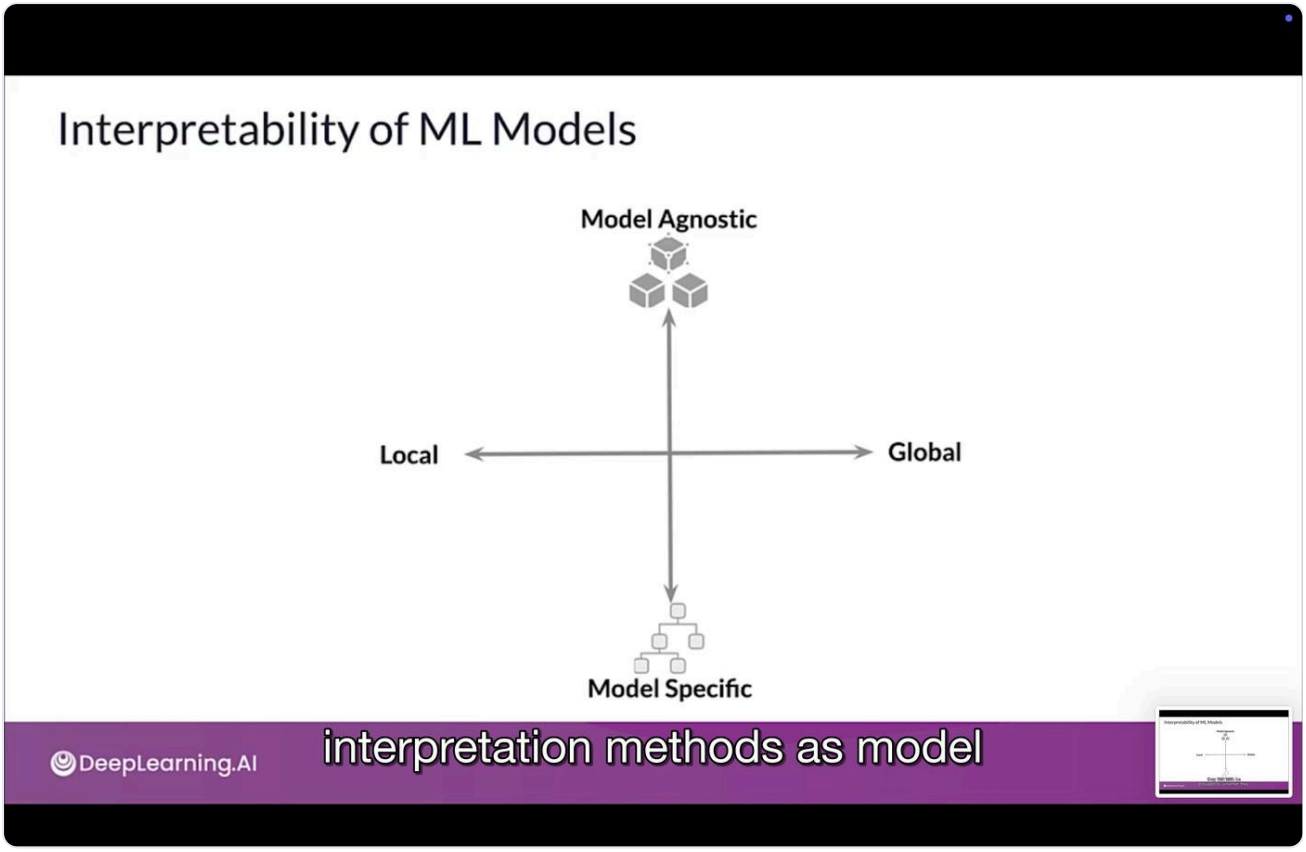

FIGURE 42.12 | Interpretability of ML Models

In addition to grouping interpretation methods as model agnostic or model specific, they can also be grouped by whether they generate interpretations which are local or global. Interpretability methods can be local or global based on whether the method explains an individual prediction or it explains the entire model behavior. Sometimes the scope can be in between local and global. A local interpretability method explains an individual prediction. For example, feature attribution in the prediction of a single example in the dataset. Feature attributions measure how much each feature contributed to the predictions for a given result.

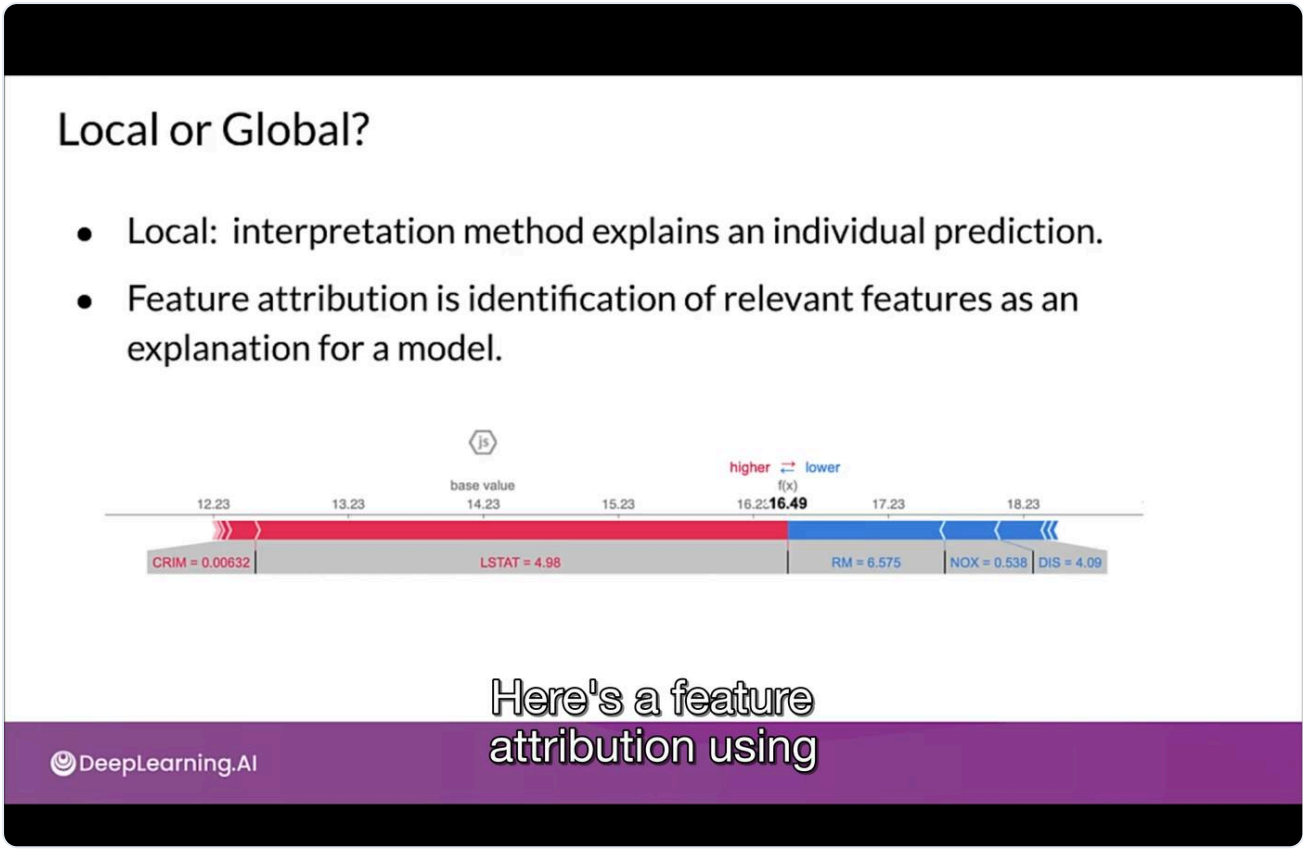

FIGURE 42.13 | Local or Global?

Here's a feature attribution using a library called SHAP for the prediction of a single example by a gradient boosted ensemble tree trained on the Boston house price dataset. This example is used to demonstrate what a local explanation looks like. The diagram shows the contribution of features in pushing model output from the base value towards the actual model output. Features in red push the model towards a higher value and features in blue try to push the model output towards a lower value. Interpretability methods can also be global. Global methods explain the entire model behavior. For example, if the method creates a summary of feature attributions for predictions on the entire test set, then it can be considered global.

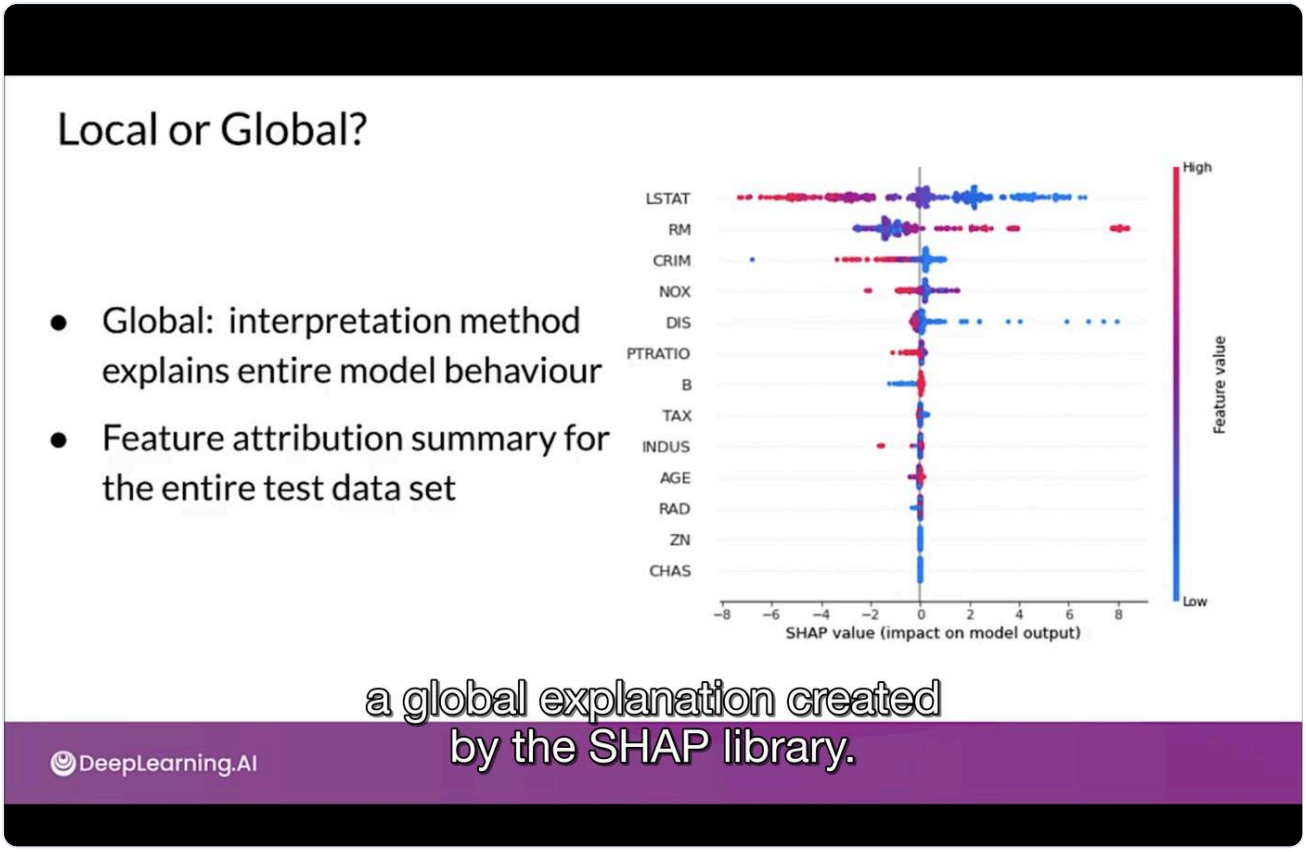

FIGURE 42.14 | Local or Global?

Here's an example of a global explanation created by the SHAP library. It shows feature attributions, the SHAP value of every feature for every sample for predictions in the Boston house prices dataset. The color represents the feature value. Red is high and blue is low. You can see how as LSTAT, the percent of lower status of the population increases, it tends to lead to a decrease in the predicted house price. Since this explanation shows an overview of attributions of all features on all instances in the dataset, it would be considered global.

LECTURE 43

More Advanced Models: TensorFlow Lattice

Since the early days of statistical analysis and Machine Learning, there had been model architectures which are intrinsically interpretable. Let's look at those now along with some more recent advances and learn about how they can improve interpretability. What exactly do we mean by an intrinsically interpretable model? Well, one definition is that the workings of the model are transparent enough and intuitive enough that they make it relatively easy to understand how the model produced a particular result by examining the model itself. Many classic models are highly interpretable, such as tree-based models and linear models. But although we've seen neural networks that are able to produce really amazing results, one of the issues with them is that they tend to be very opaque.

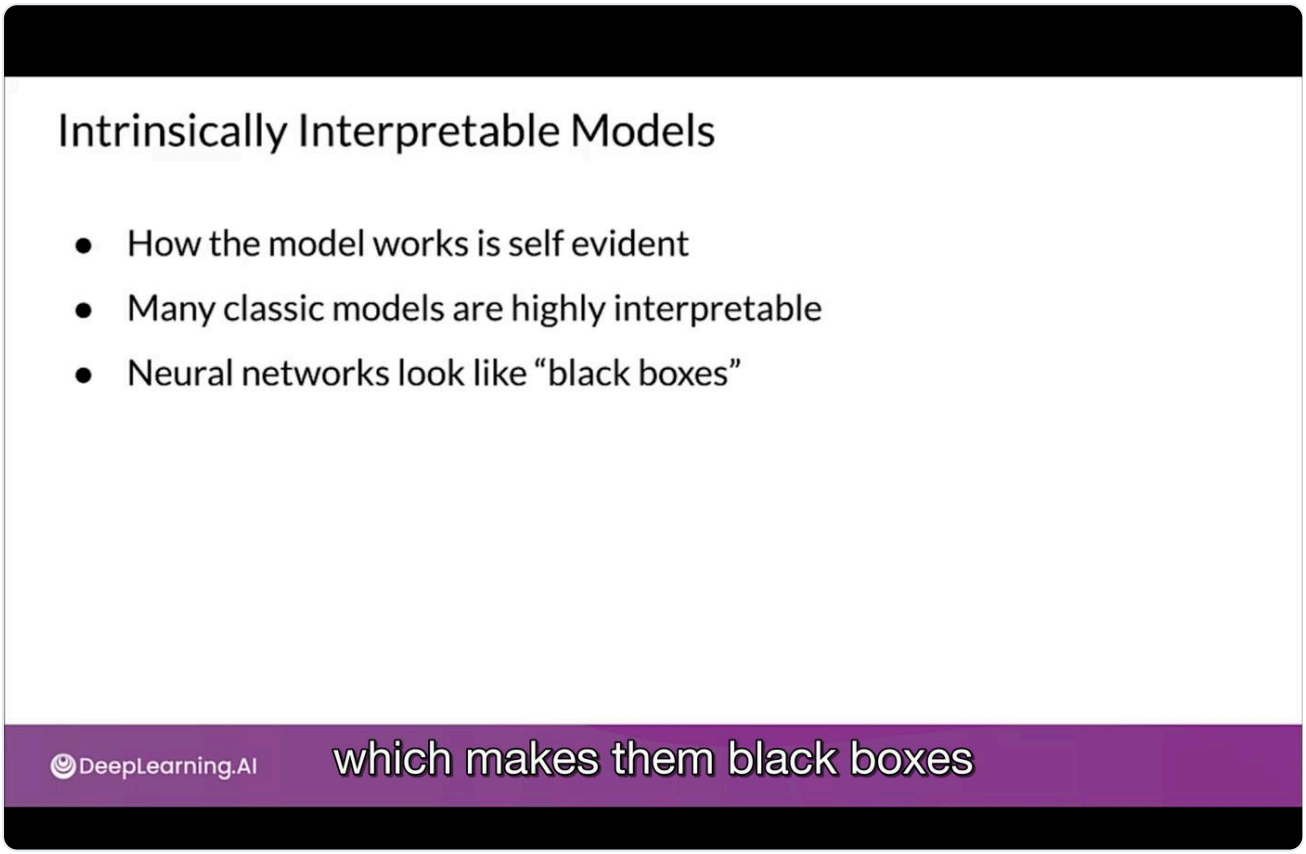

FIGURE 43.1 | Intrinsically Interpretable Models

Especially the larger and more complex architectures, which makes them black boxes when we're trying to interpret them. That limits our ability to interpret their results and requires us to use post-hoc analysis tools to try to understand how they reached a particular result.

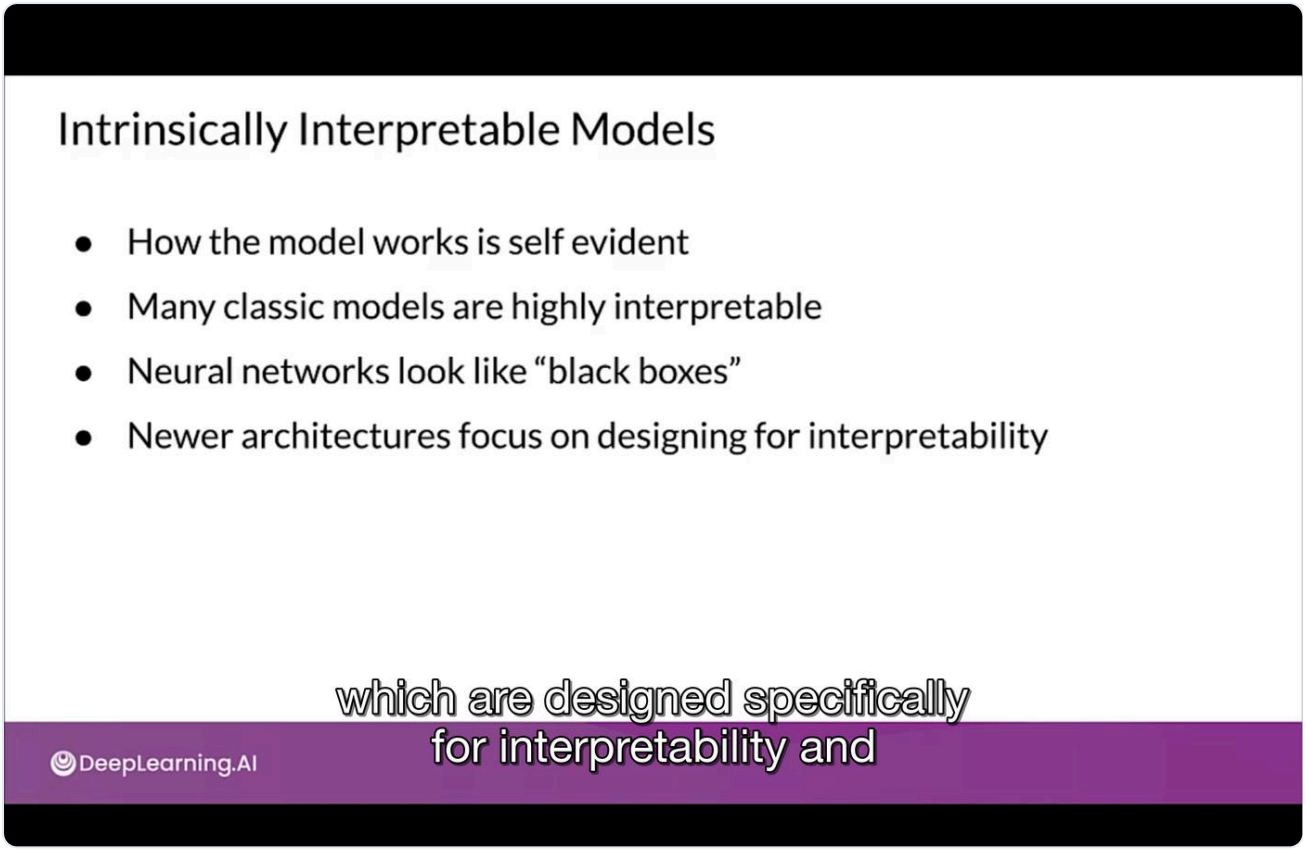

FIGURE 43.2 | Intrinsically Interpretable Models

However, newer architectures have been created, which are designed specifically for interpretability and yet retain the power of deep neural networks. This continues to be an active field of research. One key characteristic which helps improve interpretability is when features are monotonic. Monotonic means that the contribution of the feature towards them all result, either consistently increases or decreases or stays even as the feature value changes.

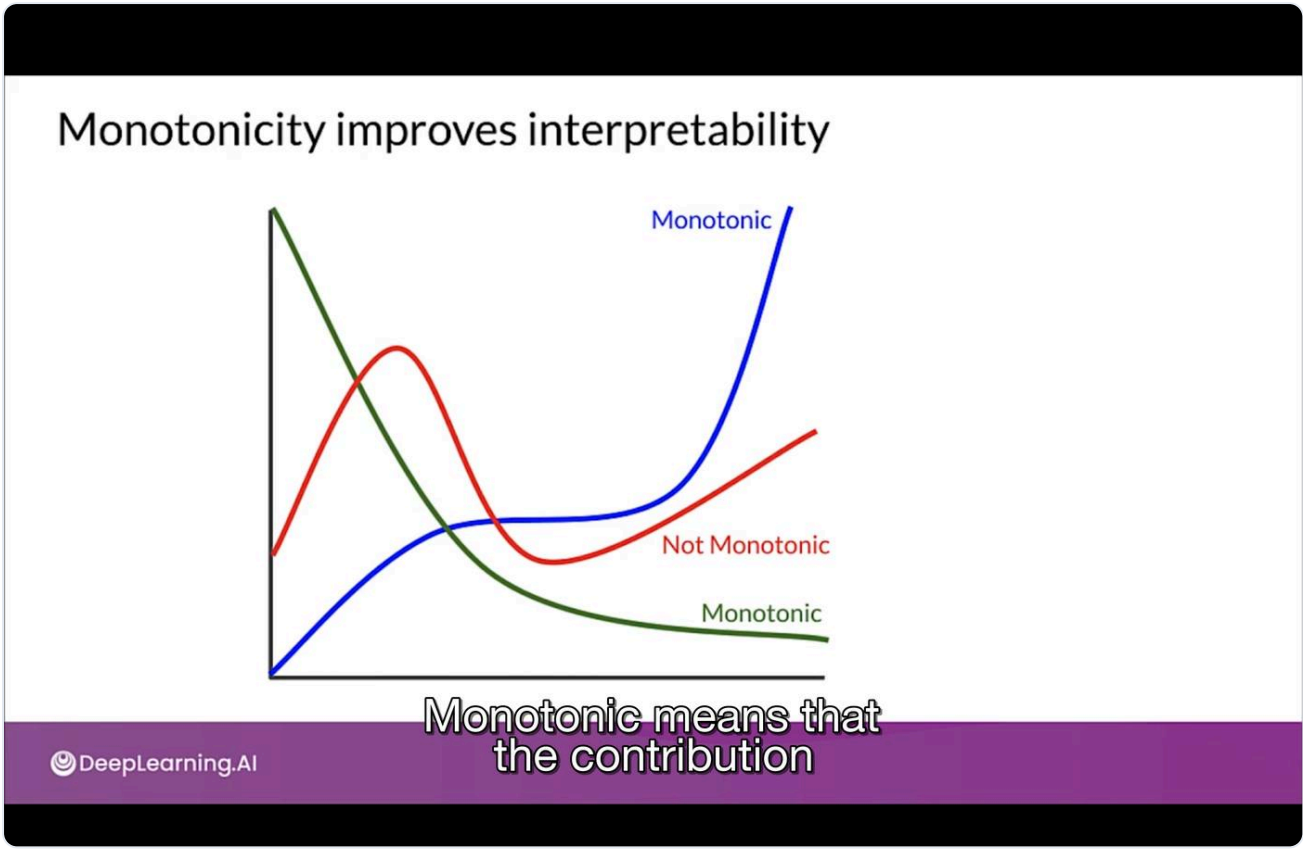

FIGURE 43.3 | Not Monotonic

The key factor here is the consistency. This matches the domain knowledge for many features in many problems. When you're trying to understand the model results, if features are monotonic, it matches your intuition about the reality of the world that you're trying to model. For example, if you're trying to create a model to predict the value of a used car, when all other features or how constant, the more miles on the car, the less the value should be.

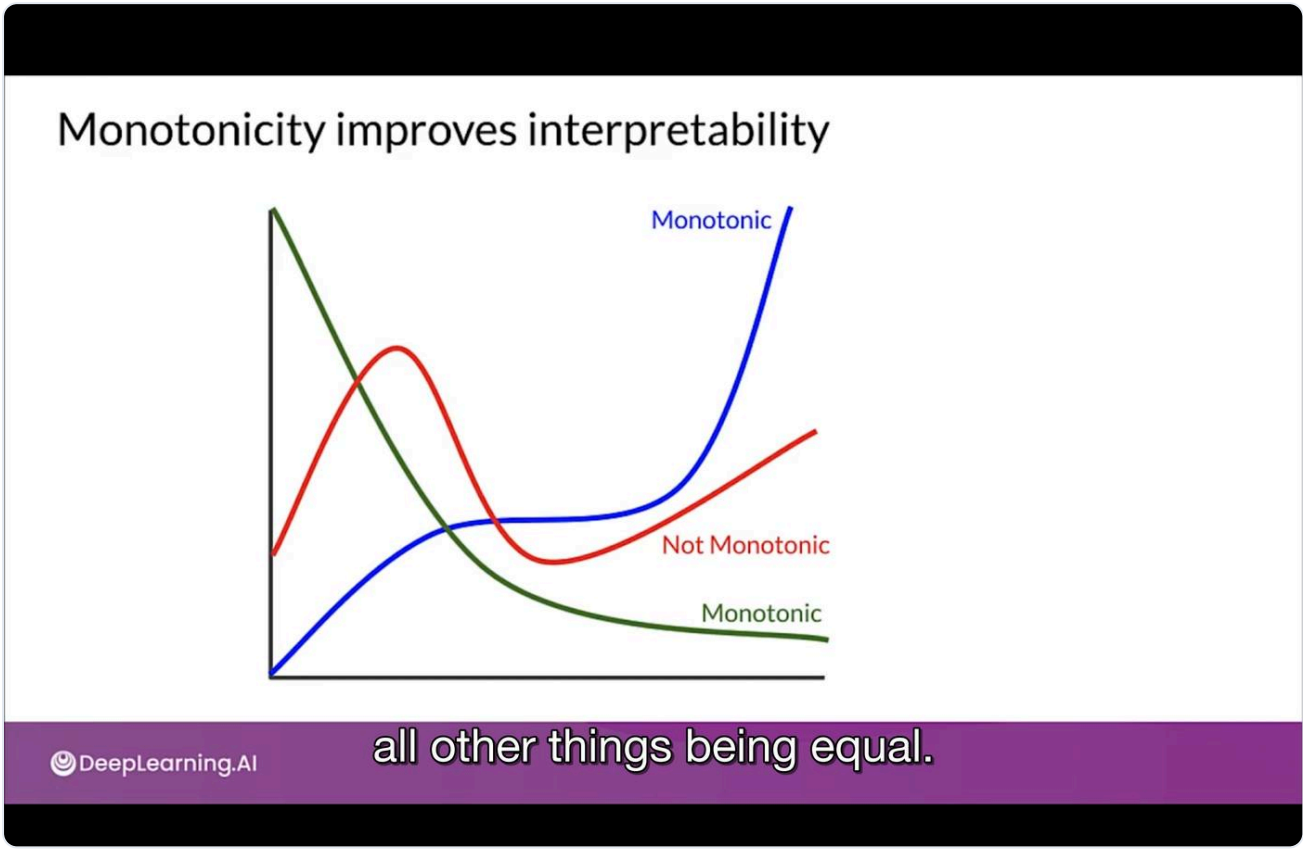

FIGURE 43.4 | Not Monotonic

But you don't expect a car with more miles to be worth more than it was with less miles, all other things being equal. This matches your knowledge of the world and, so your model should match it too and the mileage feature should be monotonic. In this graph, the blue and green curves are monotonic, while the red curve is not because it does not consistently increase or decrease or remain the same. Let's look at a few architectures which are considered interpretable. First, linear models are very interpretable because linear relationships are easy to understand and interpret and the features of linear models are always monotonic. Some other model architectures have linear aspects to them.

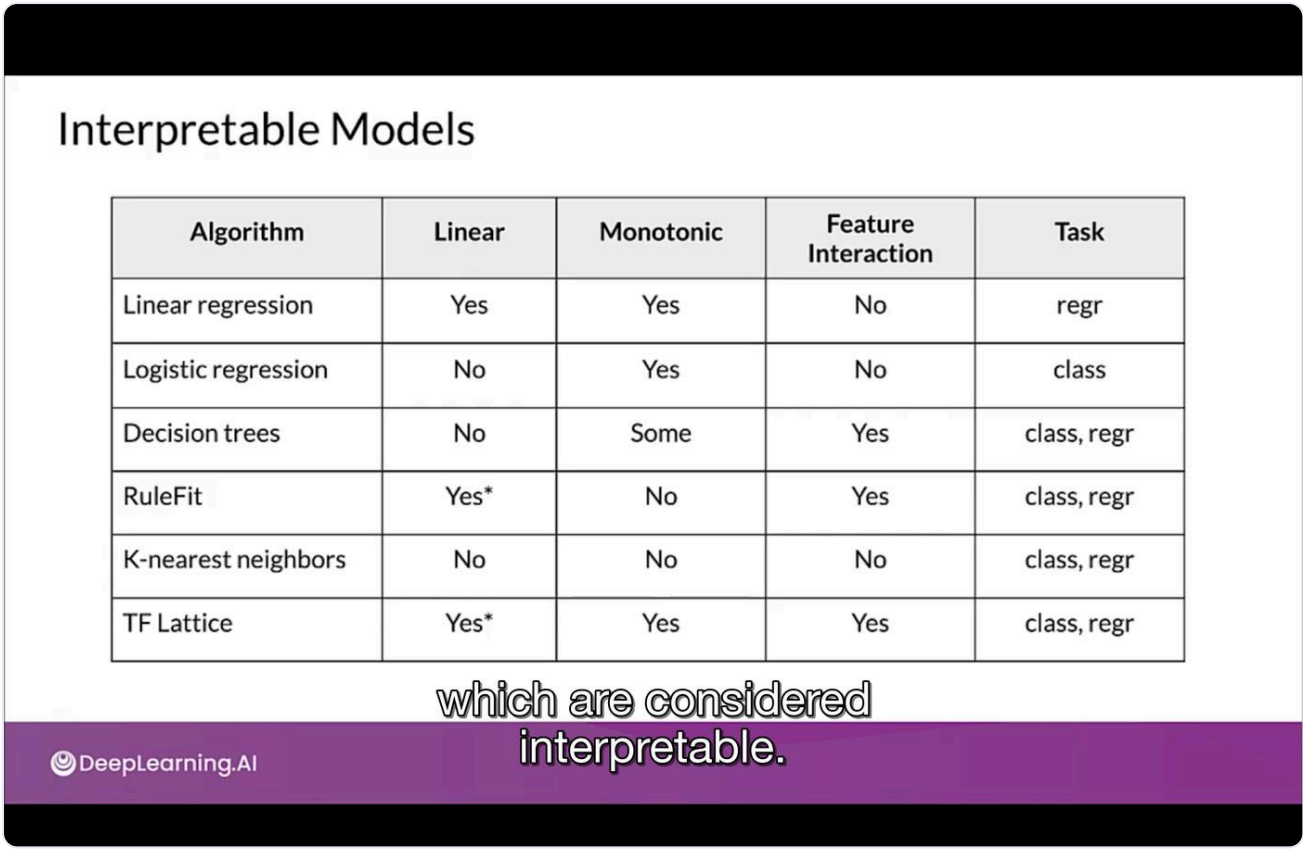

FIGURE 43.5 | Interpretable Models

For example, when used for regression, RuleFit Models are linear. In all cases, TF Lattice Models use linear interpolation between lattice points, which we'll learn about soon. Some models can automatically include feature interactions or include constraints on feature interactions. In theory, you can include feature interaction in all models through feature engineering. Interactions which matter domain knowledge tend to make models more interpretable.

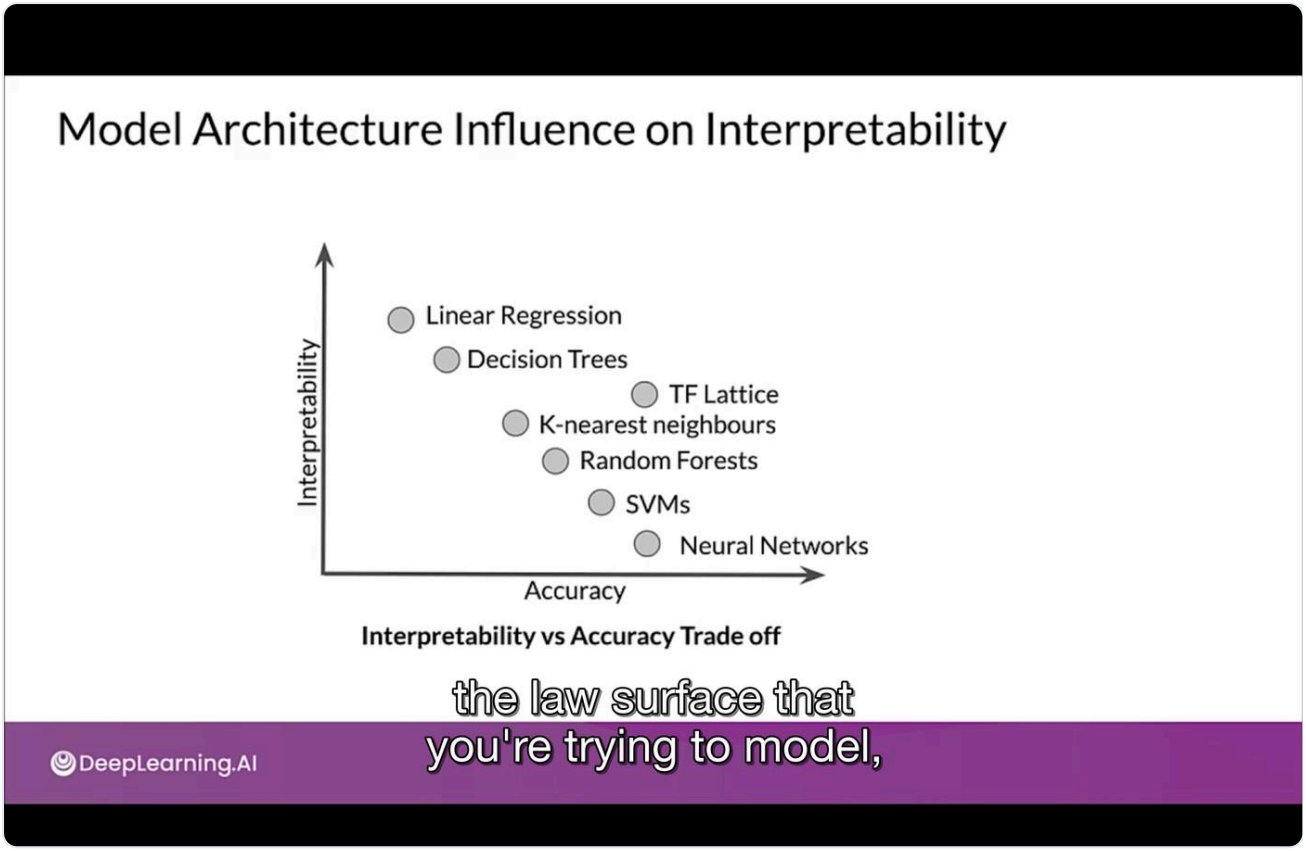

FIGURE 43.6 | Linear Regression

Depending on the characteristics of the law surface that you're trying to model, the more complex model architectures can achieve high-accuracy. This often comes at a price in terms of interpretability. For many of the reasons discussed earlier, interpretability can be a strict requirement of models, and so you need to find a balance between models that you can interpret, and models that generate the accuracy that you need. Again, some newer architectures have been created which deliver far greater accuracy but also deliver good interpretability. Tensorflow lattice is one example of these kinds of architectures. Probably the ultimate in interpretability is our old friend, linear regression.

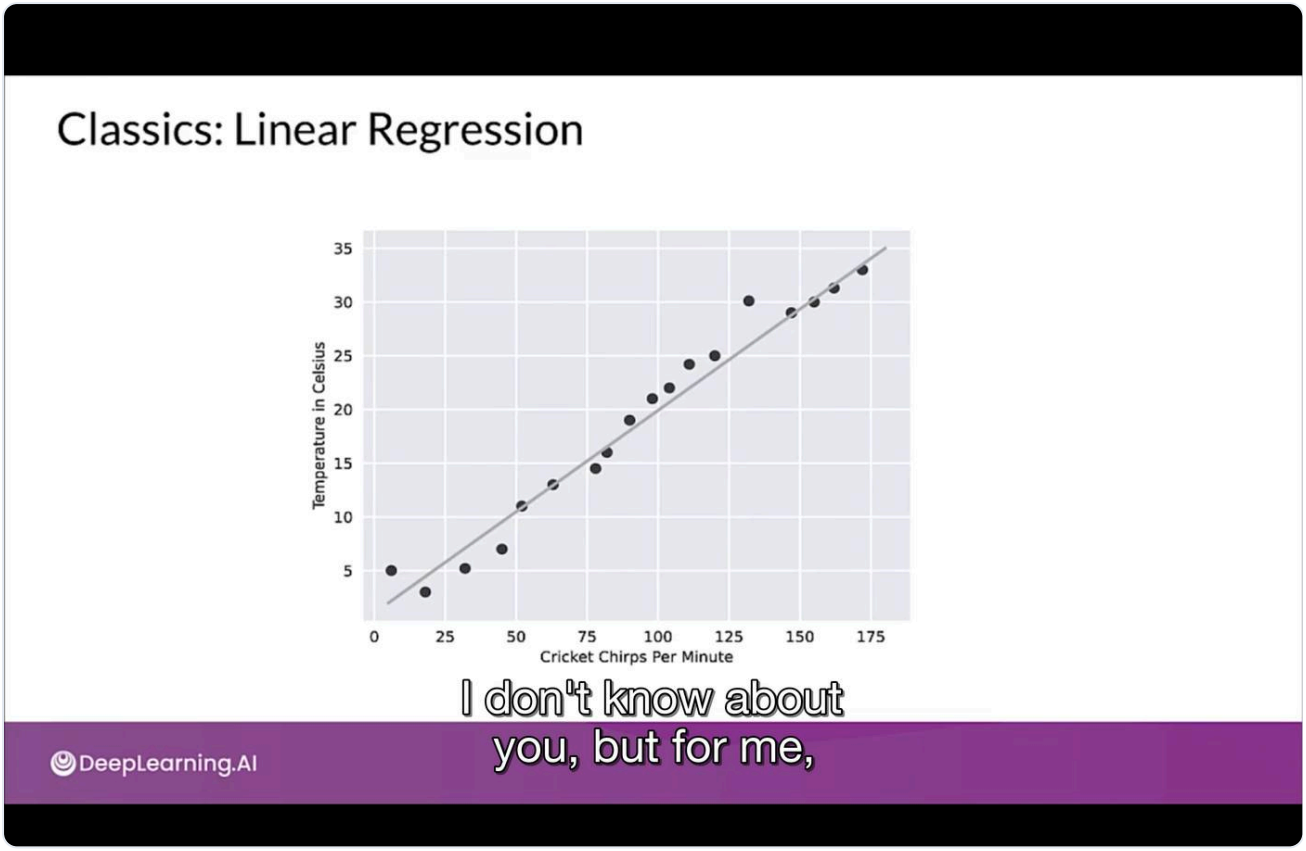

FIGURE 43.7

I don't know about you, but for me, this was the first type of model that I learned about. Is very easy to understand the relationship between feature contributions, even for a multivariate linear regression. As feature values increase or decrease, their contribution to the model results also increases or decreases. This example shown here, models the number of chirps per minute that a cricket will make based on the temperature of the air. This is a very simple linear relationship and so linear regression models at well. By the way, this also means that when you're out at night, if you listen carefully to the crickets and count how many chirps they make, you can measure the temperature of the air.

Check out dole beer's law to learn more. Of course, the actual contribution of a feature to the results of the model will depend on its weight. This is especially easy to see for linear models. For numerical features an increase or decrease of one unit in a feature increases or decreases the prediction based on the value of the corresponding weight. For binary features, prediction is increased or decreased by the value of the weight.

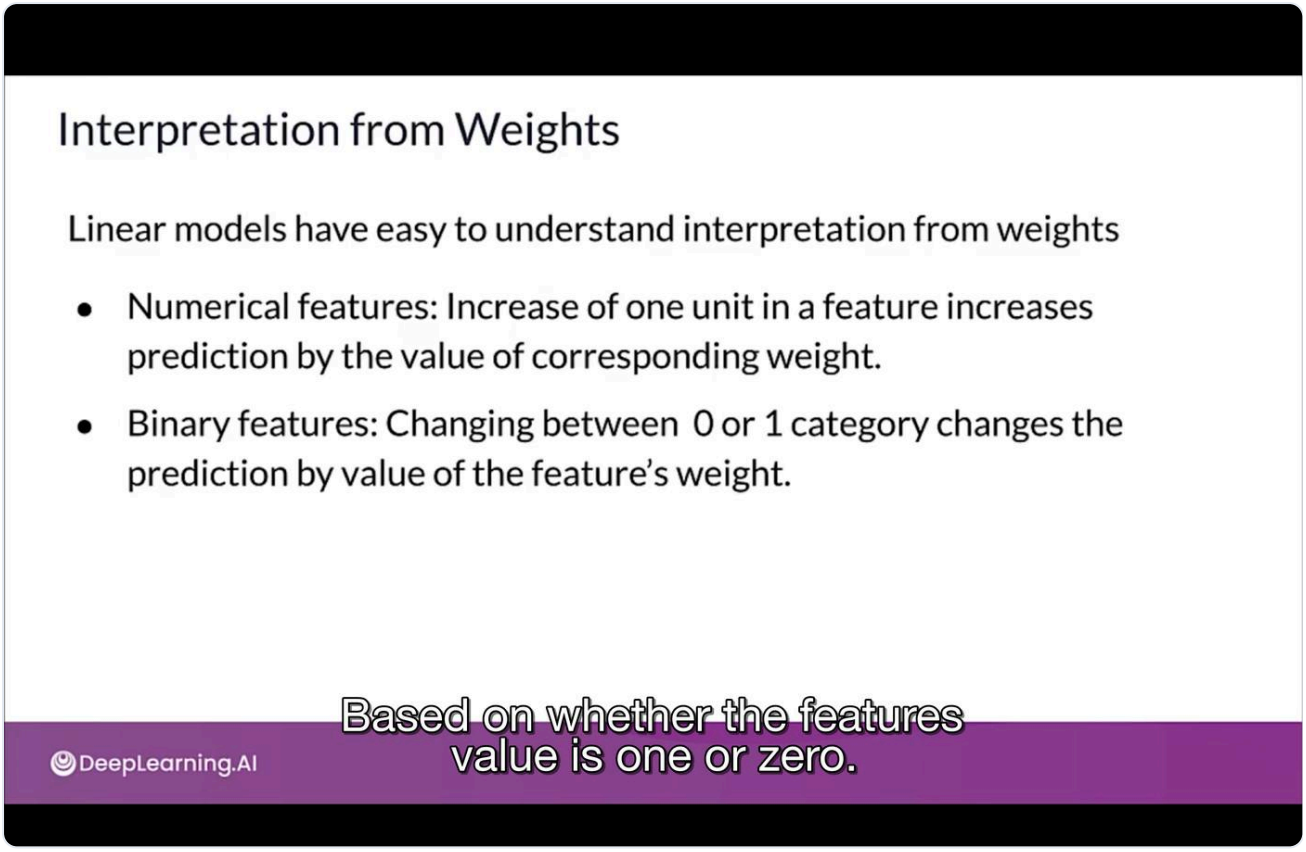

FIGURE 43.8 | Linear Models Have Easy to Understand Interpretation from Weights

Based on whether the features value is one or zero.

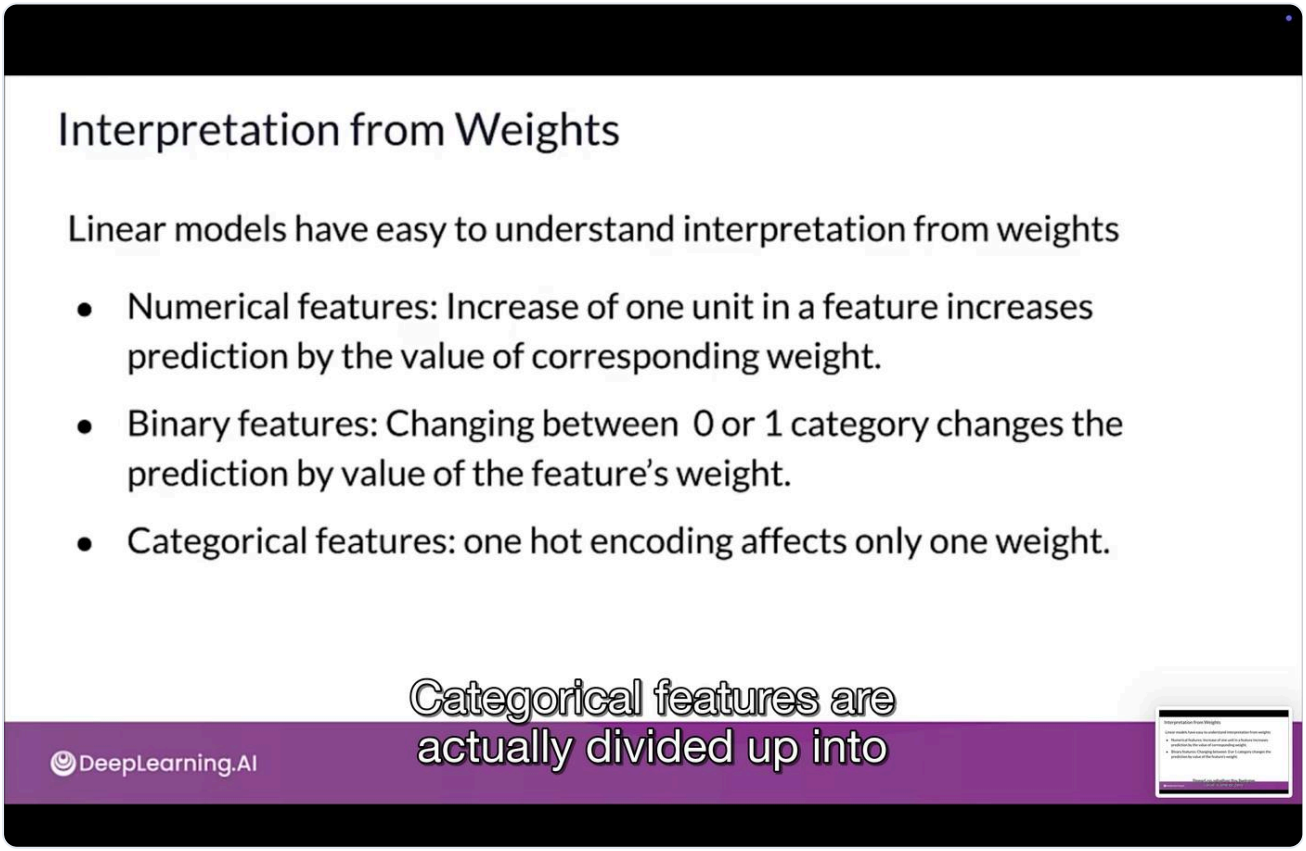

FIGURE 43.9 | Linear Models Have Easy to Understand Interpretation from Weights

Categorical features are actually divided up into several individual features were one-hot encoding, each of which has a weight. With one-hot encoding, only one of the categories will be set. Only one of the weights will be included in the model result.

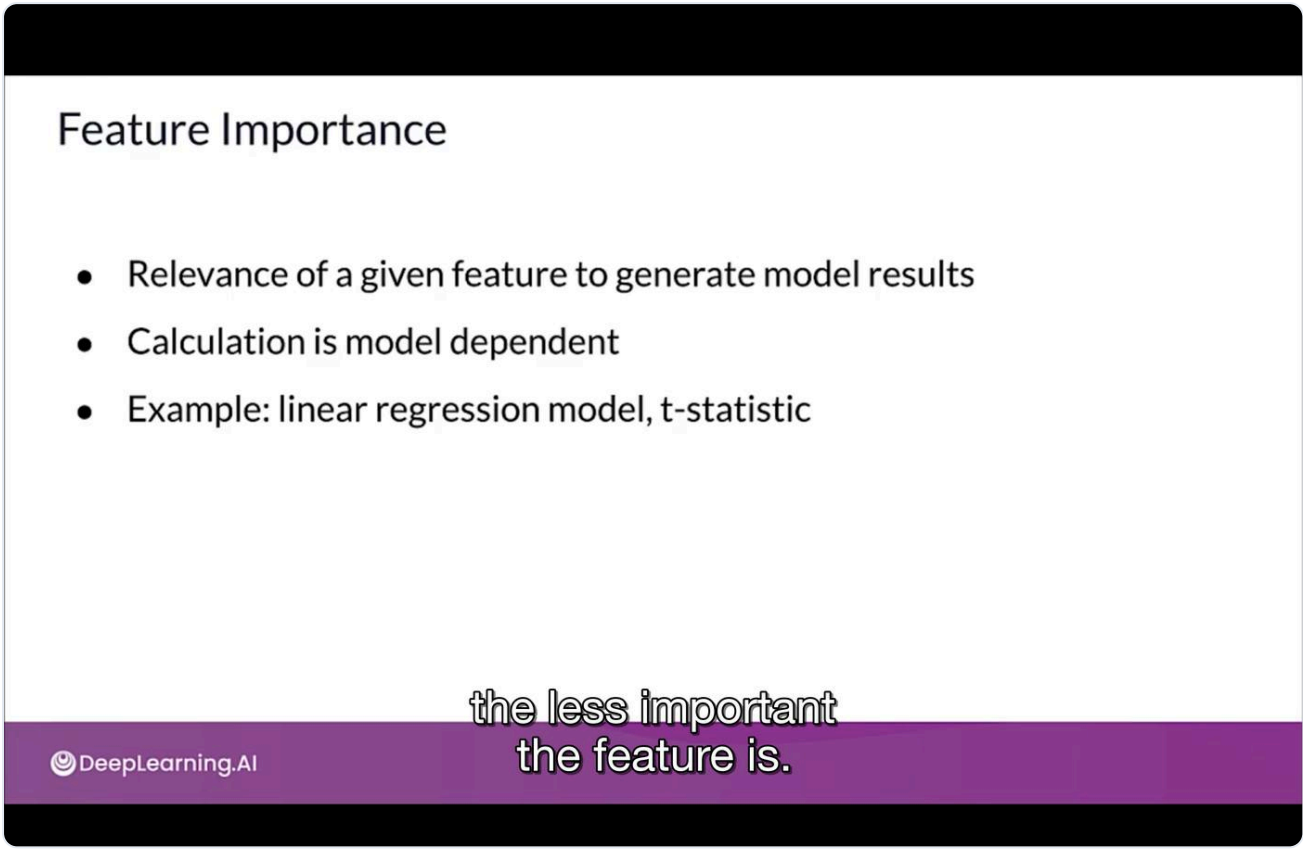

FIGURE 43.10 | Feature Importance

How can we determine the relevance of a given feature for making predictions? Feature importance tells us how important the feature is for generating a model result. The more important feature is, the more we want to include it in our feature vector. But feature importance for different models is calculated differently because different models calculate results differently. For linear regression models, the absolute value of a features t-statistic is a good measure of that feature's importance. The t-statistic is the learned or estimated weight of the feature scaled by its standard error. The importance of a feature increases as its weight increases.

But the more variants the weight has; in other words, the less certain we are about the correct value of the weight, the less important the feature is. You may not be familiar with lattice models. Let's take a look at how they work. Lattice models overlaid grid onto a feature space and set the values of the function that it's trying to learn at each of the vertices of the grid.

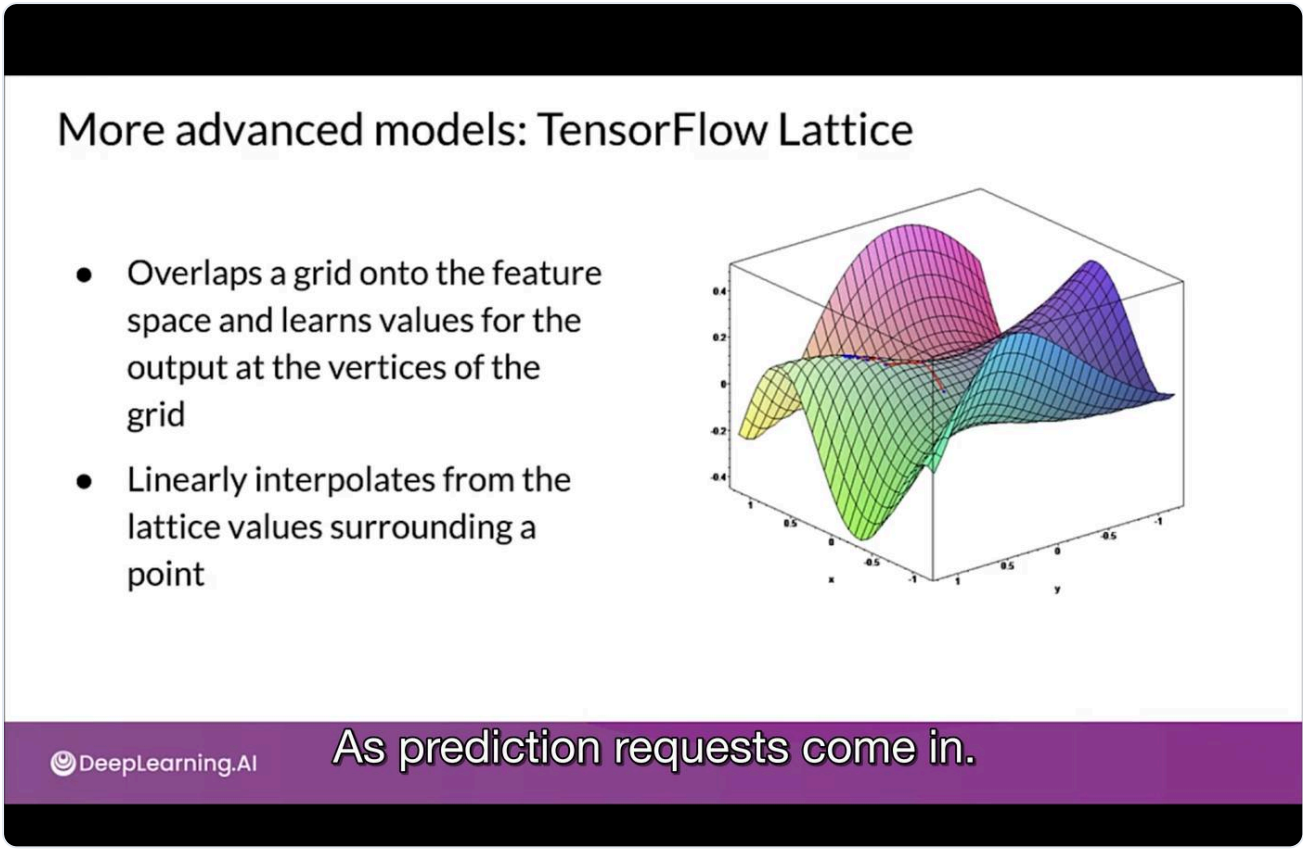

FIGURE 43.11 | More Advanced Models: TensorFlow Lattice

As prediction requests come in. If they don't fall directly on a vertex, then the result is interpolated using linear interpolation from the nearest vertices of the grid. One of the benefits is to regularize the model and greatly reduced sensitivity. Even two examples that are outside the coverage of the training data, by imposing a regular grid on under the feature space. However, TensorFlow lattice models go beyond simple lattice models.

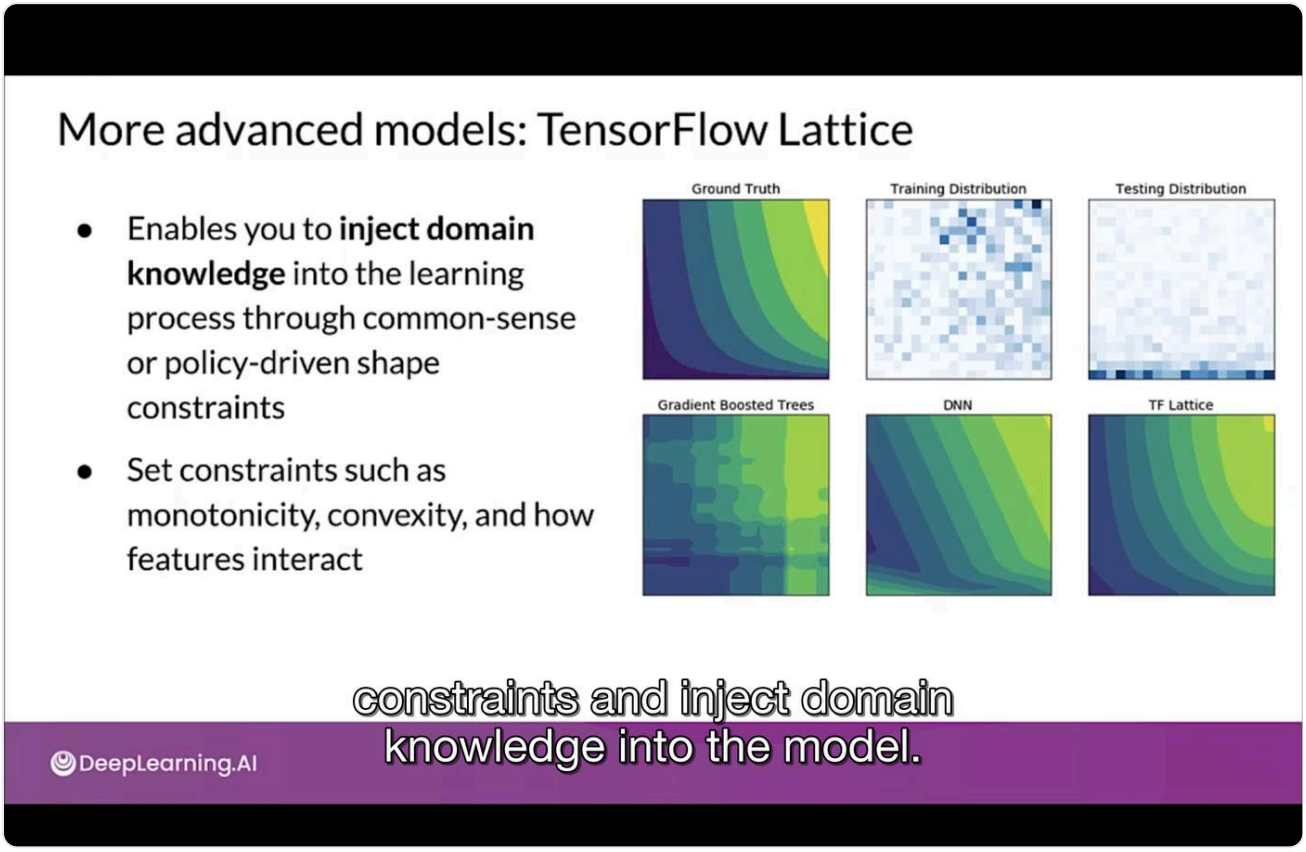

FIGURE 43.12 | More Advanced Models: TensorFlow Lattice

Tensor-flow lattice further allows you to add constraints and inject domain knowledge into the model. These graphs show the benefit of regularization and domain knowledge. Compare the one on the top left to the one on the bottom right, and notice how close the model is to the ground truth compared to other kinds of models. When you know that certain features in your domain are monotonic, we're convex, or that one or more features interact. You can inject that knowledge into the model as it learns. For interpretability, this means that feature values and results are likely to match your domain knowledge, for what you expect your results to look like.

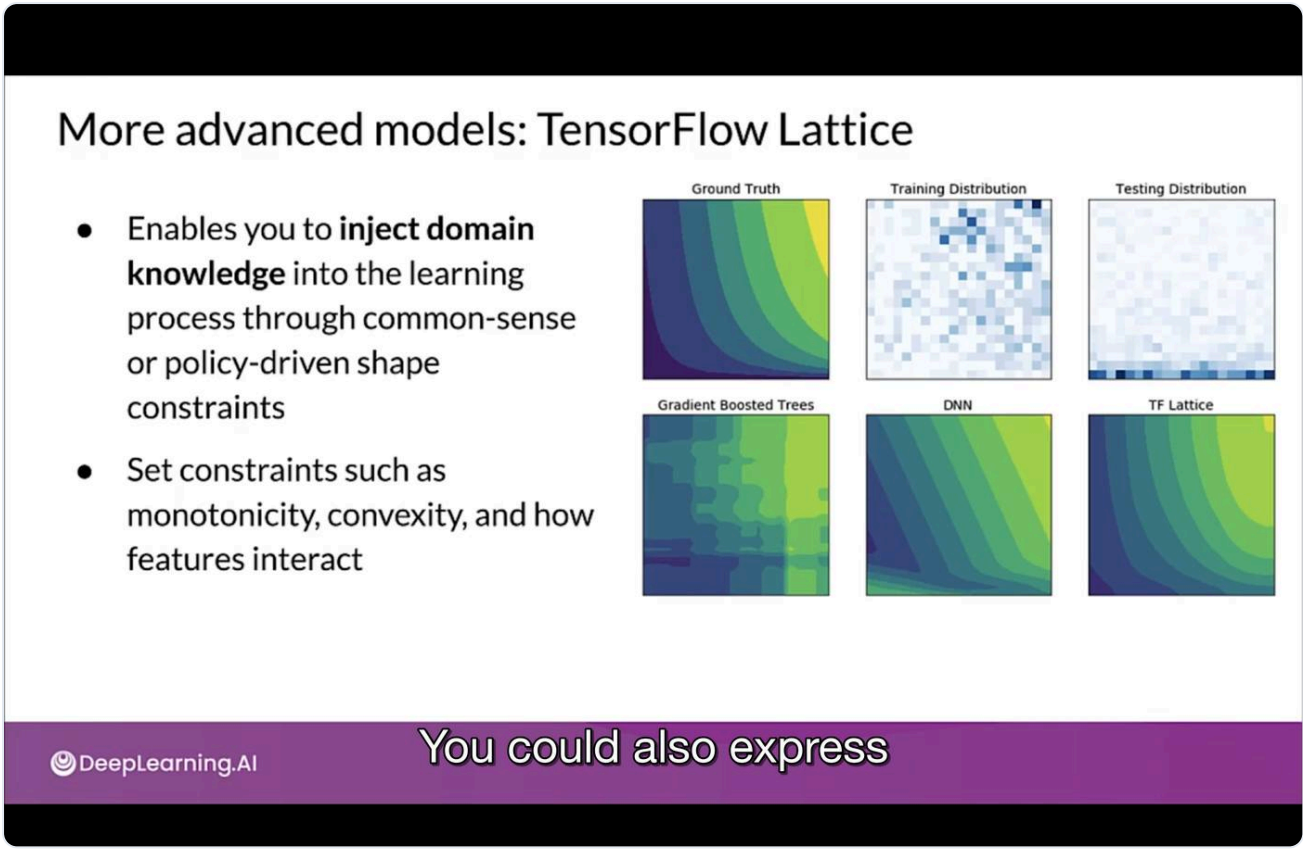

FIGURE 43.13 | More Advanced Models: TensorFlow Lattice

You could also express relationships or interactions between features to suggest that one feature reflects trust in another feature. For example, a higher number of reviews makes you more confident in the average star rating of a restaurant. You might have considered that yourself when shopping online. All of these constraints based on your domain knowledge or what you know about the world that you're trying to model. Help the model produce results that make sense, which helps make them interpretable. Also, since the model uses linear interpolation between vertices, it also has many of the benefits of linear models in terms of interpretability.

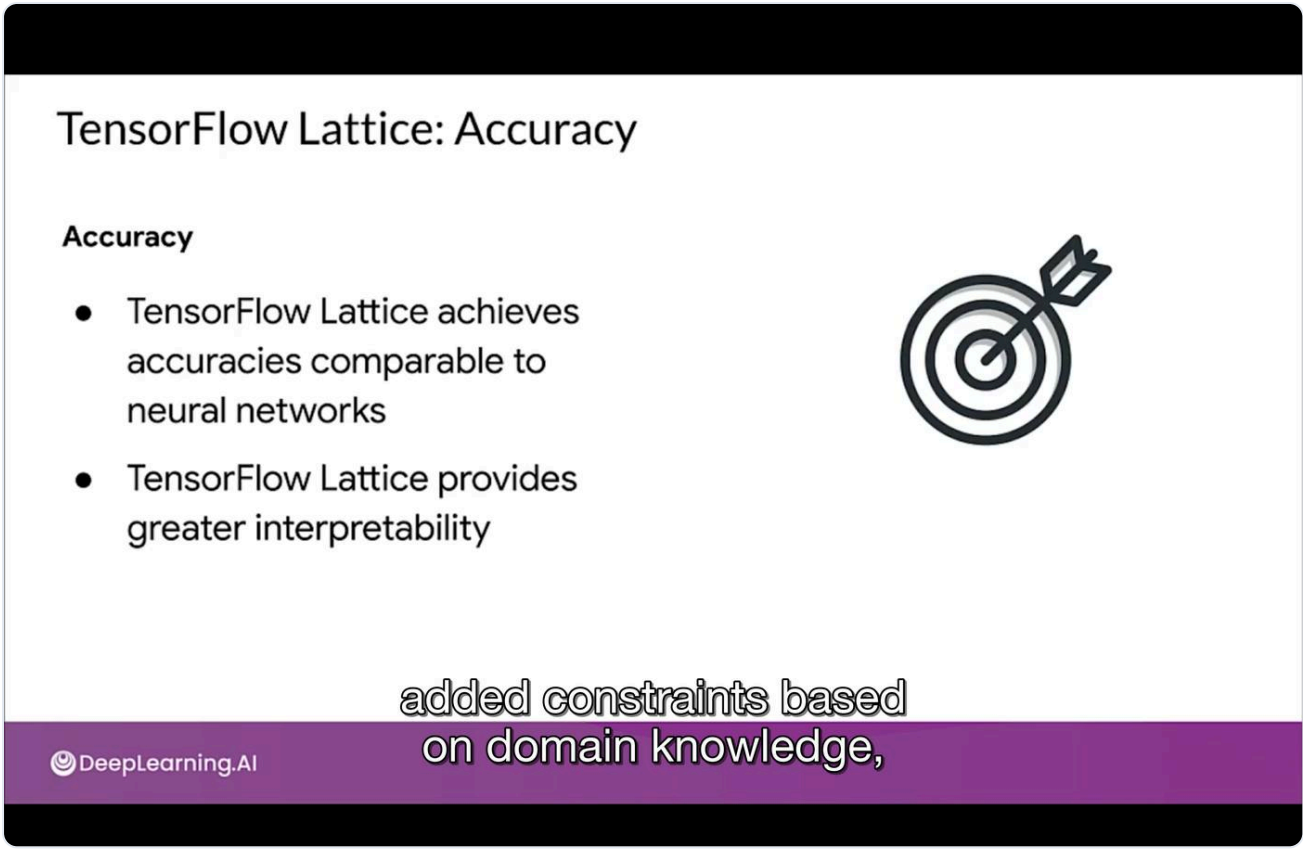

FIGURE 43.14 | TensorFlow Lattice: Accuracy

But along with all the benefits of added constraints based on domain knowledge, Tensorflow lattice models also have a level of accuracy on complex problems that is similar to deep neural networks. Of course, Tensor-Flow lattice models are also easier to interpret than neural networks. However, lattice models do have a weakness. Dimensionality is there kryptonite? The number of parameters of a lattice increases exponentially with the number of input features, which creates problems with scaling for datasets with a large number of features. As a rough rule of thumb, you're probably okay with 20 or less features.

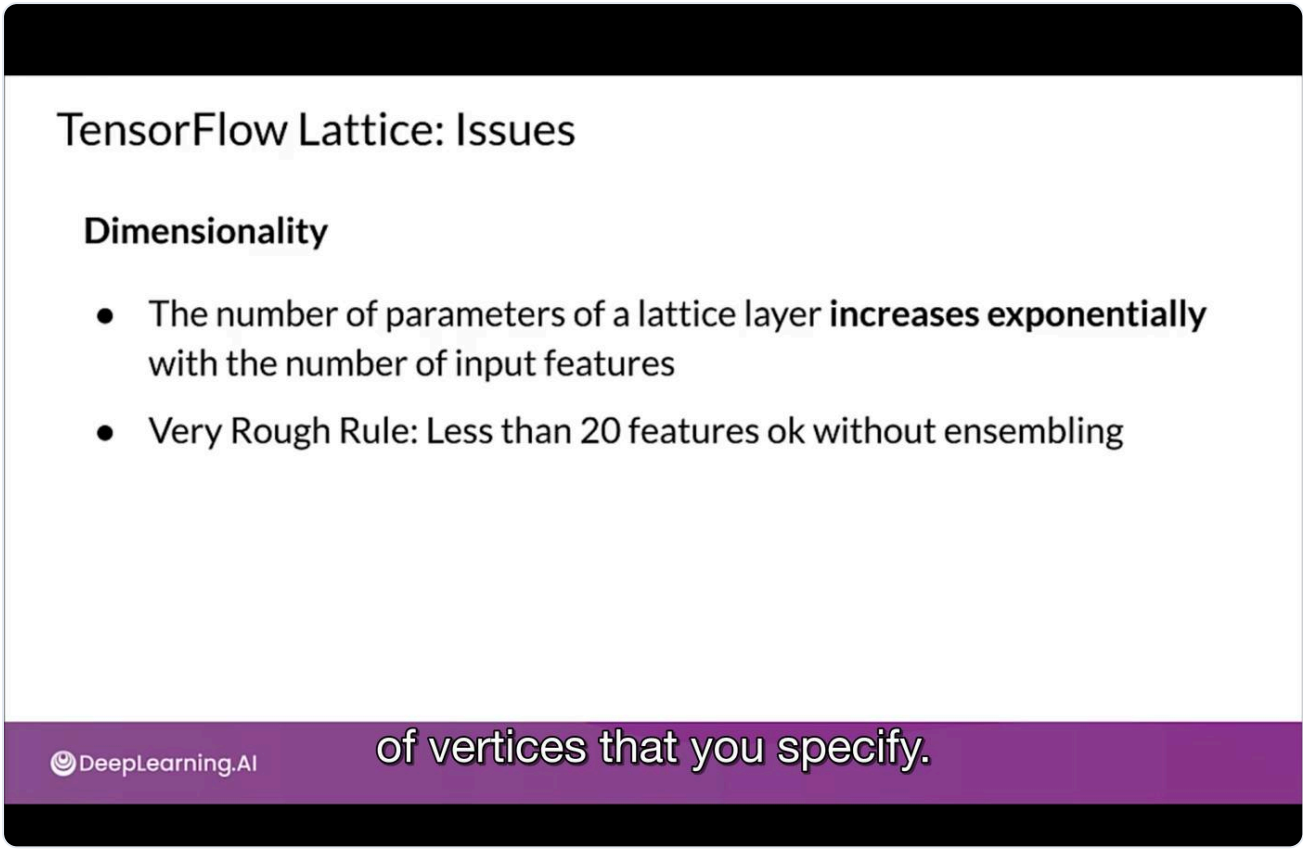

FIGURE 43.15 | TensorFlow Lattice: Issues

But this will also depend on the number of vertices that you specify. There is another way to deal with this dimensionality kryptonite however and that's by using ensembling. That is beyond the scope of this presentation.

LECTURE 44

Model Agnostic Methods

Unfortunately, you can't always work with models that are intrinsically interpretable. For a variety of reasons you may be asked to try to interpret the results of models that are not inherently easy to interpret. Fortunately, there are several methods available that are not specific to particular types of models. In other words, they are model agnostic. Let's look at some of those now. Model agnostic methods separate the explanations from the model. These methods can be applied to any model after it's been trained. For example, they can be applied to linear regression or decision trees or even black box models like neural networks. The desirable characteristics of a model agnostic method include model flexibility and explanation flexibility.

The explanation shouldn't be limited to a certain type. The method should be able to provide an explanation as a formula or in some explanations it can be graphical like perhaps for feature importances.

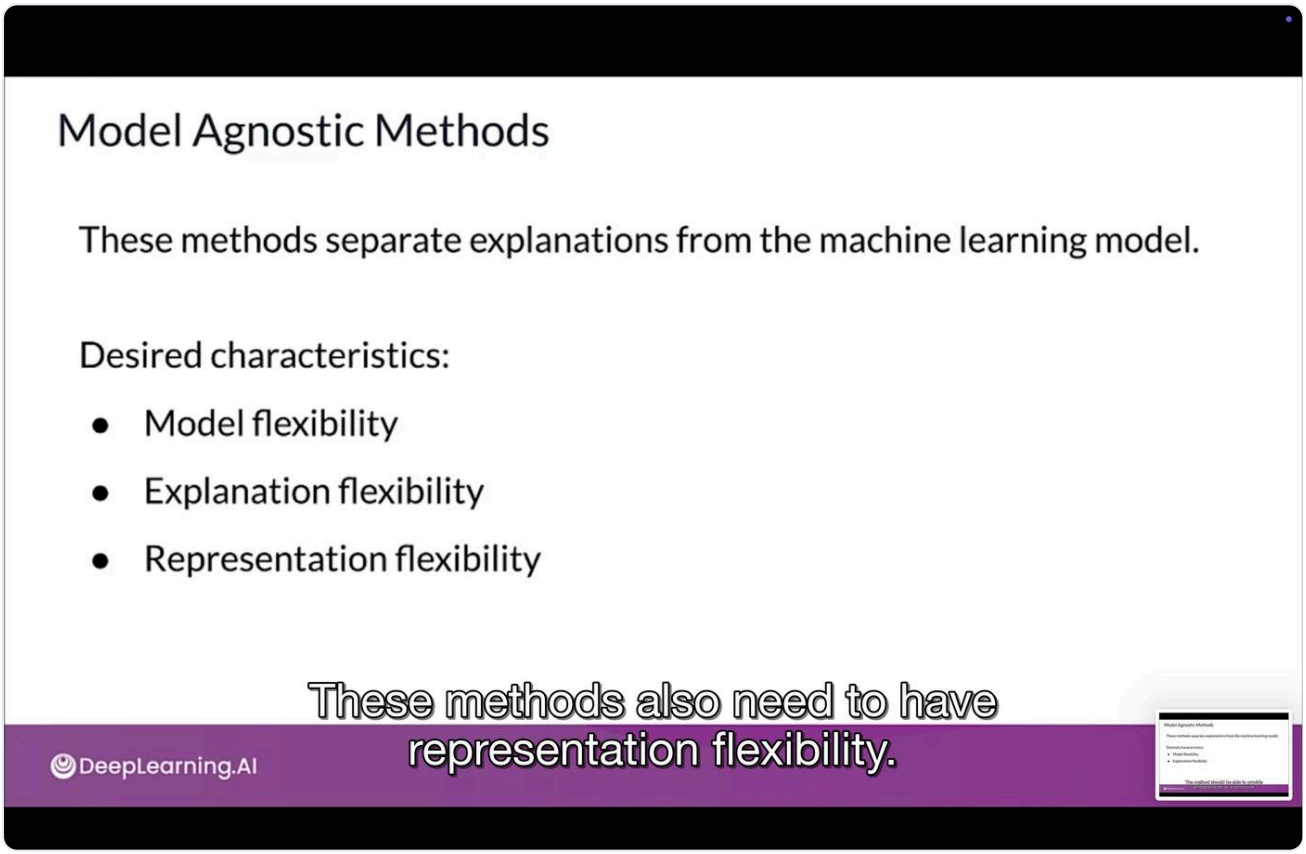

FIGURE 44.1 | Model Agnostic Methods

These methods also need to have representation flexibility. The feature representations used should make sense in the context of the model being explained. Let's take the example of a text classifier that uses word embeddings. It would make sense for the presence of individual words to be used in the explanation in this case.

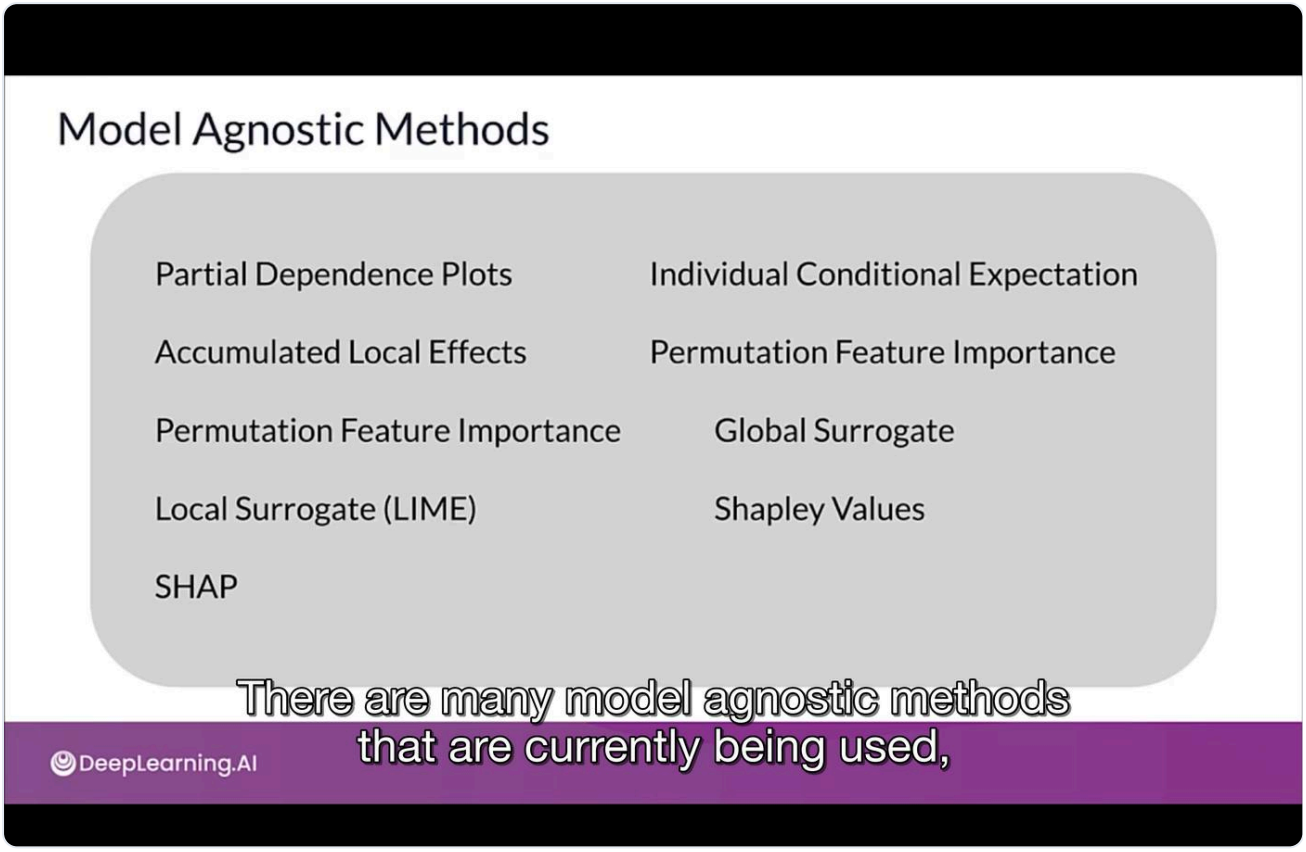

FIGURE 44.2 | Model Agnostic Methods

There are many model agnostic methods that are currently being used, too many to go into all of them in detail here. I'll just discuss a few to give you a flavor for the kinds of things that you can do with model agnostic methods to explain the results generated by your models.

LECTURE 45

Partial Dependence Plots: Examples

One widely used method is partial dependence plots or PDP. Let's start with PDP. Partial dependence plots help you to understand the effects that particular features have on the model results that you're seeing, and the type of relationship between those features, and the targets or labels in your training data. It typically concentrates on the marginal Impact caused by one or two features on the model results. Those relationships could be linear and or monotonic, or they could be of some more complex type. For example, for a linear regression model, a partial dependence plot will always show a linear monotonic relationship.

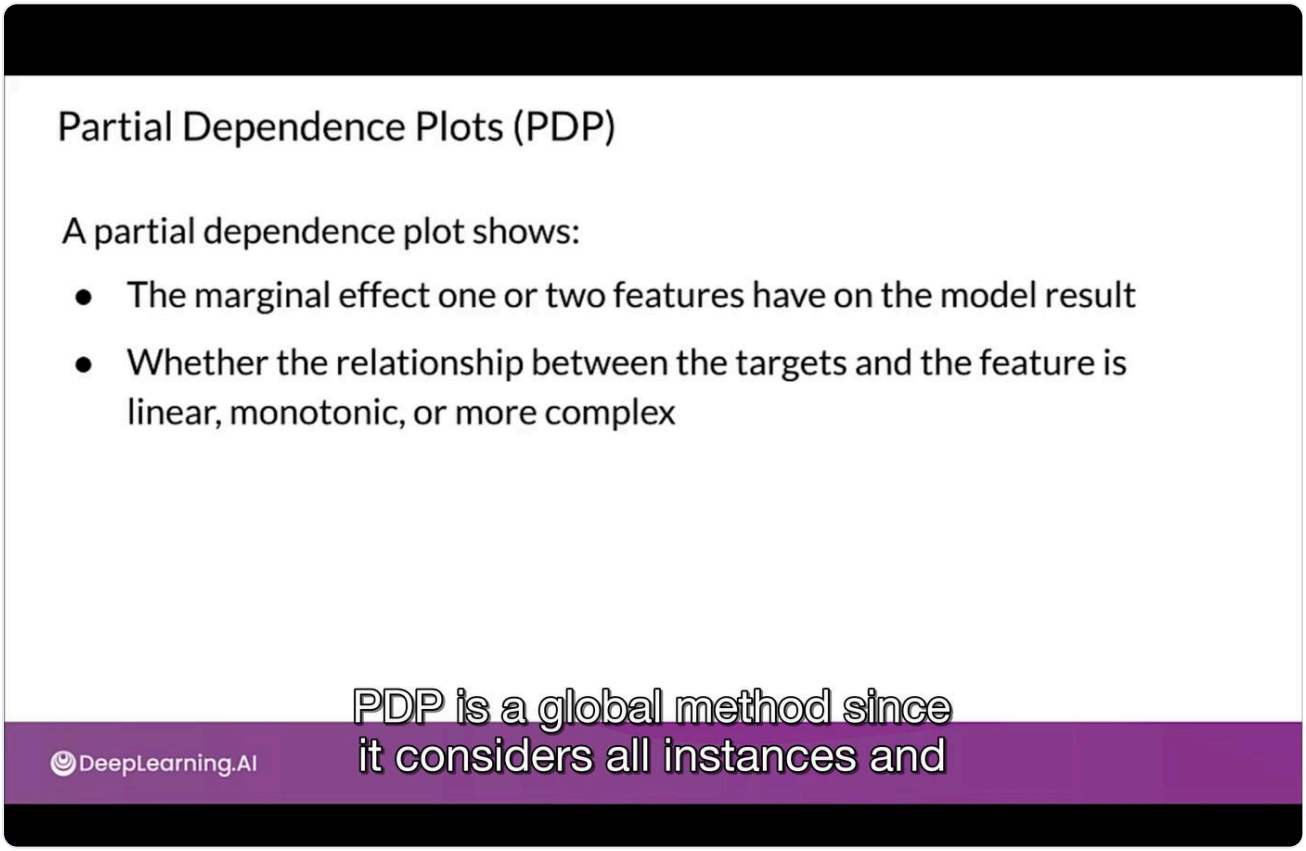

FIGURE 45.1 | Partial Dependence Plots (PDP)

PDP is a global method since it considers all instances and evaluates the global relationship between the features and the results.

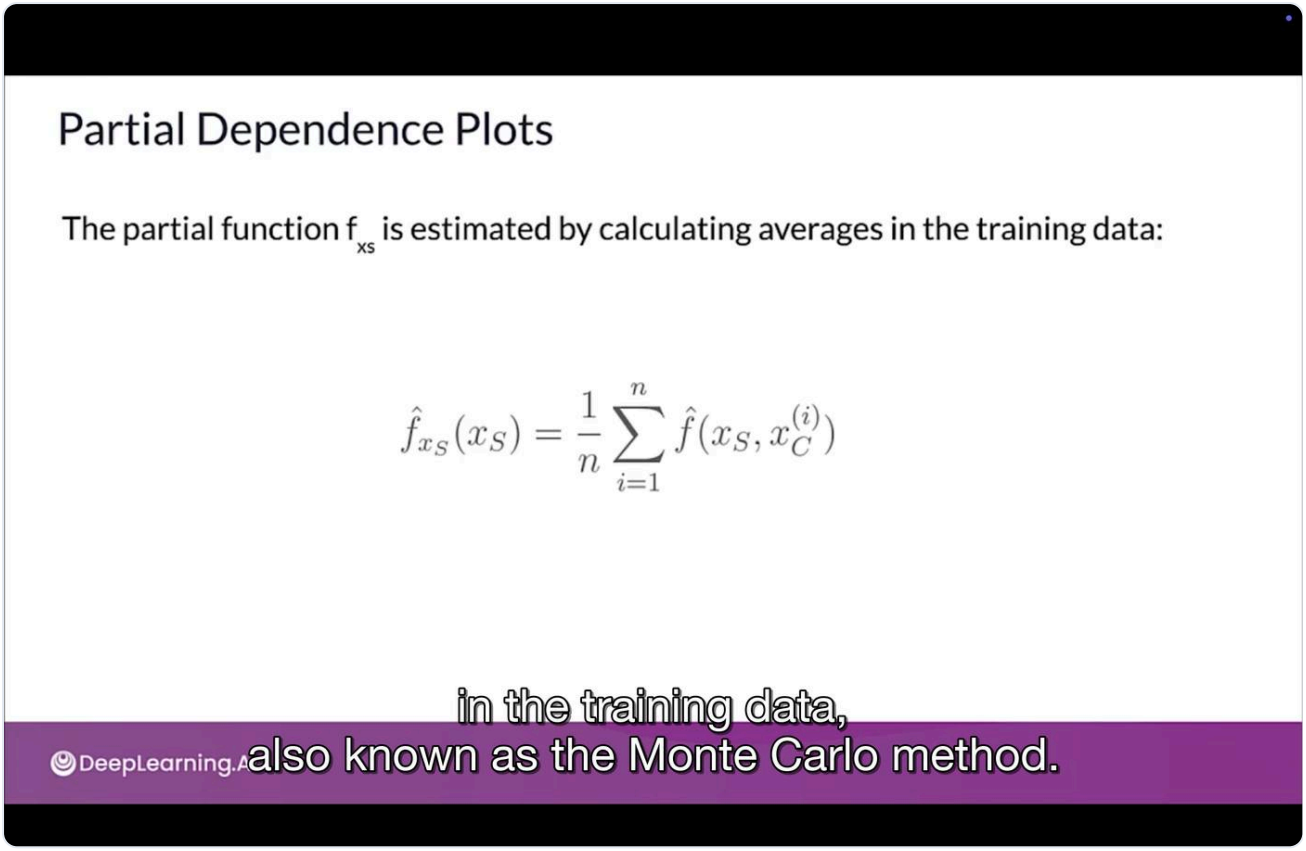

FIGURE 45.2 | Partial Dependence Plots

The partial function f sub xs is estimated by calculating averages in the training data, also known as the Monte Carlo method. This equation shows the estimation of the partial function where n is the number of examples in the training data set, s is the features that we're interested in, and c is all of the other features. The partial function tells us what the average marginal effect on the results is, for given values of the features in s. In this formula xci are feature values for the features that we're not interested in. PDP makes the assumption that the features in c are not correlated with the features and s. If this assumption is violated, the averages calculated will include data points that are very unlikely or even impossible.

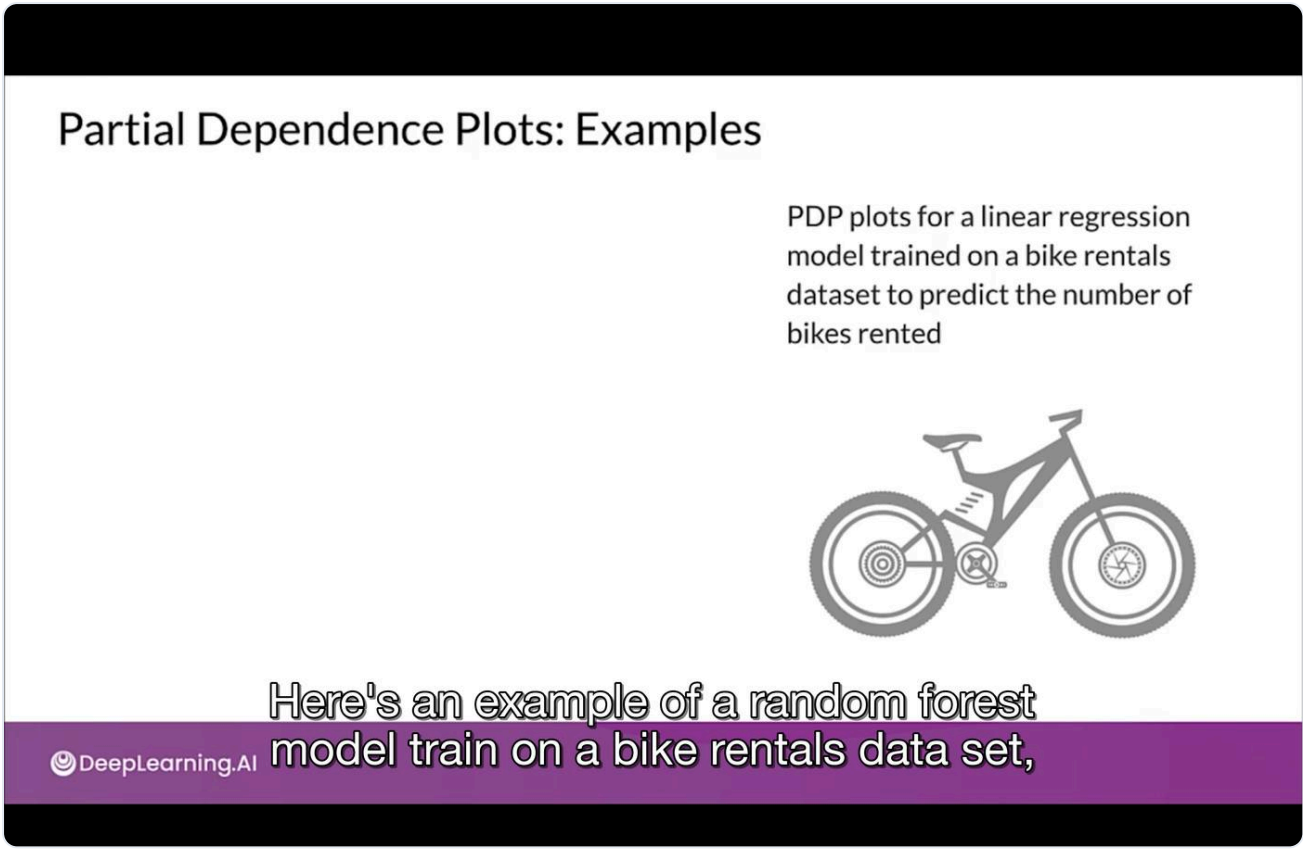

FIGURE 45.3 | Partial Dependence Plots: Examples

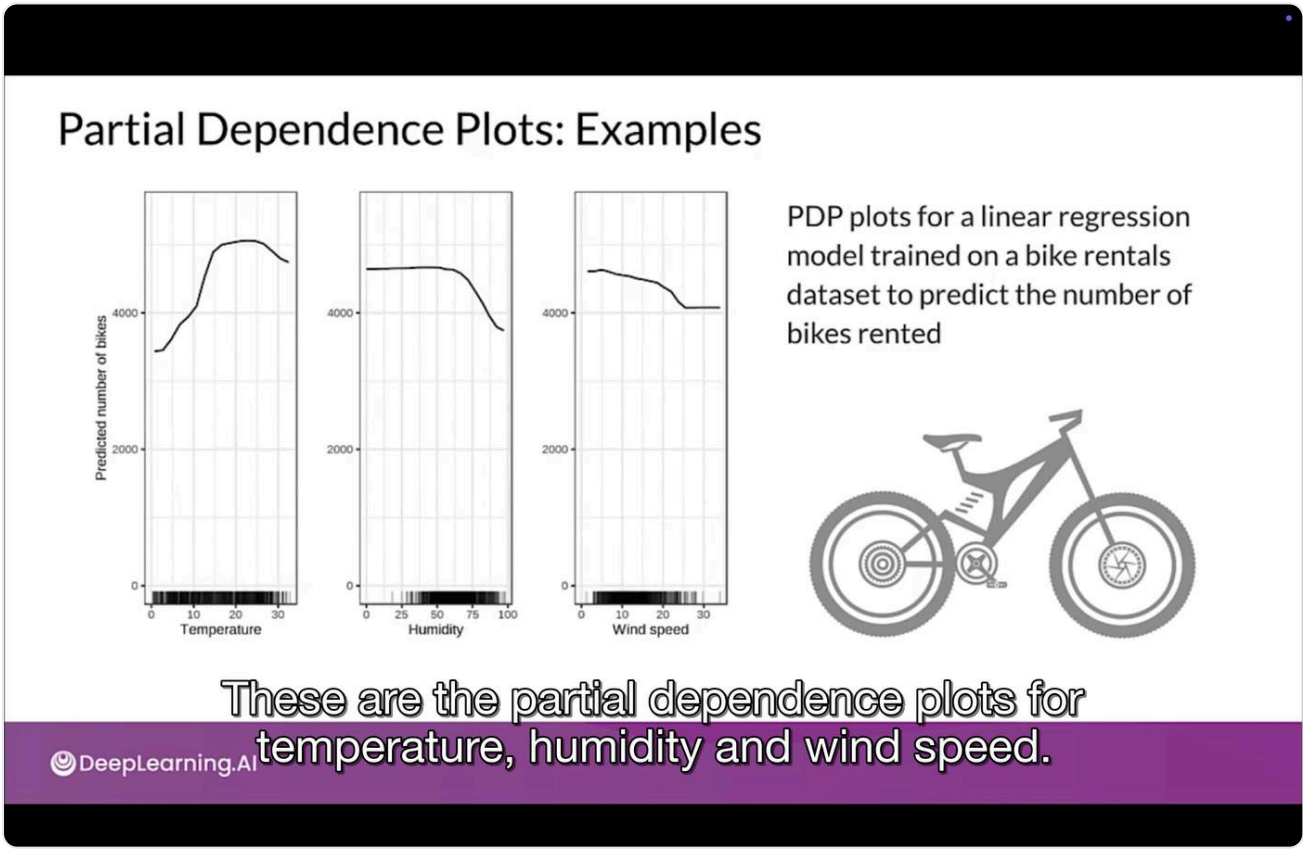

FIGURE 45.4 | Partial Dependence Plots: Examples

Here's an example of a random forest model train on a bike rentals data set, to predict the number of bikes rented per day given a set of features which include temperature, humidity and wind speed. These are the partial dependence plots for temperature, humidity and wind speed. Notice that as the temperature increases up to about 15 degrees C or 59 degrees F, more people are likely to rent a bike. This makes sense because people like to ride bikes when the weather is nice, and at that temperature I'd say, it's just starting to get nice. But notice that this trend level is off and then starts to fall off between above about 25 degrees C or 77 degrees F.

You can also see that humidity is a factor and above about 60 humidity, people start to get less interested in riding bikes. How about you, do these plots match your bike riding preferences?

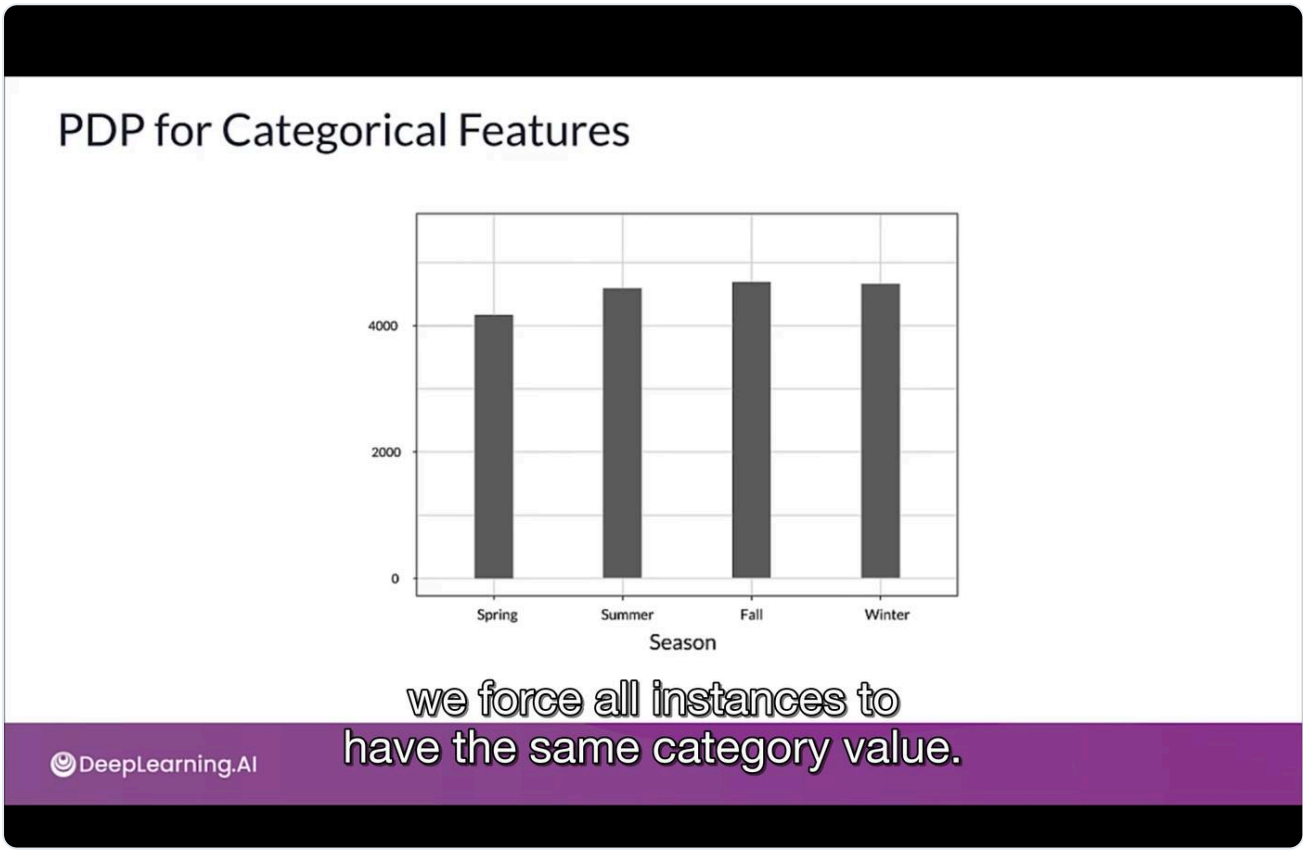

FIGURE 45.5 | PDP for Categorical Features

To calculate a PDP for categorical features, we force all instances to have the same category value. Here's the plot for the categorical features season in the bike rentals dataset. It has four possible values, spring, summer, fall and winter. In order to calculate the PDP for summer, we force all instances in the data set to have value equals summer for the feature season. This plot shows different values for season. Notice that there is not much difference of an effect if a change in seasons on bike rentals, except for in spring when the number of rentalss is somewhat lower. Frankly, I wouldn't expect that to be the case, but that's what the data is telling us.

There are some clear advantages to using PDP first. The results tend to be intuitive, especially when the features are not correlated. When they're not correlated, a PDP's plot shows the average prediction changes when a feature has changed. The interpretation of a PDP is also usually causal, in the sense that if we change a feature and measure the changes in the results, we expect the results to be consistent.

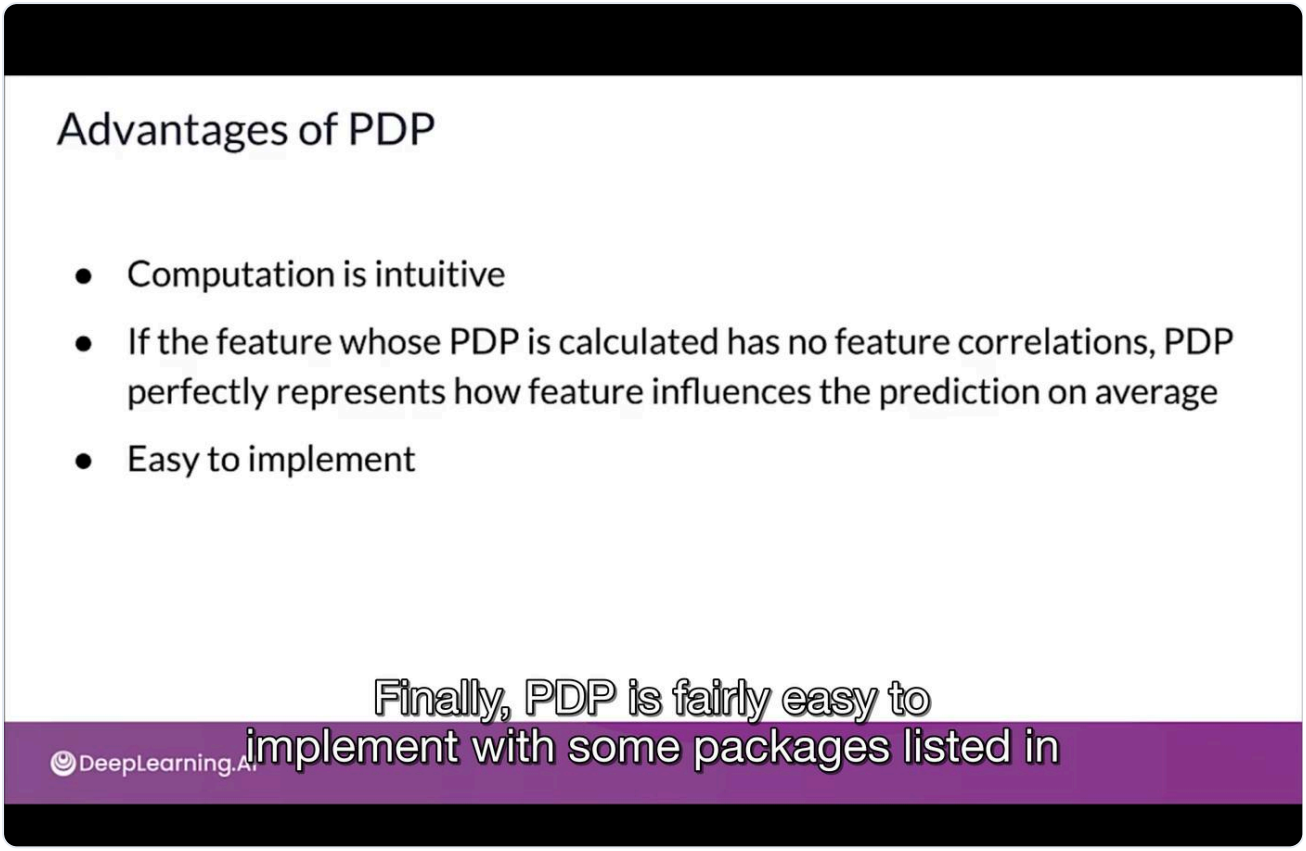

FIGURE 45.6 | Advantages of PDP

Finally, PDP is fairly easy to implement with some packages listed in the reference section at the end of this lesson. Like most things, however, there are some disadvantages to PDP. Realistically, you can only really work with two features at a time, because humans have a hard time visualizing more than three dimensions. But I'm really not sure that I would blame PDP for that. More serious limitation is the assumption of independence. PDP assumes that the features that you're analysing aren't correlated with other features.

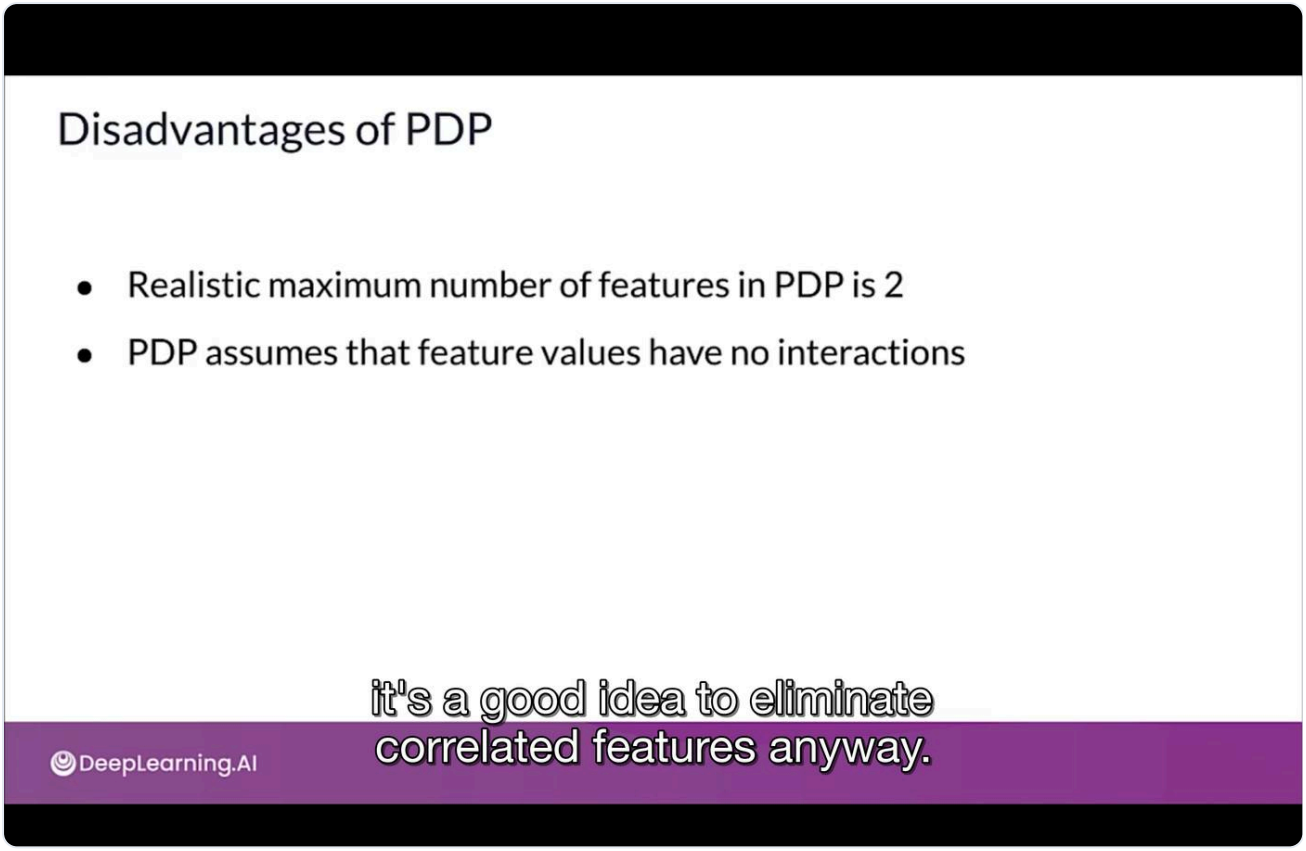

FIGURE 45.7 | Disadvantages of PDP

As we learned in our discussion of feature selection, it's a good idea to eliminate correlated features anyway. But, if you still have correlated features, PDP doesn't work quite right. For example, suppose you want to predict how fast a person walks given the person's height and weight. PDP will assume that height and weight aren't correlated, which is obviously a false assumption. As a result, we might include people with a height of two metres and a weight of 50 kg, which is a bit unrealistic even for fashion models, although when I googled this, I was shocked to learn that some are actually pretty close. But anyway, you get the idea correlated features are bad.

LECTURE 46

Advantages of Permutation Feature Importance

Now let's take a look at permutation feature importance. Permutation feature importance is just one way of measuring the importance of a feature. Permuting a feature is just one way of breaking the relationship between a feature and the model result. Essentially by assigning a nearly random value to the feature. For permutation feature importance, we measure the importance of a feature by measuring the increase in the prediction error after permuting that feature. A feature is important if shuffling its values increases the model error. Because in this case, the model relied on the feature for the prediction.

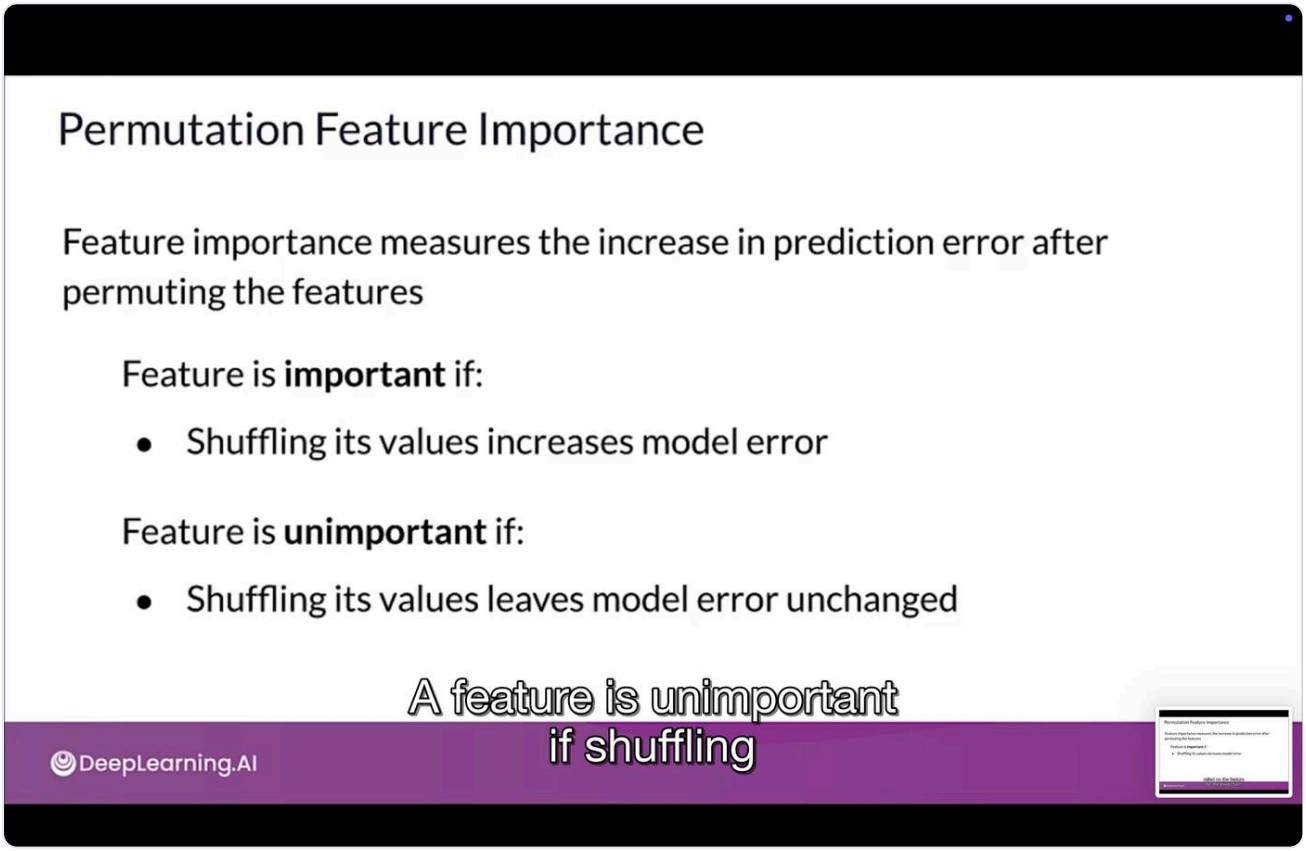

FIGURE 46.1 | Permutation Feature Importance

A feature is unimportant if shuffling its values leaves the model error unchanged. Because in this case, the model ignored the feature. If we find that we have unimportant features then we should really consider removing them from our feature vector. This is the basic algorithm. The inputs are the model, the features, the labels or targets, and our error metric. You start by measuring the model error with all of the true values of the features. Next, you started iterative process for each feature where you first permute the values of the feature that you're examining and measure the change in the model error.

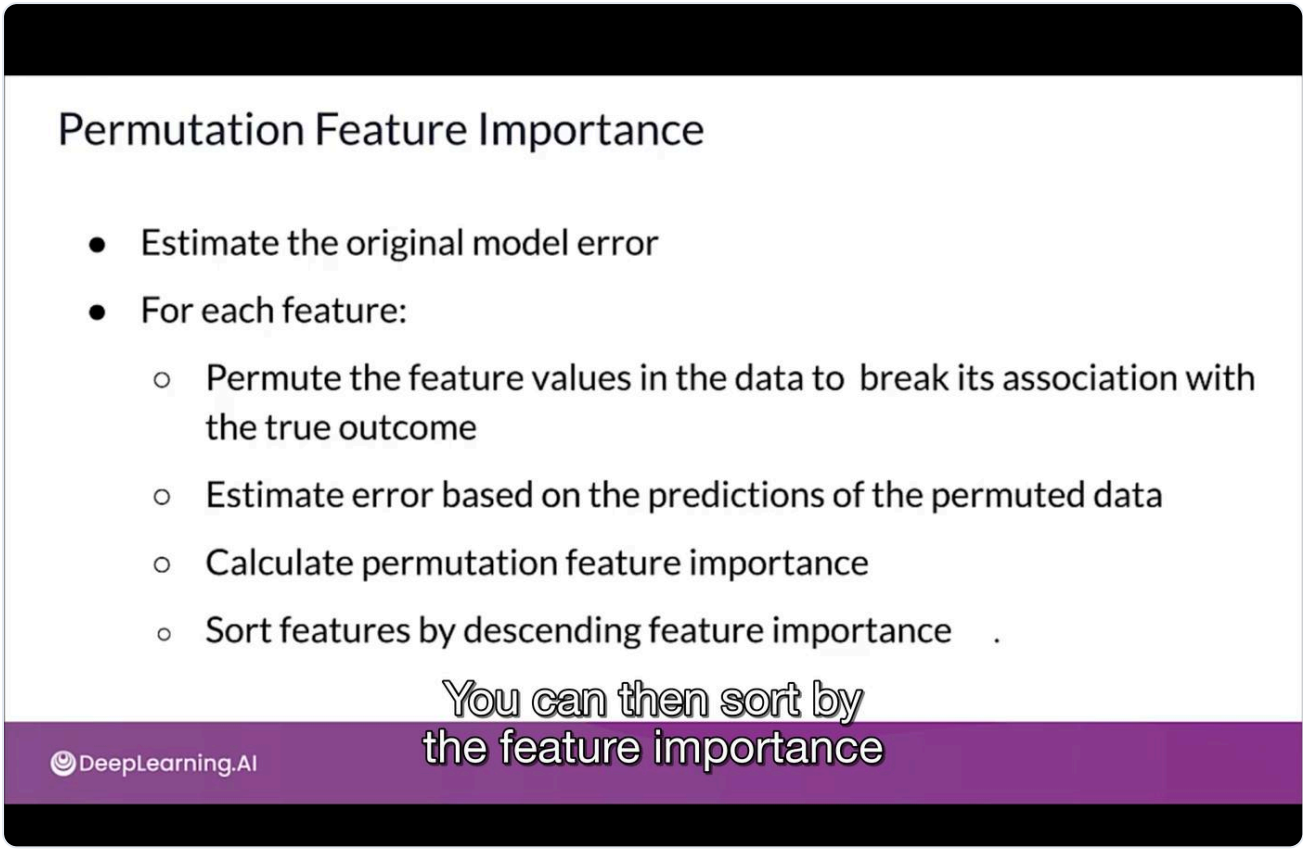

FIGURE 46.2

You can either express the feature importance as a ratio of the permuted error to the original error or as the difference between the two. You can then sort by the feature importance to determine the least important features. Let's consider the advantages of permutation feature importance. Permutation feature importance has a nice interpretation because feature importance is the increase in the model error when the feature's information is destroyed. It's a highly compressed global insight into the model's behavior. Since by permuting the feature you also destroy the interaction effects with other features, it also shows the interactions between features.

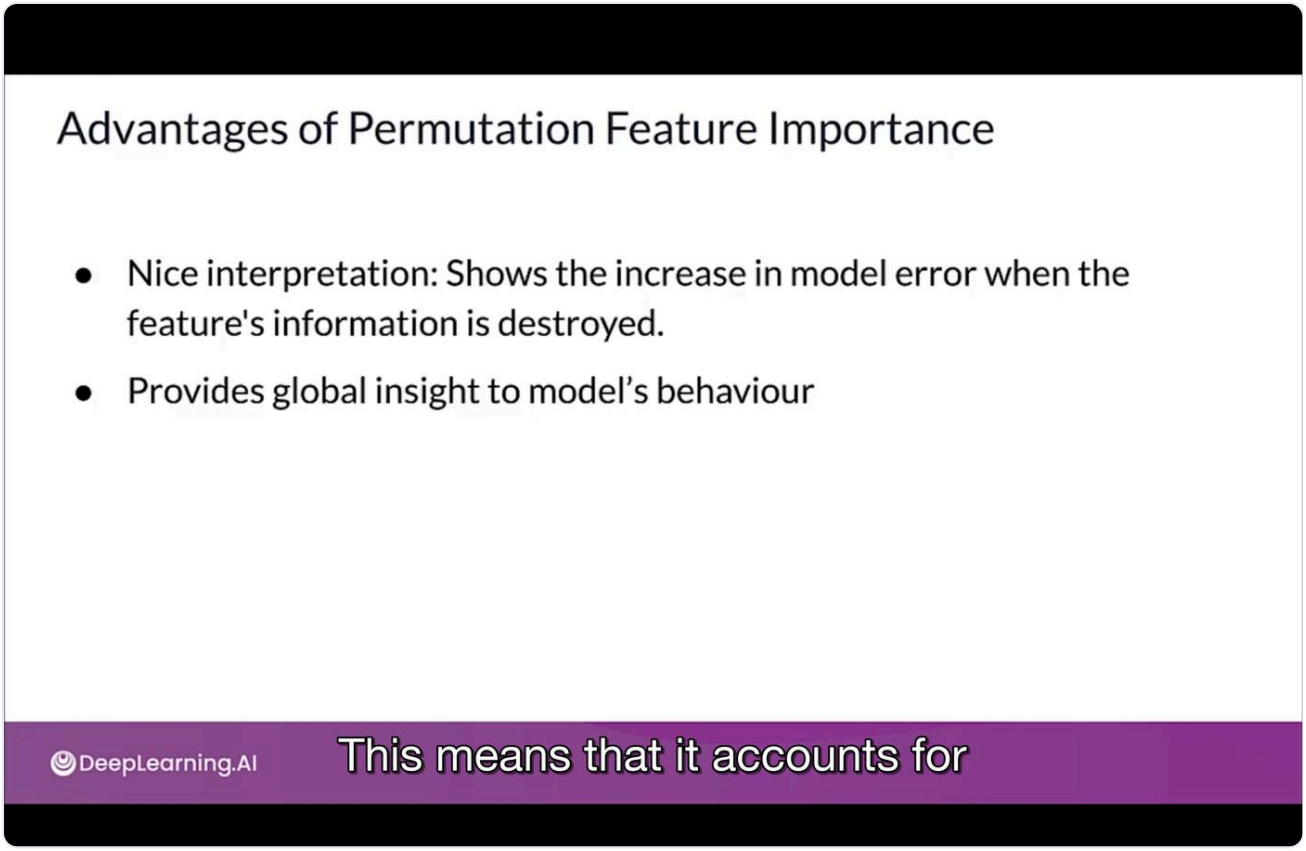

FIGURE 46.3 | Advantages of Permutation Feature Importance

This means that it accounts for both the main feature effects and the interaction effects on model performance.

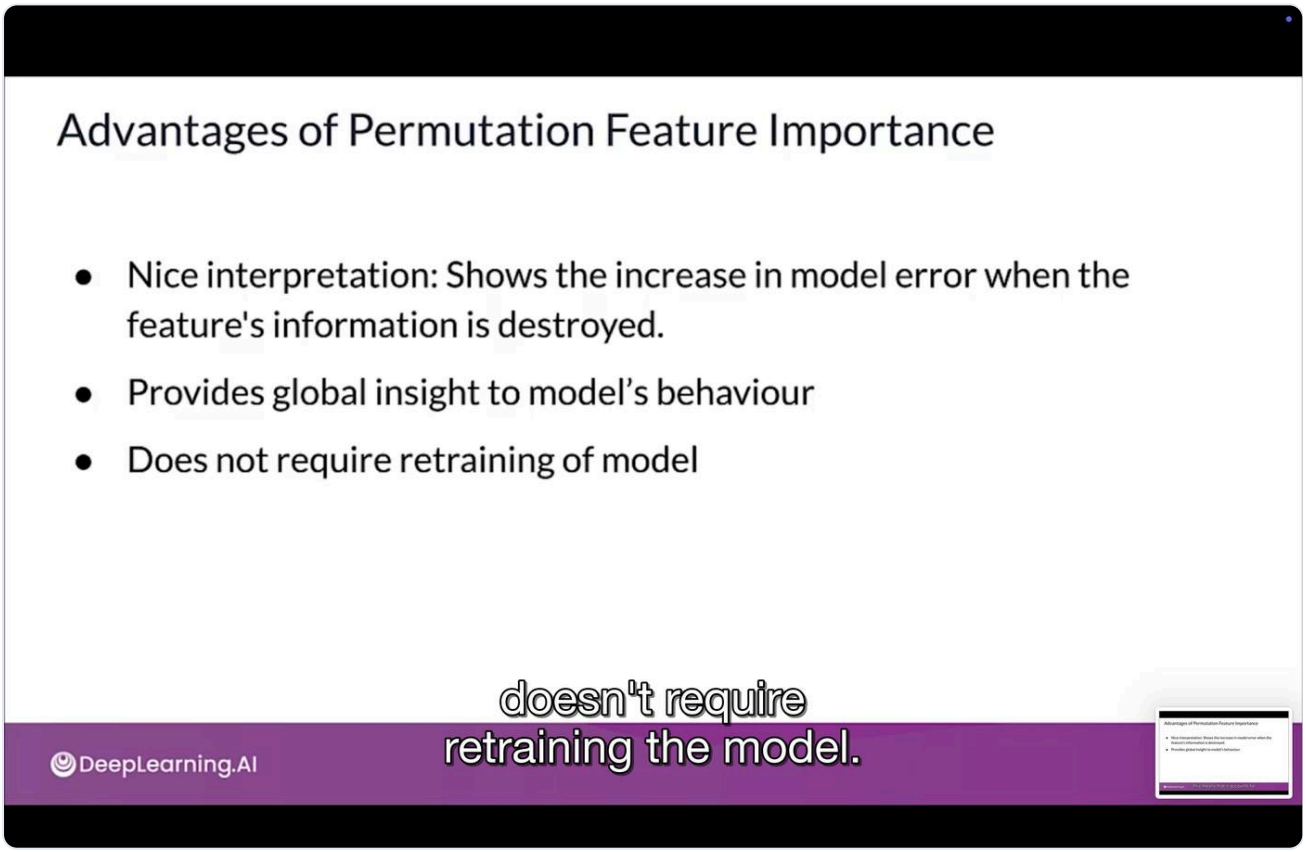

FIGURE 46.4 | Advantages of Permutation Feature Importance

A big advantage is that it doesn't require retraining the model. Some other methods suggest deleting a feature, retraining the model, and then comparing the model error. Since retraining a model that can take a long time, not requiring that is a big advantage. Let's look at some of the disadvantages inherent to permutation feature importance. One odd disadvantage is that it's unclear whether you should use your training data or your test data to measure permutation feature importance. There are pluses and minuses to both. Like PDP, correlated features are once again a problem.

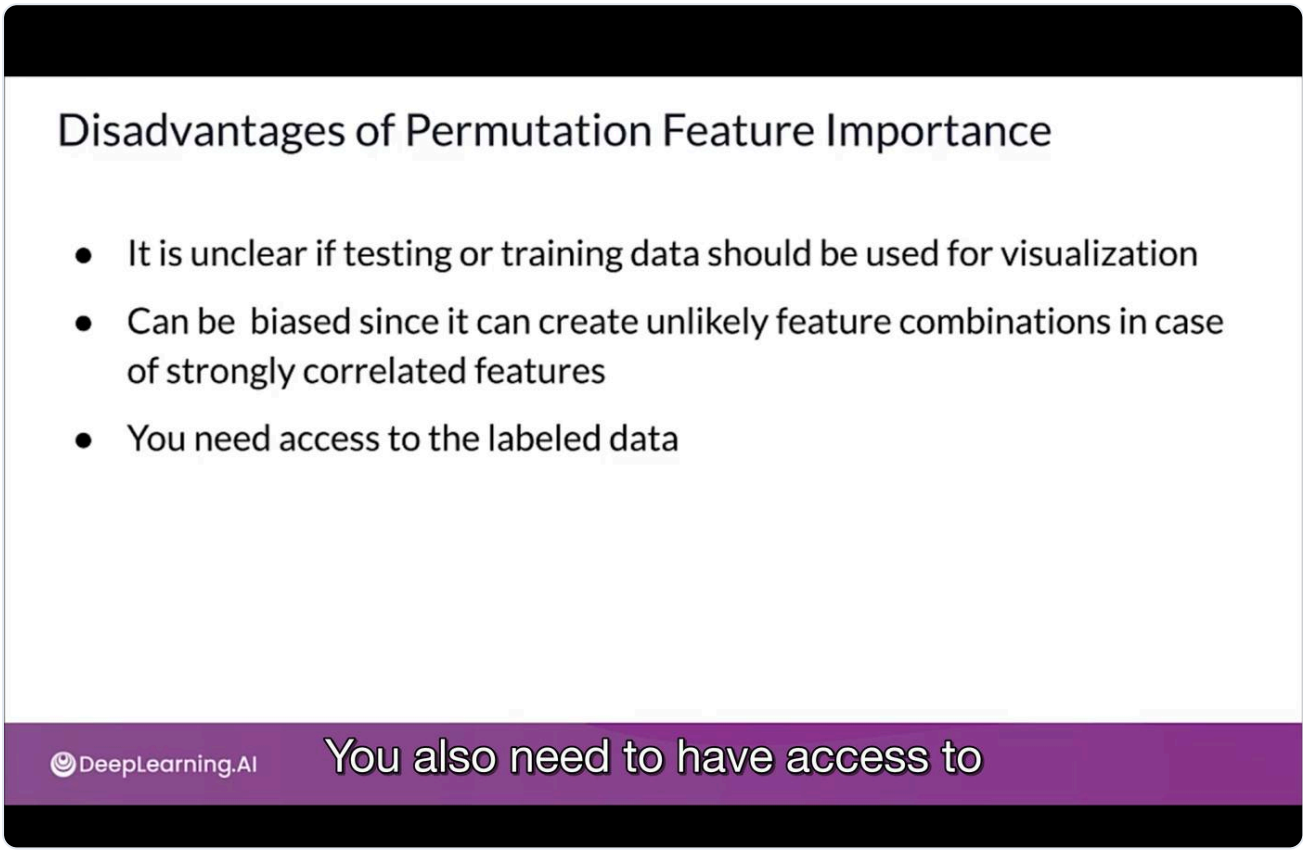

FIGURE 46.5 | Disadvantages of Permutation Feature Importance

You also need to have access to the original labeled training data set. If you're getting the model from someone else and they don't give you that, then you can't use permutation feature importance.

LECTURE 47

Disadvantages of Shapley Values

A key concept in measuring feature importance and the contribution of each feature to the models results is the Shapley value. Let's discuss that now. The Shapley value is a concept from Cooperative Game Theory. It was named in honor of Lloyd Shapley who introduced it in 1951 and won the Nobel Prize in Economics for it in 2012. To each cooperative game it assigns a unique distribution among the players of the total surplus value generated by the coalition of all players. Here's how it works in game theory. Imagine that a group of players cooperates and that results in an overall gain because of their cooperation. Since some players may contribute more than others or may possess different bargaining power, how should we distribute the gains among the players?

Or phrase differently, how important is each player to the overall cooperation and what payoff can he or she reasonably expect? The Shapley value provides one possible answer to this question. So how do you apply this to ideas in ML? For machine learning and interpret ability, the players are the features of the data set, and we're using the Shapley value to determine how much each feature contributes to the results. As you might expect the prediction is the payout.

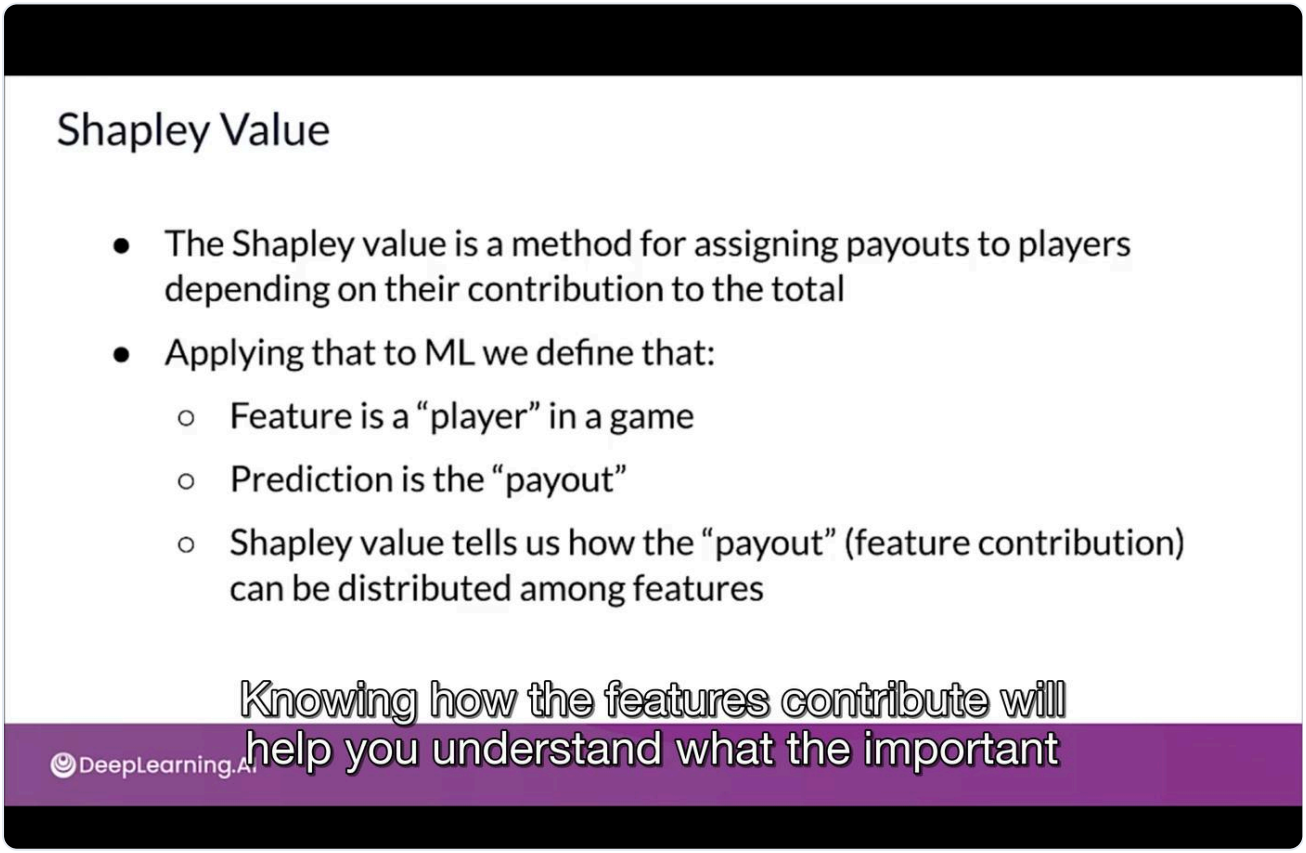

FIGURE 47.1 | Shapley Value

Knowing how the features contribute will help you understand what the important factors were in generating the models results. Because it's not specific to any particular type of model it can be used regardless of the model architecture. That was a quick overview of the ideas behind

Shapley value. Let's now focus on a concrete example, suppose you trained an ML model to predict apartment prices.

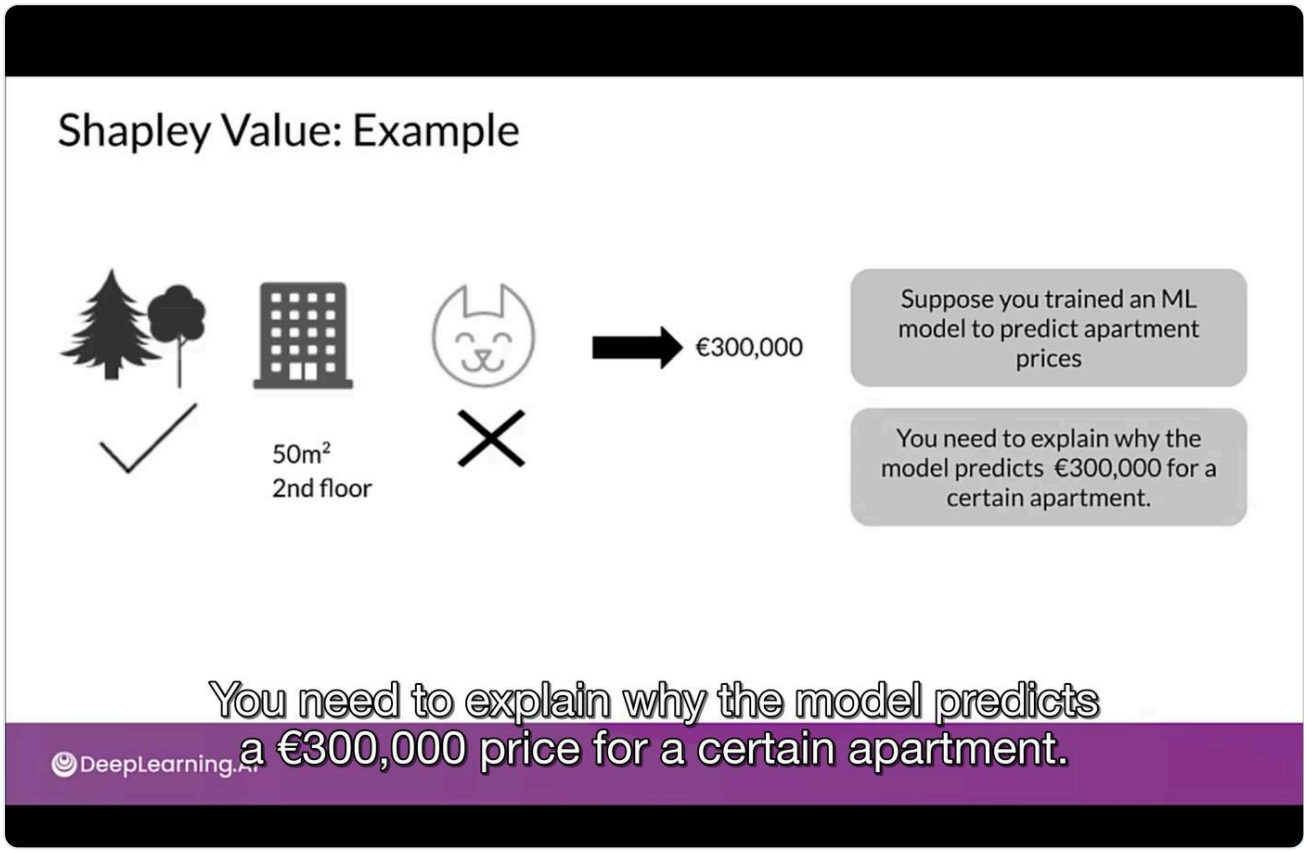

FIGURE 47.2 | Shapley Value: Example

You need to explain why the model predicts a €300,000 price for a certain apartment. What data do you have to work with? Well, in this example the apartment is 50 square meters. It's located on the second floor, it has a park nearby and cats are banned. You're not a cat, so you're okay.

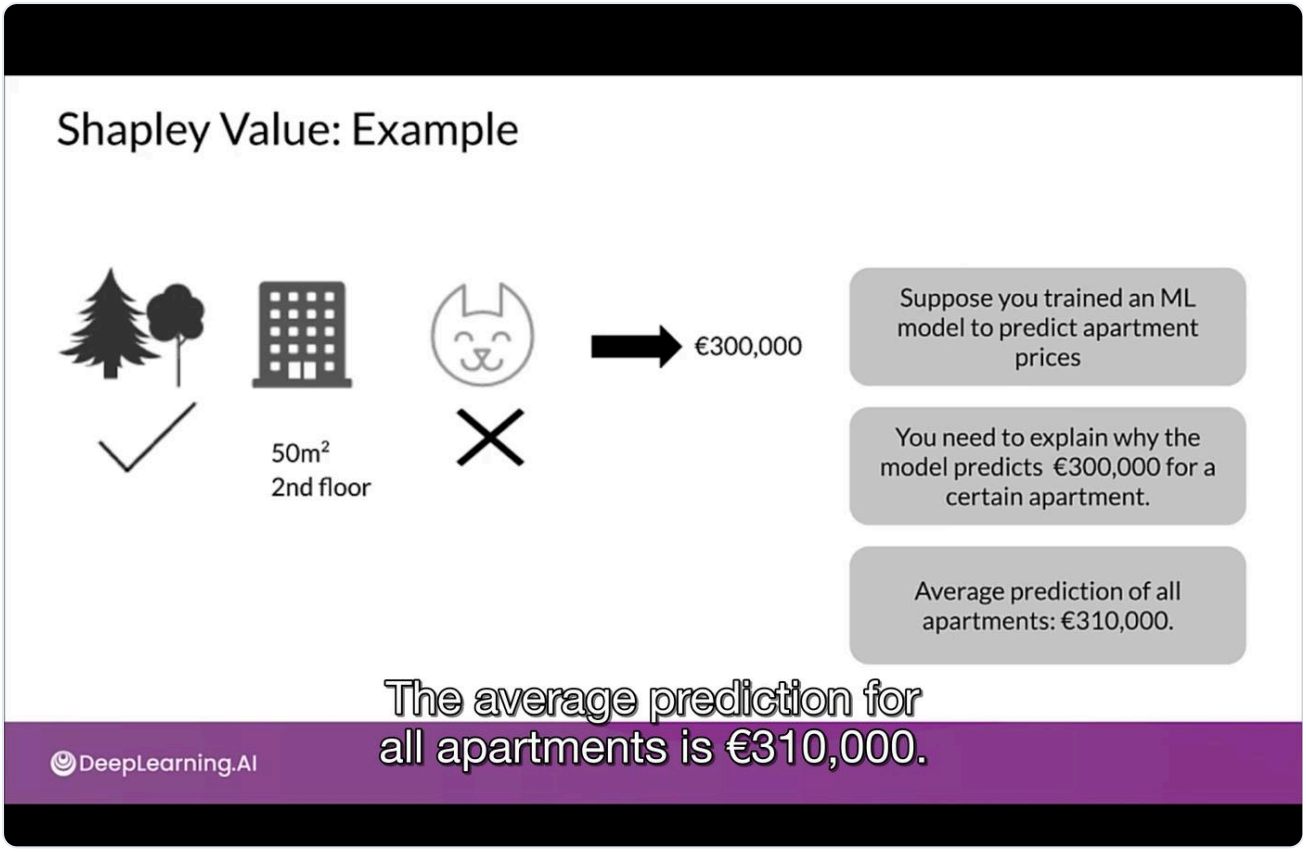

FIGURE 47.3 | Shapley Value: Example

The average prediction for all apartments is €310,000. Shapley values come from game theory, so let's clarify how to apply them to machine learning interpret ability.

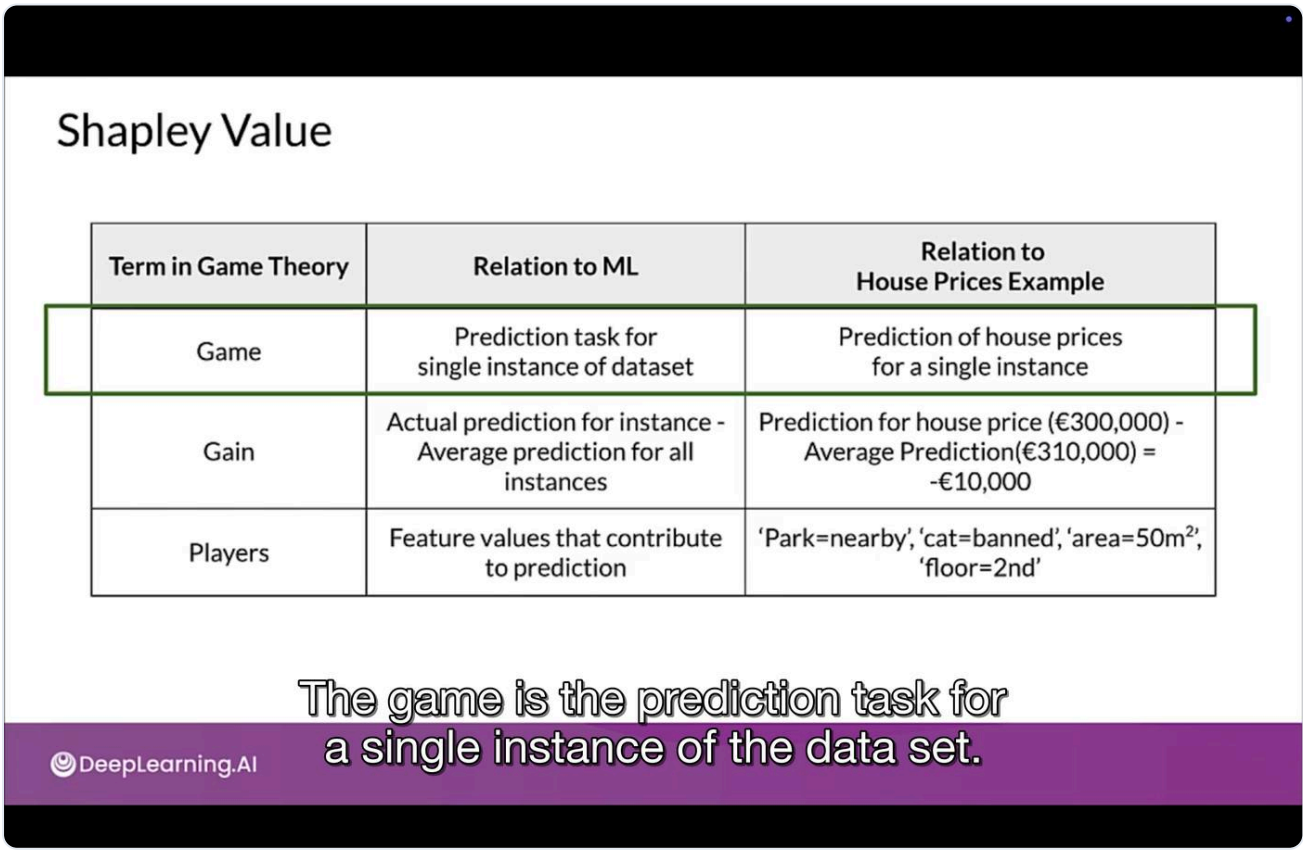

FIGURE 47.4 | Shapley Value

The game is the prediction task for a single instance of the data set. The gain is the actual prediction for this instance minus the average prediction for all instances. The players are the feature values of the instance that collaborate to produce the gain, in the apartment example the feature values park equals nearby, cat equals banned, area equals 50 square meter and floor equals 2nd, work together to achieve the prediction of €300,000.

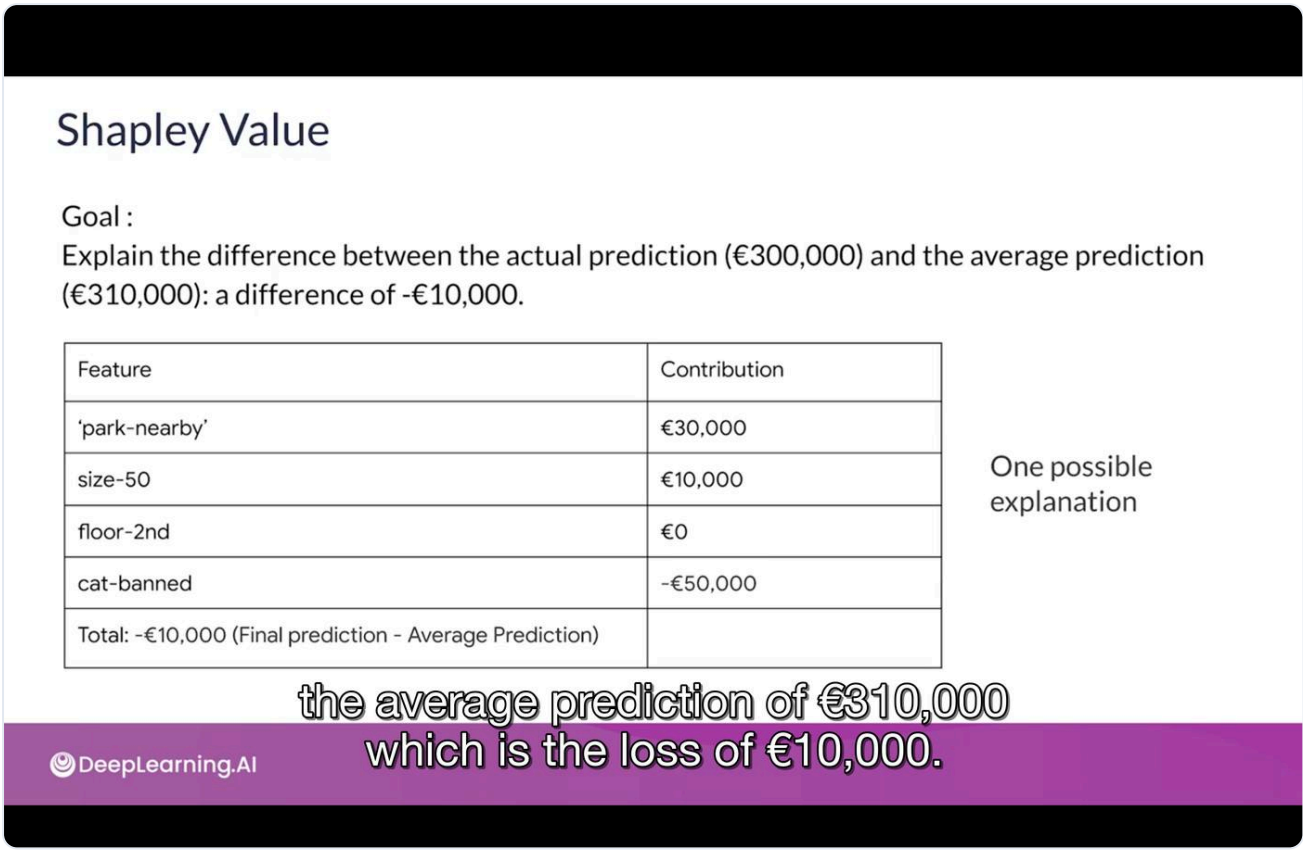

FIGURE 47.5 | Shapley Value

Our goal Is to explain the difference between the actual prediction €300,000 and the average prediction of €310,000 which is the loss of €10,000. One possible explanation could be the park nearby contributed €30,000, size of 50 square meter contributed €10,000. Floor equals 2nd contributed zero. Cat equals banned contributed negative €50,000. The contributions add up to negative €10,000. The final prediction minus the average predicted apartment price. You can think of that as the absolute value €10,000 or you could also think about it as the percentage of the average which is about 3.3%. This is one possible explanation, but how did we get these numbers?

Unlike perhaps any other method of interpreting model results, Shapley values are based on a solid theoretical foundation. Other methods make intuitive sense, which is an important factor for interpret ability but don't have the same rigorous theoretical foundation. This is one of the reasons that Shapley was awarded the Nobel Prize for his work.

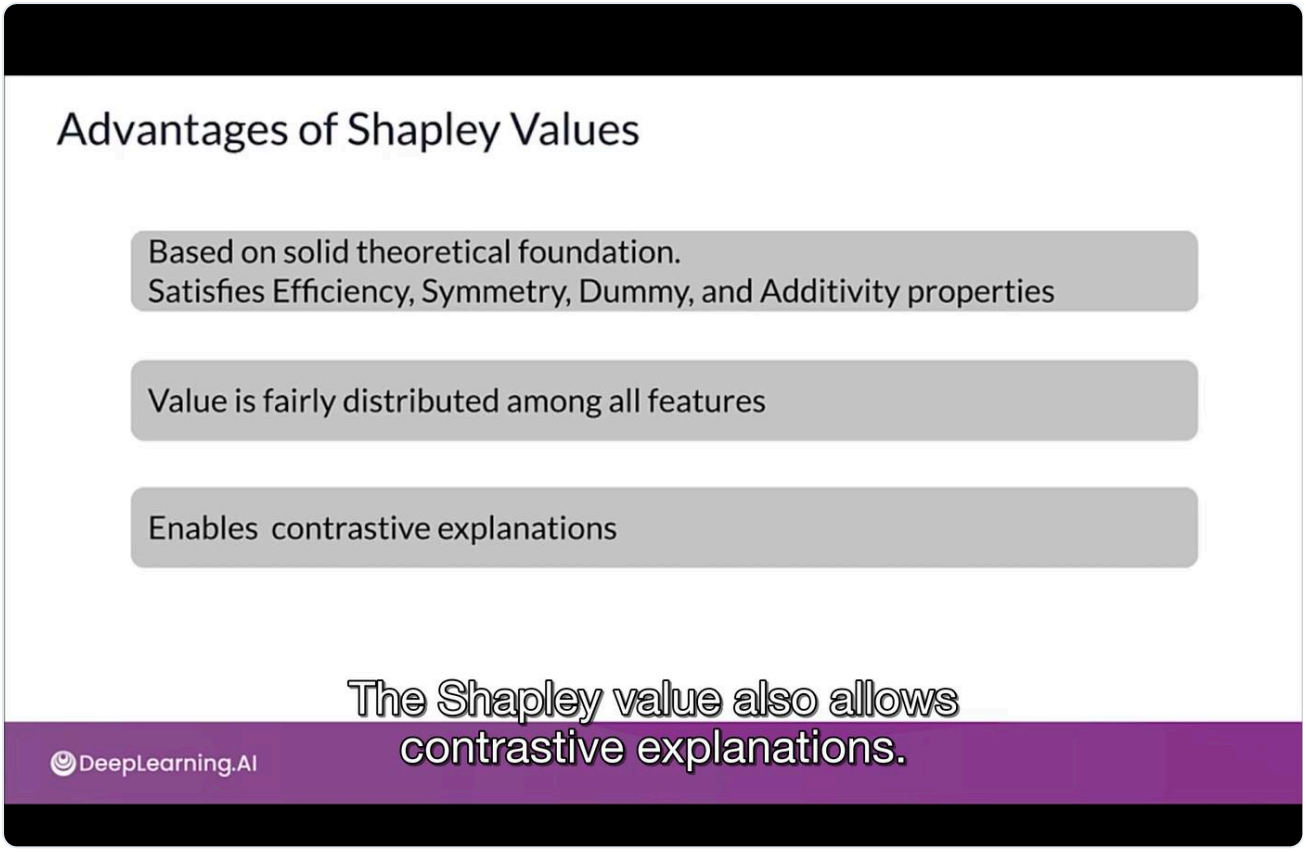

FIGURE 47.6 | Advantages of Shapley Values

The theory defines four properties which must be satisfied, Efficiency, Symmetry, Dummy and Additivity. One key advantage of Shapley values that they're fairly distributed among the feature values of an instance. Some have argued that Shapley might be the only method to deliver a full explanation, in situations where the law requires explainability like the use right to explanations for example. Some feel that Shapley value might be the only legally compliant method because it's based on a solid theory and distributes the effects fairly. The Shapley value also allows contrastive explanations. Instead of comparing a prediction to the average prediction of the entire data set, you could compare it to a subset or even to a single data point.

This ability to contrast is something that local models like Lime don't have. Like any method Shapley has some disadvantages. Probably the most important is that its computationally expensive, which in a large percentage of real world cases means that it's only feasible to calculate at approximate solution. It can also be easily misinterpreted. The Shapley value is not the difference of the predicted value after removing the feature from the model training, it's the contribution of a feature value to the difference between the actual prediction and the mean prediction. If you want to only explain a few of your features, Shapley is probably the wrong method to use, Shapley always uses all the features.

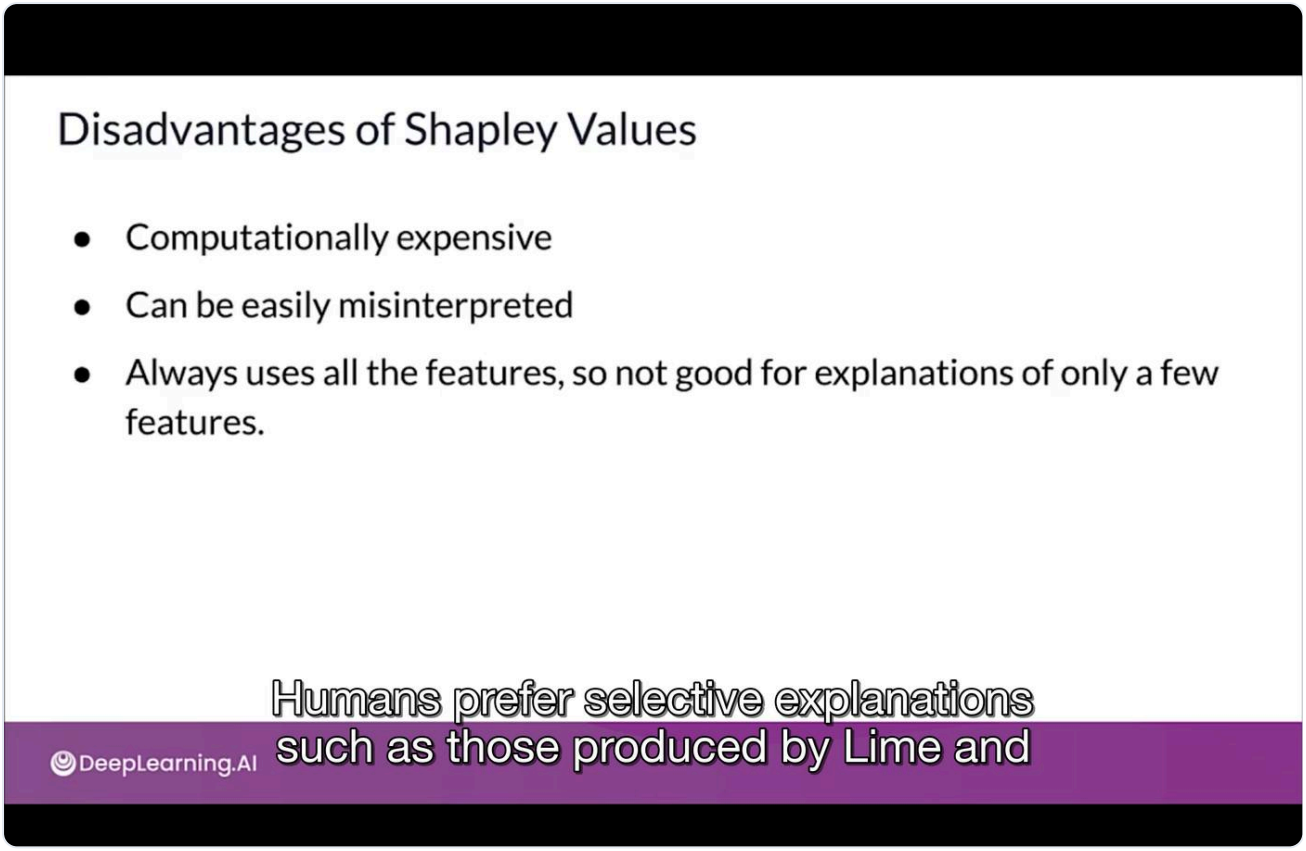

FIGURE 47.7 | Disadvantages of Shapley Values

Humans prefer selective explanations such as those produced by Lime and other similar methods. So those might be the better choice for explanations that lay persons have to deal with. Or another possible solution is to use Shap which is based on the Shapley value but can provide explanations with only a few features, we'll discuss Shap next. Unlike some other methods, Shapley does not create a model.

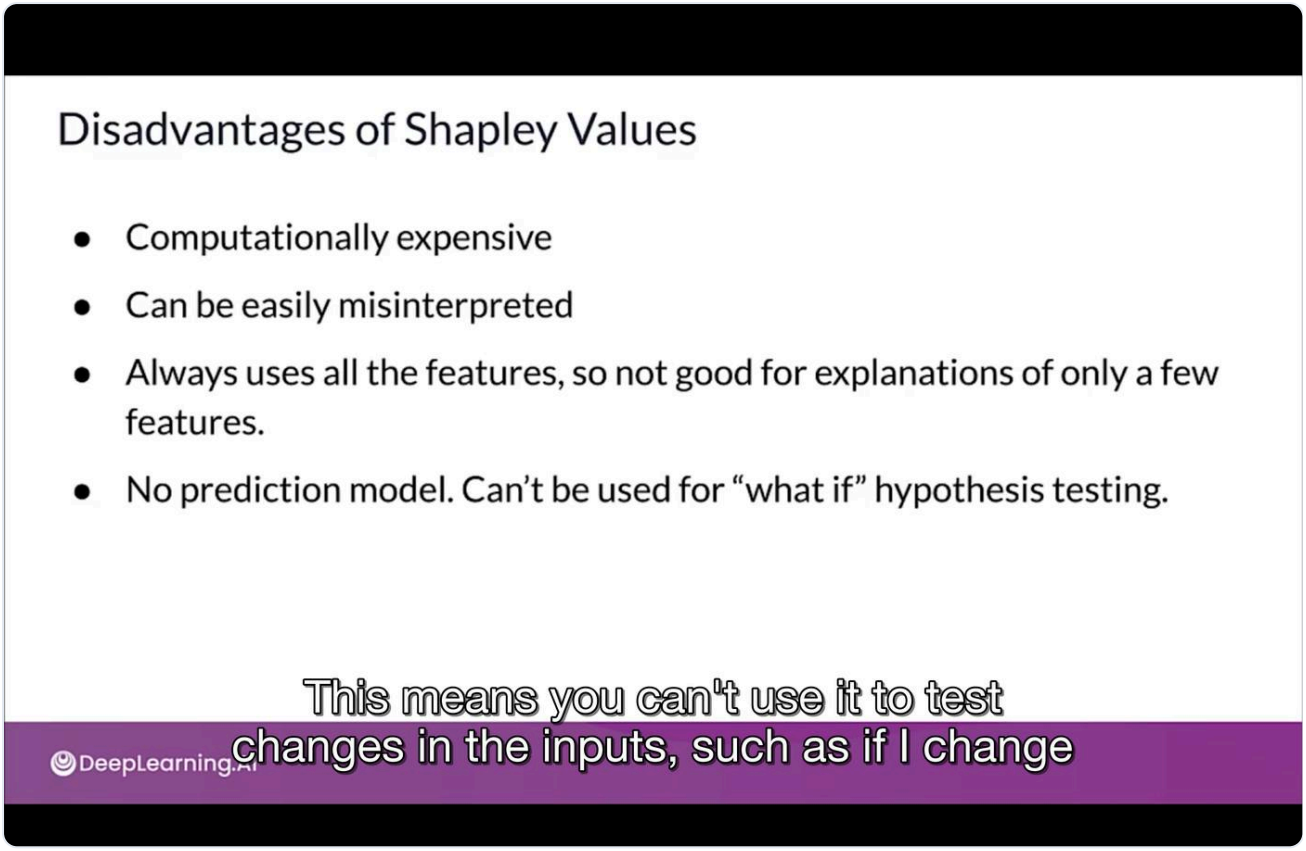

FIGURE 47.8 | Disadvantages of Shapley Values

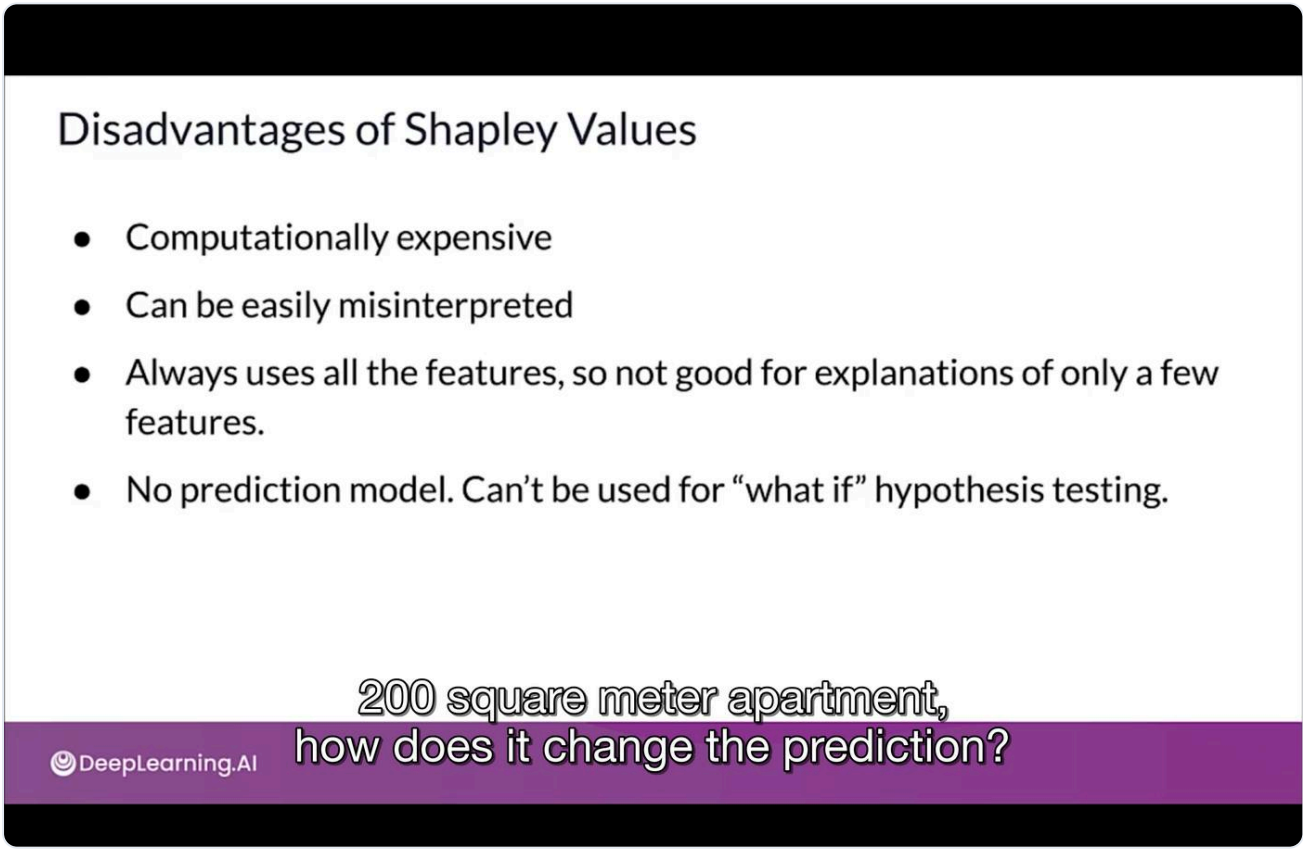

FIGURE 47.9 | Disadvantages of Shapley Values

This means you can't use it to test changes in the inputs, such as if I change 200 square meter apartment, how does it change the prediction?

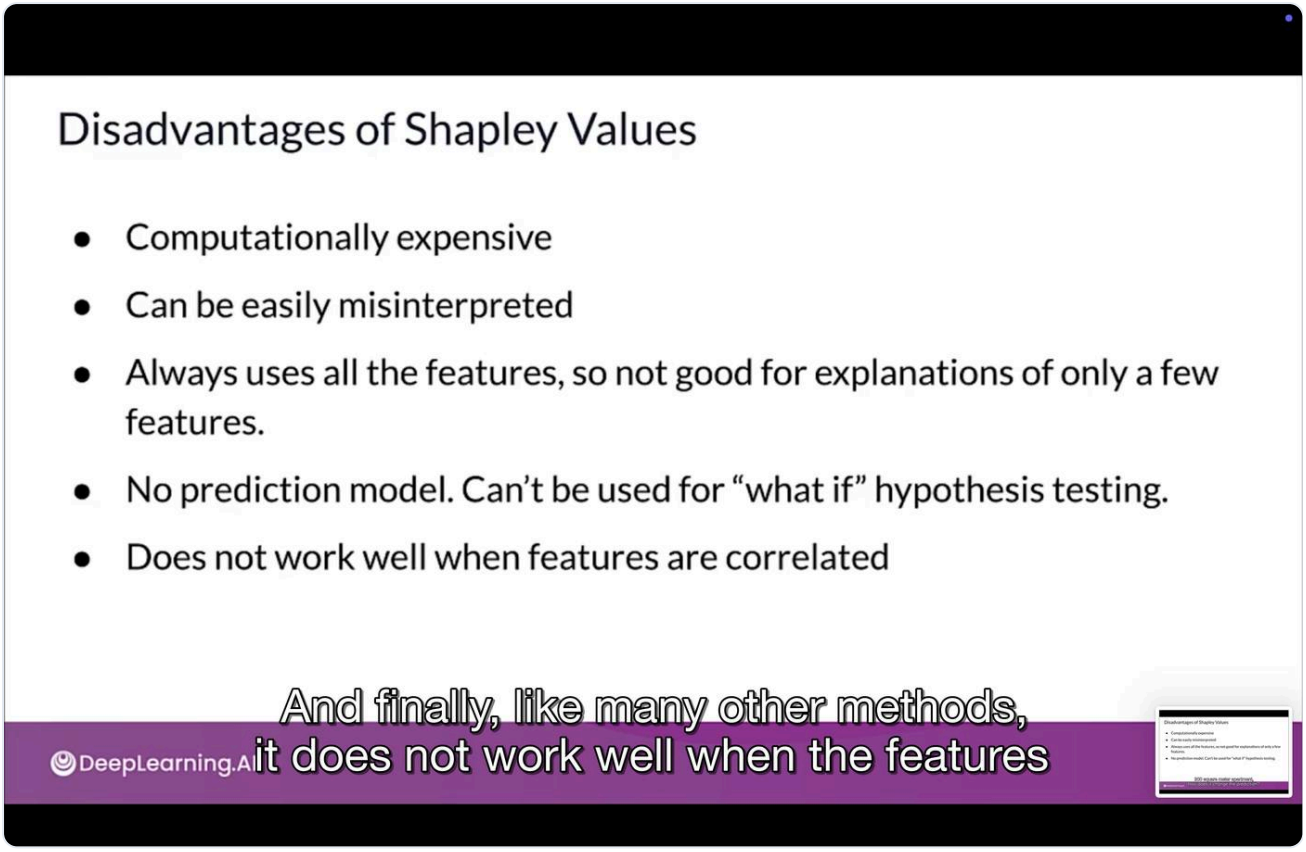

FIGURE 47.10 | Disadvantages of Shapley Values

And finally, like many other methods, it does not work well when the features are correlated, but you already know that you should have removed correlated features from your feature vector when you were doing feature selection. So that's not a problem for you, right? Well, hopefully anyway but something to be aware of.

LECTURE 48

SHAP Summary Plot

Now let's take a look at the open source Shap library, which is a powerful tool for working with sharply values and other similar measures. Shapp, which is short for shapely additive explanations, is a game theoretic approach to explain the output of any machine learning model, which makes it model agnostic. It connects optimal credit allocation with local explanations. Using the classic sharply values from game theory and their related extensions, which have been the subject of several recent papers. Remember that shapely created his initial theory in 1951 and more recent researchers have been extending his work. It assigns each feature and importance value for a particular prediction and includes some very useful extensions, many of which are based on this recent theoretical work.

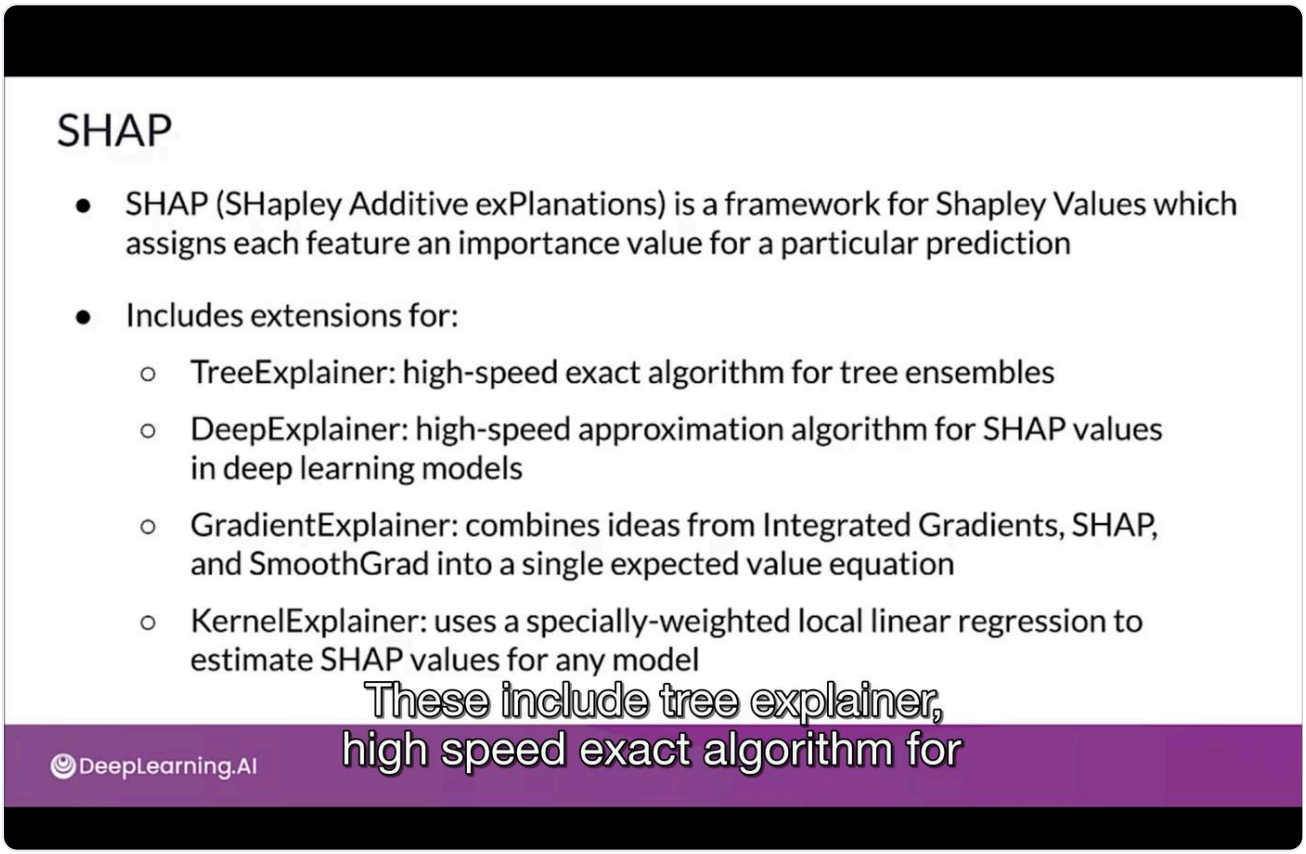

FIGURE 48.1

These include tree explainer, high speed exact algorithm for tree ensembles, deep explainer, high speed approximation algorithm for chef values in deep learning models, gradient explainer, which combines ideas from integrated gradients, Shap and smooth grad into a single expected value equation, and Colonel explainer, which uses especially waited local linear regression to estimate Shap values for any model. It also includes several plots to visualize the results, which helps you to interpret the model. You can visualize shapely values as forces.

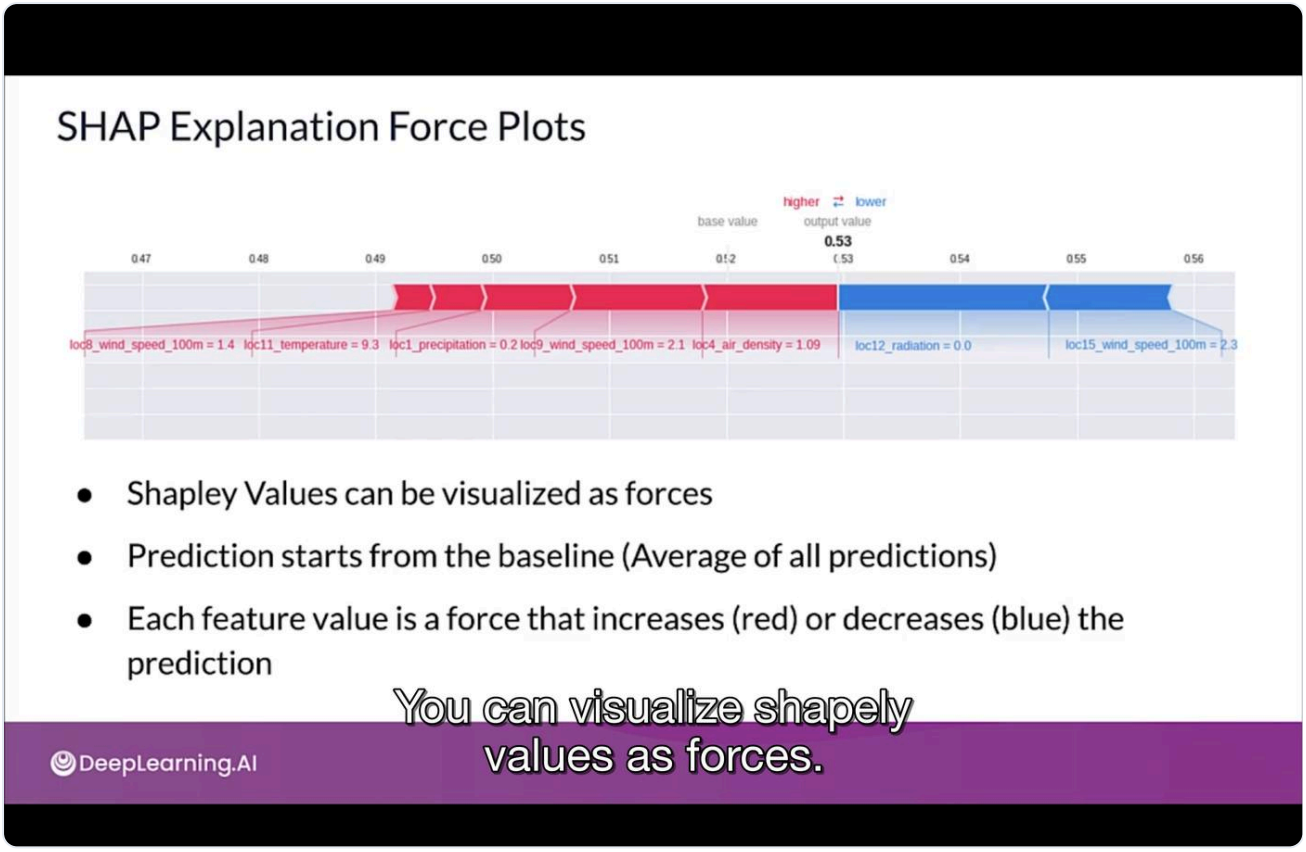

FIGURE 48.2 | SHAP Explanation Force Plots

Each feature value is a force that either increases or decreases the prediction. The prediction starts from the baseline, which for sharply values is the average of all the prediction. In a force plot, each shapely value is displayed as an arrow that pushes the prediction to increase positive values shown here in red or decrease negative values shown here in blue. These forces meet at the prediction to balance each other out.

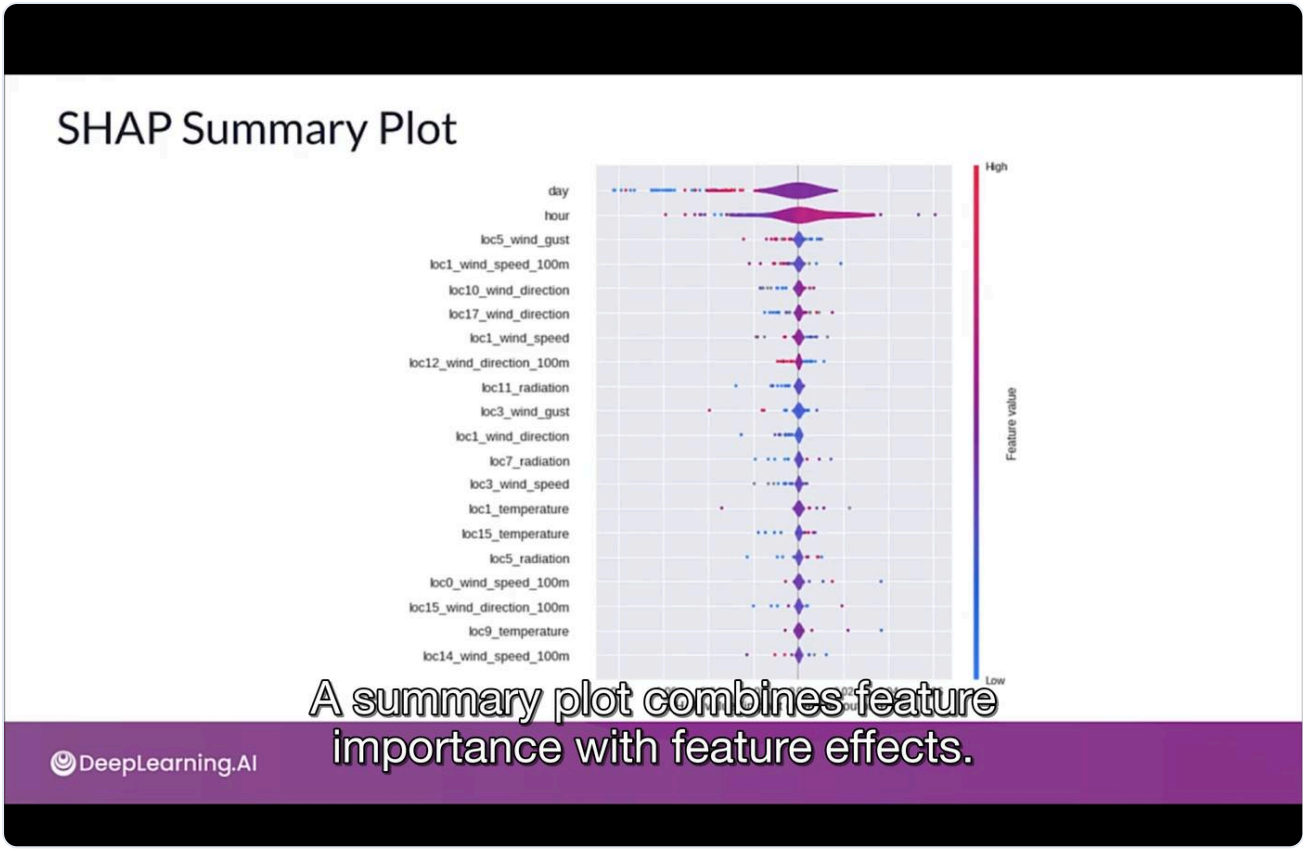

FIGURE 48.3 | SHAP Summary Plot

A summary plot combines feature importance with feature effects.

FIGURE 48.4 | SHAP Summary Plot

Each point on the summary plot is a shapely value for a feature and an instance. The color represents the value of the feature from low blue to high red, overlapping points are deterred in the y axis direction. So we get a sense of the distribution of sharply values per feature and features are ordered according to their importance.

FIGURE 48.5 | SHAP Dependence Plot with Interaction

So in this example we can quickly see that the two most important features are day and hour in a shop dependence, flatter a feature value is plotted on the x-axis and the shop value plotted on the y-axis. From the plot in this example, you can see that the correlation between our and the Lok 15 temperature feature is low. In this example, in the early hours of the morning, right after midnight, the energy price drops during the afternoon hours between 10 and 12, the price increases, and then drops back down after three. Then the price again increases at night after 8 O'clock.

LECTURE 49

Testing Concept Activation Vectors (TCAV)

Let's get started with an advanced approach known as testing concept activation vectors. The interpretation of deep learning models is a challenge due to their size, complexity and often opaque internal state. In addition, many systems, such as image classifiers operate on low level features rather than high level concepts. To address these challenges, the team at google introduced concept activation vectors, or CAVs, which provide an interpretation of a neural networks internal state. In terms of human friendly concepts, high level human friendly concepts are defined using sets of example input data for the model under inspection, for instance, to define the concept curly, a set of images of hairstyles and texture of images can be used for an image model.

Note that these examples are not constrained to training data.

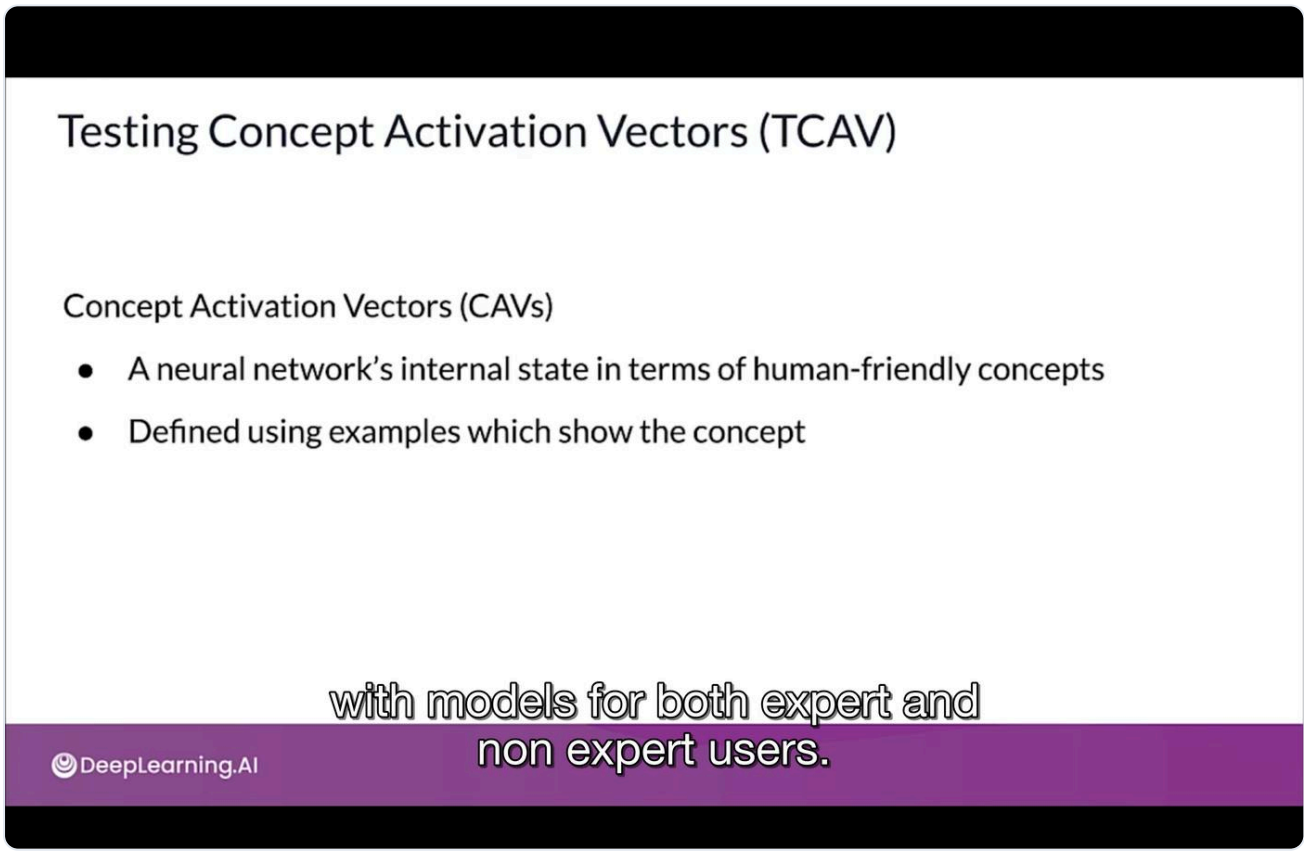

FIGURE 49.1 | Testing Concept Activation Vectors (TCAV)

They can be defined using new user provided data, examples have been shown to be effective means of interfacing with models for both expert and non expert users. You can use concept activation vectors to sort examples or in this case images with respect to their relationship to a concept.

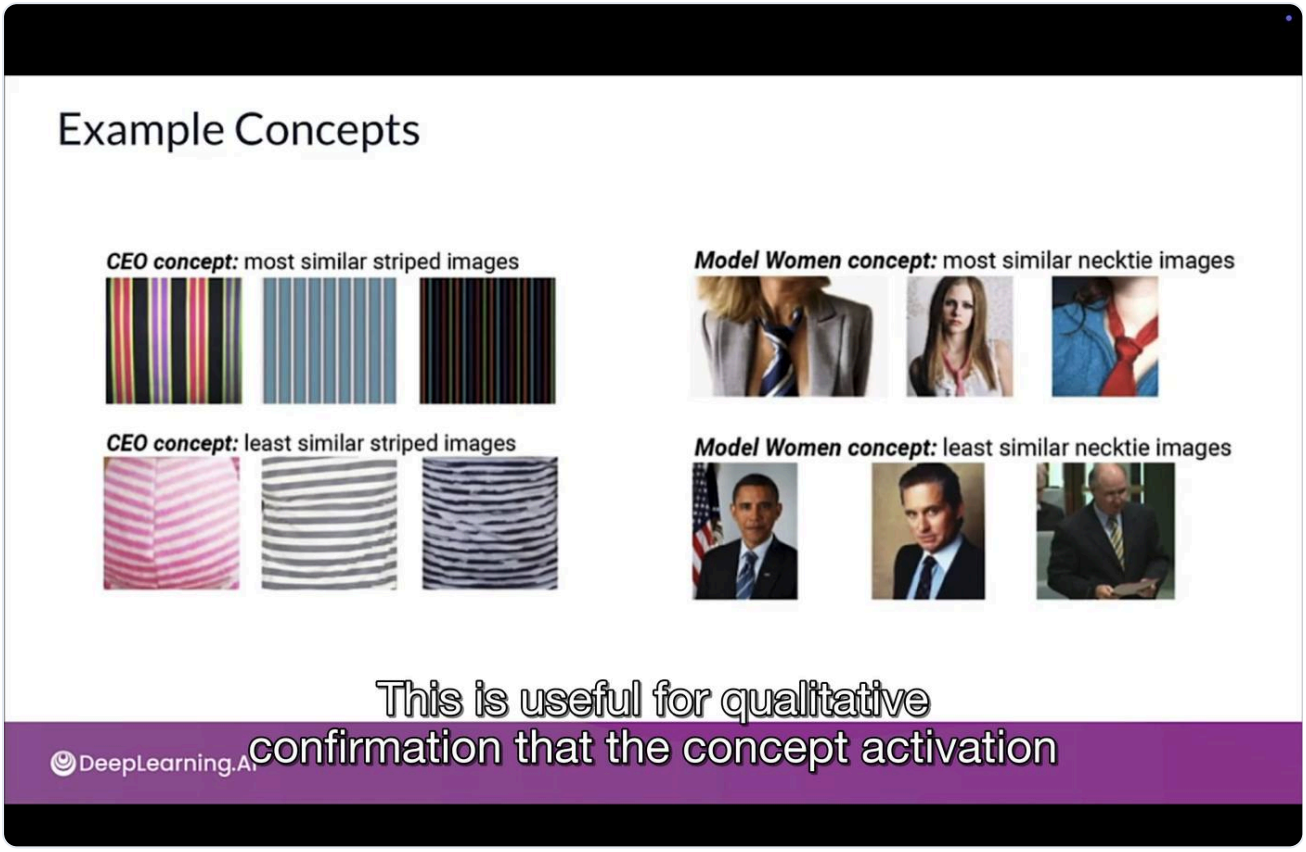

FIGURE 49.2 | Example Concepts

This is useful for qualitative confirmation that the concept activation vectors correctly reflect the concept of interest. As a concept, activation vector encodes the direction of a concept in the vector space of a bottleneck. We can compute cosine similarity between a set of pictures of interest to the casine activation vector in order to sort the pictures. Note that the pictures being assorted are not used to train the concept activation vector. Shown here are two concepts of interest CEO and model women and images that have been sorted by similarity to the concept. On the left are sorted images of stripes with respect to a concept activation vector learned from a more abstract concept CEO collected from image net.

The top three images are the most similar to the CEO concept and look like pinstripes, which may relate to the ties or suits that a CEO may wear. Which provides confirmation of the idea that CEOs are more likely to wear pinstripes than horizontal stripes. I'm told that horizontal stripes can look unprofessional, but I'm hardly a fashion expert, which you can tell by the way I dress. On the right are sorted images of neckties with respect to a model women concept activation vector. Again, the top three images are the most similar to the concept, which in this case is model women and show women in neckties, but all of the bottom three images show men in neckties.

This also suggests that concept activation vectors can be a standalone similarity sworder to sort images to reveal any biases in the example, images from which the concept activation victor is learned.

LECTURE 50

Local Interpretable Model-agnostic Explanations (LIME)

Now let's briefly discuss LIME, which is a popular and well-known framework for producing local explanations. LIME is a popular and well-known framework for creating local interpretations of model results. The idea is quite intuitive. First, forget about the training data and imagine that you only have a black-box model where you can input data points and get the predictions of the model. You can probe the box as often as you'd want. Your goal is to understand why the model made a certain prediction. LIME tests what happens to the predictions when you give variations of your data to the model. LIME generates a new dataset consisting of permuted samples and the corresponding predictions of the model.

With this new dataset LIME then trains an interpretable model, which is weighted by the distance from the sampled instances to the result that we're interpreting. The interpretable model can be anything that is easily interpretable, like a linear model or a decision tree. The new model should be a reasonably good approximation of the model results locally, but it does not have to be a good global approximation.

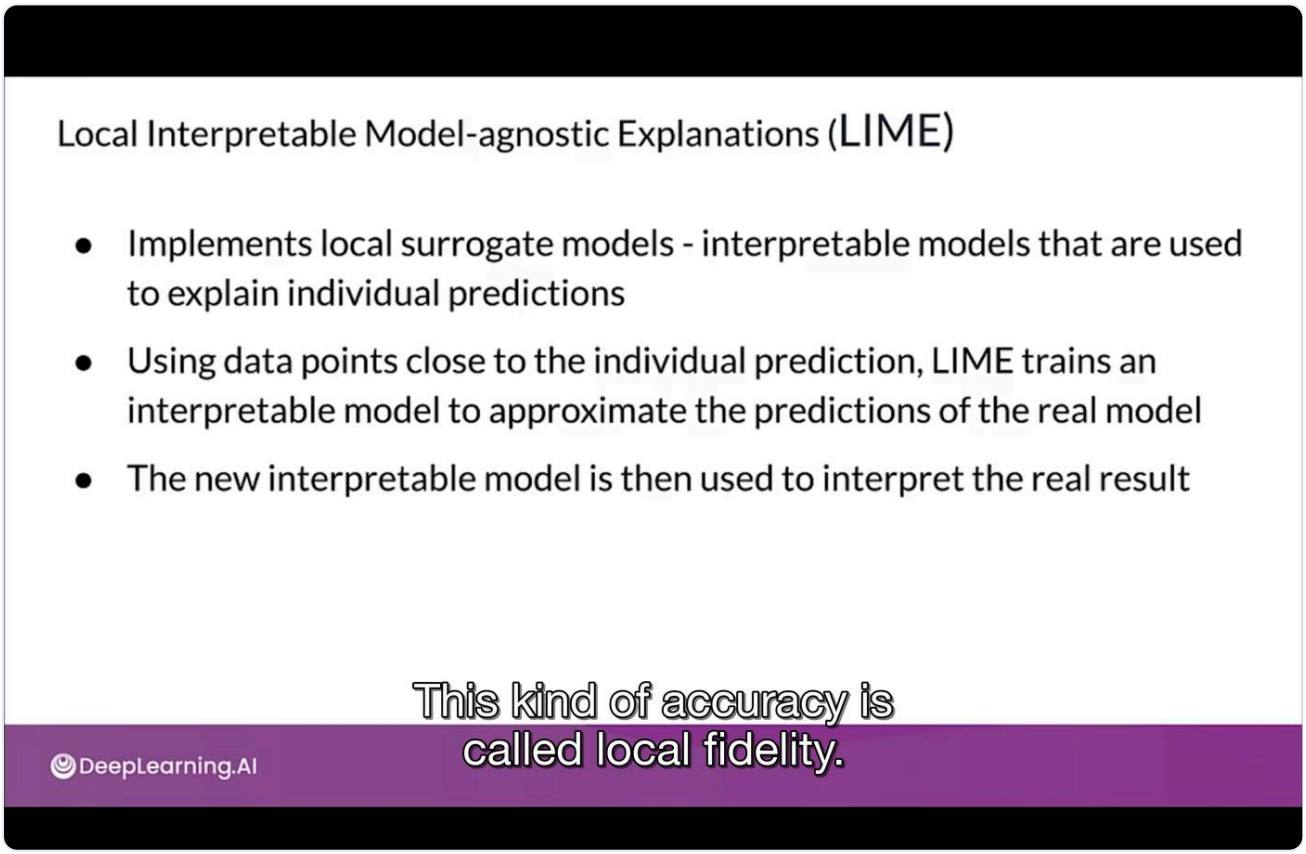

FIGURE 50.1 | Local Interpretable Model-agnostic Explanations (LIME)

This kind of accuracy is called local fidelity. You then explain the prediction by interpreting the new local model.

LECTURE 51

AI Explanations: Feature Attributions

There are also Cloud-based tools and services which can be very valuable for interpreting your model results. Let's look at one of these now, Google's AI Explanations service.

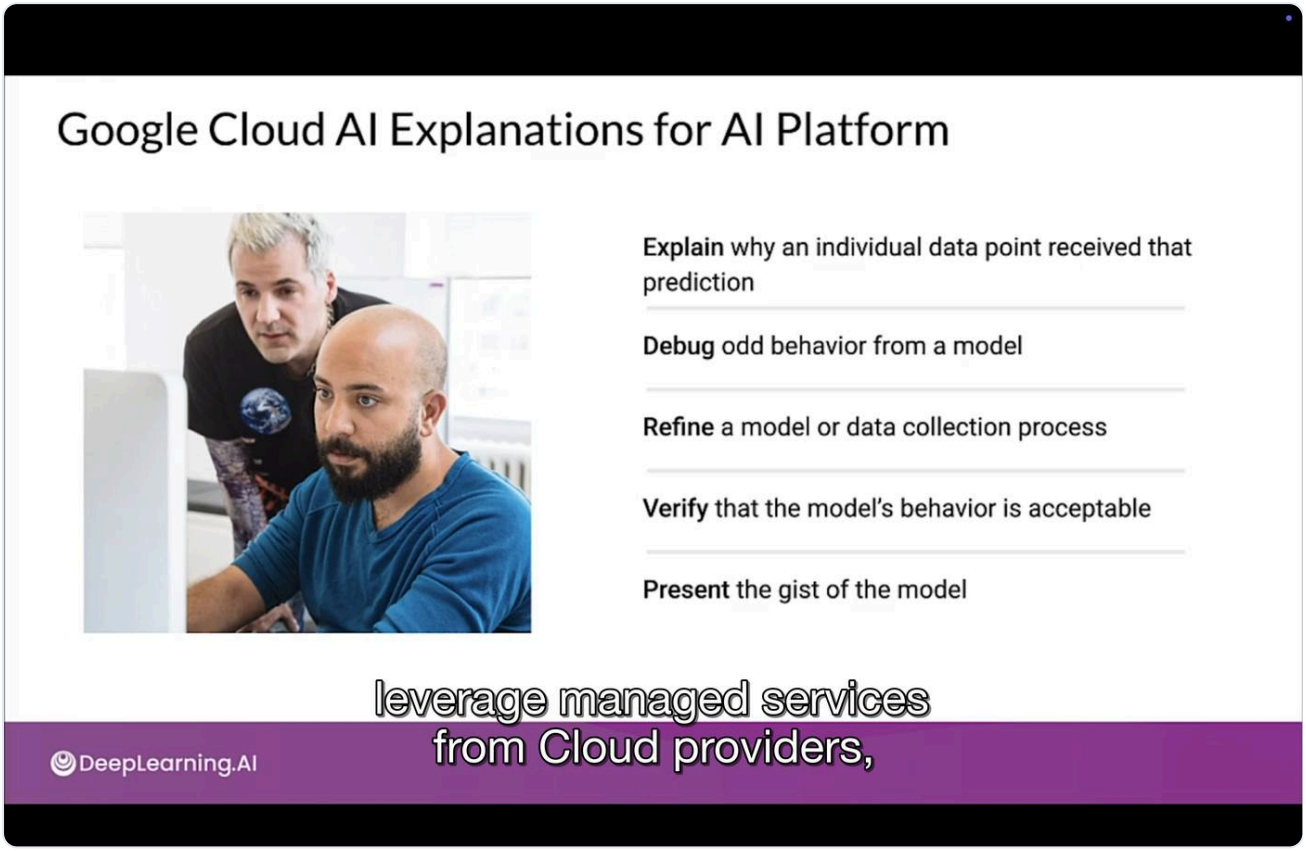

FIGURE 51.1 | Google Cloud AI Explanations for AI Platform

Another option for interpretability is to leverage managed services from Cloud providers, such as Google's AI Explanations for AI Platform. AI Explanations integrates feature attributions into Google's AI Platform Prediction service. AI Explanations helps you understand your model's output for classification and regression tasks.

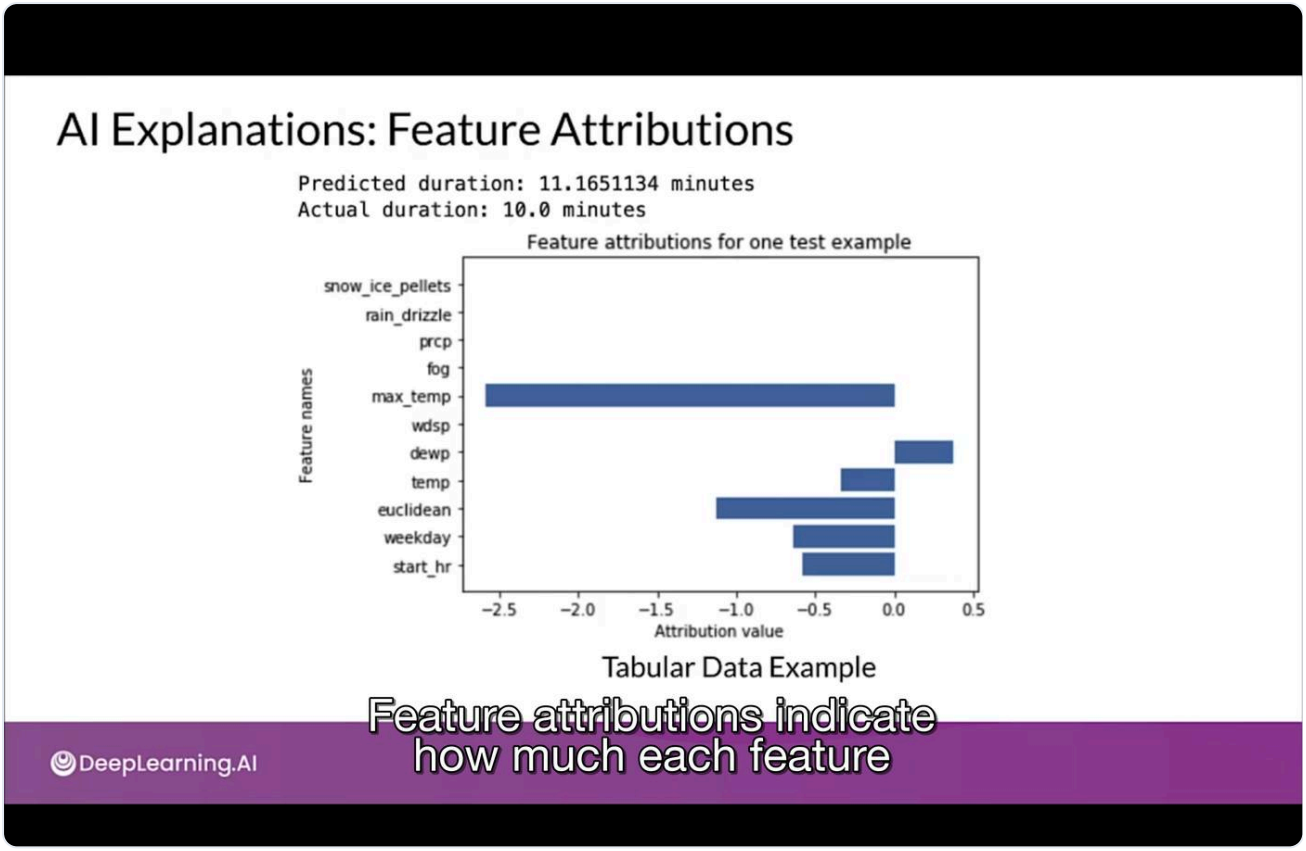

FIGURE 51.2 | AI Explanations: Feature Attributions

Whenever you request a prediction on AI Platform, AI Explanations tells you how much each feature in the data contributed to the predicted results. You can then use this information to verify that the model is behaving as expected and identify any bias in your models and get ideas for ways to improve your model and your training data. Feature attributions indicate how much each feature contributed to the given prediction. AI Explanations works with Google's AI Platform Prediction managed service. When you request predictions from your model normally using AI Platform Predictions, you only get the predictions. However, when you request explanations, you get both the predictions and the feature attribution information for those predictions.

There are also visualizations provided to help you understand the feature attributions. This chart shows an example of feature attributions for a tabular dataset. These are examples of the visualizations for image data.

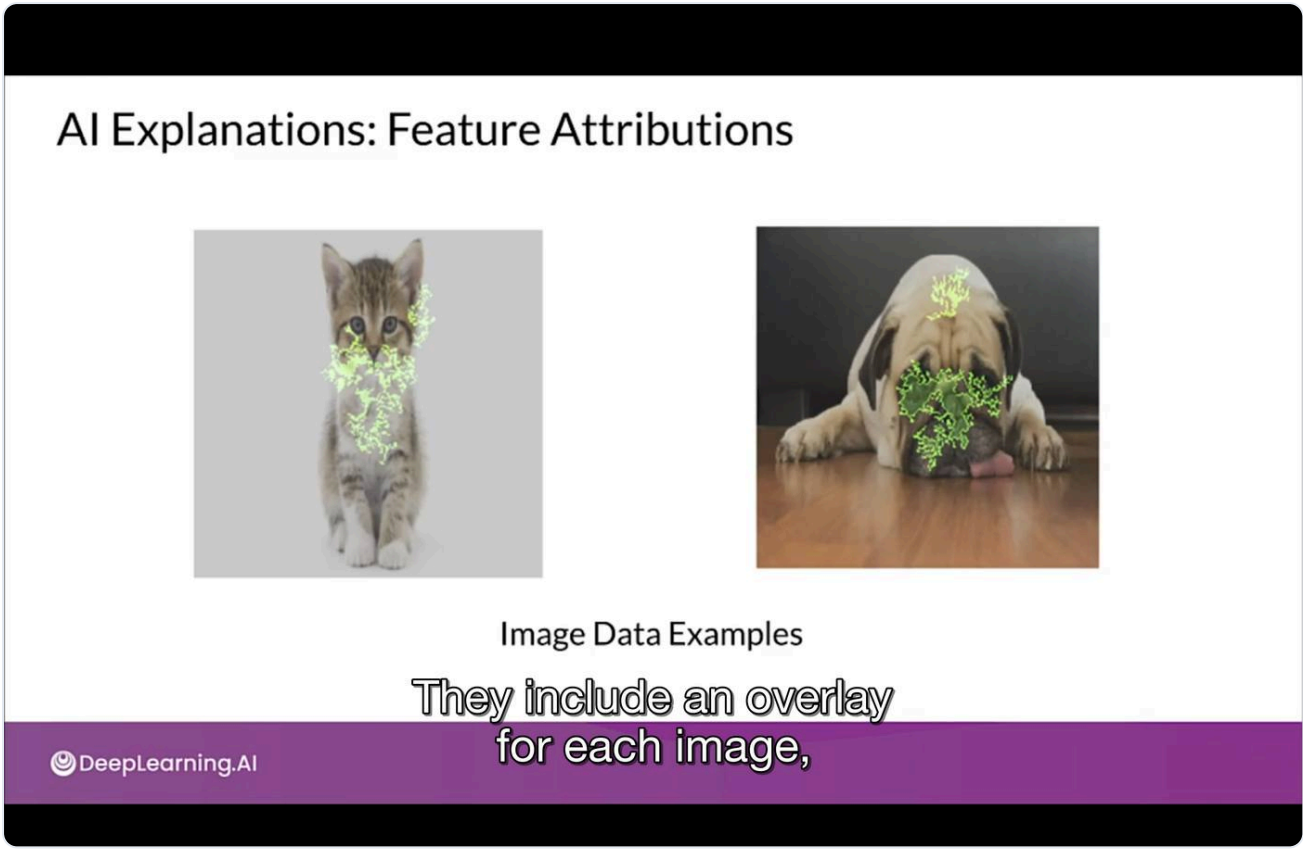

FIGURE 51.3 | AI Explanations: Feature Attributions

They include an overlay for each image, highlighting which pixels in the image contributed most strongly to the resulting prediction. AI Explanations currently offers three methods of feature attribution.

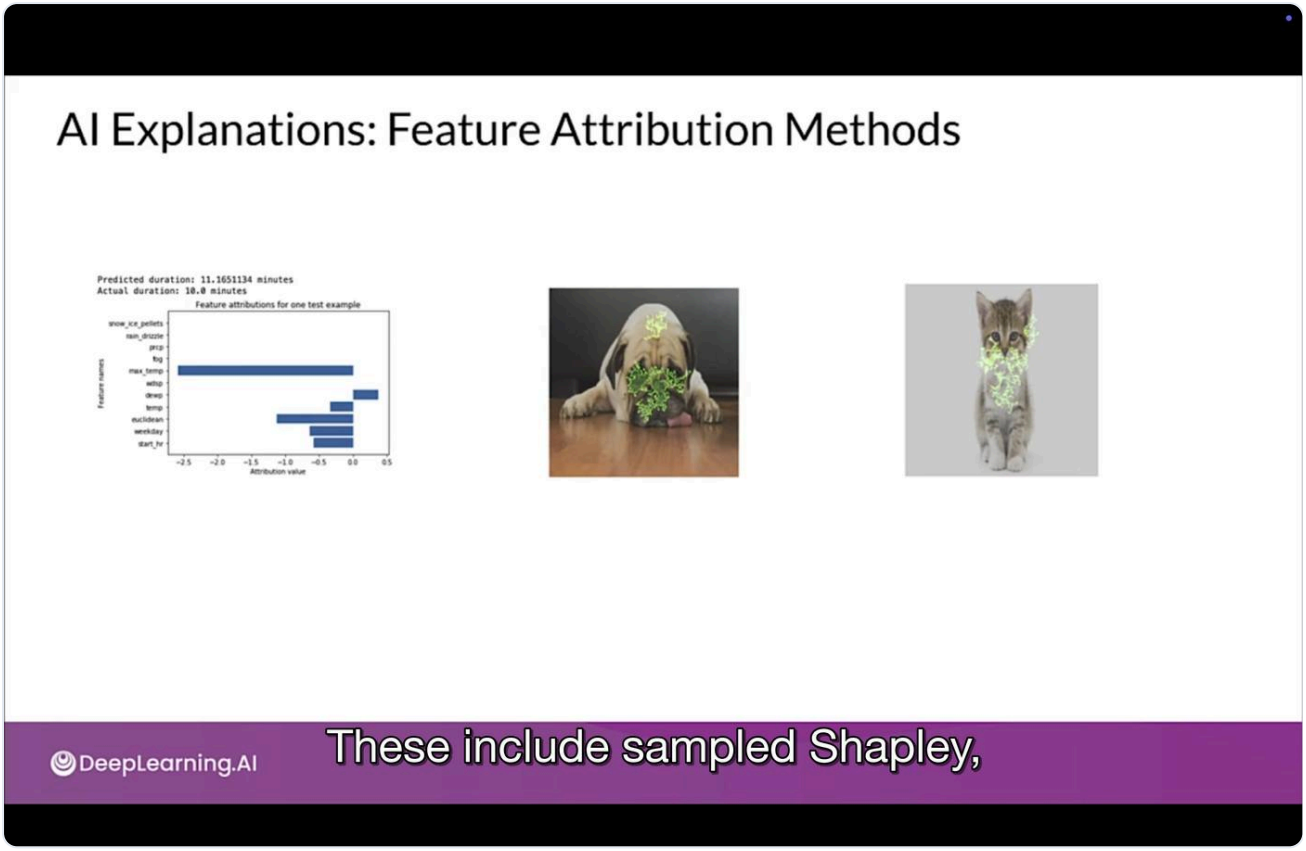

FIGURE 51.4 | AI Explanations: Feature Attribution Methods

These include sampled Shapley, integrated gradients, and XRAI. But ultimately, all of these methods are based on Shapley values. We've discussed Shapley enough that we don't need to go over that again. But let's look at the other two methods, integrated gradients and XRAI.

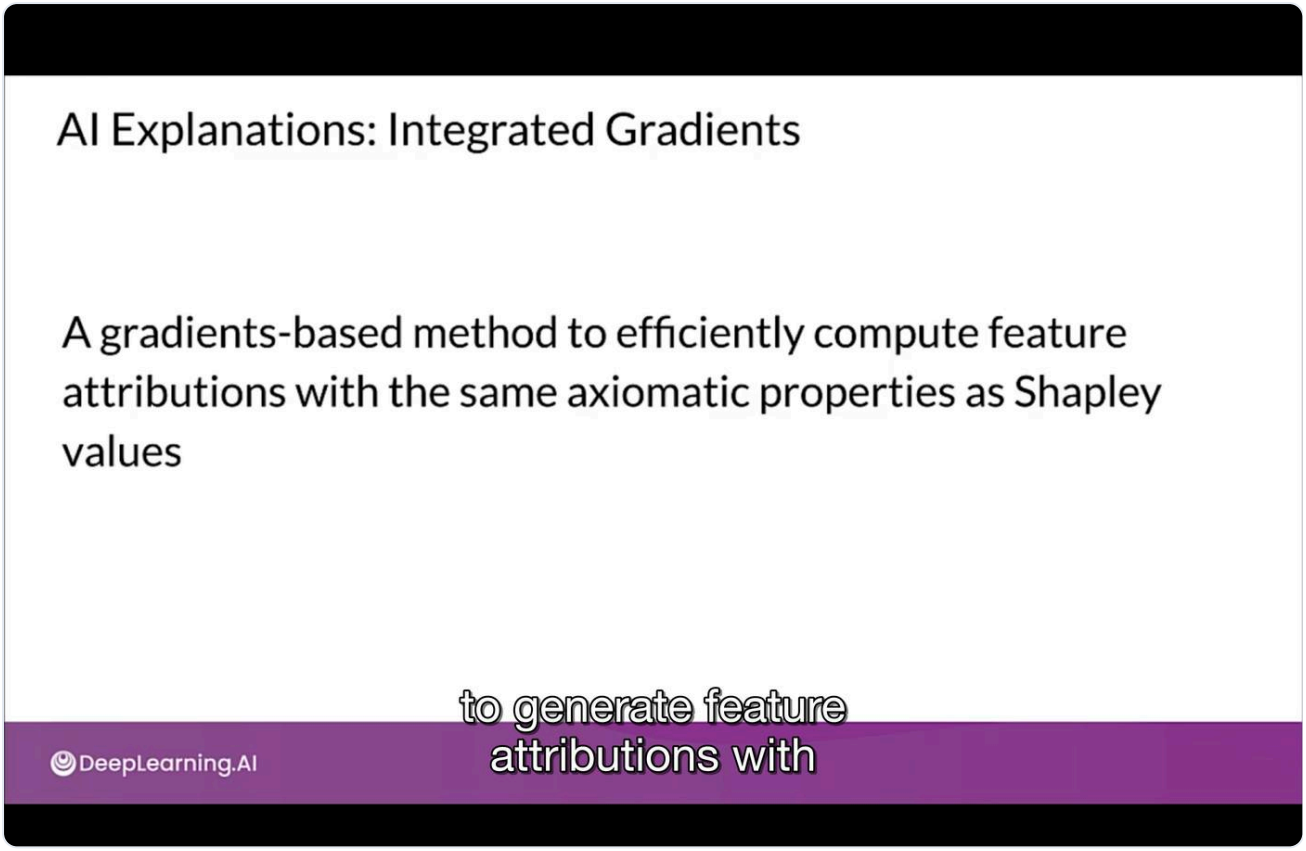

FIGURE 51.5 | AI Explanations: Integrated Gradients

Integrated gradients is a different way to generate feature attributions with the same axiomatic properties as Shapley values based on using gradients. It's orders of magnitude more efficient than the original Shapley-Shubik method when applied to deep networks. In the integrated gradients method, the gradient of the prediction output is calculated with respect to the features of the input along an integral path. The gradients are calculated at different intervals based on a scaling parameter that you can specify. For image data, imagine the scaling parameter as a slide that is scaling all the pixels of the image to black. By saying that the gradients are integrated, it means that they are first averaged together, and then the element-wise product of the average gradients and the original input is calculated.

XRAI or eXplanation with Ranked Area Integrals is specifically focused towards image classification. The XRAI method extends the integrated gradients method with additional steps to determine which regions of the image contribute most to a given prediction. XRAI performs pixel-level attribution for the input image using the integrated gradients method. Independently, a pixel-level attribution XRAI also over-segments the image to create a patchwork of small regions. XRAI then aggregates the pixel-level attribution within each segment to determine the attribution density of that segment. It then ranks each segment and orders them from the most to the least positive.

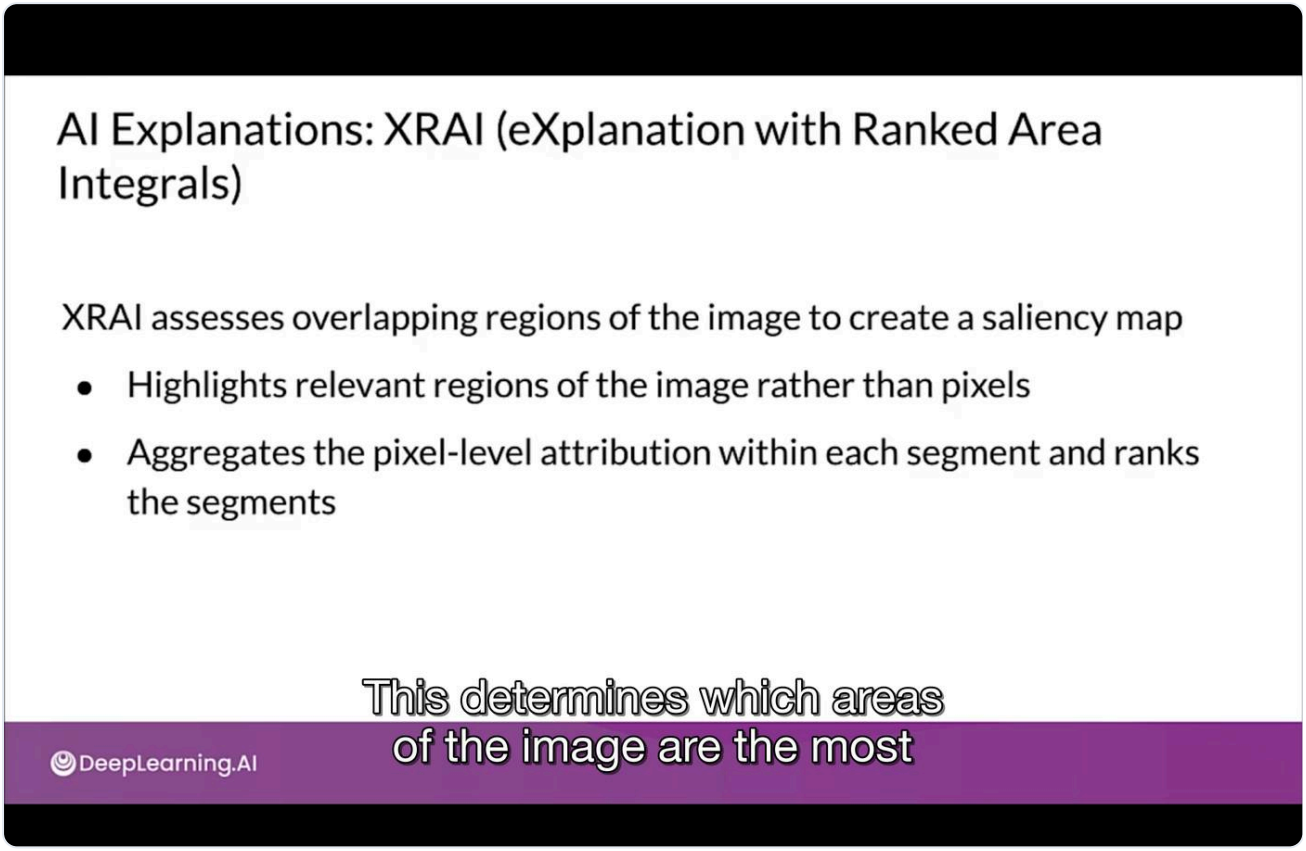

FIGURE 51.6

This determines which areas of the image are the most salient or contribute the most strongly to a given prediction. Here's an example of an image of hummingbirds. Starting at the left, the original image is both segmented into different regions and the pixel-level attributions are calculated using integrated gradients.

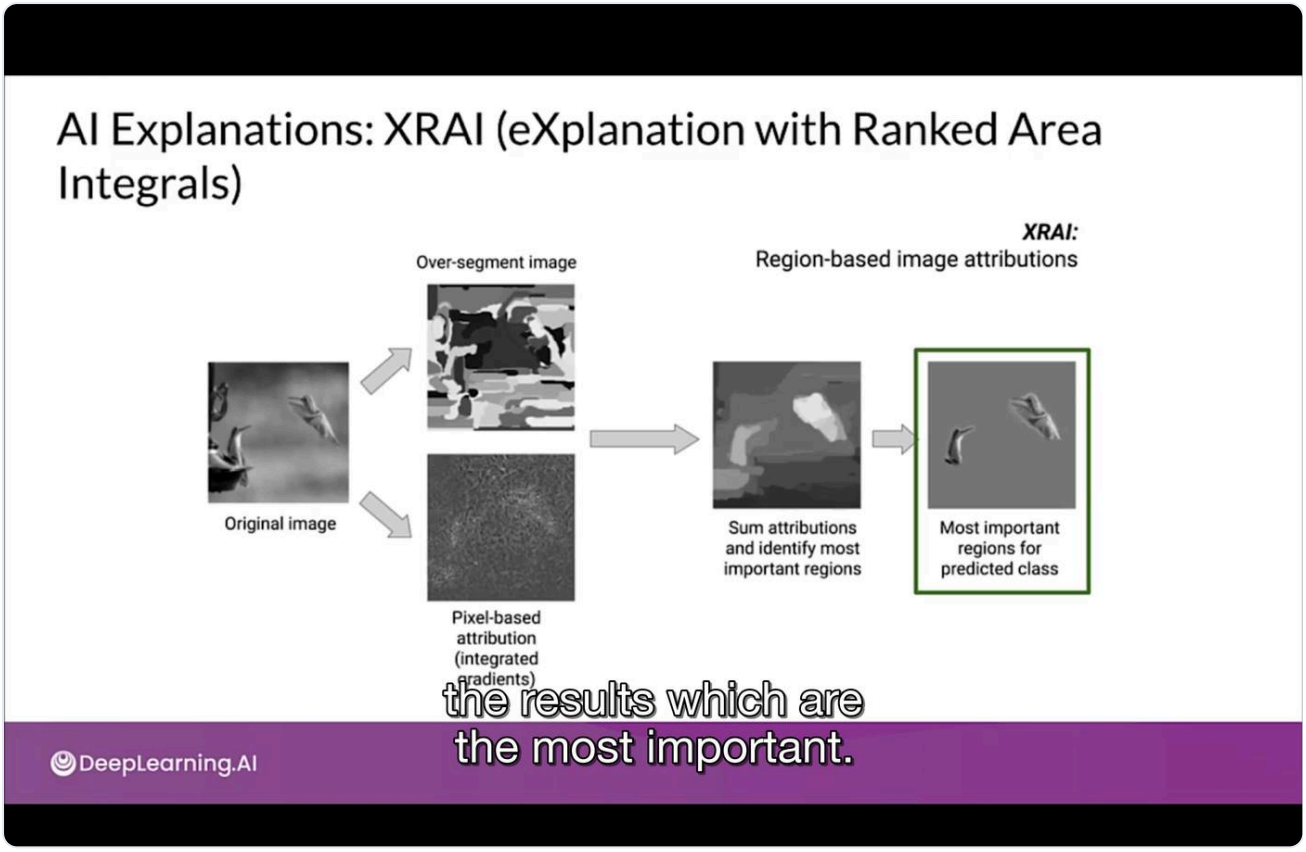

FIGURE 51.7

The attributions within each region are then summed and the regions are ranked to determine the results which are the most important. As you can see in the image on the right, the most important parts of this image are the hummingbirds themselves. Well, that was quite a week. We got fairly deep into interpretability and explainability and looked at it from a couple of different angles. We looked at some intrinsically interpretable models. We looked at the ideas of local versus global interpretation. We looked at a few different post hoc methods, including Shapley values, which are very important for interpretability, and the SHAP library, which helps us work with them.

We took a look at some of the newer methods too including T cab and lattice models. I thought it was a great week. I hope you did, and we'll see you next time.